REGULATORY INTERPRETATION AND BANK CONTROL

From SFC Circulars to Reviewable Bank Controls

Interpretation, evidence and reliable execution for Hong Kong private banking

Chak Wong

A management and research account of how a bank can turn a circular into a reviewable control. Two circulars from Hong Kong's Securities and Futures Commission (SFC) provide the cases. The book follows the meaning of each requirement through interpretation, evidence, implementation and supervised operation, and marks the point where the present evidence stops. The technical record remains available for the people who must build and audit the control.

---

5 October 2026 Research draft

Public-source and synthetic examples. Natural-language legal correctness remains not established. Checked conditional behavior is distinguished from proposed interpretation work throughout.

Executive summary

Senior management is being asked to consider a narrow question: can a bank turn a regulatory circular into a repeatable client or product decision without losing the meaning of the source along the way? The answer matters because a program can apply a mistaken interpretation perfectly. The damaging error is often made before implementation, when an exception is missed, the wrong person is assessed, or a requirement is applied to a different activity.

LegalMath now investigates competing readings of a circular and carries the resulting proposals into executable controls. It keeps the source passage, interpretation, supporting facts and review decision together. A later reviewer can reconstruct which requirement was applied, to whom, at what time and on what evidence.

What has been done

The work follows two public SFC circulars through the same chain of questions. The first concerns the streamlined treatment of sophisticated professional investors (SPIs). A synthetic client is followed through financial qualification, product category, exposure, consent, continuing duties and a proposed transaction. The second concerns incentives used when marketing authorised funds. It shows why a fee discount may fall within an exception while a voucher in the same offer may require a separate assessment. These examples are deliberately ordinary. Their value is that a small change in a fact, a date, or the subject being assessed changes the responsible answer.

The worked rules can be executed in Java and checked against boundary cases, withdrawals and amendments. Later development added the investigation that comes before those rules: generating alternative readings, looking for omitted requirements and comparing the cases on which proposed programs disagree. Official examples can inform that investigation, with their dates and limits kept visible. A separate settlement-readiness circular exposed another problem: several partial answers do not, by themselves, cover all the duties in a source.

The contribution

LegalMath connects a circular to a bank control through an explicit interpretation whose premises can be examined. The investigation states competing meanings, unresolved questions and any separate policy assumptions. Engineering implements the declared specification. The result retains its evidence and explains what remains unresolved when the record is incomplete.

The work distinguishes source duties from added bank restrictions and unknown facts from false ones. Each fact belongs to a person, product, component, date and activity. Preserving what was known at the time lets a correction explain a past decision without rewriting it. A compact flag or a large language model prompt can conceal these distinctions.

Separate duties can now be combined while rival readings remain alternatives. A gift restriction and a disclosure duty may both need implementation; two readings of the same restriction require a choice or an open question. The method also looks beyond agreement. In a controlled challenge, three programs shared a deliberately omitted exception. A separate examination of the source exposed the gap. That is why agreement alone cannot settle the review.

What the evidence supports today

The implementation can preserve alternatives and unfinished checks, build draft Java controls and revisit affected work after a source change. A local amendment exercise changed the selected control's answer while preserving the earlier version and an unrelated control. Earlier development included live model calls; the subsequent integration checks used retained responses and controlled cases. Together they establish these mechanisms on public sources and synthetic facts.

One retained interpretation can now feed two executable languages: RuleIR, the project's small rule language, and Catala, a language and compiler designed for legal rules. Their checked common cases agree. The Catala route also executes documented calculations over collections and partially known records that the RuleIR target cannot express. This extends the implementation; the available source-conversion study does not establish that either language yields better legal interpretations or faster review.

Substantive interpretation remains incomplete. Retained responsibility, exercised supervision and achieved adequacy need separate treatment. Conditions, footnotes, actors and dates still require source-bound justification, and the programs often aggregate supplied factual judgments rather than establish them from evidence. The 5 October assessment retains all 46 original challenge obligations as open. A further literature review identifies methods for precedent comparison, structured arguments and evidence-linked specifications that can address these gaps. Their proposed integration is distinguished from the mechanisms already executed.

An assessment of an actual arrangement would also require its parties, activities, dates and authentic evidence. Their absence does not prevent development of the interpreter on declared premises. The present results do not establish a live client or product approval, general legal correctness or production readiness.

The next research step

The substantive implementation program begins with paragraph 10 and paragraph 14 with footnote 5 of the retained tokenisation circular. It makes rival readings explicit, identifies their supporting and opposing arguments, connects evidence to scoped factual predicates, and checks the executable consequences that follow. Section 6.40 develops the literature and the reasoning behind this program.

The controlling research requirement is proof of an exactly stated proposition or an explicitly qualified result. Human answer keys, ratings, reviewer approval and agreement among models are not product quality evidence. Published legal texts remain premises; a proof about an encoded rule does not prove that the rule faithfully interprets those texts. Earlier institutional workflow proposals in the book are historical context, not acceptance conditions for this research. Production deployment and human-benefit studies are outside the present program.

From source documents to transaction decisions

A prospectus can permit a particular purchaser without establishing that a bank may execute the purchase. The bank must also establish which laws apply, who owns the account, whether sanctions restrict the transaction, and what its own mandate and settlement arrangements permit. LegalMath keeps these questions separate so that an answer to one cannot silently answer the others.

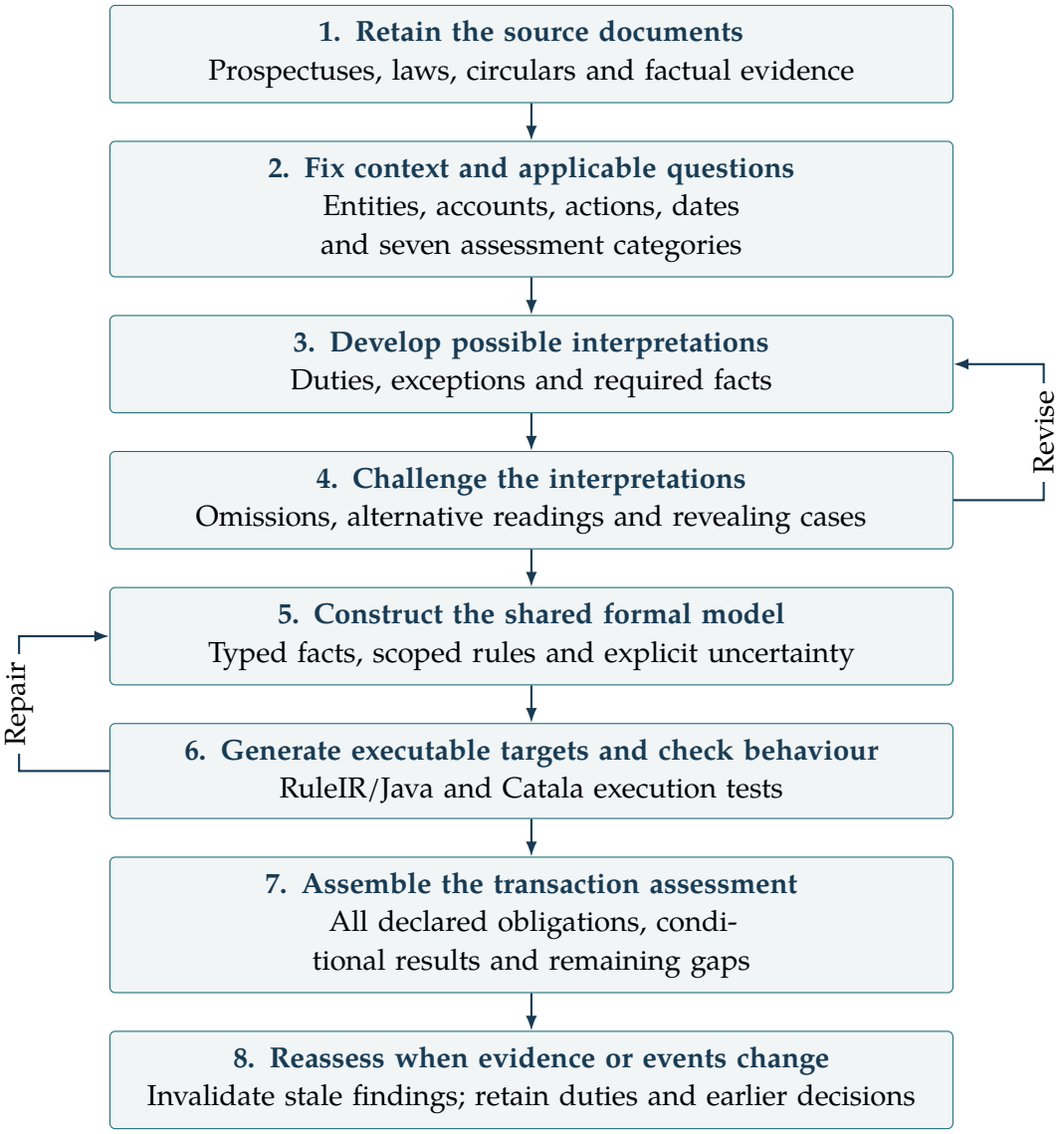
The running example compares two real contingent convertible securities, or *CoCos*: Barclays' February 2025 issue and Standard Chartered's January 2025 issue. These securities can absorb losses through conversion into shares and other contractual or regulatory measures. The same hypothetical US individual passes one selected purchaser condition and fails the other. Both complete bank assessments remain qualified because essential evidence and legal coverage are missing. The following charts trace that distinction.

There are two kinds of layer to keep apart. The *software stages* turn source documents and proposed meanings into executable calculations. The *seven assessment categories* organise the different legal and factual questions about a transaction. Every category uses evidence and scoped rules; it is not a separate programming language. The eight-step overview links to both the worked CoCo case and the implementation details.

The first software stage, called the *frontend*, proposes a meaning for the passage and records its sources, assumptions and required facts. A *translator* then turns that fixed proposal into an executable language. LegalMath has two such targets. *RuleIR*, its own small rule language, expresses conditions such as a financial threshold and the effect of missing evidence. *Catala* is a programming language designed for legal rules, with a compiler that translates those rules into executable code. Both targets can receive the same proposed meaning; Catala also supports the implemented calculations over structured collections described in Chapter 8.

Stipula answers a different question: how does a contract change when a party takes an action? It describes parties, resources and states. For example, a request available while an agreement is active may become unavailable after withdrawal. The book studies Stipula's verification methods as a comparison; LegalMath does not currently compile Stipula contracts.

Eight steps from documents to a qualified decision

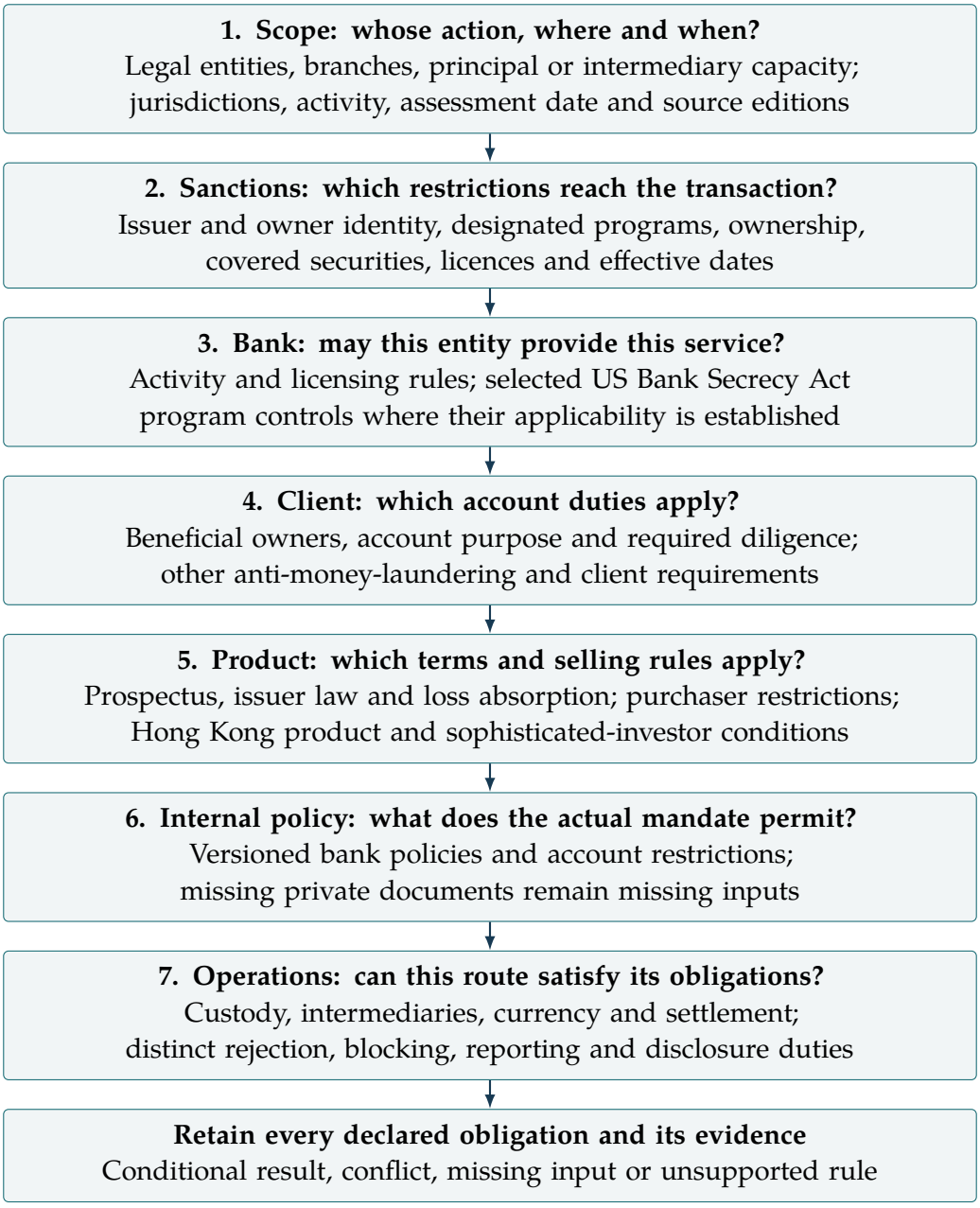


A failure or an unknown remains part of the assessment. Completing the software steps does not establish complete legal coverage or permission to execute a transaction.

The next pages expand the seven assessment categories and follow the two CoCo purchaser conditions into the complete bank assessment. Later pages explain each software step. Unsupported expressions prevent translation; missing facts can prevent execution even when a program has compiled. A different proposed interpretation remains a different model, with its own evidence.

The seven layers of a transaction assessment

Begin with a proposed action by a specified entity for a specified account. The bank's brand does not identify its booking branch, capacity or the client's status. Follow all seven categories: an adverse or unresolved finding remains in the assessment while the other questions are considered.

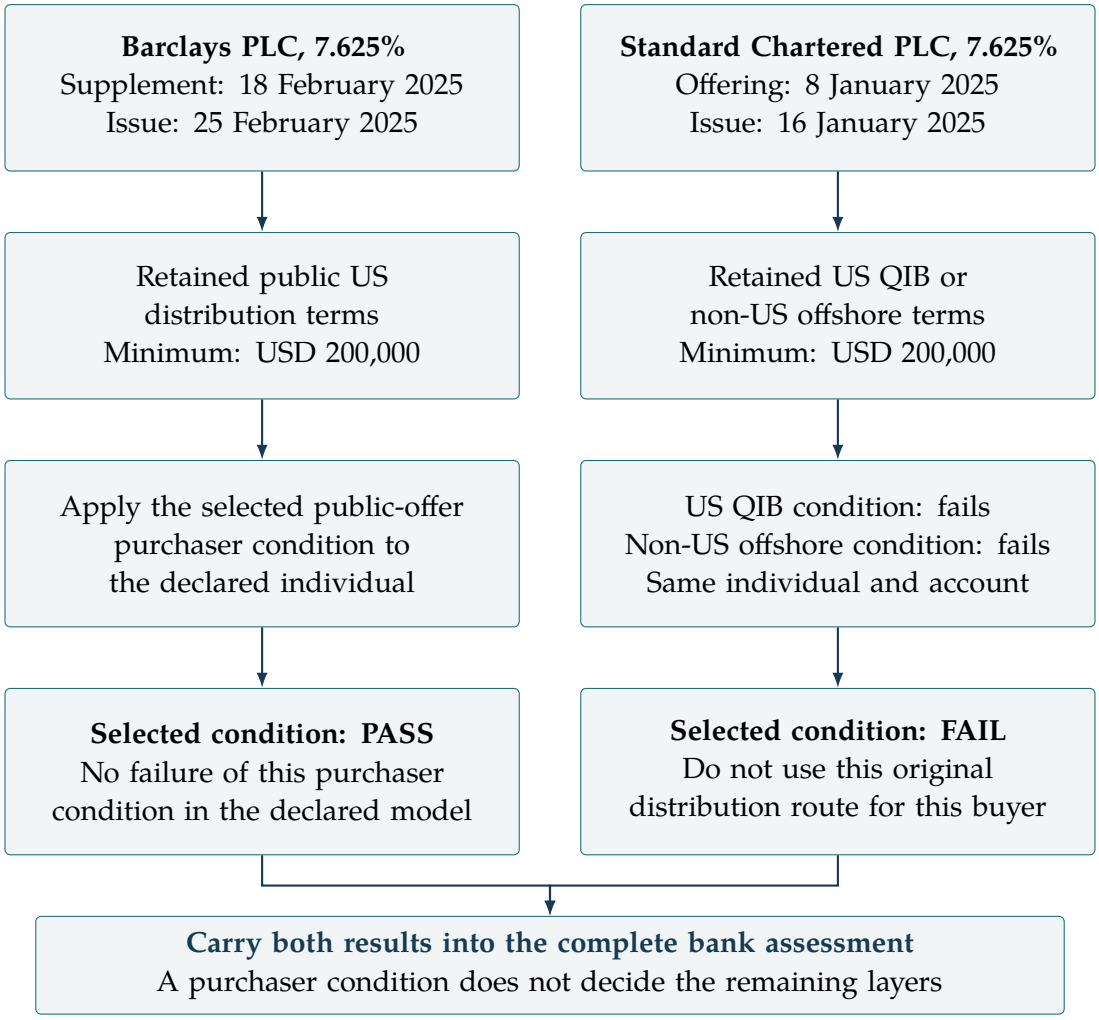


The current inventory has fourteen obligations across these seven categories. Five have conditional rule models; nine retain missing rule families or unresolved coverage. None may be removed by the caller.

The five calculated obligations concern selected sanctions, bank program controls, private-account diligence, sophisticated professional investor conditions and reporting. Supporting calculations bring the collection to eleven shared models. Complete discovery of applicable law remains unproved. Section 2.4 explains the provisions and their limits.

The CoCo example: from terms to purchaser conditions

Assume one individual, resident and located in the United States, buys USD 200,000 principal for their own account and benefit. The individual is not a *qualified institutional buyer* (QIB), an institutional and account category used by Rule 144A. Wealth alone does not turn an individual into a QIB. Regulation S supplies a different offshore distribution route; its US-person and account definitions must also be considered.



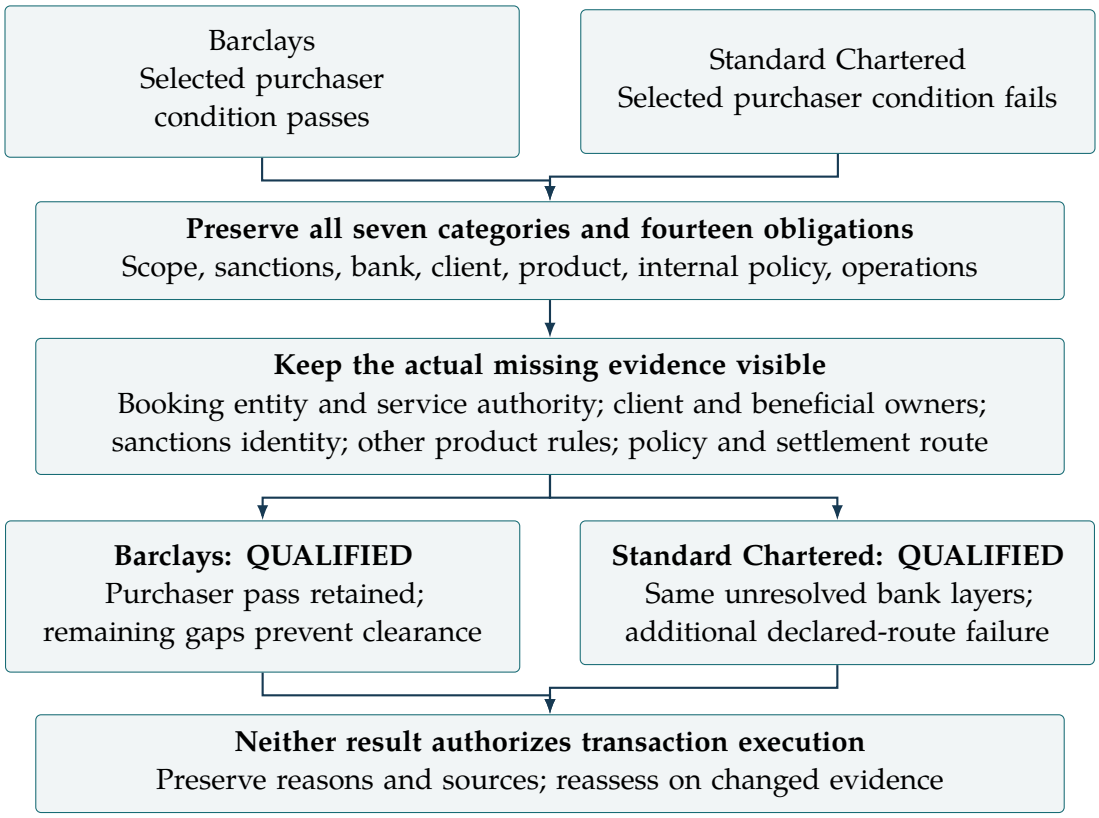
Both are real original offerings with the same minimum denomination. The difference is the selected purchaser restriction. Their offer dates differ; the comparison does not assert simultaneous live availability.

Both instruments can expose holders to conversion and regulatory loss absorption. A favourable purchaser result therefore says nothing about principal protection. The two prospectuses and the retained 2025 legal definitions support explicit interpretation proposals; page and file hashes identify that evidence without proving the proposals correct.

Changing the actual purchaser can change the answer. A QIB buying for itself can satisfy Standard Chartered's selected condition. A QIB bank acting for a non-QIB client cannot use its own status to satisfy that client-account condition. Moving an order offshore does not by itself remove US-person or US-benefit restrictions. Section 2.4.5 derives the formal rule and identifies the exact instruments and source passages.

The CoCo example: the complete bank result

The conditional purchaser finding supplements the existing bank investigation; it does not close the broader product-law obligation. A *qualified* result explains why the evidence cannot support a complete transaction decision.



The implemented attachment preserves the bank investigation and adds the purchaser finding. It cannot turn a partial pass into bank or regulatory approval. The negative result concerns the declared original distribution.

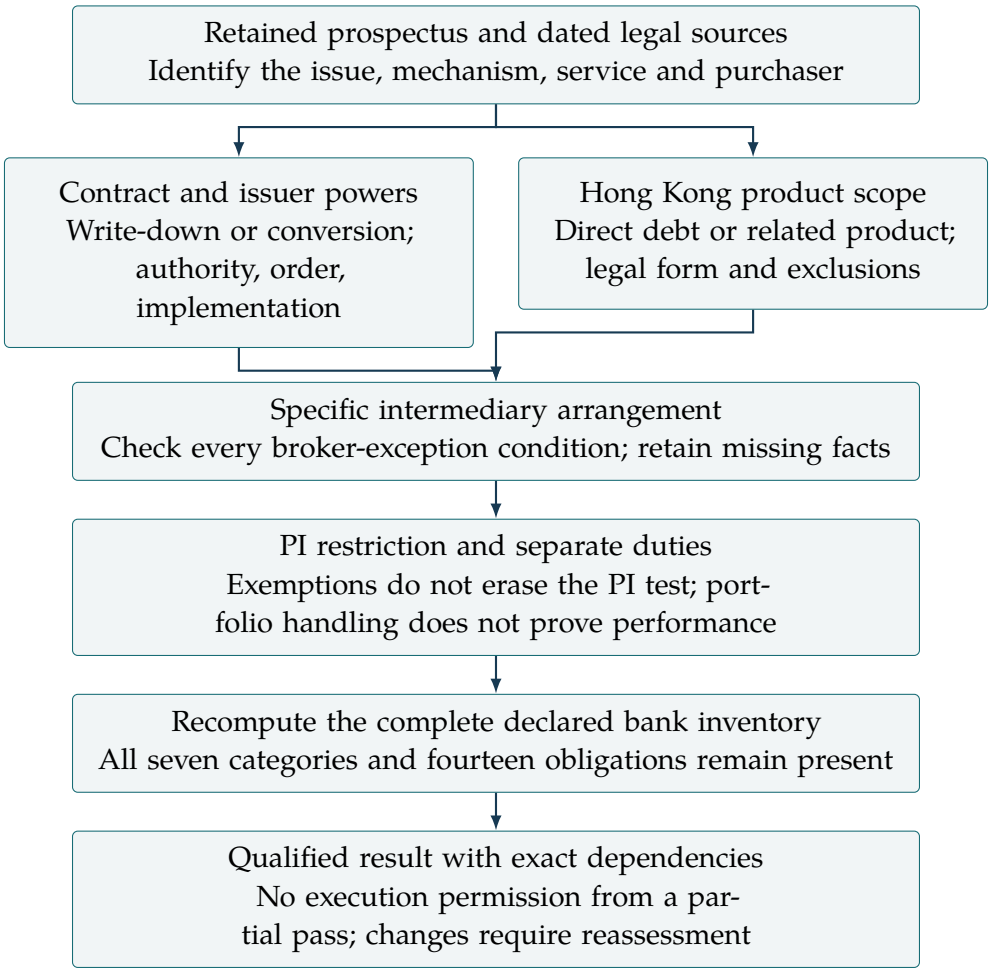
Issuer law is part of the product investigation even when the prospectus contains no decisive contractual trigger. The CoCo survey in Chapter 2 follows Swiss regulatory write-down powers and their dated legal and litigation context. That survey informs required source checks; it is not a completed executable model of every issuer jurisdiction. Swiss provisions cannot be applied to these UK issues merely because all are CoCos.

Sanctions require their own analysis of the issuer, beneficial ownership, program, activity and date. Chinese state ownership alone is not the selected sanctions trigger. A name match supplies a candidate identity; no match does not supply clearance. Restrictions on a US bank acting for a client also need the bank's actual capacity and the client's status. These questions remain separate from the US offering categories in the preceding chart.

The paired run reuses retained interpretation proposals; it does not conduct a fresh model investigation of every prospectus clause. The formal and execution checks support the stated calculation without proving its English interpretation or supplying missing bank evidence.

Join the product conditions to the bank investigation

The joined calculation keeps contractual loss, issuer powers and selling requirements distinct. Section 2.4.6 explains the implemented conditions and their remaining premises.

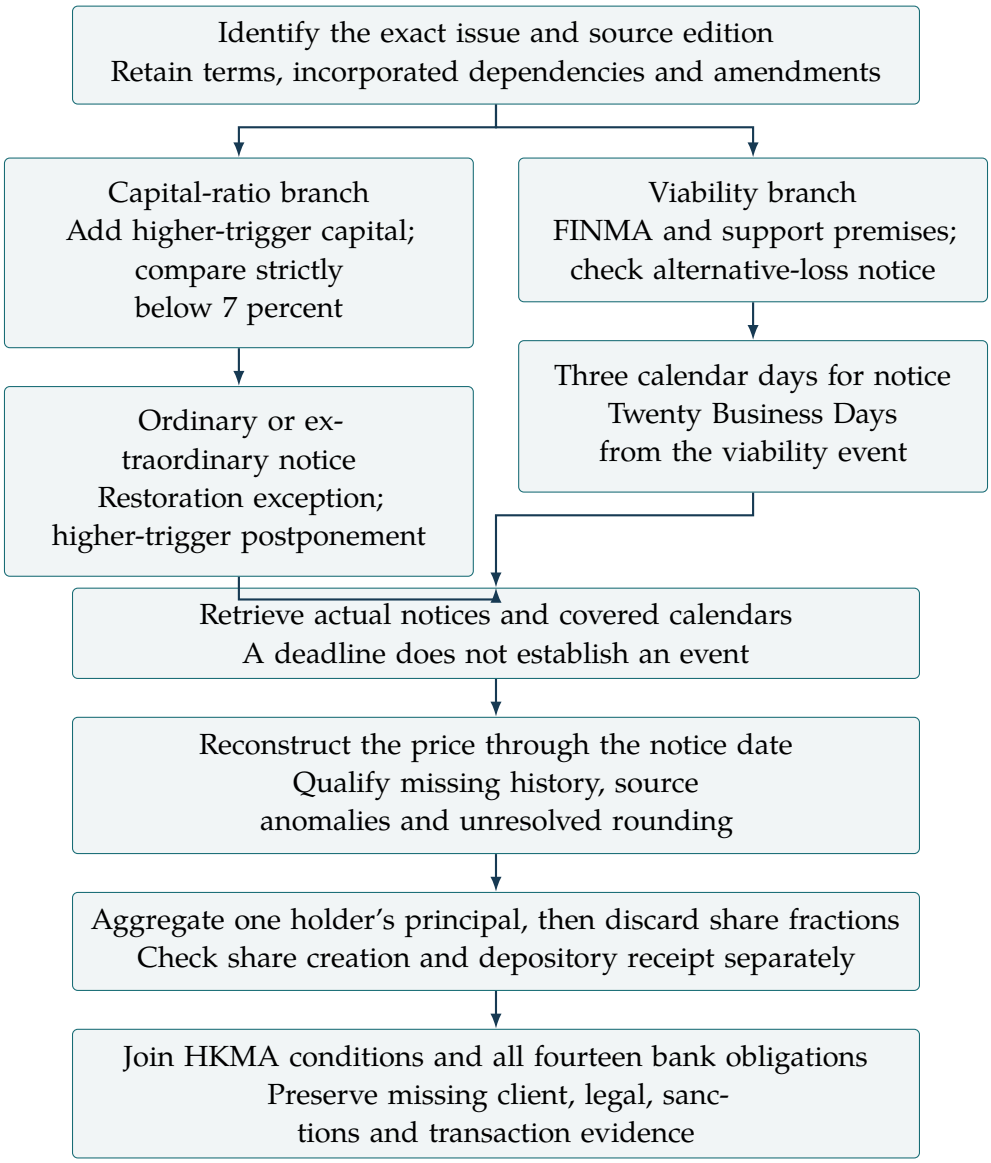


A conversion can cancel debt principal while delivering shares. Neither zero debt principal nor a PI classification settles the entire bank decision.

The retained UBS issue converts under its specified conditions. The next chart follows its implemented notice and share calculations. Swiss regulatory provisions do not establish powers over the UK issues. The actual account, mandate, bank entities and transaction route also remain necessary evidence. The joined reports for all three prospectuses retain these gaps.

Follow one UBS issue from publication to shares

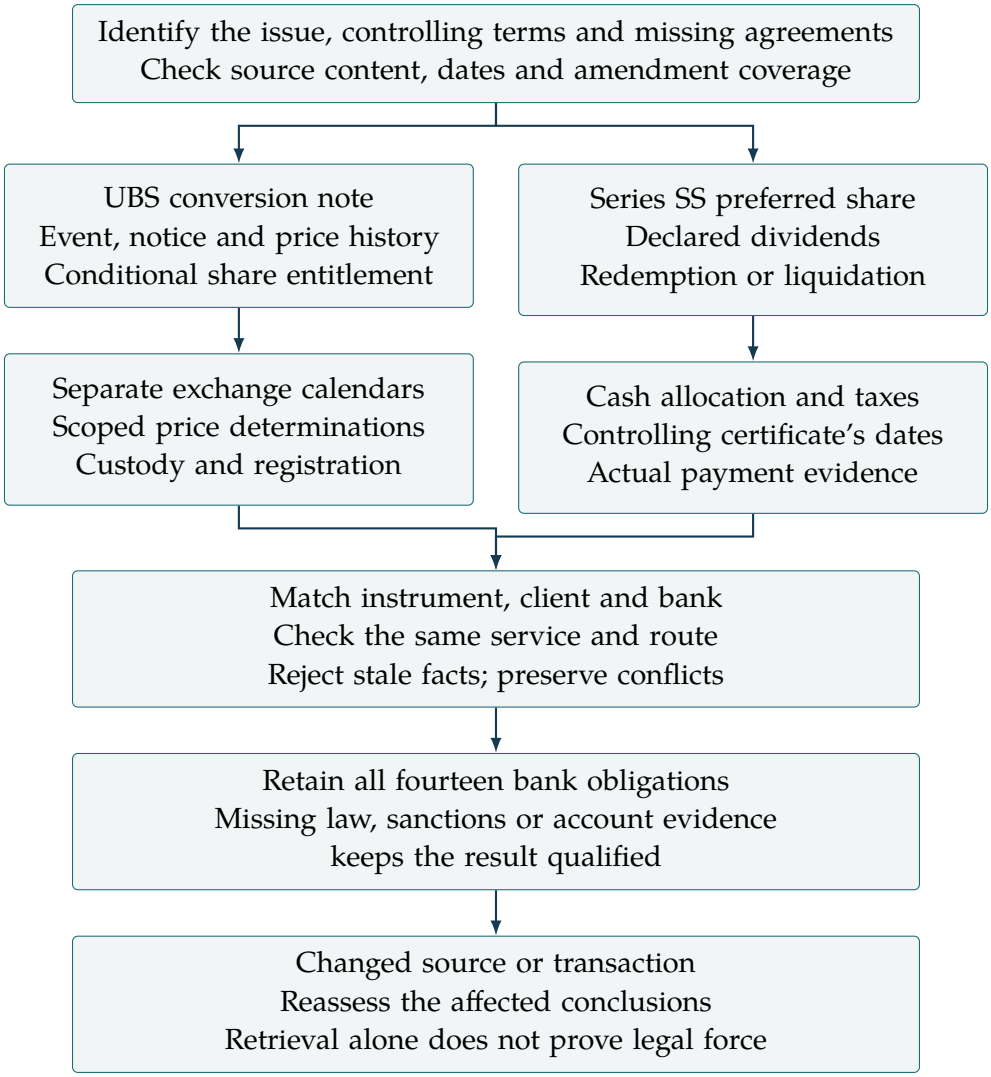
Section 2.4.7 uses Annex A of the June 2024 SGD notes. Source proposals populate the product analysis; actual client and event facts remain separate inputs.



The printed extraordinary-distribution formula has an unresolved denominator anomaly. Its literal calculation is retained; no replacement price or bank clearance is inferred.

Compare conversion notes with preferred shares

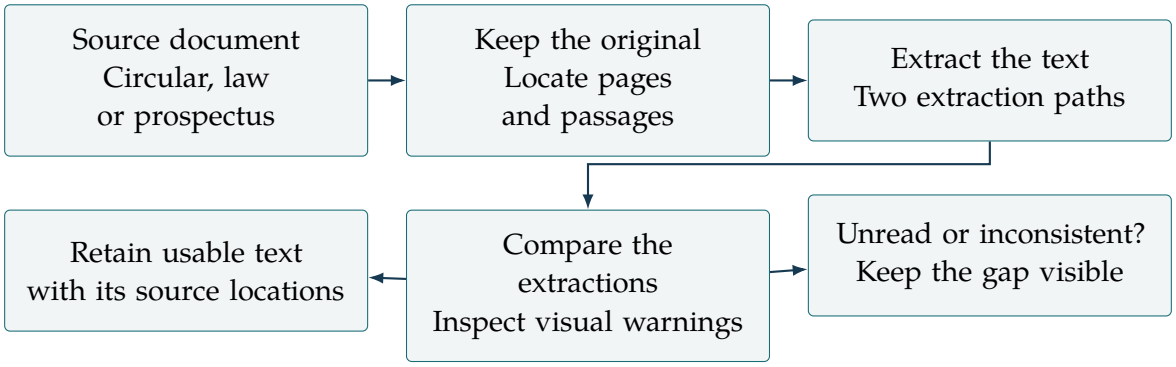
Section 2.4.8 follows the Series SS preferred share. Its non-cumulative dividend and redemption terms require different calculations from the UBS conversion note. Both still enter the same bank assessment.



Different instruments use different contractual calculations. Evidence about a distribution, conversion or delivery remains separate from permission for the bank to carry out the transaction.

1. Retain the source documents

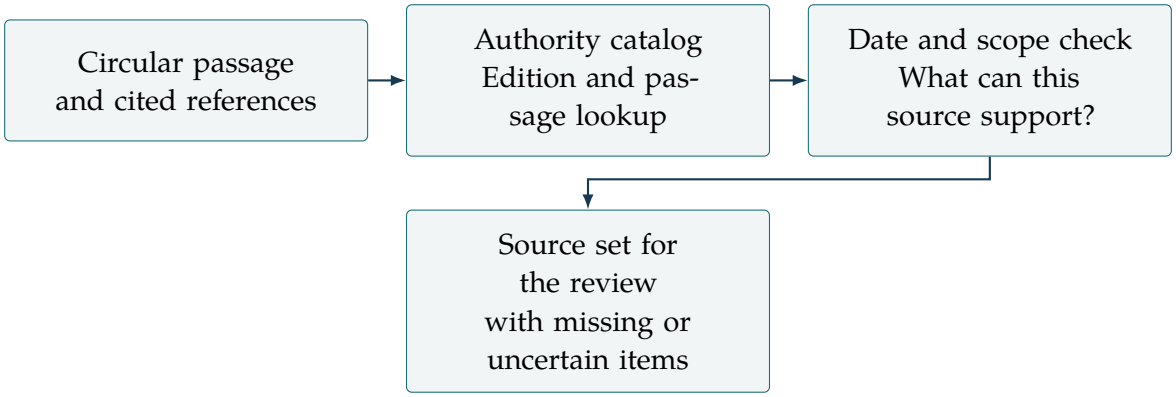
The first task is to give every later reading the same inspectable source. For PDFs, `pypdf` and Poppler's `pdftotext` are two different text extractors. Web pages have two parsing paths. Comparing their text can expose a missing line; checking the page for images can expose material that neither extractor has read.



Agreement between text extractors cannot reveal an exception present only in an image. The additional source checks in Chapter 6 read page images and compare the recognised text with the embedded text.

2. Fix context and applicable questions

A gift restriction may refer to the Code of Conduct and a frequently asked question (FAQ). A retained authority catalog connects those references to document editions and passages. A dependency search follows explicit official links; missing documents remain named questions. The relevant business scope identifies the people, products, activities and dates being assessed.

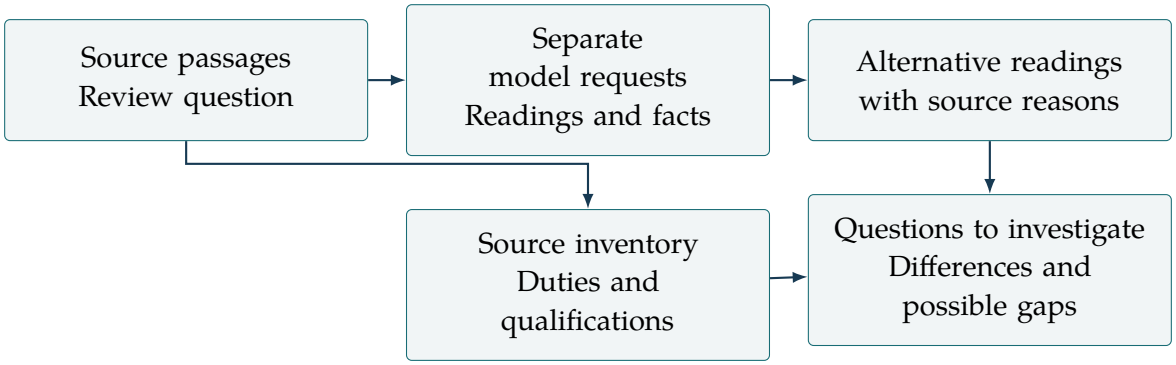


Finding the FAQ changes the question from where the source is to what its dated answer supports.

The retained FAQ page has later answer updates. Its older page date therefore does not establish everything a historical reader could have seen. Prospectus work adds issue-specific supplements, incorporated documents, issuer law, regulatory notices and the applicable selling rules. The transaction registry binds factual documents, policy and settlement-route records to their retained bytes and declared dates. Acquisition time does not establish legal force. Missing or conflicting evidence remains explicit. Chapter 2 develops these source questions; Appendix A gives the source tools.

3. Develop possible interpretations

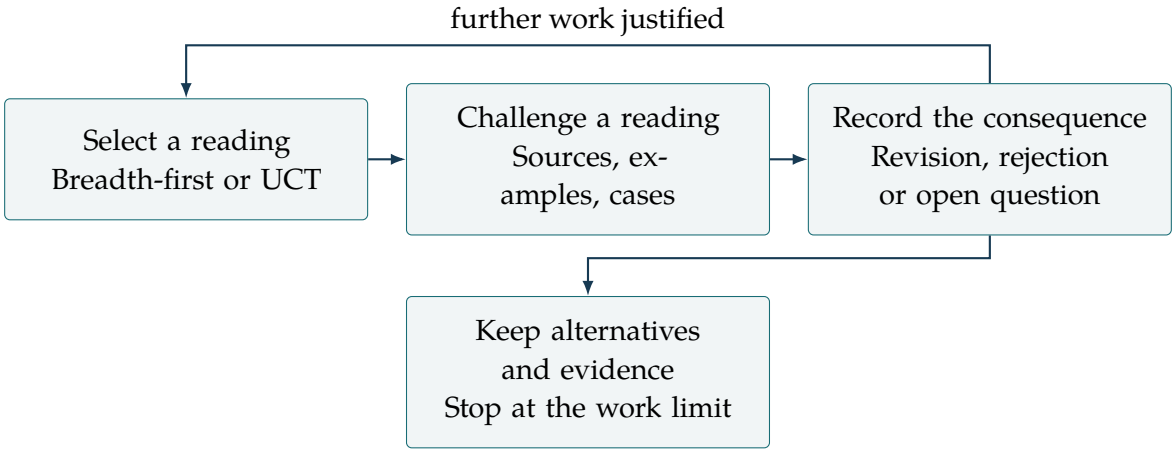
A language model can propose what a passage requires and which facts would decide a case. Separate requests produce initial readings without showing each request the other answers. Another set of requests inventories the source's requirements and qualifications. This second view helps expose an exception that all the proposed readings might miss.



Several readings reveal disagreement. A separate source inventory can reveal a shared omission. Separate requests can still share mistakes.

4. Challenge the interpretations

Tree search organises successive revisions of a reading. Breadth-first search visits the shallower alternatives first. Upper Confidence bounds applied to Trees (UCT) balances previously useful branches against less-visited ones. These are ways to choose the next investigation, not probabilities of legal correctness. Both are implemented with finite work limits.

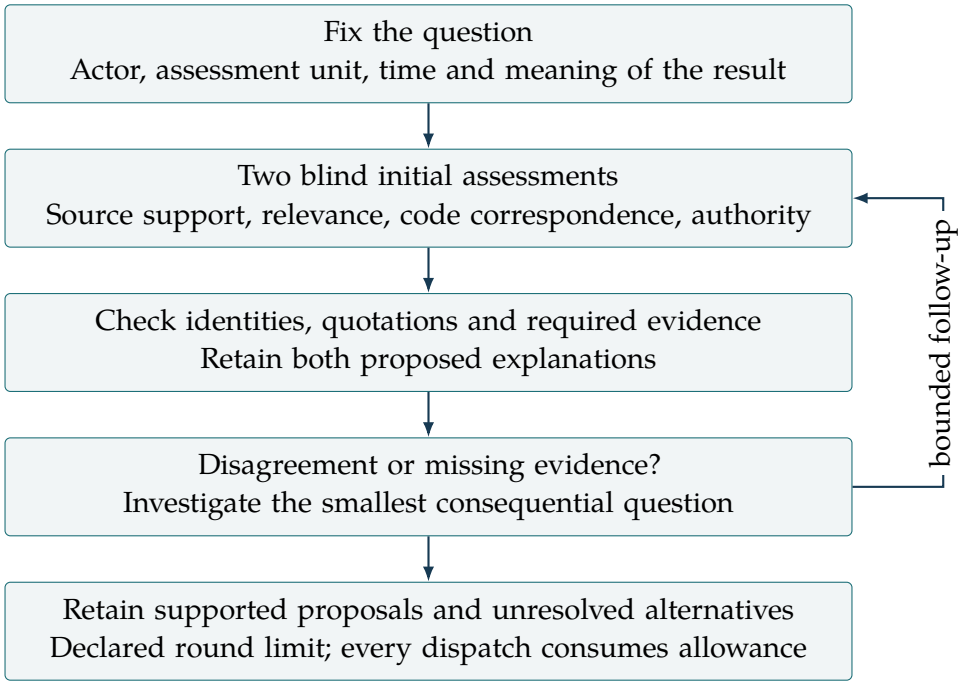


Search changes a proposed reading only with an explicit reason. Exhausting the work allowance leaves an unresolved result.

Typed arguments record support and objections; official examples retain their stated meaning. A rebate-and-voucher case can reveal where readings attach an exception. After aligning facts and outputs, the original programs execute that case. Draft compilation and step 6 therefore occur inside this loop too. Agreement still cannot establish complete source coverage. Chapters 5 and 6 develop these checks.

4 continued. Investigate the exact disagreement

The continuing investigation separates source support, relevance to the selected question, correspondence with the executable rule and availability of required authority. Two initial contexts work without peer answers. A later round can then address the precise disagreement, or repair an invalid response, within the same fixed allowance.



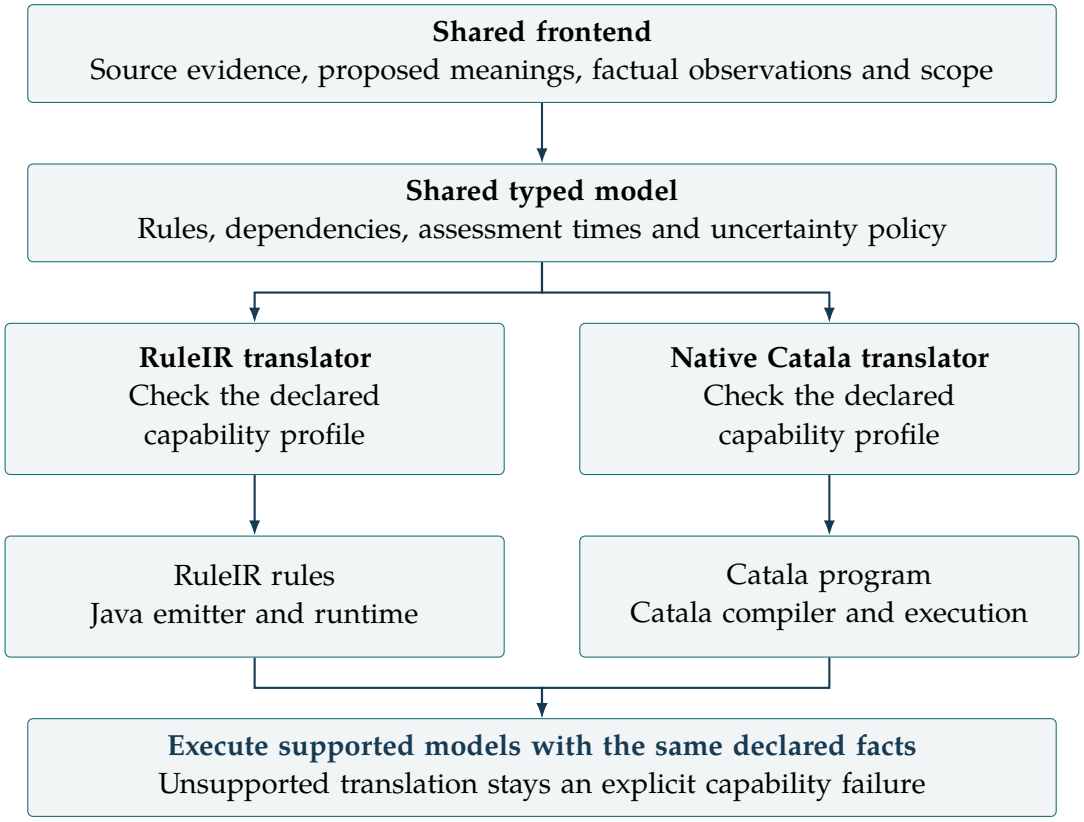
A repaired response is another proposal. Agreement cannot supply a missing source or prove an interpretation.

Rival factual models retain different actors, objects and assessment times. A proposed mapping permits a conditional comparison only after those distinctions and its assumptions are stated. Changed source bytes or methods start a new revision; the old program remains evidence for its old source. Section 6.35 follows the implemented distinctions through a contact history for a suspicious transaction report and the unfamiliar-circular study.

Capacity is checked separately from meaning. Smaller source or comparison batches retain the complete set of obligations and the original retry counts. The optional incremental check admits each validated part only while total retained size remains within its limit. A failed repair preserves earlier claims; it cannot replace them with an empty inventory. If the work still cannot finish, its missing checks remain in the final report.

5. Construct one shared model before choosing a target

A program needs the meaning of each fact and operation fixed. The shared frontend records sources, proposed interpretations, typed facts, scoped rules and uncertainty before a target language is selected. Type and dependency checks expose invalid combinations and missing or circular references. Separate translators then report which parts they can express.

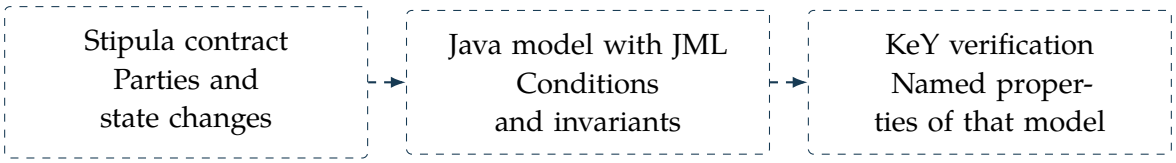


RuleIR is one target of the shared model. Native Catala does not need to pass through RuleIR. A different proposed meaning requires a different model, not merely a different target.

For example, a gift restriction and a separately specified disclosure duty can both belong in a package. Two rival readings of the restriction remain alternative packages. The proposed grouping and each unresolved question are preserved. There are two Catala routes. The compatibility route lowers a supported RuleIR profile and retains the existing result, trace and release checks. The native route consumes the shared model directly, so rich records, lists, options, payload variants and typed operations can be reported as executable Catala even when the RuleIR target returns an explicit capability failure. These routes share the frontend and evidence policy; they do not make different interpretations agree by construction.

5 continued. Where Stipula belongs

Stipula describes contracts through parties, resources, actions and state changes. It is relevant when consent can be granted and withdrawn. The published translation studied in Chapter 4 constructs a Java verification model with specifications in the Java Modeling Language (JML). KeY is a verifier that reasons about the Java model and those specifications.

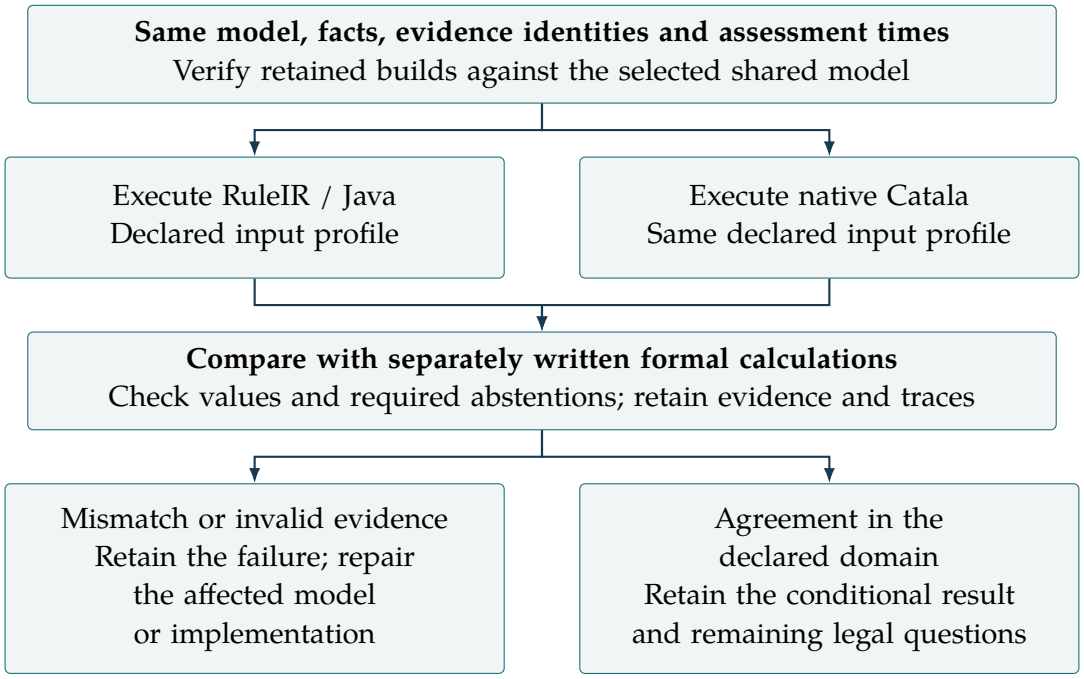


Dashed boxes show a studied research route, not the current LegalMath compiler. Its Java model is not automatically a bank application.

The retained translator examples contain a compilation discrepancy; no local KeY replay establishes their reported proofs. Adoption requires reproducible inputs and a justified mapping to LegalMath's event rules. Chapter 4 details those limits; Chapter 7 explains the implemented rule language.

6. Generate executable targets and check their behaviour

The Java emitter embeds the fixed RuleIR rules in a generated policy class that calls a shared Java evaluator. The Java compiler, `javac`, compiles that class and its supporting runtime. A separately written Python evaluator supplies another execution of the same proposed rule. The Catala translators take the same shared model when their capability report permits it: the compatibility route lowers supported RuleIR, while the native route emits richer Catala directly. Each target receives the same declared facts and assessment times for the cases in its profile.



The Java/Python pair and the Catala routes check whether a proposed rule survives translation within a declared profile. They can agree on a rule that omits a legal exception.

The bank and CoCo runs use a complete-input profile: unknown, conflicting or expired required evidence causes abstention. In the paired run, each target returned 138 complete-input results and 19 abstentions. A successful abstention test demonstrates preserved uncertainty, not a favourable purchaser finding.

The test cases must also be capable of exposing mistakes. Boundary cases place values on either side of a threshold. Missing-fact cases check that uncertainty survives. Event histories exercise consent, withdrawal and timing. Deliberately altered rules, called *mutations*, ask whether the tests notice a removed exception or a reversed condition.

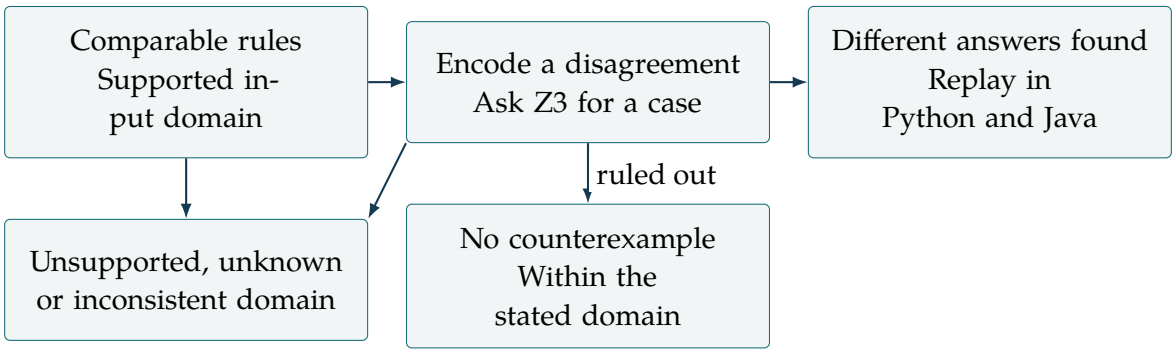
For a small finite rule, every declared input combination can be enumerated. Larger or unsupported domains require more limited conclusions. Expected legal outcomes need their own source justification, separate from Python/Java agreement. Compilation failures and disagreements return to the affected rule or implementation; successful checks accompany the draft rather than remove its open interpretation questions. Chapter 7 develops these checks and the separate Java host demonstration.

6 continued. Where mathematical verification fits

A mathematical check needs a precise proposition: whether two encoded rules differ on a permitted input, for example, or whether a transfer preserves a quantity. Such propositions concern the formal model and its assumptions.

Symbolic comparison with Z3

Z3 is a satisfiability modulo theories (SMT) solver: it searches for values that make a collection of logical and arithmetic conditions true. Here the condition asks whether two supported rule encodings return different answers. The facts, output meanings and permitted input domain must first be made comparable.



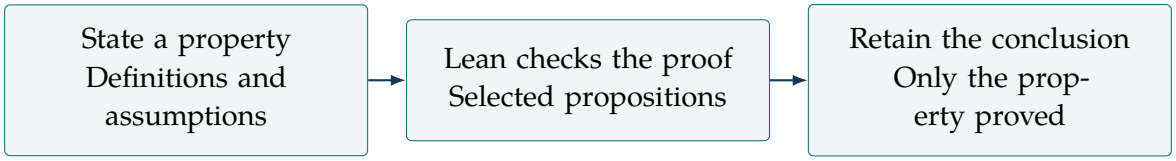
A solver result concerns an encoded comparison. An unsupported operation, timeout or empty domain cannot count as agreement.

The solver adapter covers a stated subset of the formal model and does not check an independent proof certificate from the solver. A found discrepancy is valuable because the original programs can replay it. A no-counterexample result retains the encoding and domain assumptions. Unsupported operations, timeouts and empty domains remain explicit outcomes. Chapter 3 explains those restrictions.

6 continued. Prove a translation and challenge the tests

Proofs of selected mathematical properties

Lean is a proof assistant that checks a formal derivation. The mathematical review used it for selected logical and arithmetic propositions, including three-valued logic, an exception counterexample and conservation under a stated transfer. SymPy, a symbolic algebra system, checked selected algebraic identities.



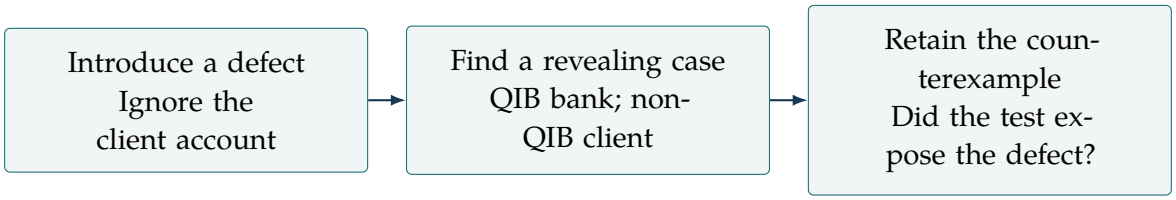
These are checks of selected mathematical claims, not an automatic proof of each generated program or of the whole compiler.

The successor also registers a narrow Lean certificate for Boolean expression lowering. Its fixed checker proves the relation between two formal descriptions; an additional finite check executes the generated Java for every declared Boolean state assignment within its size limit. Arithmetic, dates, full event histories and English meaning remain outside that theorem. A separate Java entry point demonstrates attributed contact histories, keeping the duty trigger distinct from an observed contact and from overall compliance.

For the CoCo purchaser rule, a Lean certificate checks preservation from the formal expression to the shared model. The independent solver comparison covers all 512 assignments of the nine Boolean premises. Native execution is a separate check: both targets receive the same 157 complete, uncertain or expired cases. These are different forms of evidence about a stated model.

Would the tests detect a consequential mistake?

The purchaser tests deliberately remove conditions that matter. One mutation ignores whose account benefits; another ignores the purchaser's US-person status. A revealing input must make the altered rule disagree with the independently expressed relation.

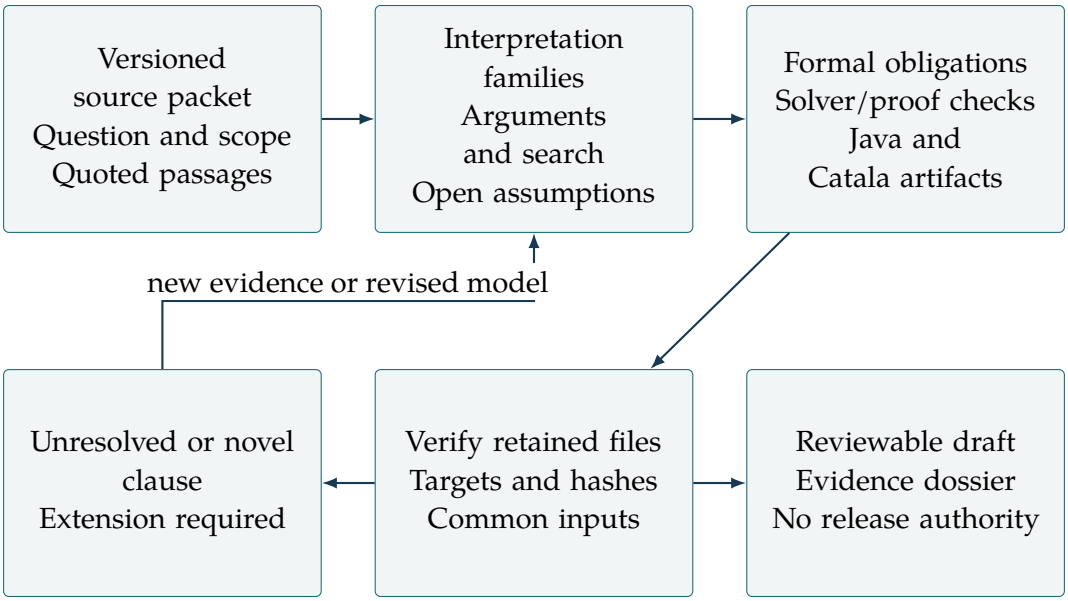


Six altered purchaser rules produced counterexamples. Mutation testing checks sensitivity to those defects; it does not manufacture legal answers or establish that the source interpretation is complete.

Proving that every supported rule preserves its behaviour through translation remains a stronger goal. The present checks do not establish that whole-compiler result or correct interpretation of English. The Stipula/JML/KeY route offers another research approach to particular Java properties.

The proof-carrying dossier and the abstention path

The preceding checks are useful only when their targets and assumptions stay visible. The integration record therefore binds a source packet, competing hypotheses, arguments, proof obligations, method-family results and executable artifacts by hash. It also records shared dependencies: agreement between Java and Catala after the same RuleIR lowering is a translation check, not two independent readings. The controller never changes an unavailable, unknown or unsupported result into agreement.



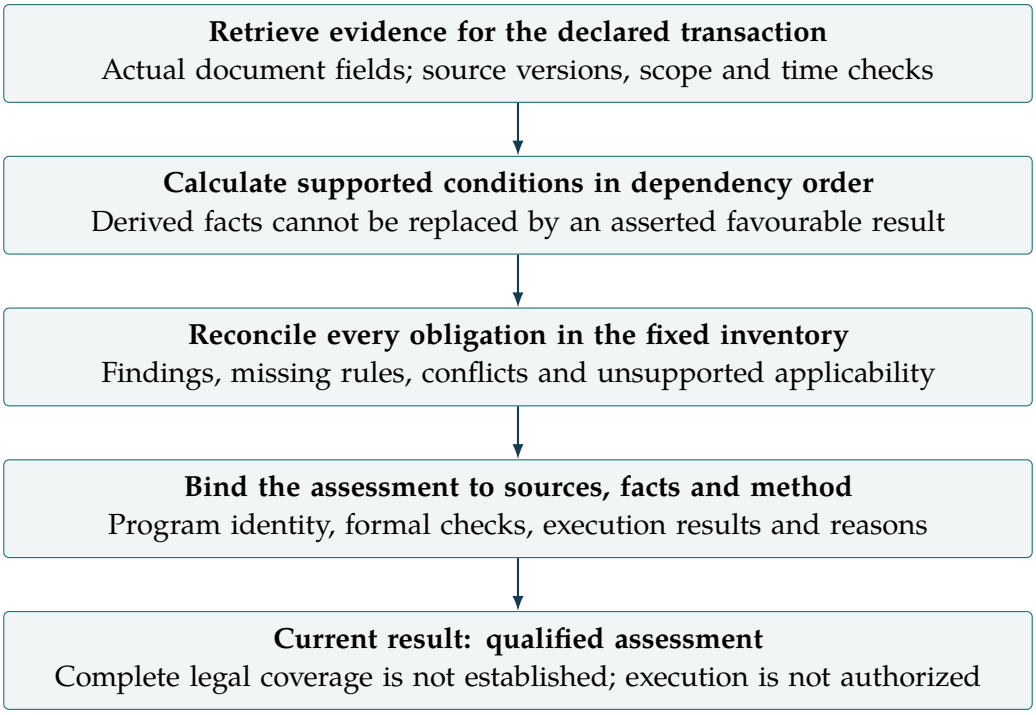
A proof or replay closes the proposition it names. An open authority, future clause, stale result or unavailable required method keeps the dossier on the abstention path.

The gift-circular replay (23EC46) imports five previously proposed readings. Two differ on whether product-directed presentation suffices without an operational connection. The solver finds a separating assignment and the original Python and Java programs replay it. The Catala routes supply another execution target over the shared model: the compatibility backend consumes supported RuleIR, while the native converter can consume a richer model without first making it RuleIR. Agreement after a common lowering is a translation check, not independent legal evidence. The disabled live provider supplies no fresh reading. The witness identifies a legal question; it does not answer it. The result can be delivered as a draft Java package with its evidence and open questions. The transaction assessment must retain the unresolved classification and any missing policy. The exact contract and master command are described in Section 6.34.

The master validates a frozen revision and binds each accepted phase to its inputs and predecessor files. Failure preserves the attempt and triggers a focused repair check before a bounded retry. Success refreshes the next-phase plan. A changed source, checker, plan or allowlist invalidates acceptance; retry exhaustion produces a report of the remaining failure. Neither a retry nor a successful document build supplies missing interpretive evidence.

7. Assemble the transaction assessment

A Java archive (JAR) or a declared Catala package contains the compiled control. Its execution result is only one part of the transaction assessment. The investigation also checks the fixed obligation inventory, applicability, retrieved factual documents and their source dependencies. Each finding keeps its scope, supporting evidence and unresolved questions.



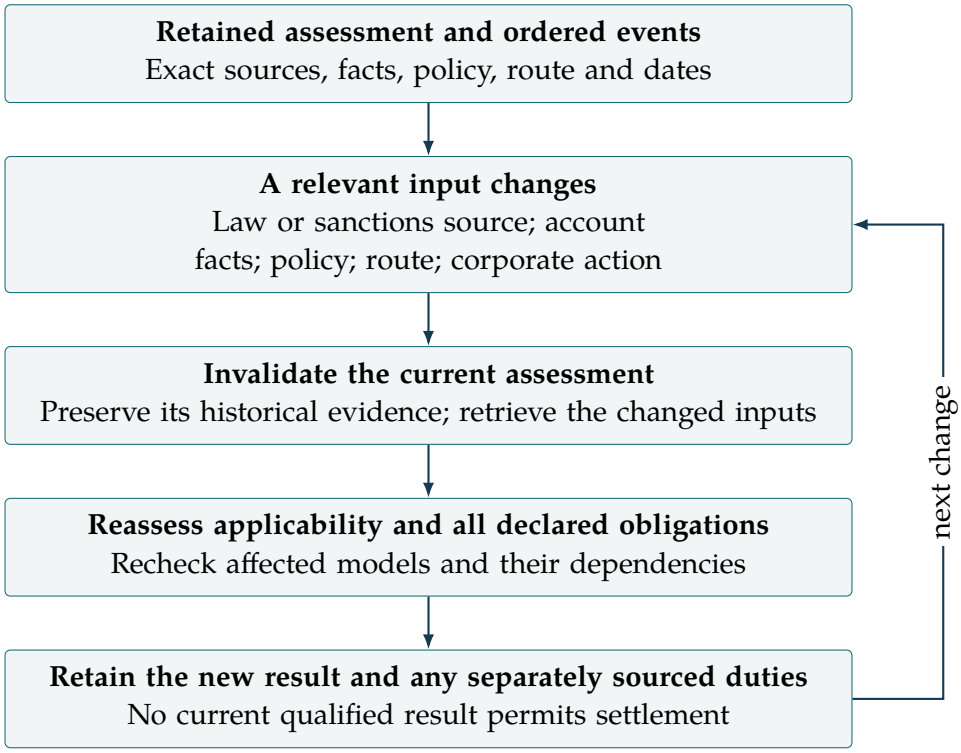
A request supplies evidence and context, not its own shorter list of requirements or precomputed favourable findings. The retained assessment can be reconstructed from its bound inputs.

For example, account amounts enter the financial condition before client qualification is calculated. A transaction threshold also needs its separately declared controls premise. Sophisticated professional investor eligibility then feeds the condition for streamlined procedures. A false sufficient condition does not prove its converse: failure to establish that a bond is complex does not establish that it is non-complex.

The CoCo purchaser result is supplementary evidence attached to this assessment. The original fourteen bank obligations remain present for each instrument. Actual JPMorgan entity, account, policy and settlement facts have not been supplied. The program therefore retains those gaps and returns no transaction execution permission. Recording a bank authorization as a business fact would not turn it into proof of legal interpretation quality.

8. Reassess when evidence or events change

An assessment belongs to particular source editions, factual records, a method and an assessment time. It cannot be reused as though a later transaction had the same circumstances. The event replay records changes and invalidates the current assessment before it considers a later settlement request.



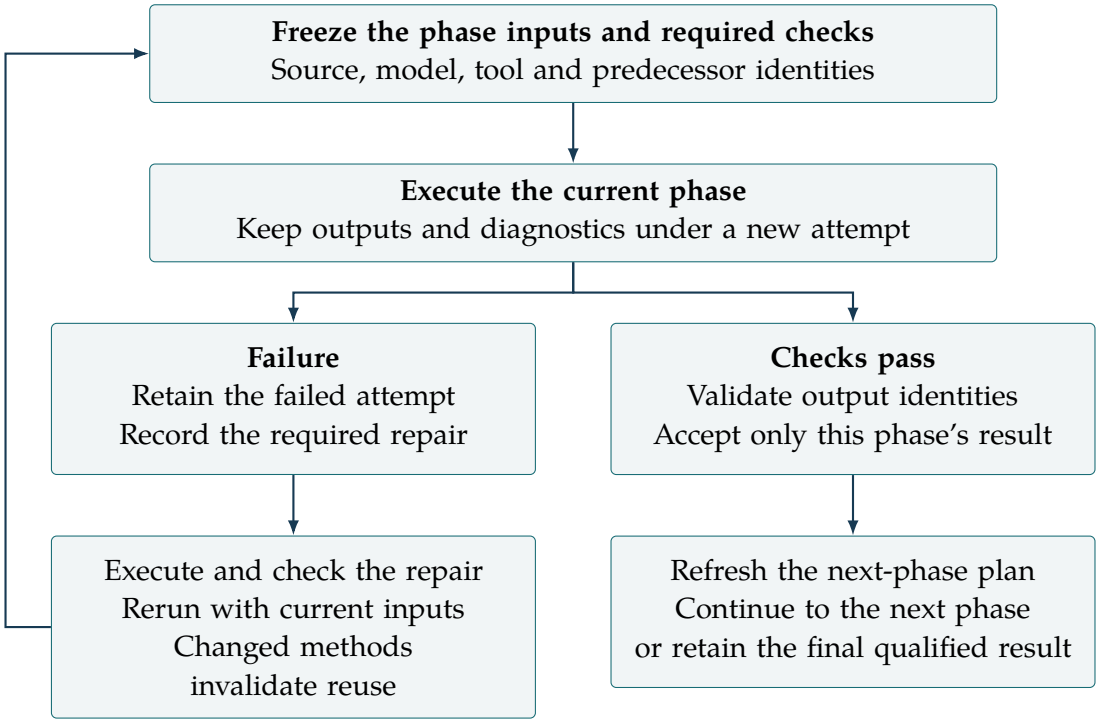
Settlement requires a current assessment for its declared time. A stale or absent assessment is rejected. Passing that freshness check still does not turn a qualified result into permission to trade.

Duties keep the responsible actor, trigger, source and due time separate. Rejection, blocking assets, reporting, rescreening and disclosure are different actions; a sanctions finding cannot silently substitute one for another. A business-day deadline needs an explicit retained calendar within its supported range. Evidence recorded as completion does not prove the action occurred.

Official sanctions snapshots can be refreshed and checked before replacement; earlier bytes remain available for historical investigations. Exact-name matches identify candidates for entity resolution. Ownership propagation for a blocking program cannot automatically be reused for a solely Chinese military-industrial company designation. These scoped mechanisms do not complete identity resolution or every sanctions program.

Repair the implementation; refresh the next phase

The executable master programs preserve the evidence from each phase. The bank and paired-CoCo campaigns validate sources and tools, run regressions, check formal relations, execute both targets, and then assemble the integrated investigation. A phase failure must remain visible until a repair has been executed and checked.



The runner records a repair request; a failed check does not repair itself. Reuse of a successful phase requires matching inputs and output hashes. An engineering pass does not remove the legal qualifications.

Evaluate new situations without human answer labels

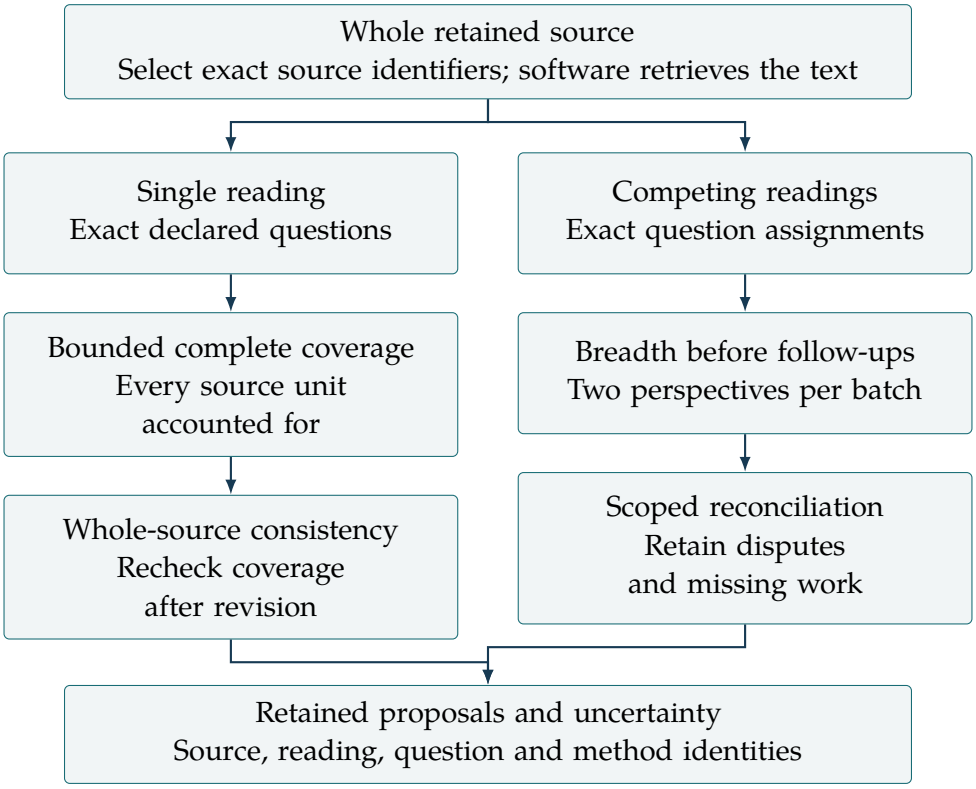
The product requires a machine-checked proposition or an explicit qualification. Human approval and agreement among models are not quality evidence. Independent equations, translation proofs, adversarial cases and source-bound challenges test different possible failures, each within its declared domain.

For future-source work, the observation mechanism freezes the method, inventory and prior sources before it accepts new content. A changed method starts a new series. New-to-inventory documents and generated amendments test novelty handling; they do not establish accuracy on future laws. The present bank series has no real prospective observations. Unknown meaning must remain qualified until evidence establishes the particular claim being made.

Complete the processing without losing the question

The same proof-or-qualification requirement applies when a source investigation must be split into smaller pieces. Finishing those pieces matters because an unexamined exception can invalidate a conclusion drawn from the rest.

Long responses and exhausted review budgets can leave parts of a circular unexamined. Two implemented paths share exact source references. The single reader accounts for its declared questions; the ensemble investigates competing readings. These paths retain their separate results. See Section 6.36 for the mechanics and checked limits.

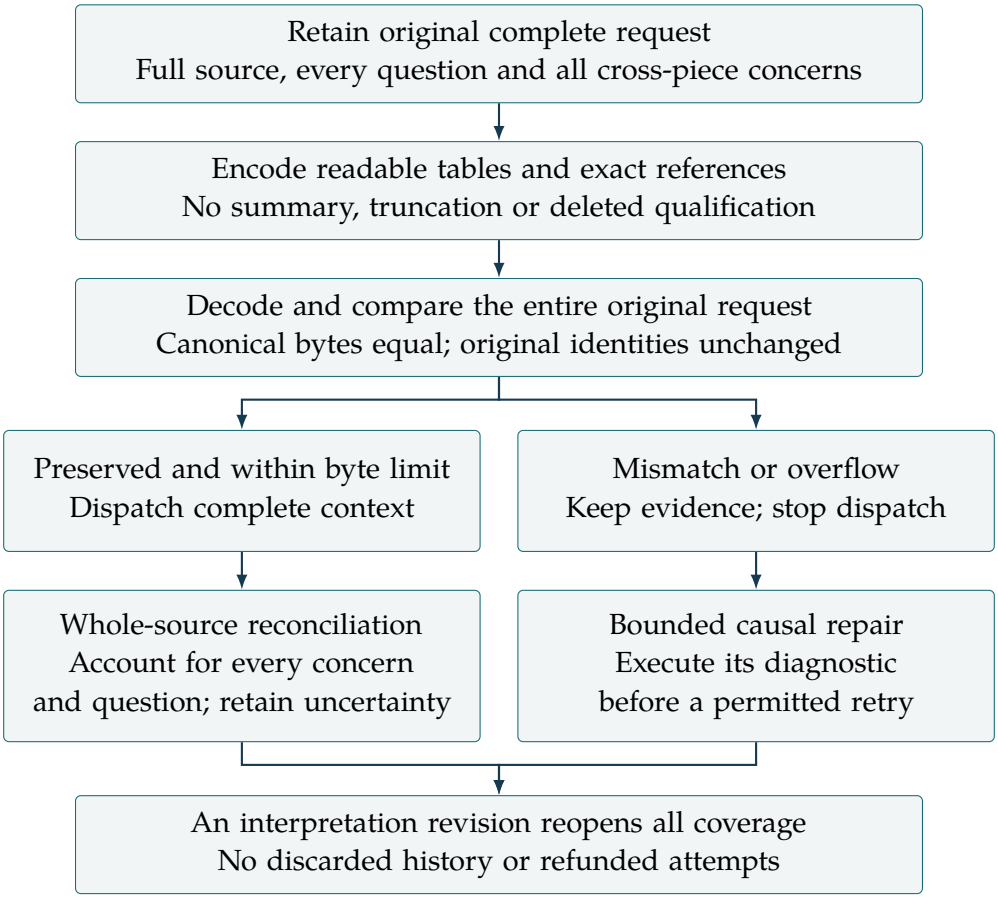


An interpretation revision reopens its coverage checks. Failed requests keep their spent attempts; no batch disappears to fit the budget.

The qualification adapter checks supported formal propositions for a retained reading and exact question. The complete ensemble investigation invokes it when enabled; the split single-reader path retains proposals for subsequent checking. A source identifier proves where the words came from. Coverage proves that the declared units and questions received dispositions. A formal theorem proves its stated relation. None of these alone proves the English interpretation. The four earlier unfamiliar investigations therefore remain incomplete unless their remaining stages actually execute under the recorded limits.

Preserve every concern before shortening a request

The retained authentication exercise reached all 263 source units but could not fit its final whole-source comparison into one request. The optional readable table encoding removes repeated representation while retaining the complete source, questions and explanations. Section 6.37 explains the encoding and its executed checks.

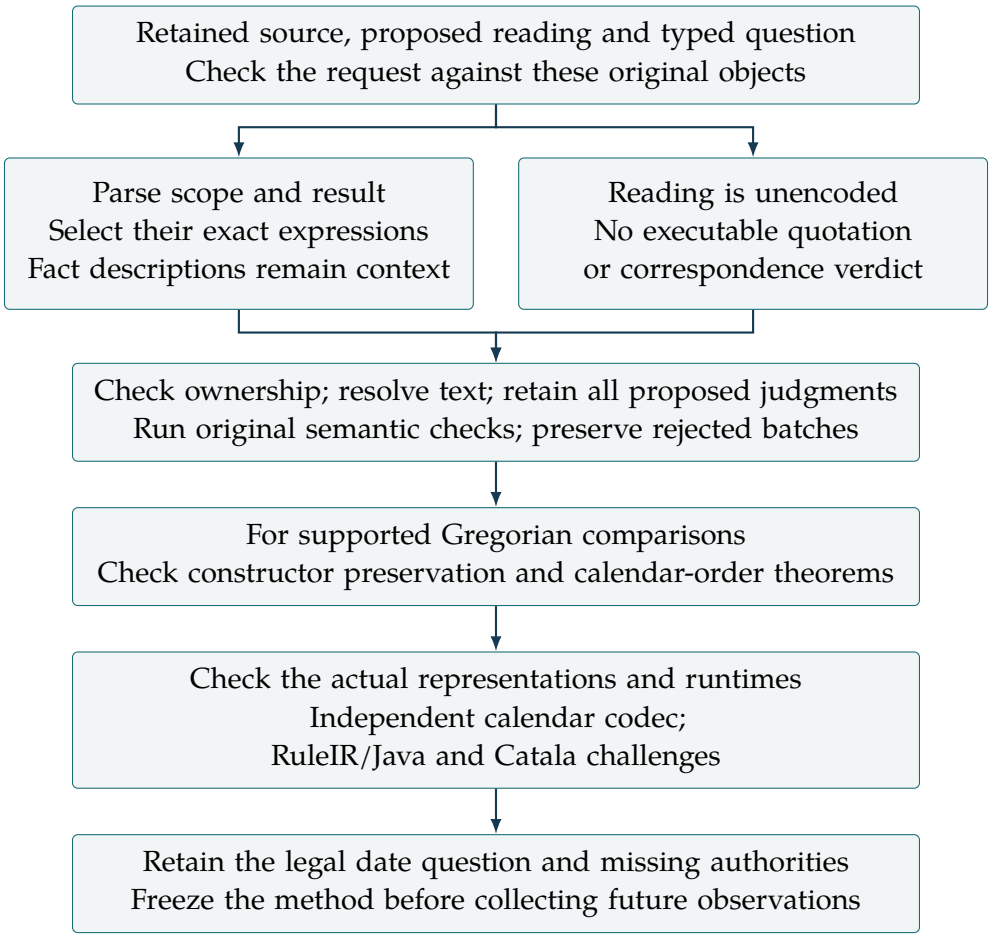


Exact reconstruction permits the capacity comparison; it does not establish legal support. The executed synthetic reconciliation retained every concern as unresolved.

The measured request shrank from 382,396 to 175,016 bytes under the unchanged 200,000-byte limit. A challenge with long unique explanations still exceeded that limit and stopped before dispatch. A future source can therefore produce an explicit capacity qualification. Model comprehension of the encoding and English correctness remain unestablished by these offline exercises.

Check program references and dates independently

Section 6.38 separates exact quotation, mathematical preservation and native execution. Each establishes a different proposition.



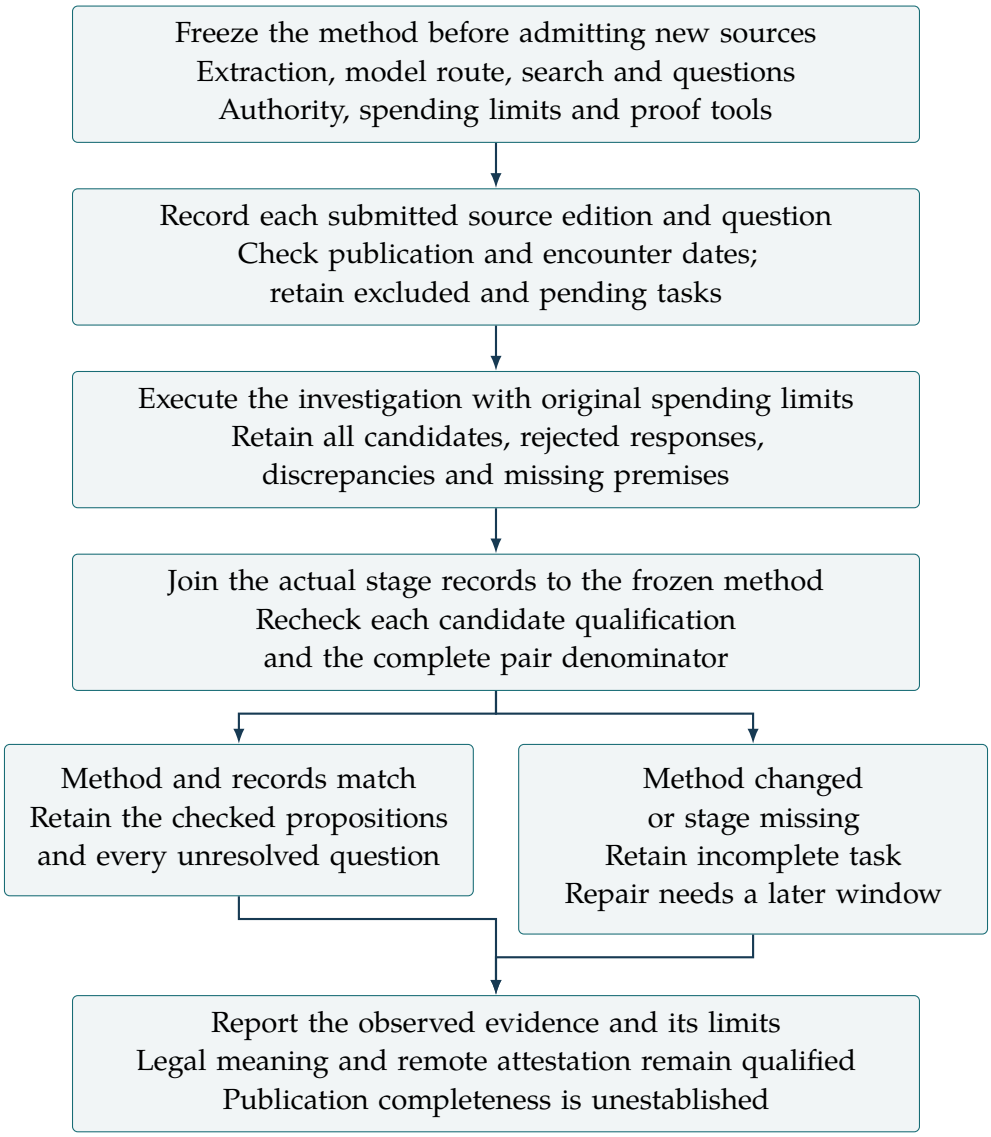
The calendar domain is valid proleptic Gregorian dates in years 1–9999. Business-day rules and selection of the legally relevant date remain separate.

The eight encoded UCITS readings can now receive date-preservation evidence. The ninth remains unencoded, and unresolved source questions persist for all nine. Tests of this implementation do not create future observations or certify the interpretation of an unknown future legal text.

Source extraction and question definition use source references. Version 2 executable references apply only at the correspondence stage. A failed response triggers bounded diagnostic processing; exhaustion retains every unresolved pair and concern. Exact quotation never establishes legal support.

Keep the method fixed when observing future sources

Section 6.38.7 explains why a study must identify the extraction and interpretation process as well as its final proof checker.

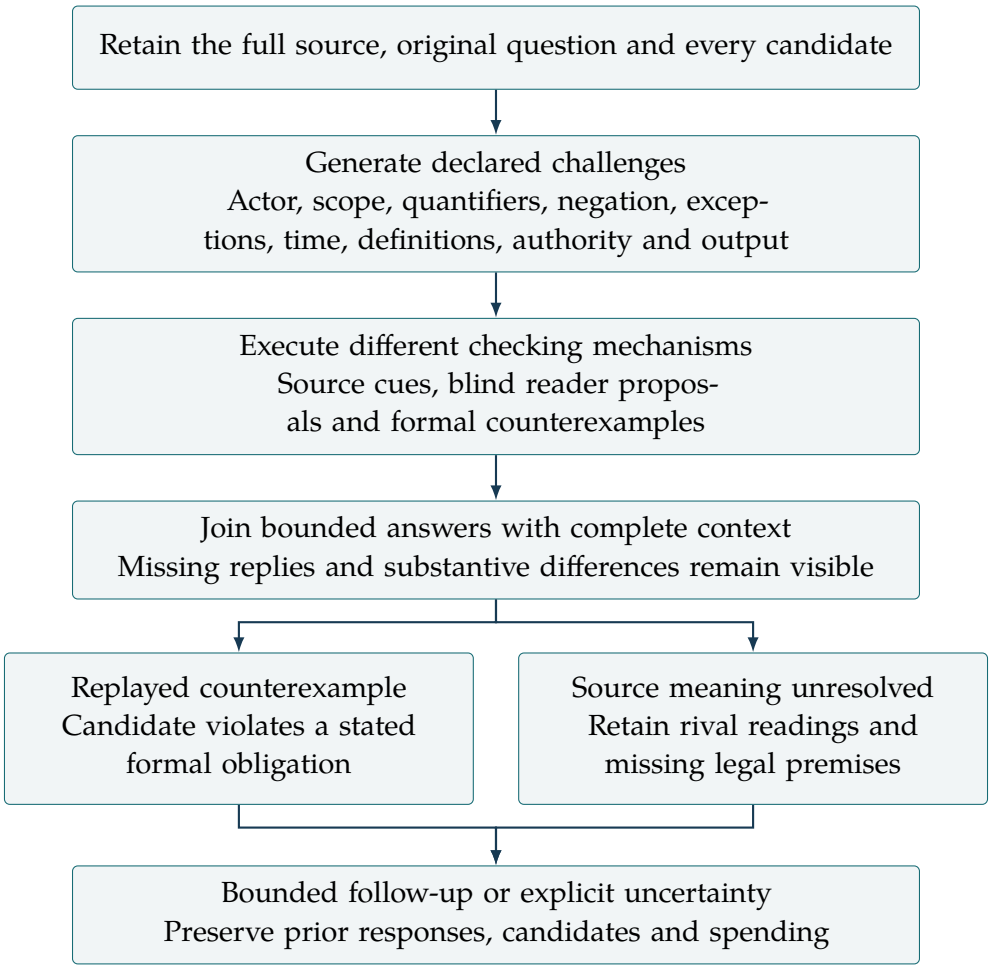


A checked program belongs to a particular investigation. The same program produced from a changed source or method is a different observation.

An additional allowance does not reset the original question's attempts or deadline. A revised source keeps its predecessor's unresolved questions. A repaired method needs a later untouched observation window; no earlier failure is silently turned into evidence for that revision.

Challenge a reading before eliminating it

Section 6.39 explains the distinction between an unresolved source reading and a checked mathematical counterexample.



A counterexample rejects a candidate relative to a declared premise. It does not establish that the English source warrants that premise.

The source scanner cannot prove that all exceptions were found. A finite grammar cannot prove that every legal interpretation was considered. These qualifications remain explicit when the checked stages are joined into one investigation.

Contents

Executive summary **i**

From source documents to transaction decisions **iv**

1 A correct program can enforce the wrong rule **1**

1.1 The decision before the decision 1
1.2 The bank workflow and its regulatory perimeter 2
1.3 The regulatory perimeter is part of the specification 3
1.4 What the existing experiment actually established 4
1.5 Five places where meaning can be lost 5
1.6 The denominator problem in completeness 8
1.7 Why agreement is useful and insufficient 9
1.8 The object being classified is part of the rule 11
1.9 Time changes the question being answered 12
1.10 From an answer to an accountable decision 13

2 Two circulars, from business meaning to executable controls **15**

2.1 SPI qualification: the conditions a wealth flag conceals 15
2.2 A complete worked change: circular 23EC35 to a Java decision 19
2.3 CoCos and preferred shares: law beyond the prospectus 27
2.4 The bank and the transaction also determine eligibility 35
2.5 New prospectuses: payment, scope and external law 48
2.6 From selected terms to a qualified prospectus answer 55
2.7 Marketing rules require exceptions and judgment 69
2.8 The source packet and the question it supports 69
2.9 Reading the surrounding circular before isolating the rule 70
2.10 Naming the six inputs precisely 71
2.11 The named cases are an argument about the rule 73
2.12 What the 729 combinations cover 76
2.13 Mutations ask whether the tests can notice an error 77

2.14 A new discrepancy: what counts as a discount? . . . 78
2.15 Finding the source does not settle its date or its meaning . . . 80
2.16 A complete dry run of the proposed workflow . . . 81
2.17 Who decides what the correct answer should be? . . . 82

3 What can be proved, and under which assumptions 84
3.1 The vocabulary before the proof chain . . . 84
3.2 Giving a correctness claim a precise target . . . 85
3.3 Quantifiers change which entity carries the obligation . . . 88
3.4 Necessary conditions, sufficient conditions and permissions . . . 89
3.5 A proof obligation should expose its assumptions . . . 90
3.6 Expressions, types and meanings . . . 93
3.7 Unknown is an information state . . . 96
3.8 Conflict, scope and errors are not extra Boolean values . . . 98
3.9 A readable specification with a fixed translation . . . 99
3.10 How a preservation argument is constructed . . . 100
3.11 Counterexamples and the domain they live in . . . 101
3.12 Proving a workflow property . . . 105
3.13 Temporal duties require histories . . . 106
3.14 An assurance claim is a conjunction of different evidence . . . 107

4 Languages that make interpretation inspectable 108
4.1 Choose the question before choosing the language . . . 108
4.2 Rules do not all behave like ordinary implications . . . 109
4.3 Controlled English: readable structure with restricted freedom . . . 112
4.4 Catala: defaults, exceptions and source-linked authoring . . . 115
4.5 Metadata is part of the meaning: LegalRuleML . . . 118
4.6 Stipula: a contract as parties, resources and transitions . . . 119
4.7 A small state contract that a reader can execute by hand . . . 122
4.8 What contract verification does and does not buy . . . 125
4.9 Other normative languages clarify different distinctions . . . 130
4.10 Decision tables and business-rule engines . . . 133
4.11 A solver can improve reasoning after a translation . . . 136
4.12 Roundtrip verification and targeted repair . . . 141
4.13 Evidence spans and distant qualifications . . . 142

4.14 Testing paths, dates and inconsistent constraints . . . 143
4.15 A genuine citation can still support the wrong answer . . . 146
4.16 Where the literature still leaves this product exposed . . . 148
4.17 A synthesis for the first implementation . . . 149

5 Constructing and comparing competing interpretations 150
5.1 Look for choices that change a decision . . . 150
5.2 Legal theories are more than predicted outcomes . . . 152
5.3 What case-based argument contributes . . . 153
5.4 AGATHA: explicit construction of rival theories . . . 154
5.5 Arguments, attacks and unresolved conclusions . . . 155
5.6 Burdens, assumptions and evidence in Carneades . . . 157
5.7 Breadth before expensive investigation . . . 158
5.8 Tree search and the meaning of a reward . . . 159
5.9 Choose questions that separate the surviving readings . . . 160
5.10 Uncertainty about meaning is not a vote count . . . 162
5.11 Reusing research without importing its assumptions . . . 163

6 Using disagreement to improve a review 165
6.1 The design objective and its limits . . . 165
6.2 Why diversity has to be designed . . . 166
6.3 The ensemble members and their responsibilities . . . 168
6.4 Build a source perimeter before interpreting it . . . 169
6.5 Freeze blind proposals before reconciliation . . . 170
6.6 Detect discrepancies at the right level . . . 171
6.7 Keep the same question open across revisions . . . 172
6.8 Finishing the work does not settle the question . . . 173
6.9 Choose a remedy that can answer the question . . . 175
6.10 What evidence would settle each disagreement? . . . 176
6.11 Resolution can reveal that the first question was wrong . . . 177
6.12 Follow the cause shared by several disagreements . . . 178
6.13 Set limits on an investigation before it starts . . . 180
6.14 No progress is a substantive stopping reason . . . 181
6.15 Choose the next useful question . . . 182
6.16 How an investigation proceeds . . . 184

6.17 Termination and the boundary of the proof . . . 185
6.18 Recovering a request whose answer never arrived . . . 186
6.19 What counts as a justified repair . . . 188
6.20 Scores that assist investigation without pretending to be probabilities . . . 189
6.21 A complete dry run: repairing an explicit omission . . . 190
6.22 A complete dry run: an ambiguity that survives the budget . . . 191
6.23 Tell the reviewer what remains uncertain . . . 192
6.24 Agreement can still conceal a missed obligation . . . 194
6.25 Interaction between uncertainty and operational action . . . 195
6.26 Acceptance conditions for the central method . . . 196
6.27 From the ensemble design to executed assurance . . . 197
6.28 Assurance through methods with different opportunities to fail . . . 197
6.29 From complementary checks to a working investigation . . . 203
6.30 A certificate requirement is a different question from a compliant submission . . . 205
6.31 A decision can agree before its interpretations do . . . 210
6.32 The errors that agreement can leave hidden . . . 214
6.33 Repairing the distinctions that a comparison can lose . . . 217
6.34 A proof-carrying investigation joins the methods without joining their claims . . . 222
6.35 From an interpretation dossier to a continuing investigation . . . 227
6.36 Preserving the whole investigation under bounded processing . . . 238
6.37 Keeping the complete question inside a bounded request . . . 243
6.38 Checking executable quotations and calendar comparisons . . . 245
6.39 Challenge each interpretation before narrowing the alternatives . . . 253
6.40 What case law can contribute to an interpretation . . . 260

7 From an accepted interpretation to verified Java behavior 268
7.1 The specification becomes the comparison target . . . 268
7.2 Writing a rule precisely enough to execute . . . 269
7.3 Several duties, several answers . . . 271
7.4 Facts are evidence-bearing values . . . 272
7.5 Exact arithmetic prevents a second kind of ambiguity . . . 273
7.6 Unknown, conflict and scope have an order . . . 275
7.7 Validation failures are part of observable behavior . . . 277

7.8 Independent oracles and their limits . . . 278
7.9 Mutation tests measure what the checks can notice . . . 280
7.10 Symbolic comparison requires a feasible domain . . . 281
7.11 Property tests should follow the semantics . . . 282
7.12 Following the accepted rule into Java . . . 283
7.13 What the bank must supply at the point of use . . . 284
7.14 Traces are part of the delivered meaning . . . 284
7.15 Event histories add obligations that a snapshot cannot express . . . 285
7.16 Approve the version that was actually tested . . . 289
7.17 A correct decision can still race with the transaction . . . 290
7.18 A verification dossier that supports a release decision . . . 292

8 What the bank would own and operate 293
8.1 Begin with the decision a person needs to make . . . 293
8.2 The shared frontend and the Catala routes . . . 293
8.3 Follow one change from source to release . . . 296
8.4 Keep the release tied to its meaning . . . 297
8.5 What the prototype has and has not demonstrated . . . 297
8.6 The boundary before a bank pilot . . . 298

9 Measuring error, uncertainty and the value of redundancy 299
9.1 What the completed RuleIR–Catala comparison can answer . . . 299
9.2 The experiment must ask the right question . . . 301
9.3 Define the unit of evaluation . . . 303
9.4 Build the reference before seeing the model's answer . . . 304
9.5 Construct a corpus that exposes the expected failures . . . 305
9.6 Use a baseline ladder with comparable resources . . . 307
9.7 Measure omissions, alternatives and useful abstention . . . 311
9.8 Severity and direction of error need explicit treatment . . . 312
9.9 Zero observed errors does not mean zero risk . . . 313
9.10 Calibration asks a narrower question than truth . . . 316
9.11 Conformal prediction can support sets under stated assumptions . . . 317
9.12 Search coverage and semantic completeness are different . . . 318
9.13 Evaluate the reviewer, not only the model . . . 319
9.14 Economic value and operating cost . . . 320

9.15 Promotion decisions and the next evidence needed 321

10 Operating the product without losing its evidence 323

10.1 The bank remains responsible for the control 323
10.2 What prevents reliance on the present product for bank decisions 324
10.3 Start with a narrow operational perimeter 327
10.4 Keep regulatory meaning distinct from bank policy 329
10.5 Shadow operation is a study, not a hidden release 330
10.6 Source monitoring begins a new interpretation cycle 331
10.7 Amendment, actor and conditional language: tokenised products 332
10.8 A worked amendment scenario 334
10.9 Bitemporal replay explains rather than rewrites history 335
10.10 Operational responses to unknown and conflict 336
10.11 Risks that shape the implementation 337
10.12 Incident analysis should locate the failed boundary 338
10.13 Availability failures must not change the meaning of a result 339
10.14 Model updates are method changes 340
10.15 A release readiness review with concrete evidence 340
10.16 Three operational cases for the implementation team 342
10.17 Maintaining a useful literature and method record 343
10.18 Decisions needed at the start of a bank prototype 345
10.19 A practical sequence from manuscript to product 345
10.20 What the prototype establishes and what must be measured 346

Chapter 1

A correct program can enforce the wrong rule

1.1 The decision before the decision

Imagine a distributor preparing an offer for an investment fund. The offer includes a fee reduction and a travel voucher. On Monday, the compliance team asks whether the promotion may be released. The answer appears to require only a few fields: is there a gift, is the fund connection established, and does the fee-discount exception apply?

Those fields already contain several decisions. The reviewer must decide whether the voucher and fee reduction are separate components, what the circular means by an investment product, which evidence establishes the promotional link, and which edition of an incorporated rule applies on Monday. A program can evaluate the resulting condition perfectly while still answering a question no reviewer intended to ask.

Suppose the adopted reading covers both a named fund and a type of fund. A test built around one named fund can pass every case written for it while omitting the second route entirely. The failure sits before the code. No amount of repeated execution can reveal a requirement that never entered the rule.

Now change only the subject of the exception. The fee reduction may qualify as a discount, while the voucher remains a separate incentive. A formula that stores one "discount" flag for the whole offer can be internally consistent and still attach the exception to the wrong thing. The missing information is not a programming detail. It is part of the regulatory decision.

These are hypothetical cases drawn from the retained 23EC46 exercise. They do not report a finding about a live bank. They show why a bank needs a reviewable route from a source passage to a business decision, with the unresolved parts named and assigned to the people who can decide them.

The work therefore begins before rule implementation. It records the business scope, the source passages, the readings that have been considered, the facts needed to apply them, and the authority for resolving an open question. Code comes after those choices and remains answerable to them.

In the rest of this book we call that package a *regulatory change assessment*. The name is less important than its contents: source material, scope, proposed meaning, affected controls, data requirements, implementation changes and review decisions. One circular may create several assessments when its provisions concern different actors or activities. One assessment may also need a code provision, an FAQ and an earlier decision.

Keeping those connections visible prevents a downloaded page from masquerading as a complete specification.

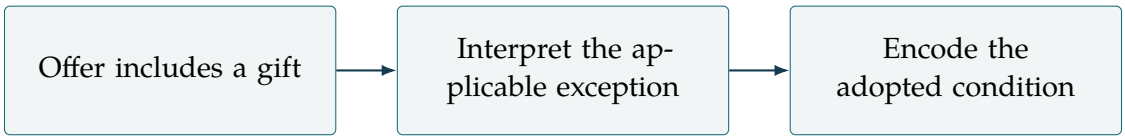


Figure 1.1: A coding decision begins with a decision about meaning.

For a synthetic promotion offering a travel voucher, the first question is whether the relevant source treats that voucher as a gift or as a fee discount. A program can apply either classification consistently. Consistency becomes useful only after the classification and the offer's regulatory scope have been reviewed.

1.2 The bank workflow and its regulatory perimeter

A private bank must turn regulatory requirements into client classifications, product permissions, disclosures, monitoring, records and staff actions. Those requirements rarely arrive as a complete algorithm. A circular may change one part of an existing regime, depend on definitions in a code, distinguish product providers from distributors, or explain an exception in an annex. Software must nevertheless decide what to do for a particular client, product and event.

The starting business concern is that manual interpretation and system updates can lose a requirement. No measured incident rate is available here. Consider a plausible process, to be checked against the bank's actual practice. A compliance officer reads a circular, writes an interpretation, and discusses it with product and operations staff. A business analyst turns the interpretation into requirements. Engineers implement configuration or code, and testers construct examples from those requirements. Each participant can perform competently while a condition disappears between documents. When the circular changes, the bank must reconstruct the chain.

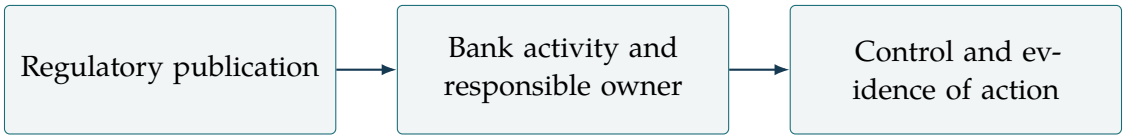


Figure 1.2: A publication becomes a banking task through its effect on an activity.

The product opportunity is to make that chain inspectable and reusable. A reviewer should be able to select a rule and see the passage that supports it, the interpretation adopted, the facts it needs, the situations in which it changes an outcome, and the version actually used. A subsequent amendment should identify the affected controls and test cases without relying on the memory of the original implementation team.

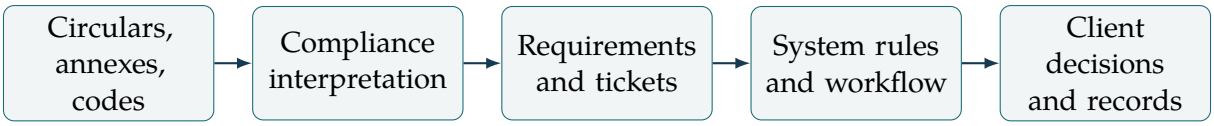


Figure 1.3: The proposed discovery exercise will map the bank's actual process onto these transitions. LegalMath preserves the source, interpretation and tests across them, so a reviewer can follow a decision back to its reason.

1.3 The regulatory perimeter is part of the specification

The main client case concerns a sophisticated professional investor, abbreviated SPI. The marketing case concerns a distributor offering incentives alongside a fund. Both require a decision about which sources govern the activity.

The supplied Hong Kong Securities and Futures Commission (SFC) product-authorization page is a useful discovery source, but it is not a complete inventory of a private bank's distribution obligations. The SPI guidance examined here is a joint circular from the SFC and Hong Kong Monetary Authority (HKMA) to intermediaries. The tokenisation circular distinguishes product-provider responsibilities from distributor responsibilities and imports other product and intermediary rules. The marketing circular refers to the Securities and Futures Ordinance, the Code of Conduct and advertising guidance. These dependencies determine the meaning of the extracted clauses (SFC, 2026b; SFC and HKMA, 2023a; SFC, 2026c, 2023b).

Discovery therefore starts with the bank's legal entities and activities. For a specified business, the inventory should record whether the institution acts as distributor, product provider, adviser or another regulated actor, and which client and product classes are involved. Being a Hong Kong office of a US banking group does not by itself settle any of these questions. The prototype will record the selected Hong Kong perimeter and leave other group and jurisdictional requirements as separately identified dependencies. A broader rollout requires the bank's authoritative inventory.

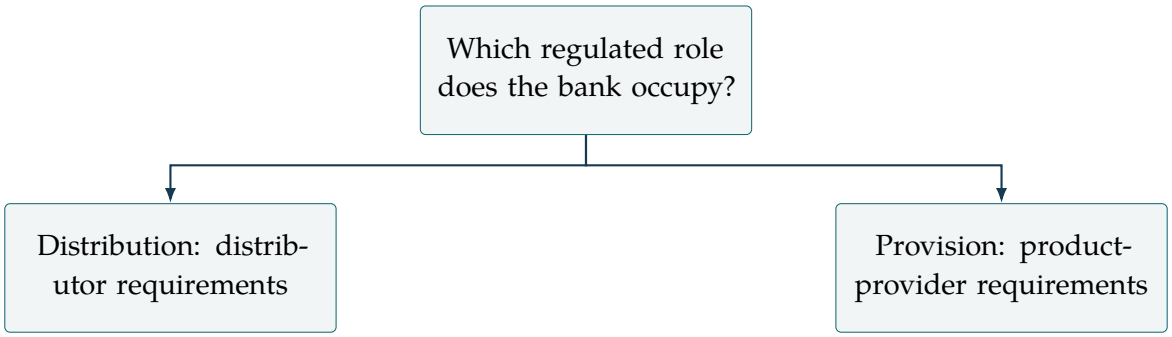


Figure 1.4: An institution's name does not settle which regulated role it occupies.

A bank that distributes a fund and also provides a tokenised product may occupy two roles. Each role needs its own applicability assessment. A requirement imposed on the provider does not become a distributor requirement merely because both activities occur within the same group.

Document classification also matters. The September 2025 joint circular announces a concurrent thematic review of distribution practices and describes what the review will examine. It should enter regulatory-change tracking; its description of the review is not, by itself, a new numerical client-eligibility test (SFC and HKMA, 2025). A translator that treats every sentence as a binding conditional will manufacture rules. Conversely, a translator that extracts only sentences containing a particular modal verb can miss definitions, incorporated requirements and material explanatory guidance.

LegalMath should preserve a passage's role: definition, requirement, permission, exception, supervisory expectation, explanation or announcement. The original wording and its authority remain alongside the adopted implementation. A bank may choose an additional operational restriction, but that choice should be recorded as bank policy with an owner and rationale. It must not be silently attributed to the SFC.

1.4 What the existing experiment actually established

The present LegalMath workbench provides a useful starting point because it has already crossed the engineering boundary from a formal rule to generated Java. Its second-circular exercise concerns a deliberately narrow interpretation of paragraph 10 of SFC circular 23EC46, dated 24 October 2023 (SFC, 2023b). The example assesses one incentive component in a specified distributor and SFC-authorised-fund profile. Whether the component is a gift, whether it is a fee discount, and whether it has a relevant product link are supplied classifications. The program does not establish those classifications by reading an offer document.

The early exercise preserved a packet of 22 named expected outcomes before constructing the candidate rule. A separate truth-table implementation enumerated 729 combinations of true, false and unknown values across six inputs. The actual generated Java class and the Python evaluator agreed on the declared results. Four deliberately altered rules were compiled and detected by the named cases. Those are substantial engineering observations: they show that the exercised implementation computes the proposed rule in the checked domain, and that the cases can expose several particular errors.

They do not provide independent evidence that the proposed rule is the correct reading of the circular. The same implementing agent authored the interpretation and the expected outcomes. Writing one before the other makes later alteration detectable; it does not create a second legal judgment. The separate calculation of expected answers avoids copying the candidate program's structure. It still expresses the same chosen formula. If that choice omitted a legal condition, all three implementations could agree on the omission.

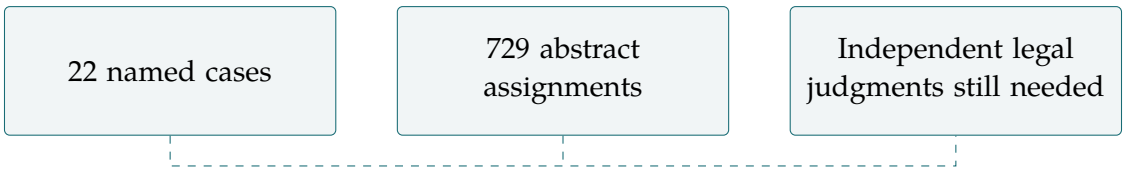


Figure 1.5: Three different denominators answer different questions.

If all 729 assignments pass, the evidence covers the chosen six-input formula. It does not supply 729 independently read legal provisions. A single omitted definition can affect every assignment simultaneously.

The candidate also retained five unresolved review issues and two unresolved source dependencies. Its attempt to enter review was rejected. It remained a draft and no release was created. That refusal is part of the useful result. The workbench did not mistake successful compilation for authority to deploy a regulatory control. The monograph preserves this example as a record of an unapproved candidate; resolving the cited dependencies here by assumption would destroy the very distinction the example teaches.

The wider application was also tested for collecting sources, preserving records, evaluating rules and controlling review and release. In that exercise, the drafting service returned prepared examples. It did not interpret unfamiliar circulars in a measured evaluation of a live language model. Its 154 passing tests therefore establish behaviour under the tested conditions; they do not estimate interpretation accuracy. The detailed execution record is retained in Appendix D of the technical companion.

Later development put the shared-error concern to a direct test. Three altered programs deliberately omitted the fee-discount exception and agreed with one another. A separate examination of the source identified the missing support; the exception-preserving control supplied a comparison. The defect was inserted for the test. Finding it shows how a source check can challenge unanimous code, without estimating how often an unfamiliar circular would produce such an error.

The interpretation service also progressed beyond prepared responses. Live development runs generated alternatives for the marketing and settlement-readiness circulars and checked them against separately collected source requirements. Both investigations retained unresolved questions. The later integration work combined explicitly scoped controls, used official examples conditionally and replayed source changes through Java. Appendix A records those results and their reproduction paths. The early draft described above remains an early draft; new source collection does not retrospectively approve it.

This distinction suggests the question the new product must answer. Given a source packet and a proposed operational scope, can it expose a material misinterpretation before that misinterpretation becomes an approved executable control? Success requires more than generating valid syntax. It requires finding missing alternatives, retrieving the context that changes a reading, and making disagreements concrete enough for a competent reviewer to resolve.

1.5 Five places where meaning can be lost

It is tempting to describe the workflow as one translation from English to Java. That description hides several different kinds of error. The first is source error: the reviewed text is incomplete, extracted incorrectly, superseded, or wrongly treated as authorita-

tive. An omitted footnote can remove a definition. A copied paragraph can lose a heading that limits its scope. A source's publication date can be mistaken for its effective date. None of these failures requires a language model to reason incorrectly; the model may never receive the material needed for the correct decision.

The second is interpretation error. The words are present, but their legal function is misrepresented. A qualification becomes an unconditional rule. An exception is attached to the wrong object. A statement about possible classification becomes a finding of fact. A normative expectation is converted into an operational block without recording the bank's choice. These are decisions about meaning, not merely extraction or programming defects.

The third is specification error. The reviewer may have a defensible interpretation, but the intermediate rule expresses something else. A conjunction is replaced by a disjunction, a threshold loses equality, or an unknown condition is interpreted as false. The specification can also fail through insufficient expressiveness: a time-dependent obligation may be forced into a single true-or-false test. A software language that cannot represent a required distinction must report the gap rather than approximate it silently.

The fourth is execution error. Java can evaluate a correct intermediate rule incorrectly because of a compiler defect, numeric conversion, time-zone mismatch, or inconsistent evidence handling. A data adapter may supply dollars where the rule expects cents. A build may package an older runtime. An apparent match on a few simple examples can miss a defect in unknown-value or conflict precedence. These failures are often amenable to strong formal or exhaustive checks once the specification and input domain are fixed.

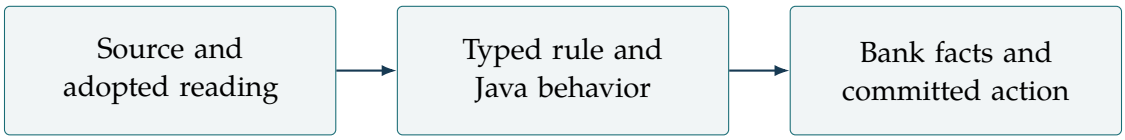


Figure 1.6: Meaning can change at each translation boundary.

A source can be read correctly while a host sends dollars into a field declared in cents. Conversely, exact arithmetic can faithfully execute a rule that omitted an exception. The investigation must locate which transition introduced the error before choosing a repair.

The fifth is operational error. The correct Java result can be used for the wrong client, product, source version or transaction time. Consent can change between evaluation and order commitment. A system can treat the absence of one prohibition as permission to proceed despite unresolved checks elsewhere. A reviewed rule can remain active after its source is replaced. These are failures of integration and change control, even when the individual rule evaluator is correct.

Figure 1.7 gives each transition its own question. The arrows do not imply that success flows automatically from one stage to the next. Every arrow needs evidence appropriate to the claim it makes.

A useful consequence follows immediately. Two checks count as meaningful redundancy only to the extent that their opportunities for failure differ. A second compiler

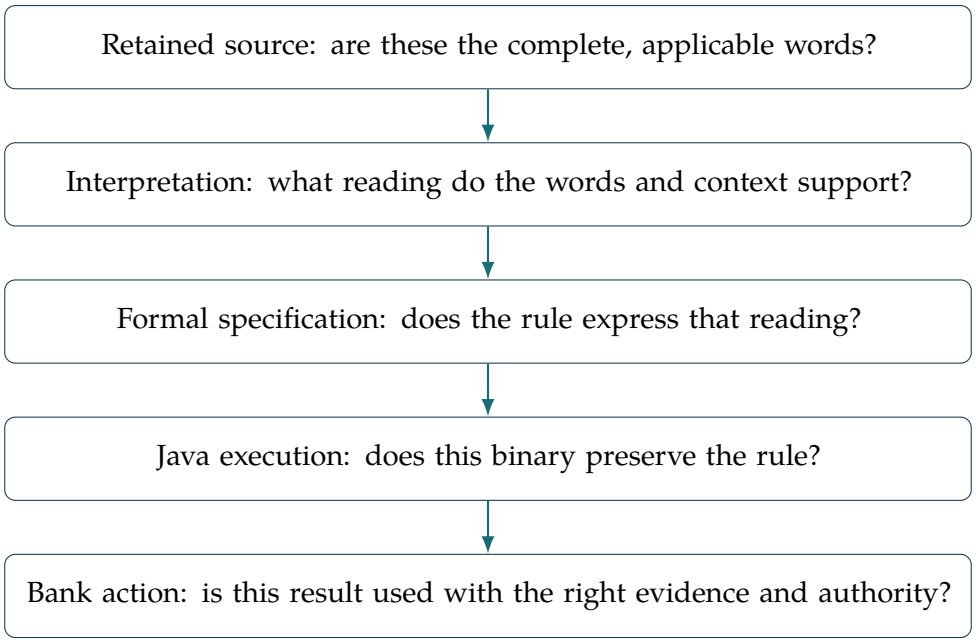


Figure 1.7: A correct downstream step does not repair a mistaken upstream premise. Each transition has a different assurance question.

can challenge code generation. It cannot detect a missing source document if it receives the same incomplete specification. A second language model may propose a different reading, but it cannot authenticate a source merely by describing it confidently. Redundancy must be designed around the failure being investigated.

1.5.1 What makes the current work expensive and hard to verify

The common failure is loss of context. Thresholds detach from definitions; exceptions detach from their parent rules; duties detach from actors and events; and historical decisions detach from the source version that justified them. Document search helps locate text, but it does not preserve those relationships through implementation. More fluent summaries can make an omitted condition harder to notice.

Review then becomes repetitive. Compliance officers must infer what an engineer understood, engineers must infer what a compliance memo leaves implicit, and testers must construct enough cases to expose mismatches. If tests are written only from the implementation ticket, a mistranslation shared by the ticket and code can pass every test. After an amendment, teams may retest too little because dependencies are hidden, or retest too much because they cannot show what is unaffected.

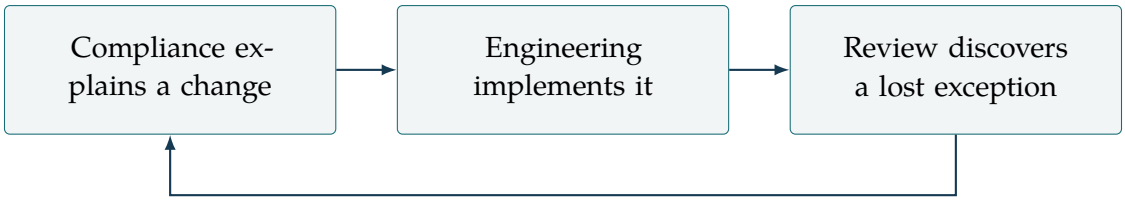


Figure 1.8: Repeated translation creates repeated opportunities for meaning to change.

The proposed product makes each material interpretation a maintained part of the system. Its value should be measured in fewer material defects, more complete explanations, and less repeated review effort. Those benefits are hypotheses for the prototype to test. The evidence needed to test them includes independently reviewed cases, disagreements that remain visible, and the time spent resolving them; a high extraction score alone cannot establish the business case.

1.6 The denominator problem in completeness

Suppose a drafting agent extracts eight requirements from a circular and produces eight rules. A coverage report then shows that every extracted requirement has a rule, every rule has a test, and every test passes. The result sounds complete. It is complete only relative to the agent's own extraction. If the circular contains a ninth operative requirement, the report has no row on which that omission can appear. The system has used its answer to define the denominator of its completeness measure.

The source inventory must therefore originate independently of the proposed rule set. At the document level it records paragraphs, headings, annexes, footnotes, tables and incorporated references. At the normative level it records potential duties, prohibitions, permissions, definitions, exceptions and qualifications. These two inventories serve different purposes. A paragraph inventory can establish that a passage was considered, but one paragraph may contain several operative propositions. A rule inventory can establish that an identified proposition has a disposition, but it cannot prove that the propositions were all identified.

For the retained circular, the existing inventory contains 18 numbered paragraphs and four footnotes. Only one paragraph has an executable control in the second-circular example. Other passages concern context, other controls, qualitative assessment or external dependencies. It would be wrong to describe the circular as implemented because its selected gift rule passes 729 input combinations. It would also be wrong to implement every paragraph mechanically: an observation about market practices is not necessarily a new obligation, and a qualitative duty may need a reviewed procedure rather than a fabricated numeric threshold.

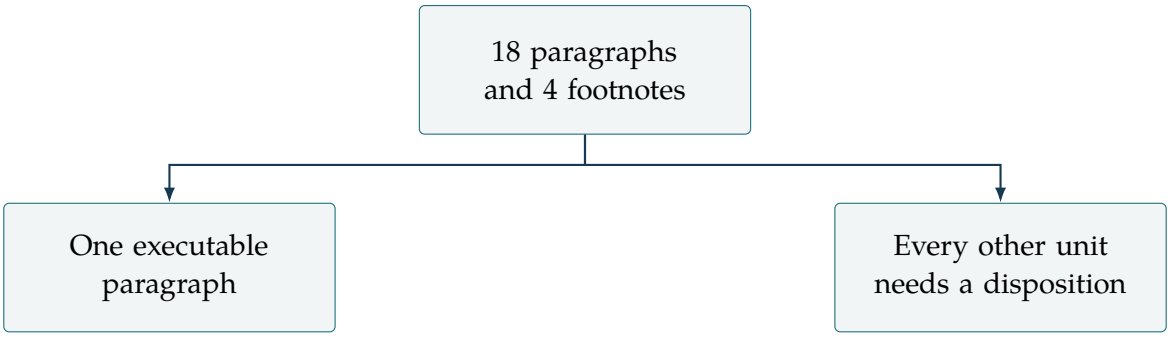


Figure 1.9: Completeness needs a visible denominator.

A report saying that paragraph 10 was encoded can be accurate. A report saying that the circular was translated requires an account of all the remaining paragraphs and footnotes, including provisions outside the selected executable scope.

A complete disposition process gives every inventoried proposition a reasoned destination. It may be linked to an executable rule, to a reviewed manual procedure, to a justified exclusion from the declared assessment, or to an open question. An open question is visible incompleteness. An exclusion requires a scope rationale and reviewer authority. A manual procedure needs an owner and evidence of performance; attaching the word "manual" to an inconvenient clause does not implement it.

The same denominator problem occurs in search. If five candidate interpretations agree, the correct conclusion is agreement among those five candidates under their recorded assumptions. The system has not measured the interpretations it failed to generate. Breadth constraints and targeted challenges improve the opportunity to find missing readings, but there is no finite list of prompts that proves unrestricted legal interpretation has been exhausted. The product must report which dimensions were explored and which remain outside the search.

This makes completeness a collection of bounded claims. We can check that every retained paragraph has a disposition. We can check that every cited reference has been resolved or explicitly left open. We can check that each represented exception has positive, negative and uncertain cases. We cannot infer from those facts that no relevant circular exists elsewhere or that every legally defensible reading has been found. A bank needs a process for setting and maintaining its regulatory perimeter as well as a translator inside that perimeter.

1.7 Why agreement is useful and insufficient

An ensemble is a group of methods whose results are examined together. In a prediction task it is common to combine answers by majority vote or averaging. For regulatory interpretation, the useful unit of combination is often a reasoned claim: one method identifies a scope restriction, another locates an exception, and another generates a case that exposes a missing promotional route. Their outputs need not be identical to contribute to the same assessment.

Agreement can provide evidence that a representation is stable across some transformations. A controlled-English sentence and its formal rule may produce the same decisions. A back-translation may preserve the formal structure. A second implementation may agree with the first. Each observation rules out some failure explanations. None rules out every explanation. In particular, methods that share the same omitted premise can agree for the same wrong reason.

Disagreement is therefore not merely a nuisance to suppress. It may be the first observable sign of a missing definition, a conflict in evidence, or a genuine interpretive choice. If one candidate treats a product-type promotion as covered and another does not, the difference points to a specific source phrase. If two candidates disagree only when a classification is unknown, the problem may be their evaluation semantics. If two sources have different effective intervals, the disagreement may be temporal rather than substantive. The controller should classify these possibilities before attempting a repair.

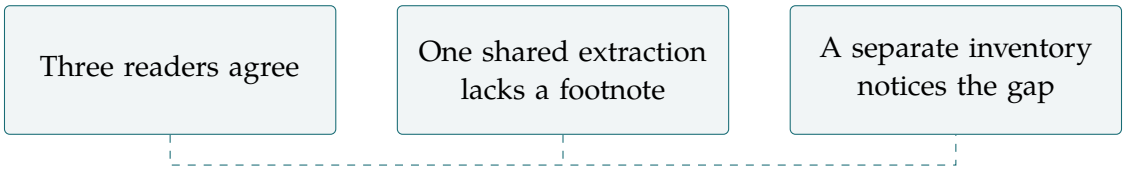


Figure 1.10: Agreement does not expose material absent from every reader's input.

The converse is equally important: disagreement alone does not make every candidate legally respectable. A generated rule that deletes an express exception may be a useful negative control but a poor interpretation. The system should retain it long enough to explain the rejection and test the detectors, while distinguishing it from a reading that remains genuinely unresolved. A shortlist filled with contrived alternatives can increase reviewer workload without improving assurance.

The proposed ensemble consequently has two obligations. It must preserve material dissent until there is a recorded reason to resolve it, and it must identify the evidentiary standing of that dissent. An objection supported by an applicable source is different from an unsupported model suggestion. A solver counterexample proves a difference in formal consequences; it does not by itself give one interpretation greater legal authority. A human reviewer can accept a reading for a declared scope, but that decision must not be mislabelled as a machine proof of English meaning.

Chapter 6 will specify how these distinctions become a finite automatic process. The core idea is that a discrepancy creates work with a defined purpose: retrieve missing authority, correct an encoding, investigate an alternative or clarify a fact. Repetition without a new diagnostic objective is not additional assurance. When the permitted actions have been exhausted, the system must make the remaining issue easier for a reviewer to understand.

1.8 The object being classified is part of the rule

The incentive example reveals a general weakness of compact decision tables: they often define a field without defining the entity to which it belongs. A field called `is_discount` could describe a payment, a fee line, a marketing offer, a product or a client relationship. These objects are not interchangeable. If a fee waiver and voucher are separate incentive components, a property of the waiver cannot automatically be transferred to the voucher. The rule's subject identity is therefore part of its meaning.

The source-to-rule specification must state the unit of assessment before introducing its predicates. In this example it is one proposed component within a documented offer. The specification must also state how components are identified and when a purported separation is legally meaningful. A data designer cannot settle that second question solely by assigning two database identifiers. Operational decomposition needs to reflect the reviewed legal analysis; otherwise a bank could accidentally change the meaning of an exception by changing how it stores an offer.

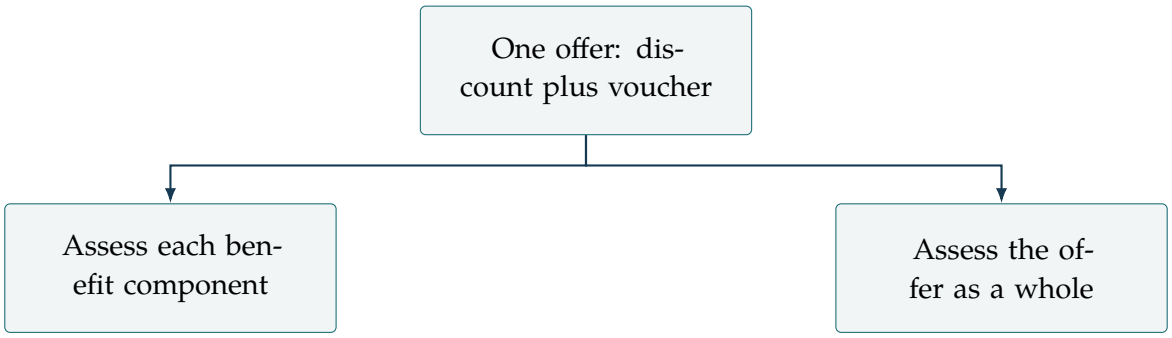


Figure 1.11: The subject being classified changes the exception question.

Suppose an offer combines a fee reduction and a travel voucher. Showing that the fee reduction is a discount does not by itself classify the voucher. The adopted rule must say whether the exception attaches to each component or to the whole offer.

The same problem appears in private-bank suitability workflows. A statement about a client's status does not necessarily apply to every account, every product category or every transaction channel. Consent may be linked to an identified service and period. A relationship manager's assessment may have a subject, scope and evidence date that differ from those of a wealth record. The intermediate representation must retain these bindings, not merely a collection of yes-or-no answers about the client.

A useful specification sentence therefore contains both a predicate and its subject: "component C in offer O is classified as a fee discount under definition D, on the evidence recorded in assessment A." That sentence is longer than "discount = true," but the extra information explains what the Boolean means. The eventual runtime can carry compact identifiers so that ordinary transactions remain efficient. The review record retains the definitions and evidence those identifiers name.

This is also why a language model's ability to extract entities is insufficient. It may correctly identify the words "fee waiver" and "voucher" while failing to determine the

legal relationship between them. Extraction can propose candidates for classification. The system must separately record the reasoning that accepts their relationship and the uncertainty that survives it. That separation lets a later source clarification update a definition without pretending that the underlying marketing document changed.

1.9 Time changes the question being answered

An apparently straightforward rule can yield different legitimate assessments at different times. The regulation may change, the facts may change, or the bank may learn something new about an earlier event. These are three distinct sources of variation. A source publication date identifies when a document was issued. An effective interval identifies when a reviewed rule is intended to apply. An evidence recording time identifies when the bank had the relevant information. Substituting one date for another changes the assessment.

Consider a gift classification entered on Tuesday for an offer made on Monday. A historical review can ask what the bank should conclude now about Monday's offer, using Tuesday's evidence. It can also ask what the system could conclude on Monday from the information then recorded. Both questions are useful. They require different knowledge cutoffs. Replaying the earlier decision with all subsequently available facts would conceal the information gap that existed at the time.

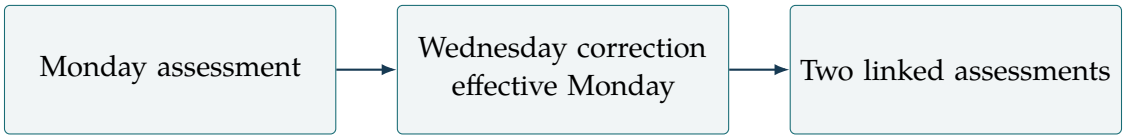


Figure 1.12: Later knowledge changes a reassessment without changing what was known earlier.

The current runtime makes assessment time and knowledge cutoff explicit through `valid_at` and `known_at`. Known evidence has a validity interval and a recording timestamp. An expired classification becomes unknown for an assessment outside its interval. Evidence recorded after the historical cutoff cannot be used in that historical assessment. A superseding record names the record it corrects; simple arrival order does not determine which assertion is authoritative.

These are engineering conventions selected for a precise replay question. They do not determine the legal effect of every late discovery or correction. A corrected factual record may establish that an obligation was performed on time, while the bank's earlier inability to demonstrate that fact remains part of the audit history. A regulatory amendment may apply prospectively, retrospectively or with transition rules that need interpretation. The translator must represent the relevant temporal question before the runtime can answer it.

The ensemble must therefore challenge time assumptions as well as the visible substantive condition. One candidate can be correct under a historical source version and

wrong under the current one. Another can use a current definition to interpret an earlier transaction without justification. A disagreement report that omits the source revision and assessment times can make these distinct questions look like a conflict about the same rule. Immutable context identifiers are necessary to know what is actually being compared.

1.10 From an answer to an accountable decision

A useful product result contains more than a sentence announcing compliance or non-compliance. It identifies the precise condition evaluated, the reviewed scope, the facts and definitions used, the source version, the proposed interpretation, the evidence of correct execution, and any unresolved dependency. These details serve different readers. A compliance reviewer needs to see the interpretation. A data owner needs to see the evidence mapping. An engineer needs reproducible inputs and the exact program version. A release owner needs to know which approvals bind the exact binary being deployed.

The design should not force every reader to inspect every detail on every case. The review interface can begin with the decision and the material reason, then allow inspection of assumptions and counterexamples. The underlying record must still support that inspection. A concise explanation is valuable when it is a faithful view of a fuller record; it is dangerous when it replaces a missing record with persuasive prose.



Figure 1.13: Responsibility accompanies the rule through review and use.

Accountability also requires distinguishing an interpretation from a bank policy. A bank may decide to prohibit a class of offers while a regulatory question remains unresolved. That can be a sensible operational choice under its own authority. The system should record the policy owner, scope and reason for the restriction. It should not attribute the restriction to an SFC sentence unless the interpretation supports that attribution. Otherwise later reviewers cannot tell whether a change requires new legal analysis or a policy decision by the bank.

Likewise, a manual review route must have a defined effect. “Refer to compliance” is incomplete if no owner, service expectation, required evidence or transaction state is specified. An unresolved assessment may hold an automated approval, route the case for review or invoke an already approved conservative policy. The choice belongs to the bank’s operating design. The translator should never invent a transaction permission merely because its retries have reached a limit.

The product’s ambition is thus substantial but specific. It should improve the quality of the questions reaching a human reviewer, preserve alternatives that would otherwise

disappear, and verify execution of the interpretation that is eventually adopted. The next chapter follows two circulars through that route. The complete SPI transaction makes execution concrete; the smaller marketing restriction then exposes why independent source judgment, explicit scope and uncertainty cannot be replaced by a large number of successful program evaluations.

Chapter 2

Two circulars, from business meaning to executable controls

The two cases begin with decisions a bank would recognize. In one, a relationship manager wants to know whether Ms Lee may use a streamlined treatment for a particular bond transaction. In the other, a distributor wants to know whether a benefit in a fund promotion falls within a restriction. The questions look like flags in a system. They become different questions as soon as the bank asks which person, product, date and evidence the flag describes.

The first case shows how a qualification can depend on alternatives, continuing consent and the event being monitored. The second shows how an exception can attach to one component of an offer and not another. Circular 23EC35 supplies the complete transaction profile; 23EC46 tests the boundary where the source leaves room for judgment. Both cases use public editions and synthetic facts, so the reader can see exactly which evidence supports the demonstrations.

2.1 SPI qualification: the conditions a wealth flag conceals

The joint circular of 28 July 2023 supplies an unusually useful test case because its apparent simplicity conceals several different kinds of rule. Annex 1 defines the qualifying criteria, eligible investment transactions, information and consent requirements, and continuing controls. Annex 2 clarifies difficult implementation cases. The examples below are synthetic and illustrate the cited provisions; they do not complete an SPI assessment or authorize a transaction (SFC and HKMA, 2023a,b,c).

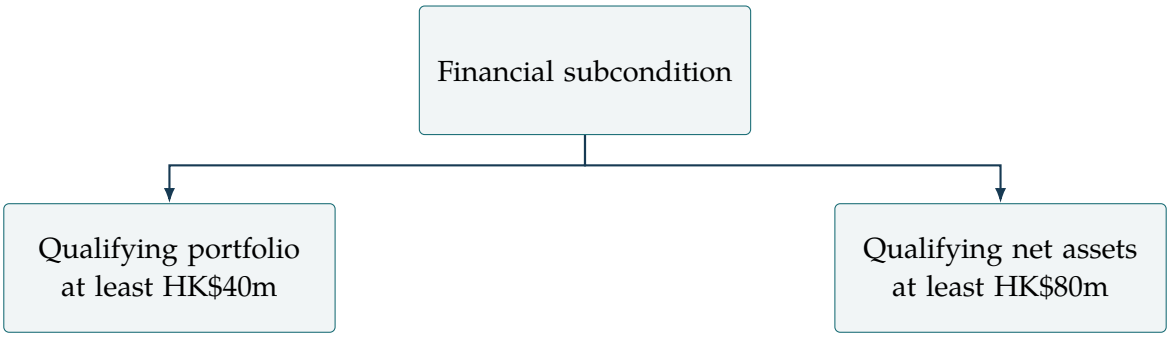


Figure 2.1: The two inclusive wealth routes remain alternatives.

A one-word error changes the financial condition

Annex 1, paragraph 3.1 provides two alternative financial routes: a portfolio of at least HK$40 million, or net assets excluding the primary residence of at least HK$80 million, including the stated foreign-currency equivalents. Consider a client with an eligible portfolio valued at HK$35 million and qualifying net assets of HK$90 million. The second route meets this financial condition. Replacing the alternative with a conjunction rejects the client incorrectly. Replacing either inclusive comparison with a strict comparison rejects a client exactly at that threshold. Both defects are easy to miss when test data use wealth amounts far from the boundary (SFC and HKMA, 2023b, Annex 1, para. 3.1).

The numbers also require definitions. The value of assets held at the bank is not automatically the qualifying portfolio, and total wealth is not automatically net assets excluding the primary residence. Paragraph 3.3 specifies the net-asset calculation. In a simple synthetic case with HK$120 million of assets, including a HK$30 million primary residence, and HK$15 million of liabilities, that calculation gives HK$75 million: $120 - 15 - 30$. Omitting the residence gives HK$105 million and changes the result of the net-asset route. The portfolio route must still be assessed independently. The bank must also establish how the source definitions map to its valuation and ownership records (SFC and HKMA, 2023b, Annex 1, paras. 3.2–3.4).

Ownership and missing evidence can reverse the result

For a joint portfolio with persons other than the client's associate, paragraph 3.2(c) refers to the client's share specified in a written agreement, or an equal share in the absence of that agreement. Suppose a two-holder portfolio is worth HK$60 million and the written allocation gives the client 20 percent. The relevant share for this illustration is HK$12 million, not the whole account and not an automatically assumed half. The rule for a joint account with an associate is separately stated. A generic "joint-account multiplier" loses this distinction (SFC and HKMA, 2023b, Annex 1, para. 3.2).

An unavailable document is also different from an established absence of an agreement. The bank must decide what evidence establishes each fact. If a feed returns a null portfolio value, replacing null with zero creates a substantive conclusion from a data failure. If qualifying net assets are known to be HK$90 million, the financial condition can still be established by that route. If they are HK$70 million, the unknown portfolio value can change the answer; the financial condition remains unresolved. The product must express that distinction directly.

Wealth does not establish suitability for streamlined treatment

Paragraph 1 combines professional-investor status, the financial criteria, knowledge or experience, and investment objectives at the relevant date. Paragraph 4.1 requires reasonable satisfaction concerning sophistication and sets out alternative supporting criteria. Paragraph 5.1 excludes conservative clients from SPI treatment

for this guidance. A wealthy client whose objective is capital preservation therefore demonstrates why a field named `wealth_qualified` must not automatically populate `streamlined_approach_available` (SFC and HKMA, 2023b, Annex 1, paras. 1, 4–5).



Figure 2.2: Wealth is one part of the selected SPI assessment.

A synthetic client with qualifying assets can satisfy the financial condition while having conservative investment objectives. The financial result remains true; the full assessment must still address the exclusion and the other conditions. Giving those two results different names prevents a wealth test from becoming a trade permission.

Some criteria are countable; others require assessment. A degree, a qualification, relevant work experience and prior transactions are distinct routes. A translator should not invent a score that trades off wealth against sophistication or investment objectives. The assessment and its supporting evidence need an owner, date and review record. Paragraph 2 also supplies a separate route for a qualifying investment-holding corporation wholly owned by SPIs. It cannot be represented by treating every client identifier as an individual.

Experience and consent belong to a product category

Suppose the client relies on the transaction-experience route and has completed five bond transactions in the relevant three-year period. That history does not automatically establish the same route for an unrelated product category. Paragraph 7.3 connects category eligibility to the alternative qualification and experience routes; paragraph 7.2 requires the client's choice of categories to be documented and category information to be provided. A global client flag can therefore remain true while a proposed category is outside the selected scope (SFC and HKMA, 2023b, Annex 1, paras. 4.1, 7.1–7.3).

The identity of the person matters too. Annex 2, FAQ 2 explains the assessment where a client acts through a power of attorney. The attorney's sophistication cannot simply replace the client's qualifying attributes. In software terms, the customer, account holder, representative, beneficial owner and person giving an instruction are different roles. Collapsing them into one record can make every subsequent logical step internally consistent and still apply it to the wrong person (SFC and HKMA, 2023c, Annex 2, FAQ 2).

Exposure monitoring is an event rule

Paragraph 8.3 permits two monitoring approaches. One concerns gross exposure on execution of transactions under the streamlined approach. The other uses designated accounts and checks the specified condition after a top-up or deposit of new funds. Annex 2, FAQ 6 explains how these approaches operate, including transferred exposures and restrictions on additional funds when a designated account exceeds the threshold. FAQ 7 addresses leverage (SFC and HKMA, 2023b,c, Annex 1, para. 8.3; Annex 2, FAQs 6–7).

A single rule that rejects every transaction whenever an account's current value exceeds a threshold loses the distinction between these approaches. In the designated-account example, the FAQ allows continued execution above the threshold subject to its conditions restricting new funds. The monitoring mode, movement of funds, gross exposure and prior events all matter. An appreciated position, a leveraged purchase, a cash withdrawal and a transferred non-cash position must not be reduced to the same change in account market value. This is a strong reason to model event histories as well as decision tables.

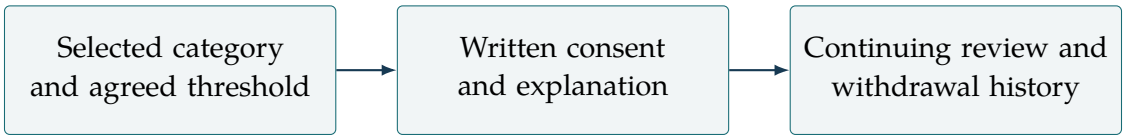


Figure 2.3: Qualification, agreement and continued availability are different questions.

Consent and continuing obligations outlive the initial decision

Paragraph 13 calls for prior written agreement and an explanation of the ability to withdraw from, or change, the arrangement. Paragraph 14 requires annual review and reminders. Assessment and agreement are therefore not permanent facts. A transaction considered before receipt of a withdrawal instruction and one considered afterwards require different state snapshots. A review reminder being queued is different from being sent; being sent is different from the review being completed (SFC and HKMA, 2023b, Annex 1, paras. 12–14).

The streamlined approach also retains specific information and conduct requirements. The solicited or recommended transaction provisions and the unsolicited complex-product provisions have different contexts. A broad "skip suitability and disclosure" switch would conceal these remaining duties (SFC and HKMA, 2023b, Annex 1, paras. 9–11). The prototype should return the relevant decision and the associated tasks, explanations and evidence requests, rather than a single apparently comprehensive compliance flag.

2.2 A complete worked change: circular 23EC35 to a Java decision

The wealth condition now meets a transaction. An adviser needs to know whether this client, category and order can use the selected streamlined treatment. The answer must survive a withdrawal of consent or a later source change. Following one case through these events shows what the bank must preserve beyond the initial qualification.

The running source is the joint SFC–HKMA circular of 28 July 2023, *Streamlined approach for compliance with suitability obligations when dealing with sophisticated professional investors*. Its main text introduces the approach; Annex 1 sets out the guidance; Annex 2 answers implementation questions. This example follows those documents through collection, interpretation, evidence mapping, evaluation, Java generation and an operational change. The facts about the client are invented for the demonstration. The regulatory provisions are identified individually so that those two sources of information remain distinguishable (SFC and HKMA, 2023a,b,c).

2.2.1 The business question and the facts the bank would need

At 10:00 Hong Kong time on 21 September 2026, an adviser proposes a solicited transaction in an ordinary corporate-bond category for Ms Lee, a synthetic individual client. The question is whether the bank may apply the selected streamlining treatment to this transaction, subject to its other controls. The example uses paragraph 8.3(a), which monitors gross exposure at execution. It does not use the designated-account alternative in paragraph 8.3(b).

Assume the bank has already assessed Ms Lee as an individual professional investor under the applicable imported definitions. Her qualifying portfolio is HK$40 million: HK$35 million on her own account plus a documented 25 percent share of a HK$20 million joint portfolio with a non-associate. Her total assets are HK$120 million, liabilities HK$15 million and primary residence HK$30 million, giving HK$75 million of net assets excluding the residence. The portfolio route meets paragraph 3.1 exactly; the net-asset route does not. The ownership agreement, valuations and assessment would be separate evidence records in the bank. The example's normalized inputs already incorporate them.

Ms Lee has no relevant degree, professional qualification or qualifying year of work experience in this example. She instead has five transactions in the chosen category, dated 8 January and 12 June 2024, 14 February and 7 November 2025, and 20 August 2026. These dates lie comfortably inside the relevant three-year window; the example does not settle a disputed endpoint convention. The bank has separately recorded its reasonable satisfaction about her sophistication and assessed her objectives as non-conservative. Five transactions are evidence for a specified route, not an automatic substitute for that assessment (SFC and HKMA, 2023b, Annex 1, paras. 4.1, 5.1, 7.3).

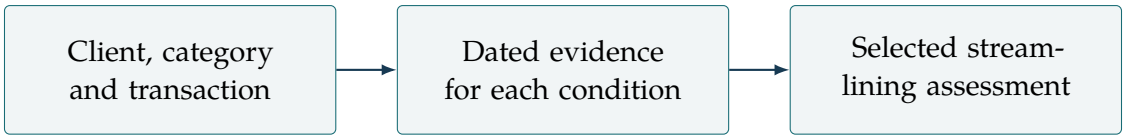


Figure 2.4: The worked change starts from a particular transaction and its evidence.

The client has selected this category, received the category information and required explanations, and given the prior written acknowledgment for the arrangement. The agreed streamlining threshold is HK$10 million. Existing gross exposure is HK$8 million; the proposed transaction adds HK$2 million, including leverage, bringing the monitored exposure to HK$10 million. The control compares that gross amount with the threshold, not the cash margin posted for the purchase. The threshold rationale and assessment are retained; the example treats the annual review and reminders as current (SFC and HKMA, 2023b,c, Annex 1, paras. 6–8, 12–14; Annex 2, FAQ 7).

Finally, current offering documents have been delivered. There is no unanswered request for product explanation or material query. The transaction has been assessed for the outsize or material-transaction warning requirement; this synthetic case does not require that warning. These are retained controls. They do not disappear because the wealth condition passed (SFC and HKMA, 2023b, Annex 1, paras. 8.4, 10.3).

The facts are intentionally close to two boundaries. A one-cent reduction in qualifying portfolio, with the other facts fixed, defeats the financial condition. A one-cent increase in projected gross exposure defeats the selected execution-monitoring condition. A test suite containing only a very wealthy client buying a very small position would miss both defects.

2.2.2 Collect the whole instruction before extracting rules

A legal decision must remain explainable after the public web page has changed. For the SPI example, the bank keeps the edition of the circular and both annexes that the reviewer actually read. It also keeps the extracted text used during review and the paragraph, page and exact passage that support each rule. If a later retrieval differs, the reviewer can see whether the source changed, the extraction changed, or the interpretation changed.

That discipline matters in an ordinary incident review. Suppose a client asks why a transaction was accepted on Monday and a revised document appears on Friday. A link to the current web page cannot answer the Monday question. The preserved edition and passage can. They also show whether the relevant financial condition came from the circular itself or from an imported definition that the bank still has to adopt and review.

The implementation record gives these preserved files stable content identifiers and checks that the recorded passage still belongs to the retained edition. Those identifiers are useful to the people building and auditing the control. They are evidence of which source was read, not evidence that the source has legal effect or that the resulting interpretation is correct.

Collection also reveals dependencies. Annex 1 imports definitions from the Professional Investor Rules and the Code of Conduct; Annex 2 and the footnotes point to further guidance. The demonstration supplies reviewed assessments for those imported matters. A bank release would have to name the adopted editions, legal entity and activity. The synthetic validity dates used in the example are test settings, not findings about the circular's commencement or continuing applicability.

Before writing a formula, the reviewer lists the requirements found in the circular, annexes and answers to frequently asked questions. Some requirements can become decision rules. Others require a calculation, a human assessment or a continuing task. A passage outside the selected business scope still needs a recorded reason for exclusion. It cannot disappear merely because it is awkward to implement. The table groups the questions that this reading must settle.

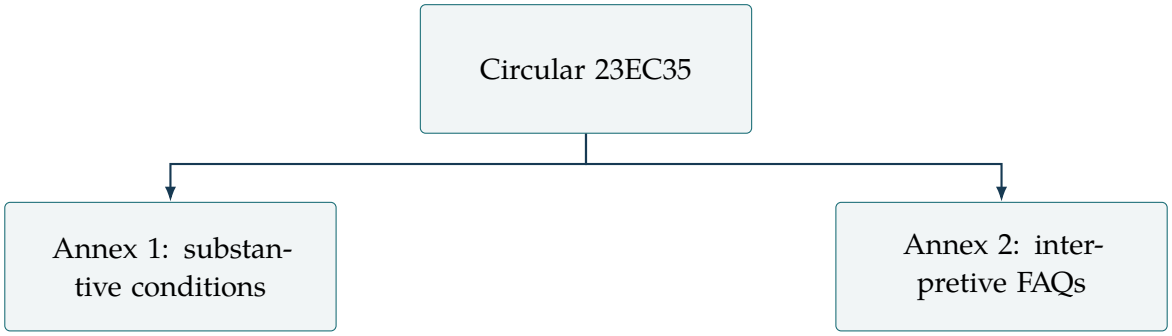


Figure 2.5: The main circular's attachments are part of the reading task.

Table 2.1: What the source review must answer before a rule is released. The full provision-by-provision inventory is retained in Appendix C of the technical companion.

Question	Why it needs a separate answer
Who qualifies, and for which product category?	Wealth, experience, objectives and category choice are separate conditions.
What exposure and transaction are being assessed?	The answer depends on gross exposure, leverage and the chosen monitoring method.
What has the client agreed to, and what remains due?	An initial agreement does not settle current consent, explanations or annual review.
Which activities need a different assessment?	Account monitoring and unsolicited complex products require their own rules and evidence.

This inventory is the first useful review deliverable. It exposes omissions before a test generator starts exploring the rules that happened to be encoded. The main text, annexes and incorporated definitions are the review boundary; the numeric threshold is only one part of it.

2.2.3 Record the interpretation and who supplies each fact

The draft interpretation separates what the circular says from what the bank would choose to do. For paragraph 3.1 it records an inclusive alternative and links both comparisons to the paragraph. For paragraph 13 it records the arrangement-specific written acknowledgment and the withdrawal right. For paragraph 14 it records the annual duties. It then proposes, separately, that the bank suspend use of the streamlined route when its annual review is not current. A compliance reviewer can accept, amend or reject that proposal without losing the duty stated in the source.

Two unresolved issues are deliberately visible. First, the bank must settle ownership, currency and valuation mappings for wealth and exposure. Second, the calendar conventions for transaction windows and annual deadlines must be adopted. A complete source-based interpretation may still rely on human assessment: "reasonably satisfied" and "material transaction" do not acquire objective numerical definitions simply because the system needs an input.

The implementation record gives each reviewed version a stable identifier. A preparer can submit a version with its supporting examples, and an authorized reviewer can accept that exact version while seeing its perimeter and unresolved questions. Editing a rule creates a new version, so the earlier acceptance no longer covers it. The example in this book is labelled `DEMONSTRATION_ONLY`: its synthetic assessments and proposed bank choices have no invented reviewer approval. That makes the demonstration useful for learning and testing without presenting it as a production control.

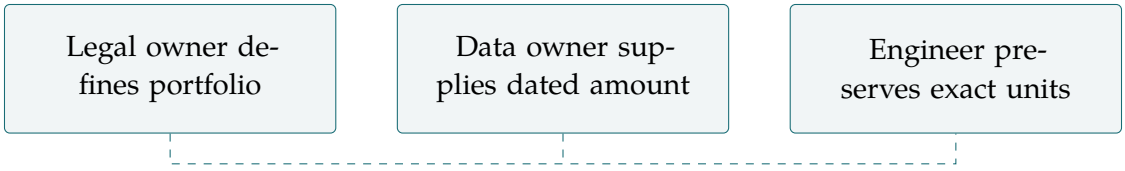


Figure 2.6: Different owners answer different questions about the same input.

Table 2.2: The people and records that supply the facts for the worked decision.

Who supplies the evidence?	What the records must establish
Compliance and relationship management	This client's qualification and chosen product category, supported by a dated assessment under the adopted definitions.
Valuation and operations	The qualifying portfolio and net assets, showing ownership, liabilities, residence, currency and valuation date.
Client and adviser	What the client was told and agreed to, with delivery records and the history of consent or withdrawal.
Order management	The agreed threshold and gross exposure at execution, including leverage and any required warning.

One answer may depend on several records. For example, the acknowledgment must link the prior written agreement, qualification explanation, chosen categories and threshold, consequences and withdrawal/amendment explanations. A rejected or incomplete checklist makes the composite false or unknown under the bank's adopted evidence policy. The reviewer can open the constituent items. The generated decision library does not authenticate documents or decide whether an explanation was adequate.

2.2.4 Construct and evaluate the executable specification

The worked bundle asks a small set of separate questions. Does the client's portfolio or net assets meet the financial route? Does the client satisfy the chosen category and assessment? Is there a written arrangement, current consent, the required explanation and a current annual review? Is the execution threshold respected, and have the required documents and warnings been delivered? The final answer combines these questions only for the selected product profile.

Keeping the questions separate lets a reviewer see why an answer was reached. The calculation uses exact money and count values. Missing evidence remains missing, while contradictory records are reported as a conflict rather than resolved by choosing whichever value is convenient. The executable specification checks that each question is well formed and refuses an operation whose meaning has not been defined.

For Ms Lee, the reasoning is straightforward. Her HK$40 million portfolio meets the financial condition, even though her HK$75 million in net assets falls below the alternative threshold. Five transactions in the relevant category establish the chosen expe-

rience route. The remaining assessments and arrangement conditions have been supplied and satisfied. Her projected HK$10 million exposure equals the agreed limit, so the order stays within it. She has no unanswered request for an explanation; the supplied assessment finds that this transaction does not require the material-transaction warning; and the offering documents have been delivered. The selected streamlining conditions are therefore met. A reviewer can follow each answer back to its source passage and supporting facts. The record also shows which rule was used, when the transaction was assessed, and what information was available then. The technical companion gives the precise record format needed to reproduce that decision.

This successful conclusion remains narrow. It identifies the selected streamlining conditions under the supplied assessments and proposed review restriction. The host must still apply other relevant product, credit, market, financial-crime, conduct and order-execution controls. Its own release process decides which approved control version is usable. A demonstration result is never automatically promoted by setting an input named “approved”.

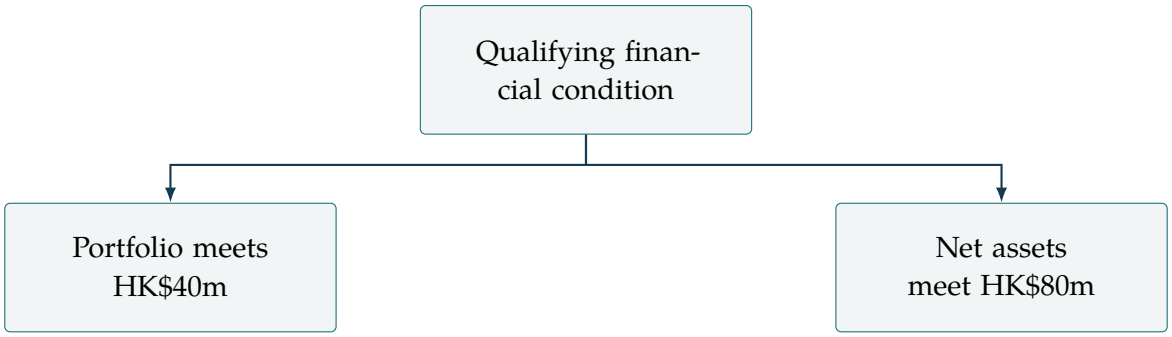


Figure 2.7: Either reviewed wealth route can establish this subcondition.

At exactly HK$40 million of qualifying portfolio and zero qualifying net assets, the inclusive portfolio route succeeds. Reducing only that portfolio by one cent makes both routes false. Replacing the alternative with a conjunction would incorrectly reject the first case.

Table 2.3: How a small change in the facts changes the response.

Change in the case	Result	Reason
The portfolio is just below the threshold, while net assets also fail.	Standard process	Neither financial route is established.
The portfolio record is missing, but net assets clearly satisfy the alternative route.	Conditions met	Missing evidence need not block a conclusion that another route already establishes.
The client's objective is conservative, despite substantial wealth.	Standard process	Wealth does not replace the separate suitability condition.
Consent has been withdrawn or a required explanation is unanswered.	Standard process	The continuing duty is part of the decision at the time of execution.
Two records give different portfolio values.	Review required	A conflict needs an owner and evidence; it is not resolved by choosing the convenient value.

The associated actions are equally concrete. A missing portfolio requests the valuation and ownership evidence. An unanswered explanation routes a task to the adviser and retains the interaction record. An exceeded execution threshold routes the order to the standard process or a separately reviewed change in arrangement. An evidence conflict routes to its data owner. These are proposals for the host workflow, not a generated instruction to place or cancel a trade.

2.2.5 Generate, build and call the Java release candidate

Once the reviewers have agreed on the meaning, the same rule is written in a small reference form and in Java. The important test is not that the Java build finishes. It is that the two forms return the same decision, explanation and request for evidence for the same client history. A separate host program then calls the Java library at the point where a bank system would use it.

The comparison includes cases that sit on the edges of the rule. A strict comparison would reject a portfolio exactly at HK$40 million. Replacing the two alternative wealth routes with a conjunction would reject a client who qualifies through only one route. Ignoring consent withdrawal would allow the arrangement to continue after the client had ended it. Each deliberately altered program must fail a case written independently of the altered code. A matching file identity alone cannot show that a semantic mistake

was detected.

The exact class names, source paths and build identifiers belong in the implementation record. For the reader, the result is simpler: a reviewer can see the case that should change, the decision that changes, and the reason the change matters.

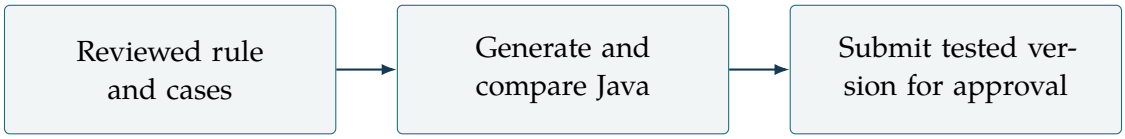


Figure 2.8: Generating a candidate precedes approval of its use.

2.2.6 Withdraw consent, replay the decision, then change the rule

At 10:05 Hong Kong time, suppose Ms Lee withdraws her consent. The event is recorded at 10:06. A historical replay asks two different questions: what was true at the time of the 10:00 decision, and what would the bank decide at 10:10 using everything it knew by then? The first answer remains reproducible. The second sees the withdrawal and routes the transaction to the standard process.

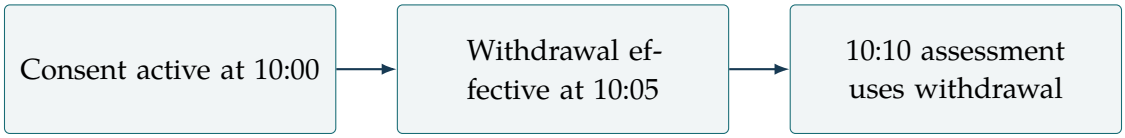


Figure 2.9: Withdrawal changes later behavior while preserving earlier decisions.

That distinction matters when information arrives late. A history that is known complete only through 10:04 cannot support a claim about a 10:10 decision merely because the last visible event is a grant. Duplicate events with the same content may be harmless; two different payloads under one event identity are a conflict. Events recorded at the same time need an agreed order. The example makes these cases visible so that the bank adapter must supply the missing history or stop for review.

Now suppose a later approved rule changes a threshold or a retained-information duty. The bank needs to see which definitions, cases and client outcomes change. It then approves the new rule and its implementation as one release while keeping the earlier version for historical replay. Replacing a defective Java package is a software action. It does not change which legal rule applied on an earlier date.

The dry run reaches the point at which a bank could call an executable control, then stops where human authority is still required: adopting imported definitions, mapping live data and calendars, and approving production use.

2.3 CoCos and preferred shares: law beyond the prospectus

Suppose Ms Lee now asks to buy a bank capital instrument. Her wealth and her consent to the streamlined approach do not answer the first product question: what, exactly, could happen to her investment if the issuing bank becomes distressed? The answer may depend on the offering conditions, banking legislation, a regulator's order and subsequent litigation. A second question concerns the intermediary: which products may it sell to this client, through this service, and subject to which obligations? Keeping these questions separate is necessary even when both eventually depend on the same loss-absorption provision.

The preserved prospectuses make this a concrete investigation. The UBS Singapore dollar notes issued in 2024 contain conversion provisions; the US preferred-share documents describe different legal interests and distribution rights. A country label or the word "preferred" cannot substitute for identifying the issuer, instrument, applicable conditions and distribution arrangement. The Swiss and Hong Kong sources examined below form a bounded survey, inspected on 29 September 2026. A dated source establishes what that source says; it does not establish that no later amendment, order or judgment exists.

2.3.1 What can disappear when a bank absorbs losses?

A contingent convertible instrument, commonly called a *CoCo*, changes the holder's rights when specified conditions occur. Bank capital instruments may instead provide for a reduction of the debt claim. *Additional Tier 1* (AT1) is a category of regulatory capital available to absorb losses. It is not a promise of safety to its purchaser. *Common Equity Tier 1* (CET1) is the core equity category; a contractual trigger may refer to its ratio to risk-weighted assets. The relevant ratio, entity, threshold, observation date and notification provisions must all be identified. A *point of non-viability* provision concerns intervention to prevent failure and need not coincide with breach of that numerical threshold (Swiss Federal Council, 2025, arts. 27, 27a and 29).

A permanent write-down can extinguish principal without delivering replacement shares. Conversion also removes the original debt claim, but replaces it with equity under specified settlement arrangements. The market value of that equity may be very low or eventually zero; this does not make conversion the same legal mechanism as cancellation without compensation. Temporary reduction and a conditional future write-up form a third possibility. Separately, cancellation of a non-cumulative distribution removes that period's payment entitlement without, by itself, cancelling principal. The implementation must therefore record debt principal, replacement shares, distribution entitlements and any restoration right separately. Its existing exact-arithmetic examples do this conditionally; they do not establish that every real instrument fits those examples.

The retained UBS conditions illustrate why the qualification matters. Conditions 7 and

8 provide for conversion following the specified trigger or viability event. Condition 7(b) also contains notification, timing and higher-trigger priority provisions, including a defined circumstance in which FINMA can agree that conversion need not occur. Condition 7(e) addresses subsequent legal changes that can displace the contractual viability clause after the required determination and notice. Thus, even this conversion example needs more than a comparison of one capital ratio with one number. The retained document contains published terms and conditions; it is not represented here as a complete final prospectus with every incorporated document and later notice (UBS Group AG, 2024, conditions 7–8, PDF pp. 25–29).

Bail-in introduces another route: an authority restructures liabilities under statutory powers. Whether that route reaches a particular claim depends on the applicable law and proceedings. A warning about possible bail-in is therefore evidence of a risk requiring investigation. It is neither evidence that an order has been made nor a sufficient classification rule for every investor-protection regime. Hong Kong's banking regulator expressly distinguishes debt that could be bailed in from the narrower product class covered by its loss-absorption guidance (Hong Kong Monetary Authority, 2022c, questions 1–4).

2.3.2 Swiss capital requirements and powers of intervention

The Swiss Financial Market Supervisory Authority, *FINMA*, supervises the relevant banks. Three different sources of loss absorption must be examined: the capital instrument's agreed conditions, ordinary banking resolution law and any exceptional measure applicable to the event. They do not supply interchangeable legal grounds.

The retained German consolidation of the Capital Adequacy Ordinance, effective 1 January 2025, is the principal legislative text used for the following description; its official-site English translation is retained as a reading aid. Article 27 distinguishes capital eligibility from the trigger written into a debt instrument. Article 29 requires provision for loss absorption at non-viability, through complete reduction of claims or conversion, and specifies FINMA's role. An instrument may consequently qualify as capital for the bank because it exposes its holder to loss. This prudential eligibility is a different question from whether an intermediary may distribute it to Ms Lee (Swiss Federal Council, 2025, arts. 27–29).

Article 27a, introduced with effect from 1 January 2025, is especially relevant to executable interpretation. Once the specified trigger for eligible AT1 liabilities occurs, FINMA orders the reduction or conversion, its timing and the amount affected. The provision also requires preservation of the contractual issue or loan conditions and their resulting priority. A program that extracts only "FINMA orders the reduction" loses those conditions. Conversely, a program evaluating a March 2023 event cannot silently use this later provision. The retained 2025 consolidation also amends article 29(2); the earlier wording quoted in the Credit Suisse judgment must remain available for the historical question (Swiss Federal Council, 2025, arts. 27a and 29, amendment footnotes).

Ordinary resolution follows a separate statutory route. Article 25 of the Banking Act states the conditions for intervention, including serious liquidity problems, suspected over-indebtedness and failure to meet capital requirements after the specified deadline. Articles 26 and 28 distinguish protective measures from restructuring proceedings. The amendment published as AS 2022 732, effective 1 January 2023, sets out capital measures in article 30b: its provisions address conversion and reduction of claims, protected claims, prior absorption of capital, creditor ordering and specified exceptions. Article 30c governs the restructuring plan, including creditor protections, and article 31 governs approval, including its specified compensation provision for systemically important banks (Swiss Confederation, 2022, arts. 30b–31 and commencement provision). These provisions are also visible in the retained 2024 consolidation. A general instruction power is not interchangeable with this structured resolution route (Swiss Confederation, 2024, arts. 25–31).

Subordinate procedural rules also change. FINMA's Insolvency Ordinance of 20 August 2025 entered into force on 1 October 2025 and repealed the 2012 Banking Insolvency Ordinance. Its transitional provision applies the new ordinance to proceedings already pending at commencement. The investigation must therefore inspect transitional rules rather than assume that the law applicable throughout a proceeding is fixed by its opening date (FINMA, 2025b, arts. 1–2 and 48–50).

There was also an exceptional measure in March 2023. The amendment published as AS 2023 136 inserted article 5a into the emergency liquidity ordinance. It addressed FINMA's ability to order the borrower and its financial group to write down additional Tier 1 capital at the specified credit-approval point. Its text did not declare all CoCos issued by every Swiss bank worthless. The amendment entered into force on 19 March 2023 at 20:00 (Swiss Federal Council, 2023a, art. 5a and commencement provision). Another amendment, adopted on 6 September and published as AS 2023 495, repealed articles 5–7 with effect from *15 September 2023*. Confusing adoption with commencement would give the program the wrong boundary. Repeal of a power for new action also does not, by itself, decide the consequences of an earlier order (Swiss Federal Council, 2023b, repeals and commencement provision).

These provisions establish why a legal investigation is necessary. They do not justify a universal rule that "Swiss bank" and "AT1" imply an immediate write-down to zero. The program needs the legal entity and claim covered by a power, its effective version, its factual conditions, the required procedure and any operative order. It must also distinguish the existence of an asserted legal basis from a final judicial determination of its validity.

2.3.3 Credit Suisse: the write-down and its judicial review

The Credit Suisse case demonstrates the cost of collapsing those distinctions. FINMA's statement of 23 March 2023 described its instruction to write down Credit Suisse's AT1 instruments. It relied on the contractual viability event and the emergency ordinance, and said the Tier 2 instruments were unaffected. That is a preserved account of the regulator's position and action. It is not a judgment resolving all objections to their

legal basis (FINMA, 2023).

In its partial judgment of 1 October 2025, B-2334/2023, the Swiss Federal Administrative Court annulled the challenged order. Its reasoning examined the contractual provisions and the alleged public-law grounds separately. In particular, the court distinguished assistance addressing liquidity from the capital-adequacy conditions in the contracts before it. It did not treat the existence of extraordinary public support as sufficient to erase the rest of the contractual wording. Its reasoning also distinguished a guarantee benefiting the acquiring entity from assistance to the entity specified in the contract (Swiss Federal Administrative Court, 2025a, reasons 5.3–5.5, especially 5.4.4–5.4.8).

On the public-law grounds, the court found that the protective-measure provision in article 26 of the Banking Act and the general remedial provision in article 31 of the Financial Market Supervision Act did not provide the required basis for the disputed extinguishment of claims. It also rejected reliance on the emergency provision, addressing legal certainty, the constitutional conditions for emergency action, delegation and property protection. These are the findings of that court in this partial judgment. They should be preserved with their procedural status, rather than rewritten as either an uncontested description of FINMA's powers or a general abolition of bank resolution (Swiss Federal Administrative Court, 2025a, reasons 6.8–6.10, 7.6–7.11, 8 and 10).

The subsequent chronology is equally material. FINMA announced an appeal on 15 October 2025. The Administrative Court's announcement of 22 October said that other pending AT1 cases were suspended and that reversal of the write-down had not yet been decided (FINMA, 2025a; Swiss Federal Administrative Court, 2025b). A later Federal Supreme Court judgment, 2C_565/2025 of 5 March 2026, records the separate merits appeals 2C_661/2025 and 2C_662/2025 and the grant of suspensive effect on 8 December 2025. But 2C_565/2025 itself concerns a procedural complaint about access to submissions and delay; its inadmissibility decision does *not* dispose of those merits appeals (Swiss Federal Supreme Court, 2026, facts D, reasons 1.1–1.4 and operative order).

The dossier therefore supports a more precise historical account than either "the regulator may always erase the bonds" or "the investors have been repaid". It records the write-down, the partial annulment, the separate appeals and the reported stay. The latest judicial text inspected for that procedural history is dated 5 March 2026. A fresh operational determination requires a check of the merits dockets and any later relief; a search that finds no later decision is not proof that none exists. Payment actually received is a further fact, distinct from annulment, an entitlement to relief or a stay.

For testing, this case supplies source documents and changing propositions, not a human-labelled answer with which the program must agree. A court's judgment is part of the law and procedural history being represented. Whether the software has represented its content, scope and consequences correctly remains a separate obligation. The project does not use human ratings, annotator consensus or agreement with a model's preferred legal interpretation as a quality criterion.

2.3.4 Which Hong Kong requirement follows from the risk?

The SFC's circular of 7 December 2018 expressly discusses bonds whose non-viability loss absorption can follow a capital-ratio event or government or regulatory action. It connects their complexity to product due diligence, suitability, disclosure and the client's ability to understand and bear the risks. This supports investigating statutory and regulatory events alongside contractual triggers. It does not make "can lose everything" a universal SFC prohibition on sale (Securities and Futures Commission, 2018, discussion of NVLA bonds and selling practices).

For a bank acting as a *registered institution*, another source is decisive: the HKMA's circular of 21 October 2022 and its two annexes. The circular consolidates and supersedes the earlier 2018 and 2019 guidance (Hong Kong Monetary Authority, 2022a). Annex 1(A) restricts sale and distribution of in-scope loss-absorption products to professional investors in both primary and secondary markets. Thus the channel and intermediary type matter: an SFC circular addressed broadly to intermediaries and an HKMA requirement for registered institutions must not be treated as identical statements with identical scope (Hong Kong Monetary Authority, 2022b, applicability and section A).

The scope must be established before applying that restriction. Annex 1 covers the specified contingent write-down or conversion features, and certain products investing mainly in, or closely linked to, such instruments (Hong Kong Monetary Authority, 2022b, applicability). Annex 2, question 1, includes qualifying AT1, Tier 2 and specified loss-absorbing-capacity debt under the relevant Hong Kong or equivalent foreign regimes. It expressly excludes instruments legally constituted as equity, including preferred shares, and deposits. Questions 2–4 explain why possible bail-in alone is insufficient. Consequently, an ordinary debt warning does not automatically place the instrument in scope, and exclusion of an equity-form preferred share from this particular circular does not establish that the share is non-complex, suitable, or freely distributable. A depositary share also requires examination of its own legal interest and underlying security; its marketing name is insufficient (Hong Kong Monetary Authority, 2022c, questions 1–4).

Exceptions have their own boundaries. Annex 1(E) removes the specified enhanced suitability and disclosure measures in sections C and D for loss-absorption funds while preserving the applicable SFC requirements; it does not remove section A's professional-investor restriction. Section F grants specified institutional and qualifying corporate professional investors relief from sections B, C and D. These are limited forms of relief, not a general permission for retail distribution (Hong Kong Monetary Authority, 2022b, sections E–F).

The execution arrangement can also change which intermediary must apply the circular. Annex 2, question 9, describes a specific arrangement involving an appropriately regulated adviser or asset manager, an execution broker with the restricted service and contact role, a written allocation of responsibility and written notification to the customer. The clarification requires all of those conditions and does not cover a customer asking that broker directly to purchase the product. Question 10 separately addresses discretionary portfolio management on a holistic basis. The procedure must therefore

preserve service roles and conditions rather than attach an unconditional "execution-only" exemption to the account (Hong Kong Monetary Authority, 2022c, questions 9–10).

SPI streamlining is a further decision. The SFC's SPI FAQ identifies loss-absorption debt and related products as requiring a specific product category, with regard to their characteristics and existing product-specific requirements. Meeting the SPI wealth threshold alone establishes neither category knowledge nor satisfaction of a separate selling restriction. The implementation should report product classification, applicable audience restrictions, remaining selling obligations and availability of streamlining as separate conclusions. A client's own mandate may add a restriction, but that restriction must come from the mandate rather than a presumed preference attached to the client's wealth (Securities and Futures Commission, 2026a, questions 3–4).

2.3.5 A source procedure that can detect the missing law

An instrument investigation begins with identity: the issuing legal entity, any relevant guarantor or resolution entity, identifier, issue and amendment dates, legal form, governing law and intended distribution arrangement. The document set should include final conditions, the correctly associated base prospectus, incorporated material, supplements and relevant notices. A later base prospectus must not be attached to an earlier issue merely because it is from the same bank. A missing final document leaves a recorded gap.

The next step follows the prospectus's legal references and independently examines the issuer-jurisdiction legislation and supervisory sources. The investigation records the provision, its effective interval, the entities and claims it covers, its conditions and the legal consequence. Ordinary resolution, contractual non-viability and emergency action remain separate entries. It then checks relevant orders, proceedings, stays and subsequent judgments. For the Swiss example, this means identifying the relevant ordinance edition and checking the Credit Suisse litigation without transferring its contractual findings to a different UBS issue.

A consolidation date is also insufficient evidence that every amendment has been incorporated. The archived Banking Act copy labelled 1 January 2023 lacks the article 30b and 30c headings present in the promulgated amendment effective on that date and in the retained 2024 consolidation. The discrepancy is retained as a source-version finding; the account above uses the amendment and the corroborating consolidation. A mechanical article-presence check detects this omission, although presence alone cannot establish faithful incorporation of the provision's meaning.

Distribution law is investigated separately for the place, intermediary, service and customer. For the present case, that search includes the SFC complex-product and SPI material and the HKMA circular with both annexes, including their exceptions and supersession history. Product-specific offering restrictions and client mandates remain additional dependencies. This is a bounded source procedure: cross-border recognition, other jurisdictions, incorporated definitions or a newly discovered restriction can

enlarge the required set. The program must then reopen the affected conclusion rather than assert that the original list was exhaustive.

Two dates are retained for every changing proposition: when it applies and when it became available in the decision's evidence history. Downloading a 2025 judgment today must not rewrite what the system knew in March 2023. Conversely, a present assessment of the 2023 event may need that judgment. Original bytes, retrieval locations and hashes preserve the evidence used; page text is a derivative. A changed source invalidates dependent results until the change is assessed. Unchanged bytes prove neither current law nor complete retrieval, so an unresolved update check remains visible.

At the end of this procedure, every required proposition is established under stated premises, refuted, unresolved, or conflicting. For example, the fact that FINMA has an applicable power cannot supply the missing fact that it has issued the relevant order. An unresolved contractual interpretation must not be converted into certainty by a compiler, by several agreeing models, or by a human verifier. The software can instead state exactly which conditional conclusion follows and which missing premise prevents an unconditional one.

2.3.6 Tests that discriminate between the possible mistakes

The implemented checks exercise these distinctions with formally stated premises. They do not use a collection of people's preferred eligibility answers. For a necessary-condition rule, the independent specification is the set of complete fact assignments satisfying every condition. Incomplete evidence describes several possible assignments. If all satisfy the rule, the conditional result is true; if none do, it is false; otherwise it is unresolved. Conflicting evidence is reported separately, so an empty set of consistent assignments cannot produce a vacuous permission.

This gives a precise test of the professional-investor restriction. Under the bounded four-premise specification, violation occurs when the seller is a registered institution, the product is in scope, the investor is not a professional investor and the specific exception is known not to apply. Enumerating partial assignments tests whether the executable predicate agrees with that independently stated relation. An unknown exception is neither assumed available nor silently treated as a known negative fact. Satisfaction of this single restriction still does not establish permission to sell under all applicable requirements.

The regression cases are designed around errors that would otherwise survive a successful compilation:

Change in the evidence or case	Required distinction
Same Swiss issuer country, different instrument conditions	Country alone cannot establish a write-down mechanism or the power's instrument scope.

Change in the evidence or case	Required distinction
Capital-ratio condition absent, viability route unresolved	Absence of the numerical trigger cannot establish absence of every loss route.
Generic statutory bail-in warning added	The warning cannot establish the HKMA's narrower direct-debt classification by itself.
Preferred share legally constituted as equity	Exclusion from the particular debt classification does not become permission to sell.
One required authority premise removed	The asserted power becomes unresolved unless another known false premise already defeats it.
Emergency provision tested at its start or repeal boundary	Apply the exact recorded effective interval; adoption and commencement are distinct.
Judgment added after the original decision	Preserve historical knowledge; permit a separate reassessment using later information.
Procedural appeal declared inadmissible	Do not mark the separate merits appeals decided.
Annulment recorded, repayment evidence absent	Preserve the annulment and its finality or stay status without inventing a payment.
SPI eligibility changed to true	A failed independent selling requirement continues to prevent a positive sale conclusion.
Applicable law bytes changed or disappeared	Dependent evidence is stale; the previous conclusion remains historical.

The current executable addition checks the authority premises, the bounded direct-debt classification, the professional-investor restriction, independent sale conditions, time intervals, proceeding types and source changes. It does not yet execute the full UBS notice and conversion schedule, all fund and depositary-share classifications, cross-border resolution recognition, or an automatic reading of court reasons. Those remain explicit implementation obligations. The source dossier also records the currentness check still needed before a live decision; no finite regression suite establishes the absence of an unknown future legal change.

2.3.7 What this adds to independent assurance

The earlier prospectus campaign preserved documents and checked conditional mechanics, source quotations and shared-language execution. That work is useful, but a correctly executed ratio test cannot compensate for a missing statutory power, a wrong legal entity or an omitted selling restriction. This survey identifies those dependencies before the translation into executable rules. The added tests establish properties of

their explicit representation, while the preserved sources expose the remaining interpretation questions.

Generalisation requires the same discipline when the next unfamiliar instrument arrives. The program should preserve the new sources before adapting its interpretation, attempt the fixed checks, and report which premises it cannot establish. If a legal change requires repair, the old result remains associated with its original sources and method, and the repaired method is tested on a subsequent untouched case. The defensible promise is conditional correctness with detectable limits and reproducible revision. Universal legal correctness on unknown future texts has not been proved.

2.4 The bank and the transaction also determine eligibility

An ordinary bond can have straightforward payment terms and still be unavailable for a particular transaction. The client may be prohibited from acquiring it; the bank may be prohibited from dealing in it; or the bank may lack information needed to establish which restrictions apply. The CoCo example therefore exposes a general question. After explaining what an instrument promises, the system must establish what the participating institutions and clients may do with it. Changing the instrument from an AT1 bond to a preferred share, a fund or an ordinary equity does not remove that question.

2.4.1 Identify the institution before selecting its rules

Consider a private-banking service provided through a Hong Kong office. JPMorgan's public Asia disclosure identifies JPMorgan Chase Bank, N.A., Hong Kong or Singapore branch as the provider of dealing, advisory, discretionary management, banking and custody services, as notified to the client. It also describes the Hong Kong branch's local regulatory supervision. The same page identifies other legal entities in other jurisdictions. Consequently, the J.P. Morgan brand alone cannot identify the legal person undertaking a trade; the account and service documents must establish that person and its capacity (JPMorgan Chase, 2026, Asia service-provider disclosure).

The US Treasury's Office of Foreign Assets Control (OFAC) includes foreign branches of US entities among US persons. A separately incorporated foreign subsidiary requires a program-specific inquiry: some programs extend obligations to subsidiaries owned or controlled by US persons. The system must preserve that relationship and apply the particular program's definition (Office of Foreign Assets Control, 2024, FAQ 11). For the Chinese securities regime, Executive Order 13959, as amended by Executive Order 14032, includes foreign branches within its US-entity definition, alongside US citizens, lawful permanent residents and persons in the United States. The investor's and the bank's statuses must therefore be recorded separately (President of the United States, 2021, replacement section 3(d)).

The account's booking entity is only the beginning. Advice, execution, custody and

payment processing may involve different entities. Each participant needs an identified role, location and applicable obligations. A decision about an advisory service cannot automatically authorize principal dealing by the bank, and permission for a purchase cannot automatically authorize a subsequent transfer or conversion. These are distinctions the proposed data model must retain even when a particular legal rule ultimately treats two cases alike.

2.4.2 A US intermediary is different from a US investor

The Chinese military-industrial securities restrictions supply a useful counterexample to an overly broad institutional rule. Suppose, conditionally, that the issuer is designated, the security is covered, the restriction has taken effect, and no relevant authorization or divestment exception applies. A purchase by a US person is then prohibited under the amended executive order. Neither Chinese state ownership alone nor an instrument's AT1 label establishes these premises; designation, security coverage and dates require their own evidence (President of the United States, 2021, replacement section 1).

Now change the bank's capacity while preserving those instrument facts. OFAC permits specified investment advisory and management services for a non-US person when the underlying transaction is otherwise permissible. Its guidance explicitly addresses the ultimate benefit of US persons and evasion. This means that a US bank's involvement does not by itself establish that every non-US client transaction is prohibited (Office of Foreign Assets Control, 2021b, FAQ 902). OFAC also states that US financial institutions may continue to intermediate qualifying transactions between non-US persons. Conversely, a prohibited transaction under this particular order must be rejected; the order does not itself require an asset freeze (Office of Foreign Assets Control, 2022b, FAQ 1048). The system therefore needs the actual service and beneficial parties, rather than an undifferentiated "US bank involved" flag.

Time and action introduce another distinction. OFAC permits continued holding after the relevant divestment period, although a purchase or sale after that period requires authorization. An instruction to sell an existing holding can therefore require a different answer from an instruction to retain it (Office of Foreign Assets Control, 2022a, FAQ 1046). The exact prohibition and divestment dates depend on the designation history; an old prospectus cannot supply later list changes (Office of Foreign Assets Control, 2021a, FAQ 899). Automatic conversions, redemptions and other corporate actions require their own analysis. The present conditional implementation supports buying, selling and holding; it leaves an unsupported action unresolved.

Ownership also has program-specific meaning. Under OFAC's blocking ownership rule, suppose blocked person X owns half of company A and A owns half of company B. A becomes blocked through X's ownership, and B becomes blocked through A's ownership. Multiplying the two fractions and treating B as merely one-quarter owned by X misses the applicable propagation. Multiple blocked owners' direct stakes must also be aggregated (Office of Foreign Assets Control, 2014, FAQ 401, examples 1–3). Under the Chinese military-industrial securities regime, however, a subsidiary must itself be

designated: the blocking ownership rule does not apply merely because its parent appears solely on the Chinese military-industrial securities list (Office of Foreign Assets Control, 2021c, FAQ 857). A reusable ownership graph consequently needs a rule that states which relationships have legal consequences in each program.

2.4.3 Compliance includes institutional and account obligations

Sanctions are one source of additional restrictions. The retained regulation of the Office of the Comptroller of the Currency (OCC) requires national banks to maintain a written Bank Secrecy Act compliance program, with internal controls, independent testing, designated responsibility and personnel training. It also addresses customer identification within that program (Office of the Comptroller of the Currency, 2025, 12 CFR 21.21(c)–(d), 2025 edition). These obligations concern the institution's conduct and controls. Reading a security's prospectus cannot establish that the bank has performed them.

Applicability still precedes assessment. In the retained 2025 rules, 31 CFR 1010.620(a) addresses defined private-banking accounts established, maintained, administered or managed in the United States. The account definition adds conditions concerning the required minimum aggregate assets, non-US owners and a liaison with the institution. Thus neither the commercial description "private banking" nor a Hong Kong office address alone decides whether that particular US provision applies. Its substantive requirements include ownership, source-of-funds and account-activity inquiries, with enhanced scrutiny for the specified foreign political figures (Financial Crimes Enforcement Network, 2025b, 31 CFR 1010.605(m), 2025 edition) (Financial Crimes Enforcement Network, 2025a, 31 CFR 1010.620(a)–(d), 2025 edition). Those editions illustrate the scope distinction; their preservation does not establish that every provision remains current for a later transaction.

The proposed system separates these institutional and account requirements from product distribution and conduct rules. It also reserves a distinct place for the bank's internal policies and the client's mandate. An internal restricted list or product policy can prevent a service even where the examined public law does not prohibit it. The repository has not received JPMorgan's actual internal policies, so their requirements and permitted exceptions remain missing inputs. They must not be reconstructed from a public marketing disclosure.

Finally, execution and custody require the actual settlement route and relevant reporting duties. A requirement to investigate an account, reject a trade, freeze specified property or make a report has a different legal effect. The decision must preserve those distinctions. A generic negative eligibility flag cannot safely stand for all four actions, and satisfying one duty does not establish that all the others have been performed.

2.4.4 Compose the obligations without erasing qualifications

The common front end now needs to supply two kinds of information together: the instrument's sourced terms and the transaction's parties, institutions, services and dates. Rule packs selected for that context supply the applicable requirements. In the Python implementation these are organized into seven assessment categories: scope and coverage; sanctions; bank regulation and licensing; client and account controls; product distribution and conduct; bank policy and mandate; and settlement, custody and reporting. The categories are a software design choice. They do not assert that every relevant rule has already been discovered or implemented.

Each required assessment carries a result, a reason, source references and a recorded validity interval. A finding that a rule does not apply needs evidence just as a finding that its requirements are satisfied does. Unknown, conflicting, expired and missing findings prevent an affirmative result. The binding also identifies the instrument, client, booking entity, establishment, service, action, transaction time, knowledge time, and content hashes for the facts, settlement route and policy snapshot. Changing any of them invalidates the earlier bound finding. Checking a source's hash establishes unchanged bytes; the scope assessment must separately address legal currentness and meaning.

The conjunction has a simple conditional justification. Let R be the finite set of declared requirements. Let S contain requirements supported as satisfied or inapplicable by valid, consistent findings for this context. Let U contain requirements with unresolved findings, and let C mean that every required assessment category is represented. The implementation's permission within that declared scope is

$$P = C \ \wedge \ (R \subseteq S) \ \wedge \ (U = \varnothing).$$

A valid prohibition excludes its requirement from S, and a conflicting favorable finding also places that requirement in U. Either prevents P. An absent bank-policy assessment has the same practical consequence for permission, but its explanation is missing evidence rather than a proved legal prohibition. A favorable product assessment cannot alter either result.

Representing every category still leaves a possible omission. A program could receive one favorable assessment in each category while never being asked about another applicable restriction. The integrated investigation therefore starts from a versioned inventory of fourteen declared obligations. A request cannot shorten that inventory or supply its own completed findings. Requirements without an executable interpretation remain visible, including other sanctions programs, licensing, issuer law and the unavailable internal bank policy. This prevents omissions within the declared inventory; discovering every obligation outside it remains a separate legal question.

The factual references now retrieve the underlying documents. For a structured account record, the program reads the named field, checks its type and preserves disagreement between records. Source dependencies, effective dates, knowledge dates and amendments are checked separately. Consider the private-account definition above: the required minimum deposit and the places where the account is administered or managed

enter the calculation as distinct premises. A large current balance cannot replace the required contractual minimum. A quoted provision or an unchanged document still does not establish that a proposed interpretation follows from it.

The resulting conditional specifications use the shared RuleIR and Catala translation machinery. They include entity scope, selected sanctions, institutional controls, private-account diligence and reporting requirements, together with the existing product calculations. Formal-expression preservation is checked separately from compiled execution. An independently written mathematical specification supplies another comparison for the Boolean rules and exact monetary thresholds. Agreement between programs therefore supports their stated formal consequences while leaving the interpretation of the legal sources explicit.

Changes after an assessment also matter. The event record distinguishes a new source, an account change, a different settlement route and a corporate action. Each requires reassessment before the earlier decision can be reused. Reporting and other duties retain their responsible party, trigger, deadline and supporting evidence. A recorded completion document establishes that evidence was supplied; it does not independently establish that the real action occurred. When the integrated example uses the preserved public documents without a real account or internal bank policy, its transaction decision consequently remains qualified.

The checks enumerate every combination of satisfied, prohibited and unknown across the seven categories, giving $3^7 = 2{,}187$ cases. Further cases challenge conflicts, non-applicability, missing policy, source changes and changed entity, action or time. The ownership calculation has an independent finite check: for each of 512 three-entity graph and initial-blocking combinations, the test intersects every ownership-closed superset of the initial blocked entities and compares that set with the implementation's result. The algorithm is monotone and terminates on a finite graph; induction shows that every closed superset contains each entity it adds. Its terminal set is therefore the least closed superset under the supplied ownership premises.

No human quality labels are used in these checks. They establish properties of the declared formal rules and the handling of missing evidence. Correctness of a new legal interpretation, completeness of an ownership register and coverage of future amendments remain separate obligations. The system can generalize by applying checked semantics to new supported facts and qualifying unsupported situations; it cannot turn today's finite legal collection into a proof about every law that might govern a future transaction.

2.4.5 One investor, two CoCo offering decisions

A useful positive and negative test can hold the investor fixed while changing the security. Consider a hypothetical individual resident and located in the United States, buying USD 200,000 principal for their own account and benefit. Compare the original offering terms of two actual contingent convertible securities: Barclays PLC's February 2025 issue and Standard Chartered PLC's January 2025 issue. The question is whether

that investor satisfies the specified US purchaser conditions. Approval below means passing this particular condition; it does not mean that a regulator approved the investment or that the bank may execute the transaction. The different issue dates are retained: this is a comparison of preserved offering terms, not an assertion that both primary offers are open today.

The relevant investor category needs explanation before either document can be applied. A *qualified institutional buyer*, or QIB, is a category defined by Rule 144A for specified institutions and accounts. An individual's wealth or Hong Kong professional-investor status does not turn that individual into a QIB. For this case, non-QIB status is an explicit hypothetical premise. Regulation S uses a separate definition of US person, which includes a natural person resident in the United States. The branch and account exceptions in that definition also show why the sanctions definition of US person discussed above must not simply be reused. The comparison retains both the purchaser's status and the beneficiary of the purchase (Securities and Exchange Commission, 2025b,a, Rule 144A(a)(1); Regulation S, section 230.902(k)).

The first security is Barclays PLC's USD 1.5 billion 7.625% Fixed Rate Resetting Perpetual Subordinated Contingent Convertible Securities, with international securities identification number (ISIN) `US06738EDC66`. The supplement is dated 18 February 2025 and identifies 25 February as the expected delivery date. It identifies its Form F-3 registration statement; the underwriting section describes an offer directly to the public. The minimum denomination is USD 200,000, with USD 1,000 increments. These affirmative offering provisions support the declared public US distribution route. In particular, "registered form", which describes how ownership is recorded, is not itself the evidence of Securities Act registration (Barclays PLC, 2025, cover; pp. S-6, S-22 and S-99).

The second security is Standard Chartered PLC's USD 1 billion 7.625% Fixed Rate Resetting Perpetual Subordinated Contingent Convertible Securities. Its offering circular is dated 8 January 2025 and specifies issue on 16 January. The restricted and offshore identifiers are `US853254DF47` and `USG84228GP72`. It has the same minimum denomination and increment. Its US distribution, however, is limited to QIBs; the alternative described in the circular is an offshore distribution to non-US persons. The purchaser representations also distinguish acquisition for one's own account from acquisition for a QIB's account. Our individual satisfies neither the institutional route nor the offshore route (Standard Chartered PLC, 2025, pp. i, 14–15, 140 and 148–149).

The order size therefore does not explain the difference. Under the declared interpretations, the comparison is:

Original issue	Distribution relevant to this purchaser	Purchaser condition
Barclays, February 2025	Public US offering	Pass
Standard Chartered, January 2025	QIB-only US distribution	Fail

Neither row establishes SFC product approval or an entitlement to an allocation from the underwriters.

The conditional rule can now be stated precisely. Let R mean that the declared route is the public US distribution, and T that it is the specified Rule 144A/Regulation S distribution. Let Q mean that the purchaser is a QIB and buys for its own account or a QIB's account. Finally, let O mean that the transaction is offshore, the purchaser and beneficial account satisfy the specified non-US conditions, and the purchaser is not an issuer affiliate. The selected purchaser condition is

$$G = R \ \vee \ \big[T \wedge (Q \vee O)\big].$$

For Barclays, the declared public distribution makes R true. For Standard Chartered, R is false and T is true, but our investor makes both Q and O false. The equation therefore yields opposite results from the same investor premises. It represents these purchaser conditions, not every condition of either securities-law exemption, every jurisdiction's selling restrictions or the effectiveness of a registration statement.

Changing the purchaser tests whether this result has been tied incorrectly to an issuer's name. A QIB buying for its own account changes the Standard Chartered result to a pass under the same declared rule. A bank that is a QIB but buys for a non-QIB customer's account does not satisfy Q merely through the bank's status. Moving the original individual offshore also leaves a problem: US-person status and US beneficial ownership do not disappear with the location of execution. These changes distinguish purchaser, account and geography without supplying an answer label for either issuer.

The loss mechanisms remain relevant to the other assessments. Both documents describe contingent conversion and regulatory powers that can reduce or extinguish an investment. Passing a purchaser condition is consequently no evidence of principal protection. Hong Kong distribution, suitability, sanctions, bank authority, internal policy and settlement still require their own conclusions. The US purchaser comparison cannot replace those conclusions (Barclays PLC, 2025; Standard Chartered PLC, 2025, Barclays cover and pp. S-16, S-102; Standard Chartered pp. i, 140–142).

The executable comparison reproduces the two results through both RuleIR and Catala. Its 157 input cases produce 314 executions: each backend computes the independently derived result for 138 complete cases and abstains on 19 cases with missing, conflicting or expired evidence. A separate mathematical check compares the implemented expression with an independently written relation over all $2^9 = 512$ assignments of its nine Boolean premises. A proof checks preservation of the declared formal expression in the shared model, and six deliberately wrong variants are detected, including omission of the beneficial account condition. The focused regression suite passes 165 tests. These results establish conditional formal and software properties. The legal interpretation and completeness of the applicable requirements remain unproved.

In the integrated bank investigation, the Barclays purchaser result remains qualified by missing transaction evidence. The Standard Chartered result additionally prevents proceeding under the declared offering route. Both retain all seven assessment categories and the fourteen declared bank obligations; neither permits execution of an actual trade. The preserved prospectuses, explicit interpretation proposals, synthetic investor premises and reproducible checks make this a useful test without human quality labels.

New issuers can reuse the checked relation when its premises and scope apply; new offering terms or unsupported legal situations require a new interpretation or a qualified answer. The finite pair cannot establish future legal correctness.

2.4.6 Connecting the loss provisions to the selling rules

The next calculation follows the same investment from its loss mechanism to the Hong Kong selling requirements. The provisions surveyed in Section 2.3 supply the questions: what legal form does the instrument take, which contingent loss mechanism applies, which institution provides the service, and what is known about the purchaser? A contractual write-down, a conversion and a regulator's power are separate facts. The retained UBS notes provide for conversion. Cancelling their debt principal would not, by itself, establish that the shares received are worth zero.

The implementation connects seven conditional calculations. It considers direct debt with the specified contingent loss features and separately records whether an investment product falls within the related-product provisions. An equity-form preferred share does not become direct debt merely because it can lose value. Related-product scope remains document-dependent; the program does not invent a percentage threshold for a fund's principal strategy. It then examines the intermediary arrangement, the PI restriction, the specified PI exemptions, the resulting duties, the issuer's legal powers and any exercise of those powers. Each conclusion retains its required premises.

The execution-broker exception shows why this decomposition matters. The earlier calculation accepted an established exception as one premise. The new calculation expands it into the specific adviser or asset-manager arrangement, an order placed through that arrangement, the adviser's regulatory status, execution and custody services only, absence of direct customer contact, the written allocation of responsibility, written customer notification and the continuing arrangement. A direct customer request and circumvention each prevent reliance on the exception. These are proposed formal consequences of the retained FAQ, not evidence that a customer's arrangements meet its conditions. A bare assertion that the exception applies cannot bypass an unknown condition or contradict a known one.

Duties are computed separately from the selling restriction. The specified institutional and corporate PI exemptions affect risk rating, enhanced suitability and disclosure; the PI selling restriction remains separate. Loss-absorption funds receive the specified treatment for enhanced suitability and disclosure, with other SFC requirements retained. Discretionary portfolio management can change the level at which an assessment is performed. It does not supply evidence that the assessment has been performed. Additional disclosure flexibility referred to by the FAQ still needs its own applicable authority and conditions.

A regulator's power requires a further chain. The jurisdiction, entity, instrument, legal basis, triggering circumstances and procedure must support the particular power. An applicable order must then have been issued, its suspension status matters, and implementation is another event. The calculation therefore distinguishes authority from an

operative order and an implemented loss. It cannot infer any of them from the word AT1. For the UK examples, the joined report retains a missing instrument-specific UK resolution basis. For UBS, it retains the dated Swiss source and its currentness qualification.

Contractual amounts use exact rational arithmetic under explicitly supplied event conditions. A hypothetical conversion of principal 3,777 at a same-currency conversion price of 37.77 gives 100 shares and zero remaining debt principal. The market value of those shares is a different quantity. If notice, timing or other instrument-specific event conditions are unknown, the calculation remains unknown. Section 2.4.7 develops the instrument-specific notice and share calculations beyond this generic example, including the price branches that still prevent a unique result.

The joined result recomputes the bank assessment and retains all fourteen obligations in its seven categories. It compares the shared PI premise across the product and bank inputs, preserving contradictory observations. A changed source, fact document, mandate or execution route requires reassessment. These calculations supplement the broader product-law obligation; they do not mark it complete. With the retained public documents and no actual account, mandate or transaction evidence, the UBS, Barclays and Standard Chartered reports remain qualified and cannot authorize execution.

2.4.7 Executing the UBS notice and conversion terms

The UBS example can now begin with a particular issue rather than a generic conversion rule. Annex A of the retained published terms concerns the SGD 500 million notes issued on 24 June 2024, with ISIN CH1357852636. The same PDF contains the later SGD 175 million reopening in Annex B. The initial conversion price for the June issue is SGD 37.77, and holdings must respect the SGD 250,000 denomination and increment. Keeping the annex explicit prevents an internal capital loan, another currency issue or the reopening's issue date from supplying the wrong premise (UBS Group AG, 2024, Annex A, definitions and condition 2).

These terms describe direct subordinated obligations that can convert into shares. The source-dependent product reading therefore supplies five previously missing premises: debt form, qualifying contingent loss absorption, absence of the plain-debt or deposit exclusion, and absence of either a related-product wrapper or a loss-absorption fund. They yield a conditional conclusion that the direct notes fall within the selected HKMA product scope. The reading remains an interpretation of the retained edition. It does not establish current legal force, the purchaser's PI status or the bank's capacity. The combined assessment continues to carry those questions through all fourteen bank obligations.

A breach creates a notice question. The capital test uses the figures attached to a particular publication. Let C be its CET1 capital, H the defined Higher-Trigger Amount, and $R > 0$ its risk-weighted assets, all expressed in the same monetary unit. The Higher-Trigger Amount represents capital capable of absorbing losses under the higher-trigger

provisions; omitting it can change the result. The published ratio and breach test are

$$r = \frac{C + H}{R}, \qquad r < 0.07 \quad \Longleftrightarrow \quad 100(C + H) < 7R. \tag{2.1}$$

The positive denominator permits multiplication without reversing the inequality. With $C = 6$, $H = 1$ and $R = 100$, the ratio is exactly seven percent, so this strict test does not breach. With $H = 0$, it does. Condition 7(d) uses the publication's figures even if they are later restated. The software binds the calculation to that retained publication snapshot (UBS Group AG, 2024, definition of Trigger CET1 Ratio; conditions 7(b), 7(d)).

A breach is not itself the defined Trigger Event. Condition 7(a) ties that event to the issuer giving a Trigger Event Notice in accordance with the terms. An ordinary publication normally starts a five-business-day notice period; an extraordinary publication calls for notice on the publication date. The ordinary branch also has a restoration exception involving a UBS request, FINMA's written agreement, restoration or imminent restoration above the threshold, their assessment of adequacy, and a strict timing condition. The agreement must precede the earlier of the ordinary notice date and the original fifth-business-day determination date. An unknown earlier notice history or unresolved ordering within one day therefore leaves the exception qualified. A bare restoration percentage cannot establish it.

Outstanding higher-trigger instruments can postpone both the notice and the conversion. The program retains the full supplied inventory and selects the latest relevant conversion date; one observed higher-trigger notice cannot stand for all of them. Notice publication is a further factual question. Condition 14 uses the first qualifying SIX publication while the notes are listed, or delivery to the intermediary after delisting. The implemented publication path does not infer a notice from its deadline; alternative exchange-rule notice methods still require their own evidence and treatment (UBS Group AG, 2024, conditions 7(a)–(b), 14).

Different events use different clocks. For the publication definitions, the Business Day definition refers to Zurich. For the other relevant calculations it requires Singapore and Zurich. The calendar input therefore records a covered interval and the days on which the specified banks and foreign-exchange markets are open in each location. An ordinary weekday or a public-holiday list alone does not establish that condition. The engineering examples use explicitly hypothetical calendars.

A Viability Event has its own FINMA and, where relevant, public-support premises. Its notice must be given within three days, while conversion must occur within twenty Business Days following the event. The latter period starts from the event, not from the notice. In a hypothetical calendar with both markets open on every weekday, a Friday event on 4 September 2026 has a Monday notice deadline on 7 September and a conversion deadline on 2 October. Starting the twenty-day count from that Monday notice would instead produce 5 October. The program rejects that extra time. A qualifying change in law can stop the viability provisions under Condition 7(e), but they cease from the specified actual notice date, not merely the date of the joint determination.

The capital-ratio trigger remains a separate branch (UBS Group AG, 2024, Business Day definition; conditions 7(c), 7(e)).

Shares, prices and delivery need separate evidence. Once the appropriate notice and price premises are supplied, the calculation aggregates the principal held by one holder before discarding any fraction of a share. For aggregate principal P and conversion price $K > 0$ in the same currency, the number of shares is

$$N = \left\lfloor \frac{P}{K} \right\rfloor, \qquad NK \leq P < (N+1)K. \tag{2.2}$$

The inequality checks the rounding independently. Rounding each lot first can discard more than one holder's terms permit. The discarded fraction does not produce cash compensation. Nor does the resulting share count establish a market value. Actual share creation, receipt by the settlement depository, cancellation of accrued unpaid interest and subsequent delivery to the holder remain distinct events (UBS Group AG, 2024, conditions 8(a)–(c), 8(g)–(j)).

The price is adjusted through the notice date, including that date. A later corporate action cannot silently replace the price applicable to the notice. Implemented arithmetic covers declared share splits and bonus issues, discounted rights, exact-cent adjustments, the par-value floor and a bounded carry-forward case. Other calculations cover the stated successor-share price ratio and the distribution of supplied net offer proceeds. Their market prices, exchange rates, relevant dealing days and contractual determinations remain explicit premises. Ambiguous rounding, overlapping adjustments, unavailable action history, amended terms and issuer substitution qualify the result rather than select a convenient convention.

The retained source itself supplies a useful challenge. Condition 8(d)(i)(C) prints the extraordinary-distribution factor as $(A - B)/B$, where A is the specified market price and B is the distribution per share. The rendered PDF confirms the denominator. At $A = 100$ and $B = 10$, that expression gives 9, whereas $(A - B)/A$ would give 0.9. The program preserves the printed expression and its literal result, records the unresolved intended adjustment, and withholds an operational price for that branch. It does not correct the contract by appealing to the more familiar formula (UBS Group AG, 2024, condition 8(d)(i)(C), PDF p. 30).

The checks exercise these distinctions without using human expected legal answers. Independent mathematical relations test the capital comparison, restoration predicates, date ordering, product scope and PI restriction; native RuleIR/Java and Catala executions test the corresponding programs. Calendar selection and whole-share inequalities separately challenge the Python calculations. Source substitutions, capital restatements, missing notices and changed scenarios must invalidate or qualify the affected result. The PI and non-PI examples change declared client premises while holding the issue fixed. They distinguish the selected selling condition without turning either example into approval by an actual private bank. Incorporated agency agreements, actual notices and market histories, applicable current law and private account evidence remain

required for a real transaction.

2.4.8 A preferred share changes the calculation

The UBS program cannot simply be reused for every security that supplies bank capital. Bank of America's Series SS provides a useful contrasting case. Each depositary share represents one thousandth of a preferred share, with a USD 25 share of its USD 25,000 liquidation preference. The preferred stock pays non-cumulative dividends when declared and has no contractual conversion right. Its equity form therefore changes the source-dependent reading of the selected HKMA debt-product scope. That conclusion leaves the other selling, sanctions and bank obligations to be assessed separately (Bank of America Corporation, 2022a, pp. S-16–S-17, S-22, S-27).

Consider a regular three-month dividend period and a holder of 100 depositary shares. The annual rate is 4.750 percent. The source specifies a 360-day year of twelve 30-day months, so a regular quarter contributes $90/360 = 1/4$ of the annual amount. Let n be the number of depositary shares and f the fraction of the regular dividend actually declared, with $0 \leq f \leq 1$. Before cash rounding and withholding, the holder's conditional dividend in US dollars is

$$D_{\text{exact}} = n \times 25 \times 0.04750 \times \frac{1}{4} \times f. \tag{2.3}$$

For $n = 100$ and $f = 1$, this gives USD 29.6875. Under an explicitly supplied convention that rounds the holder's aggregate allocation to cents, with half a cent rounded upward, the gross payment would be USD 29.69. If no dividend was declared, $f = 0$: the missed amount does not accumulate into the next period. This is a loss of that distribution, not a conversion of principal into UBS shares. The declaration and legally available funds remain factual premises, and the calculation does not establish that payment occurred (Bank of America Corporation, 2022a, pp. S-17, S-27).

The rounding unit deserves attention before money is assigned to an account. The preferred-share calculation and the depositary distribution both discuss cent rounding. Rounding a full preferred share first and then allocating its rounded dividend need not equal rounding a holder's allocation of the exact amount. For 10,000 depositary shares, these two declared computational scenarios give USD 2,968.80 and USD 2,968.75 respectively. The implementation exposes both and requires the applicable allocation convention rather than choosing one from agreement among models. Initial and irregular dividend periods also remain qualified when the necessary month-end convention is absent. Taxes, record ownership and actual depository receipts require their own evidence.

The source hierarchy changes another result. The supplement states that the Certificate of Designations controls an inconsistency. Its summary describes notice within 90 days of a capital treatment event followed by redemption; section 6(a)(ii) of the certificate places the special redemption itself within 90 days. The conditional program follows the latter timing, together with the 30-to-60-calendar-day notice interval. It also retains the whole-issue condition for this special branch. The ordinary branch begins

on 17 February 2027. Neither branch makes redemption mandatory or removes the issuer's election, lawful-funds and applicable regulatory-approval requirements (Bank of America Corporation, 2022a, pp. S-16, S-19–S-20) (Bank of America Corporation, 2022b, sections 6(a)–(b)).

Liquidation supplies a third, distinct calculation. The available amount is what remains after senior claims, not the issuer's gross assets. The Series SS claim includes the liquidation preference and the specified declared unpaid dividends; undeclared dividends are excluded. If the available amount is insufficient, that claim shares proportionately with the supplied parity claims. The executable allocation conserves the amount distributed and leaves only the excess for junior shareholders. It cannot establish the complete creditor inventory or infer an actual liquidation from a merger (Bank of America Corporation, 2022a, p. S-19).

Dates belong to a market and a purpose. The same discipline applies to calendars. A UBS successor-share calculation can involve ordinary shares and replacement shares traded on different exchanges. Each five-day average must use its own preceding dealing days. The revised arithmetic accepts two covered calendars, dates each exchange rate, and separately checks the date of the existing conversion price. A missing observation or an unexplained shared calendar leaves the price qualified. Contractual adviser or issuer determinations can supply a conditional factual price only when tied to the issue, event, action history and applicable exceptions. Such an act is evidence about the contract's operation; it is not a quality judgment about the program's interpretation (UBS Group AG, 2024, conditions 8(d)–(e), 8(m)–(n)).

Calendar retrieval itself can fail subtly. The stable SIX banking-holidays address inspected for this increment returned a schedule of clearing days for 2027. That document distinguishes SIC and euroSIC, including days on which their answers differ. It supplies neither a complete inventory of 2026 dates nor proof that the banks and foreign-exchange markets named in the UBS terms are open. The program therefore binds the downloaded content, covered interval, market and calendar purpose. It rejects missing dates instead of treating them as ordinary business days (SIX, 2027, one-page schedule).

A transaction needs facts about the same transaction. Private evidence now carries the instrument, client, booking entity, establishment, service, action and route to which it applies. Its source content, effective interval, recording time, freshness and declared coverage are checked before its factual assertions enter the bank calculation. An assertion about another route cannot satisfy the present route; conflicting PI assertions remain conflicting. A machine-readable request identifies the remaining facts while retaining all fourteen obligations. The coverage check concerns that declared inventory: complete discovery of every applicable law remains unproved.

The UBS event assessment also assembles the new evidence branches. An alternative permitted notice method can supply the effective notice date without inventing a SIX publication. A scoped price determination retains the earlier reconstructed price and

its unresolved conventions. Holder delivery is checked against the resulting share entitlement, and it cannot establish completed conversion while the notice sequence, share creation or depository receipt remains unresolved.

Five additional Lean propositions establish rational floor and half-up rounding properties and characterize the earliest date selected by a cumulative count of open days. Separate tests challenge the actual Python calculations. Four further declared relations are checked against independent equations and compiled for RuleIR/Java and Catala. These checks establish formal consequences of the supplied premises. The source-dependent readings, missing incorporated documents and actual legal force retain their stated qualifications.

Later observations also require careful dating. A new private fact, an old law newly added to the collection, and a document published after the method was frozen are recorded separately. Declared publication dates do not become externally attested timestamps. The current frozen method has no future observations, and the prepared full-document model comparison has no fresh responses under this increment's offline execution. Neither missing result is replaced by successful development tests.

2.5 New prospectuses: payment, scope and external law

The results in this section are dated observations from the 4 October campaign. Later OCR and source-admission work changed the evidence without establishing complete interpretation. Section 2.6 corrects the current-state reading, diagnoses the remaining failures and develops the solution.

A bond can promise repayment at its principal amount while another provision allows an authority to reduce that claim. Equally, a document can describe a reduction of outstanding principal because the investor has already received part of the money. These sentences can look similar to a text classifier, but they describe different changes in the investor's rights. The distinction must be settled before the resulting product classification is used in a selling decision.

On 4 October 2026, the unchanged prospectus reader was exercised on 24 newly acquired public PDFs. Twenty were issue-specific documents and four were programme or base prospectuses. They represented 16 issuer families; several documents covered multiple series, and the Ecuador and Volcan pairs included an initial offering and a later tap of the same series. The reader abstained on 21 documents and returned three conditional positive readings. An abstention means that its premises remained unresolved, not that the bond was found to lack a loss mechanism. One positive used demonstrably incorrect supporting clauses. Appendix B identifies every document, its source date, retained PDF and observed result.

2.5.1 A smaller outstanding balance need not be a loss

Mizuho Markets Cayman's August 2026 programme provides the clearest error. Its form of final terms says that amortizing notes are partially redeemed at the specified Amortization Amount and that their outstanding principal is reduced accordingly. A neighbouring zero-coupon option likewise reduces outstanding principal *upon payment* of the amortization amount. Both passages became principal-write-down witnesses in the reader's positive answer.[1]

The payment relation explains why that inference is wrong. In a simple illustration, an investor owed 100 receives 20 and retains a claim for 80; the reduction records repayment. If 20 of an unpaid claim is extinguished, the investor instead retains 80 without having received the missing 20. The programme passage describes the first relation. Recognising the verb *reduced* without the payment that causes the reduction loses the distinction. Moreover, these passages are unselected form options. Correctly interpreting one option would still not establish which terms govern a particular issued bond.

Ethniki's September 2026 restricted Tier 1 notes supply a contrasting principal-loss clause. Their Prevailing Principal Amount may be Written Down, and redemption can occur at that reduced amount without a later entitlement to the difference. The reader found this mechanism but left 57 candidate records unresolved. MUFG's August 2026 base prospectus similarly states that the principal of *Subordinated Notes* is written down to zero following a Non-Viability Event. That clause supports a reading of the subordinated option; it does not classify every note issued under the programme. [2]

2.5.2 The principal must belong to the right instrument

A provision's subject matters as much as its action. All three unresolved records in EDF's 2023 perpetual-note document discussed separate OCEANE convertible bonds due in 2024. An apparent exclusion of principal write-down elsewhere in the same document applied to *Exchanged Notes or Varied Notes*. It was therefore unsuitable as an exhaustive negative statement about the original notes. Volcan's 2025 memorandum supplied an even sharper contrast: several candidate forfeiture clauses concerned mining concessions or water licences. Those assets can affect the issuer's financial position, but forfeiture of a mining concession is not itself cancellation of the principal of its Notes.[3]

Missing context can also produce a misleading impression of completeness. Zurich Finance Ireland's eight-page July 2026 pricing supplement identifies senior notes and a redemption amount, but expressly incorporates the 19 May 2026 base prospectus. The isolated-document reader left debt status unknown while marking its local clause coverage complete. Both PDFs had been downloaded, but the diagnostic input had not

[1] Mizuho Markets Cayman LP, US$10 billion medium-term note programme, 31 August 2026, PDF pp. 125–126 and 128.

[2] Ethniki, EUR 100 million 6.875 percent RT1 notes, 7 September 2026, PDF pp. 44–45; MUFG, US$50 billion programme, 7 August 2026, PDF pp. 1, 20 and 88.

[3] EDF, US$1.5 billion perpetual notes, 15 June 2023, PDF pp. 15, 33, 49, 57 and 136; Volcan, US$750 million senior secured notes, 14 November 2025, PDF pp. 35, 80 and 116–117.

declared their dependency. The useful repair is to connect the correct issue documents and interpret their labelled fields. The absence of a clause from a short supplement cannot establish that the complete terms contain no such clause. [4]

2.5.3 The applicable legal power needs a date and a subject

HPB's July 2026 senior preferred notes illustrate the next step beyond a repayment promise. The disclosure says that an authority may order a write-down of the Notes or convert them into CET1 instruments, and describes cancellation of liabilities including the Notes. The reader left these clauses unresolved. The source supports investigation of a principal-loss power; the phrase *CET1 instruments* still needs interpretation before it can establish the narrower common-share conversion feature. An exclusion of contractual loss terms in the definition of replacement Qualifying Securities does not negate statutory powers over the original Notes.[5]

Insurance-resolution language shows why the date cannot be inferred from the presence of a legal reference. AXA's June 2024 prospectus calls the Insurance Recovery and Resolution Directive proposal a proposed measure and says *If adopted*. SCOR's June 2026 prospectus describes national implementation and application in January 2027 and states that French implementation was pending. These passages disclose a possible future change. They do not, by themselves, establish an already applicable power at the relevant assessment date. Existing contractual provisions and other national powers must be examined separately.[6]

Nor is the question confined to regulatory capital. Prudentia's AirBlu compartment terms extinguish the right to further amounts after the compartment assets have been realised and distributed. Orange describes court restructuring capable of binding dissenting creditor classes. Whether these mechanisms satisfy the declared principal-loss predicate requires analysis of the unpaid claim and the authority for the change. Neither can be excluded merely by calling it ordinary credit risk or observing that the security is not bank capital.[7]

The 24-document exercise therefore identifies several different engineering tasks: interpret payment and extinguishment correctly; bind a clause to its instrument; assemble the issue documents; and establish which law applies when. The sources and results remain preserved with the original reader unchanged. The findings are based on inspected extracted text and are provisional pending rendered-source and independent legal review. Passing software tests cannot supply those missing interpretations.

[4] Zurich Finance (Ireland) DAC, Series 70, tranche 1, pricing supplement of 8 July 2026, PDF pp. 1–3; Zurich EMTN base prospectus, 19 May 2026.

[5] Hrvatska Poštanska Banka, EUR 150 million senior preferred notes, 9 July 2026, PDF pp. 28, 31 and 59.

[6] AXA, EUR 125 million senior notes, 12 June 2024, PDF pp. 11–12; SCOR, EUR 500 million subordinated notes, 2 June 2026, PDF pp. 27–28.

[7] Prudentia AirBlu, EUR 35 million notes, 13 October 2022, PDF p. 44; Orange, US$6 billion notes, 13 January 2026, PDF p. 45.

2.5.4 Three further historical documents

A further search retained three historical documents for testing the boundary between contractual promises and local-law investigation. Banco Espírito Santo's final terms dated 6 May 2014 identify EUR 750 million of 2.625 percent senior notes due 8 May 2017, Series 36, tranche 1. Page 2 promises redemption at 100 percent of nominal amount, subject to the stated prior-redemption and cancellation qualifications. Page 1 incorporates the July 2013 prospectus and four dated supplements. Thus the visible senior-status and repayment fields are precise contractual evidence, but the five-page final terms alone cannot answer a question about later Portuguese resolution measures or their recognition abroad.[8] The proposed follow-up is to match the exact security to the relevant Portuguese measures and judicial decisions. That matching has not yet been established by the retained external-law sources.

The Greek document gives a directly visible example of a broad promise qualified by another jurisdiction's law. Its March 2008 offering circular covers a EUR 600 million tap of inflation-linked notes due in 2057. Condition 4 floors the contractual redemption amount at principal. Condition 14 selects English law and contains a broad waiver of immunity, but then expressly subjects execution and attachment of the Republic's property to international conventions and Greek law. A repayment entitlement, jurisdiction over a lawsuit and access to assets for enforcement are therefore three different propositions. The country name also cannot justify attaching a Greek-law restructuring rule to these English-law notes without checking that rule's scope and the actual series history.[9] No specific seizure, restructuring outcome or recovery amount is established by that qualification alone.

The Slovenian search produced SID Bank's March 2011 memorandum, rather than an NLB subordinated issue. It describes a EUR 350 million tap of 3 percent notes due in 2015 and an unconditional, irrevocable state guarantee. Its description of the guarantee refers to article 13 of the Slovenian Export and Development Bank Act, transactions under articles 11–12 and a creditor's written request. The Notes themselves use English law. The correct investigation must therefore establish the statutory guarantee's scope and conditions, while keeping SID Bank distinct from another Slovenian bank with different instruments or intervention history.[10] This document is retained as an issuer-and-guarantee control. It does not demonstrate that the guarantee was overridden or that these notes suffered a principal write-down.

These new documents extend the tests in two directions. The BES terms provide a concrete repayment promise requiring a later-event investigation. The Greek and SID documents prevent that investigation from becoming a country-wide presumption: governing law, enforcement limits, issuer identity and the statutory guarantee each have

[8] BES, Series 36, tranche 1, final terms of 6 May 2014, PDF pp. 1–4. Page 4 confirms ISIN PTBEQKOM0019. The retained PDF is a scan; these fields were inspected on rendered pages.

[9] Hellenic Republic, 2.085 percent inflation-linked notes due 25 July 2057, offering circular of 28 March 2008, PDF pp. 10 and 17–18. The permanent ISIN is XS0292467775 (PDF p. 25).

[10] SID Bank, EUR 350 million 3 percent notes due 2015, information memorandum of 9 March 2011, PDF pp. 1, 18 and 24. The retained filename records the original search candidate; the document's issuer is SID Bank.

to be bound to the particular claim. The first retrieval round did not yield readable judgments. The extended search has now retained the relevant European judgments and the BES base prospectuses and named supplements. These sources make the legal questions more precise, while leaving the exact identifier-to-measure link open for the May 2014 issue.

2.5.5 A repayment promise after a liability moves

The clearest new comparison begins with another BES issue. Final terms dated 14 July 2011 promise redemption at par of EUR 81.4 million of senior notes, paying 6.875 percent and maturing on 15 July 2016. The scanned pages identify Series 23, Portuguese law and ISIN PTBEQBOM0010. They incorporate the 3 November 2010 base prospectus and six supplements, all now retained. The 2013 base used by the two later BES issues is a different edition; its cover cannot silently replace the earlier terms.[11]

The Court of Justice's 2024 *Novo Banco* judgment describes a senior bond with that coupon and maturity. Its purchaser acquired the bond in November 2014, after the original transfer of liabilities to Novo Banco, and received the following year's coupon. At maturity, Novo Banco paid neither the next coupon nor principal. It relied on the Bank of Portugal's 29 December 2015 decisions transferring the liability back to BES. Thus a historical coupon payment cannot establish who remains liable at a later date. The court describes the bond by name, coupon and maturity but does not print its ISIN. The retained final terms are consequently a strong candidate match; the missing Portuguese Annex 2B still prevents an exact identifier-level conclusion.[12]

Timing changes the recognition question. In the 2024 case the measures preceded the Spanish proceedings. The Court held that the omitted publication required by article 6 of Directive 2001/24 did not itself prevent recognition, while preserving effective access to an appeal in the home state. In C-504/19, by contrast, the later retransfer sought to change liability during proceedings in which the claimant had already obtained a favourable decision. The Court rejected unconditional recognition that would retroactively remove the defendant and call those decisions into question. That dispute concerned a share-sale agreement, not the senior bond. A useful engine must retain both the procedural dates and the type of claim; a single rule saying that all retransfers succeed or all fail would misread these decisions.[13]

[11] BES, Series 23 final terms, 14 July 2011, PDF pp. 1–3 and 6; BES, EUR 20 billion programme, 3 November 2010, PDF p. 1. The final terms were inspected as images. The base expressly says it does not affect notes already issued.

[12] Joined Cases C-498/22 to C-500/22, *Novo Banco*, 5 September 2024, ECLI:EU:C:2024:686, paras. 58–63. This account is the factual history recorded in the judgment, not a finding about ultimate recovery from BES.

[13] The 2024 judgment, paras. 89–97; C-504/19, *Banco de Portugal and Others*, 29 April 2021, ECLI:EU:C:2021:335, paras. 49 and 59–66.

2.5.6 Payment can be delayed, reduced or enforced elsewhere

HETA's notices separate changes that a loss classifier could easily collapse. The March 2015 notice describes amended maturities: the affected amounts would not yet become due and payable. The April 2016 notice reports a later decree reducing eligible nonsubordinated liabilities, including interest due through 28 February 2015, *to* 46.02 percent of par. Interest falling due after that date was reduced to zero, and the latest maturity was set at 31 December 2023. A remaining amount of 46.02 percent corresponds to a 53.98 percent reduction in that stated bucket. The notice adds that the legal relationship and trading could continue. Continuing existence and tradability therefore do not establish unchanged payment rights.[14]

Adriatica Finance offers a different kind of local-law problem. Its 2007 securitisation finances claims arising from the Abruzzo region's payment undertakings for healthcare liabilities. The prospectus describes those undertakings as irrevocable and unconditional, but also explains why obtaining payment can be harder than reading that promise suggests. Certain public funds cannot be attached if the stated statutory conditions are met. Execution against the region is subject to a 120-day period after notification of the enforceable instrument and payment request. Accelerating the Notes does not accelerate the region's remaining scheduled payments. These qualifications concern available assets and timing; they must not be recast as immediate principal cancellation.[15]

The same prospectus distinguishes maturity from extinguishment. If funds are insufficient at December 2021 maturity, unpaid amounts remain outstanding until the last business day of December 2026. Condition 7(b) then cancels outstanding amounts in the absence of the issuer's gross negligence or wilful misconduct. Neither the earlier maturity nor the later cancellation can be interpreted without its condition. The seller's letter of undertaking also expressly disclaims being a guarantee; public receivables and transaction support are not interchangeable with a guarantee of the Notes.[16]

2.5.7 A foreign-law objection can fail

Dana Gas supplies a useful control against assuming that local law always defeats a payment promise. The 2017 English judgment addressed a transaction with UAE-law and English-law documents. For a preliminary issue, the court *assumed without deciding* the alleged invalidity under UAE law. It nevertheless held the English-law purchase undertaking valid and enforceable, having examined its construction, allocation of risk,

[14] HETA, notice concerning the decree of 1 March 2015, PDF p. 1; HETA, notice dated 11 April 2016 concerning the decree of 10 April 2016, PDF p. 2. These are issuer reports. The original FMA decree and exact instrument list were not obtained, so they do not establish a final recovery amount or the treatment of an unidentified HETA security.

[15] Adriatica Finance, EUR 140.95 million asset-backed notes due 2021, prospectus of March 2007, PDF pp. 17, 43–46. The retained source describes the historical law; the cited statutory versions have not been independently acquired. The acquisition key records a discarded Hypo search hypothesis, not the issuer's identity.

[16] Adriatica Finance, PDF pp. 12, 37 and 121, Condition 7(b). These provisions establish contractual possibilities, not evidence that cancellation or a shortfall actually occurred.

mistake and public policy. An engine must identify which obligation a particular law governs and whether the court adopted a proposition as a finding or only as an assumption for argument.[17]

Lloyds' enhanced capital notes expose another distinction. Changes to regulatory capital requirements were held by the majority to satisfy a Capital Disqualification Event, enabling optional early redemption. The conversion trigger was a separate provision. The judgment does not establish that the notes were converted or their principal written down. The dissent disagreed about redeemability, so extracting an isolated judicial sentence without its author and disposition could reverse the result.[18]

The Ukrainian sovereign-note decision illustrates a procedural limit. The Supreme Court refused summary judgment and allowed the specified alleged force-based duress defence to proceed to trial. That is neither a finding that duress occurred nor a final cancellation of the debt. The majority rejected the capacity, authority and countermeasure defences. Those different dispositions belong in the answer together.[19]

2.5.8 What the additional tests actually found

The extension records 30 distinct questions, supported by 51 checked passages, across offering documents, judgments and issuer notices. They include the differences between a state-aid communication and a national power, shares and bonds in prospectus litigation, and deposit guarantees and investor compensation. The companion gives each question an identifier, its source and the inference it must prevent.

Seven new offering PDFs were submitted to the unchanged reader. All seven produced abstentions, with 29 unresolved clause records and no runtime errors. Three additional page-limited probes reproduced a more specific error: clauses extinguishing *unpaid* principal were classified as ordinary cancellation following redemption or repurchase. Two occur in Adriano Lease Sec.'s conditions, including cancellation after a servicer certificate and representative notice; the third is Adriatica's later Cancellation Date. The original conditions preserve exceptions that a repaired interpretation must carry forward.[20]

This is a wrong clause classification relative to the described unpaid extinguishment. It did not produce a complete negative issue answer in these runs: the reader still abstained. The result therefore identifies a concrete repair target without measuring population accuracy. The remaining legal questions are source-supported specifications for an integrated evaluator and independent adjudication; they are not 30 reported engine

[17] *Dana Gas PJSC v Dana Gas Sukuk Ltd*, [2017] EWHC 2928 (Comm), paras. 1–2, 75–84 and 86, PDF pp. 2 and 17–19. The original 2013 listing particulars remain missing; the judgment is retained as a legal-reasoning source, not submitted as a prospectus.

[18] *BNY Mellon v LBG Capital No 1*, [2016] UKSC 29, paras. 10–15, 35–54 and 55–62, PDF pp. 3–4 and 8–14.

[19] *Law Debenture Trust Corporation v Ukraine*, [2023] UKSC 11, paras. 209–211, PDF p. 78. Subsequent proceedings and ultimate payment are outside this retained-source comparison.

[20] Adriano Lease Sec., Class A notes due January 2041, 28 December 2011, PDF p. 140, Condition 8.1.3, and pp. 144–145, Condition 9.2.3; Adriatica Finance, PDF p. 121. Raw outputs and the failing regression checks are preserved under `docs/implementation/prospectus-corner-cases-2026-10-04/`.

successes or 30 proven instances of investor loss.

2.6 From selected terms to a qualified prospectus answer

Outstanding principal falls when an issuer repays a bond, when the contract extinguishes an unpaid claim, and when an authority imposes a write-down. These events have different legal and financial meanings. A reader that notices only the reduction can give the same answer for all three. Interpretation must preserve the selected obligation, its conditions and exceptions, and the sources that determine its effect.

The 6 October 2026 investigation traced the actual application. The ordinary classification command still used the original reader. The repaired candidate ran from a separately retained source tree; OCR and BASF admissions supplied evidence through explicit consumers. Success in one path did not establish that the ordinary application used the repair. Earlier completion notes recorded bounded campaigns, not delivery of a general interpreter.

2.6.1 Corrections established by the evidence

Seven constructed examples separated interpretation from coverage. Both preserved readers correctly handled explicit repayment and principal write-down. Both returned a negative when principal obligations ceased permanently *without payment*, and when payment depended on an unavailable Condition 14. Both returned a compulsory-conversion positive for an explicitly non-operative example and for a conversion sentence governed by the holder's option in the preceding sentence. The candidate corrected the blank-operative-page case; the ordinary reader still returned a negative. These are counterexamples to the respective behaviours, not estimates of their frequency in prospectuses.

Conversion required after exercise of an option does not make exercise itself compulsory. Interpretation must preserve the election owner and the earlier sentence supplying its antecedent. Likewise, a repayment promise cannot establish absence of principal loss while another operative condition remains unread. Affirmative absence requires a complete scope for that question.

Two historical statements need dated qualifications. The empty BES scan was a failure of the 4 October intake; later OCR supplied text and source-review receipts. Legibility did not settle every incorporated document or the precedence of duplicate risk factor 1.17. The subsequent BASF admission accounted for 55 checkboxes and eight bilingual responses. It expressly did not construct the complete German contract. Neither result establishes complete interpretation.

2.6.2 What the literature contributes

ContractNLI requires supporting sentences and list items to be self-contained, even when they occur in separate passages, and distinguishes local from distant exceptions (Koreeda and Manning, 2021, Sections 2–5 and Appendix A.1). This supports retaining a governing paragraph with its subordinate clauses. Its 607 nondisclosure agreements and 17 hypotheses are a different task from prospectus analysis. Scans and multicolumn documents were excluded during collection. The results therefore do not validate BES OCR or German marginal instructions, and the label *not mentioned* cannot prove contractual absence.

Holzenberger and Van Durme connect statutory reasoning to typed spans, events and argument relations (Holzenberger and Durme, 2023, Sections 3–5 and Appendices B, D and E). A calculation needs the actor, amount and date, not merely an event name. Their empty-knowledge-base control shows why negation-as-failure can produce apparently correct answers despite missing extraction. The prospectus service retains missing premises explicitly and does not adopt their research implementation's date-filling defaults as contractual dates.

LegalRuleML separates sources, interpretations, contexts and the authority selecting between alternatives (Athan et al., 2014, Sections 3–4). Structured argumentation distinguishes attacks on premises, conclusions and inferences (Prakken, 2010, Sections 3 and 6). These distinctions represent conflicting language versions, duplicate provisions and incorporation ranges. They do not decide which document governs. Priority needs a source-backed interpretation.

Catala executes exceptions and conflicts after formalization (Merigoux et al., 2021, Sections 4.1, 4.4–4.5 and 6.1). Its compiler correctness argument concerns translation between formal languages; it does not prove a faithful translation of the prospectus. The same boundary applies to deterministic calculations and witness replay here. Repeating a recognizer establishes reproducibility; independent reading tests interpretation.

Month addition can depend on rounding and operation order: two one-month adjustments need not equal one two-month adjustment (Monat et al., 2024, Sections 2, 3.2 and 4.3). Business-day movement answers a different question from calendar-month rounding. Neither a library default nor a guessed date establishes the contractual convention. The recent *Know Your Limits* preprint also separates translation from solver execution and documents unsupported assumptions (Wang et al., 2026, Sections 3–5 and Appendix A). Its retained version has inconsistent dataset totals and reports manual program corrections; its numerical rankings supply no adoption threshold here.

Technical sections and relevant appendices were inspected for all seven papers, along with retained SARA implementation files, related PyArg code and selected official Catala excerpts. These sources support explicit context, typed arguments, alternatives, exceptions and time. None supplies a complete prospectus engine or a transferable accuracy claim.

2.6.3 Constructing the selected contract

BASF SE's EUR500 million 4.25% notes due 8 March 2032, ISIN XS2595418596, select German Option I from the September 2022 base prospectus. The operative text must be assembled from the main column, marginal instructions, nested alternatives and completed fields. A checkbox selects an alternative; it is not itself the operative clause.

Several selections must survive together. The scheduled holder put is excluded; the change-of-control right retains its rating condition and exercise procedure. The Finance-only initial guarantee is excluded for BASF SE, while the conditional successor guarantee in §10(d) remains relevant to substitution. The clean-up call requires both acquisition of at least 75% of original principal and the corresponding reduction recorded in the global note. It concerns the whole residual issue, with 30–60 days' notice and repayment at par plus accrued interest. Removing every guarantee reference, every holder right or the second clean-up-call condition would construct a different contract.

Every page is inventoried before semantic filtering. Text units retain source hashes, page coordinates, reading order and language. Assembly records the source and reason for each deletion and substitution and preserves unresolved instructions. Continuous text remains a candidate until relevant selections and references are resolved. BASF's annual-report incorporation exposes a remaining disagreement: the broad heading ends at page 209, whereas the detailed notes range ends at page 290. Both references must survive review.

Interpretation separates six questions. The first concerns contractual principal loss and reports statutory disclosure separately. The second records conversion, election owners, and possible common shares versus common shares as the sole outcome. The third concerns question-specific completeness. The fourth combines a dated legal basis with separately supplied jurisdiction, entity, instrument, event and procedure facts. The fifth computes a supported conditional financial quantity. The sixth retains the bank's existing obligations. A product feature does not discharge those obligations or authorize a transaction.

Completeness is local to the question. An unread payment condition can defeat a negative answer about principal loss; a missing financial metric need not defeat a supported statement about a selected option. Excluding a dependency requires a relevance reason. A positive also needs disposition of exceptions and amendments that could defeat it. Consistent readings that change the answer produce uncertainty; contradictory applicable assertions without resolved priority remain a conflict.

2.6.4 Delivery and acceptance

The delivery design requires frozen questions and baselines, source and selected-term construction, clause interpretation, and a report that consumes the separate law, fact and financial results. The repaired jobs now consume those results, as the execution checks below show. Failed attempts are retained, and changed source or method hashes trigger reassessment. The next-phase record identifies unfinished work; it does not

itself perform the semantic repairs. Missing agreements and reviewers block affected conclusions while engineering continues.

The first acceptance exercise concerns a finite assisted service with a frozen scope. It requires useful supported positive and negative answers and no wrong accepted answer in that set; blanket abstention is insufficient. The ordinary reader, frozen candidate and automatic successor must answer a common versioned question when compared. Supplying independently reviewed clauses diagnoses execution; it cannot count as automatic extraction accuracy.

Independent first readings must be made without engine predictions, with disagreement resolved by an adjudicator or recorded reasoned agreement. The 1,467 issue-context records, 1,307 unique spans and 25 review forms are different counting units whose overlap must be reconciled. Exposed development counterexamples supply neither population error bounds nor independent acceptance.

2.6.5 What the executed repairs establish

The ordered repairs of 6–7 October 2026 connect source intake, contract construction, interpretation, law and financial calculations to the installed application. They correct demonstrated software errors, including a legal evaluator that ignored an additional necessary premise and a BASF bank request that inherited a UBS instrument classification. The complete interpretation of the BASF contract remains unfinished. This distinction matters: the application now consumes the intervening work, so the next construction and interpretation repairs can change its report.

The earlier execution passed 50 development tests but left 18 adversarial findings. Its ten BASF edits left 163 bracket branches and an unmatched opening bracket. Those receipts remain preserved as historical observations. The repaired service uses versioned products with exact parent identities. A changed selected contract invalidates its interpretation; separately admitted legal facts and financial scenarios replace their corresponding results. The installed command rejects corrupted products from each of the five upstream stages.

Three paired checks establish actual dataflow. Making a fully interpreted synthetic repayment clause unresolved changes Q1 from no to unknown. Changing a necessary synthetic legal premise from true to false changes Q4 from yes to no, without rebuilding the source inventory. In the Deutsche hypothetical write-down calculation, increasing the required loss from 50 to 60 with the same principal pool changes the note's allocated loss from 25 to 30. Installed and worktree reports agree exactly in these checks. They test software consumption and arithmetic, not an independent reading of a prospectus.

The legal evaluator now executes the supplied Boolean expression over exact source occurrences. Each fact records its instrument, jurisdiction, validity interval, knowledge interval and observation date. Missing facts remain unknown; contradictory active observations remain conflicts. Necessary false premises cannot disappear behind a fixed checklist. The clause evaluator likewise preserves repeated and multiline occurrences, executes supplied conditions and exceptions, and distinguishes compulsory common

conversion, possible common shares and common shares as the sole conversion outcome. Conditional repayment no longer supplies unconditional evidence of absence.

Source intake retains the same four BASF documents, 519 pages, 43,626 text units and 489 detected references. The supplement's date is recovered from its first page, and the annual report's publication date from page 296. Existing OCR supplies word geometry for four further retained documents; profile pages from Deutsche, BBVA and SEB preserve their native word boxes. Corrected OCR transcriptions remain separate from raw coordinates until their character mapping is reviewed. These inventories support inspection; they do not establish reading order or meaning.

Visual inspection of BASF's printed page 112 confirms that the long-coupon alternative lacks an outer closing bracket. Its marginal instruction and the next definition delimit the unselected branch. The repair excludes that branch under the final terms' annual, no-stub selection without inventing punctuation. The same margin says the reference-period definition applies to every Actual/Actual option. It therefore survives, while its inapplicable short- and long-coupon additions are removed. Treating an unchecked optional field as authority to delete that common definition would be wrong.

Further selected branches retain their contractual wording after removal of the selection delimiters. Every copied character and substituted value has a recorded origin. There are now 83 unresolved brackets and no unmatched delimiter in the retained candidate. The smaller count measures construction progress; it cannot prove completeness. Clause numbering, other fields, marginal attachments, incorporated agreements and the annual-report range disagreement still require work. The supplement really prints both the overall range 195–209 and the detailed notes range 209–290; neither may be silently changed.

The production law stage also executes 25 retained development scenarios with checked source quotations, and the financial stage executes the three named Deutsche, BBVA and SEB subcalculations. Under their declared hypothetical premises, the results include a Deutsche loss allocation of 25, a BBVA conversion price of EUR5.01 and three shares, and a SEB price of USD3 and three shares. Allocation now groups by an explicit legal holder identifier rather than a display name. These calculations do not establish actual exercise, complete settlement or the correctness of every translated legal premise. Corpus availability dates also do not establish when a statute or judgment applied.

For the incomplete BASF bundle, the installed report remains:

Question	**Recorded result**
Q1: contractual principal loss	Unknown
Q2: conversion and election rights	Unknown
Q3: completeness of all five declared substantive questions	Partial
Q4: dated legal applicability	Unknown
Q5: conditional financial calculation	Unsupported
Q6: preservation of bank obligations	Conditional

Q6 now uses a sourced senior-bond classification and an explicit mapping between the issue-date contract question and a hypothetical current purchase investigation. All 14

bank obligations are compared by their actual catalogue records, rather than their number alone. Client, entity, capacity and policy facts remain unresolved, and the report grants no transaction permission.

The broader deterministic regression run passed 983 tests. Subsequent focused checks include the source-specific BASF correction. Passing these tests and reusing verified receipts establish the recorded engineering behavior; the automatic grammar remains a bounded English implementation. The general construction representation supports text, sequences, choices, fields, references and dated overrides, but it has not replaced every BASF-specific operation or completed the other agreement sets.

Occurrence coverage, partial Boolean completion, temporal admission and exact holder allocation have conditional correctness arguments for the supplied representations. Their production implementations now have regression evidence. The arguments still require faithful translation of the source contract and complete dependencies; neither requirement follows from passing these tests.

Independent readers have not supplied an unexposed cohort, adjudicated labels or accepted sign-offs. P7 consequently records no independent evaluation, and P8 blocks legal release. The remaining program includes substantial engineering: complete and review the selected contracts, attach governing exceptions and indirect references, evaluate the same source through reviewed and automatic interpretation, and supply the contractual conventions needed for settlement. Actual dated law, event and client facts must then support the particular application. The new receipts, regression records and unresolved obligations are preserved under `docs/implementation/prospectus-repair-2026-10-06/`.

2.6.6 What the tool trials changed

The next investigation, on 7 October 2026, tested whether established tools could repair specific failures in the prospectus service. It retained the existing extraction and finite Boolean evaluator as comparators. Nine documents supplied 326 selected file pages and 3,047 retained text units. For 289 of those pages, the printed page label remains unresolved; a file index is therefore not presented as a verified printed citation. These documents had already informed development and cannot supply an independent acceptance sample.

The construction repair makes omissions visible. Every interval in the selected BASF body now has a recorded disposition, including unresolved material. The initial checker reported 1,018 unresolved intervals and 83 retained brackets. Complete accounting consequently establishes where the work stops, not a complete German contract. A shared definition can serve several branches: excluding one use no longer erases another. A field whose rendered value differs from its cited quotation also remains unresolved until that transformation is justified. Versioned runtime validation checks the admitted construction; a separate published JSON schema has not been added.

The next source inspection exposed a different failure: retaining every character does not preserve a sentence if the words appear in the wrong order. In the printed German terms on pages 120 and 121, the clearing-system sentence ends before the instruction beginning “Falls die Schuldverschreibungen in Form einer NGN” (if the notes take the form of an NGN). The bold opening word has a slightly higher bounding box; sorting fragments by their top edges moved it ahead of the preceding words. A similar error affected a coupon placeholder, and two Fiscal Agent address blocks interleaved their labels and addresses. Five occurrence-specific permutations, checked against the original pages, correct these orders while preserving all 1,578 body occurrences and their text. The Canadian-agent exclusion now includes its whole label and address field. That ordering repair left 1,008 unresolved intervals and still 83 brackets; the change reflects corrected ordering and branch boundaries, not additional legal adjudication. Other reading-order and clause-scope decisions remain subject to source review.

Reading the governing instructions then exposed a substantive selection error. On base-prospectus page 132, the Luxembourg-listed alternative contains website publication and a clearing-system substitute, the latter subject to the Luxembourg exchange’s rules. A separate alternative provides clearing-system notices for securities that are not listed on an exchange. Final-term page 7 checks both publication media, while page 13 selects the Luxembourg regulated market. Our constructor had selected both alternatives by matching their labels. It now excludes the unlisted alternative and retains the exchange-rule condition within the listed alternative. The error was in our interpretation of clause scope; installing a different text extractor would not have corrected that rule. These selections describe the issue configuration, not its observed current listing or any notice actually delivered.

The same review distinguishes the BASF SE clauses from the BASF Finance alternatives and resolves supported clearing, currency, agent and call choices. It preserves the guarantee condition within BASF SE’s successor-debtor clause and the change-of-control right, although other guarantee provisions and the specified-price holder put do not apply. Twelve scalar occurrences copy exact final-term values or substrings: for example, the annual date supplies “8. März” where the template already says “eines jeden Jahres”, and the spread supplies “0,300” before the template’s percent sign. Each of the 62 reviewed operations binds its source occurrence and premises; changed text, geometry, instrument identity or selections cause rejection.

All 1,578 body occurrences remain intact in the source record. That stage retained 19 brackets and 545 unresolved intervals. Fourteen of those brackets concern numbering or cross-references, four are spare call-table cells, and one requests both the calculation agent’s name and designated office. The final terms name NatWest Markets N.V. but do not supply that office; filling the whole field with the name would hide missing information. Governing margins, continuous-text review and the conflicting incorporation ranges also remain open. The source review and the passing software checks therefore support these particular repairs, while independent legal acceptance still requires a complete reviewed contract.

A further inspection of page 111 explains why counting brackets misses some construction errors. The different-rate table places its interest-date caption below the clos-

ing bracket in the rightmost cell. Our bracket-based exclusion removed the table but left that caption beside the surviving single-rate definition, repeating "jeweils ein Zinszahlungstag". The repaired rule binds the caption's two source lines to the table's governing margin and requires the table itself to have been excluded. It preserves the single-rate definition and the following first-payment sentence. On page 120, four empty cells belong to two unused call-table rows; the final terms supply one date range and one amount. Removing those cells leaves the supplied row and its notice provisions intact. The new candidate differs from the preceding candidate by exactly these five deletions. It retains 15 brackets and 537 unresolved intervals.

The remaining source questions have concrete consequences. A reference from §13(3) still points to judicial enforcement in §14(4); renumbering the surviving paragraphs consecutively would require changing that reference too. The retained instructions do not specify such renumbering. NatWest's address does appear in the final terms, but under its role as a lead manager, which does not establish the designated calculation-agent office. The original supplement also prints both the broad annual-report range 195–209 and the detailed Notes range 209–290. The annual report confirms the latter section's extent but cannot amend the supplement. These unresolved source decisions, all 522 margin and header occurrences, and the remaining prose review keep the construction partial despite the verified local repairs.

The optional Z3 adapter agrees with finite evaluation on the declared Boolean test corpus. The checks cover a small grammar with all its partial and conflicting assignments, and selected formula families across 8,190 complete assignments using one to twelve facts. This is not an exhaustive test of every formula over twelve facts. Larger examples exercise the installed interpretation path. An inconsistent premise set produces a conflict; missing premises and computational timeouts remain distinguishable forms of uncertainty. Typed reference checks also retain missing targets and unsupported cycles as unresolved. Translating German definitions, exceptions and priorities faithfully remains a separate task.

The Docling trial exposed a mismatch between normalized display text and exact legal quotations. The pinned layout model processed BASF file page 112 and Deutsche file page 52 on the CPU, with OCR and table recognition disabled. After earlier repairs to word order and list markers, the strict adapter still failed to map 14 of 53 designated critical source lines. Inspection on 8 October traced all five remaining failed text blocks to six deleted line-final hyphens. For example, joining "Insolvenz-" to the following "oder" changes a German compound construction rather than merely removing a discretionary line break. The package's text assembly removes these hyphens; our adapter then rejects the whole block. The earlier suggestion that these residual failures also reflected unexplained geometry or reading-order errors was unsupported.

The repaired adapter uses Docling's regions as structural proposals and retains the original words, punctuation and offsets as the quotation authority. It stores the normalized candidate separately and records each proposed hyphen deletion. Other character changes and ambiguous source ownership are rejected. On the same two pages and model configuration, all 53 designated lines now have exact source maps, while the old adapter still misses 14. This controlled comparison supports the integration repair; it

does not measure population extraction accuracy. The existing extractor remains the source of the quotations.

The governing margin requires a separate decision. On BASF file page 112, the margin states that the adjacent reference-period definition applies to all Actual/Actual options. A supplied relation checked against the original page passes the new evaluator; wrong scope and omitted margin text fail. The layout model supplies no such legal relation, and its empty proposal fails the same check. This separates preserved text from an interpreted governing relationship. The source review belongs to the implementer, so independent legal adjudication and expansion beyond these development pages remain open.

The annotation exchange now works through a running INCEpTION 38.0 server. The trial used a Unicode example and four retained BASF lines joined into a development excerpt, with provisional labels. Browser edits to one label in each example persisted after reloading. Twelve exported annotation files—the original and subsequent records for an administrator and two synthetic readers on each example—matched their expected contents exactly. The edited labels changed in the administrator's records while the two reader records retained their originals. The checks preserved source editions, quotations, the supplementary Unicode character and combining mark, repeated text, discontinuous evidence groups and semantic relations.

The bridge converts between Unicode code-point offsets and UTF-16 code units. The displayed spans must omit whitespace at their boundaries, so separate source offsets retain the complete quotations, including trailing spaces and line breaks. The importer checks the relation between both sets of offsets and rejects altered text or broken group links. Editing also requires the server's current storage timestamp. Ordinary XMI export removes that internal field, so the tested workflow uses the browser to update existing annotations and REST to import initial records and retrieve exports. It does not replace existing records with a newly encoded XMI file.

The exported project contained no recommenders. Opening integrated curation left both curation records without evidence annotations despite the completed synthetic reader inputs, confirming the disabled automatic merge in these trials. Each synthetic reader session could open its own annotations; four requests for the other reader's or administrator's annotations received explicit access denials. These checks establish preservation of supplied annotations, the tested curation behavior and access restrictions within those sessions. The guest display names establish no human identity or independence. The server trials and failed attempts are retained under `docs/implementation/prospectus-inception/`.

A further trial now starts with exact source text and its edition identity but no evidence annotations. A separate draft schema lets the reader select text, assign a group and label, and connect groups. The importer derives quotations and offsets from those actual selections and checks the immutable source; the reader need not type provenance fields. Eight server exports preserve newly created spans, deletion, a boundary replacement by deletion and reselection, a discontinuous group, and a semantic relation. Reloading confirms persistence, and the other reader's record remains empty. Earlier failures came from our selectors and from treating network inactivity as completion of the editor's

queued save handlers. Waiting for the actual queue repairs that race.

This trial uses two ordinary password accounts with guest entry disabled. Wrong passwords are rejected, and four attempts to inspect peer or administrator annotations are denied. These are synthetic accounts testing authentication and access control. Actual first readings still require separately identified people, recorded prior exposure and a distinct adjudicator. The source maps, raw exports, failed attempts and attribution analysis are retained under `docs/implementation/prospectus-integration-repair/`.

Financial testing supports a conditional fixed-coupon calculation. Nine independently derived year fractions agree with QuantLib 1.38 within 10^{-12} years, and six holiday-adjusted dates agree exactly. For example, consider a semiannual ICMA coupon accruing from 31 January to 31 August 2024, with supplied reference periods 31 August 2023–29 February 2024 and 29 February–31 August 2024. The first 29 days fall in a 182-day reference period; the remaining 184 days fill the second reference period. Its year fraction is therefore

$$\frac{29}{2 \cdot 182} + \frac{184}{2 \cdot 184} = \frac{29}{364} + \frac{1}{2} = \frac{211}{364}.$$

This derivation corrected an erroneous expected value in the original test fixture. The failed check is retained. The calculator also checks explicit calendar coverage, entitlement, event order and rounding after aggregation by legal holder. The tests supply hypothetical conventions; the complete issuer-specific calendar, entitlement and event premises have not been admitted. Tax, foreign exchange and full settlement remain outside this calculation.

ACTUS and FINOS CDM remain different decisions. The inspected ACTUS service requires a particular numerical core, and the official Python library delegates event calculation to a backend. Public repository checks did not provide the required core bytes; that access result cannot establish a numerical defect. The service build and its copied older README also name different core versions. An actual comparison must first identify the engine, its applicable licence and explicit contract terms. CDM supplies a settlement representation, but no receiving system or version profile has been selected for this service. Its official Python documentation also identifies Java-native functions that raise `NotImplementedError` without Python replacements. This is a documented limit of that binding, with its relevance determined by the functions required for the chosen exchange. Neither deferred trial supports rejection of the whole package.

The controller records observed inputs and their absence, immutable snapshots, tool identities and the installed sidecar's file hashes. Generated mutation sequences compare cached results with fresh computation; installed-package checks verify that changes to construction, law and finance reach the report. Broad source hashes remain necessary because native process reads are not confined by the Python input recorder. Each phase refreshes the next plan from its exact parent results. A completed phase can preserve a rejected candidate or an unresolved premise; it does not imply acceptance.

The legal-source reconciliation retains 59 records, but current applicability and actual exercise still need evidence for the particular question. The next delivery step is a source-complete selected contract and reviewed interpretation, followed by a frozen cohort with independent first readings and adjudication. Actual-event questions ad-

ditionally require dated event and client records. The executable program, failed attempts, comparisons and remaining work are preserved under `docs/implementation/prospectus-adoption/`.

2.6.7 Integration repairs and remaining evidence

The next repair program traced accepted package outputs through the application: did the extracted text still identify its source, did the selected clause apply to this instrument, and did the calculation actually use its terms? Several earlier failures arose between these steps. A valid layout file had no consumer in the assessment service. A quotation could come from an excluded alternative. A coupon calculation could use a supplied rate without deriving it from the selected contract. These were defects in our integration, even when the underlying parser, solver, or financial library worked as configured.

The closure program therefore separates five decisions: execution, artifact validity, the engineering criterion, actual consumer integration, and external acceptance. It records them in immutable phase receipts under `docs/implementation/prospectus-integration-closure/`. A failed but well-formed attempt remains evidence and cannot be retried with unchanged inputs. A current assessment revalidates source derivatives and admissions; an explicitly labelled frozen replay does not silently acquire the guarantees of the new profile.

What changed in the service. The source graph now validates exact OCR page sequences, unique unit identities, document hashes, raw intervals, and recorded applicability times. A clause premise must refer to the selected occurrence and question. Construction authority has a separate checked admission: its source must have been consumed by the selection or override. Dated-law premises retain their own source, jurisdiction and time checks. These checks establish consistency with supplied authority records; the legal authority itself still requires review. Supported references resolve through raw and assembled intervals. Repeated copies need an explicit occurrence use; deleted, replaced, or ambiguous targets cannot silently resolve to the first matching quotation.

The construction and interpretation products use the same bounded predicate interface. Finite reasoning remains the compatibility path; an explicit Z3 request reaches construction, clause, and law stages. Conflicts, witnesses, unsat cores, limits, and typed unknown reasons survive into the report. A large Z3 request now returns UNKNOWN/`DOMAIN_LIMIT` before starting the solver rather than raising an unclassified exception. This is a resource boundary, not a claim that the legal question is false.

The package adapters now have real consumers. A retained layout projection is bound to the source graph and reviewed governing edges; losing a governed unit makes the affected question unresolved. Authored XMI spans are decoded with the actual type system, mapped through raw and assembled occurrences, and translated into typed clauses. Authored condition and exception relations must correspond exactly to the

clause links consumed by the interpreter. A reviewed cross-document link binds both exports and their group endpoints; named, relative, external and amendment references have a separate reviewed admission. This supplies an assisted route while automatic discovery remains incomplete. Bounded acyclic priorities have an independently enumerated three-rule control; cycles remain unresolved. Corrected OCR readings preserve raw quotations, and inserted or uncertain text cannot acquire borrowed source geometry. The reading-to-evaluation mapper uses the same correction rules. A signed administrator record binds the submitted export and assignment; its signature cannot establish the reader's qualification or independence.

The BASF construction exposed a more specific loss of evidence. Removing a branch delimiter changed a line without removing its surviving provision, but the adapter discarded that line's character map. A final-term amount inserted into the selected conditions likewise lost its usable source occurrence. The repair carries copied intervals into interpretation and binds an inserted term to its exact final-term quotation. A deleted template cannot supply that quotation. Older substitutions identified only by page-text offsets remain unmapped until their exact source occurrences are reviewed.

A second defect kept a recognizer's unresolved proposal active even after its reference had been reviewed. The new admission explicitly names the proposal it replaces, for the selected occurrence and question; other occurrences and questions retain their own obligations. On the inspected German BASF pages, this connects §5(10)(b) to the make-whole call in §5(5), §13(3) to the ownership-proof provision in §14(4), and §1(3) to the United States definition at the end of §6(2). The construction and raw numbering are unchanged. These three connections establish reference consumption under the supplied review, while principal-loss and conversion questions remain unknown and contract completeness remains partial. The 537 unresolved source intervals still require individual meaning and authority decisions.

The OCR path records a versioned CPU profile, language-model hashes, page images, word geometry, and blank or unread dispositions. A missing Poppler executable, worker, or language file is a typed extraction failure. The current default is project-local and configurable; the historical training environment is no longer an implicit production dependency. Native subprocess reads are not claimed to be confined. Their receipts therefore cannot be reused as fully current unless the read set is observed or the invocation is run under a tested restriction.

For finance, selected terms now include principal, calculation amount, rate, currency, period dates, maturity, day-count, frequency and rounding. A scenario cannot replace them. The distinction between rounding per holder and per calculation amount matters: two EUR100.10 units at five percent yield EUR10.01 if rounded after aggregation, but EUR10.02 if each unit's EUR5.005 is rounded to EUR5.01 first. The repaired kernel represents both supplied conventions explicitly, including the published fixed coupon when applicable.

The retained Compass final terms provide a real development example: XS3232968985 specifies EUR1,000 calculation amounts, a EUR35 annual fixed coupon, 15 January 2026 commencement and 15 January 2035 maturity. For a hypothetical entitled EUR100,000 holding, the service calculates EUR3,500 for the first annual coupon and generates the

nine anniversary dates in the final terms. The dates agree with QuantLib 1.38; the annual fraction is exactly one. Altered rate, date and issue-principal values fail their source bindings. The result assumes timely presentation, no early redemption or default, and the declared limited future calendar. The registered-note record-date provision is not imported into this bearer-note example. Incorporated annual-report material and full-contract completeness remain unresolved, and Compass's related family is now excluded from any claim of unseen evaluation.

Evidence from the executed phases. The combined successor and selected financial regression passes 681 checks. Standalone XMI and ZIP exports carry an authored exception through the service. Separate English and German scan controls preserve their designated text and a blank page. On retained BASF page 112, all three supplied shared-definition dependencies resolve; removing one required line makes them unresolved. Eight day-count examples and four calendar adjustments agree with independent fractions and dates and with QuantLib under identical conventions.

A fresh, non-editable wheel runs nine cases through the public assessment command, constructing all five products inside the isolated installation. They include Compass, the three reviewed BASF references, an AST-selected definition, a cross-document exception and a thirteen-fact Z3 construction and law case. Relocation and offline restoration preserve the answers. Docling's CPU packages and model restore twice from a verified binary snapshot and preserve fourteen mapped regions on the BASF page. Restored Java, INCEpTION, Chromium and Playwright also run twice: the browser authenticates and the local server exports a synthetic project. These results establish the recorded host profiles. Native libraries remain unconfined and a fresh operating-system image has not been tested.

Failed attempts exposed additional local defects. The codec and finance exercises initially imported pytest-only fixtures; dependency-free fixtures repaired that error. The first Compass run repeatedly rebuilt the full source index; a local index removed that unnecessary work. A controller comparison loaded different source versions in parent and child processes; the corrected experiment snapshots the same source. The first restored authoring request used a human title where the API required a slug. The corrected request works. An installed retry also rejected the older Compass input after Poppler's executable changed. The complete source-graph comparison found identical text and geometry, with only the executable identity changed. Rebuilding the request from those sources and repeating the exact coupon, schedule and adverse controls passed. The installation harness now verifies the chosen successful attempt's saved files instead of hardcoding the older request. Each failed attempt is retained alongside its repair.

Boundary	Current evidence	What it does not establish
Source and solver	Exact occurrence, applicability and routing controls	complete source meaning or unrestricted solver capacity
Layout, OCR, and XMI	BASF shared definition; two scan languages; authored exception	automatic governing-relation accuracy or broad language/document coverage
Financial terms	Conditional Compass coupon; exact rounding and schedule controls	actual entitlement, full callable/tax/FX settlement
Runtime and replay	Fresh wheel relocation; layout and live authoring restores	operating-system portability or native-read confinement
Review and attribution	Signed-record mutations and executed prediction checks	genuine qualified readers, population accuracy, or legal release

What the literature contributes. The literature reviewed in Section 2.6 remains relevant to these repairs. ContractNLI's self-contained supporting spans and separate treatment of distant exceptions motivate retaining context and refusing to infer absence from an unmentioned sentence (Koreeda and Manning, 2021, Sections 2–5 and Appendix A.1). Typed spans, events, and argument relations in Holzenberger and Van Durme motivate the explicit actor, effect, condition, and date fields (Holzenberger and Durme, 2023, Sections 3–5 and Appendices B, D and E). LegalRuleML and structured argumentation separate a source, an interpretation, and the authority selecting between conflicting alternatives (Athan et al., 2014, Sections 3–4) and (Prakken, 2010, Sections 3 and 6). Catala's compiler argument supports preserving a formal execution boundary, but cannot certify that a local prospectus translation is faithful (Merigoux et al., 2021, Sections 4.1, 4.4–4.5 and 6.1). Date-arithmetic work explains why calendar rounding and operation order must be selected terms rather than library defaults (Monat et al., 2024, Sections 2, 3.2 and 4.3). These sources justify the architecture and its tests; none supplies a complete prospectus engine or a transferable accuracy result.

Root-cause disposition. The evidence supports local integration and usage defects as the cause of the reproduced failures. It also leaves substantial work in our program: broader reference discovery, actual multilingual and table cases, wider schedule and entitlement semantics, and native-read restrictions. These are not all missing external inputs. BASF's field meanings, incorporation and calculation-agent authority additionally require source decisions, while independent acceptance requires actual readers and an adjudicator. ACTUS and FINOS CDM still lack their declared runtime or receiving-profile prerequisites and were not trialled here. None of these results establishes whole-package unsuitability.

The remaining work therefore begins with named source cases and discriminating comparisons. Complete the BASF source and meaning table without guessing away its unresolved intervals. Extend the financial comparison to source-reviewed stubs, transfers and entitlement events. Test native filesystem or network restrictions with an

undeclared-read counterexample. Obtain genuine first readings on a frozen, family-separated cohort and preserve adjudication before exposing predictions. The phase controller retains these obligations even when all implemented checks are current. The service remains a conditional assisted system; the full integration program and independent legal acceptance are incomplete.

2.7 Marketing rules require exceptions and judgment

The October 2023 distributor circular examines additional returns and incentives, redemption restrictions and marketing of fund-related services. Its treatment of guaranteed-return arrangements depends on the arrangement and the relevant legal classification. A potential structured-product classification should produce a supported assessment task; a language model should not resolve it from a slogan alone (SFC, 2023b, paras. 5–11).

The circular's discussion of gifts expressly distinguishes discounts of fees or charges. Losing that exception makes an apparently conservative implementation incorrectly broad. Its redemption discussion distinguishes administrative procedures or cut-off times from reducing a fund's dealing frequency. A daily dealing fund offered with weekly redemption presents a different issue from a different daily processing cut-off. These are excellent paired test cases because they differ in the fact that determines the treatment (SFC, 2023b, paras. 9–14).

The advertising discussion addresses misleading impressions and comparisons with deposits. Keyword detection can nominate material for review, but it cannot establish the whole classification: meaning depends on the presentation and the service described. The system should retain the advertisement, the relevant source passages and the reviewer's conclusion. That is a useful result even when there is no defensible fully automated predicate (SFC, 2023b, paras. 15–17).

2.8 The source packet and the question it supports

The running example begins with an identifiable public document: SFC circular 23EC46, issued on 24 October 2023, concerning additional returns and other services or arrangements offered when marketing SFC-authorised funds (SFC, 2023b). The retained experiment selected its paragraph-10 gift restriction. The circular as a whole discusses more than that restriction. A reader needs to know both what was selected and what remains outside the control before a result from the control can be interpreted.

The experiment preserved the official source response, a readable text derivative and anchors connecting the selected rule to its passages. On 22 September 2026, a fresh retrieval matched the retained response. That check allows another reviewer to reproduce exactly what was read at that time. It does not establish the complete current legal position or resolve the effective versions of incorporated sources. The technical

record uses a content identifier for this reproducibility purpose; the identifier does not establish authority, completeness or legal effect.

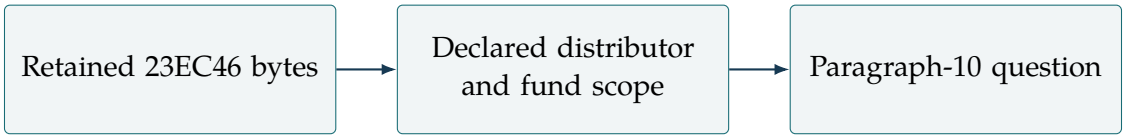


Figure 2.10: A source packet and an activity jointly define the question.

The selected assessment concerns one proposed incentive component, offered by a distributor in promoting a fund within the declared profile. It is a question about a specific restriction, not a whole-offer compliance verdict. A positive result means that the selected prohibition trigger is established. A negative result means only that this trigger was not established on the supplied, resolved facts. It supplies no general permission to market the offer.

The retained packet explicitly leaves the applicable Code paragraph and cited FAQ unresolved. This matters twice. Those sources may settle the meaning or scope of a term; they may also establish that the selected profile is narrower than the underlying requirement. Their absence is therefore a release-blocking dependency, not a bibliographic inconvenience. The example is useful precisely because it continues to expose that absence after its executable tests pass.

The book uses this historical, selected packet to teach implementation and assurance. New commercial offers, cashback definitions and mixed-component scenarios introduced below are synthetic investigations. They are not reported SFC rulings. The unresolved references are not silently filled using a language model's recollection of a Code provision. Doing so would convert a visible gap into an untraceable premise.

2.9 Reading the surrounding circular before isolating the rule

A translation that begins by copying the most convenient sentence can miss why that sentence is there. The circular's opening establishes its distribution context. The next passages describe observed practices, including additional features, standing investment arrangements and marketing. Those observations help identify the activity being discussed, but they are not automatically individual executable prohibitions. A source inventory should retain their function without manufacturing a rule for every paragraph.

The discussion then turns to additional returns and their possible regulatory consequences. Some questions concern the structure of an arrangement and public promotion. Others concern impressions created about the underlying fund. The gift discussion appears within that broader context. Consequently, a finding that one separate classification is absent does not clear the gift issue. The gift control is not conditional on first finding that every other regulatory concern applies.

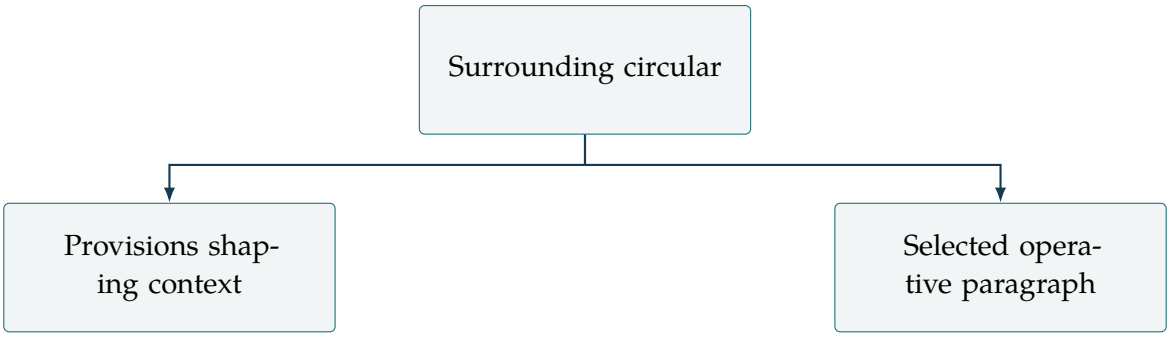


Figure 2.11: Isolating a formula must not isolate it from its qualifications.

Later passages address redemption arrangements, administrative cutoffs, advertising and the description of services. Some concerns require event or workflow evidence; others require qualitative assessment of communication. Those mechanisms differ from the paragraph-10 test of classified facts. An adequate implementation of the whole circular would need to preserve those differences. It cannot simply accumulate a larger expression around the gift predicate and call the circular complete.

The existing disposition inventory has 18 numbered paragraphs and four footnotes. Its assignments are agent-authored and unreviewed. They include context, separate controls, qualitative review and unresolved dependencies. Paragraph 10 is the only numbered passage with an executable control in this exercise. A future independent reviewer should challenge both the assignments and the completeness of the inventory. Merely approving the final formula would leave the earlier decision about what to implement unexamined.

This provides an immediate requirement for the ensemble. One path must inspect the source independently of the candidates and ask what obligations, exceptions and dependencies need a disposition. Another path may develop the selected rule. Comparing those paths creates a place for an omitted requirement to appear. If both start from the same already-filtered list of paragraphs, the apparent redundancy begins too late.

2.10 Naming the six inputs precisely

Six questions determine the worked marketing decision. The first two establish whether the rule applies; the remaining four assess the incentive itself. First, is the assessed actor a distributor in the declared business scope? Second, does the activity concern promotion of an SFC-authorised fund within that scope? Both answers are reviewed classifications. The prototype does not infer them from the actor's name or the presence of the word "fund" in a document.

The first body input records whether the proposed component is classified as a gift. The second and third record a promotional link to a specific product and to a particular product type. A link to funds from a particular fund house requires an explicit classification where the relevant example is used; the engine does not assume that every mention of a fund house creates such a link. The fourth body input records whether

the entire assessed component falls within the fee-or-charge-discount exception.

The word "entire" in that final definition is an engineering expression of the proposed component-level reading. It prevents an assessed voucher from inheriting the classification of a separate fee waiver. It does not prove that the decomposition is legally correct. A reviewer must establish the relevant unit of assessment from the source and context. The definition and the review issue must travel together until that question is resolved.

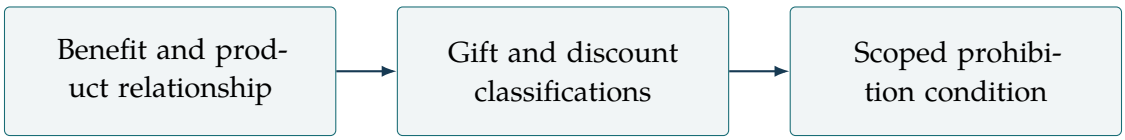


Figure 2.12: The six inputs carry separate interpretive commitments.

A payment can be linked to the type of fund even when it is not linked to one named fund. Treating those two routes as one input would conceal the omission detected by the product-type case.

For a compact mathematical account, let A denote established actor scope and S the fund/activity scope. Let G denote the gift classification, P the specific-product link, T the product-type link and D the discount exception. For now each answer is known to be true or false, the two values called Boolean in programming. Write B for the condition that triggers the selected restriction. The symbols $\wedge$, $\vee$ and $\neg$ mean "and," "or" and "not," respectively. The condition is

$$B = G \wedge (P \vee T) \wedge \neg D. \tag{2.4}$$

It asks three things together: is the component a gift, does either promotional route hold, and is the exception absent? The two alternatives occur inside one parenthesized condition. Replacing that disjunction by a test of P alone changes the rule for offers linked only to a product category.

Scope is assessed separately. When both scope inputs are true, the body answers the selected question. When a scope input is known false, the limited profile returns `OUT_OF_SCOPE`. When scope is unresolved, it returns `UNKNOWN`, even if a body condition might otherwise be false. The distinction prevents an unresolved applicability question from being concealed inside an apparently definitive in-scope result.

The prototype checks for contradictions in every fact on which the requested rule depends before it calculates an answer. For example, two unresolved records that classify the same benefit differently produce `CONFLICT`, even if a later finding about scope would make that classification irrelevant. This is a deliberately conservative software policy, not wording taken from the circular. Chapter 3 explains why the policy must be fixed before comparing implementations.

2.11 The named cases are an argument about the rule

The 22 named cases are most useful when read as a sequence of challenges to an interpretation. They do not merely fill rows in a test fixture. Each changes a fact or evidentiary condition that a plausible implementation might mishandle. The baseline case is a component classified as a gift, linked to one specific fund, with no fee-discount exception, and known in-scope actor and activity.

2.11.1 The alternative promotional route

Case g01 establishes the baseline prohibition trigger. Under equation (2.4), G and P are true and D is false, so the body is true. This is a necessary example but a weak test on its own. An incorrect expression $G \wedge P$ would also return true, even though it omits both an exception and an alternative route.

Case g02 changes the promotional facts: no individual fund is named, but the component is linked to a fund category. Now P is false and T is true. The disjunction remains true. A candidate that omitted the product-type route would return false, making this case a concrete witness to the omission. The source's inclusion of that route supplies the interpretive objection; the different program outputs make its consequence inspectable.

Case g15 uses the fund-house example through a supplied reviewed product-type classification. Its purpose is not to write a text classifier for fund-house mentions. It checks that the chosen formal vocabulary has a place to represent the classification when review establishes it. This distinction prevents a source example from being silently generalized into a universal inference from an entity name.

Case g09 establishes a specific-product link while leaving the product-type link unknown. The known route is sufficient for the disjunction. Unknown information about a second route does not erase the first. Case g14 reverses the situation: the specific-product route is false and the remaining route is unknown. Here the unresolved fact can change the outcome, so the correct result under the declared semantics is unknown.

Taken together, these cases explain the meaning of "either" in an executable rule with partial evidence. The system must neither require every route to be known true nor convert every missing route to false. This is a semantic distinction, not a preference about how verbose the explanation should be.

2.11.2 The exception and the object it qualifies

Case g03 changes the baseline component to one classified as solely a fee or charge discount. The exception settles the selected body condition as false. The result says that this particular gift restriction is not triggered under the supplied classification. It does not resolve advertising, authorization or any other requirement associated with the offer.

Case g04 returns to a voucher while another component of the same offer is a fee waiver.

The assessed component remains the voucher, so the waiver does not set its D value to true. This case challenges an implementation that stores a single offer-level discount flag and reuses it for every component. If that flag is the only available data, the adapter has insufficient information to apply the proposed component-level rule. It should request a finer assessment rather than derive a convenient value.

Case g07 leaves the discount classification unresolved for a component otherwise meeting the trigger. If D is false, the prohibition is triggered; if D is true, the exception applies. Both possibilities remain consistent with the supplied evidence. Unknown is therefore the informative answer. A system that turns this unknown into false would remove the exception by default; one that turns it into true would invent the exception.

Case g16 provides known discount evidence while leaving the gift and promotional classifications unknown. Within known scope, a true D makes $\neg D$ false, which settles this conjunction as false. The case demonstrates that every missing fact need not prevent every narrow conclusion. Its safety depends on the specific conclusion being reported. It would be unsafe to broaden the result into a complete clearance of the marketing arrangement.

The contrast between g04 and g16 is particularly useful for reviewer training. An established exception can settle a condition, but only for the object to which it applies. A precise value attached to the wrong subject is not better evidence than an unknown value attached to the right one.

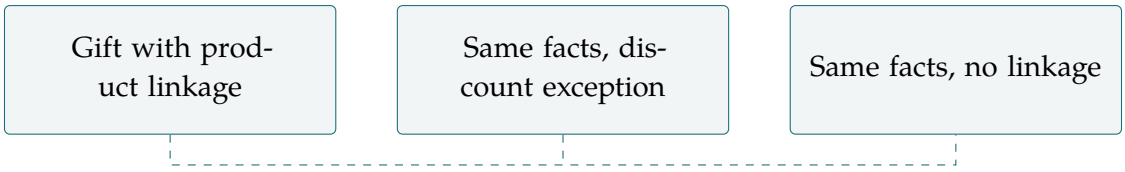


Figure 2.13: Paired cases isolate the condition that changes the answer.

2.11.3 Classification is not supplied by the formula

Case g08 concerns additional returns whose gift classification has not been settled. The circular's qualified treatment of such returns motivates an assessment; it does not establish G for every arrangement. The proposed candidate must retain unknown when the remaining facts would make that classification decisive. Replacing the missing judgment by an unconditional "additional return implies gift" would create a new interpretive rule.

Cases g06 and g22 establish that no gift is offered. In g06 the discount classification is also missing. The false gift predicate settles the body as false under the declared Boolean semantics. Case g22 makes the point with a known product link: a promotional link alone does not establish that a gift exists. These cases catch a candidate that assumed G from the context instead of requiring evidence for it.

Case g05 establishes that neither product-link route holds. This also makes the body false, while leaving other regulatory questions outside the result. It does not establish

that every incentive unrelated to an identified product is permissible under all applicable requirements. The selected condition is only one element of a broader assessment.

Case g17 separates regulatory classifications. A separate assessment finds no structured product, but the component is a non-discount gift with a relevant product link. The gift trigger remains true. The case challenges a workflow that exits early after resolving one concern and never evaluates another. Correct integration therefore requires an inventory of outstanding controls, not a single reassuring flag named `regulatory_check_passed`.

2.11.4 Applicability precedes an in-scope answer

Cases g10 and g11 establish that, respectively, the actor and the product/activity are outside the selected profile. The response is out of scope. The underlying Code requirement may have a broader reach; the example cannot answer that broader question. Treating out of scope as false would erase the distinction between an in-scope condition that is not triggered and a question the selected control does not purport to decide.

Case g12 leaves the fund/activity scope unknown. The system cannot establish whether the profile applies. Case g13 leaves actor scope unknown while establishing no gift. A Boolean programmer might be tempted to use the false gift fact to return false immediately. The declared profile instead reports unknown scope. This preserves the fact that the system has not established the context in which its body answer would be meaningful.

This is a design choice that an independent implementation must reproduce. Another language could define a different scope-combination policy, but it would then implement a different specification. Differential testing only has a clear target after the policy has been made explicit. A reviewer must also decide how an unknown scope result affects the transaction workflow.

2.11.5 Contradiction, expiry and later knowledge

Case g18 supplies conflicting gift evidence. The result is conflict, not unknown and not the value favored by the latest-arriving record. Unknown means the evidence does not determine a value. Conflict means admissible records assert incompatible values. These require different remediation: obtaining missing evidence in the first case, and resolving or superseding contradictory evidence in the second.

Case g19 combines an out-of-scope actor with conflicting discount evidence. The current static-closure conflict policy returns conflict before scope evaluation. This is deliberately conservative. The example must teach the policy honestly because a reviewer might otherwise expect out of scope and mistake the difference for a Java bug. The policy is recorded as engineering semantics, subject to separate operational review.

Case g20 makes the gift evidence stale at assessment time. The rule does not continue treating an expired classification as current. Case g21 places the recording time after the

historical knowledge cutoff. A historical assessment cannot use that later knowledge. Both become unknown when the missing gift classification remains decisive, but their reason codes and repair actions differ. Refreshing an expired record does not justify rewriting what was known at an earlier point in time.

All 22 cases therefore contribute a distinct aspect of the specified behavior. Some concern the wording of the selected restriction. Others concern evidence, scope and operational conventions needed to execute the reading. The case packet should label those sources of expected behavior separately. Otherwise an implementation convention can acquire the apparent authority of a legal sentence simply because both are stored in the same software record.

2.12 What the 729 combinations cover

Each of the six abstract inputs takes one of three values: true, false or unknown. The number of assignments is therefore

$$3^6 = 729. \tag{2.5}$$

This is a statement about a finite abstraction. It does not count legal interpretations, marketing documents, evidence histories or real client offers. The enumeration is exhaustive for that abstraction because it visits every combination of the six declared values.

The separately written calculation of expected answers handles scope first. For the four body facts it replaces each unknown by both possible Boolean values and evaluates the formula under every completion. If every completion returns true, the calculation returns true; if every completion returns false, it returns false; otherwise it returns unknown. This calculation is written without reusing the candidate rule's representation or evaluator. This makes it useful for detecting a defect in how the expression was encoded or evaluated.

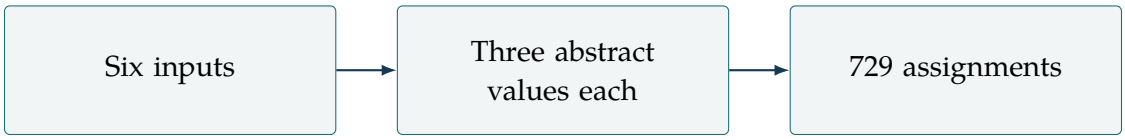


Figure 2.14: The enumeration covers a finite abstract input space.

For this particular body, each input occurs once and with a single polarity. The completion calculation agrees with the current three-valued operations. That agreement should not be generalized to every expression. For example, a formula containing a variable and its negation can be true under every Boolean completion while a compositional three-valued evaluator returns unknown. The next chapter derives that difference. The expected-answer calculation is written separately from the candidate's expression tree, but its semantic scope remains specific.

Conflict, stale evidence and later-recorded evidence are not extra members of the three-value enumeration. Their normalization and precedence are exercised by named cases and other application checks. The count 729 must not be used to imply exhaustive testing of those mechanisms. Nor does it exhaust numerical inputs, dates, arbitrary rule graphs or the Java runtime's entire implementation. A result summary should always name the finite object that was exhausted.

The comparison checked full semantic responses and relevant traces, while respecting each backend's own identity. It would be wrong to require the Python and Java engine identifiers to be identical: those identifiers describe different implementations. It would also be wrong to ignore all metadata in a comparison, because source, evidence and rule identities are part of an explainable decision. Chapter 7 specifies the distinction between semantic agreement and implementation-specific provenance.

2.13 Mutations ask whether the tests can notice an error

Successful cases can be uninformative if they would also pass a wrong program. Mutation testing addresses this by deliberately changing the rule and asking whether the checks detect the change. A mutation is not a plausible legal interpretation merely because it can be compiled. It is an instrument for testing the sensitivity of the evidence.

The first retained mutation removed the fee-discount exception. Cases g03, g07, g16 and g19 detected the change. Some of these detections concern the logical exception, while g19 also exposes the effect of changing the static dependency closure. Removing an expression can remove a referenced fact from conflict analysis. A mutation report should therefore explain why a case detects a change rather than treating every detection as the same kind of evidence.

The second mutation omitted the product-type route. Cases g02, g14 and g15 detected it. Their relationship to the source is direct: they isolate the route that the mutation removed, including a case where that route is unresolved. A baseline case with a named fund could not provide this evidence.

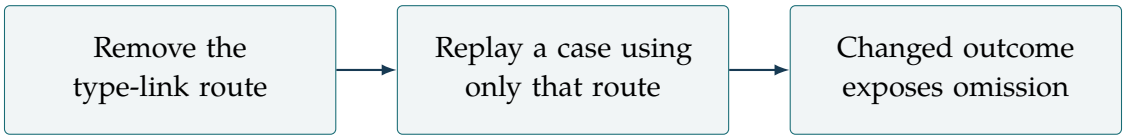


Figure 2.15: A meaningful mutation changes a consequential interpretation.

The third mutation assumed that a gift existed without requiring its evidence. Cases g06, g08, g18, g20, g21 and g22 detected the resulting change. The set includes known absence, missing classification, conflict, expiry and later knowledge. This shows why evidence-handling cases are necessary even for a small logical condition. A wrong classification default can alter outcomes across several superficially different scenarios.

The fourth mutation ignored the selected actor/product scope. Cases g10 through g13 detected it. The expected difference includes both out-of-scope and unknown results. If the test harness compared only a final allow/deny bit after flattening all other statuses, it could conceal these errors.

All four mutations were compiled into Java before comparison. The result thus concerns executable altered candidates, not a textual inspection of a proposed patch. Still, detecting four mutations does not establish detection of every possible implementation or interpretation defect. Mutation operators are chosen by people or software and can themselves omit an important failure mode. The ensemble evaluation must maintain a documented fault model and extend it when new failures are discovered.

2.14 A new discrepancy: what counts as a discount?

The existing tests assume that a reviewer can supply the discount classification. To investigate the missing interpretation step, consider a synthetic offer in which a customer pays a fee and later receives cash back. Two candidate definitions might be proposed. One restricts the exception to a reduction of the fee initially charged. Another includes a documented refund of a fee that was actually paid. The source packet alone has not established that both readings are legally tenable. Their purpose here is to expose a question whose answer affects implementation.

Fix the actor, activity and promotional link, and assume for the comparison that the component otherwise meets the gift condition. Let the fee paid be HK$100 and the later rebate also HK$100. Under the first proposed definition, the later transfer is outside the defined exception. Under the second, it may fall within the exception if the required refund relationship is established. Those conditions must be derived from authority or review; the example does not introduce them as new SFC requirements.

The next action should be a targeted investigation of the applicable definition, Code and FAQ, not a request for the same model to vote again on the word “discount.” A source that explicitly includes the relevant refund treatment would support refinement of the second candidate. A source that excludes it would defeat that candidate in the applicable scope. A source discussing another product, actor or period may be informative without settling the question. The retrieval result needs an applicability assessment before it can resolve the discrepancy.

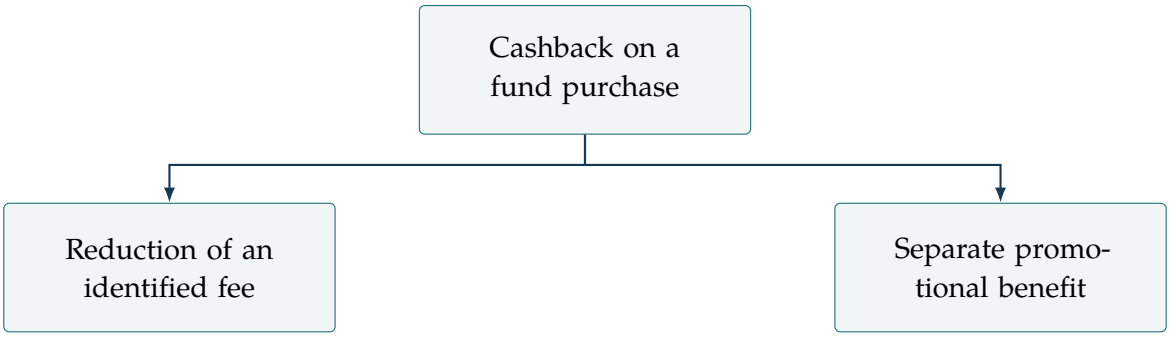


Figure 2.16: A cash payment's label does not establish the discount exception.

Suppose a client pays a fee of HK$100 and receives HK$100 later. Amount equality alone does not establish that the payment reduces that fee. The contractual relationship, purpose and applicable definition still matter. Those are evidence questions that another truth-table run cannot settle.

Now alter the cash amount to HK$150. If a candidate definition treats only a refund of an actual HK$100 fee as falling within the exception, the extra HK$50 raises a further classification question. It does not follow that the system can automatically split the transfer into a lawful and unlawful part. The legal unit of assessment and treatment of mixed benefits must first be settled. An arithmetic decomposition is easy; its legal significance is the question under investigation.

A second variation changes the evidence. The marketing material describes the transfer as a fee refund, but no record establishes that a fee was paid. The candidate's definition may be clear while its decisive factual premise remains unknown. This discrepancy belongs to fact classification or evidence availability, not necessarily to interpretation. The resolution controller should request the relevant evidence or return unresolved, rather than changing the definition to make the missing record unnecessary.

A third variation combines a refund with a separate voucher. A whole-offer exception would remove the need to assess the voucher independently. The proposed component-level interpretation would retain it. This is a different branch in the hypothesis space from the timing of a refund. Preserving separate issue identifiers prevents a source that resolves the timing question from being incorrectly treated as resolving the mixed-offer question as well.

These variations show how an ensemble can make review more precise. It should produce a small set of materially different definitions, the source reasons for each, and the smallest scenarios on which they diverge. The reviewer then decides an identifiable question. A long narrative saying that an offer appears reasonable would provide less actionable evidence.

2.15 Finding the source does not settle its date or its meaning

The missing Code and FAQ in the early gift example were concrete obstacles to further investigation. Later work retained the September 2023 and January 2026 Code editions and located paragraph 3.11 in each. The paragraph states the gift restriction and its fee-or-charge discount exception (SFC, 2023a, 2026a, para. 3.11). Selecting an edition for a test date is useful progress. It does not decide every question about when a provision applies, and the source collection still has unresolved statutory definitions.

The FAQ illustrates the dating problem. Its page is associated with 30 September 2010, but the retained text says that answers to Questions 2 and 4 were updated on 15 October 2024. That does not establish that Question 1 changed. It does establish that the page cannot be treated as an independently preserved copy of everything a reviewer would have read in 2010. A historical use must retain that uncertainty (SFC, 2010).

Question 1 also sharpens the business question. It asks about gifts used to promote a product category. Its answer brings gifts linked to certain types of funds or funds from particular fund houses within the requirement (SFC, 2010, Question 1, answer and section reference). That helps establish why a product-type link matters. It does not remove the discount exception from the Code. For a proposed fee rebate, the reviewer must still determine whether the benefit falls within that exception. "This requirement applies" and "this benefit is prohibited" answer different questions.

A useful source search therefore changes the next question. Once the relevant passage is available, the work can turn from locating it to establishing its date, its subject and the proposition it supports. The earlier blocked draft remains part of the history; it is not silently rewritten to suggest that these materials had already been reviewed.

2.15.1 A settlement deadline can conceal several different duties

The later development work also examined the SFC circular on the shortening of US securities settlement to one business day after trading. A cash calculation alone cannot express its requirements. The circular asks licensed corporations to consider liquidity, staffing and training, operational processes, technology and communication with affected clients. It separately addresses management companies of authorized funds (SFC, 2024c). A rule about staff availability and a rule about funding may both be necessary, while neither covers the communication duty.

Several generated proposals covered a narrower part of that subject. Comparing them with the source exposed incomplete coverage rather than a choice between mutually exclusive readings of the whole circular. This is the reason to retain separate controls and ask how they fit together. Chapter 7 returns to that problem after explaining the rule language. The settlement exercise does not replace the SPI teaching example, and its development results do not establish complete implementation of either circular.

2.16 A complete dry run of the proposed workflow

The following dry run uses the early gift candidate and its original unresolved dependencies to explain the workflow. It is an illustration, not a further executed investigation. Start by creating an assessment whose context contains the retained circular revision, declared distributor/fund scope, selected effective interval and an inventory of dependencies. The historical engineering example uses a synthetic September 2026 interval; the workflow must not infer a regulatory commencement date from that setting.

The source-review path inventories the circular and flags the incorporated Code and FAQ as unresolved. The interpretation path proposes the component-level gift rule. A second interpretation path omits the product-type route. A definition-focused path asks whether cashback is a discount and whether a combined offer changes the assessed unit. At this stage the system has a candidate set and several issues, not a winner.

The comparator identifies a decision difference between the complete promotional condition and the candidate omitting the product-type route. It constructs a case with a gift linked only to a fund category and no exception. The source-grounding check points to the relevant paragraph. The candidate author can propose a repair adding the missing route. The system stores the repaired candidate as a new revision, reruns the discrepancy case and the retained regression cases, and preserves the original candidate and its rejection reason.

The cashback definition issue follows a different route. Retrieval attempts to obtain the incorporated sources with the right identity and version. If the sources are unavailable, another language-model explanation cannot close the issue. The controller records the failed acquisition and the remaining dependency. It may retry an authorized retrieval route within its budget, or stop automatic work because no permitted action can supply the missing authority. The issue remains unresolved either way.

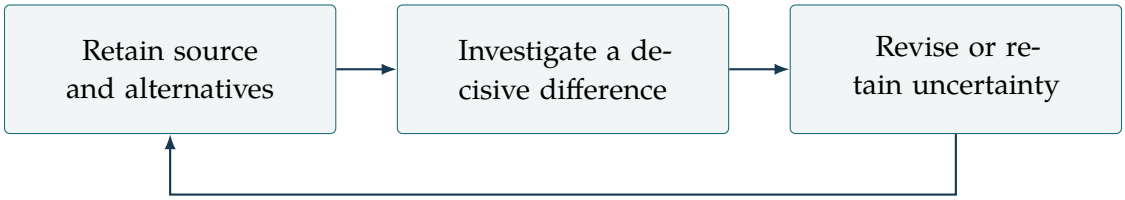


Figure 2.17: An investigation can finish with a useful unresolved result.

The mixed-offer issue can produce a useful witness even before legal resolution. The comparison holds the voucher facts fixed and toggles the presence of a separate fee waiver. If a candidate's voucher decision changes solely because of the other component's exception, the report explains the dependence. A reviewer must then confirm the unit to which the exception applies. The witness does not grant the controller authority to make that legal decision itself.

The accepted formal candidate can be compiled and checked in parallel with this research. Candidate compilation is valuable because it exposes implementation defects early. It does not change the issue status. The report may legitimately say that the candi-

date passed all specified execution checks and remains unavailable for release because source interpretation is unresolved. Keeping those two observations together is more informative than suppressing either.

When automatic work reaches its finite limit, the controller freezes a report. It identifies the source revision, surviving definitions, rejected encodings, discriminating scenarios, completed actions, failed actions, unresolved dependencies and the exact effect of each unresolved issue on the control. There is no final vote to select a legally preferred answer. The state is ready for targeted human review or blocked on a named dependency, according to the reason for stopping.

A designated reviewer can later resolve an issue using newly obtained authority. That creates a new context revision. Dependent candidate scores and comparisons are invalidated where appropriate, and the formal rule and tests are updated. The earlier report remains a faithful account of what was unresolved before the new material arrived. This is essential for reproducibility: a current reviewer must be able to see why the earlier system abstained.

2.17 Who decides what the correct answer should be?

The expected answers in this demonstration were prepared by the same agent that constructed the candidate rule. Saving those answers before building the rule makes later changes visible, but it does not supply an independent judgment. If both records share a mistaken reading, agreement between them can conceal the error. The reviewer needs to know who prepared the expected answer and how that person reached it.

Independent adjudication should begin from the retained source packet and declared task, before exposing reviewers to the proposed formal expression. One reviewer identifies requirements, definitions and difficult cases. Another can independently develop or challenge the interpretation. Disagreements are recorded and, where possible, resolved with source reasons. A third role may adjudicate material disagreement. Independence here concerns information and responsibility as well as the identity of a person or model.



Figure 2.18: Freezing an oracle and making it independent are different steps.

After that source-based stage, reviewers can inspect the candidate rules and their discriminating cases. This order reduces anchoring on the generated answer. It does not make reviewers infallible. The reference packet must allow an unresolved category, later correction and competing defensible readings. If a reviewer cannot determine the treatment of a cash rebate without the referenced FAQ, the correct reference outcome is not a guessed Boolean chosen to make benchmark scoring convenient.

The original 22 cases should remain frozen as historical engineering evidence. New adjudicated cases belong in a separately versioned packet with its own authors, source context and review decisions. A changed expected outcome should explain whether the earlier interpretation was rejected, the scope changed, new evidence became available or the evaluation semantics were corrected. Those causes imply different repairs and different consequences for existing releases.

Following a source into an executable rule can expose definitions that still need judgment. An investigation can preserve those questions and show which decisions they affect. A formal check must then state precisely which relation it establishes. Without that precision, a solver's success can be mistaken for the missing judgment about meaning.

Chapter 3

What can be proved, and under which assumptions

A green test result can be reassuring and still answer the wrong question. A reviewer may ask whether the program preserves an adopted rule for every allowed case, while the legal team is asking whether that adopted rule is a justified reading of the circular. Those questions meet at the same example but they do not share a proof.

3.1 The vocabulary before the proof chain

Before asking whether a translation is correct, we need to identify what it translates. In the gift example, the circular supplies the wording, a reviewer proposes which benefits and exceptions it covers, and a program computes that proposal for a particular offer. A *controlled natural-language specification* (CNL) expresses a rule using a limited set of sentences with declared computational meanings. LegalMath's shared frontend records a proposed reading, its source passages, assumptions and required facts in a *LegalRuleModel*. This is a typed model: each quantity has a declared kind, such as a Boolean condition, an amount of money or a date. It also states how missing evidence affects the answer.

RuleIR, short for rule intermediate representation, is LegalMath's small executable rule language. Its Python and Java implementations calculate individual values and conditions, retain missing or conflicting evidence, and support a bounded set of duties. *Catala* is another programming language for legal rules, with its own compiler. A translator can express the same supported RuleIR rule in Catala; a second route translates a richer shared model directly, including calculations over lists of records. A record groups fields about one item, such as a holding's amount and classification. These routes fix the proposed meaning before choosing how to execute it.

Stipula names a different kind of formal object. It models a contract as parties, resources, actions and state transitions, so its natural question is how a permitted action changes a contract over time. The Java Modeling Language (JML) states properties of Java programs, and KeY is a verifier for those programs and properties. The studied Stipula-to-Java translation uses them to check a model of a contract. LegalMath does not currently compile Stipula contracts. Chapter 4 introduces the language families in detail, including Catala in Section 4.4 and Stipula in Section 4.6; Chapter 8 records the executed RuleIR and Catala routes.

This vocabulary fixes the order of the claims that follow. First the source and interpreta-

tion are reviewed; then a typed shared model is translated; then each target is executed and checked against its declared reference. A proof of the last step cannot establish the first.

3.2 Giving a correctness claim a precise target

The second-circular exercise leaves us with two questions that sound similar but have different evidentiary requirements. Does the program compute the proposed gift rule correctly? Does the proposed gift rule express a justified reading of the circular? The first question becomes mathematical once the rule and its inputs have precise meanings. The second includes deciding which meanings the words, context and authority support. A proof of the first cannot silently include a proof of the second.

To see why, suppose a programmer writes a formula that checks gifts promoting a specific product and omits the product-type route. A proof assistant can verify that an implementation returns exactly that formula's value for every Boolean assignment. The proof is correct about its target. The target is incomplete relative to the proposed source interpretation. Making the proof longer, using a more powerful solver or verifying another implementation will not add the missing route. The source-to-formula mapping has to be challenged directly.

This is not a special weakness of legal language. Formal verification always depends on the proposition that was formalized. The virtue of a formal method is that it makes the conditional claim exact and inspectable. It does not remove the need to establish that the proposition is the one the institution intended to rely on. Catala's separation between legal authoring and compiler semantics is useful precisely because it keeps these responsibilities distinct (Merigoux et al., 2021, core language and compilation sections).

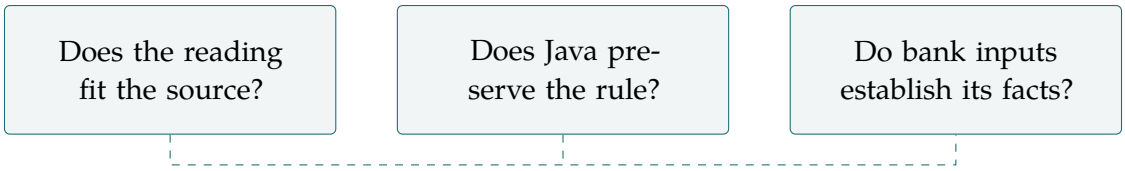


Figure 3.1: Correctness has several distinct targets.

Suppose the reviewer has accepted the rule that the relevant gift restriction includes either a specific-product link or a product-type link. We want the same case to receive the same decision, with the same required explanation, whether the rule is read in restricted English or executed as software. This is the agreement the equation below describes.

Write E for the source documents and their versions, and S for the accepted specification in controlled natural language (CNL). Here controlled means that each allowed sentence has a fixed computational meaning. Write F for its RuleIR representation, the small rule language used by the prototype, and J for the generated Java program.

The subscript on eval says which of these three representations is being evaluated; $\text{eval}(F, x)$, for example, means "apply rule F to case x."

The assumptions A fix the definitions, permitted facts and treatment of time. The set D_A contains the cases allowed by those assumptions. Finally, π selects the parts of an answer that must agree: the decision and the specified explanation and evidence behaviour. It leaves out an implementation's own name and version, which should differ between Python and Java. With those meanings in place, the proposed preservation obligation is

$$\forall x \in D_A, \qquad \pi\big(\text{eval}_{\text{CNL}}(S, x)\big) = \pi\big(\text{eval}_{\text{IR}}(F, x)\big) = \pi\big(\text{eval}_{J}(J, x)\big). \tag{3.1}$$

Read the initial $\forall x \in D_A$ as "for every permitted case." The two equal signs require agreement across all three forms of the rule. In the gift example, a voucher linked only to a product type must not disappear from the Java decision merely because an intermediate translation kept only the specific-product route. Chapter 7 makes the result comparison concrete. Equation (3.1) is a proposed target for the supported language, not a theorem already proved about the entire application.

The Catala comparison uses the same idea with the shared model as its source of execution. On the common scalar profile, a Catala result can be compared with the RuleIR/Python and Java results for the same cases. On a richer model, the Catala result is evidence that the native target executed that model, while a RuleIR capability failure is evidence that the RuleIR target does not accept it. Those observations cannot be folded into the equality above, because the targets do not claim the same input language or incomplete-evidence policy.

The claim that S is justified by E remains a separate interpretation claim. It can be supported by source anchors, explicit alternatives, reviewed definitions, adversarial cases and an attributable judgment. When some part of E has itself been translated into a formal language, a theorem can relate those formal objects. The choice that made the formal source representation faithful still needs justification. Repeating that translation boundary does not eliminate it.

The later source checks make this separation practical. Follow the gift exception through three questions. First, do we have the relevant passage and the edition that applies to this decision? Second, does the proposed reading preserve the exception and attach it to the right benefit? Third, does Java execute that proposed reading? A retained quotation answers part of the first question. A comparison of the source with a proposed rule addresses the second. Execution against the rule addresses the third.

These checks can disagree without contradiction. A model may label a passage as supporting a rule while another examination finds a missing qualification. The label is a recorded judgment that can be challenged. Equation (3.1) can still hold for the wrongly chosen rule: agreement among its representations preserves the mistake just as faithfully as it preserves an exception. The source comparison therefore adds a way to find interpretive defects; it does not extend the equality into a proof of English meaning.

3.2.1 The computational preservation objective

The preservation chain in equation (3.1) puts interpretation, RuleIR and Java in one equality, but it does not yet say which parts of the result the middle link must preserve. That narrower contract sits inside three assurance questions. The first is interpretive: does the adopted specification represent the relevant sources for the chosen business scope? The second is computational: does the implementation compute that specification? The third is evidential: do the facts and event records represent the relevant circumstances? A complete change package needs evidence for all three, and a test of the middle link cannot answer either of the other two.

To make the computational question precise, let R_v be an approved version of the formal rules. Let $[\![R_v]\!](x)$ denote the reference meaning of those rules on input x, and let $C_v(x)$ be the result of an implementation or compiled adapter. Define D_v to include the declared input types, admissible values, explicit unknowns and relevant state. The target is

$$\forall x \in D_v : \quad C_v(x) = [\![R_v]\!](x). \tag{3.2}$$

Equality here includes the named decision and required obligations represented by the specification. If explanations are part of the verified output, their relationship must be stated too. The meaning of the output cannot shrink from a set of duties to one Boolean while retaining the same assurance claim.

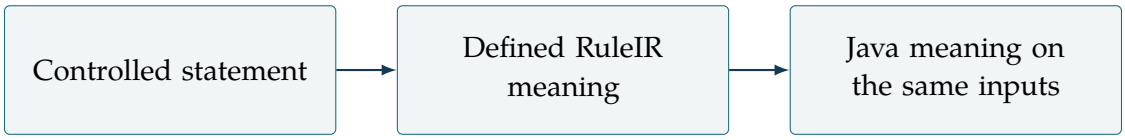


Figure 3.2: Preservation compares specified meanings across representations.

Equation (3.2) is a proposed verification objective, not a result already established by LegalMath. For a finite input space it may be checked by enumeration; for a restricted expression language it may be amenable to an SMT encoding or a proof by induction over expressions. For a production adapter, the mapping from bank data into x and the implementation of calendar and currency operations also belong to the argument. A test suite samples the equation and provides useful defect evidence, but it does not generally prove its universal quantifier.

The interpretive question does not become a theorem simply by naming a formal target. Source text, legal context and justified interpretations determine R_v. Compliance review, source coverage, competing readings and independently constructed cases address that step. Likewise, a theorem cannot establish that a recorded balance or signature is authentic. These limits are operationally useful: each review task is directed to the person and evidence that can answer it.

3.3 Quantifiers change which entity carries the obligation

The phrase "for each" introduces a choice that a Boolean expression can easily hide. To see it without assuming knowledge of predicate logic, consider a synthetic requirement: every relevant benefit in an offer must satisfy a specified condition. Let $B(o)$ be the finite set of benefit components in offer o, and let $P(b)$ mean that component b has the property under investigation. The universal reading is

$$\forall b \in B(o),\ P(b). \tag{3.3}$$

It requires every component to satisfy the condition. An existential reading,

$$\exists b \in B(o),\ P(b), \tag{3.4}$$

requires only one. These expressions are not two styles of writing the same rule. For an offer with two components, one satisfying P and one not, the universal reading is false and the existential reading true.

The distinction becomes more consequential when an exception is involved. Suppose $G(b)$ identifies a gift and $D(b)$ an excepted discount. A component-level prohibition trigger might be

$$\exists b \in B(o),\ G(b) \land \neg D(b). \tag{3.5}$$

An alternative whole-offer encoding might be

$$(\exists b \in B(o),\ G(b)) \land \neg\,(\exists b \in B(o),\ D(b))\,. \tag{3.6}$$

The first asks whether any component is a non-excepted gift. The second suppresses the trigger whenever any component is a discount. A voucher combined with a fee waiver distinguishes them immediately: the second formula can let the waiver cancel the voucher's effect, while the first continues to assess the voucher. These are proposed logical readings for analysis, not a legal determination about mixed offers under 23EC46.

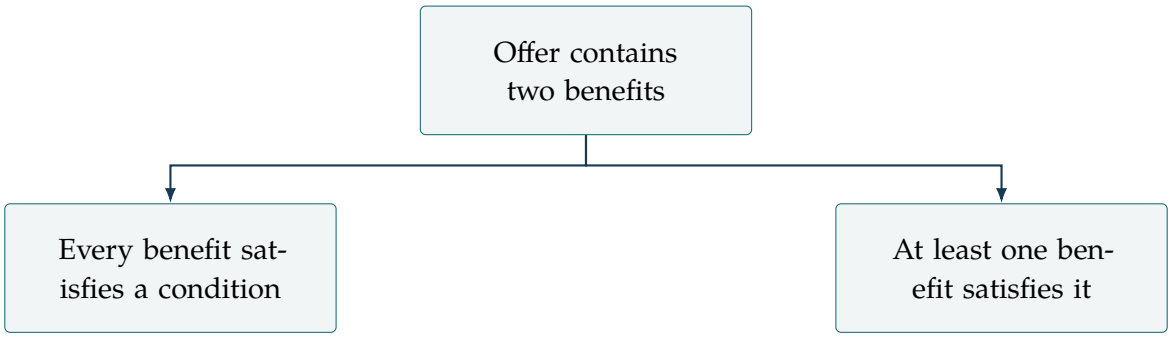


Figure 3.3: Changing the quantifier changes which offers satisfy the rule.

For two benefits, let the condition hold for the fee discount and fail for the voucher.

An existential statement is true because one benefit qualifies. A universal statement is false because the voucher does not. No compiler error is needed for those readings to disagree.

A source interpretation must decide what the variable ranges over before a proof about either formula is useful. Is the subject a benefit, a contract, a campaign, a transaction or a client relationship? Are components defined by economic substance, contractual terms or an operational product record? The formal notation exposes this question because $B(o)$ cannot remain undefined in an implementation-ready specification.

The current RuleIR fragment has scalar facts rather than unrestricted quantified collections. A reviewed adapter can create one snapshot per component and a host orchestration can aggregate results under an explicit rule. That aggregation is another specification obligation. It must define how true, false, unknown, conflict and out-of-scope component results combine, and how the host knows the component list is complete.

An empty list deserves attention. In classical logic, a universal statement over an empty set is true and an existential statement is false. That convention is mathematically coherent, but a missing component inventory must not be treated as an established empty offer. The adapter needs an evidence status for collection completeness. Otherwise a data omission can pass a universal check by vacuity. The same pattern occurs when no argument extensions are returned or no test cases are selected: an empty computation can look successful unless existence and coverage are checked separately.

Unknown classification also affects aggregation. If one component is known to trigger a prohibition and another is unknown, the selected logical aggregation may already establish a trigger. Yet the unknown component may remain relevant to another obligation or to the completeness of the offer review. A top-level known result should not erase the record of missing facts. The distinction between missing and blocking inputs in the runtime supports this more informative report.

3.4 Necessary conditions, sufficient conditions and permissions

Another frequent translation error changes the direction of implication. Consider a synthetic statement that a process may use a streamlined route only if consent has been obtained. Let S mean use of the streamlined route and C mean established consent. The necessary-condition reading is $S \Rightarrow C$: using the route requires consent. It does not follow that $C \Rightarrow S$ or that consent alone authorizes the route. Other eligibility conditions may still apply.

A truth-table example makes the difference concrete. With consent true and an unmet financial condition, the necessary-condition statement is not violated by refusing the streamlined route. An implementation that treats consent as sufficient would proceed. Both programs may mention the same two words, but their decisions represent different logical relationships.

Likewise, an exception to one prohibition is not a general permission. If the selected condition R is false, we know only that this particular trigger is not satisfied under the accepted facts and semantics. We do not infer that every other prohibition is false or every positive obligation fulfilled. The host must compose independently applicable controls without upgrading the absence of one trigger into universal clearance.

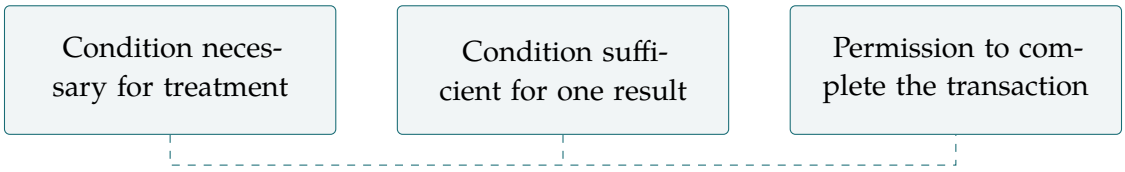


Figure 3.4: Logical direction determines what a satisfied condition establishes.

In a closed, complete formal theory, one can define permission in terms of the absence of a prohibition if that is the intended logic. Legal systems and practical source packets often do not provide the completeness needed for that move. Explicit permission, lack of evidence of prohibition and a bank's operational choice should therefore have different representations. The normative-language literature examined in Chapter 4 provides several treatments; RuleIR's narrow condition results do not define a complete logic of duties and permissions.

The words "unless" and "except" also require a structural decision. A simple propositional rewriting may express an exception as a negated condition, while a default system represents a defeasible conclusion that can be overridden. These can agree on a selected complete fact pattern and differ under missing evidence, conflicting exceptions or additional rules. The translator must identify which behavior the approved specification requires.

For the circular example, using $\neg D$ inside the selected body is an explicit modeling decision tied to the reviewed factual definition of discount. If D is unknown, the program should not act as though the exception is certainly absent. The three-valued semantics makes that uncertainty visible. A closed-world database query that treats a missing discount row as false would encode a different assumption, which requires separate evidence and approval.

3.5 A proof obligation should expose its assumptions

A useful proof statement includes the source-independent mathematical property, the accepted semantic definitions and the domain assumptions. For example: for every well-typed Boolean expression in a specified grammar and every normalized fact assignment without conflict, the reference and target evaluators produce the same three-valued result. This statement is precise enough to prove by structural induction. It does not include the claim that a particular English paragraph was mapped to the right expression.

The interpretation assumption can be represented separately as a reviewed proposition:

under source packet S, profile P and definitions D, expression e represents the selected requirement. A preservation proof then establishes that target program $J(e)$ behaves like e. Composing these steps is useful, but the first premise remains a source and review judgment unless a restricted controlled language and its translation semantics are themselves the authoritative input.

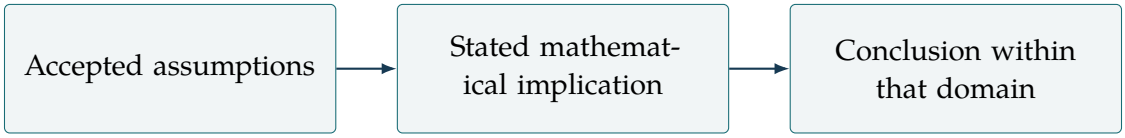


Figure 3.5: A proof exposes the assumptions under which its conclusion follows.

A proof assistant will faithfully carry an unjustified premise into a theorem. If we assume that every cashback is a discount, it may prove the corresponding exception applies. The theorem is conditional on that assumption. It is not a discovery that the assumption is legally correct. The proof report must display such premises in language the reviewer can understand.

The same principle applies to abstraction. A model may replace a complex offer with six Boolean inputs. The proof can establish behavior for every assignment of those inputs. To apply it to the real offer, the adapter must establish that the six inputs faithfully represent the relevant facts and that the omitted features cannot change the selected requirement within the declared scope. That relationship is an abstraction argument, not an automatic consequence of exhaustive truth-table testing.

The workbench should therefore attach an assumption register to every formal result. Each assumption names its provenance, operational meaning and review status. A proof that depends on an unresolved definition is retained as a valid conditional engineering result, while release remains blocked. This preserves the value of formal work without allowing its precision to conceal an uncertain starting point.

3.5.1 Why written agreement needs an executable meaning

Consider the instruction to obtain written agreement before applying the streamlined approach. A checklist can remind an officer to obtain it. A database can store a scanned agreement. Neither tells a program what to conclude if the agreement covers bonds while the proposed transaction is in a different category, or if the client withdrew yesterday. A formal language supplies a precise way to state the relevant conditions and the changes that events produce. Its purpose here is to preserve an agreed interpretation closely enough that people can inspect it and computers can execute and check it.

The word “formal” does not mean that the language resolves legal judgment. It means that a well-formed statement has a specified computational meaning. We can then distinguish disagreement about that meaning from a programming mistake in carrying it out. The original circular, the adopted interpretation and the formal statement must remain connected.

3.5.2 From a sentence to a calculation with a defined meaning

A programming language needs three ingredients. Its *syntax* says which statements are allowed. Its *types* say which kinds of value can be combined. Its *semantics* says what each allowed statement does. For example, `portfolio >= HKD(40000000)` has comparison syntax, compares two money values, and means an inclusive monetary comparison. Comparing that portfolio with a consent date is a type error. Using `>` instead of `>=` is well-typed but changes the meaning at the boundary. A type checker catches the first error; an independently chosen boundary example catches the second.

For the SPI financial condition, a readable formal statement is:

Listing 3.1: Illustrative controlled syntax for Annex 1 paragraph 3.1. This is a teaching notation, not Catala or Stipula source.

```
financial_condition(client, date) :=
    portfolio(client, date) >= HKD(40000000)
    OR net_assets_ex_home(client, date) >= HKD(80000000)
```

Here `client` and `date` are arguments: every occurrence refers to the same person and valuation date. `portfolio` and `net_assets_ex_home` name defined inputs. The source, paragraphs 3.2–3.4, determines the ownership and net-asset definitions; the host bank must provide values calculated under the adopted mapping. The notation makes no assumption that the portfolio is simply assets held at this bank.

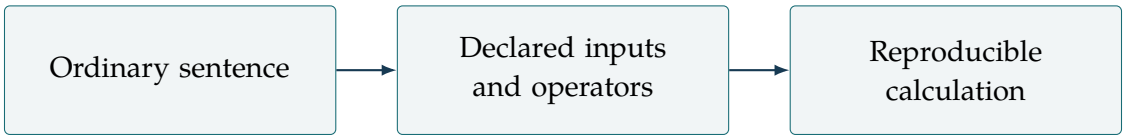


Figure 3.6: An executable meaning requires decisions about inputs and operations.

The compiler stores the statement as a tree. The root is `OR`; its two children are inclusive comparisons; each comparison has an input and a monetary constant. This tree is called an *abstract syntax tree*: it retains the operations and their arguments without depending on the layout of the original sentence. The stored expression in Section F.7 of the technical companion is a serialized version of this tree: a text record preserving its operations and arguments. An intermediate representation, or IR, adds stable identifiers, source locations, types and interpretation records to that executable structure.

Evaluation works from the leaves towards the root. For a portfolio of HK$35 million and net assets of HK$90 million, the comparisons give false and true; their alternative gives true. Missing portfolio evidence produces an unresolved comparison. An established net-asset route can still make the alternative true. If neither route is established and one remains unresolved, the answer remains unresolved. This is why an ordinary Java `boolean`, which has only two values, is insufficient as the complete result type.

The financial predicate answers one question. It does not produce a duty or an authorization to execute an order. The larger model must combine qualification, the selected

product category, consent, exposure and retained requirements. Keeping these named subquestions separate is useful both to the reviewer and to the engineer diagnosing a rejected request.

3.6 Expressions, types and meanings

A formal language is a collection of permitted expressions with rules for deciding what they mean. It need not look like a conventional programming language. A decision table, a restricted English sentence or a state-transition diagram can have formal semantics if each permitted construct has a precise interpretation. An unrestricted paragraph usually has more interpretive freedom than such a language allows.

For the gift example we need statements that can be true or false, references to facts, and the words “not,” “all” and “any.” A fixed true or false value is called a literal. A reference asks for a named fact, such as whether this benefit is a gift. Combining smaller statements makes a larger expression.

The following grammar is a compact catalogue of the allowed forms. The symbol e means an expression, a is a declared fact name, and $e_1, \ldots, e_k$ are k smaller expressions, with at least one required. Read $::=$ as “may take one of these forms” and each vertical bar as “or.”

$$e ::= \mathsf{true} \mid \mathsf{false} \mid \mathsf{fact}(a) \mid \mathsf{not}(e) \mid \mathsf{all}(e_1, \ldots, e_k) \mid \mathsf{any}(e_1, \ldots, e_k), \quad k \geq 1. \tag{3.7}$$

Here a is the identifier of a declared Boolean fact. The grammar specifies which trees are valid expressions; it does not decide whether the fact denotes a legally appropriate classification. That meaning belongs to the declaration and its source-linked interpretation.

Types prevent several avoidable errors. A Boolean condition is different from a monetary amount. A date is different from a number of days. Money denominated in HK cents is different from a foreign-currency amount before conversion. The current RuleIR scalar types are Boolean, integer, HKD money and date. A comparison accepts compatible ordered values. Addition preserves a numeric type. No implicit conversion turns a missing Boolean into zero or a dollar amount into cents.

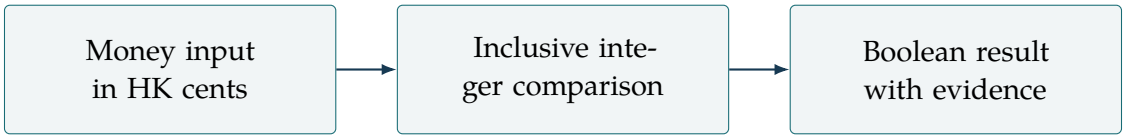


Figure 3.7: Types give an expression a defined interpretation.

Typing is valuable because it can reject an entire class of malformed specifications before any client case is evaluated. It is also limited. Two variables can both have the Boolean type while referring to different subjects or definitions. A type checker cannot infer that an offer-level discount flag was wrongly substituted for a component-level clas-

sification unless the type and declaration system explicitly represent that distinction. The proposed extension should therefore retain subject, scope and definition identities alongside the scalar type.

An expression's semantics is its evaluation rule. For known Boolean inputs, `all` is true exactly when every operand is true, and `any` is true when at least one operand is true. The gift body is a tree whose root is an `all`, with children for the gift fact, the two-route disjunction and the negated exception. The tree makes parentheses explicit. A source sentence can be attached to the root and narrower evidence anchors to its children.

The chosen language is deliberately finite and acyclic. A reference cycle such as rule A depending on B while B depends on A is rejected in the current profile. This avoids an undefined recursive evaluation. Richer legal formalisms can assign meaning to recursive or mutually defeating rules, but doing so requires a specified semantics. A compiler must not guess that an unsupported recursive construct means the same thing as the current acyclic profile.

3.6.1 A precise example: money, alternatives and unknown values

Let P denote the client's portfolio amount under the adopted definition, and let N denote net assets excluding the primary residence. Both are valued at the relevant date and expressed in HK dollars under an explicit conversion policy. For complete, valid inputs, the financial condition is

$$F(P, N) = (P \geq 40{,}000{,}000) \ \vee \ (N \geq 80{,}000{,}000). \tag{3.8}$$

The source of the constants, inclusive comparisons and alternative is Annex 1, paragraph 3.1. Equation (3.8) formalizes that subcondition; its inputs incorporate the separate definitions in paragraphs 3.2–3.4 (SFC and HKMA, 2023b). RuleIR stores monetary amounts as integer HK cents encoded as decimal strings. Conversions must take place before the comparison, with the exchange-rate source and valuation instant recorded. Floating-point rounding must not silently move a boundary.

Bank evidence is often incomplete. Define the comparison with a missing amount to return U, and let T, F and U mean established, refuted and unresolved for this condition. For the alternative in Equation (3.8), use the following truth table:

$\vee$	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

(3.9)

This table follows the intended treatment of incomplete evidence: a known sufficient route establishes the alternative, both refuted routes refute it, and otherwise missing information can matter. With unknown P and $N = 90{,}000{,}000$, the second comparison is true, so $\mathsf{U} \vee \mathsf{T} = \mathsf{T}$. With unknown P and $N = 70{,}000{,}000$, the result is $\mathsf{U} \vee \mathsf{F} = \mathsf{U}$. Figure 3.9 gives a flowchart view.

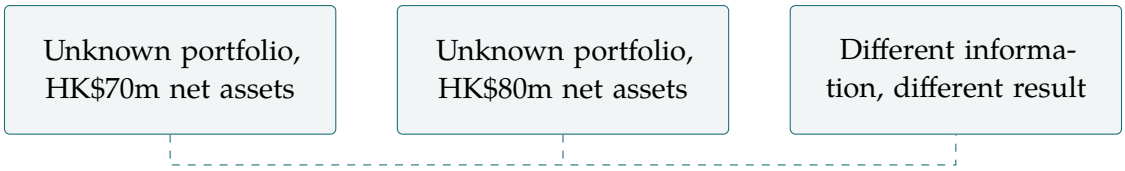


Figure 3.8: A sufficient known route can settle an alternative despite missing information.

The first case stays unknown because the missing portfolio could change the answer. The second establishes the financial condition through the inclusive net-asset route. Neither case treats missing portfolio evidence as zero.

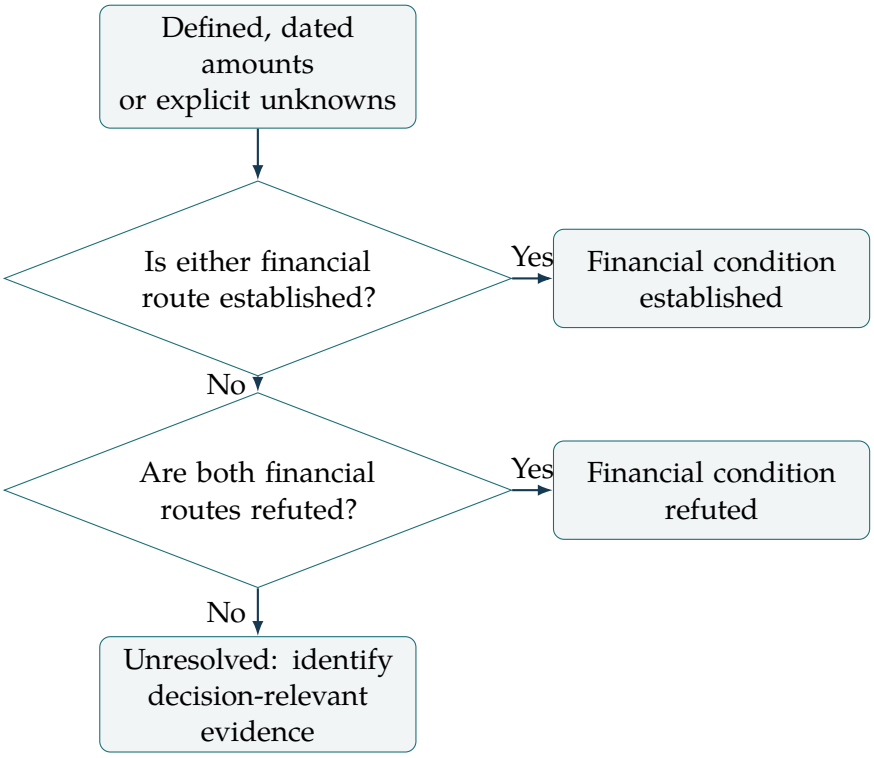


Figure 3.9: Proposed review flow for the financial subcondition, after resolving input conflicts. Passing this test alone does not establish SPI status.

The other Boolean operations need equally explicit tables. Under the proposed conservative three-valued semantics, conjunction is false if an operand is false, true if both are true, and otherwise unresolved; negation swaps true and false and preserves unknown. This semantics does not discover every conclusion implied by correlations among unknown facts. For example, a general expression $A \vee \neg A$ remains unresolved when A is unresolved under these tables. A more powerful completion-based analysis may establish additional conclusions, but it must be a specified refinement with its own evidence. No accidental change of logic should occur when switching backends.

Contradictory input records are handled before this evaluation. If two authoritative-looking records disagree on the same ownership allocation, the workbench creates an evidence conflict and identifies the disputed fact. It must not insert both as classical premises and infer arbitrary results from an inconsistent theory. Being outside a rule's

scope, having insufficient evidence, and detecting a data conflict are also distinct operational outcomes. These statuses should wrap the logical result rather than being encoded as false.

3.7 Unknown is an information state

The bank often lacks a decisive fact when a rule is evaluated. In the gift example, the evidence may not establish whether an additional return is a gift. The program should distinguish that absence from evidence that no gift exists. We therefore need an unknown value, written U, alongside known true T and known false F.

The current Boolean operations follow strong-Kleene tables. The name identifies a precise three-valued calculation, which can be taught through the two cases already encountered. If a product-specific promotional link is true, then the disjunction of that link with an unknown type link is true. If the specific link is false and the type link is unknown, the disjunction is unknown. A true route suffices; an absent route cannot settle the other one.

a	b	$a \wedge b$	$a \vee b$
T	T	T	T
T	F	F	T
T	U	U	T
F	F	F	F
F	U	F	U
U	U	U	U

Table 3.1: The omitted rows follow by exchanging the operands. Negation swaps true and false and leaves unknown unchanged. The table describes logical values; evidence conflict and errors are handled separately.

An established exception can make the gift conjunction false even when another body fact is unknown, because one false conjunct is enough. That is the logic behind g16. It is not a general instruction to ignore missing information. Unknown scope is handled before the body, and conflicting evidence has its own precedence. The evaluation result must retain which question was answered and why the remaining unknown body facts did not change that answer.

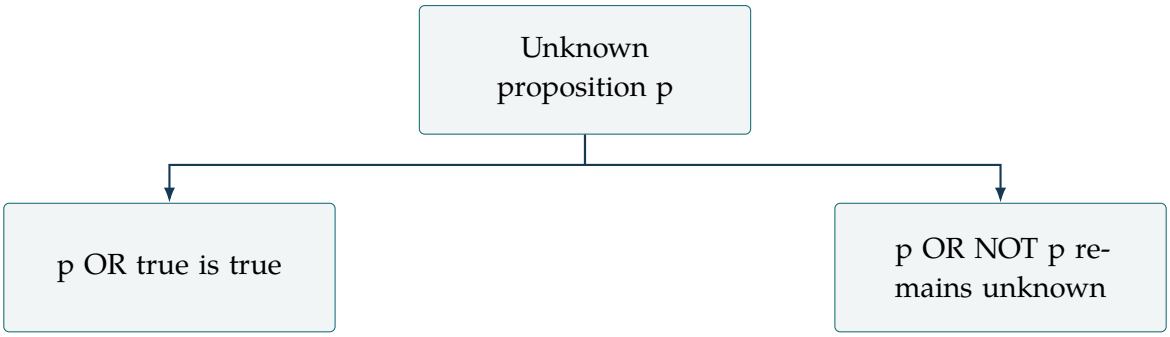


Figure 3.10: Three-valued evaluation follows information rules, not every classical identity.

A convenient symbolic encoding represents a three-valued Boolean by two ordinary Boolean indicators: one means "known true" and the other "known false." The unknown value has neither indicator. If t_a and f_a encode a, then

$$t_{a\wedge b} = t_a \wedge t_b, \qquad f_{a\wedge b} = f_a \vee f_b, \tag{3.10}$$

$$t_{a\vee b} = t_a \vee t_b, \qquad f_{a\vee b} = f_a \wedge f_b, \tag{3.11}$$

$$t_{\neg a} = f_a, \qquad f_{\neg a} = t_a. \tag{3.12}$$

The domain excludes $(t_a, f_a) = (1, 1)$. That pair would assert both truth and falsity, which belongs to a different conflict mechanism in the current design. The equations reproduce Table 3.1: for example, true and unknown have true indicators 1 and 0, so their conjunction cannot be known true; neither has a false indicator, so the result is unknown.

This encoding allows a solver to search disagreements while preserving missing information. Replacing each unknown by an arbitrary ordinary Boolean inside one execution would ask a different question. It could produce a definite answer chosen from a possible completion without showing that other completions differ. A comparison tool must name the semantics it encoded, not merely report that it used a solver.

There is a subtler distinction between compositional three-valued evaluation and evaluating every Boolean completion. Let p be unknown. Strong-Kleene evaluation gives

$$p \vee \neg p = \mathsf{U} \vee \mathsf{U} = \mathsf{U}. \tag{3.13}$$

Under every consistent Boolean completion, however, the expression is true: if p is true the first term holds, and if p is false the second holds. A completion-based semantics can therefore return true. Neither calculation is ambiguous once its rules are specified; they are different semantics.

For the read-once gift body, the retained truth oracle's completion method agrees with the compositional calculation. That particular equivalence does not make the completion oracle a general replacement for RuleIR semantics. A future ensemble should treat an unexpected disagreement as a possible semantics mismatch before assuming that one interpreter contains a bug. This example is also a reason to test repeated variables when extending the comparison fragment.

3.8 Conflict, scope and errors are not extra Boolean values

Unknown truth and conflicting evidence should not be combined casually into one truth table. The current runtime first normalizes records by validity, recording time and explicit supersession. Two admissible non-superseded records with incompatible values produce conflict. That result records a problem in the evidence supporting a fact. It does not ask the Boolean evaluator to choose which record is right.

The static dependency closure contains every fact reachable from the requested rule through its scope, body, referenced rules and exception branches. Conflict in that closure prevents evaluation from treating a convenient branch as a way to evade contradictory evidence. A conflict in an unrelated declared fact does not automatically block the requested rule. This distinction makes the conservative policy bounded and reproducible.

After successful structural validation and conflict handling, scope determines whether the body is an appropriate question. A known-false scope produces out of scope. An unknown scope produces unknown with a scope reason. Only a known-true scope permits the in-scope body result. Thus an out-of-scope result is a statement about the relation between the query and the selected rule, not a false answer to the rule's substantive condition.

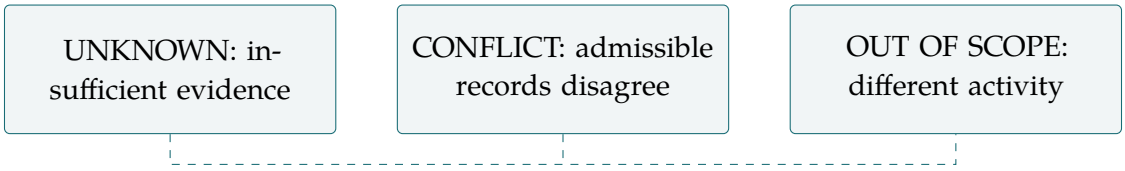


Figure 3.11: Different reasons for withholding a Boolean answer require different remedies.

Malformed data and unsupported constructs introduce a further distinction. An input with an invalid date or duplicated conflicting identifier may be an error, not legal uncertainty. If the system cannot parse a request, it cannot honestly report that the regulatory condition is unknown on a well-defined request. The API should identify the boundary error and the location of the malformation. Operationally both may prevent automatic action, but their causes and repair routes differ.

We can therefore describe the result as a tagged value whose tag identifies the kind of answer. The current contract includes true, false, unknown, conflict, out of scope and error, together with diagnostics and provenance. The tags are not interchangeable labels for a single confidence score. The ensemble discrepancy taxonomy must preserve them so that a failed parser is not processed as a legal disagreement and an unresolved definition is not processed as a transient network failure.

The precedence is part of the specification. Reordering checks can change both a result and its explanation. The g19 example deliberately returns conflict even though an actor scope fact is false. A Java implementation that short-circuits on scope before consulting

the required conflict closure would disagree with the current target. That disagreement is wrong relative to the declared execution semantics, independently of whether the bank later chooses to revise the policy.

3.9 A readable specification with a fixed translation

Controlled natural language keeps the familiarity of English while restricting how sentences are formed. The restriction matters because the reader should be able to determine the formal structure from the sentence without asking a model to interpret it again. Logical English provides one developed example of this approach through explicit templates, variable binding and restrictions on ordinary linguistic ambiguity (Kowalski et al., 2023, language and law examples).

For our narrow gift example, a specification can state: "For an assessed component in the established distributor/fund scope, the selected gift restriction is triggered if the component is a gift, at least one of the two specified promotional links holds, and the component does not meet the defined discount exception." The specification must then define each term and its evidentiary state. This sentence is reviewer-facing exposition. A machine translation needs an accompanying restricted structure in which the three conjuncts, disjunction and negation are explicit.

An appropriate authoring interface might present one condition per line with typed connectives and named definitions. The reviewer sees the English label and can inspect the exact expression beneath it. Free text remains available for reasoning and caveats, but it does not secretly alter the executable condition. A change to a definition or connective creates a new specification revision. Editing an explanatory paragraph must not bypass the formal diff.



Figure 3.12: Readability and precise translation meet in a restricted language.

The controlled specification also records the legal character of the source. A statement framed as an obligation, a prohibition, a permission, a discretion or an expectation should retain that category. Translating all of them into a Boolean called `allowed` would destroy distinctions before verification begins. The current RuleIR chiefly computes conditions. A reviewed operational policy determines how those conditions affect a bank action. Richer normative and temporal languages are introduced in Chapter 4.

A fixed translation from restricted syntax to RuleIR is a valuable target for proof. Every permitted construct has a known translation rule; unsupported sentences are rejected or left as unresolved authoring work. If a reviewer writes an unrestricted sentence outside that syntax, a model can propose a restricted rendering, but that proposal returns to

the interpretation process. The system cannot claim deterministic controlled-language compilation merely because an LLM usually produces valid JSON.

Roundtrip verification supplies an additional check. Translate a formal candidate into readable language, formalize the rendering again, and compare the two formal candidates. A mismatch can reveal drift. Agreement can show stability across the tested transformations. The same omitted source condition can survive both translations, so the roundtrip result remains one component of the evidence, as acknowledged by Amrollahi et al. (2026, method and limitations). It cannot replace the independent source inventory established earlier.

3.10 How a preservation argument is constructed

The phrase "prove the compiler correct" becomes more useful when the reader can see the structure of such a proof. Consider only the finite Boolean expression grammar in equation (3.7). Let its source evaluator use Table 3.1. Suppose a translation maps literals, fact references and each connective to corresponding target operations. Assume the target fact lookup returns the same normalized tagged value for every declared fact and that its primitive operations implement the same tables.

Proposition 3.1 (Preservation for the Boolean expression fragment)**.** *Under those assumptions, translating any well-formed finite expression in equation* (3.7) *preserves its three-valued result for every normalized fact assignment.*

Proof. Use induction on the construction of the expression. A true or false literal has the same value by the literal translation. A fact reference has the same value by the lookup assumption. For a negation, the induction hypothesis gives the same child value in both evaluators, and the target negation table maps that value to the same result. For a conjunction or disjunction, the induction hypotheses give equality for every child. Applying the identical connective table to the identical sequence of values gives the identical parent value. These cases cover every constructor in the grammar. The finite syntax tree ensures the induction reaches a base case on every path. □

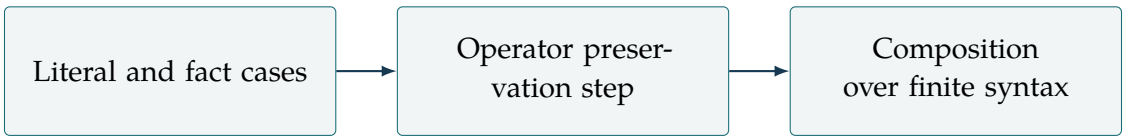


Figure 3.13: The preservation argument follows the structure of the expression.

For an alternative of two comparisons, first establish that each comparison has the same value in both implementations. Then establish that both use the same unknown-value rule for the alternative. This local argument still needs separate treatment of errors, scope and observable traces.

This small proof is exact, but its scope is intentionally visible. It assumes correct target primitives and lookup. It does not establish the current Java implementation satisfies those assumptions. It does not cover scope, conflict normalization, numeric operations, exceptions, errors or traces until separate lemmas include them. And it says nothing about whether the expression is a justified interpretation of paragraph 10.

The next proof obligations follow the same pattern. A normalization lemma must show that both implementations select the same admissible evidence under the same times and supersession relation. A scope lemma must preserve the tagged result and the decision to skip the body. A conflict lemma must preserve the static closure and conflict precedence. A trace lemma must connect each evaluated or skipped node to the required source and evidence identifiers. Only after these claims are assembled can the implementation claim be expanded from a Boolean value to the full semantic result.

The actual generated Java class currently binds an immutable validated bundle and invokes the Java semantic runtime. The proof boundary must follow that architecture. Proving a small emitted wrapper correct while assuming an unchecked runtime would leave most of the computation outside the theorem. Conversely, a proof about a generic runtime still needs to show that the generated class embeds the intended bundle and that the host invokes the correct class. The theorem's trusted components should be named rather than hidden under the word "compiler."

There is a useful discipline here for an implementation agent. Begin with a small formal fragment whose semantics and generated behavior can be inspected. Prove or exhaustively check its components, then extend the fragment with new constructors and corresponding obligations. A failed lemma is information about the specification or implementation. It is not a reason to weaken the claimed target without explaining the change.

3.11 Counterexamples and the domain they live in

To compare two candidate interpretations, the system first needs a common question and compatible inputs. Let F_1 and F_2 compute tagged results on the same declared input domain. A solver can search for an assignment satisfying

$$D(x) \wedge (\pi(\text{eval}(F_1, x)) \neq \pi(\text{eval}(F_2, x))) \,. \tag{3.14}$$

Here D states the permitted facts and relationships. In the product-type example, a witness sets gift true, specific-product link false, type link true and discount false, with scope established. The assignment has different consequences under the complete and omitted-route rules.

The first solver check should establish that D is satisfiable. If the domain simultaneously requires a fact to be true and false, the difference query is unsatisfiable for an unhelpful reason: there are no admissible inputs. The correct response is `INCONSISTENT_DOMAIN`, not equivalence. This precheck is a concrete safeguard against a formally successful but meaningless verification result.

If the domain is satisfiable and the difference query is satisfiable, the assignment is a counterexample to equivalence in that domain. Before displaying it to a reviewer, replay it through the ordinary interpreters. That replay checks that the solver encoding and the runtime agree on what the witness means. If replay disagrees with the solver prediction, the comparison harness has failed and the witness cannot be used to adjudicate the candidates.

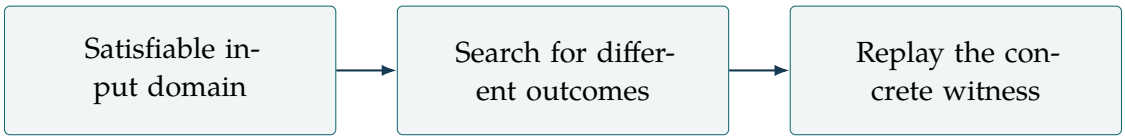


Figure 3.14: A counterexample concerns the domain actually encoded.

If the difference query is unsatisfiable, the supported conclusion is absence of a counterexample under the encoded semantics and domain. The strength of that result depends on the solver procedure and any checked certificate. A machine-checked certificate provides a different assurance level from trusting an unchecked solver response. A timeout, unsupported construct or unknown solver result remains unresolved; it must never be converted to successful equivalence because no witness was printed.

Domain assumptions can hide exactly the error we hoped to find. If a comparison asserts that every product-type promotion also names a specific product, the omitted-route mutation may become indistinguishable from the proposed rule. That assumption would need a source or business justification. It cannot be introduced merely because it makes the proof succeed. The report should show which constraints exclude a witness and where those constraints came from.

Minimal witnesses are useful for human review, but "minimal" also requires a definition. One option minimizes the number of facts changed from a familiar baseline. Another minimizes monetary amounts within meaningful bounds. Neither guarantees that the resulting scenario is legally or operationally feasible. Source and domain validation remain necessary. The smallest formal assignment may be a poor real-world question if it combines classifications that a reviewer would never jointly accept.

3.11.1 A concrete SPI witness and its knownness encoding

For many regulatory changes, a small counterexample is more useful than a large proof transcript. Suppose the proposed implementation accidentally replaces the first inclusive comparison in Equation (3.8) with a strict one. Let that candidate be G, and write D for the declared set of admissible, complete inputs (P, N). A solver can search for

$$\exists (P, N) \in D : \quad F(P, N) \neq G(P, N). \tag{3.15}$$

The concrete values $P = 40{,}000{,}000$ and $N = 0$ witness the disagreement: F is true and G is false. A compliance officer can understand the error without reading the encoding. Replacing the alternative with a conjunction is similarly exposed by $P = 35{,}000{,}000$ and $N = 90{,}000{,}000$.

This technique also compares two candidate interpretations or old and new rule versions. It must check the admissibility and consistency of its assumptions first. If a test domain accidentally demands that the same amount is both positive and negative, the absence of a counterexample proves nothing useful about real cases. Unsatisfiable assumptions should be reported as a separate failure, with the conflicting constraints identified. Solver timeout and unknown results should never be rendered as "equivalent."

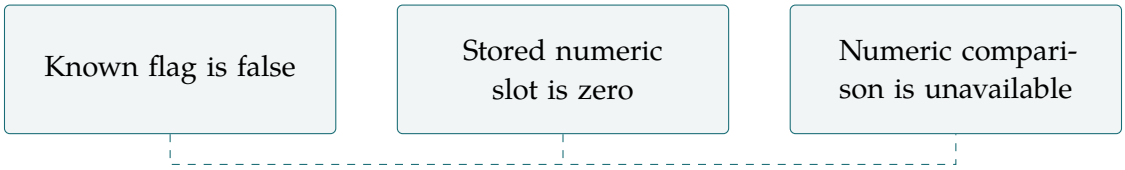


Figure 3.15: Knownness prevents a placeholder from becoming evidence.

The initial Z3 adapter supports a smaller language than the interpreter: Boolean, integer and money expressions with explicit knownness and bounded declared domains. Dates, scaling, defaults and events return `UNSUPPORTED`. Encode a Boolean expression a by two bits (t_a, f_a): true is $(1, 0)$, false $(0, 1)$ and unknown $(0, 0)$; prohibit $(1, 1)$. Conjunction then has the encoding

$$t_{a \wedge b} = t_a \wedge t_b, \qquad f_{a \wedge b} = f_a \vee f_b. \tag{3.16}$$

Disjunction uses the dual equations and negation swaps the bits. A numeric value carries a knownness bit, and a comparison produces neither Boolean bit unless both operands are known. These equations implement the stated truth tables. Giving an unknown fact an unconstrained classical Boolean would instead make $A \vee \neg A$ always true and verify a different language.

The service checks domain satisfiability before searching for a difference in tagged results. Display a satisfying model only after replay through both interpreters. An unsatisfiable difference query yields `NO_COUNTEREXAMPLE_IN_DECLARED_DOMAIN`; an empty domain yields `INCONSISTENT_DOMAIN`; timeout stays `UNKNOWN`. Preserve formula, domain, rule hashes and solver version. A solver answer is a different class of evidence from a certificate replayed by an independent checker.

For event rules, the corresponding search concerns traces. A useful first property is that, in the adopted consent model, no transition representing a new streamlined decision uses a withdrawn consent state. Another is that processing the same evidence event twice does not create duplicate obligations. Each property has assumptions about event identity and ordering. Bounded trace search can find violations, while a proof of an invariant can establish a stronger claim if the transition model and induction obligations are checked.

3.11.2 Solvers, explanations and search in ordinary programming terms

A satisfiability solver searches for values that make a set of logical constraints true. To compare the correct wealth alternative with a mistaken conjunction, give it both formulas and ask for inputs where their answers differ. HK$35 million and HK$90 million are one such counterexample. An SMT solver extends this idea with theories such as exact integer arithmetic. It is useful here because money can be encoded as integers. Before treating "no counterexample" as evidence, the system must establish that the allowed input domain is nonempty and that both formulas were encoded faithfully. A timeout means that the search did not finish.

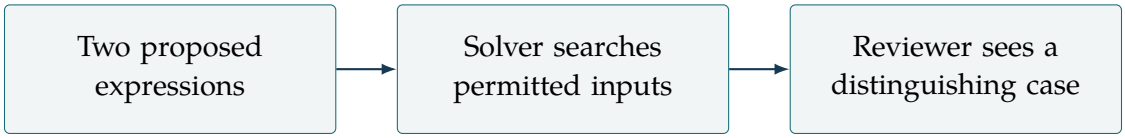


Figure 3.16: A solver turns a formal disagreement into a concrete example.

Two names distinguish a reasoning engine from its legal application. s(CASP) is a logic-programming system that can return the assumptions and rule steps supporting an answer. s(LAW) uses that kind of reasoning to represent legal rules and explore their consequences. Here a logic program consists of facts and rules for deriving conclusions, rather than a sequence of instructions for updating a client record.

A justification engine answers a related question: which rules and facts support a conclusion? In a rule program, `eligible(C) :- financial(C), consent(C)` means that both supporting predicates are needed to derive eligibility for the same client `C`. s(CASP), used by s(LAW), can expose the supporting derivation and assumptions rather than returning only a label. An assumed consent predicate in a hypothetical answer is still an assumption; it must never be written back as an observed agreement (Arias et al., 2024).

Search over interpretations sits above these calculations. An agent may propose two readings of a clause, retrieve a referenced definition, and ask a solver for a case that separates them. A tree search can use an upper-confidence-bound (UCB) score to choose which investigation to try next, balancing promising branches against less-explored ones. Its score measures usefulness under the chosen search objective; it does not certify the resulting legal reading. The adopted meaning must come from source-based adjudication. Model Context Protocol (MCP) provides an interface through which an agent can call tools, such as a solver or a document retriever. The available integrations can assist these bounded jobs, as explained in Section F.4.1.

3.12 Proving a workflow property

Some of the strongest useful guarantees concern the review process rather than the legal reading. For example, the proposed ensemble should never mark an assessment releasable while a material discrepancy remains unresolved. This is a property of explicit records and transitions. It is amenable to a state-machine argument once the terms "material," "unresolved" and "releasable" have precise stored meanings.

Let Q be an assessment state, $M(Q)$ its set of unresolved material issues, and $R(Q)$ the predicate that a release is authorized. A required invariant is

$$R(Q) \implies M(Q) = \varnothing. \tag{3.17}$$

The invariant does not say that an empty issue set proves the interpretation correct. It says that known unresolved material issues cannot be ignored by the release transition. A faulty detector may fail to create an issue, which is why independent coverage and detector evaluation remain necessary.

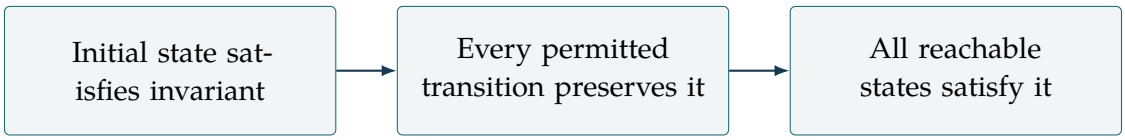


Figure 3.17: A workflow invariant needs an initial case and a transition argument.

To establish the invariant, the initial state has no release. Every transition that creates, reopens or materially changes an issue must invalidate dependent release eligibility. The only transition that authorizes release must check the issue set and all other required conditions in the same transaction that records the authorization. If a concurrent worker can add an issue after the check but before the release without a version check, the implementation does not preserve the invariant. Database transaction boundaries are therefore part of the proof assumptions, not an incidental engineering detail.

A second property concerns termination of automatic processing. Suppose a run has a finite number of reserved actions, and every dispatched action consumes one reservation even if the worker fails. If all automatic paths either consume a reservation or enter a terminal state, then no run can dispatch more actions than its budget. This argument requires crash and retry accounting; counting only successful responses would permit unlimited failed calls. Chapter 6 gives the exact controller and its termination measure.

These guarantees directly support the user's requirement. They can establish that the system reports remaining uncertainty and respects its retry limit. They cannot establish that the detectors discovered every possible uncertainty. The two responsibilities should be tested separately: detection quality against independent cases, and workflow integrity against state-transition and failure injection tests.

3.13 Temporal duties require histories

A Boolean assessment of an offer is not enough for every circular requirement. Some duties become active after an event and must be performed before a deadline. Others must remain satisfied throughout an interval. Consent can be granted, withdrawn and granted again. A formal specification for these cases needs histories and states, not just a snapshot of current fields.

Consider a hypothetical duty activated at time a with an exclusive deadline d. Under a declared non-preemptive achievement interpretation, a matching performance at time t satisfies the timing condition exactly when

$$a \leq t < d. \tag{3.18}$$

The inequality states an engineering profile; a source may require a different boundary or permit performance before activation. Those choices must be reviewed and encoded explicitly. The event must also match the duty's actor, action and subject. A timely action by the wrong party is not satisfaction.

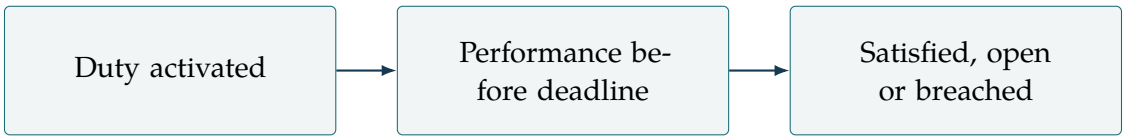


Figure 3.18: A duty concerns events and observation completeness.

Failure to find a performance record is not by itself evidence that the duty was breached. The observation stream may be incomplete. The current event runtime therefore requires a completeness assertion through the deadline before inferring breach from absence. That assertion has provenance and a recording time. A process clock cannot create it. This mirrors the source-coverage problem: an empty retrieved set does not prove that the relevant object does not exist.

Late performance and breach are also distinct. A duty can be performed after its deadline without erasing the fact that timely performance was absent under complete evidence. A later correction may establish that an earlier performance was missed; that creates a new assessment with a different knowledge cutoff. The original assessment remains available. Normative logic and contract-state languages help express these distinctions, but their semantics must be selected to match the intended duty (Governatori and Rotolo, 2006; Hashmi et al., 2016; Crafa et al., 2021).

The current profile supports a limited event model. It does not automatically implement every waiver, sanction, compensatory duty or maintenance obligation that a legal document could contain. Unsupported temporal constructs should remain explicit implementation gaps. A translator that invents a deadline or converts a continuing duty into a single check would create a new rule while appearing to simplify the software.

3.14 An assurance claim is a conjunction of different evidence

We can now state the overall structure without pretending that one theorem settles the entire process. Source retention establishes which material was reviewed. Interpretation review establishes an attributable decision about that material within a declared context. Formal checks establish consequences and preservation properties for specified languages and domains. Integration checks establish how a result reaches a bank action. Each supports a distinct premise in the justification for using a control.

This structure prevents two opposite mistakes. The first is to dismiss formal verification because it cannot prove the meaning of unrestricted English. Preserving a reviewed specification across implementations is valuable, and workflow invariants can rule out important classes of operational failure. The second is to present formal verification as if it eliminated the interpretation and evidence premises. A perfectly verified formula still needs the right subject, source version and fact classifications.

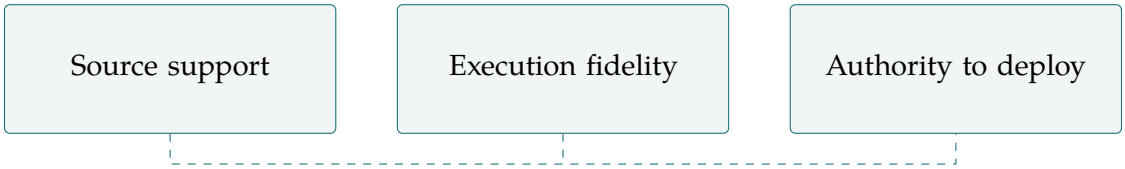


Figure 3.19: Each part of assurance needs its own evidence.

For the bank's reviewers, the practical outcome is a set of answerable questions. Which parts of the interpretation are accepted and by whom? Which formal properties have actually been established? What lies outside the supported fragment? Which data mappings remain unresolved? What would reopen the conclusion? Those questions can be supported by exact records rather than an undifferentiated confidence statement.

The next chapter examines existing languages and methods in that light. Each is introduced by the problem it solves: readable authoring, exceptions, alternative interpretations, events, or symbolic checking. Their combination must preserve those distinctions instead of assuming that choosing one formal language supplies the whole assurance argument.

Chapter 4

Languages that make interpretation inspectable

4.1 Choose the question before choosing the language

The gift example needs a condition over classified facts. A consent example needs a history of grants and withdrawals. A payment agreement may need parties, assets and deadlines. A dispute about the meaning of an exception needs alternative readings and their supporting reasons. These are related questions, but a representation designed for one does not automatically solve the others. The purpose of reviewing formal legal languages is to identify which structures help make each question precise.

A formal legal language is a language in which statements about a legal or normative domain have defined syntax and computational meaning. The word "legal" identifies the intended subject matter and useful abstractions. It does not mean the language has legal authority, that a program written in it is enforceable, or that translation into it establishes a correct reading. Authority still comes from the relevant source and institutional decision.

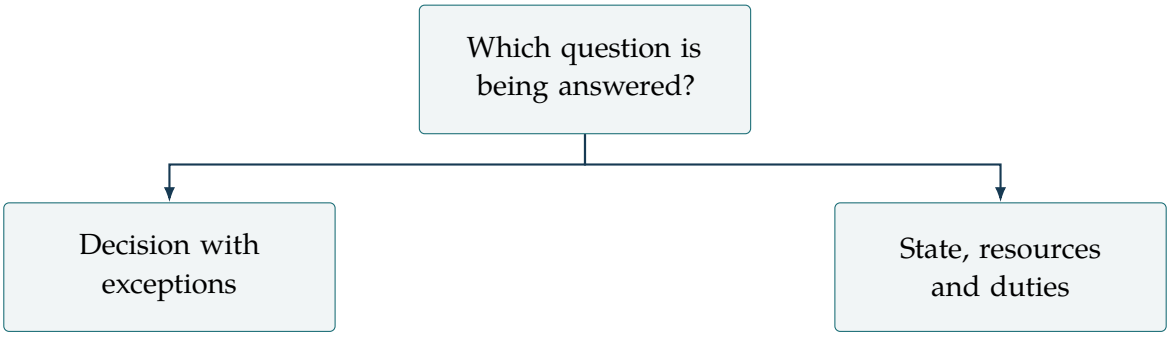


Figure 4.1: Different legal questions call for different computational languages.

For our product, a useful language must support at least three forms of inspection. A reviewer needs to see how a condition corresponds to source text. An engineer needs to know exactly what values and events the program expects. A verification tool needs unambiguous semantics. A language that is expressive enough to represent any program may make verification harder; a language that permits only simple decision tables may fail to represent important temporal or exception structure.

This chapter develops the relevant mechanisms through examples. It does not choose a universal replacement for the bank's Java system. The recommended architecture keeps a bounded executable representation, a richer interpretation record and explicit

adapters for specialized analyses. The relationship between those representations must itself be checked. Adding more languages without defining that relationship would create new translation boundaries rather than redundancy.

4.1.1 The research families behind the design

Computational law has developed several answers to different parts of this problem. Logic programming makes statutory dependencies executable. Languages such as Catala organize definitions and exceptions close to legal text. Normative languages represent duties, permissions and violations. Contract languages model interactions through time. Recent language-model research attempts to cross the remaining boundary from prose to these representations. A useful product combines these contributions selectively, because their guarantees concern different objects.

4.1.2 How the literature bears on the banking question

The review follows the distinctions a bank control needs to preserve. A numerical eligibility rule raises a different question from two competing exceptions; a duty that persists through time raises another. The selected research therefore covers executable statutes, exceptions, obligations, dates, source authority, formal verification and the translation of prose into rules. The comparison asks what each method can express and check, and where a reviewer must still supply meaning or evidence.

A language's definition and its software implementation are separate sources of evidence. A paper may prove a property of a small calculus while its compiler supports a larger language; a repository may demonstrate an example without establishing a general theorem. The discussion keeps those distinctions beside the relevant result. Source versions, technical reading notes, citation searches and unresolved retrieval gaps are retained in Appendix F of the technical companion. The published benchmarks and proof developments discussed here were inspected, not independently reproduced.

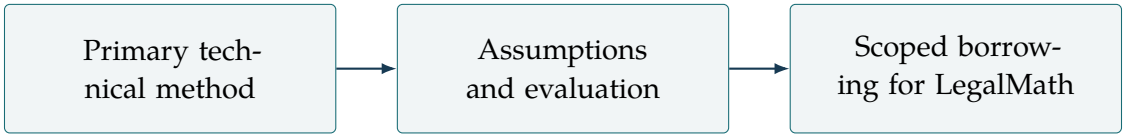


Figure 4.2: Literature review must inspect the mechanism being borrowed.

4.2 Rules do not all behave like ordinary implications

An ordinary material implication says that if a proposition is true, another proposition is true. A legal norm often says that, under specified circumstances, someone ought to do something. The existence of the duty does not establish that the action occurred.

A rule requiring a document to be supplied cannot be used as evidence that the document was supplied. The distinction between facts and normative outputs is central to input/output logics (Makinson and van der Torre, 2000, secs. 2–4).

For a simple illustration, suppose an adopted specification says that a distributor must provide a named warning when a condition holds. The condition can create a duty record. Performance requires separate evidence of the warning being delivered to the correct recipient in the required context. If the program inserts the duty's content into its factual database as though it had happened, every obligation can appear automatically satisfied.

Prohibitions and permissions also need care. Failure to derive a prohibition may mean the act is permitted under one explicitly defined logic; in a bank workflow it may instead mean the relevant sources or facts are missing. The literature distinguishes strong and weak permissions under particular defeasible-logical definitions (Governatori et al., 2012, secs. 2–4). Those definitions are not a general license to treat an empty query result as authorization. The selected RuleIR gift result remains a subcondition, with permission supplied only by a separate approved decision policy.

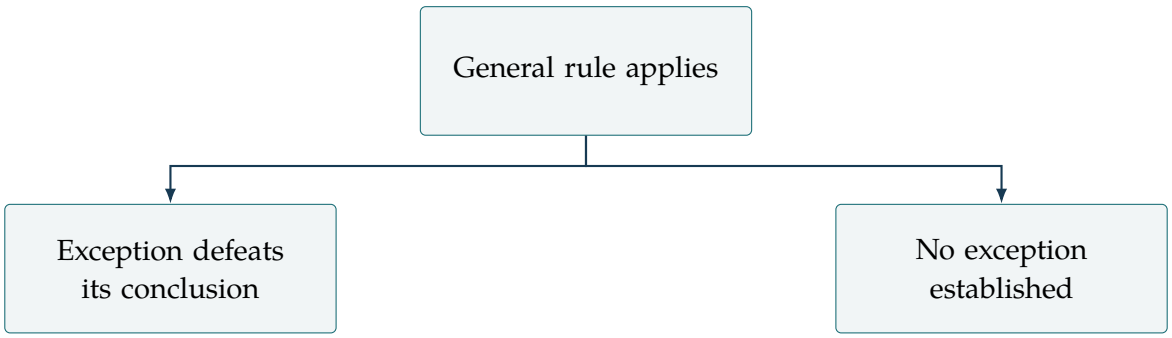


Figure 4.3: Defeasible reasoning makes the exception relationship explicit.

A general fee rule and a special discount exception cannot be combined by treating every sentence as an unconditional implication. The formalization must state when the exception applies and what happens when its applicability is unknown.

Many legal rules are defeasible: they normally support a conclusion, but an exception or stronger opposing rule can override that support. A defeasible rule is not the same as an ordinary implication with an informal comment attached. Its evaluation must account for applicable opposing rules and priorities. Defeasible-logic systems formalize different ways to do this, including rules that support conclusions and defeaters that block opposing support (Antoniou et al., 2000, sec. 2).

The engineering implication is simple but demanding. "Has exceptions" is not a complete semantics. The specification must say what happens when no exception is known, one is established, two compete, or an exception remains unknown. It must also say whether a priority is supplied by legal authority, an explicit interpretation decision or a programming convention. File order is not an adequate hidden source of legal priority.

The early logic-program treatment of the British Nationality Act demonstrated how a

selected body of legislation could be decomposed into rules with inspectable reasoning (Sergot et al., 1986). That work is valuable here as an example of disciplined formalization, including attention to time and negative information. It should not be read as a claim that ordinary legal prose can always be converted automatically into a complete logic program.

4.2.1 Why exceptions, duties and permissions need different logics

A threshold calculation asks whether a proposition follows from evidence. A duty asks what an actor ought to do in those circumstances. Treating both as ordinary implication loses the possibility of noncompliance: an instruction to conduct an annual review is not evidence that the review occurred. Input/output logic makes this separation explicit. A normative system transforms factual conditions into required outputs, without requiring every input to be an output or permitting unrestricted contraposition (Makinson and van der Torre, 2000, secs. 2–3).

Let A contain factual propositions and let G contain pairs (a, x) expressing that condition a calls for outcome x. Writing Cn for classical consequence, the paper's simplest operation is

$$\text{out}_1(G, A) = \text{Cn}(G(\text{Cn}(A))) \,. \tag{4.1}$$

Here $G(S) = \{x \mid (a, x) \in G \text{ and } a \in S\}$ collects the required outputs whose conditions are in S. Thus the two uses of G denote the same normative system, first as a set of pairs and then as the operation of applying those pairs. First derive the input conditions, then apply the normative pairs, then unpack their consequences. Section 3.2 characterizes this operation by strengthening inputs, conjoining outputs and weakening outputs. A pair connecting "review due" with "review performed" supplies a required outcome; it does not create a recorded fact of performance. Other variants change disjunction and reuse. LegalMath borrows this separation rather than implementing all input/output logics.

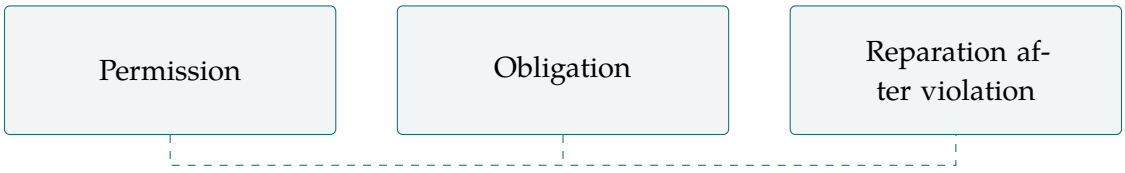


Figure 4.4: Permission, duty and the consequence of breach are different normative relations.

Exceptions ask a different question: which applicable rule defeats another? Antoniou et al. (2000, sec. 2) distinguish facts, strict rules, defeasible rules, defeaters and an explicit superiority relation. Their proof theory records both definite and defeasible provability, and explicit failure to establish either. A defeasible conclusion needs supporting premises, failure of definite contrary proof and a response to every applicable opposing rule. Different supporting rules may defeat different opponents. This "team defeat" differs materially from selecting one table row or finding one Catala exception.

An unretrieved agreement is therefore not a defeater merely because the database has no row. The product must distinguish explicit negative evidence, a closed-world assumption and a reviewed default. LegalMath 0.1 chooses nested, source-linked defaults with conflicts between competing sibling exceptions. It does not implement the full defeasible proof theory. General priorities remain an extension requiring a stated defeat policy and conformance tests.

Permissions add another distinction. In this proof theory, a weak permission requires a derivation establishing that the opposite obligation is not derivable; an unfinished search or an empty database response is insufficient. A strong permission has explicit normative support. Governatori et al. (2012, secs. 2–3, 6) distinguish permissive rules from defeaters and distinguish obligation from factual fulfillment. Their algorithms transform a finite theory while preserving its defeasible extension. The soundness, completeness and linear-complexity results in section 5 concern that theory and specified data-structure assumptions, not arbitrary regulations. For the bank, failure to derive a prohibition must never by itself become a positive trade authorization.

A remedy is different again. The logic of violations of Governatori and Rotolo (2006, secs. 2–5) uses an ordered reparation connective: perform the primary duty; if it is violated, a further duty can become relevant. Replacing that structure with ordinary disjunction can make the remedy appear an equally acceptable way to satisfy the original requirement. A warning delivered after a deadline can be useful remediation without making the original control timely. LegalMath retains the breach assessment and records subsequent performance separately; general compensation and sanction reasoning remain outside its first event profile.

4.3 Controlled English: readable structure with restricted freedom

Ordinary English permits pronouns, elliptical references, implied subjects and several possible attachments of a qualification. Those features make writing compact, but they leave a translator with choices. Controlled English restricts some of that freedom so a sentence can be mapped into logic by explicit rules. Logical English develops this approach for law and education (Kowalski et al., 2023, language specification and examples).

Consider the sentence “If it is a gift and is linked to the product or its type, it is prohibited unless discounted.” The sentence is short, but “it” could refer to the offer or component, “its” needs a binding, and “discounted” does not identify the exception. A controlled rendering should repeat the subject where necessary and name each definition. Repetition in the language is useful if it removes a legally consequential ambiguity.

One possible authoring form for our own restricted profile is:

Listing 4.1: Illustrative LegalMath authoring notation. This is a proposed template, not claimed to be valid Logical English or Catala source.

```
subject: component C in offer O
scope:
  actor(O) satisfies distributor_profile
  activity(O) satisfies authorised_fund_promotion_profile
condition:
  C satisfies gift_definition
  AND (
    C has specific_product_link
    OR C has product_type_link
  )
  AND NOT (C satisfies fee_discount_definition)
consequence:
  selected_gift_restriction_triggered
```

The form makes four decisions inspectable: the subject, the scope, the condition and the named consequence. Every definition identifier refers to a versioned record. A reviewer can ask why a definition applies without reverse-engineering the syntax tree. An engineer can translate the connectives deterministically once the template is fixed.

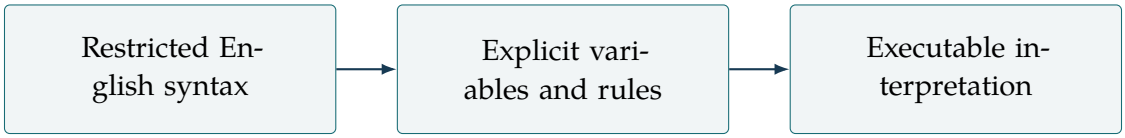


Figure 4.5: Controlled English reduces freedom to improve inspectability.

Variable binding is a practical source of error. If one occurrence of C refers to the voucher and another to the whole offer, the sentence has changed its subject midway. The authoring language should enforce a declared variable and type environment. A reference to an undeclared subject should be rejected. An association such as "component C belongs to offer O" should be explicit when the rule uses facts about both.

Negation requires similar precision. "There is no evidence that C is a gift" is not the same assertion as "evidence establishes that C is not a gift." Logical-programming systems may distinguish explicit negation from failure to prove a proposition. Our evidence model records known false, unknown and conflict separately. A controlled sentence that asks about missing evidence should use that information state explicitly; it should not rely on a model to guess which sense of negation the author intended.

Controlled English improves the boundary between reviewed meaning and implementation. It does not remove the earlier task of choosing the controlled sentence from the original circular. A reviewer can still choose the wrong definition or omit a phrase. That is why the product retains source anchors, alternative templates and contrasting scenarios alongside the readable rule. The language makes a chosen interpretation precise enough to challenge.

4.3.1 Statutory programs, literate rules and collaboration evidence

The British Nationality Act logic program is an early demonstration that selected legal provisions can be represented as executable dependencies. Its significance for this project is architectural: a conclusion can be explained through the conditions supporting it, while rules and facts are represented separately. The work also confronts negative information and temporal conditions, which remain central to bank data. It does not show that arbitrary legal prose has a unique automatic translation (Sergot et al., 1986).

The literature turns that mechanism into a review question. If a source packet contains a base rule and two exceptions, the reviewer should see the dependency from each exception to the rule, the source span supporting each guard, and the result when both guards fire. Catala's default calculus makes this failure visible: two applicable exceptions can conflict even when their computed values agree (Merigoux et al., 2021, secs. 3–4). The point for LegalMath is the traceable comparison, not another automatic priority rule.

For LegalMath, this suggests a representation in which an exception names the rule it qualifies and preserves the source supporting its priority. A discount exception to a restriction on gifts should remain visibly attached to that restriction. Two independently imported exceptions that both apply should trigger a review of their relationship, not an arbitrary choice by the language model. The product should also distinguish this normative priority from a document's publication date: a newer explanatory document does not mechanically override every older source it mentions.

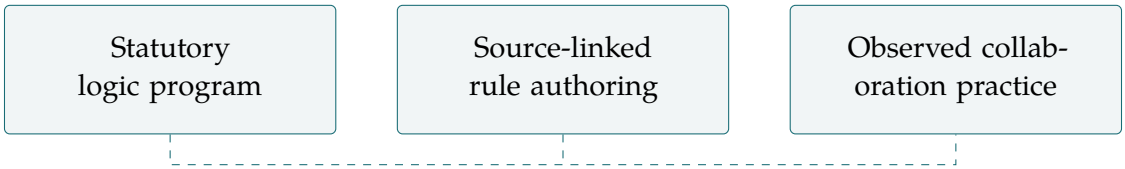


Figure 4.6: A formal language and evidence that people can use it answer different questions.

The Catala compilation result is relevant but narrower than the phrase "verified legal software" might suggest. The paper proves properties of a translation from the core default calculus into a lambda calculus with exceptions, with a mechanized development in F*. It distinguishes that model from the separately implemented production compiler. The inspected repository's formalization notes additionally disclose an admitted supporting lemma; this review has not replayed the development (Merigoux et al., 2021; Catala contributors, 2026, sec. 4). The useful precedent is to define a small semantics and state exactly which translation preserves it. It provides no theorem that a human or model has captured the meaning of an SFC circular.

Catala's literate presentation also addresses the organizational problem. Huttner and Merigoux (2022, sec. 4.2) describe collaboration between legal experts and programmers around a common representation. The legal expert can challenge a condition

without reverse-engineering application code, and the engineer can make an ambiguity concrete. The exploratory lawyer study in the original Catala paper gives useful interface evidence, but its small educational setting does not establish reliable defect detection by bank reviewers (Merigoux et al., 2021, sec. 6). LegalMath should borrow the collaboration model and test its own interface with the intended users.

Controlled language offers another way to make formal statements reviewable. Logical English retains a readable rule syntax while assigning a computational meaning to predicates and variables (Kowalski et al., 2023). For this product, a controlled paraphrase should be generated from the adopted rule, so changing the rule changes its explanation. It should display how variables are bound: "the client" must refer to the same person throughout a condition. Readability cannot substitute for a defined semantics, and unconstrained English should not be presented as if it were executable merely because it looks regular.

4.4 Catala: defaults, exceptions and source-linked authoring

A source-linked exception needs two things at once: a defined value when a guard applies and a record of which text gives that guard authority. Catala joins those concerns by placing its exception calculus beside the legal text (Merigoux et al., 2021, secs. 2–4). The question for this section is how a bank profile should expose competing guards; the example below treats their nesting and evidence as observable cases.

The useful structural question is whether an exception is nested inside a qualification or sits beside another exception. A nested exception is considered only after its parent condition is active; two sibling exceptions are competing answers to the same base rule. Moving a condition one level can therefore change which evidence is needed before a value is available, even when every word of the policy remains present.

Take a base value $b = \mathsf{false}$ and two sibling guards g and h, each of which returns true. If g is established and h is refuted, the exception supplies the value. If g is established and h is unknown, the result remains unresolved because the second exception may still apply. If both are established, the result is a conflict even when their values agree. Catala's empty-applicable-value and conflict distinction motivates this treatment; the bank profile adds the separate information state for incomplete evidence. Equal Booleans therefore do not settle the priority question.

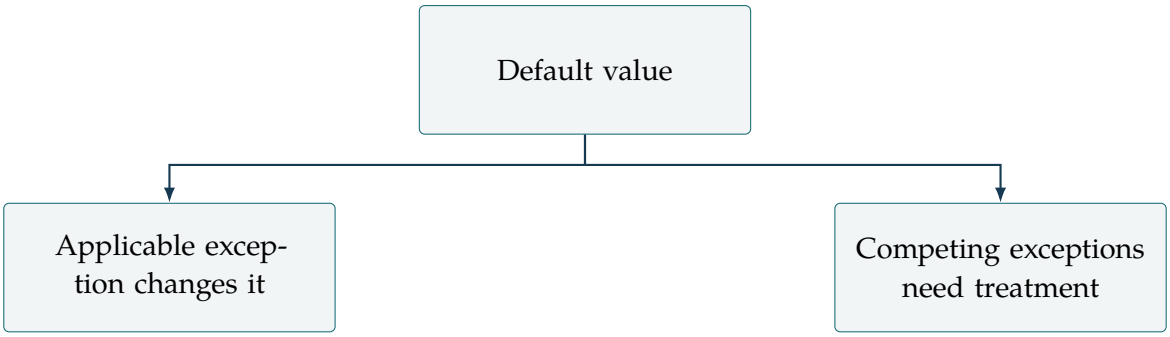


Figure 4.7: Catala makes default and exception structure part of the program.

LegalMath borrows explicit exception structure but adds its own treatment of incomplete evidence. In the present RuleIR profile, every sibling guard is evaluated. Two true guards conflict. One true guard with an unknown competitor remains unknown because the competitor may become active. One true guard with all competitors false selects its value. When all guards are false, the base is selected. This is a separately specified design; it is not a claim that RuleIR implements the whole Catala calculus.

The distinction matters for ensemble comparisons. A Catala model and a RuleIR model can be useful independent representations, but they do not become equivalent merely because both have an "exception" construct. The adapter must define how missing evidence, no applicable value and competing exceptions are represented. A comparison outside that common fragment should return unsupported or expose the difference, not choose a convenient interpretation.

Catala's compiler work also illustrates how formal claims should be scoped. The paper establishes properties for a core translation; the production compiler and surrounding tooling are separate implementation objects. The retained repository inspection also records an admitted supporting lemma in the formalization notes. This project has not replayed those proofs. We borrow the explicit language structure and preservation discipline, while requiring independent evidence for any concrete adapter used in the bank product.

Huttner and Merigoux (2022, sec. 4.2) discuss legal/programmer collaboration as a way to keep source meaning and executable rules connected. The useful lesson is organizational as well as technical. A legal reviewer can challenge an exception's interpretation while an engineer checks that its computational consequence matches that interpretation. Neither role should be expected to certify the other's entire domain from a single code review.

4.4.1 Applying the exception model to SPI explanations

Apply that distinction to the SPI explanation requirement. In Annex 1 paragraph 10.3, the adopted profile makes the explanation guard active when the client asks for one or raises material queries. Catala's source-linked style motivates keeping that guard beside the text (Merigoux et al., 2021, secs. 2–4), while the banking profile must still specify how missing evidence is represented.

Write the guard explicitly rather than splitting the two conditions into competing exceptions:

Listing 4.2: An exception with an explicit guard. The combined guard avoids creating two competing exceptions for the same requirement.

```
normally: product_explanation_required = false
exception when client_requests_explanation
               OR client_has_material_queries:
    product_explanation_required = true
always retain: current_offering_documents_required = true
```

The *guard* is the condition after "when". If the guard is true, use the exception; if false, use the general value. If evidence about the guard is missing, LegalMath leaves the requirement unresolved. The separate offering- document requirement is not inside the explanation exception. A programmer who implements one switch for both has changed the model (SFC and HKMA, 2023b, Annex 1, para. 10.3).

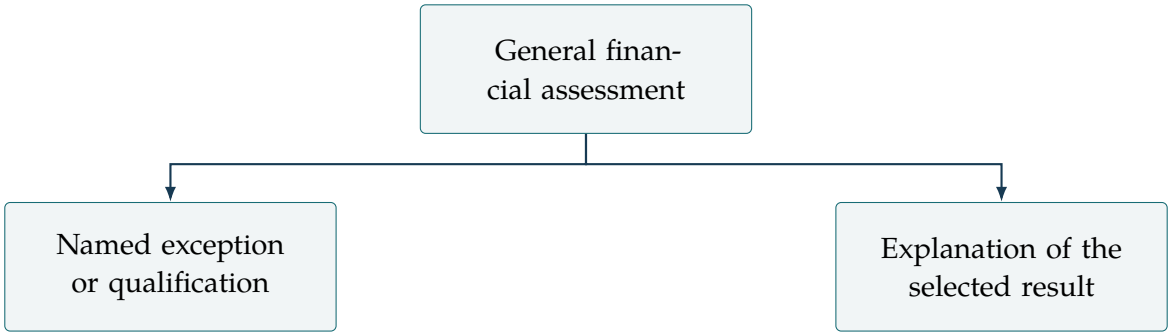


Figure 4.8: An explanation should show why the default or exception was selected.

The generic distinction now has a concrete test. In the SPI packet, the request flag is true and the material-query flag is false, so the combined guard selects the exception. If the request flag is true while the query flag is unknown, the combined OR guard is still true: the request alone establishes the condition. Writing the flags as two sibling exceptions would change that result. One true guard with an unknown competitor leaves that default unresolved; two true sibling guards conflict even when both return "explanation required". A single guard containing their OR expresses one reviewed exception and avoids that sibling conflict.

This is the difference between borrowing Catala's organization and implementing Catala itself. Catala distinguishes an empty applicable value from a conflict; LegalMath adds incomplete bank evidence as an explicit unknown state. Its proposed default operator evaluates every sibling guard, uses nesting for an adopted priority, and never treats file order as legal priority. The earlier SPI-Demo1 Java example uses the equivalent explicit Boolean condition and rejects the general default operator. The subsequently completed v0.1 runtime implements that operator with its own conformance tests; the two compiler profiles remain distinct.

The completed Catala implementation preserves this distinction deliberately. The com-

patibility backend and the version-2 shared translator encode the declared strict exception policy, including conflict when two guards are true and their consequences agree. Directly authored native Catala has a separate development record: the pinned compiler coalesced identical literal consequences in a checked example, while distinct competing consequences raised an execution error. That observation describes the tested native program and compiler, not a new priority rule for a banking specification. The shared translator emits explicit selection and result states to preserve the policy chosen in the shared model. Section 8.2 explains that model and Section 9.1 compares the resulting execution evidence.

4.5 Metadata is part of the meaning: LegalRuleML

A formula alone does not say which provision it interprets, which jurisdiction it concerns, when it applies, or whether a competing formula is also defensible. LegalRuleML addresses representation of legal rules together with such contextual and provenance information. The work on legal interpretations explicitly represents alternative readings associated with source material (Athan et al., 2014, secs. 2–4); the later OASIS specification provides a broader standardized vocabulary (OASIS LegalRuleML Technical Committee, 2021).

For the fee-discount issue, two candidate rules can point to the same source span while naming different interpretation records. One record might restrict the exception to an initial reduction; another might include a documented refund. Their source identity is shared, but their assumptions and consequences differ. The system should not assign a single interpretation identifier to both merely because the source paragraph is the same.

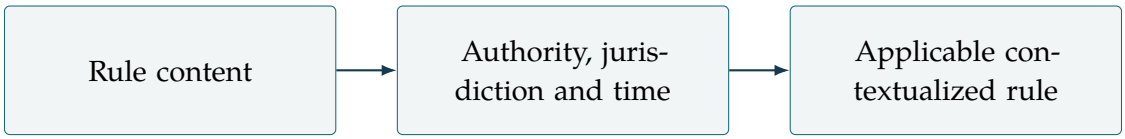


Figure 4.9: LegalRuleML identifies context that a bare formula does not express.

The representation must also preserve the authority of a selection. A legal reviewer may adopt one interpretation for a defined product and date range. That selection is an event with an author, reason and scope. It does not erase the alternative or establish that the alternative was syntactically invalid. A future source clarification may require reconsidering the choice. The provenance record is what lets the system determine which decisions depend on it.

LegalRuleML is a representation standard, not a turnkey engine for resolving every legal dispute. A serialized permission or priority still requires a specified evaluation semantics. Different reasoning engines may support different fragments or make different assumptions. For the prototype we should borrow the discipline of explicit context and alternatives without requiring the entire standard to become the first runtime dependency.

This leads to a useful separation in the proposed storage model. A source span identifies words. An interpretation claim identifies an adopted or proposed meaning. A formal rule identifies a computation. A review decision identifies who accepted which claim under which context. Keeping those identities separate prevents a hash of the source text from being used as if it also identified the meaning chosen from that text.

4.6 Stipula: a contract as parties, resources and transitions

Stipula becomes relevant when a legal arrangement changes through actions over time. Its original paper describes agreements, parties, fields, assets, functions, states and scheduled events (Crafa et al., 2021, secs. 3–6). A function can be available only to a specified party in a specified state, with a precondition on the current data. Executing it changes fields or resources and moves the contract to another state. This is a much richer object than a single decision-table row.

To understand the mechanism, consider a synthetic service agreement. It begins in a state awaiting agreement on terms. After the designated parties agree, it enters an active state. An authorized party can request a permitted service only while the agreement is active. A withdrawal changes the state so that the same request is no longer available under that agreement. A timed event may move the agreement into an expired state. This example illustrates a state model; it is not a claim about the legal effect of any particular bank consent.

The state matters because the same action name can have different consequences at different points in the history. A function called `requestService` cannot be understood by its input parameters alone if its availability depends on whether consent is active. A current Boolean copied from a stale database may also be inadequate. The state should be derived from an attributable history or a reviewed snapshot plus a complete subsequent event stream.

Stipula's agreement construct explicitly identifies who participates and who must agree on the initial terms. This is useful as a modeling discipline: the identity of the party agreeing is not an untyped annotation on a field. For LegalMath, the analogous review question is who has authority to approve a meaning, data mapping or release. We borrow the importance of explicit parties and transitions, not a claim that a regulatory circular is itself a private contract between the bank and regulator.

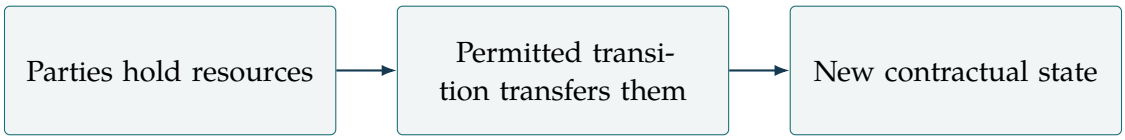


Figure 4.10: Stipula models contracts through state and controlled resource movement.

Assets in Stipula are distinguished from ordinary fields. An ordinary numeric field can be copied as information; an asset transfer must account for the resource leaving one

location and arriving at another. In the original semantics, asset movement prevents creating a negative quantity and preserves the relevant resource through transfer. This supports questions about duplicate spending, locked resources and the availability of contractual actions. It is not a model of market liquidity or a substitute for the bank's general ledger.

For a synthetic escrow, suppose a contract holds a resource amount b and transfers q to a recipient. A local arithmetic description is

$$b' = b - q, \qquad r' = r + q, \qquad 0 \leq q \leq b, \tag{4.2}$$

where r is the recipient's modeled holding. Adding the first two equalities gives $b' + r' = b + r$. This is a conservation property of the modeled transfer. It says nothing about whether the underlying asset exists off-system or whether the transfer is legally authorized. Those are separate premises. The example shows how a formal language can support a useful exact property without claiming to settle the whole legal arrangement.

Scheduled events add another dimension. A Stipula event can schedule a continuation at a time and execute it only if the contract remains in a specified state. An action taken before the event can change whether that continuation is applicable. The ordering of an event and a function call at the same time is consequently significant. The inspected original semantics gives scheduled events precedence in the relevant equal-time situation. A bank adapter must not assume that this is the legally correct ordering for its own deadlines. The source, clock precision and event-order authority need review.

This example also distinguishes observation from permission. A formal contract may describe which function calls its own execution platform permits. A bank audit system must still record actions that occurred outside a permitted workflow. Deleting an observed transaction because the policy would have forbidden it would erase evidence of a possible violation. A normative monitor therefore needs a clear separation between advice about an action and recording its actual occurrence.

4.6.1 Applying the state model to client consent

A wealth calculation is a function of facts. An agreement also has a history. Before signature, after signature, after withdrawal and after an amendment, the same attempted action can have different consequences. Stipula is a language for expressing those interactions. The requested 2021 paper defines parties, agreement terms, fields, assets, states, guarded functions and scheduled events (Crafa et al., 2021, secs. 3–4). The original language in that paper is type-free; the typed decision representation proposed here is a separate design.

Read the rental example as a state machine rather than as a second glossary. The party identifies who may act; the agreement binds the terms; fields hold ordinary information; assets carry conservation obligations; the state gates which actions are available; a guarded function changes fields or resources and moves the state; and a scheduled event attempts a transition later if its state condition still holds. This mapping matters because a function's name alone cannot tell the reader whether a withdrawal, expiry or

payment has changed what that function means.

For example, the rental contract in the paper first records an offer. The borrower accepts by supplying the required payment, which moves the contract into its using state. The contract provides the use information and schedules an event for the end of the rental period. Either an appropriate borrower action or the scheduled event can finish the interaction and release the held payment. The language gives a precise account of which party can trigger each step and when the payment moves. Assets have special transfer behavior; they are not ordinary numeric fields that an assignment may freely duplicate (Crafa et al., 2021, sec. 3, rental example and grammar).

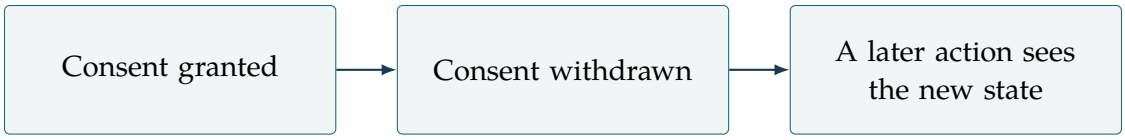


Figure 4.11: A consent history is a state problem, not a permanent client flag.

The banking analogue uses the state and actor ideas but does not transfer client assets. The following is explanatory pseudocode, deliberately distinguished from compilable Stipula syntax:

Listing 4.3: A consent model inspired by Stipula. "Record" observes an event; "allow" describes the control's decision.

```
parties: Client, Bank
terms: client_id, product_category, threshold, monitoring_mode
states: NOT_ESTABLISHED, ACTIVE, WITHDRAWN

NOT_ESTABLISHED:
    record Client.written_agreement(terms, evidence)
    when Bank.confirmed_required_explanations(evidence)
    -> ACTIVE

ACTIVE:
    record Client.withdrawal(instruction_id) -> WITHDRAWN
    allow Bank.apply_streamlining(transaction)
    when category_matches(transaction, terms)
         AND all_other_required_conditions_hold(transaction)

WITHDRAWN:
    do not allow Bank.apply_streamlining(transaction)
    record a new, fully evidenced agreement -> ACTIVE
```

The pseudocode makes two boundaries testable. The terms bind consent to a client, category and arrangement, so a new agreement after withdrawal creates a new consent generation rather than deleting the withdrawal. A transaction carrying the old generation must remain distinguishable from one carrying the new one. The regulatory duties still come from the applicable sources: client and bank agreement can establish

the consent contemplated by those sources, but it cannot amend an SFC requirement. The figure therefore supports a state-history test, while the source decision remains a separate review object.

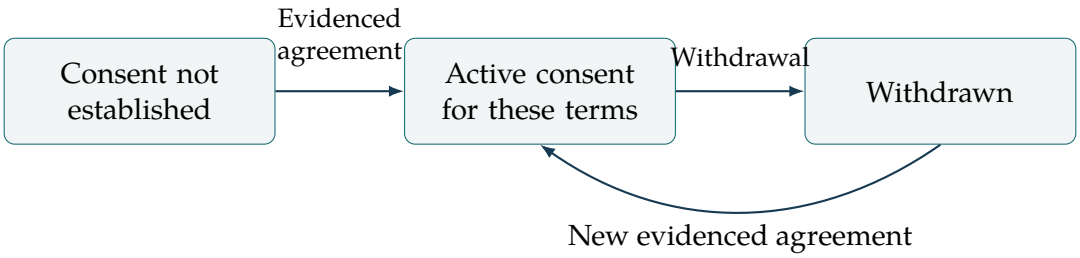


Figure 4.12: Consent transitions for the example. An incomplete event history produces an unresolved assessment outside these established states. It is not evidence of an active agreement.

Execution requires a state, stored terms and a history of ordered events. In Stipula's operational semantics, the runtime also maintains pending scheduled events and a clock. In the inspected 2021 semantics, due events take priority over function calls at the same time. That is a language rule, not a rule of Hong Kong banking law. LegalMath must adopt an explicit ordering policy for withdrawal and order events, record the authority for their sequence, and retain an unresolved result where the ordering cannot be established. It cannot import Stipula's clock convention as if the circular prescribed it.

There is a second adaptation. A control may prohibit streamlined treatment after withdrawal, but the bank might nevertheless have processed a transaction. Deleting that transaction from the history because it is an illegal transition would hide the very incident a compliance system needs to detect. LegalMath therefore separates the observed event record from the decision about whether the proposed action is permitted by the selected control. Normative languages such as eFLINT and Symboleo help make this distinction explicit: a duty or violation is represented separately from the fact that an action happened (van Binsbergen et al., 2020; Sharifi et al., 2020).

4.7 A small state contract that a reader can execute by hand

A complete miniature model helps distinguish the legal-language idea from its implementation in a particular tool. Consider a synthetic escrow agreement in which a payer deposits a fixed amount, a recipient may receive it after an accepted delivery, and the payer can recover it after an expiry if delivery has not been accepted. The example is chosen to teach states, parties, resources and time. It is not a model of an actual bank product or a proposed interpretation of the gift circular.

Let the contract state be one of `UNFUNDED`, `HELD`, `PAID` and `REFUNDED`. Let b be the amount held by the contract, p the payer's modeled external holding and r the recipient's holding. All amounts are nonnegative integer units. Let d be the agreed deposit, τ the expiry

time and t the current event time. The parties and the meaning of accepted delivery are agreed inputs to this model.

Start with a payer holding 100 units, an empty contract and an agreed deposit of 40. Depositing moves 40 from the payer into the contract, leaving holdings $(p, b, r) = (60, 40, 0)$. Accepted delivery before expiry moves those 40 units to the recipient. Expiry without prior release returns them to the payer. The contract needs to remember which event happened first so that it cannot transfer the same deposit twice.

The table makes those three movements exact. A *guard* is the condition that must hold before the action is allowed. A prime marks the value after the action: p' is the payer's new holding. The table uses neutral mathematical notation, not tested Stipula source code.

Table 4.1: Synthetic escrow transitions, inspired by state-contract languages.

From	Action and caller	Guard	Effect and next state
`UNFUNDED`	Deposit by payer	$p \geq d$, $d > 0$, $t < \tau$	$p' = p - d$, $b' = d$, $r' = r$; enter `HELD`.
`HELD`	Release by authorized acceptance process	Accepted delivery and $t < \tau$	$r' = r + b$, $b' = 0$, $p' = p$; enter `PAID`.
`HELD`	Expiry event	$t \geq \tau$	$p' = p + b$, $b' = 0$, $r' = r$; enter `REFUNDED`.
`PAID` or `REFUNDED`	Any repeat transfer	No enabled transition	No second transfer is permitted by this model.

In the numerical example, release leaves $(p, b, r) = (60, 0, 40)$ and state `PAID`; refund leaves $(100, 0, 0)$ and state `REFUNDED`. At the exact expiry instant, $t = \tau$, the release guard fails and the expiry guard holds. That endpoint is a stated choice in this miniature model, so the table settles a case that an informal arrow from "held" to "paid" would leave open.

A reader can now check a conservation property without trusting a compiler. Define $W = p + b + r$, the total modeled resource. For the deposit transition,

$$W' = (p - d) + d + r = p + r = W, \tag{4.3}$$

because the initial contract holding is zero. For release, $W' = p + 0 + (r + b) = W$. For refund, $W' = (p + b) + 0 + r = W$. Each permitted transition therefore preserves the total. Since the initial holdings are nonnegative and the deposit guard requires sufficient funds, these transitions also preserve nonnegativity. This is a local derivation about the stated miniature model.

The terminal states prevent a second transfer. After release, no transition from `PAID`

can release the same balance again. A Java implementation that updates balances but forgets the state change could violate that property under a repeated call. This is why the state transition is part of the contract, not just a user-interface label. An idempotency mechanism can support repeated requests, but must be specified consistently with the state model.

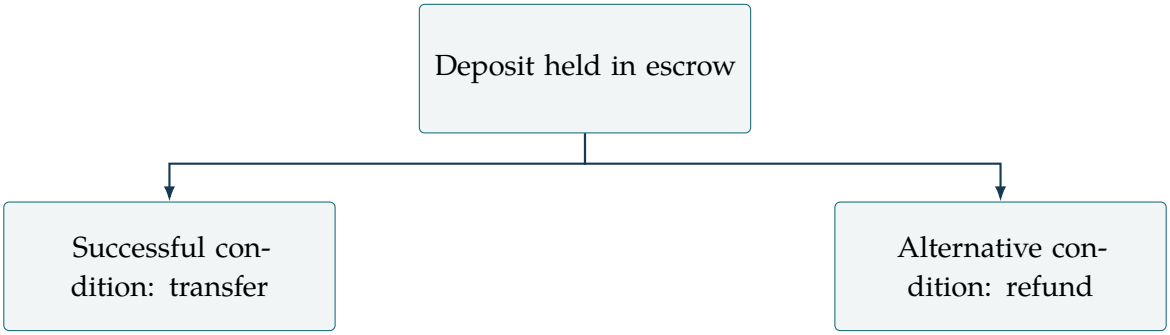


Figure 4.13: The escrow example separates authorized paths through the same resource state.

For a synthetic deposit of 100 units, a complete refund and a complete transfer cannot both spend the same held deposit. Follow one transition at a time and check the remaining balance. This illustrates the resource argument; the local contract's permitted transitions determine when either path can occur.

Time is equally substantive. The table uses $t < \tau$ for release and $t \geq \tau$ for expiry. At exactly τ, only expiry is enabled. If the intended agreement allowed delivery at the deadline, the guards would need a different boundary or an explicit ordering rule. A test performed one minute before expiry cannot establish the exact-deadline behavior. The Stipula literature's event ordering is therefore a modeling choice to understand, not a universal answer for every contract (Crafa et al., 2021).

The model also exposes a dependency hidden by ordinary prose: what establishes accepted delivery? It might be an authorized person's recorded acknowledgment, an external system event or a disputed factual assessment. The transition table assumes that this predicate is supplied correctly. Conservation and no-double- transfer proofs do not establish that assumption. If acceptance can be forged, the formal transfer properties may hold while the wrong person receives the resource.

The distinction between safety and progress is now visible. Conservation is a safety property: no permitted step creates or destroys the modeled resource. The ability to reach `PAID` or `REFUNDED` is a progress question. If the platform never processes the expiry event and delivery is never accepted, the resource can remain held. Proving eventual refund requires assumptions about time advancement and event execution, not merely the transition table's arithmetic. The contract-liquidity literature concerns such availability of actions and resources; it should not be confused with market liquidity (Laneve, 2023).

This miniature also clarifies what a Java/JML translation would need to preserve. A precondition specifies the caller, state and guard. A postcondition specifies the new balances and state. An invariant states nonnegative holdings and any conserved to-

tal. The verifier reasons about all allowed calls under those conditions. It must also account for the event mechanism and any loop abstraction used by the translation. The inspected Stipula-to-Java work provides a relevant route, with the restrictions already described (Hähnle et al., 2025).

For LegalMath, the corresponding benefit is a disciplined way to model review, consent and obligation lifecycles. A report awaiting review is not an approved report; a withdrawn consent is not an active generation; an opened duty is not fulfilled merely because a document was eventually delivered late. Explicit states and transitions make these distinctions executable and testable.

The correspondence is methodological rather than a claim that every circular is an escrow contract. Regulatory obligations can apply even when a party never agreed to a private contract, and actual events must be recorded even if a formal platform would reject them. A bank's monitoring system therefore separates the normative question of what should occur from the factual record of what did occur. The state model helps express that separation only when it is designed for the intended purpose.

A practical specification exercise follows directly. Before implementing a new history-dependent control, write its states, actors, events, guards and effects; execute one normal path, one boundary path and one violating observation by hand; identify the facts that the model assumes; and state which invariants and progress properties matter. Then choose a language or verifier that supports those properties. This sequence gives Stipula, Symboleo, eFLINT or a restricted custom monitor a concrete role rather than selecting a language because its name occurs frequently in the literature.

4.8 What contract verification does and does not buy

Once a contract has a transition semantics, tools can ask whether certain states are reachable, whether a resource can become trapped, or whether two contracts have corresponding observable behavior. Stipula's legal-bisimulation work gives a formal account of selected observable equivalences (Crafa et al., 2021, behavioral equivalence section). A bisimulation relates states so that an observable step on one side can be matched by a corresponding step on the other. It is a statement about modeled behavior under that definition of observation.

For the bank, an analogous question might ask whether a revised consent implementation preserves the reviewed behavior for grants, withdrawals and transactions. The observation set must include what matters: subject, category, time, permitted use and recorded violations. If the comparison ignores withdrawal, equivalence of the remaining observations would not establish the property the bank needs. Formal observation boundaries deserve the same scrutiny as ordinary input-domain restrictions.

Hähnle et al. (2025) demonstrate a translation from Stipula contracts into Java with JML specifications and verification using KeY. JML expresses properties of Java behavior, such as preconditions, postconditions and invariants; KeY reasons about whether the Java program satisfies them. This is especially relevant to our Java delivery target be-

cause it illustrates a route from a specialized contract language into a general-purpose verification environment.

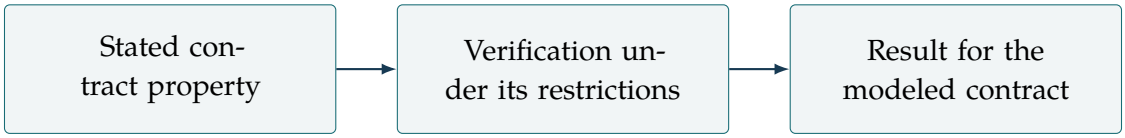


Figure 4.14: A contract proof inherits the model's expressiveness and assumptions.

The inspected work has explicit restrictions. Its event conditions are modeled symbolically, and the automated encoding keeps certain loop inputs constant across iterations. Its examples establish feasibility for the examined models, not complete fidelity to arbitrary calendars or external event streams. The official translator source was inspected in the earlier project review; no KeY proof was replayed. Adopting a similar route would require a precise mapping from our RuleIR and event profile to the Java/JML semantics, including the unknown and conflict results that ordinary Boolean contracts omit.

General verification cannot be promised for every expressive contract language. Delzanno et al. (2026) establish undecidability results for clause reachability in Stipula fragments and identify a decidable fragment under particular restrictions. The restrictions concern disjoint function/event source states and immediate events; they should not be confused with the different disjoint-cycle condition used in the Java/JML translation work. A shared word in two papers does not identify a shared theorem.

The practical consequence is to define bounded questions and supported fragments. A tool can report that no bad state was found up to a specified depth, or prove a property under a restricted transition model. It cannot turn a finite search into a complete reachability decision for a language whose general problem is undecidable. The report must retain the bounds so a reviewer can understand which behaviors were left unexplored.

Stipula's amendment literature is also relevant. A contract can change its declarations, state and permissible behavior through a specified amendment mechanism (Laneve et al., 2023, secs. 3–5). For regulatory controls, an amendment must similarly identify whether existing obligations keep their opening source version or migrate to a new one. Hot-replacing code without a state-migration interpretation can change the treatment of historical events. The versioned control and its transition obligations need separate approval.

4.8.1 From a consent property to a Java verification model

A property is a precise question about the model. For the consent example: "After a known withdrawal, and before a new qualifying agreement, this control never returns that the streamlining conditions are met." Testing checks selected histories. Model checking explores all histories within a specified finite model. A mathematical proof establishes the property under explicit assumptions. None establishes that a real client signed a document, or that the selected provisions are the complete legal regime.

Stipula's behavioral-equivalence work compares the observable interactions of two formal contracts. Two implementations can use different internal state names yet be equivalent if the relevant party actions, communications and asset behavior match. This is useful for comparing formal implementations. It is not a test that the English circular and the contract mean the same thing (Crafa et al., 2021, sec. 5).

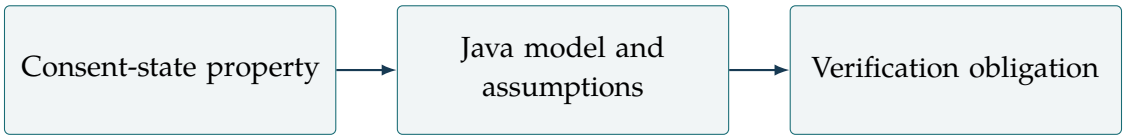


Figure 4.15: A Java verification model must preserve the event meaning it is meant to check.

The Stipula–KeY work supplies a particularly relevant connection to Java. Its translator builds Java verification models from a supported class of Stipula contracts and adds specifications in the Java Modeling Language (JML). KeY reasons about Java programs and these specifications. A JML precondition states what must hold before a method; a postcondition states what the method must establish; an invariant states a condition preserved at the designated program points. For instance, an illustrative postcondition can require a withdrawal method to leave consent inactive, while an invariant can prevent a successful streamlining decision in that state. The proof obligation concerns all executions covered by the model and its assumptions (Hähnle et al., 2025; Stipula contributors, 2026b, secs. 3–5).

That translation is a useful research route, but its Java verification model is not automatically a deployable bank integration. The paper restricts cycle structure and abstracts some event conditions; the adapter still has to preserve timestamps, evidence and the bank's actual interface. The concrete Java demonstration in this proposal instead compiles the stated decision subset directly. It compares Java with a separately implemented reference evaluator and source-derived examples. A later KeY or Lean task can strengthen the compiler evidence for a chosen property without changing who approves the legal interpretation.

4.8.2 The evidence and restrictions behind Stipula verification

Stipula makes parties, agreement, state, assets, permitted functions and scheduled events explicit. Its semantics describes how a contract evolves when parties act and time advances. This is well aligned with consent, withdrawals, review events and requests for evidence. It also provides a notion of legal bisimulation, relating contracts through their observable interactions (Crafa et al., 2021, secs. 3–5).

The important borrowing is the state model, not a presumption that bank compliance requires a blockchain. The original paper discusses centralized as well as distributed execution. A bank can record a source-linked event history in its ordinary systems. Moreover, a similarity of formal interaction does not establish equivalence of the source texts as a matter of law. The accompanying discussion of legal calculi highlights how

executable arrangements interact with external intervention and legal choices that code can otherwise suppress (Crafa et al., 2021; Crafa, 2022).

Several semantic choices need explicit review. In the studied Stipula semantics, scheduled events take precedence over function execution at the same time. That is a clear rule for the language; it is not an automatic answer to the bank's cut-off or consent-ordering questions. The prototype should construct a trace in which a withdrawal and a transaction have the same recorded time, and identify the finer ordering evidence or adopted policy required to decide it.

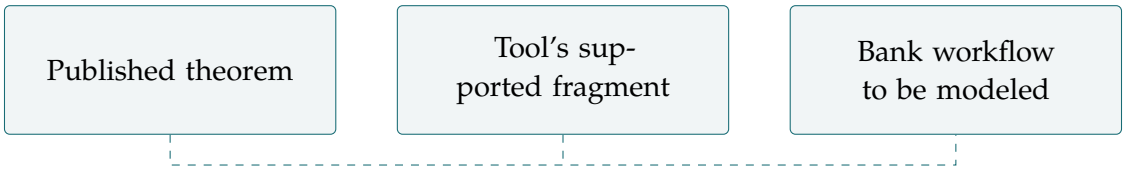


Figure 4.16: The theorem's restrictions must survive the proposed transfer.

The Stipula-to-Java/JML work offers a concrete verification route. It translates contract structure into Java together with specifications and applies KeY to properties of the result. The authors report automatic verification of four contract examples for the selected translation and properties. This is reported feasibility evidence whose exact executable inputs must still be reconciled with the appendix. The inspected implementation includes a translator and generated examples (Hähnle et al., 2025; Stipula contributors, 2026b, secs. 3–4 and apps. A–B). The retained extended version also contains a concrete reproducibility problem. Appendix B.3 declares Boolean parameters in methods that add those parameters to integer asset balances. Java rejects that arithmetic; the printed example is not compilable as written. A rendering check confirmed the declarations, and a minimal compiler diagnostic confirmed the type error. The pinned official repository also contains the same Boolean parameters in `Deposit.java`; compiling that complete file produces eleven Java type errors (Stipula contributors, 2026b). This does not determine what files were used for the reported experiment. Adoption requires the exact verified sources, correction of the discrepancy and a replay of the proof obligations (Hähnle et al., 2025, app. B.3, pp. 31–32).

This is worth a bounded experiment if workflow verification proves important.

The encoding also defines the limit of the evidence. Event conditions are abstracted through symbolic Boolean values rather than a complete implementation of banking calendars. Automated handling of the relevant loops uses inputs that remain constant across iterations. A proof about that model does not establish correctness for arbitrary changing portfolios, unbounded event queues or all real-time orderings. LegalMath must state and test any adapter's correspondence to its own event semantics before using the proof in a compliance package.

The 2026 reachability paper supplies a direct design constraint. Through counter-machine reductions it establishes undecidability for general and several restricted microStipula fragments. It also identifies a decidable fragment, called DI, with both disjoint function/event source states and immediate events, using a well-structured

transition-system argument (Delzanno et al., 2026, secs. 2–4). DI is not the same condition as the disjoint-cycle construction in the Java/JML paper.

The product implication is to ask bounded, named questions. For example, can an approved rule set permit streamlined treatment after a recorded withdrawal in any trace of at most k events under a specified ordering model? A counterexample is useful immediately. Failure to find one supports only the checked bound unless an additional invariant proof establishes a broader claim. A timeout or unknown result must remain unresolved. The research does not imply that a small acyclic financial decision is intractable; it limits the promise of a universal verifier for unrestricted event-based contracts.

4.8.3 Amendment and resource analysis

The amendment literature asks what happens to a running agreement when its code changes. Higher-order Stipula lets an amendment remove declarations, add declarations and run a transition, changing both program and current state. Restrictions can preserve parties, constant fields or asset conditions; an agreement predicate can require relevant parties to accept the change (Laneve et al., 2023, secs. 4–6). The retained source is the 2023 author manuscript *Legal Contracts Amending with Stipula*. It is not mislabeled as the different 2024 chapter *Programming Contract Amending*, whose full text was not successfully retrieved.

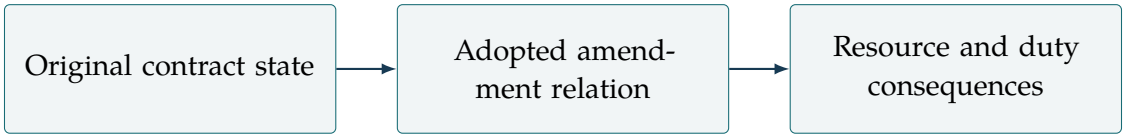


Figure 4.17: Amendment changes more than the text of a condition.

This poses a precise SFC transfer question. A new circular can change future decisions while existing consent arrangements or open duties need a separately justified transition. A replacement marker alone cannot decide that migration. The MVP creates an immutable new rule version and retains the version that opened each obligation. Any migration is an explicit reviewed operation. Runtime code replacement is unnecessary for the initial prototype and would make historical replay harder to explain.

The platform paper describes a visual editor, interpreter, type inference, unreachable-clause analysis and liquidity analysis (Laneve, 2026, secs. 3–7). Its liquidity property concerns resources that might remain trapped in a program. The underlying type analysis represents asset effects symbolically; cheaper sufficient tests differ from more precise bounded exploration (Laneve, 2023, secs. 4–5 and proof appendices). This is program-resource liquidity, not market or funding liquidity. The platform account assumes cooperative, strategyless behavior for the discussed property and states that the liquidity implementation is still being developed. That is the paper's historical status: the separately retained workbench repository includes a Python liquidity analyzer and a configurable bound on function repetitions (Stipula contributors, 2026a). Source inspection

confirms its presence, but neither its correctness nor production suitability has been established by execution here. Borrowing editor and explanation ideas is therefore a different commitment from claiming a production verifier for all bank workflows.

4.9 Other normative languages clarify different distinctions

Symboleo models legal contracts using concepts such as obligations and powers, with lifecycle distinctions that are easy to lose in a Boolean implementation (Sharifi et al., 2020, secs. II–IV). Creating a duty is not always the same as activating it. A power can change a normative relationship, for example by suspending or terminating an obligation. These distinctions are useful when a bank workflow has review, activation, withdrawal and corrective actions with different legal effects.

The retained Symboleo paper and software inspection do not provide a complete implemented axiom set for LegalMath. Some supporting references in the early paper are not fully available in the retained edition. We can use the lifecycle questions to improve requirements, while treating any direct engine adoption as a separate interface and semantics audit. A familiar conceptual diagram is not a substitute for an executable contract.

eFLINT separates facts, actions and duties in a normative specification (van Binsbergen et al., 2020, secs. 3–5). A particularly useful distinction is that an impermissible action may still occur, change state and create a violation. This matches the difference between a bank's pre-transaction advice and its post-event audit. A system that refuses to represent an unauthorized observed action cannot explain what happened after a control failure.

To understand a stable model, consider the two rules "accept if rejection is not established" and "reject if acceptance is not established". One self-consistent interpretation contains acceptance; another contains rejection. Selecting the first answer chooses between them without additional legal authority. In answer-set programming (ASP), a solver enumerates such stable interpretations after accounting for the program's default-negation rules. Absence of a stable model can indicate inconsistent constraints, while several models can leave the answer dependent on the chosen interpretation. This miniature example explains the backend issue without equating stable-model semantics with the three-valued fact evaluation used by LegalMath.

Clingo is an answer-set solver: given a logic program, it searches for stable models satisfying that program. It is the execution engine used by the eFLINT translation discussed below.

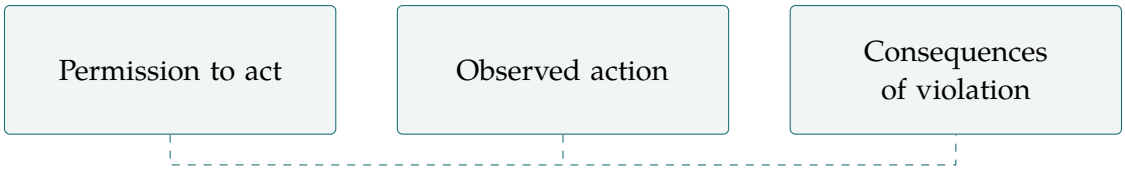


Figure 4.18: A normative model can record an action that was not permitted.

The eFLINT literature also shows why language versions matter. The later stable-model semantics provides a particular formal grounding and can produce no model or several models (Esterhuyse et al., 2025b, translation and evaluation sections). Those outcomes carry information; selecting one model silently would conceal ambiguity or inconsistency. The newer language survey discusses implementations whose semantic guarantees differ (van Binsbergen et al., 2026). A proof or test result for one version should not be attributed to every interpreter bearing the same name.

s(LAW) combines legal formalization with logic programming that can explain positive and negative outcomes and explore possible models (Arias et al., 2024, secs. 2–5). This makes assumptions visible in a way that a single opaque classification may not. However, a model containing a hypothetical fact does not establish that the fact is true for the client. Counterfactual reasoning should propose what would change an answer, while the evidence service establishes whether the corresponding condition actually holds.

These methods suggest an interface in which normative classification, state transition and factual evidence are separate inputs to a decision. The implementation should reject an unsupported duty kind explicitly. A maintenance obligation, which must hold throughout an interval, cannot be implemented by checking that it was true once. A non-preemptive achievement obligation cannot be discharged by a performance before it came into force. The taxonomy in Hashmi et al. (2016, sec. 3) helps identify these choices; the precise profile still needs a source-grounded interpretation.

4.9.1 Authority, default negation and competing normative backends

LegalRuleML is relevant because it treats legal meaning as more than a Boolean expression. The standard addresses modalities, defeasibility, temporal aspects, sources and contextual information. Earlier work on legal interpretations shows how alternative readings and their contexts can be represented rather than collapsed prematurely into one rule (Athan et al., 2014, sec. 4). The standard describes the relevant requirements and contextual annotations in sections 2.2–2.3 (OASIS LegalRuleML Technical Committee, 2021). This is a strong basis for LegalMath's metadata: every executable condition should identify its source, scope, adopted interpretation and relationship to other rules.

The standard is an interchange representation, however. XML that names an obligation does not by itself define the event processor that creates, satisfies or breaches it. LegalMath should adopt a deliberately small execution profile with explicit meaning, and use LegalRuleML concepts for interoperability. Requiring compliance officers to review raw XML would not solve the bank's usability problem.

s(LAW) illustrates a different benefit of formal legal reasoning: explaining conclusions under alternative factual assumptions. Its use of s(CASP) combines logic-programming rules, explicit negative information and default negation, and produces justifications. The school-admission examples demonstrate discretionary conditions and hypothetical cases, including assumptions that affect which conclusions can be derived (Arias et al., 2024, secs. 4–5).

This distinction matters when an SPI record lacks an agreement or a qualification. A hypothetical answer can say which additional fact would establish a condition. It cannot treat that fact as already established. LegalMath should display "would hold if this evidence were confirmed" separately from an evaluated client fact. Different treatments of negation must also remain explicit: absence from a database, a documented negative statement and a defeasible assumption are three different things. The s(LAW) examples support explanation and exploration; the linked implementation could not be fully audited in this review.

eFLINT contributes an especially useful operational distinction. A prohibited or otherwise impermissible action can still occur in the real world and have legal effects; the system must be able to record the resulting violation. A pure authorization engine that simply removes such transitions cannot audit what actually happened. The language's duties, actions and open types are relevant to missing evidence and ongoing controls (van Binsbergen et al., 2026, secs. 3–5).

The 2026 eFLINT account also describes differences between implementations and versions. It notes the absence of a formal operational semantics and type system for version 4, and limitations concerning fine-grained explanations and normative priority; another implementation has a stable-model foundation. Guarantees must therefore be attributed to the actual version and engine used (van Binsbergen et al., 2026, secs. 6–7). The prototype should borrow the distinction between permission, occurrence and violation without assuming that installing an eFLINT interpreter supplies all the required proof evidence.

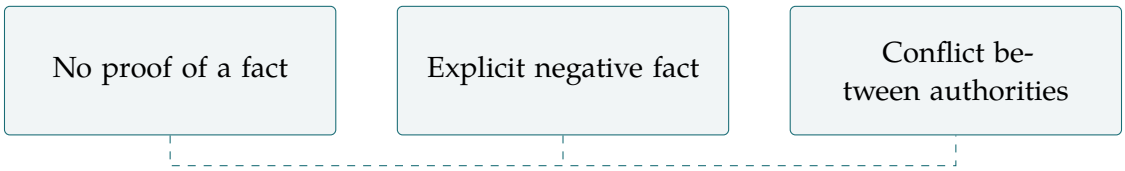


Figure 4.19: Different normative backends distinguish absence and negation differently.

The original eFLINT paper makes the operational idea concrete through a transition-system interface: declarations establish institutional concepts, events change a configuration, and queries inspect permissions and duties. Action-compliance and duty-compliance are independent. Its GDPR example forms part of a small data-sharing/KYC exploration with ABN AMRO and ING, using separate policy and agreement specifications. This is relevant domain evidence, although it is a selective experimental case rather than a deployed private-bank control system (van Binsbergen et al., 2020, secs. 3–5). Operational connectors qualify observations into facts; the

normative engine evaluates those facts; a reviewer owns the interpretation connecting them.

The stable-model implementation supplies a sharper warning about backend substitution. Esterhuyse et al. (2025b, secs. 4–7) encode states and transitions in Clingo and translate a restricted normal form of eFLINT clauses. The translation makes state explicit, normalizes aggregation and permits scenario search as well as checking a supplied scenario. Section 8 compares 108 specification–scenario pairs over 43 specifications. Agreement with the earlier interpreter is expected only where their semantics coincide; default reasoning is deliberately changed.

Their Example 9 shows how a prior implementation can retain a conclusion based on absence even after further facts invalidate that absence. Examples 10 and 11 distinguish no stable model from multiple stable models, while the older engine produces a particular result. A future ASP adapter that selects the first model silently would therefore choose an interpretation policy. LegalMath 0.1 avoids that problem by rejecting recursive definitions and defining unknowns directly. Richer semantics remain an option when the selected regulatory scope requires them. Pinning a package name alone does not pin the meaning of a rule.

The accompanying eFLINT release makes these limits inspectable: it separates tests expected to agree from tests expected to differ, and documents excluded cases involving cyclic aggregation and unsatisfied models. The compiler's `Clingo.hs` also keeps action violations separate from duty violations. The source was inspected, not compiled or benchmarked. This supports a small differential test before adopting an engine, rather than extrapolation from the paper's overall claims (Esterhuyse et al., 2025a).

Symboleo contributes an explicit lifecycle for obligations and legal powers. A trigger creates a conditional obligation; its antecedent activates it; later events can fulfill, violate, suspend or terminate it. A power concerns the ability to change these legal positions, which differs from a duty to act. The initial paper gives statecharts, temporal predicates and selected axioms, with a sale-of-goods example checked using a Prolog prototype (Sharifi et al., 2020, secs. III–IV). It is preliminary feasibility evidence; the retrieved preprint does not contain all 27 referenced axioms. The official editor and model-checker repositories provide concrete examples and a nuXmv route, but were not executed (Smart Contract Modelling, University of Ottawa, 2026). The borrowing for LegalMath is to distinguish obligation creation, activation and performance. Suspension and legal powers require an extension to the MVP's simpler profile.

4.10 Decision tables and business-rule engines

Decision Model and Notation (DMN) is a standard for describing decisions, including decision tables. A table row associates input conditions with an output; its *hit policy* specifies what to do when more than one row matches. These definitions let us ask whether the table preserves the intended exception priority.

Decision tables remain useful when their rows and hit policy have a precise meaning.

A table can show, for example, that a known discount exception changes the selected trigger while an unknown exception leaves a material question. The reader must know what happens when several rows match: choose the first, require a unique row, collect results or apply a priority. DMN makes such hit policies explicit (Object Management Group, 2024, sec. 8.2.11).

That explicitness is the valuable part. A spreadsheet whose row order silently resolves competing legal exceptions is harder to review because its legal priority is hidden in presentation. Changing the sort order can change the answer. A unique-hit table can reject overlap, but it cannot determine whether the missing row represents an omitted legal situation. Independent source coverage and boundary cases remain necessary.



Figure 4.20: A table needs an explicit rule for multiple matching rows.

If two rows match the same client, a first-match policy and a collect policy compute different outputs. Visual neatness does not choose between them. The bank must adopt the intended meaning before the table becomes an interchangeable backend.

Drools, OPA and other rule engines can execute policy logic and integrate with enterprise applications. The question for this product is not whether they are capable software. It is whether the selected engine's semantics, evidence model, explanation behavior and deployment interface match the adopted specification. Unknowns, conflicts, exception priorities and event ordering need explicit mappings. A backend name is not an assurance claim.

For the prototype, a small RuleIR is easier to specify and compare than a large engine with many unsupported features. Other engines can serve as independent reference encodings for a supported fragment after that fragment is defined. The comparison must include the behavior that ordinary Boolean examples leave out. Engine diversity without semantic alignment can produce many discrepancies that reflect adapter choices rather than flaws in the legal interpretation.

4.10.1 Software adoption and interchange standards

The open-source landscape is sufficiently rich that a prototype need not build every component. It is not sufficiently uniform that selecting one package settles semantics, review, execution and proof at once. Table 4.2 identifies a practical role for each candidate. These are design recommendations based on the inspected papers and official documentation; installation, performance, security and integration have not been tested here.

The candidates fall into four useful families. The point of the comparison is not to choose the most famous package. It is to ask which distinction the package can preserve,

and which part of the bank's problem would still require a separate record or review. The compact table gives that map; Appendix E of the technical companion retains the candidate-by-candidate dispositions and citations.

Table 4.2: What the main language families can contribute.

Language family	What it contributes to the bank's control
Source-linked rule languages	Make definitions and exceptions explicit while retaining their source passages. The bank still supplies the evidence policy and event history.
Normative and contract languages	Represent actors, duties, permissions and changes of state. Reviewers still settle the legal reading and its relation to bank records.
Decision and policy engines	Execute rules as tables or services. Their treatment of overlaps, missing facts and explanations must match the adopted rule.
Solvers and proof assistants	Find counterexamples or establish specified mathematical properties. Their conclusions depend on the rule and assumptions supplied to them.

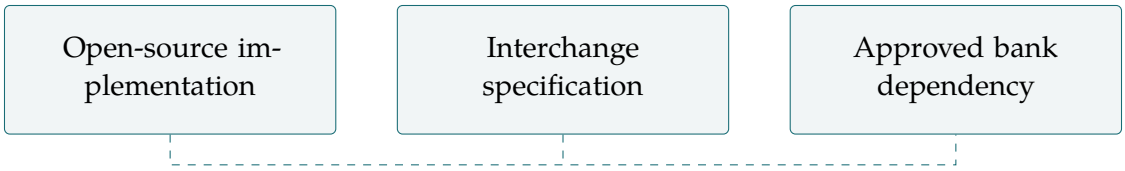


Figure 4.21: Availability, interoperability and adoption are separate questions.

L4 is a language and software environment for expressing legal rules. Its authoring tools help people write and inspect a rule; its execution tools evaluate that rule on supplied facts. That separation makes it relevant to both the reviewer's task and the engineer's task.

L4 is a serious alternative to building the authoring environment from scratch. Its inspected repository exposes a parser, type checker, evaluator, IDE, decision service, generated schemas and graphical traces. Its regulative `DEONTIC` documentation names actor, action, deadline, successful continuation and breach continuation, and distinguishes `MUST`, `MAY` and `SHANT` (Legalese contributors, 2026). The 2023 L4 paper on deontics and time was identified but its full text was not retrieved, so no theorem from it is used here. Before adopting L4 as a core dependency, test exact money, missing evidence, simultaneous events and historical replay against the RuleIR cases. The proposal fixes that comparison target rather than making development wait for another language-selection exercise.

DMN deserves particular attention because its accessible tables can conceal a priority choice. Under a Unique hit policy, matching rows must be disjoint; Any permits overlaps with the same result; First selects by row order. Those policies are not interchangeable. A default exception tree imported from Catala cannot be exported as an arbitrary First table without checking that the meanings agree (Object Management Group, 2024, sec. 8.2.11). Review annotations also need their own source store: a visual note on a table is not automatically available as runtime provenance.

Licensing and reproducibility are separate from public availability. Catala, Apache KIE, OpenFisca, Z3 and the other public projects are candidates for reuse, but the precise pinned version, dependencies and intended distribution must be reviewed. A paper PDF, a hosted service and a public research repository carry different rights and maintenance expectations. The Stipula and tax prototypes should not be described as approved bank components merely because their source can be inspected. No third-party implementation has been imported into LegalMath as part of this document work.

4.11 A solver can improve reasoning after a translation

A neural-symbolic system separates tasks that language models and formal engines perform differently. A language model proposes a formal representation of prose. A symbolic engine evaluates or reasons about that representation. The engine can give a reproducible answer to a precise logical question and can expose inconsistency, unsupported syntax or a counterexample. The translation that supplied its premises remains a separate source of error.

Logic-LM uses language-model formulation followed by an appropriate solver and feedback-driven refinement (Pan et al., 2023, sec. 3 and prompt appendices). LINC combines language-model formalization with first-order theorem proving and analyzes failures at the translation and reasoning boundary (Olausson et al., 2023, method and error analysis). These methods support the design choice that a model should not be asked to simulate every logical calculation in prose. They do not support treating successful solver execution as evidence that a legal requirement was fully captured.

A syntax repair illustrates the benefit and the limit. Suppose a candidate uses an undeclared symbol in the gift rule. The parser identifies the symbol and rejects the encoding. A repair prompt can ask for the missing declaration or corrected reference while preserving the intended interpretation. The repaired candidate is then validated again. This process may solve a concrete representation error. It does not justify adding a new legal assumption merely because doing so makes the formula easier to satisfy.

ARc develops policy-model creation, human vetting, structured inspection and symbolic checking in a combined system (An et al., 2025, sec. 3 and appendix A). Its policy creation procedure divides a document into units, proposes types, variables and constraints, and composes them into a model. Its vetting mechanisms include linting, template-based English inspection and testing. These are relevant to our product because they make the proposed model available for correction before individual answers rely on it.

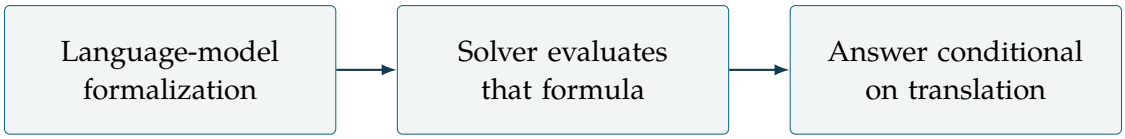


Figure 4.22: A solver can reason correctly about an incorrectly translated question.

Composition creates its own interpretation problem. Two units may use the same word for different legal concepts, or different words for the same concept. ARc uses embedding-based clustering in its variable-unification procedure. For our high-stakes specification, such a merge should be a proposal with provenance and validation. Merging "offer discount" with "component discount" because their descriptions are similar could remove the distinction that g04 was designed to test. A similarity score cannot establish identity of legal concepts.

ARc's answer-verification procedure also uses redundant translations and symbolic agreement checks. Its reported finding categories distinguish ambiguous translation, impossible premises, invalid conclusions, entailed conclusions and satisfiable but unentailed claims. The distinction is useful: if a model permits both an outcome and its opposite under incomplete premises, the system can show scenarios for each rather than invent a definitive answer. For LegalMath, these scenarios become review questions tied to the candidate set and the source context.

The word "soundness" in ARc's empirical evaluation needs careful translation into a product claim. The paper defines an empirical quantity based on false positives across requests; it is not a theorem that the English policy was formalized correctly. The denominator differs from precision among approvals. The manuscript therefore borrows the inspection and disagreement mechanisms without importing a claimed legal-error probability. Chapter 9 defines the denominators our own evaluation would need.

The tax-reasoning work of Jurayj et al. (2026) further illustrates the importance of task boundaries. Supplying gold rules, retrieving annotated cases and extracting facts are different assistance conditions from generating rules from a new circular. A comparison must account for those inputs and the human effort that produced them. A system given a reviewed rule library should not be presented as evidence that the same system can independently interpret unreviewed prose.

4.11.1 Language models can propose formalizations; their inputs remain the weak point

The SARA tax-reasoning dataset makes explicit the ingredients an evaluation needs: statutory text, cases, formal rules and expected conclusions. Its corpus uses selected and simplified US tax provisions and constructed examples. It is a valuable experimental design, but neither a current tax authority nor a test set representative of SFC circulars (Holzenberger et al., 2020, secs. 2–3).

The later work connecting statutory reasoning to information extraction associates facts with spans in case descriptions and evaluates their effect on reasoning. This

is a useful model for recording where a client attribute came from, which person it describes and how it was normalized (Holzenberger and Durme, 2023, secs. 3–5). Its task-specific date imputations and semi-synthetic construction require caution: an extraction convention suitable for that dataset is not an evidential rule for missing bank records (Holzenberger and Durme, 2023, sec. 7 and apps. B, E).

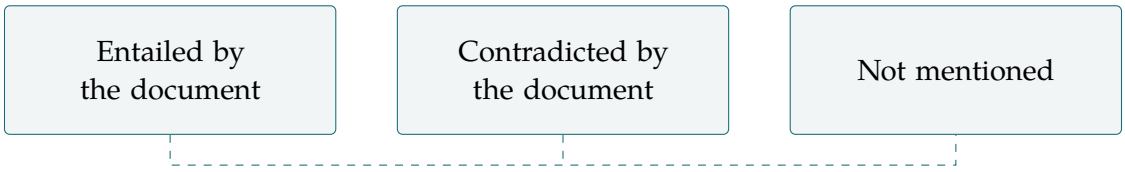


Figure 4.23: Evidence-based classification preserves the difference between contradiction and silence.

ContractNLI adds a complementary annotation pattern. It separates supported, contradicted and not-mentioned hypotheses, with evidence spans in contracts. Its 607 nondisclosure agreements and fixed hypotheses expose the importance of exceptions and evidence spread across a document. It supplies useful ideas for an evidence-review interface, while evaluating document inference rather than compilation of regulatory rules (Koreeda and Manning, 2021, secs. 2–5).

Logic-LM and LINC both divide language interpretation from symbolic inference. Logic-LM routes formalized problems to solvers and uses error feedback to refine malformed programs. LINC translates into first-order logic and uses a theorem prover, with multiple samples and voting in its experiments (Pan et al., 2023, sec. 3) (Olausson et al., 2023, secs. 2–5). The transferable idea is a clear boundary: the model proposes the statement, and the solver reasons over that statement. Their benchmarks do not establish legal translation fidelity. Successful parsing can leave a wrong quantifier untouched, and repeated models can agree on the same missing exception.

The inspected Logic-LM repository also offers fallback strategies when formal execution fails, including a language-model answer or random selection for its benchmark setting. Those are not acceptable defaults for this prototype's regulatory decisions. A failed formalization should produce an unresolved issue with a reason. Research code can be valuable while requiring a different operational policy (LegalMath research notes, 2026).

The ARc paper is especially close to the proposed product. It separates creation and human vetting of reusable policy models from subsequent verification of natural-language material. Its workflow uses source passages, variable identification, repeated translations and deterministic explanations. The verification algorithm checks consistency and relationships between the policy, premises and claim, and distinguishes invalidity, validity, satisfiability and several forms of inability to decide (An et al., 2025, sec. 3 and Algorithm 1).

The method can be understood with three formulas. Let M be the reviewed policy model, P the premises extracted from a message and C its proposed conclusion. For example, the message may say that a client has a HK$40 million qualifying portfolio and

therefore satisfies the financial condition. The verifier first checks whether M allows the premises at all. If $M \wedge P$ has no possible assignment, the premises are impossible under that model; declaring the conclusion true by vacuous implication would be misleading. With consistent premises, if $M \wedge P \wedge C$ has no assignment, the conclusion is refuted. If $M \wedge P \wedge \neg C$ has no assignment, it is entailed. If both have assignments, the premises leave the conclusion undecided; the assignments show which extra facts distinguish the outcomes. For a purported complete SPI assessment based only on wealth, those missing facts could include objectives and knowledge. These checks reason within M; they do not reconstruct an omitted circular paragraph.

ARc's redundant-translation step addresses an earlier source of uncertainty. Several translations of the same message produce candidate premise/conclusion pairs. Algorithm 1 counts the fraction of translations that support a pair's implication while not contradicting its premises. It does not merely compare the strings or require formula identity. A confidence threshold can make the system abstain and display competing readings. LegalMath can borrow that review interaction, but should preserve the candidate formulas and a distinguishing client example. Agreement among generated candidates is evidence of agreement, not independent proof that the interpretation is correct.

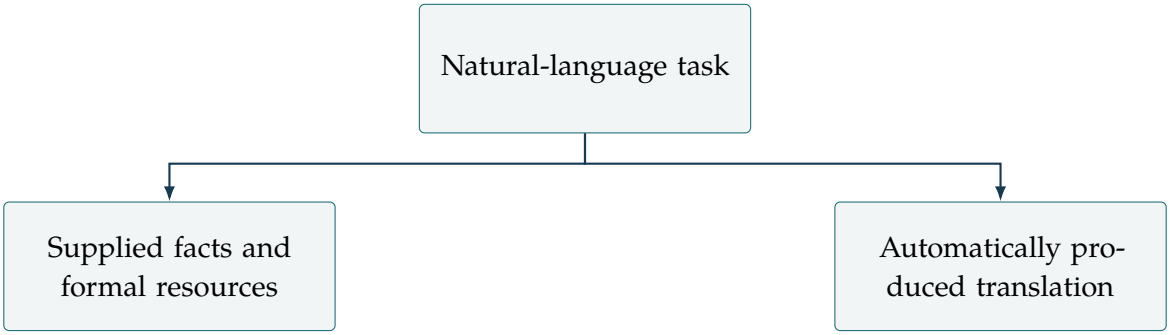


Figure 4.24: The information supplied to a benchmark limits what its success establishes.

This is a strong precedent for an interpretation workspace with explicit abstention. It is also a warning about terminology. The paper's reported "soundness" metric is $1 - \mathrm{FP}/N$, an empirical rate over its decisions; it is not a theorem establishing the translator's semantic soundness. At the 3/3 threshold in Table 1, 10 false positives among 1,689 decisions yield about 99.4 percent on that metric, while reported precision is 94.4 percent and valid recall is 14.9 percent. The 563 items were each run three times, so the total is not 1,689 independent policies (An et al., 2025, sec. 4, Table 1).

Those results motivate measuring both errors and abstention. A cautious system may produce few confident errors by deciding very little. Its business value then depends on whether the unresolved cases become easier to review. The ARc paper also discusses difficulties with cross-references, tables and implicit assumptions, all present in the bank's problem (An et al., 2025, sec. 5 and app. A.1). Its semantic comparison of candidate translations should not be simplified into a claim that independent generation proves equivalence. ARc is a published commercial-system comparator, not an open-source implementation established by this review.

The requested AAAI paper on trustworthy tax reasoning evaluates combinations of language models, facts and logic programs, including reviewed statutory rules and examples. It provides a particularly useful distinction for prototype design: reasoning with gold rules is a different task from generating the rules. The human work needed to prepare those rules and examples belongs in the comparison (Jurayj et al., 2026, 2025, Tables 2–3). The reported setting uses 100 numerical tax cases; its application-specific penalty for errors and abstention cannot be imported as a Hong Kong bank's cost model.

In implementation terms, the tax method separates at least three jobs: encode the statute as a reusable logic program, extract the current case's facts, and execute the program to obtain the requested amount. Supplying human-reviewed rules removes the first job from the model's evaluation. Supplying formalized facts also removes much of the second. The corresponding LegalMath experiment must therefore hold the source packet and human preparation budget fixed, and report separately rule errors, fact-mapping errors and execution errors. A Java calculation can be exact while still using a net-asset input that includes the home; scoring only the final answer hides which stage needs repair.

The inspected companion code confirms that extraction and Prolog processing are separate stages and that execution failure can lead to abstention. LegalMath should adopt that separation and evaluate facts and rules independently. It should not quote a tax benchmark's best configuration as evidence that the same method will reduce private-bank compliance errors. The reading notes also retain a discrepancy between the paper's narrative attribution of a cost result and the configuration shown in its table; the proposal relies on the methodological distinction rather than that ranking (LegalMath research notes, 2026).

The recent *Know Your Limits* preprint studies failures in legal reasoning and autoformalization, including assumptions that change the intended entailment task. Its examples strengthen the case for testing entity binding, implicit assumptions and distinctions between legal interpretation and strict entailment (Wang et al., 2026, secs. 3–5). It remains under-review evidence. Inconsistencies in the reported dataset denominators require clarification before quantitative reuse, and the choice of which ordinary or legal assumptions are admissible is itself a modeling decision. This proposal borrows its failure taxonomy, not a general error-rate estimate.

LegalBench broadens evaluation beyond a single reasoning dataset. Its tasks separate several kinds of legal reasoning; the evaluation appendix gives task-specific procedures. For rule application, legal reviewers check both correctness and whether an explanation contains the inferences connecting the facts to the result (Guha et al., 2023, sec. 3 and evaluation appendix). Repeating a circular accurately is not an explanation of its application to a client. Nor can a benchmark score become a banking acceptance criterion: the numerical SARA task permits error within ten percent, whereas one cent can change the intended threshold predicate. The limitations appendix identifies US-law, language, document-length and ambiguity gaps. LegalMath borrows independent answer guides and error analysis while constructing exact boundaries, source-dependency tests and amendment histories for its own task.

4.12 Roundtrip verification and targeted repair

Roundtrip verification asks whether a formal representation survives a journey through another representation. In the method of Amrollahi et al. (2026), source English is formalized, the formula is translated back into English, and the reconstructed English is formalized again. A solver compares the first and last formulas. If they differ, diagnosis identifies a likely translation stage to repair, and dependent stages are regenerated.

This is attractive for our user interface because the intermediate English can make a formal candidate easier to inspect. Suppose the original formula contains both product-link routes but the back-translation mentions only a specific product. Re-formalizing that sentence may lose the product-type route. The resulting counterexample exposes a defect in the explanation or translation chain. A reviewer should not be shown that explanation as if it faithfully described the formal rule.

The same construction can miss a shared omission. If the first formalization already omits the product-type route, a faithful back-translation of that incorrect formula and a faithful re-formalization will agree. The roundtrip then passes. What survived is the first formula, including its error relative to the source. The paper explicitly recognizes this limit; our ensemble must retain a source-grounding path that does not simply inspect the same back-translation.

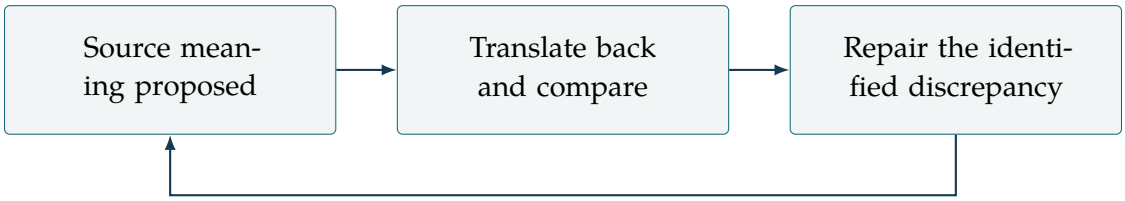


Figure 4.25: Roundtrip checks can expose loss, but can also repeat a shared omission.

Repair therefore needs a named target. If the formal rule is justified but the explanation drops a condition, repair the explanation and its dependent roundtrip outputs. If the original rule omits a source condition, create a new interpretation candidate and revalidate its downstream artifacts. If the two formulas use incompatible vocabularies, repair the mapping before claiming the formulas disagree about one question. A generic "make these consistent" instruction does not distinguish those cases.

The stopping rule is equally important. A roundtrip procedure can run for a bounded number of attempts and still end in disagreement. The product should report the original and revised formulas, the counterexample, diagnosis, actions attempted and the remaining issue. It should not keep repairing until some pair agrees and then discard the path that made the original source objection visible. The ensemble controller extends this principle from one translation chain to several methods and issue types.

An explanation generated by deterministic templates provides a useful additional path. It is constrained by the formal syntax and can be checked for completeness of its displayed operands. It may be less elegant than free prose, but it offers a stable view of what the program actually says. A freely generated explanation can still help readers if

it is compared against that view and does not replace it as the sole account of the rule.

4.13 Evidence spans and distant qualifications

An answer supported by a source span is easier to review than an unsupported assertion, but the span itself may be insufficient. A qualification can appear in a footnote, definition or distant section. ContractNLI studies document inference with evidence annotation and highlights challenges involving references and exceptions (Koreeda and Manning, 2021, task, annotation and error-analysis sections). For our product, the relevant lesson is to retain the supporting and limiting passages needed for the inference, rather than a single convenient quotation.

The distinction between "not mentioned" and "false" is particularly useful. If a retained source does not settle the definition of a fee refund, the absence should remain visible. A retrieval engine returning no matching paragraph does not establish that the exception excludes refunds. It may have searched the wrong document, missed a synonym or lacked the referenced source. The evidence report should distinguish unsuccessful retrieval from reviewed absence within a specified source set.

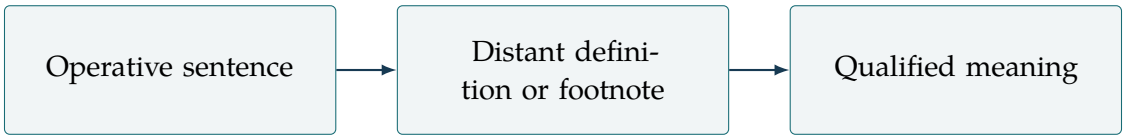


Figure 4.26: The supporting span can lie beyond the nearest sentence.

Source spans also require stable identity. Character offsets depend on the extraction version, while page numbers may not survive a changed layout. Retain raw-byte identity, derivative identity, exact span text or hash, and human-readable location. A reference to paragraph 10 without a source version can become misleading after an amendment. A matching text hash without its source context may identify a sentence copied into a secondary summary.

The formalization boundary should distinguish source claims from supplied facts. A sentence defining a gift exception is part of the normative context. A bank record establishing that a fee was paid is factual evidence. A reviewer's classification relating the record to the exception is an interpretive application. Combining all three into an undifferentiated "evidence" list makes it harder to see where disagreement actually arises.

Recent faithfulness work analyzes how language models and formal solvers can diverge when implicit assumptions enter formalization (Wang et al., 2026). The retained paper also has annotation and denominator questions that prevent us from transferring a general accuracy figure. Its examples nevertheless reinforce a practical requirement: an implicit assumption that changes the answer should become an explicit, challengeable premise. The system must show where a model supplied information that the source packet did not.

4.14 Testing paths, dates and inconsistent constraints

Other methods contribute different checks once a formal model exists. CUTECat uses concolic execution to explore paths in Catala programs (Goutagny et al., 2025, secs. 2–5). Concolic execution combines a concrete run with symbolic constraints describing the path taken. By changing a path condition, the tool seeks an input that exercises another branch. For our gift example, this is analogous to moving from a named-product case to the case where only the product-type route can determine the answer.

Path-directed generation can expose an exception branch that ordinary examples never visit. It cannot visit a branch that the formal program omitted entirely. Independent source-derived cases remain necessary. The reported costs on larger legal examples also show why a prototype needs explicit resource limits; small-program speed should not be treated as a promise that every circular can be exhaustively explored.

Date arithmetic is another place where a precise program can embody a wrong choice. Adding a month to a date near the end of a month may require a rounding convention. The choice can affect deadlines. Monat et al. (2024) formalize date arithmetic and analyze ambiguity in legal applications. The transfer to this project is to make calendar rules, boundary conventions and their source basis explicit. A verified date library does not decide which rounding convention a particular provision requires.

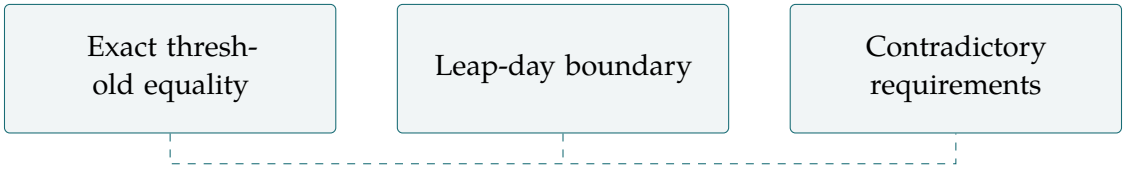


Figure 4.27: Boundary cases test different sources of failure.

ContractCheck analyzes consistency and executability of structured contract models (Khoja et al., 2025, secs. 3–8). A satisfying assignment shows that a modeled set of conditions can jointly hold. An unsatisfiable set can help locate conflicting clauses or assumptions. Neither result proves that the original legal document has been translated faithfully. The study's manual translation is an important part of its task boundary.

An unsatisfiable core is a useful diagnostic: a subset of constraints that already suffices for inconsistency. It is not necessarily the smallest such subset unless a minimization procedure establishes that property. For a reviewer, a small and source-linked explanation is valuable, but the product should label a returned core accurately. It must also distinguish an encoding inconsistency from legitimate defeasible conflict that the chosen logic was intended to resolve.

These analyses fit into the ensemble as specialized questions. Does this representation have a hidden path? Does a date convention alter a boundary case? Are the supplied assumptions jointly possible? Does a revised rule preserve the reviewed behavior? Their evidence can challenge a candidate without claiming to replace the source interpretation process.

4.14.1 Dates and test generation expose defects that ordinary examples miss

The date-arithmetic work of Monat et al. (2024) demonstrates why even a simple phrase such as adding a month needs a convention. Calendar months have different lengths; leap days create boundary cases; sequences of date operations need not behave like addition of integers. The paper develops explicit rounding choices, formalizes date semantics and selected properties, and uses static analysis to detect ambiguities in legal computations. Its application to benefit rules connects the formalism to realistic source language rather than treating dates as a generic programming-library problem.

The immediate lesson for annual reviews and three-year transaction windows is to preserve the chosen calendar operation and its source or interpretation. "Three years" must not silently become a fixed number of days. A leap-day case can expose disagreement between two plausible implementations. The paper's day-level formalization does not settle intraday questions such as withdrawal instructions arriving at a trading cut-off, nor does the mechanization make the whole static-analysis implementation verified (Monat et al., 2024, secs. 2–6). Those are distinct prototype obligations.

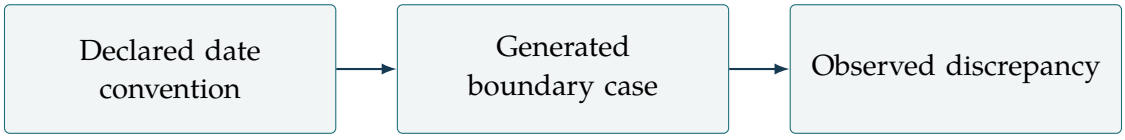


Figure 4.28: Date semantics needs explicit calendar assumptions.

For a synthetic anniversary beginning on 29 February, a non-leap year forces a policy choice. February 28, March 1 and rejection are distinct outcomes. A library default cannot establish which convention the adopted requirement uses.

CUTECat complements this work by generating test cases from Catala programs using concolic execution: an execution carries concrete values and symbolic conditions, then a solver seeks inputs that take a different branch. The method specializes its exploration to default expressions and uses preferences to obtain inputs that are easier for humans to understand. In the reported evaluation, generated cases exposed 20 manually validated seeded mutations. A much larger example also shows that exhaustive-looking exploration can produce a substantial volume of cases and runtime (Goutagny et al., 2025, secs. 3–5).

In the running example, a concrete test with portfolio HK$45 million takes the successful portfolio branch. The symbolic execution retains the condition $P \geq 40{,}000{,}000$ instead of forgetting how that result was reached. Negating it asks the solver for $P < 40{,}000{,}000$; holding the other financial route false yields a case that changes the overall answer. To distinguish inclusive from strict comparison specifically, request $P = 40{,}000{,}000$. Concolic exploration automates this alternation between a real execution and constraints describing another path. Mutation testing asks a different but complementary question: whether the examples notice an intentionally wrong operator. Both mechanisms are understandable and useful without importing the entire CUTECat ex-

ecution environment.

LegalMath should borrow branch-directed and mutation-based testing, especially for inclusive boundaries, alternatives, exceptions and rounding. The result is evidence about the encoded program. A branch corresponding to an omitted paragraph does not exist for the test generator to discover. Independent source-derived scenarios are therefore essential. The official CUTECat artifact documentation was inspected, but its large execution image was not run; the prototype should first implement a small set of useful mutations before adopting that full toolchain (LegalMath research notes, 2026).

4.14.2 Consistency of clauses and compliance of processes

ContractCheck models legal contracts through structured clauses, an ontology and constraints, and uses automated reasoning to identify consistency and execution problems. Its share-purchase-agreement case study is particularly relevant to the presentation of contradictions: reviewers need to know which claims cannot coexist and which circumstances make an obligation executable (Khoja et al., 2025, secs. 3–7).

Two distinctions are essential when borrowing this approach. First, the study's input formalization is manual; its reasoning results do not measure automatic translation fidelity. Second, showing that each claim can be executed under some circumstances differs from showing that all duties can be satisfied in one execution, and both differ from showing that every possible execution complies. LegalMath should label each of those questions separately. If a solver returns an unsatisfiable core, that identifies conflicting constraints; a smallest or minimal explanation requires a further minimization procedure rather than an assumption about the core (Khoja et al., 2025, secs. 4–5, 8).

Process-compliance research reinforces the need to connect rules with activities, actors and the effects of execution. Ghose and Koliadis (2007) describe effect accumulation, constraint checking and repair in business processes. For the bank, the analogue is the difference between determining that a warning is required and showing that the relevant warning was delivered before the required event.

Tosatto et al. (2015, sec. 2) distinguish achievement obligations, which must be accomplished, maintenance obligations, which must hold across an interval, and punctual obligations. They also distinguish partial compliance, where some execution complies, from full compliance, where all executions do. Their complexity results concern specified process and obligation languages (Tosatto et al., 2015, secs. 3–4). The prototype should borrow those questions and delimit its model rather than import a complexity label for all regulation. The practical consequence is that a flowchart showing one successful path is insufficient evidence about the paths that omit a review or mishandle an exception.

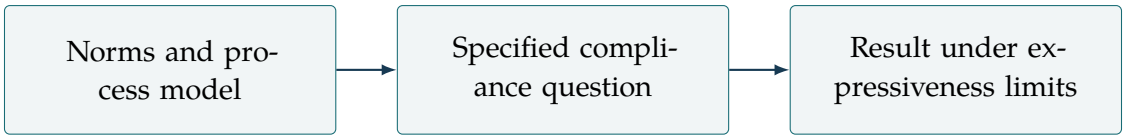


Figure 4.29: Clause consistency and compliance of a process are different analyses.

Hashmi et al. (2016, sec. 3, Definitions 10–17) give a more detailed taxonomy. An obligation may apply at a point or persist across an interval; a persistent obligation may require an achievement once or maintenance throughout; achievement may or may not count before activation; and a violated obligation may persist or have specified compensation. A Boolean `completed` field cannot distinguish early action, timely performance, late performance, or a continuing control that failed briefly and recovered. The paper takes as input a function specifying which obligations are in force at each trace position. Supplying that function is a substantive modeling step, not something the taxonomy infers from prose. The printed definition of maintenance violation also needs correction: Definition 14 on page 15 uses membership in the set of fulfilled conditions where the adjacent prose and Figure 3.5 require non-membership. As printed, a trace that satisfies the duty throughout could be called a violation. An implementation must check the intended condition explicitly instead of copying that formula. The MVP consequently supports one named profile, non-preemptive achievement, and explicitly rejects unsupported maintenance and compensation templates.

Governatori et al. (2016, secs. 3–4) connect LegalRuleML contexts and versions to annotated business-process tasks and a Regorous/PCL reasoner. Their procedure accumulates effects, derives obligations in force, carries pending obligations forward and checks fulfillment, violation and compensation. It distinguishes all-trace compliance from existence of a compliant trace, and conditions its reasoning claim on appropriate rules and complete annotations. Omitting a trigger can prevent an obligation from appearing at all, producing a deceptively clean result.

In the reported telecommunications pilot, roughly 100 paragraphs and associated definitions became 176 normative statements; initial legal mapping took about two weeks, followed by process-modeling workshops. These quantities show that formalization and operational mapping are real work; they do not estimate the time required by this bank. The proposal budgets compliance participation and counts preparation in every comparison instead of treating gold rules as free preprocessing.

4.15 A genuine citation can still support the wrong answer

The legal-AI evaluation literature adds a failure that a source-preservation system alone cannot detect. An answer may link to an authentic document and still misstate its consequence, use an inapplicable jurisdiction, or rely on a position that has changed. The review therefore needs two questions: whether the stated proposition is correct, and whether the cited authority supports it for the activity and time under assessment.

Magesh et al. (2024, sections 4–5 and appendix C) distinguish factual correctness from legal groundedness when evaluating legal research tools. Their 2024 study used 202 queries, recorded product responses and manually applied a published coding rubric. A source can be real yet fail to support the proposition attributed to it. Their treatment of partially correct answers and refusals also shows why a label must be read with its scoring definition. The study's products, dates and United States legal questions do not estimate LegalMath's error rate on Hong Kong circulars. Its useful contribution here is the separation of answer correctness, citation support and incomplete answers.

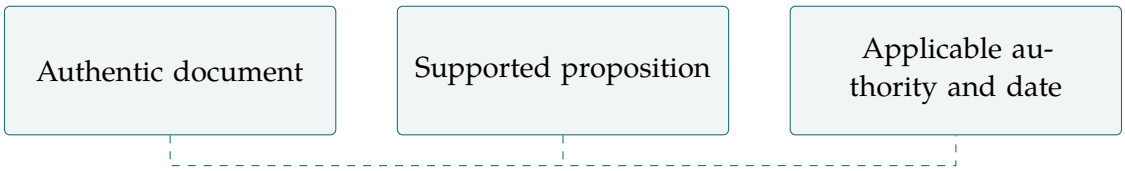


Figure 4.30: Three checks are needed before a citation can support a regulatory control.

Consider a synthetic drafting error. A candidate states that an SFC request is always required before a particular action, citing a paragraph that requires a response *when* the SFC requests it. The document exists and the quoted words are accurate, but the candidate has changed the trigger. The audit should retain the candidate's precise claim, the relevant sentence and its surrounding conditions, and the reason the implication is unsupported. A quotation is useful because another reviewer can inspect that comparison; quotation marks themselves do not establish that the inference is sound.

Dahl et al. (2024, sections 4.2–4.3 and appendix C) distinguish reference-based checks from a method that detects contradictions between model responses. They inspect United States case-law tasks and validate the contradiction labelling against human recoding of a sample. The distinction matters to an ensemble: detecting a contradiction identifies an inconsistent pair, while agreement leaves shared error possible. The workbench consequently needs source support even when repeated answers are consistent. The paper's published error estimates and its model-based labelling are not a substitute for an independent Hong Kong reference process.

A citation audit must also record the edition actually read. A preprint, a conference version, a later journal article and a changing software repository may support different statements. If a later edition changes a theorem or a benchmark denominator, retaining only the newest file destroys the explanation of an earlier claim. The appropriate record binds the document bytes, section or page, a short exact quotation, the manuscript proposition and the support judgment. The full retained document allows examination beyond the short quote.

4.16 Where the literature still leaves this product exposed

The collection is strongest on executable legal languages, formal comparisons and candidate-generation methods. Those contributions make the implementation question concrete. They leave several questions on which the product's actual banking value depends.

First, the product needs evidence about omissions across a *complete* regulatory packet, including incorporated definitions and attachments. A benchmark whose input already supplies the decisive facts and formal rules cannot answer whether a workbench discovers those facts and rules. The next comparison should preserve independently inventoried source units and measure which material units and alternatives are missed. Existing engineering cases can remain as development checks without serving as the legal reference.

Second, comparison of language backends needs matched meanings. Before choosing among RuleIR, Catala, a decision table or a normative language, give each the same unknown facts, conflicting evidence, exceptions, dates and duties. Record where an adapter cannot express the accepted meaning. A familiar syntax or a successful example does not establish interchangeability. The restricted initial runtime keeps this comparison manageable.

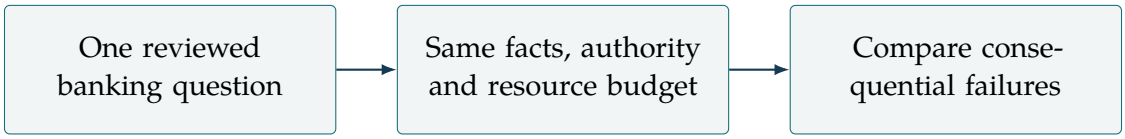


Figure 4.31: Coverage improves through matched questions, not by accumulating unrelated benchmark scores.

Third, common-mode failure deserves a direct study. Changing the model vendor while keeping the same defective extraction may leave the most serious omission untouched. A useful experiment varies extraction, inventory and interpretation separately and retains which activity first detected a defect. The numerical dependence example in Chapter 6 explains the mechanism; it supplies no measured correlation for this product.

Finally, the proposed human workflow needs independent evaluation. Reviewers must be able to identify a plausible but unsupported source claim, notice a minority reading and act on unresolved issues without an unmanageable queue. The study in Chapter 9 therefore measures consequential errors and review effort together. A legal-language theorem, model benchmark or successful browser test does not answer that human question. These are defined coverage gaps and research tasks, rather than a claim that no relevant work exists beyond the retained literature.

4.17 A synthesis for the first implementation

The immediate product should keep a small executable core whose semantics are fully specified. Around it, the interpretation record should be richer than the executable language: it needs alternative readings, source versions, definitions, unresolved qualitative judgments and supporting arguments. Controlled templates give reviewers a readable view of formal structure. Specialized analyses can then inspect declared fragments through adapters.

This choice avoids forcing every legal question into the first compiler. A paragraph requiring judgment can become a reviewed assessment input or manual procedure, with explicit evidence and responsibility. A temporal duty can use an event profile whose limits are stated. A disputed definition can remain in the interpretation set until review resolves it. The system can still generate Java for a reviewed subset while keeping the broader assessment visibly incomplete.

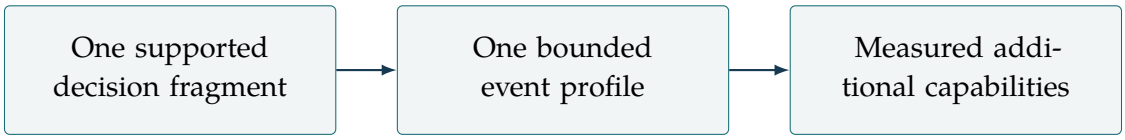


Figure 4.32: The first implementation gives each borrowed mechanism a testable purpose.

Every adapter needs a semantic contract. It declares supported constructs, result categories, unknown and conflict behavior, vocabulary mapping, resource bounds and how a returned witness is replayed. A paper's name or a repository's existence does not supply that contract. A failed or unsupported analysis is reported as such and cannot be counted as an additional agreeing ensemble member.

The literature thus supplies complementary mechanisms: source-linked authoring, precise exceptions, contextual alternatives, normative state, symbolic consequence, path-directed testing and translation repair. The implemented investigation now combines a selected subset: competing readings, bounded search, explicit challenges, source checks and executable comparisons. An official-example register adds conditional analogies and distinctions. These adaptations keep the small executable core while preserving questions that it cannot settle. They do not implement a general procedure for extracting judicial reasoning, deciding the priority of legal authorities or reproducing all the capabilities of the research systems discussed here. Before specifying that controller, we need to explain how competing interpretations are generated and defended. That is the task of the next chapter.

Chapter 5

Constructing and comparing competing interpretations

When two reviewers disagree about an incentive, the bank needs more than a winner. It needs to know what the disagreement changes, which source passage supports each reading, and what fact or example would separate them. A drafting service that produces one fluent answer may hide that choice instead of solving it.

5.1 Look for choices that change a decision

Suppose three models explain paragraph 10 in different words. The first says that product-linked gifts are prohibited unless they are fee discounts. The second calls the same arrangement a marketing incentive and repeats the exception. The third turns the sentence into an identical decision table. We have three presentations but possibly only one interpretation. Their agreement tells us little about the alternative that a mixed offer should be assessed component by component, or about the evidence required to classify a cashback payment. An ensemble that rewards the number of agreeing explanations may therefore become more confident without becoming more informative.

A useful hypothesis changes a consequential choice. It might change the unit of assessment from a whole offer to each benefit, distinguish reimbursement of a paid charge from an unrelated reward, or require an incorporated definition before applying a term. The hypothesis must also explain why that choice is supported. Merely deleting the words about a product type creates a different program, but an unsupported deletion is a test mutation, not an equally respectable interpretation. The system needs both kinds of variation and must label their roles accurately.

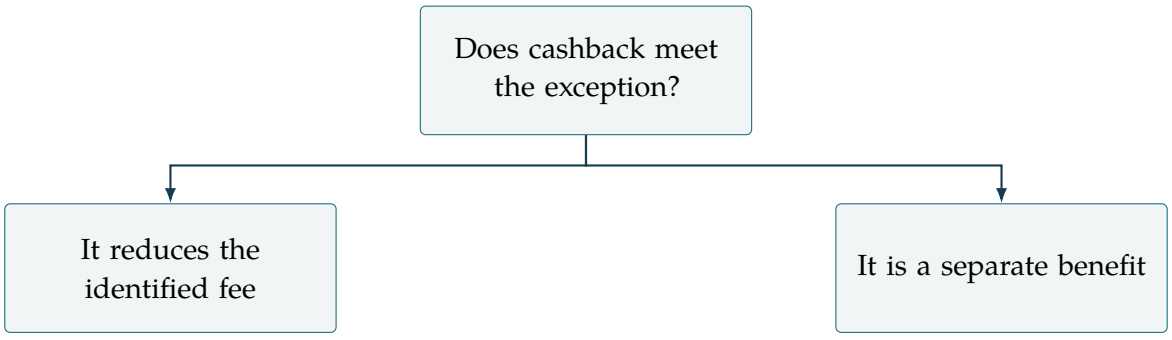


Figure 5.1: Search explores consequential readings of the same facts.

We will use a candidate record with five substantive parts: a proposition about the source; the assumptions under which it holds; a formal rule that expresses its consequences; supporting and opposing reasons; and a description of cases that distinguish it from another candidate. The candidate also carries source versions and an immutable identifier. This makes a claim such as "this cashback is a discount" answerable: a reviewer can inspect the proposed definition, identify the facts it requires, and observe the transactions on which a competing definition would disagree.

The search objective follows from that representation. We seek a useful set of supported alternatives, together with reasons for rejecting other proposals and questions that remain unanswered. Finding a single high-scoring sentence is a different objective. The former fits the bank's need to know where judgment is required; the latter can conceal the judgment in an optimization procedure.

5.1.1 What a drafting service should propose

The drafting service should propose clause segments, roles, definitions, cross-references, candidate rules and review questions. Every proposal must cite the exact source passage and expose the definitions it imports. The service should use constrained structured output, followed by validation against the rule grammar. Retrieved text is evidence to interpret, not an instruction that can modify the workbench or execute arbitrary tools.

Independent candidate translations can help discover uncertainty. Their value comes from meaningful differences, not a vote count. When two candidates disagree, the solver should try to construct a case that distinguishes them. When both depend on a missing FAQ, the next action is to retrieve and inspect that source. When the language requires legal judgment, the result is an explicit assessment task. The model's confidence and the solver's ability to execute a formula must remain separate from approval of the interpretation, consistent with the lessons of ARc and the tax-reasoning work (An et al., 2025; Jurayj et al., 2026).

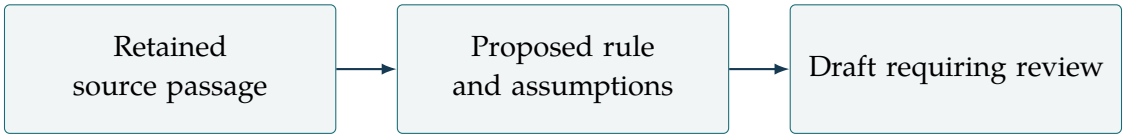


Figure 5.2: A drafting service proposes an interpretation together with its dependencies.

The reviewer should receive a short set of consequential questions. For example: does the selected evidence establish that an agreement is absent, or merely that it has not been retrieved? Does a new circular replace the entire earlier source or a specified part? Does a control apply to the distributor or the provider? Question selection should reduce ambiguity in the actual decision rather than produce generic compliance checklists.

5.2 Legal theories are more than predicted outcomes

Theories of legal reasoning provide a richer starting point than generic text search. Bench-Capon and Sartor (2003, sections 3–5) represent a theory through cases, legally relevant factors, rules, preferences between rules, and values that explain the preferences. A factor is a legally significant characteristic, not simply a word that occurs in a judgment. A rule relates such characteristics to a conclusion. A preference explains why one applicable rule should prevail over another when both cannot determine the outcome together.

To understand the distinction, consider an invented training example about a reimbursement policy. One decision allows reimbursement because the expense was necessary for an authorized task. Another refuses it because the claimant had already been reimbursed. A theory containing only the first rule predicts payment whenever necessity is established. A theory containing both rules, but no priority, encounters a conflict in the double-payment case. A theory in which the bar on double payment defeats the necessity rule explains both outcomes. This is a logical illustration, not a statement about an actual court or SFC policy.

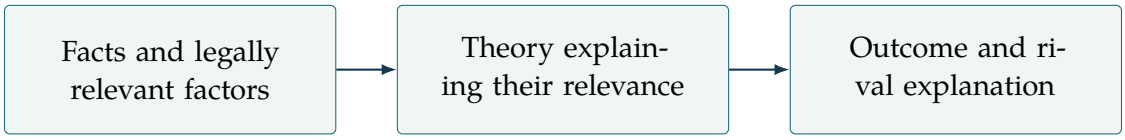


Figure 5.3: A legal theory explains why the factors matter.

The theory's values might explain a preference by reference to preventing double recovery. That explanation is itself contestable: it must be connected to applicable authority rather than invented as the bank's preferred commercial outcome. The theoretical machinery helps represent such an explanation; it does not decide which values an institution is legally entitled to prioritize.

For the circular, the analogous factors include a product link, a type-of-product link, the character of the benefit, and evidence of a charge reduction. A theory might treat an incentive's relationship to the subscription charge as decisive; another might require that the charge itself is reduced in the contractual fee schedule. Their predictions may coincide for a direct fee waiver and diverge for cash paid after settlement. The divergence identifies an interpretive question that a bare outcome label would obscure.

The source paper does not claim to recover complete judicial reasoning from raw opinions. Section 5.4 explicitly discusses limits of the representation. Our implementation must preserve that limit: converting a judgment to factors is another interpretive task, with evidence spans, assumptions and review. A model that extracts a convenient factor list and then proves a conclusion from it has not bypassed the original source problem.

5.3 What case-based argument contributes

The history surveyed by Bench-Capon (2017, sections 2–3) explains how an initial reading can be challenged through a concrete case. HYPO organizes exchanges in which one side cites a favorable case, an opponent distinguishes it or presents a counterexample, and the first side replies. This exchange encourages a search for facts that make an apparent analogy fail. The purpose is not to collect the largest number of vaguely similar documents.

Imagine that an earlier, applicable authority addresses a benefit described as a fee rebate. A candidate relying on it must state the shared features: perhaps a charge was first incurred, the later payment could not exceed that charge, and both arose from the same transaction. An opposing candidate may distinguish an offer in which the payment exceeds any charge or can be earned without incurring one. These are questions about material similarities. A search engine's textual similarity score does not answer them.

HYPO's claims lattice orders relationships between cases using legally relevant dimensions. A lattice is a partial ordering: some comparisons remain incomparable. Two precedents can each support a different aspect of the present case, without either being globally "more similar." Preserving this structure is useful for an interpretation workbench. A numerical embedding can nominate cases for inspection, while the argument record explains which feature makes a case relevant and which feature might defeat the analogy.

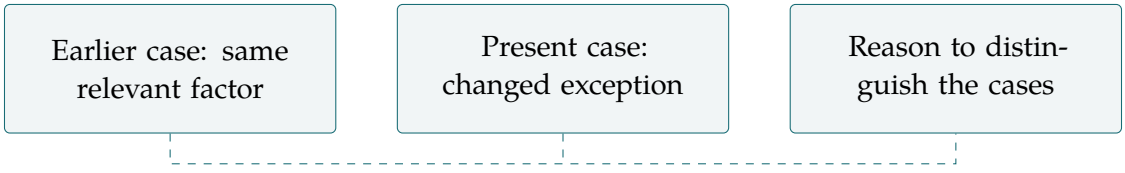


Figure 5.4: Case-based reasoning needs reasons for similarity and difference.

Two promotions can both involve cash yet differ because only one reduces an identified charge. The similarity in payment form does not settle the exception. A useful comparison isolates the relationship that the legal reading treats as decisive.

CABARET, as described in the retrospective, combines statutory rules with case-based reasoning about difficult terms. BankXX searches a network of pieces of legal argument. These suggest a division of labor for the circular: apply explicit logical structure directly, and investigate open-textured classifications through sources and arguments. They do not justify allowing analogy to override an unambiguous express restriction without an applicable legal reason. The original CABARET and BankXX implementations have not been audited for this monograph; the historical claims here come from the inspected retrospective.

A production case record therefore needs more than a citation and a summary. Record the issuing body, jurisdiction, date, procedural context, proposition relied on, relevant factual features, passage that supports the proposition, and known treatment by later

authority. Distinguish a court's holding from an advocate's submission or an incidental observation. Those classifications must remain reviewable. An invented or misclassified precedent is more dangerous when attached to a polished formal argument than when presented as an uncertain search result.

This book supplies no actual case-law ruling on the cashback example. Its role is to specify how such authority would enter the system if found. The immediate 23EC46 exercise remains tied to the circular and its unresolved incorporated materials. A reviewer should never have to infer that a hypothetical analogy is a reported judicial decision.

The implemented example register now supplies a narrower, actual application of this idea. It retains the two branches of FAQ Question 1 discussed in Section 2.15: links to certain types of funds and links to funds from particular fund houses. Each branch keeps the issuer, passage, date uncertainty and proposed relevant features. A candidate can explain a similarity or identify a difference, with those assumptions exposed for review.

The distinction between applicability and prohibition controls the argument. If a candidate predicts that paragraph 3.11 applies to a product-type promotion, the FAQ branch can provide conditional support for that proposition. It cannot thereby establish that a fee discount is prohibited. Nor does attaching a real quotation to a supposed court case make the invented case real. The register requires the example's identity and source as well as its words.

In the retained exercise, two branches produced twenty conditional argument moves across five candidate readings. Those are repeated uses of one official answer, not twenty independent authorities. Historical applicability and the choice of material features remained open. This limited implementation gives analogy a reviewable form without claiming a general method for deciding the force of judicial precedents.

5.4 AGATHA: explicit construction of rival theories

AGATHA supplies a direct precedent for searching over competing legal theories. Chorley and Bench-Capon (2005, sections 2–4) define moves that extend or challenge a developing theory: analogy, distinction, counterexample and changes to preferences. Rather than asking a model to "think harder," the system asks what legal move would change the current account and why. This makes the search history inspectable.

In our running example, an analogy constructor could propose that two payments belong to the same classification because both reverse a charge. A distinction constructor could split them by whether a charge was actually paid. An exception constructor could attach the fee-discount qualification at a particular point in the rule. A counterexample constructor could search for a scenario that the candidate treats as permitted despite an express product-type link. Each move should identify its parent, the changed proposition and the source support.

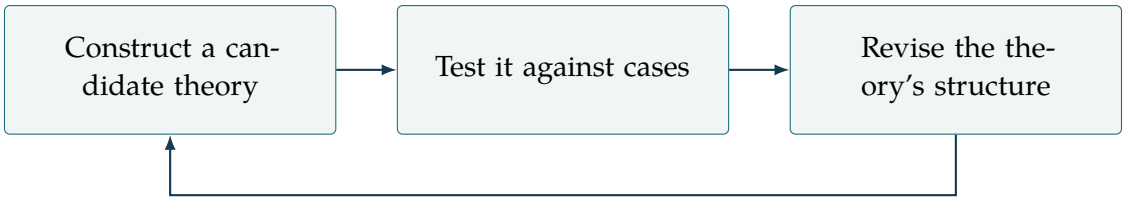


Figure 5.5: AGATHA makes rival theory construction explicit.

AGATHA evaluates theories using explanatory power, simplicity and completeness. Its search is a modified A* procedure, and the paper explains why the modification can miss the best theory. We should not import the word A* as an optimality certificate. The evaluated cases already have factor descriptions. The reported exercise is therefore substantially easier than discovering the factors, resolving cross-references and determining current authority in raw banking materials.

The paper's adversarial lookahead is also instructive. A theory can look strong because a limited opponent fails to find the appropriate counterargument. Optimizing for a victory against that opponent can favor a weak justification. For a bank, the objective must be resistance to informed challenge, not persuasion of a convenient critic. The system should preserve the strongest located objection even when an advocate agent prefers to stop discussing it.

We borrow the explicit construction operators and the record of competing reasons. We do not adopt the paper's heuristic weights as calibrated legal confidence, nor its advocacy objective as the bank's release criterion. A successful local adaptation must be evaluated on whether it finds material alternatives and omissions within a declared source perimeter. That evaluation has not yet been performed.

5.5 Arguments, attacks and unresolved conclusions

An argument is a chain or tree from premises to a conclusion. A tree joins the separate reasons needed for a conclusion with several premises. Some steps are strict: under the stipulated meaning of conjunction, two true components make the conjunction true. Other steps are defeasible: evidence that a payment reverses a charge may support a classification while remaining vulnerable to a contrary source or missing condition. ASPIC+ separates these kinds of inference and formalizes ways to challenge them (Prakken, 2010, sections 2–3).

There are three useful attacks. An attack on a premise disputes a starting fact, such as whether the charge was paid. A rebuttal supports a contrary conclusion, such as classifying the payment as an additional benefit. An undercut attacks the inference itself: even if the payment and charge have equal amounts, equality of amount alone may not establish that one legally discounts the other. These attacks require different remedies. Retrieving a payment record may settle the first; retrieving a definition may address the third. Repeating the same classification prompt need not address either.

An extension is a set of arguments satisfying a specified acceptability condition. A con-

clusion is credulously accepted when at least one extension supports it, and skeptically accepted when every relevant extension supports it. A conclusion may be supported by different arguments in different extensions; requiring one identical argument to appear everywhere would be a stronger and different condition. The selected semantics and treatment of premises must be stored alongside the result.

Let an argument graph contain arguments a and b that defeat each other, with no stronger argument resolving the dispute. Under the usual preferred-extension semantics, the graph has two alternative acceptable sets, $\{a\}$ and $\{b\}$. Neither argument appears in both sets. Under grounded semantics, neither is accepted in this simple graph. The result expresses unresolved competition; it does not say both conclusions are legally true.

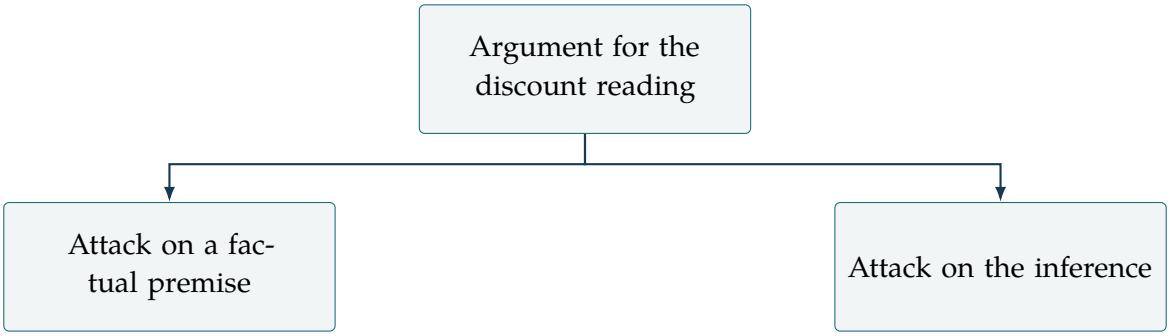


Figure 5.6: An objection can challenge the evidence or the step from evidence to conclusion.

An invoice may establish that a fee exists without establishing that a later payment reduces it. One objection disputes the invoice's authenticity; another accepts the invoice but disputes the inference. The workbench should preserve the distinction because the remedies differ.

Carneades is an argumentation system that represents reasons for and against conclusions. Its software includes an implementation of Dung-style argument graphs, in which arguments are nodes and attacks are directed edges. We first use that graph component to illustrate an execution error; the next section introduces Carneades' treatment of premises and burdens of proof.

There is an implementation trap at this point. If a semantics yields no extensions, code that applies a universal predicate to an empty list may report that every proposition passes. The inspected Carneades Dung implementation illustrates why callers must handle the existence of extensions explicitly. The workbench will require a nonempty computed collection before reporting skeptical support. If no extension exists, or enumeration is incomplete, the result is a separate unresolved status. It cannot become a positive conclusion through vacuous truth.

Argumentation computes consequences of the supplied arguments, attacks and preferences. It cannot establish that all relevant arguments were supplied or that the legal preferences are justified. Moreover, ASPIC+ rationality results depend on conditions on rules and knowledge bases; the author's correction to definition 6.8 must accompany

any implementation relying on those results. A free-form graph of model-generated sentences does not automatically inherit the paper's theorems.

5.6 Burdens, assumptions and evidence in Carneades

Carneades distinguishes ordinary premises, assumptions and exceptions (Gordon et al., 2007, sections 3–6). An ordinary premise needs support. An assumption may stand in a particular dialogue until challenged. An exception can defeat an otherwise supported argument. These distinctions explain why absence of evidence has different effects in different reasoning arrangements.

For example, the workbench might require affirmative evidence that a transaction falls within the bank's activity profile. It must not silently assume that fact because most transactions in a training set did. Conversely, a procedural review may temporarily accept an explicitly declared factual assumption to compare the consequences of two legal readings. That temporary acceptance is a property of the review exercise, not a permission to use the assumed fact in production.

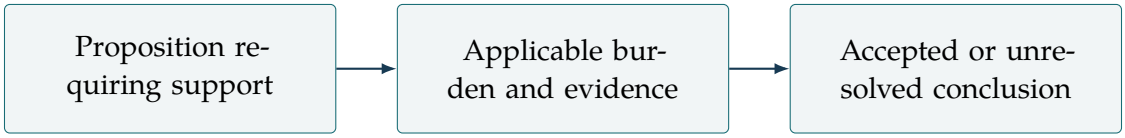


Figure 5.7: A burden of proof concerns what support is sufficient for a proposition.

A burden of proof specifies what support is needed for a proposition in a given context. The 2007 paper includes illustrative proof standards and explicitly notes that they have not been validated as legal proof standards. It would be wrong to rename one of its numerical or dialectical conditions "SFC approval" and install it as policy. The bank needs its own reviewed rules about which questions require legal adjudication, what factual evidence is sufficient, and which roles can accept residual operational risk.

The original model is acyclic. Newer Carneades software has a different computational model and cannot be treated as an executable copy of every result in the paper. Our source inspection of the newer implementation found useful argument utilities but also potentially expensive extension enumeration. A prototype should first support small, bounded graphs with explicit semantics and known reference examples. Introducing a general argumentation engine before its resource and semantic behavior are understood would create another opaque component in the assurance chain.

For the first release, even a simple structured argument register is valuable. Require each candidate to name its supporting premises, distinguish assumptions from evidence, identify opponents and specify what would defeat it. This can be implemented before automatic extension computation. The eventual engine then formalizes an already visible review practice rather than replacing an undefined practice with a numerical score.

5.7 Breadth before expensive investigation

The initial search should cover named dimensions: actor and activity scope, subject identity, definitions, logical attachment of exceptions, modality, quantification, time and incorporated context. These dimensions are prompts for possible differences, not a Cartesian product of equally plausible answers. If a choice has no source support, retain it as unsupported or as a negative control; do not inflate the shortlist with implausible alternatives.

Consider three dimensions with two alternatives each. Exhaustive combination would generate eight candidates, but some combinations may be inconsistent. A whole-offer interpretation combined with a component-only factual schema may not be executable without a transformation. A candidate that uses a definition from one historical version and an exception from an incompatible later version may not describe any applicable regime. Rejecting these combinations requires an explicit reason, because an encoding defect could otherwise be mistaken for a refutation of the underlying reading.

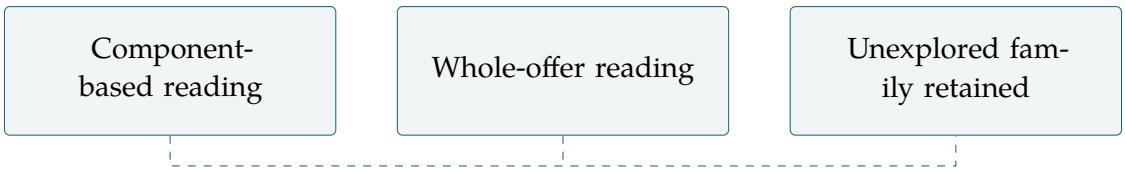


Figure 5.8: Search breadth concerns distinct meanings, not paraphrase count.

A practical initial frontier reserves room for each material family before ranking refinements within a family. The reservation is a search policy, not a claim that all families have equal legal merit. If the budget permits only six candidates and eight supported families are identified, the report must say that two families remain unexplored. It should not display a six-candidate shortlist as exhaustive. Increasing a beam width can improve exploration, but no finite width establishes semantic completeness for unrestricted legal language.

The search tree records how candidates were created. The argument graph records shared dependencies. These are different structures. Two branches may rely on the same disputed definition, and a later clarification should affect both. Representing all dependencies only as parent-child edges would duplicate facts and make invalidation unreliable. A directed acyclic graph of immutable candidate revisions can coexist with a possibly cyclic argument graph, provided each structure has its own semantics and resource limits.

Deduplication also needs layers. Exact duplicate bytes can be merged by hash. Canonical syntax can identify expressions that differ only in irrelevant formatting. Formal equivalence can establish equal decisions over a declared domain. None of these establishes that source assumptions are identical. Two candidates with equivalent Boolean bodies but different definitions of “gift” remain separate interpretive commitments. Their shared body may be stored once, while their source and factual bindings remain distinct.

5.8 Tree search and the meaning of a reward

Tree of Thoughts separates candidate generation, evaluation and search (Yao et al., 2023, section 3). Its inspected breadth-first implementation retains a beam selected by model-based values or votes. That is bounded search, not exhaustive breadth-first enumeration. In the value-based branch, an exact duplicate string receives a zero score; it is not necessarily removed from the candidate list. This check cannot establish that the retained thoughts express different legal hypotheses.

Language Agent Tree Search combines tree search, model judgments, reflection and environment feedback (Zhou et al., 2024, sections 3.2–4 and appendix A). A commonly used selection expression illustrates the tradeoff. For a parent visited N times and child i visited $n_i > 0$ times, let $\bar{r}_i$ be its observed mean reward. A UCT-style selector evaluates

$$U_i = \bar{r}_i + c\sqrt{\frac{\log N}{n_i}}, \qquad c > 0. \tag{5.1}$$

The first term favors a child that has produced useful outcomes. The second encourages investigation of less-visited children. A separate rule handles unvisited children; setting their denominator to zero is not an implementation. The constant c, reward definition and visit accounting are material design choices and must be recorded.

For a numerical illustration, set $c = 1$ and suppose the parent has been visited 100 times. One child has an average reward of 0.6 after four visits; another has an average of 0.8 after 25 visits. Their scores are approximately 1.67 and 1.23. The rule therefore investigates the less-tested child even though its observed average is lower. That is the purpose of the second term: it buys information about an alternative that might otherwise be neglected. Scores above one are perfectly possible because this is an investigation priority, not a probability that a legal reading is correct. The chosen $c = 1$ illustrates the arithmetic; it has not been established as a suitable setting for this project.

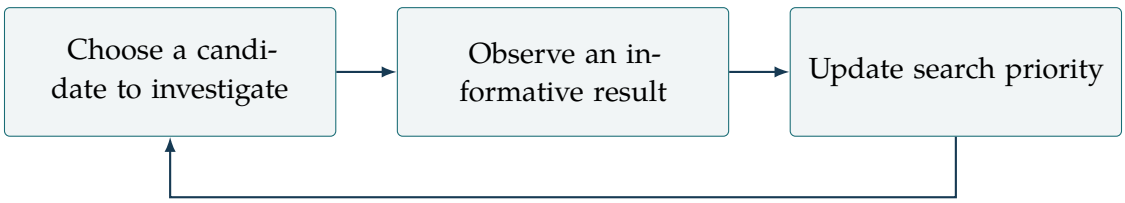


Figure 5.9: A search reward directs computation; its definition determines what is favored.

For a game, the reward can be a terminal win. For a software task, a reward may include tests. For interpretation, neither model agreement nor passing a candidate's own tests is an external truth signal. A useful action reward could instead record a verified dependency retrieved, a replayable disagreement scenario found, or a previously open question resolved by authenticated evidence. Such a reward measures progress in an investigation. It must not be displayed as the probability that the resulting legal reading is correct.

The distinction is especially important when reflection uses the model's own explanation of a failure. A model can learn to satisfy a flawed test or to agree with its earlier answer. The resolution controller must keep the authoritative question and evidence fixed while allowing the candidate to change. The oracle cannot be rewritten merely to improve the candidate's reward.

Tree search also assumes that trial actions can be undone or replayed. The LATS paper makes this environmental assumption explicit. A bank cannot treat a completed payment or a disclosure of client information as a reversible trial. Here, search operates on hypothetical cases and retained evidence; authority to perform a real transaction belongs to the separately controlled host workflow.

The prototype includes deterministic scheduling as well as a bounded search option using UCT. The former makes explicit action priorities easier to inspect; the latter explores an alternative allocation of search effort. Neither has been established as the better policy for legal interpretation. Both operate under the same action, budget and release constraints. A comparison would ask whether it finds more independently judged material issues at equal cost. It would not ask whether it accumulates a larger self-assigned score. Chapter 9 specifies the needed experiment.

5.9 Choose questions that separate the surviving readings

Suppose four supported candidates remain. Three agree on a direct fee waiver, but they differ on a payment larger than the fee, a payment made without any fee, and a mixed voucher-and-waiver offer. Rechecking the direct waiver provides little discrimination. The system should identify the question whose answer is most likely to separate the remaining positions.

Generalized binary search formalizes a related problem for a finite hypothesis class and an answer oracle (Nowak, 2011, Figure 1 and sections I–II). Its identification guarantees have assumptions about the hypothesis space, queries and oracle. Legal interpretation may violate all three: a defensible reading may not yet be generated, a question may not admit a definitive answer, and the reviewer can be uncertain. We borrow question selection without claiming those guarantees.

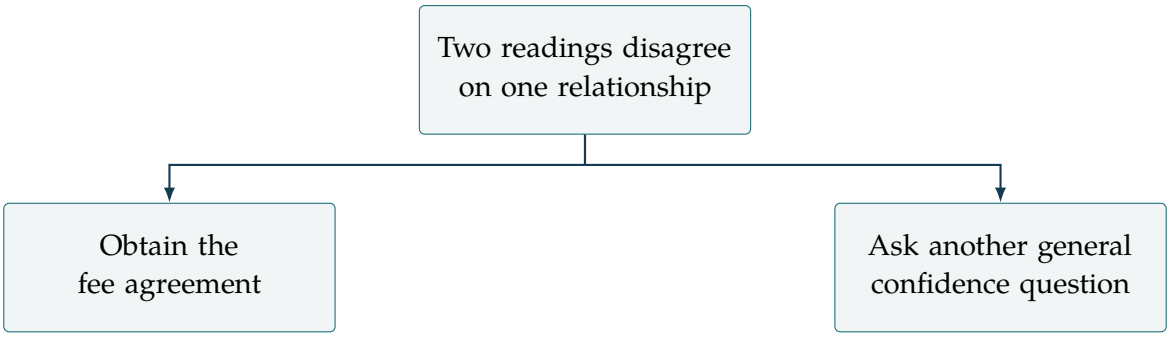


Figure 5.10: A targeted document request can separate readings that another vote cannot.

Before counting disagreements, both programs must be answering the same question. A true answer meaning "the restriction applies" cannot be compared with a true answer meaning "the offer is compliant." Nor can one program's fact about a particular voucher be silently substituted for another's fact about the whole promotion. Subject, outcome meaning and corresponding facts must be aligned first.

Within such a comparison, let H contain the retained candidates whose outcomes are defined on the same scenario x. Let $w_h \geq 0$ be descriptive priority weights and $o_h(x)$ candidate h's predicted outcome. A transparent heuristic counts the weighted pairs that differ:

$$D(x) = \sum_{\{h,j\} \subseteq H} w_h w_j \, \mathbf{1}\{o_h(x) \neq o_j(x)\}. \tag{5.2}$$

Each unordered pair contributes when the scenario makes its predictions differ. With unit weights, this simply counts separated pairs. A different common weight multiplies that count by the square of the weight. Dividing by the expected cost of answering the question can help scheduling, but the cost estimate is itself uncertain. Neither version converts the weights into posterior probabilities.

The implemented version uses unit weights and runs the original candidate programs on the proposed facts. It retains each Java result and its agreement with the Python calculation. Candidates with incompatible fact meanings, unsupported expressions or unfinished execution stay outside the checked set, with the reason recorded. Thus the number of separated pairs always belongs beside the coverage of the comparison. A zero after two comparable programs agree means something different from a zero because no pair could be checked.

A controlled gift-and-discount example separated two executable readings and influenced which repair the system attempted next. The retained marketing circular could not supply the same kind of comparison because the relevant fact bindings were incompatible; the settlement-readiness investigation had no checked pair. Neither was assigned an invented distinguishing case. A source check can still reveal an omitted exception when no program pair separates, so it keeps its place ahead of this ranking.

For two candidates, the useful scenario is any feasible one on which they differ. An SMT solver can sometimes construct such a scenario, subject to the domain conditions explained in Chapter 3. For classification disagreements, the solver may expose a vari-

able assignment but cannot manufacture evidence that makes it legally meaningful. A witness with a "discount" flag set to true is not an answer to the question of what establishes that flag.

The review question should therefore contain the facts, the rival conclusions, the source passage responsible for the disagreement, and the additional evidence or authority sought. "Which answer is right?" is usually too vague. "Does the incorporated definition require a reduction in the contractual charge, or can a later reimbursement qualify when it is capped at the paid charge?" identifies the missing premise. If the available authority does not resolve it, that is a substantive result that the report should preserve.

5.10 Uncertainty about meaning is not a vote count

Self-consistency samples multiple reasoning paths and aggregates their answers (Wang et al., 2023, section 2). Its evaluations show benefits on the studied reasoning benchmarks. The method deliberately distinguishes diversity of paths from a single greedy trajectory, but the samples still share a model and prompting context. Forty sampled answers are not forty independent legal experts. The paper does not establish omission detection or calibrated legal confidence for SFC circulars.

Semantic uncertainty goes further by grouping answers by meaning before estimating uncertainty (Kuhn et al., 2023, sections 3–4). This is helpful because ten paraphrases of one answer should not look like ten separate answers. In the idealized formulation, strings expressing the same meaning belong to a class c, and the probability of the class is the sum of the probabilities of its strings. The class entropy is then

$$H(C) = -\sum_{c} p(c) \log p(c). \tag{5.3}$$

Here the nonnegative class probabilities sum to one, and a zero-probability class contributes zero, using the limiting convention $0 \log 0 = 0$. A concentration on one class produces lower entropy than an even distribution across several classes. This measures uncertainty of the model's answer distribution under the selected grouping. It is not a proof that the concentrated answer is correct.

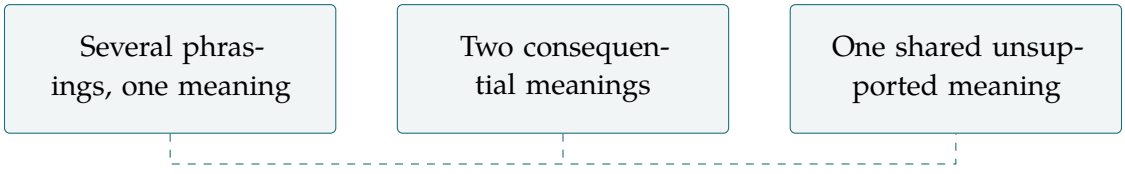


Figure 5.11: Linguistic variety and uncertainty about meaning are different quantities.

The paper approximates semantic grouping with natural-language inference. Its appendix discusses nontransitivity and sensitivity to clustering order. Our inspection of the official source identified a further implementation issue: the examined clustering condition tests the absence of label zero in either direction, rather than explicitly re-

quiring entailment in both directions. The retrieved model configuration maps label zero to contradiction, one to neutral and two to entailment: the inspected code therefore also permits neutral relations (Kuhn and collaborators, 2023; Microsoft, 2026). The model configuration was retrieved separately from the pinned code; its identity must be pinned as well before reuse. Even a perfect implementation of the stated mutual-entailment test would remain an empirical language-model judgment, not formal equivalence of bank controls.

Natural-language inference (NLI) is the task of judging whether one text supports, contradicts or leaves open the meaning of another. In this setting, the model compares two proposed readings; its score is evidence for a review decision, not a proof that the readings are interchangeable.

The workbench can use semantic similarity to propose clusters for review. It must not delete a dissenting candidate merely because an NLI model assigns a high score. Formal equivalence can justify merging execution paths only within its checked domain, while source assumptions and classification definitions stay visible. Chapter 9 develops calibration and set-valued reporting without assuming that a model's internal distribution is a distribution over legal truth.

5.11 Reusing research without importing its assumptions

A search method is useful here only if it helps answer a legal question. Consider the cashback dispute. The next useful task might be to retrieve the definition of "discount," compare two supported readings on a concrete offer, or ask a legal owner to settle a point that the available authority leaves open. A method that scores fluent answers but never changes any of these questions would add cost without addressing the uncertainty.

The inspected adjacent projects offer three different kinds of help. Source retrieval and retained reading records support the question "What evidence did we inspect?" Hypothesis and dependency records support "Which alternatives remain, and what could distinguish them?" Formal proof tools support "What follows from this precise rule under these assumptions?" Their source versions and the detailed reuse assessment are retained in Appendix F of the technical companion. Each still needs a legal adapter: a source store must understand annexes and amendments, an investigation record must retain competing readings, and a proof request must name the rule's actual meaning.

None of these tools supplies an authoritative reading of English. A numerical score used to prioritize an investigation is also not a probability that the reading is legally correct. The prototype should begin with a finite work queue ordered by materiality and dependencies. A more elaborate search policy becomes worth testing when there is evidence that this queue spends effort badly. Its comparison would concern useful questions resolved at equal cost, while source review continues to determine which interpretation is justified.

That leads to the ensemble design. Separate source inventory, competing readings, argument and formal comparison can expose different mistakes. The next problem is to make their disagreements produce a useful investigation, with an honest stopping point when the evidence runs out.

Chapter 6

Using disagreement to improve a review

Imagine five careful readers examining the same circular. If all five receive the same missing annex, their agreement says little about the omitted rule. If one reader inventories the source, another tests exceptions, and a third checks the consequences in a worked case, a disagreement becomes useful evidence. The bank can ask which reading is supported and what still needs a human decision.

The ensemble proposed here is built for that kind of disagreement. It assigns different readers different jobs, freezes the initial readings before debate, and gives each discrepancy a finite investigation. The controller's success is a clear account of what was resolved and what remains open, not a high vote count.

6.1 The design objective and its limits

The proposed system must make a dangerous path difficult: a plausible reading is accepted, translated correctly, and released without anyone discovering that it omits a material qualification. Multiple generators alone do not close that path. They may share a source extraction, a vocabulary, a training distribution and the same mistaken assumption. The design therefore uses redundancy across different activities, not just repeated answers to one question.

One activity inventories the source and its dependencies. Another identifies normative statements. Several candidate constructors express different readings. An opponent seeks a missing condition or counterexample. A formal evaluator checks consequences. A separate reference process supplies expected decisions for reviewed scenarios. A Java verifier checks preservation. Each activity has a defined responsibility and a defined way to fail. A failure or discrepancy is recorded even when every other member appears successful.

The main output is an interpretation dossier. It contains the proposed control, the source it covers, the readings considered, the reasons for their dispositions, the tests and formal checks, and the uncertainty that remains. The controller may prepare this dossier for review. It does not acquire authority to approve legal meaning merely because it has completed its work.

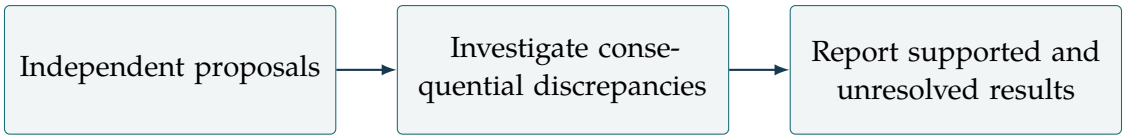


Figure 6.1: The ensemble's purpose is to investigate, not merely count votes.

There are three success conditions at different levels. An engineering run succeeds when it follows the specified state machine and produces a complete report within its limits. An interpretation investigation succeeds when it resolves its declared questions with adequate evidence or precisely identifies what requires judgment. A release succeeds only after the required people approve the exact interpretation and executable package. A run can satisfy the first condition while ending with an unresolved legal question. That is a valid and useful outcome, not an excuse to fabricate certainty.

No finite ensemble can establish that it has found every possible reading of unrestricted language. The operational objective is more precise: cover a reviewed source perimeter; require explicit treatment of every inventoried unit; search a declared set of interpretive dimensions; investigate every detected discrepancy under finite limits; and prevent a material unresolved question from being silently converted into release. These obligations can be implemented and tested. Their empirical effectiveness at discovering unknown omissions requires the evaluation in Chapter 9.

6.2 Why diversity has to be designed

Repeated sampling can reduce some random mistakes while leaving a shared mistake untouched. A simple probability model makes the distinction clear. Let q be the probability of a common failure that makes every member wrong. Conditional on that failure not occurring, assume for this illustration that each of m members has independent error probability p. The probability that all members are wrong is then

$$\Pr(\text{all wrong}) = q + (1 - q)p^m. \tag{6.1}$$

The two events are disjoint: either the common failure occurs, or it does not and all residual errors occur together. This derivation is an illustrative model, not an estimated error law for language models or lawyers.

If $q = 0.02$, $p = 0.1$ and $m = 3$, the expression gives 0.02098. Each member then has marginal error probability $r = q + (1 - q)p = 0.118$. A fair independence comparison holds that marginal probability fixed: three independent members would all fail with probability $r^3 = 0.001643032$. The smaller value $0.001 = p^3$ describes residual errors conditional on the common failure being absent; it is a different comparison. Adding members reduces the residual term in this example, but the common failure remains. A missing footnote in the shared extraction is a concrete way that the common failure could arise. Three careful interpretations of the same incomplete text may agree perfectly.

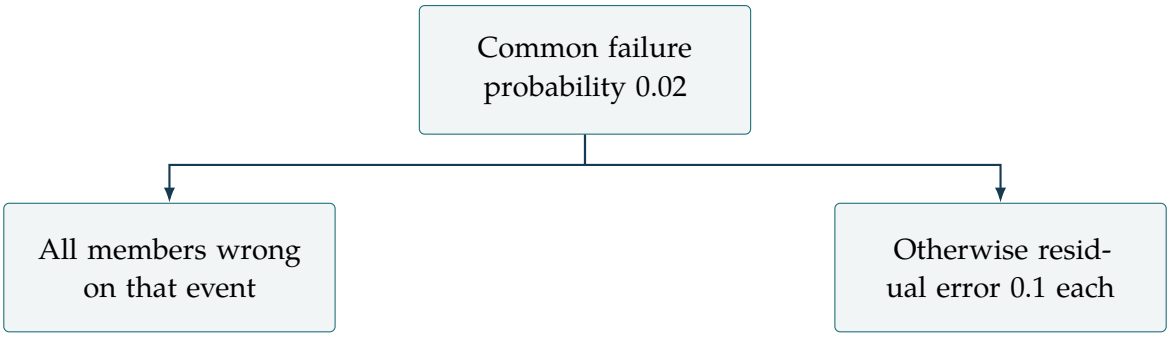


Figure 6.2: The common failure creates dependence between members.

In this synthetic model, every member has error probability 0.118. Holding that probability fixed makes the independence comparison meaningful. Adding members reduces the residual joint-failure term, while the shared-failure contribution stays at 0.02.

A second calculation concerns the variability of a mean of member indicators. Suppose X_i records whether member i makes a particular error. For $m \geq 2$, assume equal nonzero variance σ^2 and common pairwise correlation ρ. These must describe an attainable joint distribution. In particular, its correlation matrix requires $-1/(m-1) \leq \rho \leq 1$; Bernoulli indicators can impose further restrictions. There are m variance terms and $m(m-1)$ ordered covariance terms when the squared sum is expanded, giving

$$\begin{aligned} \operatorname{Var}\left(\frac{1}{m}\sum_{i=1}^{m} X_i\right) &= \frac{1}{m^2}\left(m\sigma^2 + m(m-1)\rho\sigma^2\right) \\ &= \frac{\sigma^2}{m}\{1 + (m-1)\rho\}. \end{aligned} \tag{6.2}$$

The expression explains why counting correlated members exaggerates the amount of independent information. It does not identify their correlation in practice, and it does not supply a legal error probability. Its role is to motivate measurement of shared failure and to discourage confidence claims based on panel size alone.

Diversity should therefore be recorded along several axes. Members may use different model families, prompts, representations, source acquisition routes, retrieval methods, factual schemas or verification methods. These differences are opportunities for complementary errors; they do not prove independence. Two vendors can share training material. Two parsers can both omit an image. Two human reviewers can both receive a leading case summary. The record should describe what differs and what remains shared.

The strongest inexpensive separation is often procedural. Let source inventory and initial interpretation happen before either sees the other's preferred conclusion. Freeze the initial outputs. Only then expose disagreements for investigation. This preserves evidence that one member independently noticed a condition. If all members see the first answer before responding, later agreement may merely reflect anchoring. Deliberation can still be useful, but it must not rewrite the history of independent proposals.

6.3 The ensemble members and their responsibilities

The prototype uses roles rather than assuming that every role is a language model. The source custodian acquires the official document, retains the exact bytes and metadata, and records incorporated materials. A document inventory process enumerates paragraphs, footnotes, tables, attachments and visible cross-references. A second inventory method compares the enumeration with the rendered document. Both methods may be automated initially, but unexplained mismatches remain review questions.

The normative reader identifies who must or may do what, under which conditions, with which exceptions and dates. A controlled-language reader expresses the same material using a restricted template. Their outputs are compared at the level of actor, activity, modality, object, condition, exception, time and dependency. This comparison can reveal a missing phrase even when the two prose summaries sound similar.

A counterinterpretation reader is required to propose consequential alternatives where the source supports them, and to say when it found none. It is not required to invent a controversy for every sentence. Its task includes seeking an omitted qualification, an alternative attachment of an exception, or a classification that changes a boundary case. Unsupported alternatives are preserved as such, which helps distinguish deliberate testing from legal disagreement.

The formal checker validates types, references and explicit semantics, compares candidate rules over supported domains, and replays witnesses. It has no permission to promote a source assumption to a fact. The scenario designer constructs examples that separate readings and includes unknown, conflicting and out-of-scope evidence. Its expected outcomes come from an independent review process or remain labelled provisional.

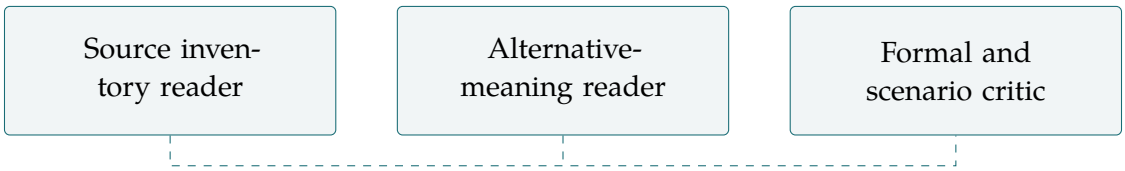


Figure 6.3: Complementary responsibilities create opportunities to notice different defects.

The implementation checker compares the accepted rule with the Java package and verifies the package's exact identity. Finally, the dossier assembler reconciles all required outputs and creates the review report. It may summarize, but it may not suppress a minority objection, change its severity or mark it resolved without a linked resolution record.

A role can fail in several ways: unavailable, malformed output, unsupported claim, resource limit, incompatible source version or abstention. Abstention is an informative response when a role lacks the evidence needed for its question. It is not a vote for the majority. If a role is mandatory for the configured coverage claim, its missing output prevents that claim from being marked complete. A replacement role must be explicitly

authorized by the configuration and its independence limits recorded.

The responsibilities are easier to understand in four groups. The first group establishes what the bank is looking at; the second asks what the source means; the third tries to break the proposed reading; and the fourth checks whether the accepted reading survives implementation and review. The short table gives the route a reader needs before the detailed record in Appendix E of the technical companion.

Table 6.1: Four jobs that must remain distinct when a circular is interpreted.

Job	Question it answers	Failure that must remain visible
Establish the source	Which passages, annexes and definitions are in scope?	Missing or inaccessible material.
Explain the meaning	Who must or may do what, under which conditions and exceptions?	Omitted actor, scope, duty or qualification.
Try to break the reading	Which supported alternative would change a real decision?	Unanswered objection or untested boundary case.
Check the delivered control	Does the implementation preserve the accepted meaning and its evidence?	Mismatch, stale release, unresolved reviewer disagreement or missing record.

The full record names the individual roles behind these four jobs. Keeping that record available matters for an audit, but the bank decision depends first on the questions and failures shown here.

6.4 Build a source perimeter before interpreting it

The source perimeter is the set of documents and versions the investigation claims to cover. It begins with the named circular, but incorporated definitions, annexes and transition provisions may expand it. Each dependency has a reason: explicit incorporation, definition of a term, amendment, background explanation, or a proposed analogy. These relationships must not all be collapsed into a generic "relevant document" link.

For 23EC46, the retained inventory has eighteen paragraphs and four footnotes. The executable slice covers paragraph 10. That is a legitimate prototype scope only if the report makes the remaining material visible and explains whether it is contextual, another obligation, a dependency, or outside the selected slice. Labelling all unimplemented paragraphs "not applicable" merely because the prototype lacks code would be wrong. Implementation capability is not a legal applicability test.

The inventory assigns each source unit a stable locator within a particular source edition. It records the extracted text, its position, and whether the unit contains a possible normative statement or reference. The second inventory can use a different parser or rendered-page inspection. A mismatch creates a source coverage discrepancy before any interpretation ranking takes place.

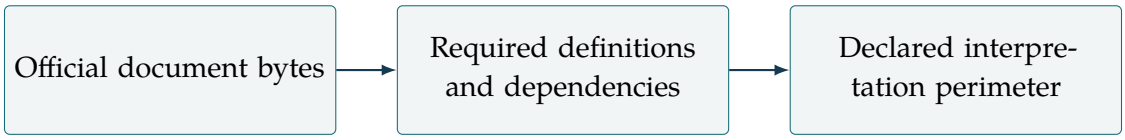


Figure 6.4: The investigation first establishes what material its answer relies on.

The controller expands dependencies until each required edge reaches a retained source, a justified out-of-scope disposition, or an unresolved dependency record. Cycles are legal as document relationships: a code and a guidance note can refer to each other. The traversal uses visited version identifiers to avoid infinite retrieval, while preserving both edges. A retrieval depth limit stops processing but leaves the unexpanded edge visible. It does not prove that the dependency is unimportant.

Publication date, effective date and retrieval date occupy separate fields. A circular published today may describe a later transition or an earlier arrangement. The source custodian cannot infer commencement solely from the HTTP response time or the first date in the text. The retained source packet is a historical evidence object. A current applicability assessment requires a separate search for amendments and supersession, with its own cut-off date.

This inventory is an important defense against common omissions, but it is not an oracle. An extractor might enumerate a footnote without understanding its legal effect. The requirement that each unit receives a reasoned disposition connects inventory to interpretation. A coverage check should fail if a unit is present but has no explanation, or if an explanation claims executable coverage without identifying the rule and verification evidence that provide it.

6.5 Freeze blind proposals before reconciliation

An initial proposal request contains the frozen source packet, the selected activity profile, the vocabulary already approved for that profile, and the member's task. It excludes other members' proposed conclusions. The response is stored with model or tool identity, configuration, prompt-template version, source hashes and completion status. Raw output is retained subject to the bank's access and retention policy; the structured candidate is a validated derivative.

A candidate cannot refer to a source version it was not given without opening a new retrieval request. This rule prevents a model's recalled description of an unseen amendment from entering as if it were inspected evidence. Recalled material can nominate a

search, but the supporting claim remains unverified until the source is acquired and checked.

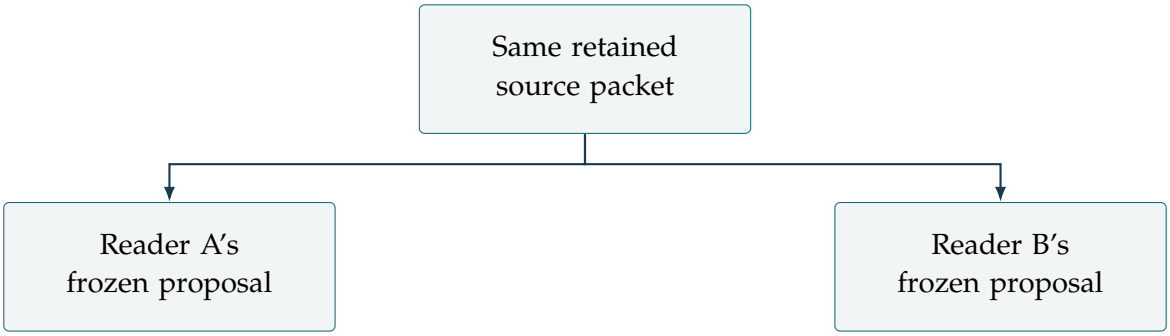


Figure 6.5: Freeze initial readings before showing the preferred answer.

The blind phase ends when every required member has responded or reached a terminal failure under the initial-phase budget. The controller records that boundary. It then compares the candidates and inventories. Later repairs may share all relevant evidence, because their purpose is informed resolution. The record must continue to distinguish the initial independent proposals from responses influenced by the shared discussion.

This separation prevents a misleading report such as "five independent members agreed" when four were shown the first member's answer. It also makes a useful experiment possible: measure which defects were caught before deliberation and which required shared investigation. A system that produces agreement by suppressing dissent should look different from one that resolves an objection with a missing official definition.

Candidate revisions are immutable. A revised rule points to its parent, states what changed, and links the evidence or diagnostic that motivated the change. The original remains available. Reviewer approvals attach to a particular exact candidate version, not to a mutable display name such as "latest rule." If a source, assumption or rule changes, the old approval cannot silently authorize the new candidate.

6.6 Detect discrepancies at the right level

A discrepancy is a structured difference that may affect coverage, meaning, execution or evidence. It need not be a pair of contradictory prose answers. One member may omit an input that another requires; two members may agree on a rule but cite different versions; one parser may miss a footnote; the Java runtime may return the same decision with an incomplete audit trace. All deserve a recorded disposition appropriate to their consequence.

The comparator first checks identity: source versions, activity profile, subject unit, candidate dependencies and evaluation times. Comparing answers to different questions creates false disagreement. A model interpreting a whole offer and one interpreting a single benefit must be identified as differing in subject identity before their truth values

are compared.

It then compares structured meaning: actor, action, object, modality, conditions, exceptions, quantifiers, temporal boundaries and references. Exact structural matches are cheap evidence of agreement about those fields, but source coverage still needs a separate check. Differences in presentation can be classified as nonmaterial only with an explicit rationale, such as identical canonical syntax or a domain-qualified equivalence result.

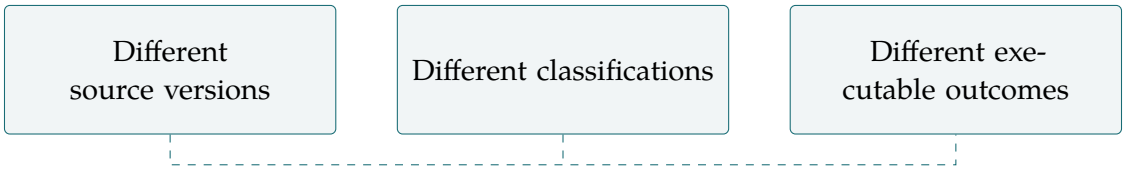


Figure 6.6: A discrepancy needs a classification before it needs a remedy.

Formal outcome comparison comes next. A counterexample is especially useful when two apparently similar rules disagree only at a boundary. For paragraph 10, omitting product-type linkage is exposed by $P = \mathsf{F}, T = \mathsf{T}$, with the other conditions established. Omitting the discount exception is exposed by a genuine fee discount. Neither example resolves whether a particular cashback is a fee discount; that is a different discrepancy with a different evidence need.

The comparator also checks omissions against independent inventories. Every required source unit, dependency and ensemble role must have a disposition. An all-agreeing set of candidates can therefore still produce discrepancies. This is the intended behavior: agreement among candidates cannot erase an uncovered paragraph or a failed source acquisition.

A discrepancy record includes its origin, affected source units, candidate identifiers, issue family, factual or legal question, observed difference, materiality, evidence needed, permitted next actions and current status. It also stores the consequence of leaving it unresolved. A classification issue might block a particular incentive category; a corrupted source record might invalidate the entire run. The scope of the block must be explicit and conservative when impact cannot yet be determined.

6.7 Keep the same question open across revisions

Suppose the question is whether the cashback payment counts as a fee discount. A model tries one reading, changes its wording and submits another. The question has not changed, so the second answer should not receive a fresh investigation budget merely because it is a new document. The proposed controller attaches the budget to the underlying question, called the *issue root*, and to the investigation as a whole.

An issue root identifies the source context, the substantive question and the affected subject or rule family. It does not include a candidate version as the only identity, because versions necessarily change during repair. The system assigns an immutable root

identifier when the question is first recorded. Subsequent discrepancies can link to that root or create a child issue. A child inherits the root's budget account and the run's limits. Creating a new display record never creates new spending authority.

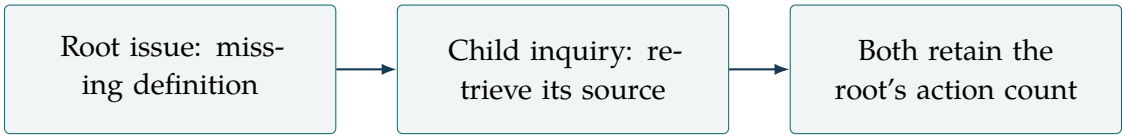


Figure 6.7: Creating a child issue must not create a fresh budget.

Automatic deduplication proposes that two issues have the same root when they concern the same source unit, semantic dimension and consequence. A reviewer or strict deterministic criterion must resolve uncertain merges. An erroneous merge could hide a second problem; an erroneous split could waste work. Global action limits protect against unlimited spending even when deduplication is imperfect, while the report shows possible duplicates for later review.

Issue identity also separates changed evidence from repeated reasoning. If an unavailable FAQ becomes available during the same run, the controller can attach the new version and continue within the remaining budget. If a later circular changes the applicable rule, that is a new source context and normally a new run. The old run remains closed with its original result. Opening the new run requires a recorded trigger and authorization; a worker cannot evade exhausted limits by automatically renaming the investigation.

A candidate that depends on several issues keeps all of their identifiers. A resolution of one issue cannot clear the others. The dependency relation also supports invalidation: retracting a definition affects every candidate that used it, even if those candidates were reached through different search branches. The service must query this relation directly rather than trying to reconstruct it from narrative summaries.

6.8 Finishing the work does not settle the question

An open question may be under investigation, awaiting evidence or waiting for a human decision. All three remain unresolved. They differ in what should happen next, not in whether the bank may assume its preferred answer.

There are three substantive ways to resolve a question. An inspected source or factual record may establish the missing premise. An authorized reviewer may make and explain the interpretive decision. Or a formal comparison may show that apparently different executions agree throughout the specified domain, with no distinct source commitment left in dispute. These conclusions require different evidence and cannot substitute for one another.

For example, an SMT equivalence result cannot resolve a dispute about whether the word "may" establishes a universal classification. Both candidates may have been en-

coded with the same unjustified classification flag. Conversely, an explicit source passage that defeats a missing-exception candidate need not wait for a probabilistic vote. The source-based resolution identifies the clause, updates the candidate and requires the affected formal and Java checks again.

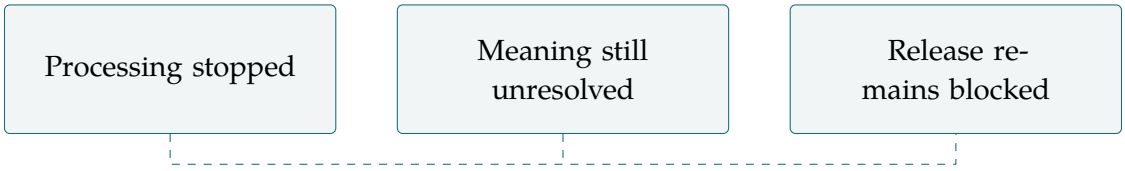


Figure 6.8: Completion of processing is not resolution of the legal question.

Automatic work can finish with the question still unresolved. The budget may be exhausted, the deadline reached, progress absent, a required source unavailable, or the investigation cancelled. An integrity failure can also force a stop. None of these reasons strengthens the candidate's legal position. The report must record both whether the work has stopped and whether the question has been answered. A single box marked "done" cannot convey both.

The implemented monitoring repairs now make a further distinction. Suppose the review requested an official-example check but that check never ran. The task is unfinished, even if the other checks returned successfully. A bounded retry may be useful. Now suppose the check did run and left two interpretations possible. Repeating the unchanged source check is not new authority for choosing one. The investigation can finish while the interpretive question remains open.

The same principle applies when a question was deferred because its comparison would exceed the allowed work, or when argument analysis could not complete. These cases retain an execution problem as well as any uncertainty about meaning. On the next monitoring visit, an unchanged source must preserve that distinction. Otherwise a temporary failure could turn into a clean-looking result simply because nothing new arrived.

Materiality is another separate field. An issue is material when its unresolved answer could change applicability, a required action, the decision, the timing, the explanation required for review, or the validity of the evidence chain. If impact is unknown, it is treated as material until assessed. Downgrading an issue requires a reason and authority appropriate to the configured review policy. A generator cannot label its own objection immaterial simply to unblock release.

A resolved issue can be reopened by new evidence, a source amendment, an invalid witness or a discovered dependency. Reopening creates a new event and invalidates results that depended on the old resolution. It does not erase the earlier record. If the original run is already closed, the successor investigation links to it and states which conclusion is being reconsidered.

6.9 Choose a remedy that can answer the question

Each discrepancy type has a permitted action set. A missing source calls for retrieval or escalation to the source custodian. A parsing disagreement calls for inspection of the original page, an alternative parser or manual transcription with provenance. A scope disagreement calls for analysis of the actor and activity language, incorporated definitions and the selected bank profile. A formal mismatch calls for a replayable witness and a correction to the relevant encoding or runtime component.

A useful action has a question, an expected evidence type, a bounded execution method and a success condition. "Ask another model" is incomplete. "Ask the alternative reader to identify source-supported reasons why the exception might attach to the whole offer, using only the retained paragraph and definition" is a bounded interpretive task. Its result is still a proposal, but the task makes clear what new information it seeks.

The controller should prefer actions with a plausible path to resolution. If the question is whether a fee was paid, querying transaction evidence may be more useful than sampling another legal explanation. If the question concerns the meaning of an incorporated term, more transaction data may be irrelevant. This matching of remedy to issue is where structured uncertainty becomes operationally valuable.

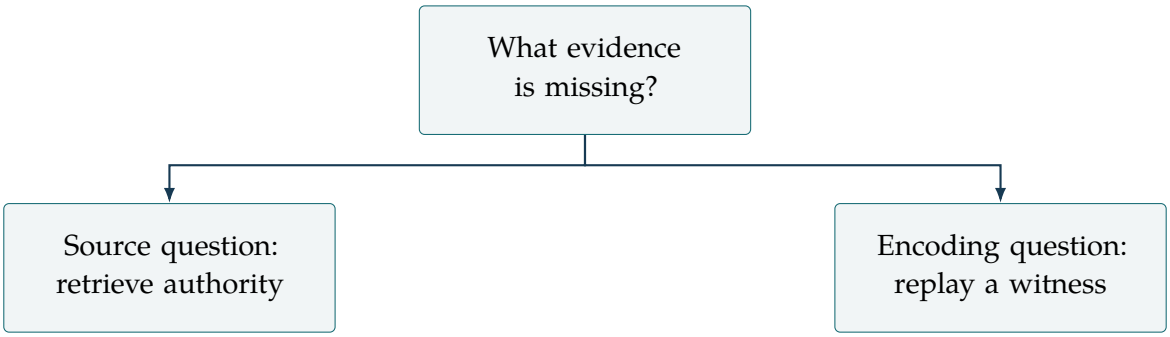


Figure 6.9: The remedy must be capable of answering the actual question.

An action can also deliberately seek a falsifier. For a proposed rule, request a feasible scenario in which it contradicts an express source condition. For a classification, request a fact pattern that the definition treats counterintuitively and ask which source justifies that boundary. For a claimed equivalence, ask the solver for a difference rather than asking a model whether the programs look similar. Falsification attempts must retain their domain and assumptions.

Repeated actions are allowed only when their rationale changes or a bounded retry policy permits recovery from a technical failure. Retrying an unavailable server can make sense; asking the identical question with no new evidence is unlikely to resolve a legal ambiguity. The report should show which retries addressed transport failure and which explored a substantive alternative. Both consume resources, but they mean different things for the investigation.

A repair is validated against the original discrepancy and against the unaffected re-

quirements. Fixing the type-of-product omission must not remove the fee-discount exception. A local witness becoming consistent is necessary evidence about that repair, but insufficient evidence about the whole rule. The controller runs the dependent validation set, records any new discrepancies, and keeps the old issue open until its resolution criterion is actually satisfied.

6.10 What evidence would settle each disagreement?

The next action depends on why the reviewers disagree. A missing annex calls for retrieval; a disagreement over whom the rule covers calls for a reading of the relevant passages and facts. Without this distinction, another fluent explanation can make a question appear settled while leaving its cause untouched. The table below gives the four questions that determine the next step. The complete row-by-row matrix remains in Appendix E of the technical companion for implementers.

Table 6.2: Choose the evidence that can answer the question.

Question	What would answer it?
Is the source complete and current?	Obtain and inspect the original pages, incorporated material and amendment dates. Record any gap that the legal owner must still decide how to handle.
Does the rule apply to this person, product and activity?	Read the scope and exceptions alongside the facts. The reviewer must explain why those facts bring the case within the rule.
Do competing readings change a decision?	Construct a case on which they differ. Use the relevant passage or definition to choose a reading, or retain the unresolved question.
Does the implementation preserve the accepted reading?	Run a feasible case on which the two implementations differ. Compare the facts, explanation and result; then check the repair or limit the supported cases.

A new paraphrase cannot repair a missing source, and a solver equivalence check cannot supply a commencement date. The action has to produce the kind of evidence that the question requires. The appendix records the finer distinctions among classification, timing, conflicts, unsupported operations and approval lineage.

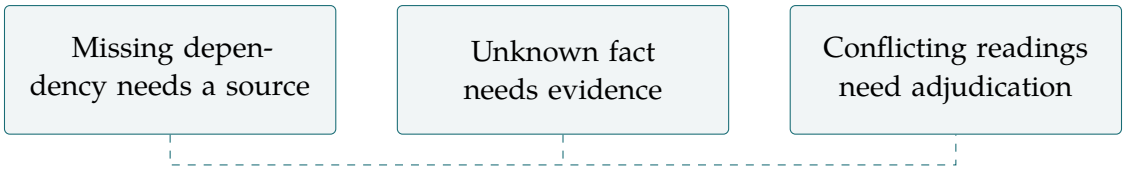


Figure 6.10: Different discrepancies have different closure requirements.

Some issues need more than one row. A cashback classification can depend on an unavailable definition and an uncertain factual relationship to a fee. Retrieving the definition resolves the dependency but may leave the factual question open. The controller should represent these as linked issues so that progress is visible without overstating closure. A single combined issue can be acceptable if its resolution criterion explicitly requires both conditions.

The matrix also guides action rejection. If a proposed action cannot produce the needed evidence, the scheduler should normally decline it and explain why. A new model paraphrase is not an appropriate remedy for a corrupted source record. A solver equivalence query is not an appropriate remedy for a missing commencement date. This does not prohibit exploratory work; it requires that exploratory work be labelled as such and charged to the investigation budget rather than counted as resolution.

Closure is evaluated against the issue's original question and current source context. Suppose a model responds to a component-level question with a correct whole-offer explanation. The response may be useful, but it has not answered the original question. The controller creates or links the relevant candidate and keeps the original issue open. This guards against a subtle form of apparent progress in which the task changes until the model can answer it confidently.

6.11 Resolution can reveal that the first question was wrong

A structured controller must still permit substantive reframing. The initial question may assume a distinction that the source does not make. For example, the team may ask whether a cashback is a discount when the relevant provision instead classifies the entire arrangement through another defined concept. The correct response is to record the changed understanding, identify which source supports it and replace the candidate with a new revision. The old question is dispositioned as superseded by a reasoned reframing, not silently deleted.

Reframing does not reset budgets. The successor issue links to the original root or consumes the same global account, and the report explains the relationship. If the reframing expands the source perimeter beyond the authorized run, the current investigation can end with a request for a successor run. This prevents a useful ability to change direction from becoming an unbounded search loophole.

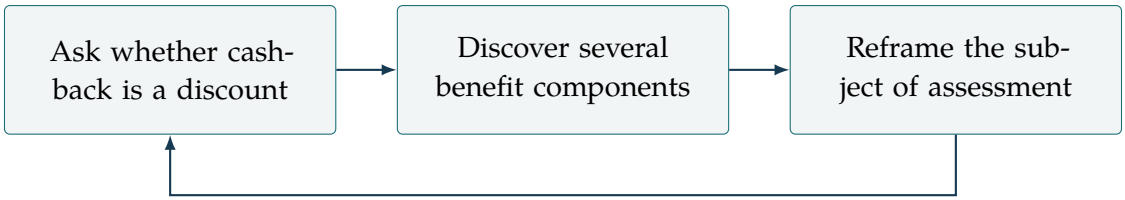


Figure 6.11: A useful investigation can correct its own initial question.

A concrete example concerns subject identity. A provider initially treats an offer as one benefit. The independent inventory identifies a source passage that requires separate treatment of components. The controller creates a component model, revises factual requirements and reruns affected scenarios. It also records that prior results used a different subject abstraction. Merely changing the formula while leaving the whole-offer adapter untouched would not complete the repair.

Another example concerns modality. A candidate initially turns guidance about factors to consider into an unconditional prohibition. Investigation shows that the source requires an assessment rather than a predetermined outcome. The repair may need a workflow obligation, evidence of consideration and a review record, not another Boolean threshold. If the current runtime cannot express the accepted requirement, the result is a supported interpretation with an implementation gap. That is more accurate than forcing the source into a familiar Boolean shape.

These cases explain why the issue register includes the affected decision and the kind of evidence required. The controller can recognize that the target has changed and invalidate dependent checks. A test suite for the previous target cannot automatically certify the new one, even if many examples happen to retain the same final status.

6.12 Follow the cause shared by several disagreements

The following diagram separates the objects that are often collapsed into a single chain of model messages. Several candidates can depend on one definition, and several discrepancies can affect one candidate. Actions investigate issues; results create evidence or revisions. The final report reconciles the graph rather than selecting the last message as the answer.

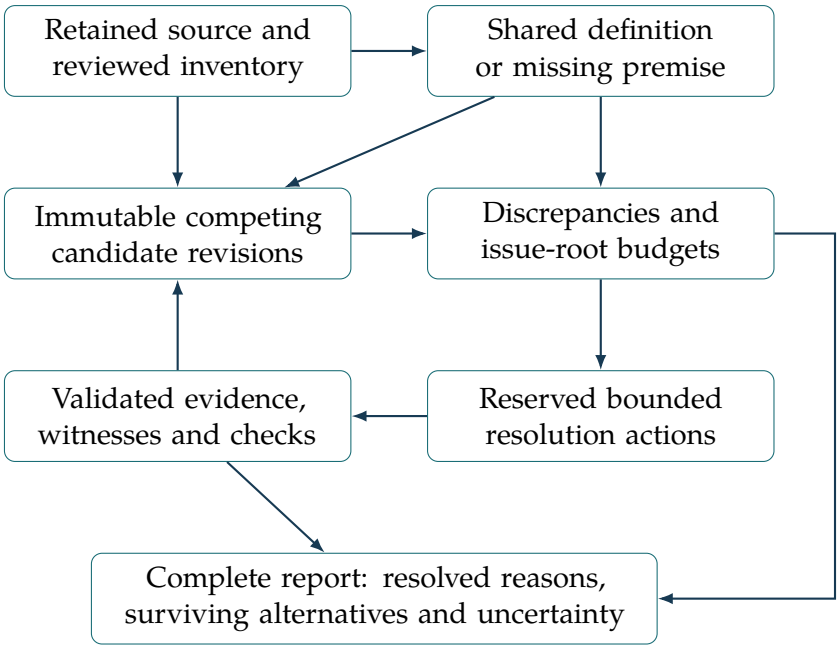


Figure 6.12: Search creates a history, while dependencies explain what must be reconsidered.

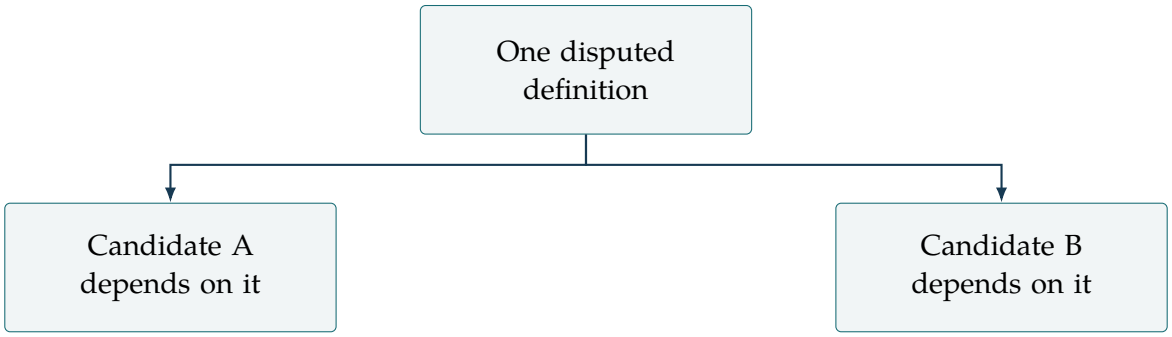


Figure 6.13: Shared causes should remain shared in the investigation graph.

An important consequence is that one new source can resolve several issues without spending a separate retrieval action for each. The action is charged once, and each affected issue receives a validated link to the result. Their root budgets do not acquire extra credits. Conversely, one failed source acquisition can block several candidates, and the report should identify that shared cause rather than presenting a long list of apparently unrelated uncertainties.

The graph also prevents misleading independence claims. If four candidate members all rely on the same extracted definition and factual mapping, the report can show that common dependency. Their reasoning paths may differ, but their evidence base is shared. This is a more useful account of redundancy than simply displaying four model logos beside an agreement percentage.

6.13 Set limits on an investigation before it starts

An investigation needs an agreed stopping point. Begin by distinguishing the initial independent readings from later rounds that try to resolve their disagreements. A resolution round consists of selecting currently open issues, reserving authorized actions, collecting bounded results and reconciling the resulting candidates. The round count increases when the controller commits the round's action plan, even if every external action later times out.

The configuration supplies a maximum number of resolution rounds, a maximum number of issued actions per issue root, a maximum number of issued actions for the run, and an absolute deadline. It also supplies per-action timeouts, candidate and graph-size limits, and any token or monetary limits. The count of technical retries in a drafting service does not determine how much substantive investigation a legal question needs. Those limits must be chosen and recorded separately.

A reference policy for deterministic fixtures uses three resolution rounds, three issued actions per issue root, and twelve external actions in total, including the initial proposal actions. It permits at most four initial external proposal actions, leaving at most eight for later investigation. These are engineering examples chosen to make boundary tests tractable. They are not empirically justified production defaults or recommendations about how much legal review is enough. Production configuration must be approved and evaluated for the selected task and cost constraints.

The reference policy also sets a ten-minute run deadline and a thirty-second external action timeout. Deterministic local checks have separate finite input and execution bounds. A mandatory check is never silently skipped when its bound is reached: it returns an incomplete result and blocks the corresponding claim. Token and monetary caps are nullable only when the provider integration cannot supply a reliable bound; such a configuration is labelled unsuitable for an unattended paid run. A paid deployment must supply conservative reservations.

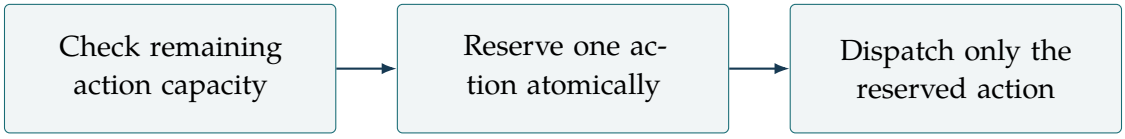


Figure 6.14: A budget controls external work at the dispatch boundary.

If eleven actions have been issued under a cap of twelve, two simultaneous requests compete for one remaining slot. Only one can obtain a reservation. Counting after dispatch would allow both requests to escape the cap.

Let B be the limit on external actions in the run and a the number already issued. For question r, let B_r be its limit and a_r the number issued against it. The current time is t and the deadline is t_{deadline}. A new action is allowed only when all three inequalities hold:

$$a + 1 \leq B, \qquad a_r + 1 \leq B_r, \qquad t < t_{\text{deadline}}, \tag{6.3}$$

The first two inequalities leave room for one more request; the third requires that time remains. For an initial proposal action, a separate initial-phase account replaces the issue-root account. The guard also checks the round, token, cost and action-type constraints. Passing one limit does not override another.

Reservation occurs atomically before dispatch. The issued count increases even if the worker crashes before learning whether the provider accepted the request. This conservative accounting prevents an ambiguous network result from becoming free work. A reservation can be cancelled without spending only before dispatch is durably committed and only through an atomic state transition that proves no worker can still send the action.

An action's effective waiting limit is the smaller of its configured timeout and the time remaining before the run deadline. The worker uses a monotonic clock for elapsed waiting, while the durable UTC deadline supports restart and audit. Clock adjustments cannot replenish an action budget. At the run deadline, pending actions receive terminal or remote-result-unknown dispositions; the controller does not extend the run simply because an action began a moment earlier.

Provider SDK retries must be disabled or included in the same accounting. A single logical action that secretly makes five billable requests defeats the meaning of the cap. When a provider offers idempotency keys, the adapter uses the recorded action identifier. When it does not, the system cannot promise exactly once execution across a lost response; it reports the uncertainty, counts the attempt, and follows the configured retry policy.

6.14 No progress is a substantive stopping reason

A finite global budget guarantees that external actions cannot continue forever, but it need not be spent in full. A no-progress condition avoids cycling through identical proposals. The controller computes a progress signature from accepted source additions, newly supported assumptions, new feasible witnesses, issue resolutions and genuinely distinct candidate semantics. Cosmetic wording changes do not count as progress.

The signature is a deterministic summary of validated changes, not a model's claim that its answer improved. If a resolution round produces no such change, the controller records that fact. The reference policy stops after two successive rounds without progress, unless an already reserved action is still awaiting a result within the deadline. This is an engineering stopping rule for the fixture profile. A production policy may choose another threshold, but must disclose it and evaluate the cost of premature stopping.

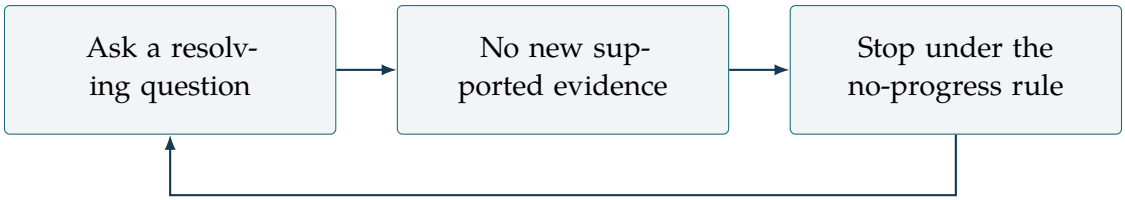


Figure 6.15: Repeated explanation without new evidence need not justify another round.

New unsupported candidates are not automatically progress. Otherwise a generator could prevent termination by inventing fresh interpretations indefinitely. Similarly, splitting one issue into several differently worded issues does not reset the no-progress count or any budget. The controller retains those proposals for review when useful, but their existence alone does not establish a reason to continue automatic investigation.

A source becoming temporarily unavailable may justify a scheduled successor run, but not an endless loop in the current run. The report records the failed retrieval, the affected question and the conditions under which a new run would be useful. A human or an authorized source-change service can initiate that run with a new recorded budget. This preserves the distinction between bounded automation and an operational process that may legitimately continue over days.

No-progress termination is not a judgment that the research direction has failed. It says that the current actions, sources and budget have not resolved the question. A later phase designed to obtain independent authority may still be justified. The system must report whether it encountered an invalid source, broken implementation, insufficient evidence, or simply a candidate that failed its checks. Those outcomes call for different next steps.

6.15 Choose the next useful question

The first implementation needs a concrete action order before a learned search policy is available. The reference scheduler uses a lexicographic key rather than an opaque weighted score. It first handles integrity and source-coverage problems, then missing dependencies and applicability, then definitions and exception structure, then formal and Java mismatches, and finally report or presentation discrepancies. This order is a proposed engineering policy: earlier questions can invalidate the inputs to later checks. It is not a legal ranking of every possible issue.

Within a class, unresolved material or unknown-materiality issues precede reviewed immaterial issues. Earlier creation sequence precedes later sequence, and the stable issue identifier breaks any remaining tie. Every selected action must be permitted for the issue kind and have remaining root and global budget. If an earlier issue is awaiting evidence that the available tools cannot obtain, it remains visible while the scheduler can make progress on another issue. Priority is not permission to spend the whole budget repeating an unavailable action.

For each issue, action selection follows a short decision procedure. Check whether an already retained source or result answers the question; if so, validate and link it without issuing a new external request. Otherwise, retrieve a named missing authoritative dependency when a permitted retrieval route exists. If the source is present but the candidate encoding differs, request a witness or a source-grounded repair. If competing supported readings remain, select a contrasting scenario or focused argument question. If no permitted action can supply the required evidence, prepare the human review question and stop automatic work on that issue.

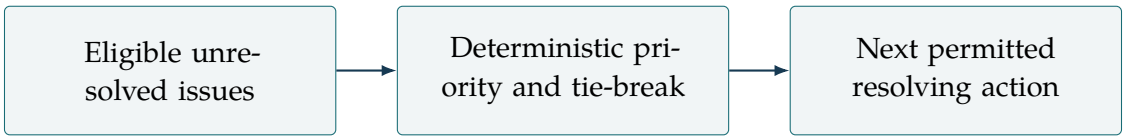


Figure 6.16: A fixed scheduler makes the first controller's choices reproducible.

Actions are not dispatched merely because the scheduler can name them. A predispatch validator checks that their inputs differ meaningfully from a prior failed substantive action, unless the retry is an explicitly permitted transport recovery. It also checks source identity, adapter support and the remaining budget. Rejection is recorded as a scheduling disposition; it does not consume an external-action slot unless a dispatch reservation has already been committed. Local scheduling and validation remain bounded by graph and execution limits.

Candidate breadth uses a similarly explicit rule. At initial reconciliation, reserve one retained slot for each supported interpretation family before allocating additional slots to refinements. If the number of families exceeds the candidate cap, retain the families in the same declared issue-priority and creation order, store the excluded family identifiers in the frontier, and open a search-incomplete issue. This policy favors reproducibility and visible incompleteness. It does not claim that its ordering finds the best legal theory.

For the optional pair-disagreement heuristic, the first prototype uses equal weights in equation (5.2). It enumerates the available feasible contrasting scenarios within the supported comparison fragment, selects the scenario separating the most retained pairs, and breaks ties by stable scenario identifier. A cost-aware or learned weight can later replace this rule under a versioned evaluation. Equal weights here are a baseline convenience, not equal probabilities that every reading is correct.

The implementation report records the scheduler version and each selection key. An engineer can therefore reconstruct why an issue was investigated before another, and a reviewer can challenge the priority policy without guessing at a model's hidden preference. Chapter 9 compares this deterministic baseline with enhanced search at equal resources. Both must obey the same budget, reporting and release constraints.

6.16 How an investigation proceeds

The controller is a deterministic state machine around fallible external tools. Its durable state contains the run configuration, source snapshot, required role results, candidate revisions, issue roots, action records, budgets and report status. External results enter through validation. They do not directly mutate approvals or release state.

Before an external request is made, the controller records both the request and its budget charge together. Either both are recorded or neither is. A worker may carry out that recorded request, but cannot quietly create extra requests. This rule prevents a restart from losing track of money already committed.

The sequence can be read as six ordinary steps.

1. Confirm which sources and business scope are being reviewed, which readers are required, and how much investigation is allowed.
2. Obtain the initial readings independently. Retain them before showing one reader another's answer, then identify missing material and disagreements.
3. For each unresolved question, choose an action that could answer it. Check that its inputs and cost are permitted, and record the commitment before sending the request.
4. Examine the returned evidence. Revise a candidate only when justified, repeat the checks affected by the change, and reconsider every open issue.
5. Stop when the required evidence is complete, or when a deadline, spending limit, integrity failure, cancellation or lack of progress requires a stop. The latter outcomes preserve the unanswered questions.
6. Produce a report in every case. Even when no disagreement was found, check that all required sources, readers and validations were present before describing the result as ready for human review.

The exact pseudocode is retained in Appendix F of the technical companion. The distinction to keep in view is between finishing an investigation and establishing a conclusion. A run can do the former responsibly while leaving cashback classification for an authorized reviewer.

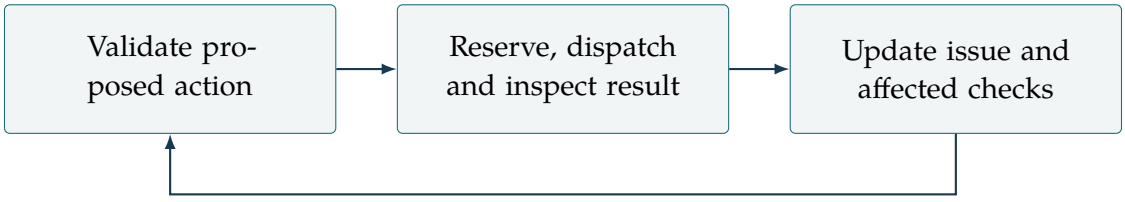


Figure 6.17: The controller changes state through validated operations.

Every detected discrepancy enters reconciliation, including differences later judged immaterial. A deterministic presentation-equivalence check can close a purely cosmetic

issue immediately with its reason; unassessed differences remain unresolved and conservatively material. The loop therefore processes unresolved issues rather than only issues already assigned a high severity. Integrity, cancellation and deadline guards are checked again before final readiness, even when no issue remains, so an empty loop cannot bypass a terminal condition.

The final coverage check is essential. If no comparator detects a disagreement, the loop might never run. The controller must still require complete source coverage, mandatory members and verification results. "No open issue" and "all required evidence present" are separate conditions. A failed member must also create an issue, but the final check protects against a defect in that issue creation path.

Actions that finish after the run deadline are retained as late observations. They do not reopen a closed run or retroactively change its result. A successor run may use them after validation. This rule makes reports stable and prevents a late worker from changing the package that a reviewer has already inspected. Late evidence can still trigger a documented follow-up when it changes risk.

The controller must produce a report even when it fails. If the primary report writer is unavailable, the durable run and action records support recovery of a minimal failure report. An integrity failure prevents a positive result, but it must not erase the explanation of what was attempted. Report generation itself has bounded resource use and a recovery path; an exception thrown while rendering a large argument graph cannot make the investigation disappear.

6.17 Termination and the boundary of the proof

We can prove a limited but useful property of the proposed controller. Assume a finite input packet, a finite configured action cap B, atomic issue accounting, finite local reconciliation with explicit graph limits, and an enforced timeout for every dispatched action. Assume also that the storage and supervisor either respond within their operational bounds or move the run to a recoverable failure state. These assumptions concern implementation and infrastructure; the proof does not establish them merely by listing them.

Define the remaining action budget as $V = B - a$, where a is the durable number of issued actions. Initially V is a nonnegative integer. Each new dispatch requires an atomic reservation and reduces V by one. No transition increases V in the same run. Therefore there can be at most B dispatches. Between dispatches, reconciliation is finite by its input and execution bounds. Waiting is finite by the action timeout and absolute deadline. When no permitted action remains, the controller enters a terminal state. Under these assumptions, automatic processing terminates.

This argument remains valid when an action discovers additional issues. New issues do not replenish the global budget. It also remains valid when a worker restarts, provided the durable counter is authoritative and a lease does not authorize a second unrecorded dispatch. The per-issue cap and round cap are useful additional constraints, but the

global cap is what prevents an unbounded stream of newly named issues from evading termination.

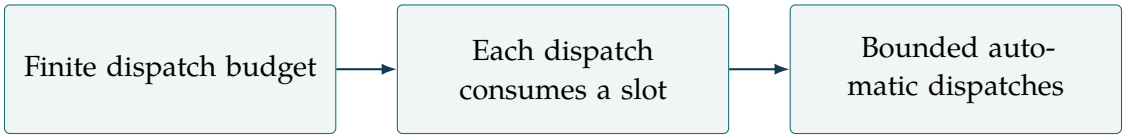


Figure 6.18: The termination argument depends on counting every external dispatch.

A finite action count alone does not stop a worker stuck inside one call. A wall-clock deadline and an independently enforced timeout are also needed for a bounded-duration service. The proof of a finite count and the engineering mechanism for stopping a call concern different quantities.

The proof is about bounded processing, not resolution. A controller can terminate with every difficult issue unresolved. It is also not a proof of wall-clock availability under arbitrary storage failure. If the process is indefinitely suspended or the database never responds, no local algorithm can guarantee a report at a fixed real-world time without an external supervisor. The operational design must therefore monitor leases and deadlines independently and recover a failure report when infrastructure returns.

A second invariant concerns release. Let $M(r)$ mean that run r has an unresolved material issue, $C(r)$ mean that its mandatory coverage and checks are complete, and $A(r, b)$ mean that the required authorized reviewers approved exact bundle b on the evidence from r. The release service requires

$$\text{Release}(r, b) \implies \neg M(r) \land C(r) \land A(r, b) \land \text{Integrity}(r, b). \tag{6.4}$$

This is a design invariant to be enforced by the release transaction. The controller's `READY_FOR_REVIEW` state supplies neither A nor release itself. An implementation test must attempt to bypass each conjunct, including through a stale approval or a source update, and verify rejection.

The invariant also exposes the limit of the design. If the inventory misses a material source and no check detects it, $C(r)$ could be incorrectly recorded. Formal verification of the release transaction would still prove only that it uses the recorded evidence correctly. Independent source review and empirical omission tests address the reliability of that evidence. The book's separation of proof claims is therefore retained inside the ensemble rather than abandoned at its most important decision.

6.18 Recovering a request whose answer never arrived

Suppose a request for a missing definition has been sent to an external service. The connection drops before the answer returns. The bank knows that it sent the request; it does not yet know whether the service completed it or charged for it. Repeating the request immediately may duplicate both the work and the cost. Recording it as a failure before submission would be equally misleading.

The investigation therefore distinguishes a request prepared for sending, one actually dispatched, a confirmed success or failure, a timeout and an uncertain remote outcome. These states describe what is known about the request. They do not settle the legal question it was supposed to answer.

Before sending, the system saves the question, its inputs, responsible issue, allowed time and budget charge together. A saved queue entry authorizes a worker to carry it out. If the worker fails, a replacement can recover that entry. The replacement receives a new ownership number, and an old worker returning late cannot overwrite the new owner's decision. This is the purpose of the software mechanisms called a durable queue, a lease and a fencing token; the full request record is retained in Appendix F of the technical companion.

A provider may accept an idempotency key, an identifier that makes a repeated submission refer to the same operation. Where that guarantee is available, a retry can recover the original work. Where it is absent, recovery must either leave the remote outcome uncertain or authorize another request and charge it as another attempt. No local status label can guarantee exactly one remote execution after an uncertain network failure.

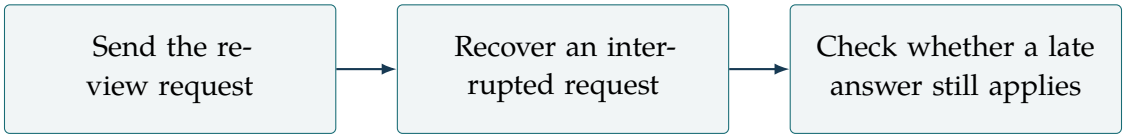


Figure 6.19: An answer to an earlier request must be checked against the current question.

The same distinction protects the spending limit. Confirmed unused monetary reservations may be refunded, but a dispatched attempt still counts towards the limit on attempts. Otherwise repeated failed requests could continue forever while appearing to spend no action budget. If a provider exceeds its documented cost bound, the system records the discrepancy and stops further spending until the integration has been examined.

An answer can also arrive for an interpretation that has changed while the request was running. The controller compares the issue revision with the one sent to the worker and checks whether the evidence still answers the current question. This prevents one worker's repair from erasing another's objection. A valid late answer remains available as evidence even if it cannot change the already closed report.

Cancellation stops new requests and attempts to cancel outstanding ones. Some remote services cannot guarantee cancellation. The report therefore keeps the outstanding work, late results and associated charges visible. Closing a browser must not erase either the cost of an investigation or evidence of an unresolved regulatory question.

6.19 What counts as a justified repair

A repair has two responsibilities: remove the identified defect and preserve the requirements that were already correct. For a missing product-type condition, the revised body must include the type route, and the previously verified gift, exception and scope behavior must remain intact. The repair record identifies the changed expression and the affected checks. It cannot simply report that a new model answer received a higher score.

The original discrepancy remains immutable. The repair links to it and supplies resolution evidence: the source clause, changed candidate, witness before and after, regression checks and any reviewer decision. A failed repair becomes another candidate revision with a failed status. This is useful evidence about the investigation and should not be removed from the dossier merely because a later revision passes.

A repair can be rejected at several layers. The source may not support its new condition. The controlled-language rendering may no longer match the formal rule. The formal rule may use an unsupported operation. The Java package may miscompute the accepted expression. These are different failures. A compiler rejection means the current encoding cannot be delivered through that compiler; it does not necessarily refute the legal reading. The next action might be a supported encoding or a documented implementation extension rather than a new interpretation.

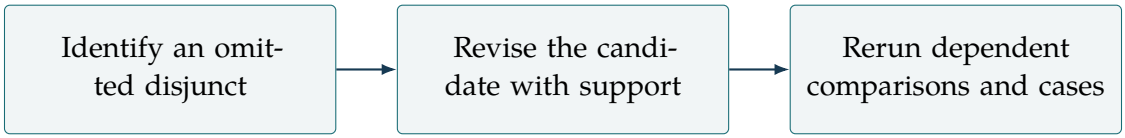


Figure 6.20: A repair changes both the candidate and the evidence needed for it.

Reference cases require special protection. When a candidate disagrees with a frozen expected outcome, the system records the disagreement and investigates both sides. It does not assume the reference is infallible, particularly in our same-author 22-case exercise. But changing the reference requires an independent resolution record with evidence and reviewer authority. A model cannot rewrite the expected result and then claim that its unchanged candidate now passes.

The same rule applies to formal domains. If a witness exposes a defect, narrowing the domain to exclude it is a substantive change. The domain change needs a source or operational justification, not merely a desire to eliminate the counterexample. Every equivalence claim includes the exact domain and assumptions, so an excluded scenario remains visible to reviewers.

Finally, repair must preserve dissent. If one supported reading is selected after review, the rejected reading, its strongest argument and the reason for rejection remain attached. Future evidence may make that earlier objection relevant. A system that stores only the winning candidate forces the next team to rediscover the same ambiguity and cannot explain why the original team chose as it did.

6.20 Scores that assist investigation without pretending to be probabilities

A single confidence number combines incompatible questions. A source can be authoritative while its application is ambiguous. A candidate can be internally consistent while missing a clause. A Java artifact can be verified against an unapproved rule. The proposed output therefore uses a score vector and explicit statuses rather than a scalar legal-confidence percentage.

The vector records source coverage, dependency completeness, supported and unsupported assumptions, unresolved material objections, scenario agreement, formal-check status, implementation-preservation status and search coverage. Some components are counts or categorical states. Others may be empirical scores, such as retrieval relevance. Their definitions are stored with the run. A score is not allowed to compensate for a failed mandatory condition.

For example, a candidate might cover eighteen of eighteen inventoried paragraphs with dispositions, yet have two unresolved incorporated sources and one material classification question. Displaying only "100 percent coverage" would mislead. The report must state that source-unit disposition is complete while dependency and interpretation resolution are incomplete. Conversely, a lower model ranking need not disqualify a candidate whose source argument remains undefeated.

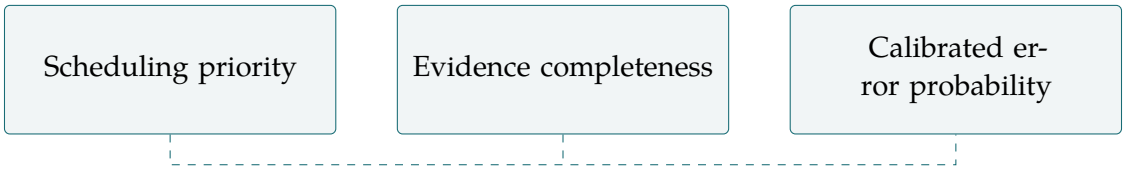


Figure 6.21: A useful priority score is not automatically a probability.

Search priority can be a weighted function of expected information gain, action cost and issue materiality. The weights are scheduling choices. They may be tuned under a declared evaluation protocol, but they are not probabilities of legal correctness. Every displayed numerical score includes its purpose and calibration status. A field named `probability_correct` remains absent or null until an appropriate target-specific calibration study supports a precisely defined event.

A useful shortlist can be a Pareto set. Candidate h dominates candidate j only when it is no worse on every declared comparable dimension and better on at least one. Even then, dominance is an aid to presentation, not legal defeat. A candidate with a unique source argument should not disappear merely because another has more completed engineering checks. Unsupported candidates can be rejected for specific reasons; incomparable supported candidates remain visible.

Set-valued reporting is especially valuable when authority is genuinely incomplete. The dossier may present two conditional proposals: under definition D_1, use rule R_1; under definition D_2, use rule R_2. It must identify who can resolve the definition and

what operations remain blocked. This is more informative than selecting one proposal with a decorative confidence interval whose coverage has never been established.

6.21 A complete dry run: repairing an explicit omission

The following trace is a designed explanation of the controller's decisions. Its particular sequence has not been executed by a live model and does not replace the retained circular investigations. The source packet contains the selected paragraph, its surrounding context and the unresolved dependency records. The activity profile and factual schema are fixed before proposals begin.

Four initial actions are reserved. The normative reader proposes $G \wedge P \wedge \neg D$, omitting product-type linkage. The controlled-language reader proposes $G \wedge (P \vee T) \wedge \neg D$. The alternative reader agrees that both product and type links require examination and identifies a separate classification ambiguity. The inventory member confirms the paragraph and footnote enumeration but reports an unavailable incorporated source. The run has issued four actions; eight remain under the reference global cap.

Reconciliation creates issue I_1 for the missing type route, I_2 for the classification ambiguity and I_3 for the unavailable dependency. The comparator does not average the two formulas. It constructs the known-fact scenario $G = \mathsf{T}$, $P = \mathsf{F}$, $T = \mathsf{T}$, $D = \mathsf{F}$, with scope established. The first candidate returns false and the second true. The source passage explicitly includes the type route, so I_1 has a concrete textual and behavioral basis.

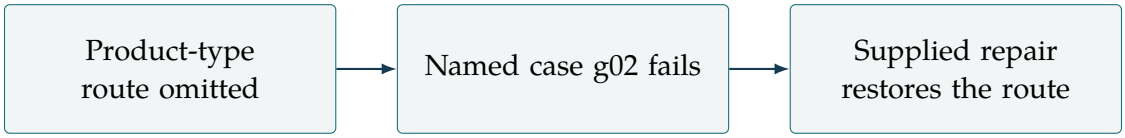


Figure 6.22: An encoding repair restores the omitted route; source questions may remain.

In resolution round one, the controller reserves a source-grounded repair action for I_1 and a dependency retrieval action for I_3. The issued count becomes six before either is dispatched. The repair produces a child candidate that adds T to the disjunction. Its controlled-language rendering, type checks and witness replay pass. The regression set checks that fee discounts remain excluded and scope uncertainty retains its established behavior. The issue is resolved by evidence with the original discrepancy and both candidate versions retained.

The dependency retrieval times out. Its action is terminal and counted; I_3 remains unresolved. The run does not claim complete circular interpretation just because I_1 was repaired. If the unavailable source could affect the selected classification, both I_2 and the relevant candidate remain blocked. The report can still state precisely that the explicit type-of-product omission was found and corrected.

This trace illustrates two kinds of progress: repairing a proposed rule and accounting for an attempted check that did not finish. As an illustration, it is not itself execution evidence. The retained development exercises separately include an executed omission repair, a deliberately shared missing exception, and bounded failures. Their results are recorded in Appendix A; none supplies an error rate on unfamiliar circulars.

6.22 A complete dry run: an ambiguity that survives the budget

Continue with the hypothetical cashback question. One candidate treats a later payment as a fee discount when it reimburses an actually paid charge and is capped at that charge. Another requires a reduction in the contractual fee itself. Their rules agree on a direct waiver and disagree on a later reimbursement. Neither definition is accepted merely because it produces a convenient outcome for the bank.

The scenario designer creates three synthetic offers. Offer A waives a charge of HK$100. Offer B charges HK$100 and later pays HK$100 back under a linked promotion. Offer C charges HK$100 and pays HK$150. The factual amounts are known, but the legal classification of the later payment remains disputed. The dossier shows the candidates' conclusions and the exact definition each requires. It also separates any excess payment into a proposed component analysis rather than silently treating the entire amount as a discount.

In round one, a retrieval action seeks the incorporated definition. If the source is inaccessible, the result is a failed retrieval, not support for either reading. In round two, a bounded argument action asks each candidate to identify the strongest source-based reason against its own definition. Suppose both return plausible but unresolved explanations and no new authority. The controller records the arguments, but a longer explanation alone does not constitute resolution.

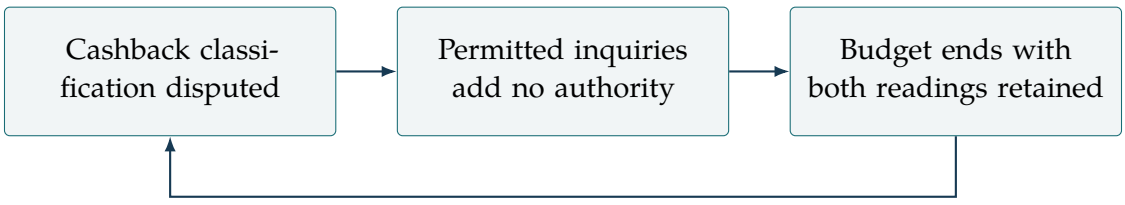


Figure 6.23: An unresolved ambiguity can survive a correctly executed investigation.

In round three, a question-selection action prepares a focused review question with the contrasting offers and the missing premise. The action can successfully produce the question while the issue remains unresolved. After the third round, no further automatic resolution round is authorized. The candidate set is preserved, the issue receives a budget stop reason, and the run closes as `BLOCKED_UNRESOLVED`.

The report states which offers are affected, what the candidates agree on, where they differ, which sources were inspected, which source remains unavailable, and what an

authorized reviewer must decide. It contains no assertion that three rounds make the leading reading probably correct. The unresolved classification is not replaced by a false Boolean, and the Java release route remains blocked for the affected control.

A later authorized reviewer may establish the definition from an applicable source or decide that a conservative bank policy should govern the uncertain category. Those are distinct decisions. A bank policy that declines all cashback offers pending clarification can be operationally useful, but it must be labelled as the bank's precautionary choice. It cannot be reported as a proven SFC prohibition unless the source supports that proposition.

6.23 Tell the reviewer what remains uncertain

An uncertainty report must let a reviewer reconstruct the decision without reading raw model transcripts. It starts with the source and activity perimeter, the proposed control, the run's terminal status and the operational consequence. A blocked report should say what is blocked. "Low confidence" is not enough: the reader needs to know whether the uncertainty concerns an incentive category, an effective date, a missing definition or the integrity of the entire source.

The report then presents the surviving interpretations in comparable form. For each, give the controlled-language statement, formal rule identifier, assumptions, supporting sources, strongest objections and distinguishing scenarios. Use the same factual scenarios across candidates so that the difference is visible. Candidate ordering may reflect a declared presentation policy, but must not imply that the first row has been approved.

Every detected discrepancy appears in an issue register with its processing and resolution states, materiality, affected decisions, attempted actions and final reason. Resolved issues remain present because their history explains why the final candidate differs from the initial one. Unresolved issues identify the missing evidence or human decision. A report that shows only unresolved issues would conceal the repairs; a report that shows only repaired issues would conceal the remaining risk.

Different residual questions need different owners. Missing code calls for an implementation repair; a missing definition calls for a source search; a missing settlement balance calls for the bank's fact owner. A disputed legal meaning needs an authorized interpretive decision. Each question should carry its exact source, surviving readings, affected controls and checks already attempted. Internal staff can often answer that narrower question without commissioning a new general review of the circular.

Coverage is reported against explicit denominators. Count source units inventoried and dispositioned, dependencies required and acquired, interpretation families identified and explored, mandatory roles completed, candidate comparisons attempted, and verification obligations passed or incomplete. The denominators must be retained even when the run exceeds a display limit. Large inventories can be paginated or stored as linked machine-readable records; they cannot be silently truncated to make the report look complete.

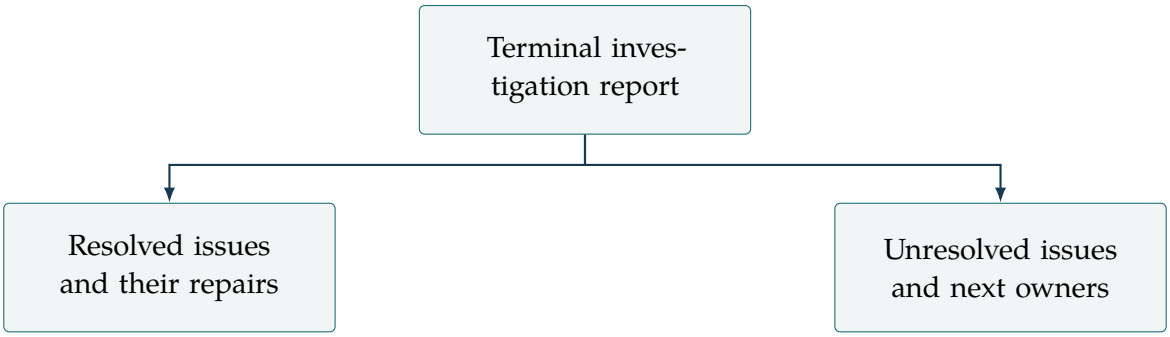


Figure 6.24: A complete report preserves both successful work and remaining uncertainty.

The execution account records configured limits, issued actions, rounds used, timeouts, unknown remote results, no-progress decisions, elapsed time, token and cost reservations, actual usage when available, and unspent budget. A run that stops early because it lacks authority should not look like a run that exhausts its resources. Both may be blocked, but their next useful actions differ.

For the unresolved cashback example, the first paragraph of a useful report could read as follows. The case is illustrative; the reasons and source passages would accompany it in an actual review.

> The cashback question remains unresolved after the permitted three rounds. Two readings remain: reimbursement of a charge, and a reduction of the fee itself. The available source does not establish which applies to this offer. The candidate cannot be released. A legal interpretation reviewer must establish the applicable definition before the affected decision can proceed.

This tells the reviewer what is uncertain, why automatic work stopped and what would change the result. It assigns no probability of legal correctness. The corresponding serialized record is retained in Appendix F of the technical companion.

The report should also state the limits of its evidence. In the second-circular exercise, the expected cases and candidate were prepared by the same author. That fact belongs beside the verification summary. A reviewer should not have to find a footnote in a separate research note to discover that the apparent oracle was not independent. Similarly, a solver timeout must appear as incomplete verification, not disappear from an average pass rate.

Reports are immutable versions. A corrected report points to its predecessor and explains the correction. Approval binds the exact report, candidate, source packet and Java bundle. The report is thus both a readable explanation and an index into reproducible evidence. Its human readability matters because a technically complete JSON object that no reviewer can understand does not solve the original manual-interpretation problem.

6.24 Agreement can still conceal a missed obligation

The hardest example is not a disagreement. Suppose every generator ignores a footnote that changes applicability. All formulas, scenarios and Java results agree because they share the same omission. Repeated self-consistency sampling would reinforce that agreement. Roundtrip translation could reproduce the same shortened meaning. Differential execution would confirm that the wrong specification is implemented consistently.

The independent inventory creates another opportunity to catch the omission. It requires a disposition for the footnote. A normative reader responsible for that unit may identify the qualification even if the candidate generators did not. A source-to-rule coverage check then opens an issue because the footnote has no corresponding assumption or rule. The repair propagates through candidate comparison and implementation checks.

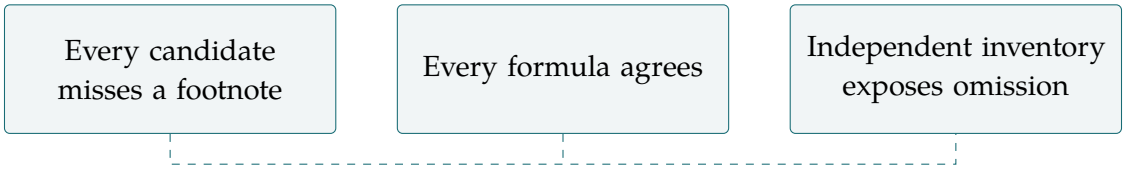


Figure 6.25: Unanimity is compatible with a shared missing obligation.

The development challenge in Chapter 1 gives this failure a concrete form. Three programs were deliberately altered to omit the gift exception. Their agreement gave a comparison of program outputs nothing to separate. Fresh source checks nevertheless left all three unsupported on the exception question, while supporting the clean control that retained it. The source inventory had a route to the missing premise that the vote did not. This is why source-omission checks retain priority even when the next scenario would separate zero program pairs.

Now suppose the inventory also misses the footnote because both extraction paths use the same defective text layer. Rendered-document inspection or a distinct acquisition route may detect it. If those are absent or also fail, the system can still miss the obligation. This remaining possibility is why the product's claims must be measured against independent source review and adversarial omission tests. Redundancy reduces opportunities for undetected error only to the extent that the redundant activities can fail differently.

A second common-mode failure concerns vocabulary. All members may agree that a payment is a discount because the prompt labels it "fee rebate." The remedy is to distinguish observed facts from proposed classifications. Present the payment, charge, contractual relationship and timing without inserting the disputed label as an established input. Then ask what evidence supports the classification. Blindness to the preferred answer is useful only if the factual packet itself has not already embedded that answer.

A third failure concerns evaluation. If the reference reviewer sees the model's answer before forming an expected outcome, the reference may inherit the model's mistake.

The ensemble may then appear accurate against a contaminated oracle. Independent initial judgments and recorded adjudication are necessary to measure whether the product actually discovers errors. They are also part of a credible operational review process, especially for novel or high-impact controls.

6.25 Interaction between uncertainty and operational action

The interpretation service should not decide every operational response. Its job is to state what is known, what is disputed and which controls are justified by the approved specification. The bank's host system then applies an explicit operational policy to outcomes such as unknown, conflict or blocked release. Those policies may include stopping an offer, routing a case to review or using an already approved fallback control.

This separation avoids a dangerous ambiguity in "fail closed." Closing an uncertain marketing offer may be appropriate in a particular process, but stopping a payment, withholding a disclosure or disabling an existing control could create other obligations or harms. The responsible business and control owners must approve the action for the relevant workflow. The interpretation engine supplies a reasoned status; it does not invent a universal response for all banking operations.

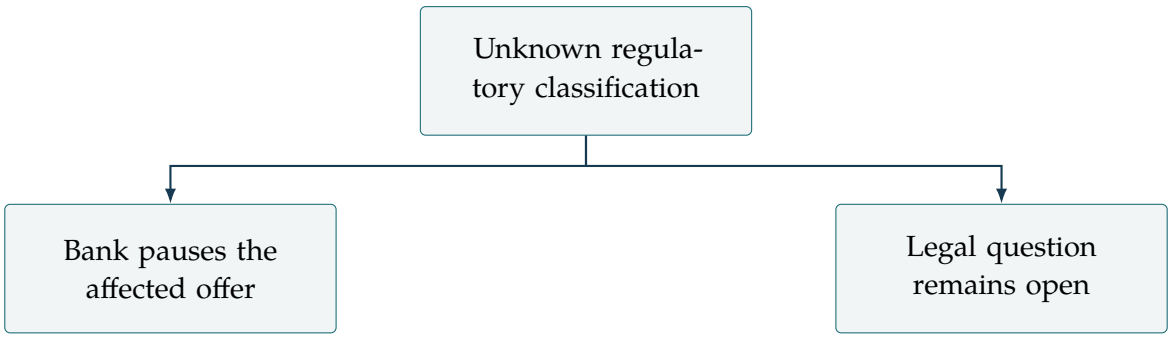


Figure 6.26: An operational precaution does not settle the legal interpretation.

For the selected gift rule, a true result indicates that the specified prohibition condition is satisfied. A false result says only that this condition is not established under the accepted facts and rule. It does not authorize the offer under every other requirement. The host must combine this result with other applicable controls, such as disclosure or suitability obligations, according to an independently reviewed orchestration policy.

An unresolved interpretation can sometimes be isolated. If the disputed cashback definition affects only one incentive category and the system can reliably identify that category, a reviewer may approve a limited control for unaffected cases while routing the disputed category to manual review. The scope restriction must be explicit, tested and included in the approved bundle. It is not enough to state informally that the system will "avoid edge cases."

If impact cannot be isolated, the block applies more broadly. The system should prefer an honest incomplete release proposal over an unsupported claim of separability. The uncertainty report gives the decision-makers the information needed to choose between narrowing scope, obtaining more evidence, retaining an existing control or postponing deployment.

6.26 Acceptance conditions for the central method

The first implementation should be accepted against deterministic scenarios before live model performance is assessed. A scripted member can deliberately omit the type route, supply a contradictory source version, fail to respond, or return a malformed candidate. These fixtures isolate controller behavior from model variability. Passing them establishes specified engineering behavior, not legal translation accuracy.

The controller must demonstrate that a unanimous but uncovered source unit opens an issue; that a supported minority candidate remains in the report; that a formal mismatch produces a replayable witness; that an unavailable dependency cannot be marked resolved by repeated explanation; and that a successful repair reruns affected checks. Each condition targets a concrete way an apparently successful ensemble could hide an error.

Budget tests must reach exact boundaries. With a cap of twelve, the thirteenth external dispatch must be rejected even if it is proposed by a new child issue. A crash after dispatch must not reset the count. A timeout must consume its issued action. A new worker with an old lease must fail to overwrite the current result. A provider retry must either reuse a verified idempotency key or consume a separately authorized attempt.

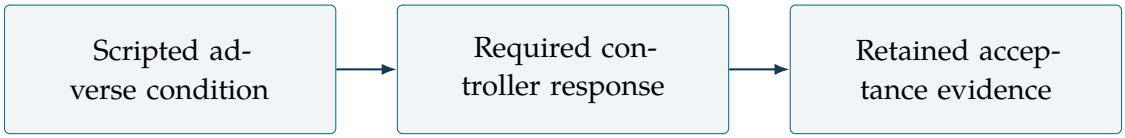


Figure 6.27: Acceptance scenarios test specific ways the controller could hide failure.

Report tests must verify completeness for successful, blocked, cancelled and failed runs. Every issue identifier must have a disposition; every unresolved material issue must appear in the blocking summary; every candidate revision must retain its parent and source references; and every displayed score must identify its purpose. Empty extension sets, solver unknowns and missing member results must remain explicit rather than being converted to success by default logic.

The release test is adversarial. Attempt release from `READY_FOR_REVIEW` without approvals, with an approval for a different bundle, after a source change, with an unresolved material issue, and after an integrity failure. All attempts must fail. A positive release fixture requires complete checks and authenticated approvals for the exact package. The local synthetic identity mechanism is sufficient to test the state transitions, but enterprise authentication and role enforcement remain a deployment requirement.

These conditions give an implementation agent a concrete target. They also preserve the intellectual purpose of the ensemble: investigate alternatives, expose missing evidence, and retain uncertainty when the evidence does not settle the question. The resulting product is useful because it makes responsibility and reasoning visible, not because a collection of fallible components can be renamed a proof of English meaning.

6.27 From the ensemble design to executed assurance

The design above names the checks that a trustworthy investigation would need. The following sections walk through one implementation of those checks. Their order follows the dependency a reader has to reconstruct: first methods that observe different failure opportunities, then the investigation that joins their results, then the distinction between a duty and a compliant performance. The later sections examine agreement across interpretations, the input meanings that such a comparison can lose, and the proof-carrying dossier that keeps each claim attached to its own evidence.

These sections report engineering behavior on retained source packets and controlled cases. They do not convert method agreement into legal authority. The source inventory, interpretation, executable target and operational policy remain separate objects throughout the investigation. A result can therefore close one question while leaving an earlier source or classification question open.

6.28 Assurance through methods with different opportunities to fail

The bank needs a reason to trust each transformation between a circular and a client decision. Consider the gift provision in circular 23EC46. A distributor may read a prohibition on gifts, encode it as a Boolean condition and deliver a Java class that implements that condition exactly. If the source packet omitted the qualification concerning discounts of fees or charges, every subsequent step could agree while the control remained wrong relative to the selected passage. The converse mistake matters too: a broad interpretation of the discount qualification could exempt something whose classification has not been established. Accuracy therefore depends on discovering missing qualifications, challenging factual definitions and checking executable behaviour separately.

There are at least three competing proposals worth distinguishing in this example. One prohibits any classified gift linked to promotion; a second excludes classified fee discounts but considers only a specific product; a third also considers a particular type of investment product. The first can miss the exception; the second can miss the wider product-type route; the third can still misclassify a voucher as a discount. These are illustrative formal proposals, not findings that all three are defensible readings of the retained circular. Their purpose is to expose where evidence is needed. The exact source passage, the referenced Code edition and the relevant FAQ must remain attached to the

question. A solver cannot supply an absent authority or establish the historical applicability of a later FAQ revision.

The additional assurance layer makes these opportunities for error visible. It requires methods that observe different parts of the problem: an image-based source reading, a structural text reading, alternative interpretation search, an argument solver, a second symbolic solver, and executable fault challenges. Each method has a precise responsibility. Failure of one method does not erase the work of another, but it prevents the affected claim from receiving a stronger status than its evidence supports.

6.28.1 Why independent executions need not mean independent errors

Separate implementations are useful because they can expose different mistakes. They do not justify multiplying error probabilities without evidence. Knight and Leveson (1986) studied independently developed programs and observed joint failures inconsistent with a simple independence assumption. The experiment concerned its particular programming task; it does not estimate a regulatory interpretation error rate. Its lesson for this product is methodological: separate development is a design choice whose effect must be measured.

Littlewood et al. (2001, section 4.1) explain one reason for correlated failure. Let X denote a randomly encountered input and let $\theta(X)$ be the probability that an independently sampled implementation fails on that input. Assume, within this illustrative model, that the two implementations fail independently conditional on the particular input. Across inputs of differing difficulty, the probability of joint failure becomes

$$\Pr(F_1 \cap F_2) = \mathbb{E}[\Pr(F_1 \cap F_2 \mid X)] = \mathbb{E}[\theta(X)^2] = \mathbb{E}[\theta(X)]^2 + \mathrm{Var}[\theta(X)]. \tag{6.5}$$

The last equality follows from the definition of variance. Both programs are more likely to fail on the same difficult input, so the joint probability exceeds the product of their marginal probabilities when difficulty varies. A distant footnote or an ambiguous actor definition is a plausible shared difficulty in our setting. The equation is an explanation of a mechanism, not a fitted model of LegalMath. We have not measured its parameters.

The language-model study by Kim et al. (2025) provides a related reason to avoid counting provider labels as independent evidence. Their evaluation examines agreement on incorrect answers in specified benchmark tasks, including cross-provider comparisons. Those tasks are not Hong Kong regulatory interpretation. We borrow the need to examine shared errors, and retain the distinction between a benchmark result and a claim about this bank's controls. If there are no observed errors in an evaluation cell, the shared-error fraction has no denominator. The report should say that the quantity is not estimable, not record zero correlation.

6.28.2 The source must be checked before interpretations are compared

An interpretation ensemble can inspect only the material it receives. The source stage therefore retains the original bytes, their hash, the source location, the selected edition and a page inventory. The new execution round uses the seven-page and three-page annexes retained for 23EC35 and the two-page appendix retained for 23EC49. These twelve pages are a bounded public-source integration corpus. Scanned exceptions and table footnotes are tested separately with explicitly synthetic fixtures; we do not assert that the retained circulars contain every difficult layout pattern.

The first extraction route uses the existing PDF text layer. ResearchAssistant supplies a second orchestration route with parser timeouts, error diagnostics and explicit partial-extraction states. Its available extractor is Poppler's `pdftotext`. Calling Poppler directly and through ResearchAssistant checks configuration and orchestration, but it is one extraction family. A count of three command invocations must not turn it into three independent readers. ResearchAssistant's reconciliation confidence is a diagnostic, not a legal reference, and missing equations or citations remain missing.

Docling adds a structured representation of document text, layout and provenance (Docling contributors, 2026). The adapter preserves its document JSON, text-item page references and bounding boxes where present. A Markdown export alone would lose the connection needed to investigate a footnote or table cell. The selected CPU configuration and model files are identified in the tool manifest. Features such as table reconstruction or picture description that are disabled in the pilot remain explicit limitations; their absence cannot certify a table's semantics.

Optical character recognition (OCR) converts the visible letters in a page image into machine-readable text. It supplies a different observation of a PDF from reading its embedded text layer, which can omit words visible to a person.

The OCR route renders every retained page, then invokes Tesseract directly (Tesseract contributors, 2026). It saves the raster hash, recognised text and TSV word coordinates and confidence values. This route observes visible pixels rather than relying on the PDF's hidden text layer. It can consequently expose a scanned qualification omitted by text extraction. It can also confuse digits, negation, superscripts and punctuation. Word confidence is used to locate uncertain regions, never to authorise a control. Two OCR routes that both use Tesseract must identify that shared backend.

Reconciliation operates page by page. It checks page coverage, empty pages, material token differences and exact text differences after whitespace handling. Negation, numbers, dates, punctuation and order are not silently removed to make the routes agree. A token comparison may help locate an omission but cannot establish that reordered words have the same meaning. Raw route text remains available beside the finding. A missing route, missing page or extraction error opens an unresolved source issue. A low discrepancy score cannot close it.

Suppose a synthetic page contains an image of the words "other than a discount of fees or charges", below otherwise machine-readable text. The text routes agree because

they see the same incomplete text layer. The OCR route supplies additional words. The controller links this difference to the page and requests a second rendering or OCR configuration within its declared action budget. If the result persists, the report retains the qualification as disputed source material. It does not discard the minority route. This is a useful positive result even when the controller cannot settle which extraction is correct.

6.28.3 Alternative models and the scope of formal checking

The existing interpretation engine already generates multiple candidates, preserves dimensions of disagreement and uses bounded search to investigate counterexamples. This round adds independent checking mechanisms around that engine. It does not replace search with a vote. Initial source-based candidates remain separate from fault mutants and from a backend reconstruction of an already chosen formula. These three kinds of object answer different questions: what the source might mean, whether a checker catches a deliberately changed rule, and whether a translation preserved its input.

For a small Boolean illustration, let G mean that the offered benefit is a gift, D that the fee-discount qualification applies, S that promotion concerns a specific product, and T that promotion concerns a particular product type. An illustrative prohibition condition is $G \wedge \neg D \wedge (S \vee T)$. A candidate using only S differs when G and T are true while D and S are false. This witness identifies exactly which factual question matters. It does not establish that the supplied definitions of G, D, S, T are legally justified.

cvc5 supplies a separately implemented SMT solver alongside Z3 (cvc5 contributors, 2026). The adapter accepts an explicit supported expression fragment and enumerated factual domains. It asks whether two expressions differ at any permitted assignment, and saves a concrete assignment when they do. Unsupported expressions, inconsistent domains, timeout and solver unknown have different statuses. None means equivalence. The first adapter covers Boolean conditions with explicit unknown handling and bounded integer threshold checks; the full RuleIR arithmetic, dates and host data mapping remain covered only by their separately declared checks until additional adapters are written.

The two solver encoders must be inspected separately. If both receive a shared translation that accidentally drops T, agreement exposes no such error. Accordingly the formal record distinguishes solver identity, encoder identity, fact definitions, supported domain and input formula hash. Exhaustive small truth tables provide a third diagnostic for the bounded fixture. Python and Java replay connect a solver witness to the executable control when that expression belongs to the generated-code profile.

clingo supplies a different formal mechanism: answer-set programming (Potassco, 2026). In the argumentation adapter, facts name arguments and attacks, and the program chooses a possible extension. Constraints exclude a set containing arguments that attack each other. A further condition requires every argument outside a stable extension to be attacked by an argument inside it. The solver enumerates all stable

extensions within the declared finite bound. The existing exhaustive Python graph evaluator provides the comparator.

Mutual attack illustrates why a single result is inadequate. For arguments a and b attacking each other, the stable extensions are $\{a\}$ and $\{b\}$. If both conclude the same proposition p, that proposition is supported in every extension even though neither particular argument appears in every extension. Conclusion-level skeptical support therefore requires, for every extension, some argument for p. Reversing those quantifiers would answer a different question. For an odd attack cycle there may be no stable extension; the absence of extensions is not universal acceptance. The adapter consequently requires a nonempty, complete extension set before making its conclusion-level statement. It preserves truncation and empty-set outcomes explicitly.

This finite graph interface is deliberately narrower than full ASPIC+ or legal argument construction. It checks the supplied attack relation. It does not establish the authority of a premise, infer a legally justified preference, or decide whether two real cases are analogous. TweetyProject, a collection of libraries for logical reasoning and argumentation, and s(CASP) remain possible extensions once their premise, preference and negation semantics are mapped to a specific product question. Adding every library before specifying that question would increase integration work without a corresponding claim.

6.28.4 Evaluation and faults in delivered Java

Inspect AI is an evaluation host with a scorer interface (UK AI Security Institute, 2026). Its role here is to execute frozen challenge cases and retain target, response and score records. The first integration uses deterministic responders, so it tests the evaluation plumbing and reference separation without spending live model allowance. The responder receives the public case while the target is kept in the scorer. A shared wrong answer must fail against the target even if every responder supplies it. A missing answer remains missing. This does not estimate a live language model's accuracy or justify selecting a model.

PIT mutates Java bytecode to challenge tests (PIT contributors, 2026). A threshold can change from greater-than-or-equal to greater-than; a branch can be negated; or a Boolean result can change. The selected pilot control and its tests are compiled with the retained Java 17 toolchain. The first run establishes that the baseline tests actually pass. PIT then reports which mutants the tests kill and which survive. The report preserves mutator identity, source line, test coverage and status. A surviving material mutant prevents acceptance of that test claim.

The PIT pilot is not a whole-application Java proof. Existing LegalMath tests already compile generated classes, replay symbolic witnesses, compare Python and Java over specified states and challenge runtime outcomes. Those tests remain mandatory and are run alongside the new pilot. In particular, a true prohibition predicate must not become a generic permission flag in the host. Passing an isolated Boolean test does not establish the output convention, fact freshness, time-zone handling, decimal units or workflow composition of the bank system.

The proposed calibration work would use separately labelled examples to assess the coverage of sets of possible answers. MAPIE is a software library for constructing such prediction sets. Chapter 9 explains why their coverage guarantees require assumptions about the reference labels and the relationship between calibration and future examples.

At the time of this assurance round, the Catala experiments were in a separate worktree. Their separately authored examples examined specification choices, while a generated adapter checked translation from a common input. The completed Catala campaign has since been merged into main; Chapter 8 describes its shared frontend and supported targets. This earlier round retains its original tool identities and does not claim to have tested the later translator. KeY proofs of selected Java/JML obligations and MAPIE calibration are later work: the former needs specific specifications and closed proof obligations; the latter needs an appropriate separately labelled calibration corpus. Neither tool is useful as an empty completion checkbox.

6.28.5 A common result and a bounded discrepancy investigation

Each assurance result names its source and claim, method family, implementation and tool versions, shared dependencies, input hashes, supported scope, completion status and evidence location. Its finding is supported, contradicted or unresolved relative to that stated claim. The controller also records whether the method actually ran. An unavailable executable is not a negative finding about the law; it is missing evidence. A complete execution with a discrepancy is successful detection, but the affected semantic claim remains unresolved.

The controller opens one issue for each material difference or unavailable required check and records the action before dispatch. A source issue may request a second OCR configuration; a formula difference requests a replayable witness; an argument difference requests complete extension comparison; a Java fault requests a narrower regression case. Each action has an immutable identifier, input hash, timeout and outcome. Repeated attempts do not increase a confidence score. The budget limits investigation work; it cannot lower acceptance criteria.

Repairs fall into two classes. A mechanical repair, such as repeating a page extraction with a declared OCR segmentation mode or regenerating a stale plan, can execute automatically. A semantic revision, such as changing what counts as a fee discount, needs new source-backed evidence and must return through the interpretation process. The controller never edits the rule merely to make backends agree. Exhausted investigation produces an explicit uncertainty report containing the remaining alternatives, consequences, missing evidence and the next discriminating question.

A crash is also an outcome. Issued actions remain consumed, partial evidence remains visible, and recovery cannot reset the budget. Evidence from a previous source or formula version cannot close the current issue. The implementation therefore binds decisions and plans to input hashes and verifies predecessor evidence before continuing. Tests must deliberately change a source, truncate an extension list, remove a required

route, interrupt a subprocess and exhaust the final action to ensure these protections actually execute.

6.28.6 The executable implementation round

The round-11 master program starts with this documentation phase. It then installs the selected tools into a separate CPU environment and a local OCR directory, records exact versions, and runs small import and fixture checks. It next runs the source routes on all twelve retained pages, tests alternative-model and symbolic mechanisms, evaluates the deterministic Inspect cases, executes the PIT pilot and runs the existing regression suite. The final phase joins evidence without upgrading the scope of any component result.

Before each phase, the program writes a plan containing current implementation hashes, predecessor outcomes, acceptance conditions and unresolved findings. A failed phase produces a repair request and retains its attempt directory. A retry requires a recorded executed repair and regression result. Successful completion refreshes the successor plan automatically. Fixed subprocess argument vectors and a narrowly scoped installation approval prevent model-generated commands from changing the toolchain or invoking arbitrary services.

There are two final decisions. Engineering completion asks whether the declared tools ran, failure controls behaved as specified and required evidence was preserved. The interpretation decision asks whether the particular legal question has sufficient source-backed support. The first can pass while the second remains unresolved. This distinction makes a useful product possible: the bank can obtain reproducible evidence and sharply stated residual questions without purchasing a false guarantee that every future circular has only one correct machine reading.

The next empirical study must measure missed material conditions, unsafe acceptance, correctly accepted coverage and residual review effort on distinct source families. Artificial faults help verify detection mechanisms but cannot estimate the prevalence of legal mistakes. The current round is intended to make that study executable and its evidence harder to misinterpret. Its results are reported in the round-11 execution record, with tool identities, retained failures and the exact next-phase proposal.

6.29 From complementary checks to a working investigation

Return to the offer containing a fee waiver and a separate voucher. A programme can implement its conditions exactly and still miss the voucher if the input record describes only whether the offer contains a discount. In that situation, adding another compiler leaves the missing distinction untouched. The necessary repair is to reconsider what the facts describe. The proposed investigation therefore treats the factual model as a revisable interpretation of the source. One proposal may describe the whole offer; an-

other may distinguish each benefit, its recipient and its connection to a product. Both proposals retain their source passages and classification assumptions. They cannot be treated as equivalent merely because both use a variable named `discount`.

This change gives breadth-first investigation a substantive purpose. Before ranking readings of the exception, separate readers propose the actors, components and dates needed to express it. A challenger then asks which relevant distinction all those proposals might have missed. Subsequent search investigates cases that separate the proposals. For the combined offer, assessing the voucher separately is such a case under the interpretation developed earlier in this book. Whether a particular benefit is legally a gift remains a classification question; a mathematical comparison cannot supply that classification.

The source itself must also remain open to challenge. Suppose the discount exception appears in a footnote recovered by OCR but omitted by the embedded PDF text. Comparing whole-page strings identifies a difference but does not explain it. The investigation needs the footnote's page region, the text recovered by each method and its attachment to the relevant paragraph. Additional work then has a specific purpose: inspect that region, recover its wording, or locate the corresponding official text. Ordinary line wrapping can be reconciled without changing words. A missing negation or uncertain table header cannot. Agreement among text parsers does not settle a disagreement with the rendered page because the text parsers may share the same omission.

Each discovered problem consequently has a history. The record states which source, factual model and proposed rule it affects, which action was taken, and whether new evidence answered the question. A retry that produces the same unsupported assertion adds no authority. The maximum number of actions is fixed before execution and persists across interruptions. If the evidence is still insufficient, the report retains the rival readings and the smallest unanswered question that distinguishes them. Such a question can often be assessed more economically than commissioning a new review of the entire circular.

Consider next an amendment to the provision defining the relevant distributor. The wording of the gift paragraph might be unchanged, yet its applicable actors could differ. The investigation must connect the proposed rule to the edition of that definition and the relevant date. A changed dependency invalidates the affected conclusion and its supporting execution checks. Historical replay requires both the rule applicable to the conduct and the evidence available at the historical cutoff. A later publication cannot silently become evidence that was available earlier. Where the authority or temporal relationship is disputed, the program reports that dispute instead of inventing a priority rule.

Once a factual model and executable interpretation are explicit, the execution checks become precise. Python, generated Java and a separately compiled Catala calculation can be run on the same cases. An independent solver can seek an input that makes two formal proposals disagree. A discovered input is then executed in the actual binaries, including their explanations and evidence references. If a compiler mutation changes an inclusive threshold to a strict threshold, the boundary case must detect the change. If two routes share the same input validation or date handling, those shared operations

still need their own challenges. Multiple compilation paths do not make every layer independent.

The distinction between these checks and a proof matters. A proof for a specified language fragment would state that every valid input receives the result prescribed by its formal semantics, under explicit runtime assumptions. Finite conformance cases are evidence for that proposition, not its universal derivation. A theorem about a mathematical evaluator requires an additional connection to the emitted Java before it establishes a property of that Java. Neither proposition, even when proved, establishes that a chosen English interpretation included every legally relevant distinction.

A useful demonstration must therefore contain both successful decisions and unresolved cases. The clear cases establish that the system can produce a working Java package and apply its stated conditions. Deliberate omissions, competing factual models and ambiguous authorities establish whether the investigation exposes the situations in which its answer needs more evidence. A system that refers every case to a reviewer has demonstrated caution but has not demonstrated the intended reduction in manual interpretation work.

Reference cases must be prepared with the same distinction in mind. An official example can provide direct evidence about the example it addresses. A constructed case tests an explicitly stated interpretation. A case and rule written by the same agent may repeat the same mistake. Their agreement remains useful for checking execution, while independent source assessment is needed for a stronger interpretation claim. The evaluation preserves these different origins rather than combining them into one accuracy percentage. Related scenarios from one circular also remain related observations; thousands of truth-table rows do not become thousands of independently checked legal judgments.

The implementation target is one investigation joining these steps and retaining their evidence. The phase plan fixes the supported scope, executes repairs after failures, and refreshes the next phase when its assumptions change. Its first demonstration concerns public circulars and declared control profiles. Measuring reviewer savings requires actual review observations; estimating rare legal errors requires a suitable independently supported corpus. Those evaluations can follow a working demonstration, provided its report makes the remaining questions visible.

6.30 A certificate requirement is a different question from a compliant submission

Consider an XML suspicious transaction report submitted without the required electronic certificate. Two correct answers are possible: the certificate requirement applies, and the submission does not satisfy that requirement. They disagree as Boolean values because they answer different questions. A system that treats them as rival interpretations can reject a useful rule, or select the wrong rule merely because it agrees with a test labelled “compliance.” Before comparing formulas, the investigation must identify

what the result is about.

SFC circular 26EC2 makes this distinction concrete. It lists three ways to submit an STR through STREAMS 2: XML, upload of the prescribed PDF form, and completion of the web form. Footnote 3 attaches the electronic-certificate requirement to the first two methods. Footnote 2 separately requires reports submitted through other channels after implementation to be resubmitted through STREAMS 2. The transitional arrangements provide direct JFIU contact for urgent reports during the blackout. Contact during the blackout, a compliant submission after launch, and a duty to resubmit an earlier report are consequently separate decisions (Securities and Futures Commission, 2026b).

For a deliberately limited example, suppose a licensed firm's original report was submitted after the operational launch and its method is known to be exactly one of the three listed methods. Let x, p and w denote selection of XML, PDF and web form respectively. Let s mean that the original event used STREAMS 2, and c mean that the certificate condition was fulfilled in the accepted technical manner. These are facts about one event. Establishing the firm's regulatory classification or the technical sufficiency of a certificate requires evidence outside these Boolean formulas. Under those assumptions, the certificate-applicability predicate A, the selected submission-control predicate C, and the original-event resubmission trigger R are

$$A = x \vee p, \tag{6.6}$$

$$C = s \wedge \big(w \vee (x \wedge c) \vee (p \wedge c)\big), \tag{6.7}$$

$$R = \neg s. \tag{6.8}$$

The scope conditions remain separate from these expressions. In particular, R is meaningful here only for an STR actually submitted to the JFIU after the selected implementation boundary; it is not a duty imposed on every object whose record lacks a STREAMS 2 flag.

For the XML example, $x = \mathsf{true}$, $s = \mathsf{true}$ and $c = \mathsf{false}$ give $A = \mathsf{true}$, $C = \mathsf{false}$ and $R = \mathsf{false}$. The failure to supply the certificate does not establish that the report went through another channel. For a completed web-form submission, $w = s = \mathsf{true}$ gives $A = \mathsf{false}$ and $C = \mathsf{true}$ even if certificate possession is unknown. An unknown method must not be represented by three false flags: the adapter preserves the method as unknown. The evaluator then cannot conclude that the certificate condition does not apply. Conflicting facts remain conflicts under the previously defined semantics.

These formulas implement only the named conditions. The circular also discusses XML schema conformity, technical testing, registration and transitional contact. The combined output is a vector of the selected decisions, with their unresolved dependencies, rather than an approval of the firm's reporting arrangements. This preserves the practical benefit of an executable rule while preventing a partial rule from inheriting the meaning of whole-document compliance.

6.30.1 Checking a smaller question without losing the rest of the circular

A typed control records its question, actor, unit of assessment, time basis, result kind, true meaning and false meaning, together with source quotations. For example, certificate applicability concerns one selected submission method, whereas a report-level resubmission condition may quantify over several events. Two rules are eligible for a direct Boolean comparison only after these commitments have been aligned explicitly. Similar variable names or a common result type are insufficient.

This distinction also changes source-fidelity checking. The earlier STR investigation retained seventeen readings and forty-three claims, including context. Checking every claim against every partial reading creates 731 pairs, many of which compare different decisions. Two separately requested readers now assign each claim and reading to the proposed controls. A scope definition may qualify several controls. A reading that combines questions remains unresolved; it is not silently narrowed. If the readers disagree, the affected comparison remains required. A control with relevant claims but no candidate is reported as missing. Claims originally labelled context are reconsidered because their relevance can change with the question.

The assignments are themselves interpretations. Agreement does not prove that a qualification is irrelevant, especially when both readers use the same model family. Every excluded pair therefore retains both assignment reasons, and the full source remains available to each fidelity request. A check can challenge the assignment as well as the formula. This reduces avoidable work without equating a shorter comparison list with completeness.

Each required pair is checked in a bounded batch. Completed responses, failed attempts and pending identities are stored separately. If a batch cannot be validated, the controller splits it and executes smaller requests within its remaining allowance. It does not discard a difficult pair to obtain a complete status. An interruption consumes its reservation; resumption reuses the exact retained response only when its source, request and method bindings still match. Thus the final result can distinguish a completed check that found an omission from a check that never ran. Neither should be reported as source support.

6.30.2 Repairing notation while preserving the proposed meaning

Some retained model responses put an English explanation in a field intended for a formal expression. Rejecting that response loses a potentially useful interpretation, but asking a model simply to “make it compile” can change its meaning. The repair must have a narrower purpose. It retains the entire original reading, including factual definitions, classifications, assumptions, citations and questions, and proposes only replacement scope and result expressions.

The expression compiler checks types and grammar. It also rejects an undeclared fact rather than supplying a convenient default. The repaired expression is then rendered

back into controlled English. A separate context compares that rendering with the original proposal and the full source in both directions: what condition was lost, and what new condition was added? The correspondence judgment and any uncertainty are retained beside the original. Even a successful correspondence check supports a conditional claim about this translation. It does not establish that the original English interpretation was legally correct.

This is especially useful for an ambiguous e-IP response. One proposed scope described a test of whether a submission was made through e-IP, while its formal body treated out-of-scope cases as automatically true. Moving the condition from the result to the scope would change a true result into an out-of-scope result for some inputs. That is a semantic decision, not cosmetic syntax repair. The record must expose the difference before any corrected expression is promoted.

6.30.3 New source evidence can settle a particular dispute

Circular 25EC71 announced the equity-option margin exemption from 4 January 2026 while saying that the corresponding Code amendment would be gazetted in due course. The retained alternatives disagreed about whether a transaction therefore needed an additional gazettal-status fact. The January 2026 Code provides more specific evidence: Schedule 10, Part III, paragraph 7(e) lists the non-centrally cleared single-stock, equity-basket and equity-index options, and footnote 9 states the extension from 4 January 2026 until further notice. The current provision supports the stated start date. It does not make a separate transaction-level gazettal Boolean an expressed condition of that provision. Historical publication timing and the possibility of a later notice remain distinct questions (Securities and Futures Commission, 2025; SFC, 2026a).

The e-IP example illustrates a different repair. Circulars 24EC33 and 24EC50 both enumerate the investment-product categories administered by IPD. The later circular expressly extends the parallel period and gives the mandatory channel date. A rule can therefore expose the individual listed categories instead of asking for an unexplained "IPD product" Boolean. A new product may still need classification, but the missing judgment is now visible. Optional payment of annual fees must remain distinguishable from an application or submission. Likewise, a sourced extension of the parallel period justifies a priority relationship for that particular date question; it does not make every sentence of the later circular superior to every sentence of the earlier one (Securities and Futures Commission, 2024b,a).

An authority relationship consequently records the provisions it connects, the question it affects, its applicable dates and the exact supporting passage. A missing interval or a cycle leaves priority unresolved. The same discipline applies to bilingual material. The retained English and traditional-Chinese versions of 26EC2 can be paired at the certificate footnote, resubmission footnote and operational-launch provision. Those pairings permit comparison; matching reference numbers and dates do not prove equal meaning. Both texts and the proposed correspondence remain available for later challenge.

The continuing resubmission duty adds a final temporal distinction. The selected

source does not supply a completion deadline. The event engine can record an opening triggered by an original submission through another channel and a later performance through STREAMS 2. Without a deadline, absence of performance leaves the duty open; the engine must not invent a timed breach. If the dependent authority changes, its current assurance is invalidated while the original opening and historical replay remain bound to their original rule version. Changing the law is not permission to rewrite the event history.

6.30.4 What the executed demonstration establishes

The development demonstration compiles the three STR decisions as separate Java outputs and evaluates twelve explicitly described scenarios, including missing methods, missing certificates and an incorrect original channel. A second rule exposes the eight e-IP categories and its scope boundaries. Python, ordinary generated Java and cvc5 compare the resulting forty-eight decisions; the STR cases also run through the Catala backend. The duty example executes both an open obligation and recorded performance, and checks that changed authority requires reinvestigation. These are conditional execution checks on authored cases. They are not forty-eight independent legal interpretations.

The PDF review has a similarly precise scope. It locates footnote attachments and the four return or yield columns in the retained advertising examples. Exact relocation of a visually inspected running page label can be resolved without altering legal wording. Changed quantities, units, negations, exceptions or column labels remain discrepancies. An unresolved OCR difference is not converted into agreement merely because another parser repeats it. The development annotations preserve the page, coordinates, source hash and image hash so that a reviewer can inspect the particular proposed resolution.

These mechanisms make the investigation more specific and more useful. They also expose what the present evidence does not measure: independent legal accuracy across unseen source families, error correlation between different model families, and actual reviewer time saved. Those questions require suitable reference judgments and observations. They must not be answered by counting compiled rules, matching outputs, resolved page labels or repeated model votes.

The gifts investigation illustrates that distinction in the executed record. It retained all fifty-three claims and eight readings, giving 424 possible claim–reading pairs. The two routing judgments proposed excluding 203 pairs because they concerned different questions. All 221 remaining pairs received an outcome. The model assessed 168 pairs, judging ninety entailed and seventy-eight not established, and raised ninety-two additional concerns. The other fifty-three pairs concerned an unencoded reading and received explicit encoding-failure outcomes. Several questions still required external definitions or authority. These counts establish completed work and the character of the findings; they do not measure the model's legal accuracy. In particular, an unestablished pair may identify a missing condition, an unencoded reading, or insufficient evidence rather than a legally false proposition.

The STR run exposed a different problem: a reader used local proposal names instead of inventory identities and subsequently omitted two claims. Both raw responses were retained. The repair requests smaller portions of the inventory while keeping the complete circular and control definitions in every request. Only an exactly accounted union can support routing. A resource ceiling may still leave checks pending, but it cannot make those checks disappear from the reported universe. In the completed continuation, 662 STR pairs remained required and sixty-nine were excluded by the proposed routing. Three hundred pairs concerned readings that could not be encoded. The model assessed the other 362, judging ninety-five entailed, two contradicted and 265 not established, and raised 223 additional concern occurrences. No required pair remained pending. The two contradiction findings concerned the same date-only formula: it treated filings before the stated 9:00am launch as post-implementation filings. They are two findings about a retained timing interpretation, not two independently adjudicated legal errors. The midnight/operational-launch question remains visible for targeted source assessment.

The repaired round passed 741 engineering tests and used ninety additional model reservations, including failed attempts. These results establish an executed investigation and conditional code conformance. The distinction between a complete investigation record and a settled interpretation remains essential: an explicit failure to encode a reading is an accounted outcome, not evidence that the reading has been checked successfully against the source.

6.31 A decision can agree before its interpretations do

Consider a suspicious transaction report sent at 02:00 on 2 February 2026. One retained reading begins the new channel requirement at the start of that calendar day; another uses the stated operational launch at 09:00. An engineer cannot erase this difference by calling both dates "2 February". At 10:00, however, the two start conditions agree. That agreement can be useful even while the interpretation of the earlier interval remains unresolved. Securities and Futures Commission (2026b)

The relevant question must first be fixed. Whether a submission method requires an electronic certificate is different from whether a particular certificate satisfies the requirement. Whether an original filing triggers resubmission is different from whether a report's entire history contains a triggering filing. The system records the actor, assessment unit, temporal basis and meaning of each Boolean result with its source passages. It compares alternatives only when those descriptions match. Shared names for facts are insufficient: the definitions, types and units must also match. An independently checked mapping would be needed to compare different vocabularies.

Let H be the nonempty set of retained readings for that one question. Let x denote the supplied facts, with their evidence and dates. For each reading $h \in H$, the executable rule returns a Boolean, an out-of-scope result, or an unresolved result. Write $E_h(x)$ for this result. The decision procedure reports a known Boolean b only under the following

condition:

$$D_H(x) = b \quad \Longleftrightarrow \quad \begin{array}{l} H \neq \varnothing \text{ and} \\ \forall h \in H : \ h \text{ is executable and } E_h(x) = b, \quad b \in \{\text{true}, \text{false}\}. \end{array} \tag{6.9}$$

An unencoded reading therefore blocks a known result. An unknown input or a conflicting fact blocks the result unless the rule itself determines its answer without that input under the existing three-valued semantics. If every reading places the transaction outside scope, the response says that explicitly; it does not turn absence of applicability into a false compliance finding.

The logical guarantee is narrow but exact. Suppose the intended interpretation h^* belongs to H, the shared facts have their stated meanings, and each evaluator implements its reading correctly. If the procedure returns b, then every member of H returned b; substituting h^* yields $E_{h^*}(x) = b$. The conclusion follows without a probability model or a majority vote. Its first premise remains a substantive interpretive question. If all readers overlook the same exception, H can contain only mistaken readings. Consequently, the returned result identifies its source edition and retained reading set and states that its agreement is conditional on them.

The 02:00 example illustrates why disagreement is information. With all other conditions fixed, the two start readings can yield different scope results; the response carries both outcomes. At 10:00, their timing difference no longer affects this decision. This does not settle certificate performance, XML-schema conformity or any other question. Those questions need their own readings and facts. A source amendment invalidates current assurance; the retained old edition can still be used for a separately identified historical replay.

6.31.1 Preserving the questions a decomposition cannot answer

Some earlier STR proposals deliberately had no formula. One refused to equate having an electronic certificate with using it in whatever technical manner the reporting system requires. Another separated the instruction to arrange an XML technical test from an unsupported requirement to have passed that test before filing. These are useful distinctions, not malformed strings.

A new proposal may separate a trigger from performance, or an event from its report history. The parent remains in the record. Every parent statement, assumption and question receives an explicit treatment identifying which child preserves it or why it remains unresolved. A fresh context examines the actual compiled children's controlled-language meanings against the parent and source. The child's successful compilation supplies no reason to remove a still unencoded parent from the relevant interpretation set.

Report history exposes a consequential asymmetry. One evidenced triggering event establishes that a trigger exists, even when the rest of the history is incomplete. Finding no trigger in a partial history does not establish absence. The implementation therefore needs an explicit history-completeness fact. Similarly, a timestamp adapter requires an

explicit timezone. Under the stated Hong Kong local-time assumption it preserves the blackout interval, the calendar-day start and the 09:00 operational start as separate facts. Neither adapter invents a deadline for resubmission.

The public JFIU materials and the cited gifts FAQ are acquired as distinct authorities. Retrieval helps only to the extent that an inspected passage answers the question. The FAQ's discussion of product-category promotions does not itself supply a dominant-character test for an indivisible package of discounts and other benefits. Technical XML specifications described as supplied separately cannot be reconstructed from a model's recollection. These missing premises remain named source dependencies. Joint Financial Intelligence Unit (2026); SFC (2010)

6.31.2 Following a fresh proposal into Java

The next development example uses the official JSON for SFC circular 23EC52, frozen before its accepted fresh model proposals. Its paragraph 28 and footnote 7 discuss the virtual-asset de-minimis trigger for fund managers and distinguish tokenised securities from the relevant virtual assets. Securities and Futures Commission (2023)

The input interface supplies a fund-manager classification, the stated virtual-asset investment objective, and intended virtual-asset investment as basis points of gross asset value. The objective and asset classifications are supplied facts requiring judgment; the experiment does not claim to derive them automatically from contracts. The numerical examples use exact whole basis points. An operational adapter must preserve finer precision or return an unknown input; rounding an exposure up to a threshold would change the legal question.

Before requesting a formula, the author freezes cases immediately below, at and above the stated threshold, an objective-only case, tokenised-only and mixed-exposure cases, and missing-input cases. These expected answers are kept out of generation and review prompts. Two methods receive the full circular: one extracts duties and conditions; the other rewrites the selected question in controlled language. Each accounts for every supplied source unit and records other provisions as separate or unresolved questions. Both methods currently use the same model family, which leaves common errors possible. The supplied packet contains the complete circular body and its footnotes; its separate appendix and referenced legislation remain external dependencies. Accounting for the supplied text therefore does not establish full coverage of all incorporated authority.

Each returned executable proposal is compiled unchanged through RuleIR into Java. A generated Java class embeds the entire retained set and calls every policy before applying Equation (6.9). Its response contains the individual outcomes and hashes of the question, source packet, facts and reading set. The API returns a conditional draft decision; it does not authorize a bank transaction. Python, cvc5 and the supported Catala backend provide further checks of the encoded rules. Their agreement tests execution of those rules, while the withheld source cases test the author's specific source-to-rule expectations. An independently adjudicated accuracy study is still a different experi-

ment.

6.31.3 What the executed increment establishes

The executed round retained all eight unencoded parents and produced 3 executable child proposals for inspection. Parent/child reviews classified 1 proposal as changing meaning, 4 as not established, and 3 as preserving meaning with stated residuals. None of these outcomes removes an unencoded parent. The fresh circular produced 2 readings, of which 2 compiled; 22 candidate–case comparisons over 11 distinct cases matched the frozen development expectations. The useful-decision criterion was met. The exclusion challenge restored 232 pairs for further investigation. Located typography resolved 2 additional PDF differences, leaving 337 with unresolved materiality. Independent executable checks covered 22 cases. These counts describe this retained development exercise; they are not an estimated legal accuracy rate.

The accepted execution followed a failed preliminary attempt. Seventy-seven model reservations failed during CLI initialization and two more belonged to an interrupted attempt; none supplied a usable interpretive judgment. An incomplete manually reconstructed circular was also rejected and replaced with the official bytes before the accepted proposals. Those failures remain in the execution record and count against the original allowance. A circuit breaker now stops dependent dispatch after a transport failure. Repairing the execution machinery does not convert the rejected attempt into evidence about the source.

The exclusion challenge revisits every one of the 272 claim–reading pairs previously omitted under two proposed control assignments. A new finding of relevance restores the pair for source-fidelity work; an uncertain response also restores it. Agreement that a pair is irrelevant remains a defeasible judgment. The procedure prevents a routing decision from quietly becoming a proof of interpretive completeness.

The PDF work preserves all 339 previously unresolved differences and their source and raster identities. A narrowly defined resolution joins parser tokens at a preserved hyphen only when the resulting printed word is uniquely located in the page's retained glyph layout. It cannot remove a negation, change an amount or unit, reassign a table column, or turn two ordinary words into one. Cases outside that rule retain their uncertainty and locations.

These results provide an executable route from multiple proposed readings to conditional decisions and explicit refusals. They leave two distinct kinds of work. Specific source dependencies and failed correspondences require further investigation. Estimating legal error rates, common failures across different model families, or savings in professional review requires independently supported reference judgments and observations of that review. Neither more passing software tests nor repeated votes supplies those observations.

6.32 The errors that agreement can leave hidden

The conditional decision in Section 6.31 is useful only if its retained readings still include the qualifications that matter to the bank's question. A workflow can lose one before any formula is compiled. An early reader might label a paragraph as background; later stages might then compare several interpretations without ever testing that paragraph against them. Their agreement would leave the exclusion unexamined.

The preceding investigation challenged 272 previously excluded claim–reading pairs and restored 232 for further examination. A restored pair is a reason to revisit the exclusion, not proof that the earlier reader was wrong. Of those pairs, 231 had an executable reading and one still lacked a supported encoding. Forty-eight of the executable pairs involved claims historically labelled as context. An initial implementation would have filtered those claims out again. Its review therefore caught a consequential procedural error: the program's actual comparison universe was smaller than the one its plan promised.

The repaired procedure schedules every executable restored pair. It supplies the complete retained source packet and the actual executable meaning while withholding earlier routing and fidelity labels. Scheduling the comparison does not assert that the claim governs the selected question. The reader must still explain that relationship. A source quote, an executable quotation and an exact pair identity make the judgment inspectable; they do not make it authoritative. The one unencoded pair remains a separate obligation. Removing it would improve the completion percentage by changing the question.

6.32.1 A child's result may answer a smaller question

The STR example makes another failure visible. Three proposed child rules compile: a rule for urgent contact, a trigger for the XML-transition duties, and a trigger for resubmission after an off-channel filing. The latter two answer whether a duty is triggered. They contain no executable condition for whether the required performance occurred. A firm intending to submit XML can satisfy the trigger whether or not it has followed the schema or arranged the technical liaison. Likewise, two histories with the same original off-channel submission have the same trigger even when only one contains a linked resubmission. The trigger can be correctly implemented while the broader compliance question remains unanswered.

The urgent-contact child's fact definition records contact for the assessed STR somewhere during the blackout. Suppose the instant being assessed is noon and contact occurred at 13:00. A fact meaning "contact occurred during the blackout" can be true, while "contact had occurred by noon" is false. A before-noon contact and an after-noon contact can therefore collapse to the same supplied Boolean. If the selected question concerns performance by the assessed instant, the data interface needs that distinction. This example identifies missing information; it does not supply a legal deadline or decide what urgency means.

The generated children's version intervals expose a separate limitation. Each begins on the development date in September 2026. When an assessment is instead made for the January or February transition, the current evaluator returns a version-time error. Nine targeted cases reproduce these refusals and the children's positive and changed-condition results in Java, Python, cvc5 and Catala. The refusal is correct for the stored interval. That interval does not establish historical coverage of the circular. The next implementation must separate source effectiveness, the time of the assessed event and the time when the system learned the evidence, without inventing a source commencement date.

These observations explain why all eight unencoded parents remain retained. The three executable children and fifteen unencoded children are partial proposals. Backend conformance can support their stated computations. It cannot establish that a child preserves every condition of its parent, or that contact, trigger satisfaction and full performance mean the same thing.

6.32.2 Preserving the threshold through the input interface

The fund example supplies a repair that can be checked exactly. Its earlier interface used whole basis points. An exposure of 9.999% equals 999.9 basis points. Rounding it to the nearest whole basis point gives 1000, which changes the answer to an inclusive 10% threshold test. A Java program can execute that rounded comparison perfectly and still answer the wrong numerical question.

Let p be the supplied decimal percentage, expressed in percentage units: $p = 10$ means 10%, not a fraction of 0.10. Parse the decimal string exactly and write $100p = n/d$ in basis points, where n is an integer and d is a positive integer. The positivity condition is supplied by the rational adapter. Multiplying an inequality by a positive denominator preserves its direction, so

$$100p = \frac{n}{d}, \quad d > 0 \qquad \Longrightarrow \qquad \left[\left(\frac{n}{d} \geq 1000\right) \Longleftrightarrow (n \geq 1000d)\right]. \tag{6.10}$$

This transformation replaces division with exact integer multiplication and comparison. It introduces no rounding rule. For 9.999%, the adapter produces $n = 9999$ and $d = 10$; the comparison is $9999 \geq 10000$, which is false. At 10%, it produces $n = 1000$ and $d = 1$, and equality satisfies the inclusive threshold. At 10.000000000001%, the exact numerator exceeds 1000 times the denominator. The development checks also retain a value equally close below the threshold.

The generated policy accepts the two integer fields, and the current Python adapter supplies them from a strict decimal string. Arbitrary externally supplied denominators fall outside this checked interface. An operational Java boundary must enforce the same positive-denominator and exact-decimal conditions before accepting bank data. Missing, conflicting or unsupported inputs must retain their explicit status instead of becoming a rounded number.

Seven numerical cases exercise this representation in the four backends. The origi-

nal whole-basis-point policy remains available as the comparator. Separate mutations change OR to AND, change an inclusive comparison to a strict one, and shift the threshold by one basis point. An objective-only case detects the first mutation; the exact boundary detects the other two. All eleven original source examples are preserved, and a conflicting-objective case checks that conflicting evidence does not become a known Boolean result.

Asset classification remains a different question. Feeding tokenised securities into the virtual-asset exposure field can change the decision even when every backend agrees. The input-mutation witness demonstrates that consequence. It does not automate the source-dependent classification of a real holding or establish which edition governs a current transaction.

6.32.3 What the completed execution can support

Round 15 completed 140 of the 231 executable restored pair checks, leaving 91 pending and the separate unencoded pair retained. Its accepted model judgments comprise 0 labelled entailed, 0 contradicted and 140 not established. These are the reader's scoped judgments, not a measured legal accuracy rate. The regression suite passed 789 tests. The three encoded children passed their generic execution probes, and the targeted historical and condition cases exposed the limits described above. The numerical mutation witnesses were detected. None of these results authorizes legal release.

The label "not established" requires careful reading. Some accepted responses identify a separate advertising or certificate control whose implementation is absent from a deliberately narrower trigger. That is not by itself an error in the trigger. Others identify a missing qualification or an unresolved authority. The current three-label contract does not distinguish these outcomes well enough for automatic defect counting. The next revision must record source support, relevance to the precise question and executable correspondence separately, retaining the original rationale and quotations. Treating all these labels as wrong interpretations would replace one misleading score with another.

Execution itself supplied a further counterexample to superficial testing. The one-attempt provider wrapper existed, but its initializer had been placed outside the reader class. An early fault test used an HTTP-error string that did not exercise the inherited retry path, so it passed without detecting the wiring error. Code inspection found the defect. The run was interrupted, the initializer was moved, and fault tests were extended to overload, disconnection and timeout conditions. Accepted responses were retained as earlier evidence; the interrupted call and malformed responses still consumed their reservations. A repeated response from the same request is never counted as a fresh independent judgment.

The PDF work keeps all 339 original difference identities and gives each an inspectable context crop linked to the full page. A unique page-number phrase may be classified as layout only when its exact words, footer position and retained page image agree. A number in a paragraph, a changed unit, a negation or an uncertain column relationship cannot be removed by that rule. The resulting record retains 330 unresolved materiality

questions, 2 earlier typography resolutions and 7 footer resolutions. These are parser-discrepancy dispositions, not findings that the remaining text has been interpreted correctly.

The source-acquisition step retrieves the official appendix for the tokenised securities circular and the related fund-manager terms. Both specified PDFs were acquired. The acquisition closes a missing-file task only. Certificate mechanics, the separately supplied XML schema, transition and resubmission guidance, mixed-gift classification, wider product provisions and incorporated authority remain named questions until passages and scoped tests address them. A document hash establishes which bytes were used; it cannot supply a missing legal premise.

The acquired 135-page appendix pack for circular 23EC44 contains clean October 2023 terms in Appendix 7 (PDF pages 74–94) and a separate version showing amendments in Appendix 7a (pages 95–135). The latter retains deleted and inserted definitions on the same printed page. Plain-text extraction captures both. This source structure needs an explicit edition selection before the text is used for interpretation. Concatenating the whole download can make a deleted definition appear to be a second operative rule. The retained source-review record identifies the inspected pages and leaves current effectiveness and supersession unresolved.

The system now retains reproducible counterexamples, explicit refusals and source-linked concerns. Its legal error rate needs a separate study with independently justified reference answers, a split by circular or provision, another model family, and explicit accounting for abstentions and missing runs. Existing comparison software supports that design. Same-family judgments and author-made examples do not supply independent answers. Targeted judgments on consequential unresolved questions may help create references without a comprehensive external review of every rule.

6.33 Repairing the distinctions that a comparison can lose

The unresolved comparisons in the preceding section do not all describe the same problem. A circular may contain a supported duty about advertisements while the selected program decides whether a gift is prohibited. Reporting that the gift formula does not implement the advertising duty is accurate as a description of the formula, but it has not identified a defect in that formula. The bank still needs to account for the advertising duty elsewhere. Conversely, an exception to the gift prohibition belongs in the gift decision even if an earlier reader called it background. A useful comparison must distinguish these two situations before proposing a repair.

Round 16 implements four separate judgments. The first asks whether the extracted proposition is supported by the supplied source. The second asks whether that proposition affects the exact question assigned to the program. The third compares the proposition with the actual executable expression. The fourth records the availability of any authority needed to decide the matter. Each judgment has its own reason. Source

support and question relevance carry source quotations; a proposed preservation or contradiction of executable meaning also carries an exact quotation from the rendered expression. The question includes the actor, unit of assessment, temporal basis and meanings of true and false. Its content hash prevents an assessment of one question from being reused after the question changes.

Consider an illustrative circular containing both an advertising disclosure duty and a gift restriction. The disclosure duty may be recorded as supported by the source, not applicable to the gift question, and consequently not applicable to the gift formula. That combination no longer needs a fictitious formalization failure to satisfy the response schema. The claim–reading pair nevertheless remains in the inventory, with its reasoned relation available for challenge. If another reader finds that the disclosure paragraph qualifies the gift restriction, the system retains both proposals and requests another investigation. Agreement on a relevance label does not establish that no distant qualification has been missed.

An omitted exception has a different diagnosis: the source proposition is supported, its relation to the gift question is relevant, and the expression omits it. A falsely extracted proposition has a different starting point again. If a reader says that no electronic certificate is required while the retained footnote states a certificate requirement, the extracted proposition must be challenged before its supposed requirement is translated into code. A missing external definition can leave authority availability unresolved even when the program faithfully preserves the words already supplied. The new contract keeps these observations separate and rejects a claimed preservation when the declared question and the expression have opposite result meanings.

Migration cannot recover these distinctions retrospectively. The 140 accepted round-15 comparisons used a combined label, and their reasons do not constitute four newly validated judgments. The migration therefore stores each complete old assessment, its provenance and its hash, while marking the new dimensions unassessed. All 232 restored pairs survive, including the 91 still pending and the one with no supported encoding. No new model call or new source judgment was made in this round. The new interface is an explicit versioned path; older experiments retain the old contract so that their recorded outcomes remain reproducible.

The reconciliation procedure does not choose a winner by majority. It retains the proposals and identifies the dimensions on which they disagree. Missing authority, an unassessed relation, a contradicted source proposition or an omission can request further processing even when two readers share the same diagnosis. The caller supplies an attempt number within a fixed maximum. Before that maximum, an unresolved case requests repair; at the maximum, it produces an explicit uncertainty report. This procedure supplies a bounded control for investigation. It neither supplies new evidence by itself nor consumes a model call without an authorized execution route.

The new investigation route executes that control with retained or local providers. Its first two perspectives receive the same source and question without seeing one another's response. Later rounds receive the earlier disputes or response-validation diagnostics. Each invocation has a persistent reservation, and the raw response is saved before validation. In the retained worked fixture, an applicability disagreement causes six

local invocations across three rounds; all six proposals remain in the final uncertainty report. A malformed-response fixture exercises an actual additional round. Resuming the same completed requests reuses evidence, whereas a changed source, question or setting invalidates the run. The first transport failure stops further dispatch, including after resume. Agreement requires a validated response from each declared perspective: repeated answers from one cannot replace a missing other perspective. These are executable local tests of the investigation procedure. The route explicitly refuses live providers pending their integration with the shared allowance; no new live judgment is implied by the fixture's six invocations.

6.33.1 What had happened, and what the bank knew

The contact example requires two clocks. Suppose the bank asks at noon whether contact has occurred for STR S. A contact at 11:00 can answer that factual question positively; a contact at 13:00 cannot. There is also a knowledge cutoff: an 11:00 contact entered into the system at 13:00 was not available to a replay restricted to information recorded by noon. A contact about STR T provides no witness for STR S. Compressing any of these distinctions into one supplied Boolean can make the rule appear correct while its input answers a different question.

The implemented event projection receives an event identity, an STR identity, an event kind, the occurrence time, the recording time, evidence identities and whether the event is confirmed or disputed. The caller states the start of the interval being examined, the assessment instant and the knowledge cutoff. A confirmed event counts only if it belongs to the specified report, falls in the stated occurrence interval and was recorded by the knowledge cutoff. The by-now interval includes the assessment instant. This is the chosen factual question; it introduces no deadline into the circular.

A positive answer can have a particular event as its witness. A negative answer requires more: a separately supplied assertion that the history is complete for the report, event kind and whole interval being assessed. That assertion has its own recording time and evidence. Without an eligible completeness assertion, absence produces an unknown result. An assertion recorded before the end of the interval cannot certify its future portion. A relevant disputed event produces conflict, and two incompatible records using the same event identity are rejected. These checks make the reliance on event recording visible; they cannot establish that an operational source captured every real contact.

Resubmission adds another relation. The resubmission event names the original submission, and that original must belong to the same STR. A missing original leaves the linkage unresolved. The adapter rejects an attributed resubmission that predates its original. A complete history with no linked resubmission can support a negative answer to this factual existence question; it does not by itself decide whether the legal performance requirements were satisfied. Schema validation and technical liaison likewise remain different from the intention that triggers their associated duties.

The source's effective interval and the generated bundle's interval are kept separately

from these two clocks. Round 16 re-executes the nine retained child cases without changing their bundles. The three historical assessments still return the version-time refusal because their requested dates precede the stored bundle intervals. The added time-context check records that refusal separately from unresolved source effectiveness. An explicitly supplied effectiveness assertion needs source anchors and an interval before it can produce even a conditional applicability result. The code has not supplied the missing legal date. A pair of trigger and performance records also retains unencoded performance instead of reporting overall compliance from a true trigger.

6.33.2 An exact number must survive the Java entry point

Equation 6.10 depends on a positive denominator. Testing that identity inside the rule evaluator does not protect an external interface that accepts arbitrary numerator and denominator fields. Round 16 adds a packaged Java entry point which accepts the decimal percentage itself. Its request declares the signed exact-percentage profile, subject, assessment and knowledge times, development mode and three facts: fund-manager status, the stated virtual-asset objective, and intended percentage exposure. The generated entry point supplies the internal rational fields. It rejects a caller who also tries to supply those fields, a raw snapshot or other undeclared fields.

The percentage is a decimal string in percentage units. Thus `"10"` means 10%, while `"0.1"` means one tenth of one percent. Suppose its signed unscaled integer is u and it has s decimal places. Java computes the integer numerator $100u$ and the positive denominator 10^s, then divides both by their greatest common divisor. The policy receives that reduced exact ratio in basis points. At 9.999%, the initial pair is 999900 and 1000, which reduces to 9999 and 10. The comparison remains false because 9999 is less than 10000. No binary floating-point representation or rounding rule intervenes. Python provides a separate reference conversion using rational arithmetic.

The accepted syntax has an optional minus sign, an ordinary integer part and an optional nonempty decimal part, within a 64-character limit. Exponents, whitespace, redundant leading zeroes, non-ASCII digits, nonfinite values and JSON numeric inputs are rejected. Very fine decimal scales remain exact when they fit the limit. A missing percentage is converted to missing rational facts; a conflicting percentage supplies conflicting facts. Their eventual effect follows the declared RuleIR uncertainty semantics. For example, the retained case with no virtual-asset objective and a missing percentage returns unknown, while conflicting percentage evidence blocks a known result.

Signed input is an explicit arithmetic profile inherited from the earlier rational experiment. Acceptance of a negative number under this profile is not a decision that negative intended exposure is permissible for a particular fund question. A deployment must state its financial domain separately. The new package is also an explicit interface choice: existing raw-snapshot APIs remain available for their earlier purposes and do not acquire this protection merely because the new JAR exists. A host must call the generated percentage entry point. Its build record binds that class, the actual JAR, the policy and the Java sources.

Thirty retained host cases exercise this JAR and the Python reference. They include all seven earlier threshold examples, additional fine-scale and signed values, missing and conflicting facts, malformed strings and direct attempts to inject denominators. Seventeen requests are rejected as specified; the other thirteen agree on the transformed snapshot and the full decision content apart from the declared engine identity and its associated hash. Separate focused checks send a floating-point JSON number directly to Java, avoiding the misleading possibility that Python alone rejects it before the JAR sees the request. These results establish the tested representation boundary; the qualifying-asset classifications supplied to the rule still need their own evidence.

6.33.3 One downloaded file can contain incompatible editions

The acquired 23EC44 appendix pack supplies a concrete source-selection test. Its clean October 2023 terms occupy Appendix 7, PDF pages 74–94. Appendix 7a, pages 95–135, shows amendments and can yield old and replacement wording in the same extracted stream. The new selection manifest binds the original PDF, each extracted page, both edition ranges and the exact selected provisions. An active source packet requires a clean edition; attempting to select marked changes as the active edition is rejected. Mixing provisions from the two editions is rejected even though both came from the same genuine download.

Selection within the clean edition must preserve qualifications as well. The executed example takes the definitions on page 76, the footnote about the structure of the collective investment scheme, the continuation on page 77 about persistent price appreciation and notification, and the application paragraphs on that page. The manifest explicitly links these provisions. Selecting the definitions while omitting the declared footnote or continuation fails with a missing-dependency result. Each excerpt has a page and exact character offsets; changed source bytes, changed extracted text or a changed manifest invalidate the earlier selection.

These structural checks operate on an inspected manifest. They do not discover every legally relevant qualification automatically. The packet therefore keeps unresolved authority questions as unresolved dependencies. The file also carries a notice that the circular was superseded and updated in December 2023; the printed October date cannot establish current applicability. The program records the clean historical edition and leaves effectiveness unresolved. This separates successful acquisition and selection from the further task of determining which provisions govern a particular assessment date.

All restored pairs, retained parents, source questions and unresolved PDF-materiality identities remain available for the next investigation. Estimating legal error or savings in review cost still requires source-grounded answers and independent reference cases.

6.34 A proof-carrying investigation joins the methods without joining their claims

The preceding sections establish why a single program, a majority vote or a successful search cannot settle the meaning of a circular. They leave a practical question for the implementation team: how should source recovery, alternative readings, legal arguments, formal checks and Java generation be joined so that each result remains attached to the question it actually answers? The answer in the current prototype is a typed assurance dossier. The dossier is a record of evidence and limits. It is not a new legal language and it is not a certificate that the regulator would accept a proposed reading.

6.34.1 The objects that travel through the investigation

The dossier starts with a *source packet*. The packet identifies the official document bytes, edition, selected passages, cited dependencies and the date and scope of the question. A digest of the packet is part of every later record. A candidate that quotes the same sentence from a different edition is consequently a different candidate, even if the visible sentence has not changed. This is the same discipline used by the authority and temporal records described earlier in this chapter.

The next object is a set of *interpretation hypotheses*. A hypothesis has an identifier, an interpretation family, the source passages on which it depends, controlled English, assumptions, supporting and opposing reasons, and either a RuleIR bundle or an explicit statement that it cannot yet be encoded. The family records a choice that changes legal consequences: for example, a later refund may be treated as a fee discount, or the exception may be attached only to the fee component of a combined offer. Repeating the same prompt with different wording does not create a new family.

An *argument record* links those hypotheses to premises, exceptions, counterarguments and source evidence. ASPIC+ distinguishes attacks on a premise, an inference and a conclusion; Carneades distinguishes ordinary premises, assumptions and exceptions (Prakken, 2010, secs. 2–3) (Gordon et al., 2007, secs. 3–6). LegalMath borrows these distinctions in its bounded graph. The graph can show that a reading is supported, opposed or blocked under the recorded premises. It does not turn an unresolved priority between two legal reasons into a Boolean answer.

A *method result* says what one assurance route did. It contains a method family, target question, input hash, implementation hash, result hash, shared dependencies, status and limitations. The result also names retained files: the exact request, response, checker output, compiled program and replay cases where applicable. The verifier reads those files and recomputes their hashes. A hash of a sentence saying "the check passed" is insufficient evidence. The status vocabulary includes `AGREES`, `DISAGREES`, `UNRESOLVED`, `UNAVAILABLE`, `NOT_COMPARABLE` and `STALE`. The validator requires at least two method families for a required comparison. It also records shared dependencies: generated Java and the optional Catala backend share RuleIR and fact preparation, so their agreement is a translation check, not two independent interpretations.

Finally, a *proof obligation* states a formal proposition, assumptions, checker and domain. It records one of `PROVED`, `CHECKED`, `DISPROVED`, `UNKNOWN`, `UNSUPPORTED` or `NOT_RUN`. A proved obligation must carry a certificate accepted by a registered proof checker; the historical v3 replay integration accepted on 26 September had no registered proof-certificate adapter. The continuing-investigation successor adds a separate, narrowly scoped Boolean Lean certificate, explained in Section 6.35; the historical replay record is preserved. An unknown or unsupported obligation must carry a limitation. For example, a finite exhaustive replay can establish that a generated Java class agrees with a Python evaluator on the listed snapshots. It cannot establish that the English phrase "discount of fees or charges" includes every future rebate.

The objects are deliberately separate because they answer different questions. The following table is the implementation boundary.

Object	Question answered	What a passing status does not establish
Source packet	Which bytes, edition and passages were read?	That the extracted text contains every visual qualification or incorporated authority.
Hypothesis	What source-linked readings are being investigated?	That a generated reading is legally tenable or complete.
Argument graph	Which premises and objections support or defeat a reading?	That a chosen priority is the regulator's priority.
Proof obligation	Does a stated formal proposition follow under its assumptions?	That the assumptions describe the source or the bank's facts correctly.
Method result	What did this checker or generator return for this target?	Independence, probability of correctness or a universal answer.
Java/Catala artifact	What executable was built from which bundle?	Approval to deploy or equivalence outside the declared domain.

Table 6.3: The dossier keeps source, interpretation, formal and runtime claims in separate ledgers.

6.34.2 A dry run on two meanings of a promotional connection

Take the retained 23EC46 source packet and the paragraph 10 gift-promotion question. The packet retains the circular, including the references to the Code and FAQ and the surrounding paragraph 9 classification language. Retaining a reference is different from acquiring its text: the referenced FAQ answer is missing from this replay packet. The implementation imports all five candidate readings and the criticism proposal from round 3, A7 attempt 08. Their origin is recorded as archived proposals. No fresh model generation occurs in this run. Two of these readings use the same five fact definitions and differ as follows:

1. an operational connection between the gift and the investment product is necessary for the selected prohibition; and
2. either that operational connection or product-directed promotional presentation can satisfy the connection condition.

Both are investigation hypotheses. Both retain gift classification and the fee-discount exception. The source packet does not decide between them. The other three readings remain visible, including differences in how product-type linkage relates to promotion and where the promotion condition enters scope. Different fact definitions cannot be equated merely because names look alike.

The scheduler first visits all five readings, then compares pairs. For the two readings above, a separating input has an in-scope distributor offering a gift that is not a fee discount, with product-directed presentation but no operational connection. One encoding returns false and the other true. The solver searches the declared three-valued Boolean domain; a found witness is replayed through both original programs. The feasibility and legal meaning of those facts remain questions for the investigation. A separating assignment locates the dispute; it does not decide which reading the regulator would adopt.

The controller records the following sequence for the same target question:

1. two extraction methods inspect the retained circular and source anchors are checked against its text; absent authority text remains absent;
2. the breadth scheduler retains every imported reading and its assumptions, with archived origin explicitly distinguished from fresh generation;
3. an argument graph records the support for each family and the absence of a retained authority deciding the disputed connection;
4. RuleIR type checking checks each restricted encoding;
5. a finite solver query finds a permitted input on which the two encodings differ, and the witness is replayed in the actual Python and Java evaluators;
6. the Java artifact is bound to the exact RuleIR bundle and snapshot set; and
7. the installed pinned Catala backend is compiled and replayed over the same declared cases; missing tools or unsupported operations are recorded as such, and the disabled live provider supplies no fresh evidence.

The result is therefore a useful but blocked dossier. Its formal and execution checks can succeed for their declared cases, while the authority question remains open. The host receives an unresolved result and the smallest question that could change it. It does not receive an approval.

6.34.3 Why a formal proof is valuable and still insufficient

The proof layer is most useful when its proposition is explicit. A Lean proof, an SMT unsatisfiability result or a checked Java/JML property can establish a statement such as

$$\forall x \in D: \quad E_{\text{Python}}(x) = E_{\text{Java}}(x), \tag{6.11}$$

where D is the declared finite or symbolic input domain and both evaluators have the same fact vocabulary, time semantics and result categories. A proof of (6.11) is evidence about translation and execution. It is not a proof that the source intended the predicate E.

The proof assistant can also check a RuleIR property, such as preservation of the three-valued exception semantics under a lowering step. The statement must include the meaning of unknown, conflict, scope and dates. If a backend omits one of these cases, the obligation is `UNSUPPORTED`; the missing case cannot be hidden by a successful Boolean test. MathDevMCP is useful at this layer because it can retain named obligations and checked derivations. Its success remains conditional on the local RuleIR semantics and theorem target.

Stipula and its Java/JML/KeY route illustrate the same boundary. A contract language can describe parties, state transitions, permissions and scheduled events, and KeY can reason about properties of a generated Java model (Crafa et al., 2021, secs. 3–6)(Hähnle et al., 2025, secs. 3–5). This is a promising backend for an event-heavy duty, but the translation from an SFC paragraph into the contract state machine is still an interpretation step. A successful KeY proof cannot close that step automatically.

6.34.4 The future-circular path

The bank is concerned with future circulars, so the product must fail visibly when a new clause does not fit its current grammar. The dossier has an `EXTENSION_REQUIRED` question for this case. It names the new source passage, the absent interpretation family, the RuleIR construct that is needed and the evidence required from the legal owner. The controller does not select the closest historical family merely because its Java emitter compiles.

This is a principled extension path rather than an assertion of universal coverage. A future source is first added to the authority catalog and source inventory. Its unresolved choices are enumerated before candidate generation. The new family receives its own controlled-language and argument records. A formal encoding is accepted only after its types, time model, result categories and source quotations are reviewed. Existing programs can be reused when the new rule is proved equivalent under the declared mapping; otherwise a new artifact is generated. Old decisions remain tied to their old source edition.

ResearchAssistant can supply retrieval, parsing, local paper and source retention. DynareMCP offers a pattern for recording hypotheses, dependency graphs and

bounded search priorities. The present dossier replay uses LegalMath's source and search adapters; it does not invoke those adjacent repositories. Tree-of-Thoughts and Language Agent Tree Search motivate breadth and environment-feedback operations (Yao et al., 2023; Zhou et al., 2024). None of these components supplies an oracle for an unseen legal meaning. Codex can serve as a bounded candidate or review-question generator, provided its output is treated as untrusted typed evidence and is subjected to the same source, argument and proof gates. The current master records the provider-unavailable state because its live budget is exhausted.

6.34.5 The executable master and its stop conditions

The implementation contract is in `src/legalmath/interpretation/assurance/integration_contract.py`. The master controller is `scripts/run_proof_carrying_master.py`; its reviewed plan and allowlist are in `docs/plans/proof-carrying-assurance-integration.md` and `docs/implementation/proof-carrying-assurance/allowlist.json`. From the repository root, the bounded execution is:

```
.venv/bin/python scripts/run_proof_carrying_master.py audit
.venv/bin/python scripts/run_proof_carrying_master.py execute
```

The application also exposes `assurance-dossier` and `assurance-dossier-verify`. The first accepts a repository root, retained input manifest, output directory, JDK and optional pinned Catala configuration; the second reread a saved dossier and verifies its evidence. The input manifest names the packet, candidate set, criticism proposal and source files by path and SHA-256, and declares the assessment time, probe and pair limits, required backends and proposal origin. No executable expression from a model is accepted: the restricted expression parser and typechecker mediate compilation. A caller can supply another circular's retained investigation through this same interface. The interface does not generate fresh readings, acquire missing authority text or authenticate source acquisition merely by accepting their file hashes.

The six phases freeze inputs, validate the contract, exercise the method families, build this monograph and process guide, run a deliberate stale-target repair, and execute the full regression. Each phase writes its exact command, environment, input hashes, result hash and predecessor manifests. After a phase succeeds, the controller refreshes the next-phase plan from those actual hashes. If a source, implementation or plan file changes, that phase and every successor become stale before another attempt can run. A failed phase cannot be retried until an executed repair note and focused regression are recorded.

The final report is deliberately capable of saying "engineering pass with retained uncertainty." The following conditions remain blocking regardless of how many methods agree: an unresolved authority or source region, an unsupported future clause, a stale result, a method unavailable for a required target, an incomparable fact vocabulary, an unknown solver result, an unproved proof obligation, or a missing independent legal decision. This is the intended connection between the flow chart and the Java system: Java receives a typed decision and its evidence, while the bank's host chooses an ap-

proved action for `UNKNOWN`, `CONFLICT`, `OUT_OF_SCOPE` or `BLOCKED_UNRESOLVED`.

An execution audit rejected the first adapter because it constructed success labels from descriptions rather than attaching executed evidence. Its original attempts are preserved, and its interrupted regression is not an acceptance result. The current repaired execution writes to `artifacts/proof-carrying-assurance/verified-execution-v3/`. The evidence verifier checks every referenced file, candidate-to-bundle link, required obligation and shared dependency; unresolved obligations block a ready status even when the method records agree. The original controller's exceptions for changed supervisor files were removed: changed controller, plan, allowlist or material evidence requires new acceptance.

The original input-manifest file is itself mandatory evidence. The verifier reparses its bounded schema and compares it with the configuration embedded in the target before checking any replay. This closes a further audit finding: an internally consistent embedded configuration had survived removal of the file that originally declared it. Tests now reject a missing manifest, a differing embedded configuration and a probe budget outside the permitted range. The earlier successful regression remains historical evidence for the earlier code; it is superseded by a new acceptance run after this repair.

Verification also checks the report that a reader sees. Every disagreement produced by the recorded comparisons must appear with its material severity and unresolved disposition. A caller cannot delete it or mark it explained without a corresponding evidence procedure. The same check preserves the proposed readings' objections, the critical questions and their required evidence, and the exact proposition and domain attached to each obligation. Consequently, evidence for a finite replay cannot be relabelled as a result about every future circular while keeping its successful checker status.

Concurrent document changes also invalidated a later attempt even though its focused tests passed. The combined revision is therefore validated in an isolated repository copy. Its controller records the actual execution root, archives input bytes, and compares source hashes before returning the results to the working repository. Each campaign retains a fixed retry ceiling; the earlier exhausted campaign remains closed without current acceptance. This distinction prevents a successful command on one revision from being presented as evidence for another revision.

The resulting engineering evidence can be reproduced and reviewed. It does not establish legal accuracy, statistical independence, complete future-circular coverage or production readiness. Those claims require a separate evaluation with independently adjudicated cases and a bank-approved operating environment.

6.35 From an interpretation dossier to a continuing investigation

Consider a firm implementing the transition to STREAMS 2. The circular says that, during the blackout, firms should contact the JFIU directly about STRs requiring urgent submission. A program can correctly identify an urgent STR and still answer the

wrong question. An obligation to make contact is different from evidence that contact occurred. A telephone call about another STR, a message sent after the assessment time, and an incomplete communication log must produce different evidential conclusions. These distinctions motivate the successor implementation described here. It joins the previously separate investigations while preserving their different meanings.

The predecessor's engineering result remains useful: its retained readings, counterexamples and generated programs can be replayed. The new work adds a continuing investigation of source meaning, a narrow proof checker, and an event-history demonstration. The execution record distinguishes three kinds of progress. A capability may be implemented and tested; a particular source question may receive additional support; and an uncertainty may remain after the allowed investigation. Only the first of these follows from a passing software regression.

6.35.1 Ask the question before judging the answer

A source-fidelity assessment needs four separate judgments. First, does the inventoried claim have support in the retained source? Second, does that claim bear on the precise question the candidate program answers? Third, does the program preserve the relevant claim? Fourth, are the required authorities actually available in the retained source set? Combining these into a single "correct" label confuses errors with differences in scope.

For example, the requirement to resubmit an off-channel STR through STREAMS 2 can be supported by the circular while remaining outside a question about the means of making one original submission. The assessment must retain the resubmission duty and explain its relation to the selected question. It must not report the original-submission program as a complete implementation of all STR duties. Conversely, an inventory statement that misreads the source does not establish a defect in every program that omits that statement.

Each proposed question therefore specifies the actor, unit of assessment, temporal basis, and meaning of true and false. The question's identity travels with the assessment. Changing "is contact required?" to "has the required contact occurred?" invalidates the old assessment even if both answers are Boolean. The validator checks the question identity, exact source quotations, candidate identity and output convention. These are necessary consistency checks. They cannot establish the entailment of an English proposition merely because its quotation is authentic.

Two fresh contexts initially assess the same bounded batch without seeing one another's answer. One starts from the source proposition; the other concentrates on qualifications and exclusions. Disagreement produces a further request containing the competing explanations and the exact unresolved question. Invalid quotations or inconsistent labels instead produce a response-repair request. Both kinds of request consume the same finite allowance. After the configured final round, remaining disagreement is reported with its evidence. The inherited backlog permits three rounds; the unfamiliar-source pilot declares two rounds before collecting answers. The system does not select the last

response simply because it arrived later.

The product's acceptance rule is a checked proposition or an explicit qualification. Human answer keys, adjudication, ratings and approval are excluded as quality evidence. A source text remains a premise; its authenticity does not prove a proposed interpretation. Model agreement also remains a proposal. The useful output therefore identifies exactly which formal relation has been checked and which source meanings, classifications or authorities remain unresolved. Future publications must enter a frozen observation window; a repaired development example cannot be presented as a fresh prospective test.

The first live successor batch illustrates why mechanical checks matter. Both initial responses marked an authority as available although the packet did not bind the required dependency to source bytes. The validator rejected both responses. A subsequent response repaired the inconsistency. This is evidence that the check can catch that failure; it is not evidence that the repaired reading is legally correct. Agreement between the initial readers would have concealed the missing source.

6.35.2 A factual model is part of an interpretation

Two formulas cannot meaningfully be compared until their facts have compatible meanings. In the gift example, one reading may assess an offer as a whole while another assesses each benefit component. In the STR example, one Boolean fact may mean that any JFIU contact occurred, while another means that the named firm contacted the JFIU about the particular urgent STR before the assessment time. Renaming both facts `contact` would conceal the disagreement.

The successor retains an explicit assessment frame: actors, objects, unit of assessment and temporal basis. A proposed bridge between frames must identify both source-bound factual models, define common facts, and give an expression for every original fact. It also states the assumptions needed to connect the frames. If the actor or unit changes, an omitted connecting premise is a validation failure. If no bridge is available, the readings remain incomparable.

Once such a bridge is proposed, the implementation can enumerate an explicitly bounded common domain, derive each reading's original facts, and execute both original Java programs. A separating case then has a precise conditional meaning: under the proposed factual mapping and its assumptions, these programs give different answers. Agreement has the same conditional meaning. Neither result establishes that the mapping is faithful to the source. The live study therefore retains the proposed bridge, its assumptions and any failed comparison alongside the program results.

6.35.3 Authentic authority can still be inapplicable

The legal-argument literature separates the structure of an inference from the acceptability of its premises and the priority of competing reasons (Prakken, 2010, secs. 2–3)(Gordon et al., 2007, secs. 3–6). The successor uses a bounded version of that dis-

tinction. An authority record states its source type, proposition, role in the document, jurisdiction, actor class, relevant factors, effectiveness interval, later treatment and supporting passages. For a case, a party's submission is not silently treated as the court's holding. A genuine quotation concerning a different actor or an earlier legal regime remains a context mismatch.

Priority is represented as a reasoned relation between authorities, with its own source evidence and premise status. A later date alone does not confer priority. The implementation reports unresolved effectiveness, adverse treatment, unsupported priority and priority cycles. It does not break a cycle by insertion order or model score. Nor does it remove a candidate solely because a language model labelled another authority applicable. These records make the open legal question inspectable; a complete judicial corpus and an automatically reliable treatment service remain distinct requirements.

6.35.4 Exactly what the Lean certificate checks

The registered successor profile is deliberately small. It accepts Boolean facts, true and false, conjunction, disjunction, negation and conditional expressions. Facts may be true, false or unknown. Conjunction with false is false, including when the other operand is unknown; a conditional with an unknown condition remains unknown. A conflicting dependency produces a conflict result. In the absence of conflict, a false scope produces an out-of-scope result. Arithmetic, dates and event histories are outside this theorem's language.

The checker parses the proposed formal expression with a parser separate from the production translator. It also reads the emitted RuleIR expression through a separate fixed encoder. Both become expressions in a fixed Lean semantics. For each supported candidate, Lean checks that the two expressions produce the same result for every valuation and the declared conflict indicator. The proof uses fixed source code generated by the checker. It does not execute Lean text supplied by a model or certificate. The checker rejects missing premises, unsupported syntax, a changed target, an incorrect lowering and a failed kernel execution. Its record identifies the source expression, RuleIR bundle, Lean executable and semantic prelude.

This is a proof about the two formal descriptions. It does not yet prove that the production Java emitter and JVM implement those semantics for every possible request. The accompanying Java check serves a narrower purpose: for up to six Boolean facts, it executes every assignment of true, false, unknown and conflict, at the recorded assessment time, and compares the result with an independently written evaluator. Larger domains remain explicitly unexecuted under this profile's 4,096-case ceiling. Even a complete enumeration of these abstract states does not enumerate arbitrary JSON encodings, dates or event histories.

These distinctions matter when comparing this work with Catala. Catala supplies an executable language and formal compilation work designed for legal rules (Merigoux et al., 2021). The existing LegalMath Catala route is useful for a different implementation of the same proposed formal meaning. Its shared fact preparation and source inter-

pretation remain common dependencies. A Lean certificate, a Catala comparison and a Java replay can therefore reinforce different formal checks without becoming three independent votes about English meaning.

6.35.5 An urgent-contact example that reaches the Java boundary

The contact demonstration makes its assumptions explicit. It interprets the circular's local clock times as Hong Kong time, identifies one licensed firm and one STR, and takes urgency as a supplied factual classification. The stated blackout begins at midnight on 28 January 2026 and ends at 9 a.m. on 2 February 2026. Conversion to UTC belongs to the recorded interpretation; the software does not infer a legal timezone from a publication timestamp. The demonstration also keeps the validity of the generated rule version separate from source effectiveness.

A contact record names the firm, STR, recipient and communication channel. It contains an occurrence time, a recording time, evidence identifiers and a confirmed or disputed status. To count as the declared witness, contact must concern the same STR and firm, address the JFIU through email, phone or fax, occur during the declared interval and by the assessment time, and be known by the evidence cutoff. The recording time prevents information learned later from silently entering an earlier assessment.

Absence needs more evidence than presence. One matching confirmed event can establish an observed contact. An empty log establishes absence only if a supplied completeness record covers the firm, STR, recipient, all permitted channels and the whole factual interval. Partial email history cannot establish that no telephone call occurred. Without sufficient history the result is unknown. A disputed relevant event produces conflict. Reusing the same event identifier for different actors is rejected.

The packaged Java entry point accepts these raw records and applies the fixed contact policy. A separately implemented Python projection checks the same cases. The demonstration includes different firms and reports, wrong recipients, late contacts, records learned after the cutoff, disputed events, missing history and partial channel coverage. The Java program then returns the trigger and observed-performance results separately. "Required contact not yet observed" is not labelled a proved breach: the circular does not supply every criterion needed to infer a deadline violation from that fact alone. Observed contact is likewise not overall STR compliance or proof that the communication was substantively adequate.

An additional packaged boundary accepts exact decimal percentages as strings. It converts the value to an exact rational representation before evaluating the selected numeric rule. Boundary tests distinguish values immediately below, at and above a threshold and reject malformed strings and attempts to submit internal numerator or denominator fields. The accepted representation does not establish a permissible financial exposure range. That range requires its own source and product meaning.

6.35.6 Learning from unfamiliar circulars without training on the answers

The new study freezes four source families before their model interpretations: private-market complex-product designation, simplified eDDA verification, UCITS post-authorisation changes and authentication controls. These are retrospectively unfamiliar tasks for the retained development record. They are not documents published after the system was frozen, and possible model pretraining exposure is unknown.

Both study arms receive the same explicitly selected sources. For authentication, that includes the incorporated appendix as well as the main circular. Merely placing an appendix in a retained catalogue does not put it into an interpretation when reference traversal is disabled; the controller must select it as an initial source. The shared-source freeze preserves every original root passage and reference quotation. A larger inventory is processed in bounded pieces with cross-piece checks, while exact text and source identifiers remain available. This prevents a source-access difference or a dropped appendix from masquerading as a difference in interpretive ability.

Sixteen conditional cases probe express source distinctions. Exactly 50 per cent exposure tests an inclusive threshold. Exposure below that threshold, without designation evidence, tests whether discretion is incorrectly converted to a definite non-complex classification. A client-supplied deposit screenshot tests the eDDA footnote requiring more than sole reliance on such documents. A pending UCITS application tests the transition provision, while an already bound device tests the authentication exception. Each expected answer is tied to a retained clause and explicit factual assumptions before the unfamiliar model answers are generated.

The reference is useful precisely because its limits are stated. It is a source-condition challenge set prepared by the executing researcher, not an independently adjudicated set of complete legal outcomes. A successful result does not estimate deployment accuracy, and these historical probes do not qualify a legal-correctness claim under the product's current machine-only evidence rule. During evaluation, models receive the case facts and question but not the expected answer or selected reference passage. They must either propose a factual mapping to an unchanged candidate program or declare that the program answers a different or unencoded question. Actual Java execution follows a validated mapping. Its factual and interpretive premises remain visible rather than becoming an invisible test oracle.

The comparator maintains one reading and revises it against the source. The ensemble also constructs rival factual models and readings, searches their differences, and pursues scoped objections. Both have the same declared per-task invocation ceiling, but their actual expenditure can differ. The report retains that difference, every selected task, failed generation, unaligned case, unsupported program and unanswered question. It records common misses and contrary decisive proposals without presenting this small study as a statistical ranking. Multiple versions can share errors; diversity of names or prompts cannot establish independence (Knight and Leveson, 1986; Littlewood et al., 2001).

The live pilot also exposes the cost of this design. Its two source inventories yield 46 retained claims, while generation and investigation retain eight readings. Reviewing each claim against each reading creates 368 pairs. If a request covers at most b pairs, two initial perspectives require $2\lceil 368/b \rceil$ calls; a second round for every batch raises the bound to $4\lceil 368/b \rceil$. These counts exclude extraction, generation, criticism, factual mapping and response repairs. They explain why a fixed allowance cannot be translated into a fixed number of circulars before their complexity is known.

Scheduling determines which questions receive attention before that allowance ends. The current scoped reviewer completes a batch's initial and follow-up rounds before moving to the next batch. This differs from the earlier breadth stage that constructs rival interpretations. Under a binding limit, later claim–reading pairs can therefore lack an initial review while earlier pairs have received repeated attention. Their identities remain in the report; they are not silently removed from the required work.

The continuation specification consequently includes a separate scheduling extension. It would allocate initial perspective opportunities across the complete matrix before routine follow-ups, while allowing explicitly justified priority for a consequential objection. That extension has not been implemented in this run. Its comparison must retain the same workload, spending history, deadlines and unresolved objections. Wider initial coverage is a processing benefit only; whether it reduces consequential interpretation errors requires separate case evidence.

The first 32-pair request reached its 300-second deadline without a completed answer. A 24-pair request grouped by candidate did so too. A synthetic size test had established that the requests fit the transport ceiling; it had not established their live completion time. The next partition uses twelve pairs, the size executed on the inherited backlog, while retaining every pair and the whole source. Earlier success supplies a capacity hypothesis for this transfer, not evidence that a different circular will finish. Both failed calls remain spent. A smaller batch must not replenish the request count or deadline of the underlying claim, reading and perspective.

The twelve-pair partition subsequently completed three batches, including both perspectives and both rounds for all 36 pairs. That observation justified starting the other frozen circular families concurrently. It did not justify stopping the unfinished matrix or declaring the first circular complete. Completed responses were reused with their original source, question, schema, route and reservation identities; an interrupted request remained spent. The full study still requires every selected investigation to finish. This distinction lets a useful mechanism check improve scheduling without becoming an easier acceptance criterion for the study itself.

A different capacity failure appeared in the UCITS comparison. Its smaller legacy batches required seventeen possible model responses. The admission check multiplied that count by the maximum permitted response size, producing a bound of 3,409,937 bytes against a 2,097,152-byte aggregate limit. It therefore stopped before obtaining a response. Reducing the batch size had made this worst-case bound larger, even though the intended responses were smaller. The bound established that completion could not be guaranteed in advance; it did not establish that the actual responses would exceed the limit.

The implemented optional repair admits validated parts incrementally. Let L be the fixed aggregate byte limit, A_k the retained aggregate after k admitted parts, and P_{k+1} the next individually bounded response. Each part must first pass its pair, quotation and representation checks. The conservative admission test is then

$$\text{bytes}(A_k) + \text{bytes}(P_{k+1}) \leq L.$$

Combining their lists removes repeated JSON envelope overhead, so the sum bounds the merged representation's size. A separate count limits retained concerns. An excessive part is preserved outside the aggregate, processing stops, and the matrix remains incomplete. Small responses can therefore finish a matrix whose worst-case forecast was too large, while an actual overflow still blocks completion. No concern is deduplicated or omitted to make the result fit. The focused tests exercise both successful accumulation and a late overflow. The live UCITS continuation then validated all seventeen batches and accounted for all 225 claim–reading pairs, including the retained diagnostics for an unencodable reading. This establishes completion of that comparison stage; its interpretation judgments and the other required stages remain separate.

One returned reading also revealed a concrete language boundary. It wrote an addition with four operands, whereas the advertised expression language accepted binary addition. A separate reading used nested binary additions and was executable. The unsupported reading therefore remains a proposal with an explicit encoding failure. Supporting its syntax requires a checked translation that preserves operand order, repeated operands, exact numeric types, unknown facts and conflicting evidence. It cannot be accomplished by relabelling the failed check. Nor would that extension put arithmetic inside the existing Boolean Lean theorem; arithmetic needs a separately stated semantics and proof.

Another retained reading makes the distinction between input meaning and program correctness particularly clear. It takes four Boolean facts: public SFC authorisation, membership in the stated fund exclusion, exposure at or above half of net asset value, and an operative case-by-case designation. Its scope is authorisation together with absence of the exclusion; within that scope, its result is exposure-above-threshold or operative designation. That Boolean program can be checked exhaustively in the supported profile. But it accepts the exposure threshold as an already classified Boolean input. It does not calculate net asset value, classify a business development company, aggregate direct and indirect holdings or establish that a designation is operative. Those tasks remain in the input meanings and their supporting evidence.

This is a useful small control when its caller can supply the required facts. It becomes misleading if presented as a complete implementation of the circular. The scoped reviews explicitly proposed omissions in the asset and designation classifications while preserving the narrower Boolean reading. The product must therefore deliver both the rule and the unresolved obligations of its caller. A proof over every Boolean valuation cannot decide whether the bank assigned the correct valuation to a real portfolio.

The same distinction applies to substantive comparisons. In the pilot, a proposed factual bridge initially carried the wrong model or source identity. Stronger validation

sent it through its remaining bounded repair opportunity. The repaired identities allowed inspection of the proposed mapping, but its numeric and date facts still lacked a declared comparison domain. The system therefore retained an unsupported comparison, rather than reporting conditional equivalence. Repairing a request makes the next check possible; it does not predetermine that check's result.

6.35.7 Retrieving a quotation without regenerating it

The authentication investigation encountered a simpler failure before it could complete its semantic assessment. Returned quotations replaced source nonbreaking spaces with other characters. The exact quotation check rejected them. The eDDA inventory separately repeated required source-unit rows. These failures concern response construction and source accounting; they are not evidence of successful detection of a legal misunderstanding.

The proposed repair moves exact copying into deterministic software. Each request would contain a closed set of source-unit and precomputed-span identifiers. An identifier resolves to the unchanged source text, its digest and a half-open interval of Unicode codepoints. The model selects an identifier; the resolver retrieves the characters. It neither asks the model to count offsets nor searches for a nearby quotation when the identity is wrong. Adjacent spans and their surrounding provision remain available so that a sentence boundary cannot silently discard an exception. Coverage partitions separately require one disposition for each assigned unit and reject omissions or duplicates. This extension is specified, but was not executed in the reported campaign. Its acceptance requires exact reconstruction on difficult Unicode, repeated text and cross-unit qualifications, followed by a retained live comparison of response validity. Even a perfect resolver establishes where the words came from, not whether those words support the proposed rule.

6.35.8 Changes reopen the investigation

An investigation's identity includes source bytes, selected question, assessment date, authority catalog, method implementation and bounded settings. A changed source or method creates a new revision and immediately marks the active investigation as unfinished. The old dossier remains valid evidence of what was checked for the old revision. It cannot be retrieved as the current result for the new revision. Successful resume reuses exact retained responses without claiming a new independent judgment or spending another model call.

Historical replay needs the historical implementation as well as historical source text. During the scheduling repair, the earlier replay correctly failed against two changed implementation files. The preserved source archive supplied the exact old bytes for verification in an isolated directory. The working implementation was left intact, and the historical receipt explicitly remained historical. Current implementation acceptance requires its own regression and executable checks. Otherwise an innocent orchestration

repair could either destroy readable evidence or, more dangerously, make yesterday's result appear to certify today's code.

The additional 500-call grant is a separate record anchored to the exhausted earlier grant. Reservations occur before dispatch and are not refunded after timeouts, invalid responses or interruption. Local arm limits sit inside this same total, so parallel requests cannot create additional authority to spend. After each phase, the master records the result, remaining uncertainty and next justified action. A failed phase needs a documented repair and an executed focused check before retry. This mechanism makes continued investigation auditable while preserving the substantive rule: exhausted processing leaves an explicit unresolved question, never an invented legal conclusion.

The execution also tests whether this discipline survives ordinary software failures. One authority request was rejected before an answer was generated because its evidence fields did not satisfy the service's structured-output format. The corrected request kept the failed reservation, original question deadline and remaining action limit. A separate resume failure arose when parallel PDF jobs reconstructed an ordered list of peer answers differently. The repair checks the completed investigation against its original journals and source images, preserving the answers actually examined. Neither incident justifies a change to a source conclusion. Both justify strengthening the machinery that records what was asked and what was checked.

A failed repair must preserve what the investigation already recorded. The UCITS exercise supplied a concrete counterexample: after source-fidelity processing timed out, both replacement inventory requests failed. The engine then replaced its active inventory with the empty replacement set, reducing 25 retained claims to zero. The initial inventories and nine candidate readings were still preserved, so the loss was detectable and recoverable. The implemented repair keeps the previous inventory until both replacement readers return validated proposals, records any unsuccessful replacement and leaves the investigation incomplete. Preserving the old claims does not establish that they are right; it prevents lack of new evidence from erasing the questions that still need examination. The resumed UCITS run retained all 25 claims and nine readings and completed its legacy comparison. A later scoped request reached its deadline, leaving 177 claim–reading pairs without both required perspectives. The inventory repair therefore succeeded without making the entire investigation complete.

Literal evidence also needs useful rejection messages. In the authentication exercise, a proposed quotation replaced nonbreaking spaces in a deadline with ordinary spaces. Its attempted repair introduced a null character. Both failed the exact-source check. Reporting the offending unit, occurrence count and character difference can help a bounded repair succeed while retaining exact quotation as the acceptance condition. Automatically normalising the quotation would answer a different question about source identity. In either case, an authentic quotation still needs interpretation in its surrounding provision.

A specified next extension would let a model select a passage by source-unit identity and character positions, leaving the software to copy its exact text. The source packet and unit text would each have to match their recorded digest; the start and end would refer to Unicode codepoints in that unchanged unit. The resolver would reject absent

units, changed text, out-of-range positions and ambiguous fragments. It would then construct the quotation and apply the existing evidence checks. This protocol has not yet been implemented. Its purpose is to reduce transcription errors, while preserving the separate judgment of whether the selected passage supports the proposed rule. Selecting an irrelevant passage perfectly still gives poor evidence for the rule.

A resource-limited study must also remain reportable. The master has a separate assessment mode for case evaluation, document production and current engineering verification after a terminal study failure. It binds those activities to the exact failed attempt and retains every selected task. It leaves the study failed and overall execution incomplete. If the study later changes, the old assessment becomes stale. This permits an informative account of unfinished work without changing the acceptance criterion to match the available results.

The execution tables accompanying this section report the actual phase results. They should be read alongside the retained stronger closure requirements: source completeness, source-supported factual classifications, legal priority, complementary error evidence and future-circular validation. The availability of an executable successor is a necessary step towards the product, but does not discharge those distinct evidential requirements.

6.35.9 Recorded successor execution

The following figures describe the bounded successor run of 28 September 2026. They are observations of executed software and proposed interpretations, with the separate evidential limits stated above.

Recorded observation	Count
Retained claim–candidate pairs	232
Pairs with both validated scoped perspectives	232
Pairs with all four scoped dimensions assessed	232
Retained PDF materiality issues	330
PDF issues retaining visual uncertainty	91
Authority questions investigated	6
Attributed-contact Java cases	20
Raw-history Java host cases	10
Exact-decimal Java host cases	30
Unfamiliar circular tasks retained	4
Complete joined unfamiliar investigations	0
Conditional source cases in the denominator	16
Cases lacking a reference answer in both arms	13
Java case executions after proposed fact mapping	19

Table 6.4: Executed successor evidence; counts do not establish legal accuracy.

Both initial contexts in the first scoped batch incorrectly proposed that unbound authority was available. The validator rejected those responses and the bounded repair loop retained the correction and original failures. The complete report retains unanswered source and performance questions; it grants no automatic release or unconditional legal-correctness claim. Human answers or approval are not quality evidence under the current product requirement. The source-case comparisons are descriptive and conditional on proposed factual mappings. No statistically supported ranking or future-error probability follows. The missing reference answers include different-question programs, incomplete generation and unresolved facts. Their count is therefore a coverage finding, not an estimate of English interpretation error. An unaligned program does not earn credit for abstaining from a question it never implemented.

The unfamiliar-source study did not complete its required investigations within the executed limits. Its failed phase remains failed. The later assessment preserves all selected tasks and reports unfinished processing; successful software regression or document production cannot turn that failure into a completed study. The complete execution record identifies the exact resource veto and the remaining claim–reading pairs.

6.36 Preserving the whole investigation under bounded processing

The unfamiliar-source investigation encountered a failure before any legal disagreement could be assessed. A reader quoted the authentication circular with ordinary spaces where the retained document contained nonbreaking spaces. A later response substituted null characters. The source validator rejected both. Rejection preserved the evidence boundary, but repeated requests to copy the same characters spent the allowance without answering the interpretive question. Exact copying is work that software can perform deterministically.

The September continuation separates that task from deciding what a passage means. Before asking a reader for evidence, the software gives each retained source unit an identifier. It also offers bounded adjoining spans for longer units. The reader selects identifiers; the software retrieves the corresponding characters from the unchanged source. Every selection is bound to the source edition and the particular request. An identifier from another request, an altered source, a repeated selection, or disjoint fragments presented as one quotation is rejected. The whole source remains available to the reader, so selecting a short passage does not remove a remote exception from the context.

For an engineer, the coordinate convention matters. A span starts at one Unicode codepoint and ends immediately before another. These are neither UTF-8 byte offsets nor Java UTF-16 offsets. In a string containing an astral character, those coordinate systems give different lengths. The implementation uses precomputed identifiers rather than asking the reader to count positions. Nonbreaking spaces, combining characters, carriage returns and line feeds survive unchanged. If a selected fragment occurs more than once in its unit, the reader must select enough adjoining context to distinguish it

or select the whole unit. This preserves the earlier quotation contract.

An authentic quotation can still be irrelevant. After retrieval, the original source-support and question-correspondence checks therefore run unchanged. A reader may select the correct paragraph while leaving its support for the proposition unestablished. That is a valid uncertain result. The resolver never changes such a judgment to supported merely because the identifier was valid. Both the original identifier response and the resolved quotation response are retained, allowing a later investigation to distinguish transcription failure from a disputed reading.

6.36.1 Give every question an initial hearing

Consider three batches of source claims, two initial perspectives per batch, and a budget of six requests. A schedule that completes both initial and follow-up rounds for the first batch can consume four requests there. Only two requests then remain for the second batch; the third receives no assessment. An exception in that last batch would remain unseen even if the early readers agreed enthusiastically.

The implemented round-first option offers both initial perspectives to every batch before allocating follow-up depth. Under the same six-request workload, it reaches all three batches. Both perspectives in a round receive the same earlier evidence; the first answer in the current round is withheld from the second. Invalid responses remain recorded, and every unattempted pair remains in the denominator. A transport interruption preserves the consumed attempt. Changing a partition or response schema cannot replenish a source issue's attempt allowance or restart its oldest deadline.

This ordering improves a specific accounting property: eligible initial work is not displaced by routine follow-ups on an earlier batch. The deterministic comparison does not establish a ranking in legal accuracy. An early objection may still matter more than a later routine question, and the current policy does not pretend to estimate that legal importance. It preserves the questions so that capacity limits cannot silently choose which ones disappear.

6.36.2 A small executable answer must not replace the circular

The authentication baseline attempted to return one interpretation and a 263-unit coverage table in the same response. The split-reader option now asks first for the interpretation and its exact questions. Each question names the actor, assessment unit, temporal basis and output meaning. Bounded responses then account for the source units against that unchanged reading. A response may link a unit to several questions, classify it as context with a reason, or retain uncertainty. Dependencies on units in other pieces remain explicit.

Merging is deterministic. Every required unit must occur once; invented, duplicate or missing identifiers fail. The merger retains every cross-piece concern. A final whole-source response must account for each concern and each declared question. It may retain the reading with uncertainty or propose a revision. A revised reading has a new

identity, so the previous correspondence table cannot certify it. The implementation rechecks coverage, subject to the same nonrefundable source-unit limits, or reports the remaining work.

This is also a guard against answering the wrong question well. Suppose a program answers whether a material investment-policy change needs prior approval. A separate transitional question asks how an application already pending at the effective date is processed. Both outputs may be Boolean, but they are different decisions. The coverage record cannot attach the first program to the second question merely because the types agree. An unencoded question remains visible beside an executable proposal. The declared question inventory is itself a proposed reading of the source; exact accounting cannot prove that the reader identified every possible legal issue.

The retained authentication source also reveals the distinction between input capacity and response capacity. Its 263 units and the initial reference schema occupied 261,822 request bytes before earlier interpretations or coverage results were supplied. The single-reader limit is 200,000 bytes, so dividing the response table alone did not remove the input limit. A deterministic diagnostic using the existing locator-compaction function preserved all unit identifiers, text and normative flags while reducing the initial transport to 153,212 bytes. The subsequent repair connects that function to the optional split-reader path. It explicitly distinguishes the original packet identity, which the coverage result must name, from the compact transport identity used by the reference table. Tests reject their substitution and check that the 263-unit initial request fits. Later whole-source reconciliation still has a size limit; a completed authentication investigation does not follow from the smaller initial request alone.

A deterministic capacity exercise then carried that exact packet through all coverage pieces using deliberately synthetic proposals. It retained all 263 source units and one cross-piece concern for each. Their whole-source consistency request occupied 382,396 bytes and stopped at the input limit, preserving the coverage and concerns. This is a measured final-context failure, not a completed interpretation. The next processing repair must reduce repeated representation while retaining the information needed to examine interactions among the concerns, or state an explicitly narrower supported capacity. Deleting concerns or labelling the completed pieces as a completed circular would give a misleading result.

6.36.3 Extend arithmetic without changing the proposed meaning

One retained private-market-fund proposal used four addition operands where the compiler previously accepted exactly two. Let D and I denote the proposal's direct and indirect exposure quantities in the same exact valuation unit. Its expression was $D+I+D+I$. The repeated terms represent a proposed doubling of aggregate exposure for comparison with NAV. Whether those exposure definitions and that classification rule follow from the circular remains a source question. A compiler repair must preserve the expression it receives, including every repeated operand.

The extension defines addition as an ordered fold. For operands x_1 through x_n, first

add x_1 and x_2, then add x_3 to that result, continuing in the supplied order. The lowering relation is

$$\mathrm{Add}_n(x_1, \ldots, x_n) = \big(\cdots((x_1 + x_2) + x_3) + \cdots\big) + x_n, \qquad n \geq 2. \tag{6.12}$$

For $D = 25$ and $I = 25$, the four terms sum to 100. Deduplicating the operands would give 50 and change the proposed test. Existing binary expressions retain their previous representation; subtraction and multiplication do not gain additional arities by analogy. Expanded expression size and depth are checked before a compiler is invoked.

The arithmetic theorem uses exact unbounded integers and a tagged missing value. Adding a missing operand yields missing. By induction over the remaining operand list, the ordered binary fold equals the specified n-ary operation; if every operand is known, it equals the integer sum; if any operand is missing, the result is missing. Lean checks these statements. The recorded proof names its standard logical dependencies explicitly. A separate constructor check compares the independently parsed proposal with the emitted model, while Java and Catala execute finite challenges, including negative values and integers larger than machine integers. Conflict, applicability and time selection retain their separate semantics and checks; the addition theorem does not establish them by integer associativity.

The twenty-operand challenge also exposed an implementation problem outside arithmetic: the schema validator recursively explored alternatives whose required operator already contradicted the expression. The repair discards only those logically impossible alternatives and retains the original schema checks for every remaining branch. Comparisons against the original validator and the unchanged backend challenge check that this processing repair does not weaken acceptance.

6.36.4 State what each completed check establishes

The investigation can now attach the exact source, proposed reading and question to the shared machine-qualification route. A supported formal proposition receives its checked domain and premises. An unencoded reading, an unavailable authority or an unresolved source question remains qualified. A formal theorem can be obtained while the product still withholds translation because source dependencies remain unresolved. Separate hypothetical execution may investigate that expression under declared mathematical inputs, but it does not remove the product's unresolved conditions.

Applying the adapter to all nine retained UCITS readings illustrates a different limit in that recorded run. Eight readings contain a date fact, which the constructor proof then did not support; the ninth has no executable formalization. Every reading also retains unresolved source questions. The adapter verified all nine identity-bound qualification records, reporting eight unsupported formal profiles and one unencoded proposal. It reported zero executed target cases. A preliminary Boolean-only challenge generator stopped as soon as it encountered the date fact; it did not replace that fact with a convenient date. Thus the new adapter accounts for the actual proposals without presenting a finite Boolean test or the integer-addition theorem as a proof about dates. The sub-

sequent declared calendar extension in Section 6.38 addresses this proof limit while preserving the unresolved source questions.

Numeric and date mappings also require a declared comparison domain. The domain record names every common fact, its type and unit, the listed values, and whether they are mathematical probes or an expressly declared finite domain. A separating case is replayed through the original programs. Matching probes establish no result outside those listed values. The system neither extracts a convenient domain from expected answers nor turns a source quotation into proof that a finite list exhausts the institution's possible facts.

Historical failures remain part of this account. The earlier unfamiliar-source phase completed none of its four whole ensemble investigations, despite the passing software regression. The continuation implements and tests specific processing repairs and pursues only live issues whose original limits permit another request. It does not rename the earlier failed phase as successful. Changes to the method also prevent its new results from entering the earlier frozen prospective window as if its method had remained unchanged. English interpretation and unknown future circulars continue to receive explicit qualifications until the corresponding propositions and premises have support.

6.36.5 What the live continuation actually resolved

The repaired live route used the remaining 37 authorized reservations. One provider stream disconnected; its reservation remained spent. The resumed run reused nine completed responses under their original identities before making further calls. It completed both initial perspectives for all fifteen selected batches and then six follow-up responses, with no current-round answer shown to its paired reader. Of the 36 returned responses, 27 passed the response contract and nine were rejected. All returned source-span selections resolved; the rejections concerned executable quotations, unencoded candidates or inconsistent assessment dimensions.

Those failures matter. In several batches, both initial readers assessed an executable omission for a candidate that had no executable representation. Agreement between them did not supply the missing program. The validator rejected the assessment and preserved both explanations. In another case, both readers questioned whether keeping a pending application under the existing approval process implied that the proposed substantive relief was unavailable. One retained unassessed correspondence; the other proposed a conflict. The source, exact question and disputed inference remained attached to both views.

The combined old and new evidence covers both validated perspectives for 165 of the 225 UCITS claim–reading pairs, compared with 48 before the continuation. Sixty pairs still lack a complete validated review. The continuation itself has 90 pairs with all four assessment dimensions filled; completing those fields does not establish that their judgments are correct. The final reconciliations retain 42 disputes. The original 500-reservation grant is exhausted, so further live investigation stops with those questions visible.

The master also detected concurrent additions to separate repository components during execution. Its unchanged-tree condition failed even though all 424 files in the repaired assurance snapshot retained their hashes. The failed phase receipts remain failed. A separately recorded final assessment checks the retained evidence, documents and current regression; it does not rewrite the live phase or declare the four-circular study complete.

6.37 Keeping the complete question inside a bounded request

The authentication circular exposes a practical obstacle to the whole-source check. Its retained source contains 263 units. A processing exercise assigned a synthetic concern to every unit, completed the smaller coverage requests, and then assembled their concerns for reconciliation. The final request occupied 382,396 bytes against a configured limit of 200,000. The source had not been lost: the unfinished step was the comparison of all the proposed concerns with the source as a whole. Finishing each smaller request could not answer that global question. In particular, an exception recorded near the end could still change a proposed duty recorded near the beginning.

This failure calls for a distinction between repeated representation and new information. The coverage records occupied 221,974 bytes. They repeated field names, source quotations, unit identifiers and some identical explanations. The source itself remained in the request, so an exact quotation could often be located there without printing it again. Removing that repetition has a different effect from replacing several concerns with a summary: an exact reference can recover every original word, whereas a summary requires a further interpretation of what matters. The capacity repair therefore uses readable tables and exact references, and checks that they reconstruct the prior request.

6.37.1 What the shorter representation says

The complete source, the proposed interpretation, every question definition and the response schema remain literal text in the request. A coverage table declares its columns once: the unit, its proposed disposition, related questions, related units, evidence and rationale. Each row supplies one value for each declared column. A separate concern table retains the concern identifier, related questions and units, evidence, explanation and original identifier from its smaller coverage request. Keeping both identifiers preserves the connection between a global concern and the earlier response that introduced it.

The source units have a fixed order. A unit number in a table refers to that position in the unchanged source list. An evidence entry contains three integers: the unit number and the beginning and end positions of the quoted text. Positions count Unicode codepoints, starting at zero; the ending position is excluded. For example, in the string

"A B C", positions zero through one select the first character. The implementation also exercises non-ASCII characters, combining marks, line breaks and non-breaking spaces. It never normalizes a quotation or uses the length of its UTF-8 encoding as a character position. A quotation must occur exactly once in its unit, preserving the pre-existing source-evidence rule.

Rationales and explanations receive a similarly exact treatment. A list stores each distinct complete string, and the row identifies its position in that list. Two different explanations therefore remain different even when they concern the same source passage. A long explanation is retained in full. Questions and concern identifiers stay literal, so the recipient can follow them without inferring which question an abbreviated label was meant to denote. The source-span selection table uses declared columns too; its literal span identifiers remain available for the response. The response still selects these identifiers under the existing schema, and the source-reference resolver still checks the selected quotations against the exact source.

6.37.2 The check that permits dispatch

Let R denote the complete original request after its source-span table has been prepared. Let $E(R)$ be the readable table representation, and let D be the deterministic decoder. The system retains R locally before applying the encoder. Its dispatch condition is equality of the canonical JSON bytes:

$$\text{bytes}\big(D(E(R))\big) = \text{bytes}(R). \tag{6.13}$$

Here canonical JSON fixes the representation of the same JSON value for the comparison. The check covers the entire request, including the unchanged source, question definitions, coverage hashes, response schema and instructions. It therefore compares more than the number of reconstructed rows.

The inverse is straightforward to inspect. A row recovers its named fields by the declared column order. A unit index recovers the original identifier from the source list. A quotation recovers the indicated contiguous substring, and an explanation index recovers its complete string. Applying these operations to every row restores the two record lists in their original order. The decoder also rejects unknown columns, wrong row lengths, invalid indices, ambiguous quotations and unused explanation strings. It rebuilds and verifies the source-reference table, then checks the reconstructed request against the hash held with the original request. The hash printed inside the transmitted object is not accepted as its own independent authenticity evidence.

This is an executed preservation check on each request. It is not a Lean proof of the Python implementation or a proof that a model understands tables. The distinction matters because a model could receive every word and still attach an exception to the wrong duty. The later reconciliation therefore remains subject to the same question, evidence and uncertainty checks. It must account for every concern and every question; a revised interpretation invalidates all its previous coverage and requires that coverage to be performed again.

6.37.3 What the executed capacity result establishes

The retained 382,396-byte request becomes 175,016 bytes under this encoding, leaving 24,984 bytes below the unchanged limit. Decoding reproduces the original request exactly. All 263 source units and all 263 concerns survive. An offline integration run then processes the same full source, recreates the same coverage records, and completes the final reconciliation using a synthetic response provider. Its final dispositions retain all concerns as unresolved. This exercise demonstrates that the whole-context control flow can finish within the byte bound; it supplies no legal answer to the authentication circular.

The first integration diagnostic also exposed an instructive harness mistake. Its synthetic respondent described an unencoded reading as an executable proposal. The validator rejected that response. The repair changed the synthetic response to retain uncertainty, preserving both the failed diagnostic and the rejection rule. Changing the rule to make the exercise succeed would have concealed the same error that the interpretation procedures are intended to detect.

A second capacity challenge gives each unit a long, unique rationale and each concern additional evidence. Its exact representation exceeds the same limit. The system retains that request and its successful round-trip check, then stops before provider dispatch. This outcome is necessary: an encoding cannot promise to fit an arbitrarily large collection of different explanations into a fixed request. A future source that reaches this limit needs a separately justified capacity or reasoning strategy. Dropping qualifications would change the question being investigated.

The new encoding is an explicit optional protocol, `readable-tables.v1`. Existing recorded routes retain their identities; a changed encoding cannot reuse a journal as though its request protocol were unchanged. No live model call was used in this capacity exercise, and the exhausted original allowance and issue histories remain intact. Model comprehension of this representation is untested. The subsequent executable quotation and date-proof extensions are described in Section 6.38. Unresolved incorporated sources and an observed prospective evaluation window remain open. The flow chart on page xxv connects the preservation check to those continuing uncertainties.

6.38 Checking executable quotations and calendar comparisons

The processing repairs leave two precise engineering questions. Can an interpretation proposal point to the exact executable expression it discusses? Can the date comparisons in the retained UCITS proposals receive a formal preservation check? Both questions can be addressed without making a model's legal judgment an answer key. They require different evidence: one concerns the identity of a quotation, the other a declared mathematical meaning.

6.38.1 A quotation must identify the actual expression

A source quotation and a program quotation have different subjects. The first identifies words in a circular. The second identifies the expression proposed to implement a reading of those words. A model may quote the right circular while describing a formula that was never encoded, or quote an older formula after its exception changed. Neither quotation supports a claim about the current executable expression.

The first optional executable-reference protocol, version 1, binds each selectable passage to the complete request, candidate reading, exact question and representation. It provides separate references for the scope expression, result expression and complete representation. That first design leaves a subtle problem: the representation also contains freely written fact descriptions. A description containing a line headed `Scope:` can look like the start of a second program expression to a text-matching routine. Accurately quoting that description would still fail to identify the proposed program. Likewise, a hash supplied alongside a representation cannot establish that the representation came from the retained reading. Both can be changed together.

Version 2 therefore starts with the reading and question retained by the investigation, independently of the request being checked. It parses the reading's scope and result, renders those two expressions using the existing controlled language, and locates their program block after the fact descriptions. Only these expressions receive selectable references. The entire representation remains visible so that fact meanings and qualifications are available for reasoning, but explanatory prose does not become executable evidence merely by resembling a heading. The transmitted reading hash, question and representation must match the retained originals before a model is called and again when its response is resolved.

Each reference records its expression kind, the reading and question hashes, the formalization and representation hashes, and the exact Unicode character positions of the quoted expression. The model selects the identifier; deterministic software retrieves the original text. Changing the request, reading, question or expression invalidates the old selections, even when a short expression looks identical. An expression owned by another candidate cannot be borrowed. The parser and renderer remain software assumptions in this transport check; the separate preservation proofs address their stated formal boundaries.

Exact identity includes the type of each recorded value. An integer character offset of zero and the Boolean value false are different JSON values, although Python's ordinary equality treats them as equal. A reference-table check based on that equality can accept altered metadata while returning the same quotation. The source and version 2 executable resolvers therefore compare canonical serializations, which preserve this distinction and reject floating-point offsets. The regression challenges these substitutions before any text is selected. This check protects the evidence record; it adds no support to the interpretation attached to that evidence.

An unencoded reading has no executable references. Its correspondence must remain unassessed, or be declared inapplicable to the question with the separately required justification. Software rejects a preserving, conflicting or omitting executable judgment

for such a reading. It does not repair a response by relabelling its substantive judgment. After references are resolved, the original source, question and correspondence checks still apply. Exact quotation establishes what was discussed, not whether the discussion is right.

The investigation uses source references while extracting claims, proposing readings and defining questions. It adds executable references at the later correspondence stage, when both a particular reading and its question exist. This routing is part of the recorded method. The resolver changes quotation identifiers into text; it preserves every status, rationale, follow-up question and additional concern. The existing semantic validator then checks source quotations, question identity and compatibility between the proposed judgments. For example, an exact result quotation still cannot support a preserving judgment when the same response says source support is unestablished.

A rejected response retains its original batch, perspective and reservation. The next bounded attempt receives the diagnostic and prior-round evidence; the two perspectives in one round receive the same earlier evidence. Failure at the attempt limit leaves the required pairs unresolved. Version 1 records remain historical records rather than being silently converted into successful version 2 observations. Replaying a response under the new selection rules is possible only where its original quotation names an available expression exactly. A longer prose quotation or a malformed paraphrase is reported as unselectable, not replaced with evidence the model never supplied.

6.38.2 What a calendar comparison means

The retained UCITS readings use a date to delimit a proposed regulatory condition. The legal selection of that date remains a separate question, but its mathematical comparison can be specified independently. We use valid dates in the proleptic Gregorian calendar, from year 1 through year 9999. "Proleptic" means applying the same calendar rule throughout this range, without reconstructing historical national changes of calendar. The accepted spelling is the fixed-width form `YYYY-MM-DD`.

Let y, m and d be a valid year, month and day. A Gregorian leap year is divisible by four, except for a year divisible by one hundred unless it is also divisible by four hundred. Thus 1900 has 365 days and 2000 has 366. The number of days before the start of year y is

$$Y(y) = 365(y-1) + \left\lfloor \frac{y-1}{4} \right\rfloor - \left\lfloor \frac{y-1}{100} \right\rfloor + \left\lfloor \frac{y-1}{400} \right\rfloor .$$

The first term counts ordinary days in preceding years. The remaining terms add four-year leap days, remove century exceptions and restore four-hundred-year exceptions. Let $M(y, m)$ count the days in the completed months of year y; it is zero in January and includes 29 February after February in a leap year. The zero-based ordinal of the date is

$$O(y, m, d) = Y(y) + M(y, m) + d - 1.$$

Consequently, 1 January in year 1 has ordinal zero. Moving to the next valid day increases the ordinal by one, including across month and year boundaries.

The checked proposition equates strict comparison of these ordinals with lexicographic comparison of valid year, month and day triples. The proof first checks that successive year starts differ by the correct 365 or 366 days. Month lengths and starting offsets put every valid day inside its year, with each month ending before the next begins. Dates in different years are therefore ordered by their year starts; dates within one year are ordered by their months and then days. The integer rank $372y + 31(m - 1) + d$ has the same order: it reserves 31 positions for each month and 372 for each year, enough to prevent overlap between components. Those deliberately padded positions express component ordering; they are not elapsed-day counts.

The independent formal-expression parser now accepts canonical date literals and date facts in supported comparison expressions. It reconstructs the same constructor tree as the shared model, and Lean checks that equality. The calendar theorem supplies a declared meaning for date order; the constructor theorem checks preservation of the expression using it. Neither theorem chooses the date intended by a legal provision. Evidence timestamps, business-day calendars and date-arithmetic library functions remain separate.

The implementation also checks the representation boundary. Java compares canonical date strings, while the Catala adapter uses a day count relative to 1970. Subtracting the same epoch from two ordinals preserves their comparison. An independent calendar implementation is compared exhaustively with accepted canonical spellings and the production Python date codec throughout the declared range. Native challenges exercise equality, inclusive and strict comparison, leap days, century exceptions and both endpoints through the actual RuleIR/Java and Catala routes. These are checks of the codecs and selected runtime cases. The Lean theorem does not prove the Python libraries, string comparator, Java compiler or Catala compiler.

6.38.3 Stronger formal evidence with legal questions retained

All eight encoded UCITS readings received constructor-preservation certificates. Their sources, readings and questions are unchanged; the ninth remains unencoded. The replay supplies no invented dates. Pending-application, incorporated-authority and legal-date questions remain unresolved.

A later audit reproduced the three Gregorian propositions, all 3,652,059 valid dates in the declared range and preservation evidence for the eight encoded readings. Its 49 known-date cases ran on each target, giving 98 native executions. Eight additional requests with unknown or conflicting inputs abstained at the shared input boundary before either target executed. These requests check the common boundary and cannot count as independent native results. Their behavior also supplies no theorem for arithmetic on unknown or conflicting dates.

Independent Boolean relations also check the HKMA calculations of Section 2.4.6 over their full Boolean domains. Omitting each broker-exception condition produces a counterexample. Both backends execute the proposals and abstain on incomplete or conflicting input. The earlier eligibility increment passed 338 affected tests and completed 234

product and date executions. Every real-document report retains missing bank and legal premises.

These results close specific software and mathematical gaps. Program agreement does not establish legal correctness. Theorems cannot supply private bank mandates or account records. Future rules require new observations under a frozen method, before further repairs. Development cases remain development evidence; future failures and qualifications must be preserved.

6.38.4 Continuing an investigation without erasing its failures

An additional computing allowance can help investigate an unresolved reading, but it changes neither the source nor the evidence already obtained. The UCITS investigation illustrates the distinction. Its earlier ensemble had reserved 92 requests under a 120-request task limit. A later allowance of 500 requests therefore left 28 requests available for that original task. Failed requests were already included in the 92. Counting only successful answers would make an unreliable method appear less expensive and give it more opportunities to produce a convenient result.

The software keeps the old allowance unchanged and records the additional grant separately. Before dispatch, it checks the original task balance and the history of the particular source claim, proposed program, question and reader perspective. Repacking the same question into a smaller batch does not create a new issue. Nor does adding an executable-reference identifier: that identifier helps locate evidence but contributes no new substantive question. In contrast, an amended source must be retained as a new edition whose relationship to the old reading is investigated explicitly.

Time limits also belong to the original issue. During the continuation, one pair's deadline expired after the initial eligibility inventory but before dispatch. The guard stopped the request before charging a model call. This case requires checking eligibility immediately before dispatch as well as during planning. The excluded pair remains in the report. It cannot disappear from the denominator merely because the other pairs can still be processed.

The first live version 2 batch supplied two validated proposals for eleven pairs; one pair was excluded by that deadline. The two responses retained an unresolved correspondence assessment: one of the eleven pairs concerned a reading that had no executable formalization. These are distinct observations: a response can obey the evidence protocol, two perspectives can both be present, and the substantive question can remain unresolved. The same model supplied both fresh contexts, so these observations do not measure independence or accuracy. The subsequent execution record keeps the pair counts, disagreements, invalid responses and expenditure separately.

The broader continuation supplied both perspectives for another 47 pairs. Together, these runs added 58 pairs with two validated responses to the earlier 165, leaving two of the original 225 pairs outside that coverage because their original deadlines had expired. Among the 58 newly processed pairs, 42 had all four dimensions assessed by both perspectives. Twenty unresolved or disputed pairs remained in these new results,

including four with differing dimension judgments. An unassessed dimension and a disagreement are different reasons for retaining a question. The earlier 42 disputes remain in their original record; the new totals do not erase them.

A later follow-up lost its upstream connection. Its reservation remained spent, bringing the continuation to 17 requests and the original task to 109 of its 120-request allowance. Concurrent work also changed the shared package while this run was active, so the controller refused to certify a stable method identity for the run. Every returned response was subsequently rechecked against its unchanged source, reading and question without another model request. That rechecking establishes the retained reference and response checks, not a retrospective method freeze. Later phases execute copied source trees so that work on another module cannot silently change their method.

6.38.5 When a correct transport still cannot finish the task

Exact reconstruction of a compressed request answers a software question: did the model receive the intended material? It does not answer whether the model can return the required result within the available time. The authentication-source diagnostic made this distinction concrete. Its input contained all 263 source units and 263 deliberately unresolved synthetic concerns. Both fresh attempts to return a disposition for every concern hit the five-minute deadline, although the request itself fitted the byte limit. The concerns were constructed to test accounting; their existence was not evidence of 263 defects in the circular.

The repair partitions the requested answer while retaining the whole input. Four groups contain 66, 66, 66 and 65 concerns. Every request still contains the entire source, the interpretation, all questions and all concerns, so an exception outside the requested group remains available to the model. Each answer must account for its selected concerns exactly once and retain the original questions. The joining check rejects repeated or foreign concerns, lists missing groups and retains differing question dispositions or proposed revisions. Accounting for every identifier does not prove that cross-group legal interactions were understood; that remains an explicit qualification.

The first smaller answers exposed a separate error. The response format required a cryptographic hash for each question, but the request supplied no such mapping. A model with no calculation tools could copy a supplied hash but could not reliably compute the required digest from the question's JSON. One response substituted zeros. The validator correctly rejected it. Earlier synthetic tests had calculated the hashes inside their test provider, concealing the missing input that the actual model faced.

The corrected transport calculates each question's digest before dispatch and supplies the mapping explicitly. The output schema permits only these supplied hashes, and the validator still checks which question owns each one. Supplying this deterministic bookkeeping value introduces no legal answer key: the question's status, rationale, evidence and uncertainty remain the model's proposal. The changed request contract also changes the cache identity, so an old result cannot silently stand for execution of the repaired request.

Both full-output timeouts and both smaller invalid responses remain in the record. The first 66 concerns have consumed two attempts per perspective; the repair has only their remaining third attempt. Later groups retain the attempt spent on the original full response. A failure stops further expansion of that perspective and leaves every missing concern visible. This example shows why a repair must address the observed cause while preserving the history that made the cause visible.

The final bounded run returned eight validated responses, one per group and perspective. Each perspective accounted for all 263 concerns; the joining check found no differing question statuses between that perspective's batches and no proposed replacement interpretation. The twelve-call diagnostic budget includes the earlier four failed requests. The result demonstrates complete response accounting for these synthetic concerns under the repaired input contract. It supplies neither a legal accuracy estimate nor evidence that the two perspectives are independent.

6.38.6 New source evidence changes a particular premise

The earlier tokenised-securities experiment in Section 6.31 received circular 23EC52 without its separate appendix. A later source revision joined the retained circular and the acquired three-page appendix into a 177-unit packet. The revision preserves the predecessor's six authority questions and its extraction findings. It does not copy an earlier model judgment into the enlarged context and call the question settled.

Both fresh first-round proposals distinguished the newly available appendix from legislation still absent from the packet. They proposed that the appendix references could now be inspected directly, while complete legislative provisions, exemptions, relevant editions and transaction applicability remained missing premises. Their precise explanations and alternative readings were retained alongside exact source selections. These are proposed answers from the configured model, rather than a finding that the English meaning has been proved. The source revision closes an availability gap within this packet; it does not close all the legal questions that motivated retrieval.

The first follow-up could not be sent within the unchanged 200,000-byte bound. It contained both earlier proposals as well as the entire revised source. Lossless table encoding reduced that exact request from 209,707 to 190,946 bytes and reconstructed it without a change. Subsequent processing retains the original role spending and deadline. If later proposals make the context too large again, the system reports that limit; it does not discard an inconvenient reading to make the request fit. Neither successful retrieval nor successful compression supplies the still-missing legislative premise.

The encoded follow-up returned another proposal from each perspective, leaving four retained proposals in total. All four reported partial source support and missing premises. The next round exceeded the byte limit again as the retained proposal history grew; both requests stopped before a model call. The source question therefore remains qualified with an explicit processing limit. It does not become resolved merely because both perspectives named similar gaps.

6.38.7 A future observation belongs to the whole interpretation method

Consider two programs that produce the same formal expression. One receives the full appendix; the other receives a text extraction that omits an exception. A proof checker may accept both instances of that expression. Identifying only the checker and expression would conceal the difference that matters most to the legal question. A prospective observation must identify how the source became an interpretation, as well as how the interpretation became Java.

The full method therefore records the extraction software and model locks, the configured model route, generation and search settings, the mechanisms for proposing questions and investigating authority, the source and executable reference protocols, the issue limits, and the qualification software and toolchains. The dispatcher derives this record from its actual configuration before beginning an investigation. Each completed investigation carries it alongside the raw source editions, selected question and individual stage records. A later verifier compares those records with the frozen method and rechecks the qualification attached to every retained candidate.

This comparison serves a specific purpose. A changed extractor, search budget or reference protocol makes the result evidence about a changed method. Missing authority processing cannot be concealed by a successful Java build. A different source edition or question cannot borrow another investigation's qualification. Every claim–candidate pair remains accounted for, and an unencoded candidate remains visible. A missing stage or failed verification leaves the submitted task pending or explicitly incomplete; it does not become a successful observation through a summary label.

Selection precedes observation. The study freezes its method, source families, publication window and development-source identities. It retains every submitted task, including an ineligible submission or an unfinished investigation. The raw primary document identifies the source even when additional background documents are attached. The retained stage times must follow admission into the study; a previously completed run cannot count as a new observation simply because its hashes still match. If observations motivate a repair, that window becomes development evidence. The revised method needs a later window whose sources were not used to make the repair. An older window that bound only the final qualification checker retains that narrower meaning; it cannot be relabelled as a study of the entire interpreter.

A first encounter after the frozen window closes remains outside that study, even if the document was published earlier. Renaming a task does not turn the same source and question into another eligible observation. Different questions about one circular may still be investigated, so the report also counts distinct primary sources. Neither count establishes independence of legal questions or of circulars issued under the same regulatory regime.

The resulting record can establish that a stated method processed particular retained inputs and that its mathematical and executable qualifications passed their registered checks. Locally retained hashes do not authenticate remote model execution or a fixed

remote model revision, establish the publisher's clock, or prove that every eligible publication was submitted. Binding dependency lock records also does not attest the installed contents of every third-party package. Those limits accompany the study report. Most fundamentally, an unchanged method and checked program do not establish that the legal premise was correctly understood. The value of the record is that future failures remain attributable and challengeable, while the propositions actually proved keep their precise mathematical scope.

6.39 Challenge each interpretation before narrowing the alternatives

The UCITS continuation makes the central difficulty concrete. Two readers can return valid source references for the same claim while leaving its executable meaning unsettled. Increasing the number of replies does not tell us whether both readers overlooked the same exception. The challenge method therefore asks what could falsify each proposed reading and records the evidence needed to act on that challenge. The implementation joins source-bound questions, independent Boolean checks and bounded reader follow-ups inside the complete investigator. Its execution record distinguishes these mechanisms from the unresolved question of whether the proposed legal premises are right.

6.39.1 A challenge is a question with a complete denominator

Consider a simplified, invented provision: "An intermediary must make the report when condition A or condition B holds, unless exemption E applies." This sentence teaches the mechanics without purporting to reproduce a circular. One proposed program computes $(A \vee B) \wedge \neg E$; another computes $A \vee (B \wedge \neg E)$. At $A = \text{true}$ and $E = \text{true}$, the second program requires a report even though the first does not. The difference is mathematical. Deciding whether the exemption applies to both conditions remains an interpretation of the sentence and its legal context.

Every retained candidate is consequently examined along nine declared dimensions: actor, scope, quantification, negation, exception, time, definition, authority and output meaning. The dimensions are a useful working vocabulary, not a theorem that all legal ambiguities have these forms. A reader can add a new concern or an alternative that the formal vocabulary cannot express. The report keeps those concerns beside the original candidates.

A challenge binds the exact source edition, original question, candidate and dimension. Its identifier is calculated from those substantive inputs, so a different batch number cannot make the same challenge new. A source scanner also locates words such as "unless", "each" and "before". These locations nominate questions for investigation. Their absence does not prove that the corresponding exception, quantifier or temporal qualification is absent: it may be expressed differently or incorporated from another

document.

The denominator is the full set of required candidate–dimension pairs. An invalid reply, an unassessed dimension and an expired attempt remain in that set. Otherwise a reader could appear more reliable merely by answering the easy questions. Every response retains its exact evidence and a proposed disposition, while the program separately records which questions have actually received the required perspectives.

6.39.2 What a counterexample permits us to reject

Return to the exemption. Suppose an investigation proposes the explicit formal premise that exemption E precludes this particular reporting duty. Write $P(x)$ for the premise that E holds in factual situation x, and $R(x)$ for the candidate's decision that the duty applies. The proposed obligation is

$$\text{for every } x \in D, \qquad P(x) \Longrightarrow \neg R(x),$$

where D is the declared domain of complete Boolean assignments to A, B and E. There are eight such assignments. The checker evaluates them independently of the production expression evaluator. A candidate that returns true at an assignment with E true supplies a replayable counterexample to this obligation.

For a finite collection of declared obligations $\mathcal{O}$, let $C(h, o, x)$ mean that candidate h satisfies obligation o at assignment x. The candidates surviving these checks are precisely

$$H_{\mathcal{O},D} = \{h \in H : \forall o \in \mathcal{O}\ \forall x \in D,\ C(h, o, x)\}.$$

If a checker finds o and x with $\neg C(h, o, x)$, elementary logic gives $h \notin H_{\mathcal{O},D}$. The executed Lean theorem checks this conditional elimination rule. It assumes the stated domain and obligations; it does not prove that the exemption sentence warrants the proposed formal premise. The original candidate and the counterexample both remain available in the investigation. A false premise can invalidate an otherwise useful elimination, so the legal conclusion cannot be silently copied from the mathematical result.

Breadth-first enumeration provides a second, distinct check. Within an explicit grammar, the program constructs Boolean alternatives before assigning search priority. It records the grammar, enumerated expressions and remaining frontier. Exhausting that finite grammar establishes only that its expressions were considered. It says nothing about an omitted actor, a different factual abstraction, an unrepresented date or a relevant authority outside the packet. Search priority therefore describes which distinction to inspect next, never a probability that a reading is legally correct.

6.39.3 Different opportunities to notice the same error

The source scanner, independent Boolean evaluator and language-model readers encounter different failure opportunities. A scanner can preserve an explicit "unless" that both readers ignored. A mathematical evaluator can replay a violating assignment that

a persuasive explanation obscures. A reader can propose that the factual vocabulary itself omits a relevant condition, which neither of the other checks can infer from Boolean structure alone.

These differences justify combining the methods, but they do not establish statistical independence. All may depend on the same incomplete source packet. Two fresh conversations may use the same model family. Two compiled programs may derive from the same mistaken interpretation. The investigation therefore records actual mechanism and provider identities and their shared dependencies. Changing a reader's name cannot create another model family. If an alternative configured provider is unavailable, that absence remains visible.

Controlled faults test the connections. For example, remove the exemption from a proposed Boolean program while retaining the declared formal obligation. The counterexample checker must expose a violating assignment, and the complete investigator must retain it. Such a test checks a specified mathematical fault. It neither supplies a human legal answer nor estimates the frequency of future legal mistakes. Detection counts describe this declared collection of faults; they cannot establish superiority among stochastic interpretation methods.

6.39.4 Bound the answer while preserving the investigation

The earlier 263-concern diagnostic showed why output size must be controlled separately from source access. The new profile requests a bounded group of challenge responses while preserving the complete source, question and candidate context. Its first round obtains blind perspectives. Follow-up rounds receive the same retained prior-round evidence and investigate differences in status, explanation or proposed alternatives. All original versions remain available.

The joining check accounts for every required challenge exactly once in each completed group and exposes missing groups. Equal status words alone are not semantic agreement: two readers can use the same status while proposing different exception scopes. The report therefore preserves substantive response differences and unresolved cross-group interactions. No majority vote may remove an interpretation. After the declared maximum number of attempts, the unresolved question and its history remain part of the result.

The profile must execute inside the complete investigator, and its actual stage files must be checked when the dossier is verified. A successful standalone helper does not establish that the product called it. Changing the input codec, challenge vocabulary or follow-up policy changes the method identity and requires a later prospective window. The earlier tested method remains a separate historical record.

Finally, evidence acquisition changes particular premises. A newly obtained legislative edition can answer whether a referenced provision is available; its effective date and application to the transaction may remain disputed. Installed-package file hashes can establish which local files were inspected; they cannot attest remote model weights or the publisher's clock. These limits belong beside the affected result. The useful product is a

progressively more checkable investigation that explains exactly what was established and what still prevents a legal conclusion.

6.39.5 The implemented rule and its assumptions

The mathematical rule needs a slightly more precise treatment of applicability than an ordinary Boolean program. Each candidate supplies a scope function $S_h : D \to \{0,1\}$ and a decision function $R_h : D \to \{0,1\}$. A proposed obligation supplies a guard $G_o : D \to \{0,1\}$ and an expected decision $e_o \in \{0,1\}$. In this implementation the obligation is satisfied at x when

$$C(h,o,x) \quad \Longleftrightarrow \quad G_o(x) = 1 \implies \big(S_h(x) = 1 \ \wedge \ R_h(x) = e_o\big).$$

Thus an active obligation that the candidate excludes from its scope is a violation of the declared obligation. This convention matters: otherwise a candidate could pass every check by declaring every relevant transaction out of scope. Whether an obligation really applies to that transaction is still a premise supplied by the investigation.

The independent checker supports total Boolean assignments to at most eight named facts. It parses the stated expressions separately and enumerates all 2^n assignments. For each active obligation it records the first violating assignment, together with the source, question, candidate and obligation hashes. Verification recomputes the certificate from those original inputs. Unsupported numeric, temporal, unknown-value or conflict semantics receive an explicit qualification; they do not produce a vacuous passing certificate.

Three propositions are checked by Lean. The first says that one active obligation and a violating assignment contradict universal satisfaction. The second says that quantifying over every item in every declared batch is equivalent to quantifying over their joined index set. The third says that two different reported values contradict a uniform report across that set. These propositions are deliberately stated over supplied functions and index sets. They do not prove that the Python parser implements the Lean definitions; the independently replayed finite checks supply separate evidence for the supported Python fragment. Nor does disagreement among explanations establish which explanation is legally right.

6.39.6 What the executed checks establish

The first complete-investigator integration run passed 107 focused checks. It exercised the new stage, retained its actual responses and checked those files when the investigator's report was verified. A further 18 checks covered the executed Lean rules, bounded panel and complete dispatcher together. These counts describe software checks, not independently established legal answers. In the controlled exception experiment, two unanimous reader proposals could not suppress the Boolean check's counterexample to the supplied exemption obligation. Altering the question, replacing an actual response, deleting an executed action or changing the kernel receipt was rejected by the

respective verification boundary.

The source work exposed another practical source of false confidence. A request for Chapter 571 as a PDF returned an HTML browser-check page. The government's bulk legislation archive was available, but its approximately 560 MB response exceeded the declared five-minute acquisition limit. The repair used the ZIP directory and HTTP byte ranges to recover the complete selected Chapter 571 XML member. It retained the range bytes, required the same strong ETag throughout, checked each Content-Range and let the ZIP reader verify the member's CRC. About 716 kB of range responses recovered a 3.47 MB XML member. This establishes which bytes were retrieved, not which historical provisions govern a particular transaction.

The original failed requests remain alongside that repair. In particular, the SFC circular endpoint returned the correct circular as HTML despite the initial expectation of JSON. Correcting its media-type interpretation did not rewrite the original expectation or response. This is why the execution record must distinguish successful accounting of an attempt from successful acquisition of the requested authority. The repair phase requires a nonempty complete chapter member before it can pass.

The XML identifies its consolidated edition as 24 August 2025 and exposes separate parent elements for Part IV and Schedule 1. These are useful structural anchors for the retained authority question about the 2023 circular. The date also supplies a reason for caution: a later consolidated edition is not, by itself, evidence of the law applicable at an earlier transaction date. The full member remains available for a provision-by-provision temporal investigation; the retrieval result does not close that question.

6.39.7 The live capacity test and its unresolved work

The test on circular 26EC22 encountered a different obstacle from the synthetic exception example. Its fourteen retained interpretations generated 126 candidate–dimension obligations. The first request occupied 270,261 bytes, above the existing 200,000-byte input limit, so neither reader received it. A recorded investigation could therefore contain all the proposed questions without having obtained any challenge response. Recording the failed request established what went wrong; it did not demonstrate that the readers had examined the interpretations.

The repair stored each repeated passage once and represented the obligations as rows with named columns. Every reference expands to its complete original text. The complete source, all fourteen interpretations, their original questions and the mathematical checks were preserved. Exact reconstruction was checked before dispatch for all twenty-two planned requests; the largest occupied 194,723 bytes. This establishes preservation of the input and its fit within the declared bound. It does not establish that a reader understands the compact representation as reliably as ordinary prose.

The continuation reused the retained interpretation and question proposals under the implementation that produced them. It did not ask the models to generate those proposals again. The new challenge requests inherited the original attempt counts and start times, so changing the representation did not renew an exhausted issue. Earlier

failures remained available beside the new responses.

Nineteen requests reached the model service. Sixteen returned responses that passed the structural, identity and quotation checks; two connections failed, and another response carried a wrong binding identity. The latter was rejected, but an error in rejection handling interrupted preparation of the final report. An offline repair reconstructed that report from the recorded actions without making another model call. Three planned requests remained undispatched. The report retains 42 obligations missing a required perspective, 75 unresolved obligations and 21 proposed alternative-reading records. The alternatives have not all been investigated, and repeated proposals are not independent findings.

The responses also make the substantive uncertainty clearer. For paragraph 10, readers distinguished retained accountability from effective performance or discharge of responsibility. An operational defect might be evidence bearing on responsibility without, by itself, determining whether this particular requirement has been breached. These are competing proposals about the source, not settled legal interpretations. Eleven original candidates lacked a supported formal program. The other three supplied no declared formal obligations for a counterexample check, so their absence of a found violation does not support their legal correctness.

The repaired software passed 2,243 regression tests. That result and the validated responses support different conclusions: the tests exercise stated software properties, while the live responses show that the repaired input reached the readers and yielded usable proposals. The investigation remains incomplete. Its next substantive work is to investigate the missing perspectives and new alternatives, obtain the required authority and define justified formal premises. Reusing this development circular cannot supply evidence about performance on genuinely later publications.

6.39.8 The supplemental review and what remains unresolved

The supplemental review completed the missing perspectives. Six calls supplied all sixty missing reviewer assignments, and thirty-six further calls assessed each of the fourteen original readings and twenty-one alternative records from both roles. Every record was retained. The alternatives are still proposals about meaning, and repeated proposals are not independent discoveries. The active source is the revision of circular 26EC22 dated 20 April 2026; the older authority questions about the 2023 circular belong to a separate historical inquiry.

The resulting seventy assessments proposed twenty-four conditional Boolean programs. Eight original readings and five alternative records received at least one such proposal; forty-six assessments remained explicitly unformalized. Each accepted program had satisfiable positive and negative obligations, so an empty premise could not produce a passing result. Exhaustive checks found no violation of those declared obligations. Separately, the existing conditional expression compiler agreed with native execution on twenty-four bounded probes per program, including unknown facts. Production translation remained blocked by the original source questions and

dependencies. None of its proposed runtime cases was executed.

Passing these checks did not make the competing readings equivalent. One reading separates retained responsibility for management, operational soundness and ownership records from achieved performance. Another additionally requires an adequately managed, sound arrangement and proper ownership records. Under an explicit illustrative identification of their responsibility predicates, exhaustive enumeration produced three assignments in which the retention program returned true and the outcome program returned false. For example, responsibility could be classified as retained while the records were classified as improper. This is a conditional distinction between programs. The identification of the predicates, the factual possibility of the example and the legally correct interpretation remain separate questions.

The authority review also caught two errors in selecting source text. Several parts of the Chapter 571 XML use the label P4; the first extraction allowed Schedule 10's transitional provisions to replace the operative Part IV on offers of investments. In the 2024 Unit Trust Code, a search for the Chapter 6 heading selected its contents entry. Selection now checks the statutory parent and the presence of sections 103–105, and requires the chapter's substantive numbered clauses. The failed selections and the responses that exposed them remain available.

Both reviewers then examined the corrected sources for the current tokenisation question. Section 6.8 expressly permits a trustee-appointed person to maintain the register. Its wording agrees across the retained 2018, 2024 and current Code editions after declared whitespace normalization. Circular 26EC22 separately retains the Product Provider's responsibility despite outsourcing. These texts make the distinction between an appointed performer and retained responsibility more concrete, but they do not by themselves determine legal title, settlement finality, the complete liability standard or effective performance in a particular arrangement. The statutory consolidation is dated 24 August 2025, with section 103 amended from 23 May 2025. It establishes neither the complete historical 2023 law nor uninterrupted applicability through April 2026. The two historical structured-product re-reviews exhausted their allowed attempts after quoting passages that occurred more than once in the supplied statutory unit; their results remain unaccepted.

Twenty-four follow-up calls reconsidered all fifty-three obligations still open after the missing perspectives had been supplied. Their complete requests included the original circular, readings and questions, both relevant reading assessments, and the corrected current-authority findings. Repeated field names were stored as named table columns, alongside the exact text pool. Independent reconstruction preserved every field and value, and the largest request occupied 193,268 bytes. The two pilot responses initially failed because quotation validation still admitted only the original circular's source identifiers. Checking the additional quotations against their retained, hashed authority sources recovered those unchanged responses without another model call.

The follow-up reduced the unresolved count from fifty-three to forty-six: fourteen questions each about time, definitions and authority, and four about the meaning of the output. Relative to the earlier seventy-five unresolved obligations, this is a narrower and fully reviewed set of questions, not a count of legally settled propositions. No new

reading was proposed. Fifteen new formal-premise records yielded thirty-seven non-vacuous checks without a violation; eight other program–premise combinations used incompatible factual vocabularies and remained unsupported. No inferred renaming converted those combinations into passes.

The continuation used seventy-nine of the one hundred additional calls. Every reservation has a final record, including rejected responses and the two offline recoveries. The old spending records and all earlier failures remain unchanged. The revised extraction and encoding implementation passed 2,331 regression tests; the later authority-citation and recovery changes passed fifty-one focused checks and replay of the actual provider receipts. These results establish the reported software behavior and the extent of the review. Resolving the remaining questions requires applicable legal standards and editions, the identities and roles of the actual parties, the governing scheme documents, and evidence about ownership records, controls and regulatory requests. Additional model agreement alone would not supply those facts.

6.40 What case law can contribute to an interpretation

A product provider can retain responsibility for an outsourced arrangement while supervising it poorly. The arrangement can also operate successfully during a period for which the provider's authority is unclear. These possibilities matter because a program that asks only whether `responsibility_retained` is true cannot discover which of these questions its input was intended to answer. It can apply the supplied judgment consistently while leaving the important legal distinction unexamined.

The tokenisation circular gives this problem a precise setting. Paragraph 10 states that Product Providers should remain ultimately responsible for the management and operational soundness of the arrangement and ownership record keeping, regardless of outsourcing. Paragraph 14 says they should "confirm and, upon SFC's request, demonstrate" the specified matters. Footnote 5 says they should "confirm and demonstrate" compliance with the preceding requirements and the specified assurance concerning smart contracts. The scope and interaction of these statements must be interpreted; neither a responsibility flag nor a general request flag settles them (SFC, 2026c, paras. 10–16 and n. 5).

The current investigation consequently has two substantive tasks. It must justify the proposed meaning of each duty, and it must explain what evidence would establish the facts needed to apply that meaning. Case law may help with both, by explaining legally significant distinctions and the reasons they matter. To use a judgment responsibly, however, the product must identify what the court decided, the facts on which that reasoning depended and the conditions under which the reasoning bears on the present question. A retrieved paragraph is the beginning of that account.

6.40.1 The distinctions still missing from the product

The product assessment of 5 October 2026 records 35 reading identities in two roles. Those 70 role records are not 70 independently established meanings. All 46 original challenge obligations remain open. The recorded 20 mechanical guards and 1,040 synthetic checks establish behavior under declared premises; the selected repair package records no accepted substantive repair. These counts describe that frozen assessment, rather than a fresh experiment or a current legal error rate.[1]

There are also narrower defects that can be corrected without deciding the law. Some operative explanations still say that referenced texts are unavailable even though acquisition records show that they have been obtained. Acquisition and applicability are different questions. Other fields state the request condition categorically while a qualification elsewhere preserves the unresolved interaction with footnote 5. The uncertainty must be expressed where the duty is described, so a reader does not receive a settled answer in one place and its withdrawal in another.

The more difficult gaps concern actors, activities, effective periods, ownership, incidents and legal consequences. A provider's supervision is not an intermediary's conduct; possession of a token does not by itself settle legal title; and an incident followed by remediation is not the same factual history as no incident. The word "should" also requires attention to the authority and context in which it appears. Satisfying one encoded predicate cannot establish overall compliance or the absence of civil liability.

These are reasons to improve the representation of meaning. They are not reasons to require a unique answer where the available sources leave the question open. A useful interpretation may identify two supported proposals, show exactly where their consequences differ and state the missing premise that prevents choosing between them.

6.40.2 A precedent supplies a reason for a distinction

HYPO provides an early, precise account of reasoning from cases. A proponent identifies a precedent and the similarities that support its application. An opponent distinguishes that precedent or supplies a more relevant contrary case. The response must address the distinction rather than repeat the original outcome. CATO develops hierarchies of legally relevant factors and ways to emphasize or downplay differences. The benefit is an inspectable argument about why a fact matters, rather than similarity between two blocks of prose (Bench-Capon, 2017, secs. 2–3).

Suppose, solely as a design example, that one proposed interpretation of responsibility turns on retained decision-making authority, while another also requires evidence that supervision was exercised. A useful case comparison would identify the authority and supervision findings in the earlier case, the proposition for which that case is cited, and the reason a difference in supervision matters here. It would not simply report that both arrangements involved outsourcing or that both disputes had the same outcome.

[1] The assessment and exact package identities are retained in `docs/implementation/assurance-evidence-master/product-gap-audit-2026-10-05.md`.

The example supplies no real precedent and settles neither interpretation; it identifies the information an actual precedent analysis must recover.

AGATHA extends this approach by constructing competing case-law theories. Its inputs include cases represented through factors and outcomes, with associated rules and value preferences. Search compares candidate theories using such considerations as explanatory power, simplicity and completeness. The formal representation makes rival explanations available for examination, but the heuristic search need not find the best theory. The advocacy setting also permits preference moves that cannot be imported as neutral findings about governing law (Chorley and Bench-Capon, 2005, secs. 2–3, 6–8 and 10).

The limitation is consequential. These systems start after much of the difficult interpretive work has been encoded. A factor such as retained authority is not the raw text of a judgment. Someone or something has selected the relevant passage, distinguished an allegation from a finding and decided what proposition it supports. We can borrow the analogy, distinction and counterexample operators while keeping that earlier justification visible. The inspected material supports design reuse; it does not establish that an official, reusable implementation of the legacy systems is available.

6.40.3 Retrieving the right case is an incomplete success

A case-law product must first find relevant material, and this is a useful task in its own right. CLERC develops case retrieval and analysis-generation tasks from a large US case-law collection. Its retrieval labels are derived from citations in the original documents. Its generation study evaluates text similarity and citation behavior, while its discussion explains why a correct citation can accompany a deficient analytical proposition. A system may cite the expected case but fail to state the proposition for which that case is relevant (Hou et al., 2024, secs. 3–5, especially sec. 5.2).

That distinction rules out citation accuracy as a substitute for an account of the holding. CLERC also concatenates majority, concurring and dissenting opinions in its case-document construction. That choice serves its retrieval task, but a precedent interpreter must preserve the speakers and opinion types. A dissent can explain a serious objection without supplying the court's governing rationale. Procedural posture and later treatment can further limit what an otherwise accurate quotation establishes (Hou et al., 2024, sec. 3.1).

The open-source eyecite library can identify citations and connect shortened references to earlier citation forms. Resolving them to actual judgments still requires a corpus or lookup mechanism. Its principally US-oriented conventions need a Hong Kong coverage check. It is not a service establishing that a case remains good law. Similarly, a search that finds no contrary authority establishes a search outcome within its coverage, not the absence of contrary law.

Commercial tools do not remove the need for this distinction. Magesh et al. (2024, secs. 5–6) evaluated 202 queries against the versions of three commercial research tools available during their 2024 study and reported hallucination rates between 17% and

33% under their correctness and grounding rubric. These are historical sample results, based on the authors' human evaluation, not measurements of current 2026 products or quality labels for LegalMath. Their relevance here is narrower: supplying retrieved legal sources did not itself establish correct analysis.

6.40.4 An argument can fail at its premise, rule or conclusion

Once a proposed reading and its support are explicit, objections need more structure than a single agree-or-disagree label. A challenge might say that the source does not establish a factual premise. It might accept the premise but contest the inference drawn from it. Or it might offer a contrary conclusion supported by another rule. These differences determine what a repair must change.

ASPIC+ distinguishes these attacks through structured arguments built from premises and strict or defeasible rules. Undermining challenges a premise, rebutting challenges a conclusion, and undercutting challenges the use of an inference rule. Preferences can determine whether an attack succeeds as a defeat. This provides a useful vocabulary for the paragraph-14 dispute: one argument may challenge the proposed scope of the request condition, while another challenges the priority or applicability assigned to the footnote (Prakken, 2010, secs. 2–3).

The semantics then asks which arguments can jointly survive the declared attacks. Grounded semantics yields one extension; preferred semantics can yield several. Here an extension is a set of arguments accepted under the specified formal definitions. It is not a set of independently established legal truths. The premises, contrary relations, inference rules and preferences remain part of what must be justified.

The inspected PyArg implementation is a practical Python candidate for a bounded prototype. Its conversion to an abstract argumentation framework chooses a last-link elitist ordering when no ordering is supplied. That default must be made explicit. A software preference cannot decide which reading the SFC intended. Argument generation must also be bounded, with any unfinished frontier reported. A truncated set of arguments cannot establish that no objection exists.

ASPIC+'s rationality results require their particular assumptions. The author's correction to the well-formedness definition is therefore a material dependency for any attempted proof reuse. Our proposed use begins with explicit argument and attack definitions; it does not import every theorem for an arbitrary generated graph. A separate independent check must compare the chosen implementation with the precise finite semantics claimed.

Carneades contributes a complementary account of ordinary premises, assumptions, exceptions and burdens of proof. This helps explain why an unanswered challenge and a rebutted proposition should not have the same status. The original paper does not claim that its proposed proof standards adequately model legal proof standards (Gordon et al., 2007, secs. 3–5). The later Carneades 4 implementation has different semantics and numerical conventions. Its default proof standard, weights and unsupported-statement labels must not be transferred into Hong Kong law or read as factual negation.

We should borrow the premise distinctions immediately and assess an engine choice separately.

6.40.5 Specify the reading before compiling it

Arguments require propositions with stable meanings. LegalRuleML provides ways to associate alternative interpretations with their sources, authorities and contexts, including temporal and jurisdictional scope. Its treatment of competing readings of an income-year provision shows why two formalizations may need to coexist. It also distinguishes prescriptive rules, which impose duties, from constitutive rules, which establish legal classifications (Athan et al., 2014, secs. 3–4).

For the circular, the representation should therefore state who must do what, for which activity and product, during which interval, under which condition and subject to which exception. It must preserve whether a condition governs confirmation, demonstration, third-party verification or a legal opinion. These are local design requirements motivated by the disputed source, not a claim that adopting the LegalRuleML schema determines its interpretation.

Logical English supplies another useful component: controlled predicate templates and explicit conventions for binding variables. These can make a proposed specification readable while retaining executable structure. The language reduces selected ambiguities by restricting the syntax accepted as a specification (Kowalski et al., 2023, secs. 1–2). It does not establish that the chosen specification faithfully translates unrestricted English. Its basic untyped syntax and treatment of negation also require care when facts are incomplete. A noun used as a variable name is not itself a type constraint, and failure to establish supervision must not automatically become evidence that supervision did not occur.

Figure 6.28 isolates one consequence of the request-condition dispute. Both branches are proposed readings; the other relevant facts are held fixed for the illustration. The question is whether the demonstration duty is triggered, not whether the provider complies overall.

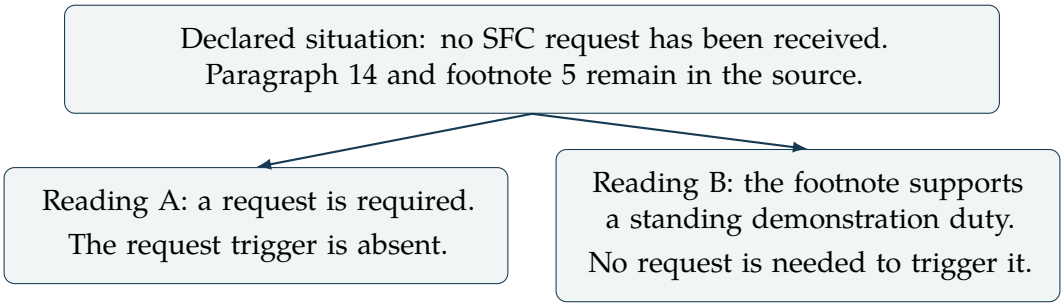


Figure 6.28: The same declared fact has different consequences under rival readings. The diagram isolates their difference; it does not decide which reading the source warrants.

A demonstration of this difference is useful even while its legal resolution remains open. It tells the implementation exactly which choice must not be hidden in a default. Pro-

ducing a controlled-English sentence and executable expression from the same record can then prevent them from drifting apart, while a separate source challenge continues to examine the record itself.

6.40.6 Evidence has to establish the right predicate

A precise rule is still conditional on its facts. Holzenberger and Van Durme connect legal information extraction with symbolic statutory reasoning by representing events and their arguments through source-linked spans. The representation ties a value to the passage from which it was obtained, allowing downstream reasoning to expose the consequences of extraction errors. The study also shows why a missing event argument can matter to the resulting legal calculation (Holzenberger and Durme, 2023, secs. 3–5 and apps. B, C and E).

That method gives us a useful evidence structure, with limits that should shape its reuse. The study uses a tax-specific ontology and curated formal statutes. Its date-completion conventions and closed-world treatment of exceptions cannot silently become rules for incomplete regulatory evidence. The associated implementation was inspected as source, not executed here; its repository reuse licence was not established. The immediate proposal is to implement the published span-linked idea within our own typed evidence records.

For a judgment, the record should also say whether a passage reports an allegation, a party's submission, a factual finding or judicial reasoning. For an operational record, it should identify the provider, arrangement component, event and time. A correctly quoted supervision log concerning Provider A does not establish supervision of Provider B. A record entered after the assessment cutoff was not available to the earlier assessment, even if it describes an earlier event.

Positive and negative support must be kept separately. There may be support for a proposition, support against it, both, or neither. These are states of the declared evidence, not guarantees that its contents are true. A complete record of inspections might support an inference that no inspection occurred during a specified interval; an incomplete folder cannot support the same inference merely because no inspection is found. The completeness premise belongs in the argument.

Legal classifications require a further warrant. A blockchain record may establish recorded token possession under its stated identity assumptions. The conclusion that this proves legal title additionally needs the relevant legal and contractual rules. Likewise, a recorded supervisory act need not establish adequate supervision. The product should expose this distinction between an observation it can derive and an evaluative standard it has not established.

6.40.7 A precise account of disagreement is an executable result

The first implementation should evaluate the consequences of each retained reading under explicitly declared facts. Let $\mathcal{R}$ be the nonempty finite set of retained reading

branches, including their semantic settings and assumptions. Let x be the scoped factual record. For each branch r, let $d_r(x)$ return its declared duty outcome, retaining unknown, conflicting or unsupported outcomes where appropriate. The set

$$\mathcal{D}_{\mathcal{R}}(x) = \{\, d_r(x) : r \in \mathcal{R} \,\} \tag{6.14}$$

is a local definition of the outcomes reported across those branches. The full report also retains each branch's outcome and premises; the set removes duplicate values only for this comparison.

If every branch is evaluated and this set consists of exactly one settled duty outcome, the evaluated branches agree on that outcome. A settled result alongside an unknown result is not agreement. If it contains two distinct settled outcomes, they disagree. If a branch remains unsupported, its status must remain visible rather than being dropped to obtain agreement. Equation (6.14) therefore organizes a conditional report; it does not show that $\mathcal{R}$ contains every legally possible reading or that a common interpretation is correct. A resource limit or an unvisited alternative prevents a claim of completed evaluation.

For the no-request illustration, the two branches report different trigger statuses. The product can establish that difference relative to the specified rules while leaving the source question unresolved. With incomplete evidence about whether a request was made, it must preserve a further factual uncertainty. Keeping the two uncertainties separate tells the next investigation whether it needs a record of events, an applicable authority, or a repair to the representation.

Existing RuleIR, Catala and formal checking mechanisms can help verify these declared consequences. They cannot supply the missing relationship between the original English and the proposed formal specification. That relationship must remain explicit even when multiple executable targets agree.

6.40.8 The first repair program

The proposed program begins with the two clause groups, preserving the current source editions and every original obligation. It first corrects demonstrably stale operative wording. It then builds the typed representation, constructs the rival clause readings and evaluates their arguments under explicit semantics. Precedent analysis adds opinion-aware propositions and factor comparisons; evidence derivation adds actor, activity and time to the facts the program actually uses.

The next stage joins explanation and execution inside the complete investigator. Unsupported readings remain in its accounting. Independent finite checks and narrowly stated formal propositions test preservation of branches, evidence states and dependencies. Only then does the program extend coverage and freeze a method for observations on genuinely later sources. Each phase rewrites the next phase's plan in light of its actual results. A counterexample normally identifies the next repair; corrupted evidence or an invalid checker stops the affected execution.

This is proposed product work. No phase of that implementation program was executed for this literature and document review. The practical dependency choices are also bounded: PyArg, Logical English, Carneades and eyecite have identified open-source licences; SARA-IE's code reuse licence and CLERC's repository-wide coverage remain unverified. The retained source snapshots and licence findings accompany the detailed program.[2]

The resulting product should explain why a conclusion follows under a stated reading and what would change it. Under the controlling research requirement, human answer keys, reviewer approval and agreement among language models do not qualify legal correctness. Published legal texts remain premises, and an independently checked formal proposition establishes only its stated domain. Where the source, applicability or evidence remains unsettled, the appropriate result is a specific qualification accompanied by the consequences that can already be established.

[2] See `docs/plans/legal-interpretation-master-program.md` and the individual plans under `docs/plans/legal-interpretation-program/`. The source review and pinned versions are in `docs/research/legal-interpretation-reuse-2026-10-05.md`.

Chapter 7

From an accepted interpretation to verified Java behavior

Once a reviewer adopts an interpretation, the problem changes. The bank now needs to know whether the program behaves like that interpretation when a client is exactly at a threshold, when evidence is missing, and when a withdrawal or amendment arrives later. A successful compilation answers none of those questions.

The detailed semantics here belong to the RuleIR profile and its Java runtime. The Catala compatibility backend preserves that profile. The richer shared translator has additional operations and an explicitly different partial-input policy, introduced in Section 8.2.

7.1 The specification becomes the comparison target

The accepted rule is more than a Boolean formula. It includes the source versions, activity profile, factual definitions, subject identity, time policy, unknown and conflict semantics, exception structure, result interpretation and coverage limits. If any of these is omitted, two implementations can agree on a formula while giving different advice in practice. A Java adapter that labels all rebates as discounts has changed the rule's application even if its Boolean operator code is flawless.

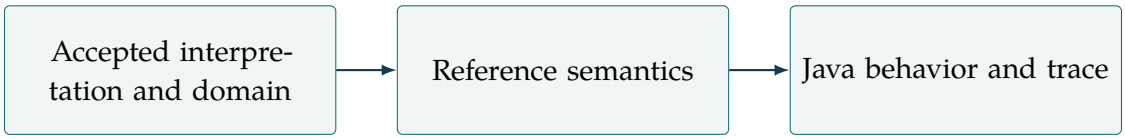


Figure 7.1: Verification starts from the complete accepted specification.

The comparison therefore fixes the complete rule, its version and the cases it is intended to cover. Two programs must agree on the decision, missing facts, errors, source references and explanation. They may have different program identifiers because they are separate implementations. Those identifiers help locate what was run; they are not the meaning that the comparison asks them to preserve. Each program's record must still be checked against its own version.

The existing workbench already provides a finite RuleIR interpreter, a Java runtime, generated policy classes and a release workflow. The investigation in Chapter 6 now supplies explicit draft candidates to that delivery path. Source checks and composition

preserve their open questions; review remains necessary before release. A proposal must fit the declared language rather than silently expand it whenever a model suggests a new legal construct. Unsupported meaning remains an explicit engineering and review issue.

7.2 Writing a rule precisely enough to execute

The approved sentence must retain the same meaning when a program reads it. For the financial route, "at least" includes equality and "or" allows either wealth measure to suffice. The intermediate representation, RuleIR, records these choices in a fixed vocabulary. It connects the reviewer's sentence to the calculation, rather than asking each implementation team to interpret the words again. Its rule expressions have no circular dependencies; histories are handled by the separate event model explained later in this chapter.

Each rule needs more than its condition. A reviewer must be able to find the source edition and supporting passage, and know which actor, activity, product, client class and time the rule addresses. The record also states whether its meaning has been accepted or remains under review. It must distinguish a derived fact from a duty, permission or prohibition. A duty also needs an actor, trigger, action, timing rule and evidence by which performance can be assessed. A missing source deadline remains missing; operational escalation times belong to a separately identified bank policy.

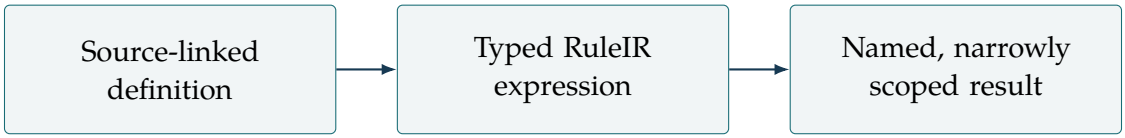


Figure 7.2: An executable rule keeps its source and interpretation context.

Definitions should be reusable and versioned. A field for portfolio value must refer to the adopted portfolio definition, relevant ownership rules, valuation date and currency conversion convention. The application mapping records where each fact comes from, its freshness and who can resolve a conflict. A rule whose necessary definition remains unresolved can be reviewed and discussed, but it cannot be released as an executable control for that use.

In ordinary language the first financial comparison says, "The client's qualifying portfolio is at least HK$40 million." Its executable form records three things: the portfolio fact, the threshold and the inclusive comparison. The second comparison asks whether qualifying net assets excluding the primary residence reach HK$80 million. The two comparisons are joined by "or."

Amounts are represented in cents, so the first threshold becomes 4,000,000,000 cents and the second 8,000,000,000 cents. This conversion changes the unit, not the wealth requirement. A supplied portfolio must already reflect the adopted ownership, valuation and currency definitions. Calling an account balance "portfolio" does not perform

those adjustments.

The result concerns this financial condition alone. It does not establish all SPI qualifications or authorize a trade. The exact serialized expression is retained in Appendix F of the technical companion; its field names add no further legal meaning to the comparison taught here.

7.2.1 Preventing meaningless calculations

A statement that is true or false, a transaction count, a money amount and a calendar date are different kinds of input. The program rejects combinations that have no defined meaning: adding a date to a portfolio, for example. Integer and money inputs retain exact values, with money measured in HK cents. Missing information remains missing; it is not silently converted into zero. Dates must be valid, including on leap days. Foreign-currency normalization records the rate, valuation instant and rounding policy before producing an HKD input. A missing exchange rate therefore leaves an unknown amount.

The expression grammar contains typed literals, fact and rule references, `all`, `any`, `not`, equality/inclusive/strict comparison, addition, subtraction, rational scaling, conditional choice and explicit defaults. Addition and subtraction require matching integer or money types. Scaling uses a nonnegative integer numerator and a positive denominator; a known result must divide exactly. A half-cent allocation returns `E_INEXACT_SCALE` until an external reviewed rounding policy resolves it. Date arithmetic is kept in the separate scheduler so ordinary library behavior cannot choose a legal anniversary convention unnoticed.

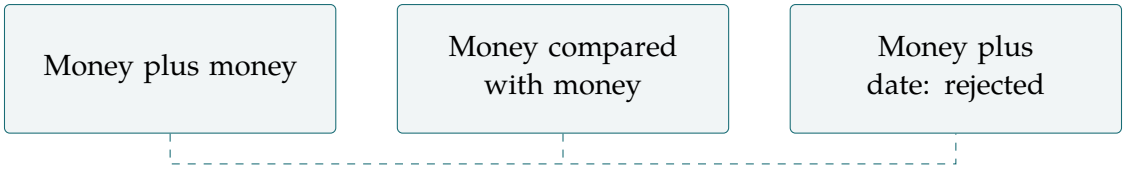


Figure 7.3: The grammar permits only operators with defined type behavior.

The permitted operations form a small vocabulary: take a fact or a fixed value; combine conditions with “all,” “any” or “not”; compare compatible values; calculate an exact sum, difference or proportion; or select between explicit alternatives and exceptions. Each expression has an identity so that a reviewer can locate its place in an explanation. A rule also records when it applies, what it calculates, and which interpretation and source passages justify it. The exact record grammar is in Appendix F of the technical companion.

A comparison requires two matching integer, money or date operands. Dates compare in calendar order; Boolean comparisons are not admitted in this version. A conditional requires a Boolean condition and matching branch types. Default values match their base type, and all guards are Boolean. An empty exception list simply selects the base.

There are no loops, quantifiers, dynamic names or arbitrary function calls. More elaborate assessment and aggregation enter through separately reviewed fact mappings.

Every node has a unique identifier. References must resolve, types must agree and the complete dependency graph must be acyclic. A rule reference may refer only to an unconditional definition; a scoped result cannot quietly become a Boolean false in another expression. Authors can expose the scope explicitly as a named Boolean definition. Unknown operators and additional JSON properties are rejected. This keeps a model from extending the language merely by emitting a plausible field name.

7.3 Several duties, several answers

Return to the settlement example in Section 2.15.1. Establishing that staff are available does not establish that funding is ready. Both questions may need answers. By contrast, two competing definitions of staff availability are alternatives for one question. Putting every proposed formula into one large "and" would confuse these two relationships.

A smaller illustration makes the distinction visible. Suppose a control package contains the gift restriction and a separate, explicitly specified disclosure duty. This illustration supplies the disclosure duty as a second control; it does not infer a new duty from the gift paragraph. One reading of the restriction recognizes a particular rebate as a discount, while another does not. The package can pair either reading with the disclosure duty. It must keep those two possible packages separate until the rebate question is decided.

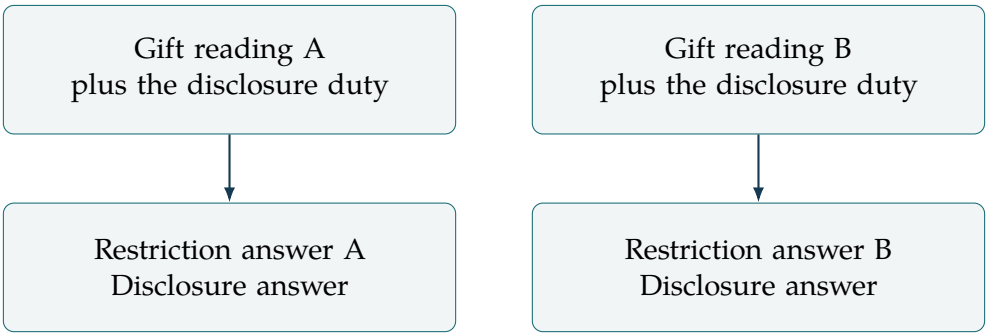


Figure 7.4: Each proposed package contains both duties. The rival gift readings remain in separate packages, and each duty keeps its own answer.

The result is an ordered collection of answers, often called a vector. One answer can concern a restriction and another whether an action is required. A true restriction is not a true permission. Each component therefore carries its output meaning, subject, scope, exceptions and dates. It also retains the questions that remain unresolved. The institution can then specify how these answers govern an offer; composition alone does not supply that policy.

Facts need the same care. A short name such as "eligible" may refer to the recipient of a gift in one control and the audience for a disclosure in another. The implementation

keeps the names separate unless an explicit mapping says that their meanings correspond. Missing facts in one component remain unknown there; a value from another component cannot fill the gap merely because its name looks familiar.

The local composition exercise built and ran two controlled packages with two components each, as well as the five retained alternatives for the gift control. This establishes that separately scoped answers can survive Java translation in those cases. The assignment of passages and candidates to components remains a proposed interpretation. Neither the gift circular nor the settlement circular has acquired complete coverage simply because its selected components compile.

7.4 Facts are evidence-bearing values

A production fact is not simply a field such as `gift=true`. It has a type, value or uncertainty status, evidence references, validity interval and recording time. The validity interval says when the evidence applies to the underlying situation. The recording time says when the system knew it. These times answer different questions and both are needed for historical reconstruction.

Suppose the bank learns on 10 October that a fee record effective on 1 October was incorrect. A decision replay asking what the system knew on 5 October should not use the later correction. A corrected assessment made on 10 October may use it to reassess the earlier event. The two assessments should coexist. Replacing the old record would erase the evidence needed to explain the original decision.

The existing normalization boundary selects records valid at `valid_at` and recorded no later than `known_at`. A correction names the evidence it supersedes. Two admissible, non-superseded records with different values create a conflict. Arrival order is not a source-priority rule. If both records agree, their evidence may be combined without inventing a conflict.

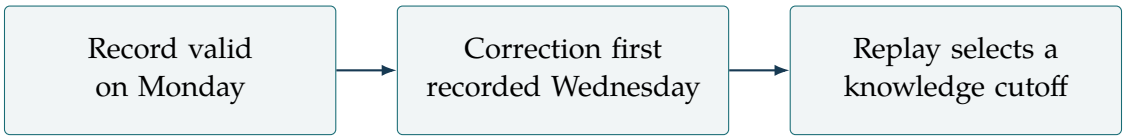


Figure 7.5: A fact has both a validity period and a recording history.

Intervals are half-open: $[a, b)$ includes the start and excludes the end. If a record is valid until midnight, it is not valid at that midnight. This convention avoids overlapping adjacent intervals when one record ends exactly as another begins. Tests must exercise equality at both endpoints, not merely dates well inside an interval. All assessment timestamps use the canonical UTC format with six fractional digits; legal calendar dates are separate values.

Unknown facts have reasons. Missing evidence, stale evidence, an unreviewed classification and unresolved event order can all lead to unknown, but they call for different

remedies. A host should not display the same uninformative message for every case. Conflict similarly preserves the competing evidence identifiers. The evaluator does not decide which source is legally superior without a reviewed normalization rule.

This design puts factual interpretation where it belongs: at a visible boundary. For the circular, classifying a benefit as a gift or fee discount is an evidenced assessment. The decision engine consumes that assessment under its approved definition. If the ensemble has not resolved the definition, the adapter cannot supply a known value merely to satisfy a required input field.

7.4.1 Evidence is an input to computation, with its own history

Use one synthetic correction to see why a fact carries its own history. A portfolio record says HK$60m at valid time 1 October and is recorded on 2 October. A second record, recorded on 10 October, says the value was HK$55m for the same valid interval and explicitly supersedes the first. The first assessment may use the record only when its `known_at` cut-off reaches 2 October; a later corrected assessment may use the second. Both assessments retain their evidence identities.

The same case distinguishes the three information states. If no eligible record exists, the value is unknown because evidence is missing or stale. If two admissible, non-superseded records disagree, the value is conflicting and both records remain visible. Matching records can combine their evidence. Arrival order is not legal priority. Every request still supplies `valid_at` and `known_at`, and the half-open interval convention makes an amount ending at midnight unavailable at that endpoint.

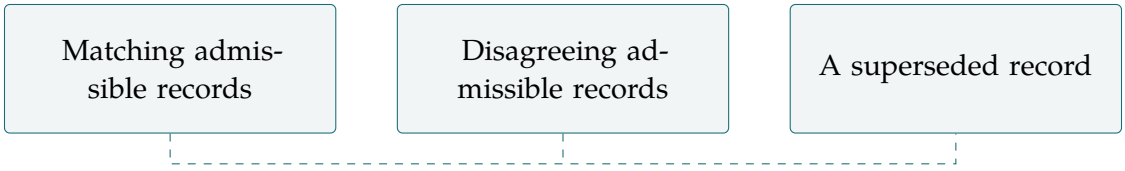


Figure 7.6: Evidence normalization determines which values reach the evaluator.

Before Boolean evaluation, the engine checks conflicts in the static transitive dependency closure, including branches that might later be skipped. A conflicting portfolio amount therefore returns `CONFLICT` even if the net-asset route could establish the financial condition. An unrelated conflict does not block that rule. This conservative policy is a proposed engineering choice; it is deliberately different from ordinary unknown-value propagation.

7.5 Exact arithmetic prevents a second kind of ambiguity

The existing scalar types are Boolean, integer, Hong Kong dollar money in cents, and Gregorian date. Integer and money values use canonical decimal strings and arbitrary-

precision arithmetic. A monetary value of HK$100 is represented as `"10000"`. The string representation avoids a transport layer converting the value through a floating-point number before the exact arithmetic begins.

Units are part of the contract. A number expressed in dollars and the same number expressed in cents differ by a factor of one hundred. A compiler cannot infer which was intended from the digits alone. The host mapping must identify the source unit, conversion and evidence. Declaration-specific bounds can reject implausible inputs, but a plausibility check is not a substitute for a reviewed unit mapping.

Consider proportional ownership of a monetary amount. If the accepted operation is exact scaling by a rational fraction n/d, the runtime computes the product and quotient using integers. Let x be cents. It checks

$$nx = dq + r, \qquad 0 \le r < d \tag{7.1}$$

for nonnegative nx; for signed values the implementation's quotient and remainder convention must still make zero remainder the exactness test. A nonzero remainder means that the result is not an integer number of cents. The current contract returns `E_INEXACT_SCALE`; it does not choose a rounding policy on the user's behalf.

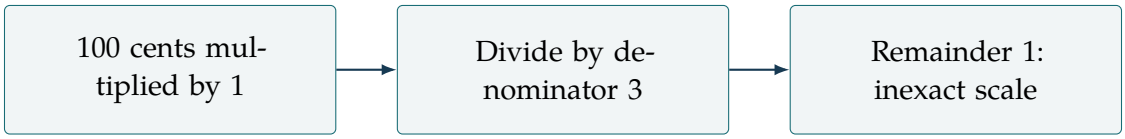


Figure 7.7: Exact money arithmetic exposes a required rounding decision.

The quotient is 33 cents with one cent left in the numerator calculation: 100 = 3 times 33 + 1. Returning 33 cents would introduce a rounding choice. The present exact-scaling operation reports an error until a reviewed policy supplies that choice outside this operation.

If a bank requires rounding, that is a substantive extension. The rule must state which quantity is rounded, at what stage, in which direction and under whose policy. Rounding each component before summation can differ from rounding the sum. Two correct implementations of different policies can therefore disagree. The ensemble should recognize this as a specification issue rather than treating one result as a numerical accident.

Foreign-currency conversion is outside RuleIR 0.1. A reviewed adapter supplies an HKD amount with rate, valuation instant and rounding evidence. A missing rate creates unknown evidence. It must not trigger a guessed conversion or reuse an unboundedly old rate. The bank's data owner is responsible for the freshness and valuation policy, while engineering verifies that the adapter implements it.

Dates require similar precision. A date is not a timestamp, and an obligation measured in business days needs a calendar policy. The RuleIR expression language compares validated dates but leaves date arithmetic to reviewed event scheduling. This restricted design prevents a generic arithmetic library from silently deciding whether a public hol-

iday counts toward a regulatory deadline. The appropriate calendar must be versioned with the assessment.

The shared model's version-2 translator now supplies separately declared rounding and calendar operations. It makes those choices executable in Catala; it does not remove the need to justify the currency or calendar convention.

7.6 Unknown, conflict and scope have an order

The three-valued Boolean rules in Chapter 3 govern unknown values, but they are not the entire runtime semantics. Before evaluating the requested rule, the existing implementation checks conflicting evidence in its static transitive dependency closure. That closure includes scope, body, exceptions and branches that might later be skipped. A conflict in the closure returns conflict even if another route could determine the Boolean result.

This is a conservative evidence policy. For example, an actor may appear outside the selected scope, but a conflicting fact in the rule's static closure still produces conflict before scope evaluation. The second-circular case that exercises this behavior is deliberately different from ordinary short-circuit intuition. The specification must teach it because an engineer using conventional Java `&&` or `||` could otherwise implement a different result.

After conflict checking, scope is evaluated. False scope returns `OUT_OF_SCOPE`; unknown scope returns `UNKNOWN` with a scope reason; true scope permits evaluation of the body. A body that would be false does not turn unknown scope into a known result. The system is answering a scoped condition, and the applicability question remains unresolved.

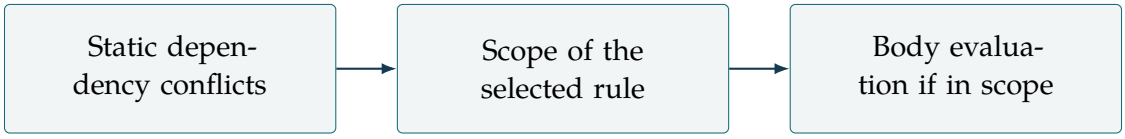


Figure 7.8: Evaluation order is part of the specified behavior.

Within the body, conjunction and disjunction evaluate all operands in declared order. This eager behavior retains complete evaluated evidence and exposes errors or conflicts even if an earlier operand would determine an ordinary Boolean answer. Missing inputs can appear in a known result, while blocking inputs are empty when the unknowns do not prevent that result. The distinction is useful: the bank may know the condition is false while still needing to repair a stale input feed.

A conditional expression behaves differently. It evaluates the condition and only the selected branch. An unknown condition returns unknown without evaluating either branch, even when the branches appear equal. Skipped branches remain type checked and part of the static dependency closure. Their trace roots are marked skipped so that the explanation does not invent an evaluation that never occurred.

Defaults add another layer. All exception guards are evaluated first. Two true guards produce conflict even if their values would agree. An unknown guard can prevent selection of another true guard because the unknown might become a competing exception. One true guard with all others false selects its value; all false guards select the base. Priority is expressed only by explicit nesting with a reviewed interpretation. Array order is not hidden legal priority.

These rules are not universal truths about law or logic. They are the existing workbench's specified execution choices. The ensemble may identify a regulation that needs different semantics. The correct response is to propose and review an extension or keep the case outside the supported fragment, not to reinterpret the current operators ad hoc.

7.6.1 Defaults, branch selection and failure behavior

The conformance fixtures should force the evaluator to reveal which rule it uses. Start with a false base and two exceptions: one true guard with all other guards false selects its value; all false guards select the base; two true guards return conflict even when their values agree; and one true guard plus an unknown competitor returns unknown. The guard carries its interpretation identifier and source spans, so the trace can explain why the result was selected or blocked. Priority enters only through reviewed nesting; row order cannot supply it.

The fixtures also separate eager combinations from selective conditionals. An `all` or `any` expression evaluates its operands in source order, retaining errors, conflicts and evidence even when an ordinary Boolean shortcut would already determine the answer. A conditional evaluates only its selected branch; an unknown condition evaluates neither branch and returns unknown. The unselected branch remains type checked and appears as skipped in the trace. These cases are more useful than another rule recital because each one distinguishes a possible Java implementation from the specified behavior.

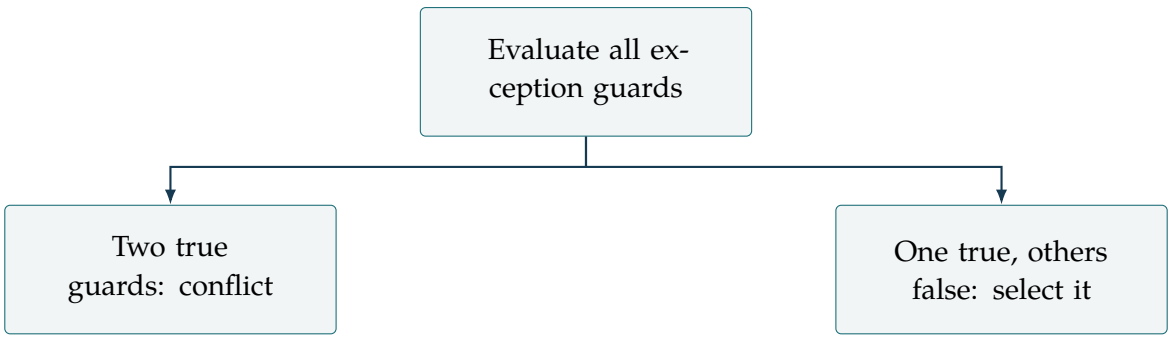


Figure 7.9: Array order does not choose a legal priority.

Even when two matching exceptions propose the same value, the current contract reports a conflict. The result records that the rule's exception selection is unresolved. Silently returning the shared value would hide that specified distinction.

The rule's scope is evaluated separately. A false scope yields `OUT_OF_SCOPE`; an

unknown scope yields `UNKNOWN`. In scope, Boolean bodies return `TRUE` or `FALSE`, other known bodies return `VALUE`, and missing evidence returns `UNKNOWN`. Conflicts and errors are distinct tagged results. An invalid version time is an error, not a negative assessment. No exception handler may fall back to false.

The response carries all encountered missing inputs and the conservative subset blocking the answer. If a known net-asset route establishes the alternative, the unknown portfolio remains visible but is not a blocker. For an unknown result, the union of encountered unknown dependencies is reported; the prototype does not claim it is a mathematically minimal explanation. Traces record evaluated nodes in postorder, evidence, source spans and explicitly skipped branch roots. Explanations and diagrams are rendered from that trace.

7.7 Validation failures are part of observable behavior

A verifier must compare failures as well as successful decisions. Suppose one request gives two different portfolio values under the same field name. Choosing the last value would make the answer depend on how the message was arranged. The request must instead be rejected. A different case supplies a readable request but asks a rule to add money to a date. Here the system can identify the invalid operation within the rule. The first failure concerns reading the request; the second concerns the meaning of what was read.

The implementation distinguishes malformed JSON, duplicate keys, incorrectly written numbers and invalid Unicode from a well-formed request containing an invalid rule or fact record. The former can fail before a semantic result is available; the latter can return a structured error identifying the problem.

The current validation order is schema, references and duplicates, types, cycles, time, static-closure conflicts and evaluation. A deterministic order makes error reports reproducible when an input has several defects. It also prevents a runtime from accessing an invalid reference before discovering that the bundle should have been rejected. Fixed error codes and pointers are compared across backends; dependency-library exception text is not the public contract.

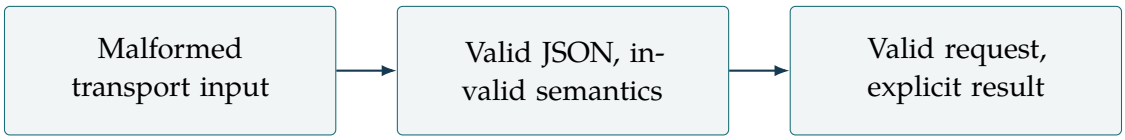


Figure 7.10: Failure at different boundaries produces different evidence.

Resource limits are explicit. The present canonical JSON boundary is bounded in size and depth, and the rule graph has node and expanded-call-depth limits. An oversized acyclic expression is a resource error; a cycle is a cycle error. An implementation must not catch either and continue with a truncated rule. The same principle applies to the new ensemble's graph and report limits.

Tests should combine defects when precedence matters. A snapshot can contain a conflict and an expired validity interval. A bundle can contain a type error in a branch that would otherwise be skipped. A default can have two applicable guards whose values contain errors that should never be evaluated. Isolated operator tests cannot establish the behavior of these combinations.

A useful specification test states both the expected result and why a tempting implementation would differ. For example, a short-circuit implementation might return false before encountering an error in the second operand. The accepted eager semantics requires the error. This counterexample explains the required behavior and makes the test more durable than an assertion copied from the current function's output.

7.8 Independent oracles and their limits

An oracle is a source of expected behavior against which an implementation is compared. Independence has several meanings. A separately written evaluator can be independent of the production code while relying on the same mistaken interpretation. A human reference can be independent of the model's answer while sharing the same incomplete source packet. The verification report must specify which independence it has actually achieved.

The retained 22-case oracle was frozen before the candidate bundle, which protects against some forms of adapting expected outputs to the implementation. It was nevertheless prepared by the same author. The 729-case enumeration gives complete coverage of six ternary abstract inputs for the selected formula. Neither fact establishes independent legal interpretation or whole-document coverage.

A stronger reference process begins with independent source reading and scenario judgment. Reviewers state the expected result, assumptions, supporting passage and unresolved questions before seeing the candidate output. Disagreements are adjudicated with reasons or retained as alternative acceptable outcomes. The reference then has a defined relationship to the evidence rather than being a single unexplained label.

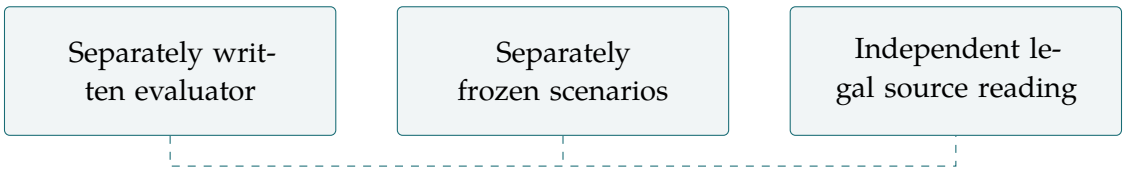


Figure 7.11: Oracle independence must identify what was independently established.

For engineering checks, a small executable oracle can be valuable when its method is genuinely different. The second-circular body has a read-once unate structure, which permits its Boolean-completion check to agree with the specified three-valued result. The general expression $p \vee \neg p$ shows why that argument cannot be extended to every RuleIR expression: under the specified semantics, unknown p yields unknown, while every classical completion yields true. An oracle based on the wrong generalization

would reject correct implementations or bless incorrect ones.

The oracle itself therefore needs a stated domain and a small set of discriminating examples. The workbench should retain oracle author, method, source versions, review status and any shared components. A generated oracle that imports the same evaluator under a different function name supplies little independent evidence. A reference that is changed during repair remains useful only when the change is separately justified and versioned.

7.8.1 The small financial fixture within the complete dry run

Section 2.2 follows the complete selected transaction profile. The smaller conformance fixture imports 23EC35 and Annex 1, verifies their hashes and opens paragraph 3.1 beside the financial rule. Its two wealth definitions are review issues until the relevant source and data mappings are accepted. An engineer loads the supplied draft bundle and the synthetic `f01` case: portfolio HK$40 million and qualifying net assets zero. The trace evaluates both inclusive comparisons and their alternative, returning `TRUE` for `spi.financial`. It also identifies the exact source revision, input snapshot and engine version.

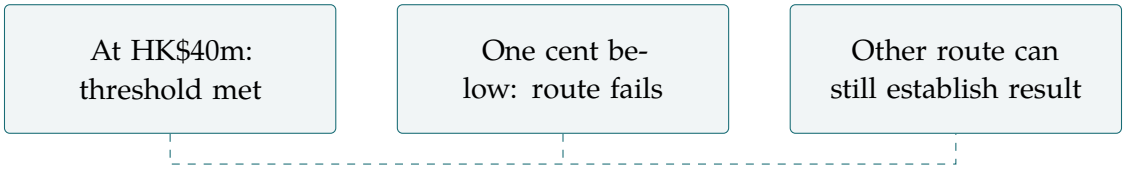


Figure 7.12: Boundary cases reveal the intended comparison operator.

The reviewer next opens `f02`, one cent below the portfolio threshold, with the other route still false. This returns `FALSE`. In `f07`, the portfolio is unknown and net assets are HK$70 million, so the answer is `UNKNOWN`. In `f06`, changing only net assets to HK$90 million establishes the financial condition despite the still-missing portfolio. These cases let a compliance reader inspect the interpretation before any system rule is exported. A proposed strict comparison is challenged with `f01`.

Once dependencies and meanings are reviewed, the evidence report is attached to the bundle hash, reviewers approve it and the release service exports the package. The production change package includes deterministic Java source and its tested JAR against the same decision contract. A host adapter supplies the bank's facts and enforces the released-version and concurrency requirements. A changed rule creates another hash and invalidates release approval. Historic decisions still point to the old hash and their original evidence.

7.9 Mutation tests measure what the checks can notice

A mutation deliberately changes a candidate or implementation in a way that represents a plausible error. If the verification process rejects it for the intended reason, the test demonstrates sensitivity to that error. It does not estimate the frequency of that error in real use, and a high mutation score does not prove complete correctness.

The second-circular exercise used four meaningful mutations: remove the discount exception, omit product-type linkage, assume gift classification and ignore scope. The retained named cases detect all four after compilation. This is more informative than mutating arbitrary characters because each change corresponds to a failure a reviewer or programmer could plausibly make.

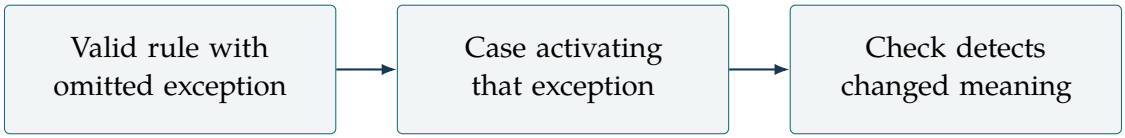


Figure 7.13: A semantic mutation should remain executable enough to test meaning.

The ensemble extension should add source and process mutations. Remove a footnote from one extraction; supply a stale dependency to one member; label a speculative classification as established; suppress a minority candidate in the report; reset an issue counter after a revision; or reuse an approval after a source change. Each mutation targets a distinct assurance boundary. Passing only formula mutations would leave the new controller largely untested.

Some mutations are equivalent under the selected domain. Changing an expression may not change any supported outcome, and failure to detect that mutation is not necessarily a defect. The mutation report must distinguish equivalent mutations, invalid encodings and live semantic defects. Counting all undetected mutations as failures, or silently excluding inconvenient ones, would distort the result.

A mutation is killed when a required property changes and the check catches it. A compiler crash is not always the intended detection: if the task is to test whether the oracle notices an omitted exception, a syntax-invalid mutant does not answer the question. Mutation construction should preserve validity where the research question concerns semantic sensitivity. Invalid-input tests remain a separate and necessary category.

7.9.1 Targeted tests and counterexamples

The initial suite should include exact threshold boundaries, small amounts below them, and cases in which only one financial route holds. It should distinguish unknown from zero, inconsistent from missing evidence, an absent agreement from an unretrieved agreement, and client attributes from representative attributes. These cases test meaning that common “typical client” examples fail to expose.

Category tests should change only the relevant category or qualification route. Ownership tests should change only the allocation agreement or associate status. Temporal tests should change only the withdrawal sequence, review date or fund movement. These paired cases help reviewers identify the controlling condition. For facts designated irrelevant to a narrow rule, changing only those facts should leave its result unchanged. Such metamorphic checks must name their domain: adding wealth might be monotone for Equation (3.8), but does not establish monotonicity of every suitability or product-distribution decision.

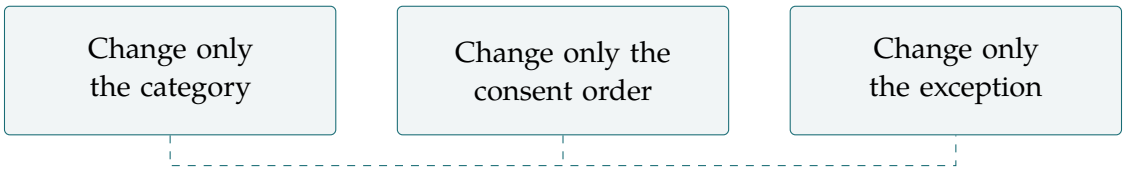


Figure 7.14: Paired scenarios expose the condition responsible for a change.

Mutation tests should deliberately replace inclusive with strict comparisons, alternatives with conjunctions, category-specific predicates with global ones, conditional requests with unconditional duties, and named exceptions with generic fallbacks. Event tests should include duplicated, delayed and out-of-order records. The test report must identify which defect each case would catch. This borrows the defect-oriented lesson of CUTECat without claiming that automated case generation establishes source completeness (Goutagny et al., 2025).

7.10 Symbolic comparison requires a feasible domain

For supported expressions, symbolic comparison can search for inputs on which two candidates differ. The domain records admissible values, knownness conditions and relationships among facts. Before asking whether the outputs differ, the solver must establish that the domain has at least one model. Otherwise an inconsistent domain makes every equivalence query vacuously true.

Suppose one domain constraint says that a payment exceeds the fee and another says that it cannot exceed the same fee. A query finding no differing outcome under both constraints has taught us nothing about the real candidates. The workbench returns an inconsistent-domain result. The domain must be repaired or reviewed before any equivalence claim is made.

A satisfiable difference query returns a witness. The witness must be decoded to a concrete fact snapshot and replayed in both the reference evaluator and Java. This replay checks that the symbolic encoding corresponds to executable semantics. A solver result alone can be wrong relative to the claimed runtime if the encoding mishandles unknown values, scope or arithmetic. The witness and both results belong in the report.

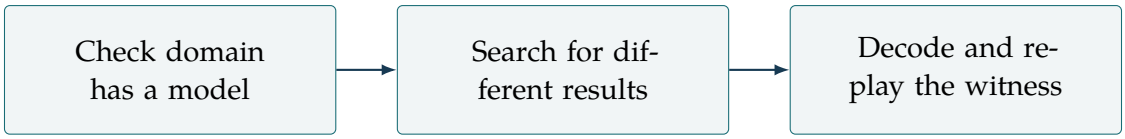


Figure 7.15: An inconsistent domain cannot support a useful equivalence conclusion.

A domain requiring an amount to exceed 100 and also be at most 100 has no admissible input. Finding no difference under those constraints says nothing about the two rules. The domain check must detect the contradiction before interpreting the comparison.

An unsatisfiable difference query establishes no differing result within the checked fragment and domain, subject to the solver and encoding assumptions. It does not establish that the candidates use the same legal definition, cover the same source material or behave identically outside the domain. The result label must therefore include the domain and comparison projection.

The current comparison fragment supports documented Boolean and exact integer/money reasoning with explicit knownness. It rejects unsupported date, scale, default and event constructs. The two solver checks share a bounded time budget, with a separate worker deadline. Unknown, timeout, missing Java replay or unsupported constructs never become equivalence. The ensemble may schedule another permitted check, but cannot relabel the incomplete result as a pass.

Counterexample-guided refinement is useful when a witness exposes an imprecise abstraction. The engineer identifies why the witness is infeasible or why the encoding omitted a required condition, changes the abstraction with a documented reason, and reruns the query. The discipline matters: excluding a witness because it is inconvenient is not refinement. The revised domain must correspond to a reviewed fact about the application.

7.11 Property tests should follow the semantics

Property-based tests generate many inputs and check a relation that should hold. They can explore combinations that named examples miss, but the property must be valid for the specified semantics. Classical Boolean identities cannot all be transferred to unknown or conflict behavior. Likewise, reordering operands may preserve a value while changing the required trace or error precedence.

Useful properties include deterministic evaluation for identical canonical requests, rejection of undeclared facts where the schema requires it, exact integer arithmetic, preservation of known values under irrelevant evidence additions, and stable source references under deterministic rendering. Each property needs a precise precondition. Adding a conflicting record in the static dependency closure is not an irrelevant addition.

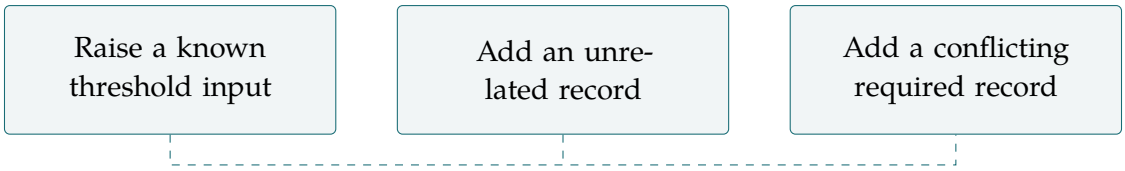


Figure 7.16: A property test needs a domain in which the proposed relation is valid.

Metamorphic tests compare related inputs when a direct oracle is difficult. If a known amount increases while all other facts remain fixed, a simple lower-bound threshold condition should not change from true to false. This monotonicity applies only to the stated threshold fragment. It need not hold for a default with an upper-bound exception or for a scope condition that changes at the same time. The test generator should derive its applicable domain from the property, not assume the property is universal.

For evidence time, a carefully limited relation is useful: replaying an earlier `known_at` should exclude records first recorded later. Increasing `known_at` does not necessarily preserve the decision, because a correction or conflict may become visible. A test that demands decision monotonicity in knowledge time would impose a false property. The intended invariant concerns which records are admissible, not whether later knowledge always makes the same answer more certain.

Fuzzing transport boundaries is a different activity. It tests duplicate keys, size limits, malformed timestamps, unusual Unicode and nesting limits. Its success criterion is controlled rejection or valid deterministic behavior, not legal interpretation. These tests protect the integrity of the evidence and execution chain and should be included in the engineering report under that role.

7.12 Following the accepted rule into Java

For a transaction, the adopted rule has to work without asking a language model what it means again. Otherwise the answer could change between review and use. The workbench therefore prepares a fixed Java library that the bank can call locally. Python can help prepare sources and compare results during development; it is not needed when the released Java library evaluates a transaction. Neither is a network connection to a solver or language model.

This division makes two questions testable. First, has the intended version of the rule been included in the library? Second, does that library calculate the same answer as the reference evaluator? A versioned release record answers the first question by identifying the exact rule and program. Repeated comparison on the same facts and histories addresses the second. Using the same displayed title, such as “SPI control,” is too weak: two files with that title can contain different rules.

There are two retained demonstrations of the Java route. The earlier *SPI-Demo1* covered comparisons, alternatives and consent histories within a deliberately small set of supported operations. The later implementation covers the decision operators and finite duty histories specified in this chapter. Their interfaces and test sets differ. The ear-

lier example has 32 executed decision scenarios and eleven consent histories; the general rule implementation subsequently exercised the 35 original rule-contract fixtures. Those numbers locate the evidence. They do not make the narrower demonstration an implementation of every later feature. Appendix F of the technical companion retains the reproducible commands, interfaces and results for each version.

The current generated policy class contains the fixed rule bundle and calls a shared Java evaluator. Thus the release includes both the rule and the evaluator version that gives its expressions meaning. Generating a class is not itself a proof that those meanings are preserved. The conformance, counterexample and mutation checks already introduced must be applied to that exact release.

7.13 What the bank must supply at the point of use

Return to Ms Lee's proposed order. The bank first selects a release approved for her legal entity, activity and assessment date. It then gathers a consistent set of client, consent and exposure records. The request identifies both the date being assessed and the information available at the chosen knowledge cutoff. These are the two times introduced earlier; they allow the same question to be repeated later without silently adding information that arrived afterwards.

The Java library evaluates those records and returns a decision with its explanation. The bank then decides how that answer fits the wider transaction. For the marketing example, a selected prohibition being true may require the relevant incentive to stop. A false result only allows the bank to continue its other checks. Missing evidence, conflicting records, a case outside the supported scope and a processing error each require their own operational response. Treating all nonempty result strings as "yes" would erase those distinctions.

Calling a library also does not grant release authority. A caller cannot approve a draft by putting the word "production" in a request. The bank must establish that the selected version has the required approvals and is applicable at that time. The released library is one control within that decision, not the person or service authorized to approve it.

Finally, the bank records the result with the transaction and verifies that the facts have not changed before commitment. The exposure race later in this chapter shows why that last step matters. The local demonstrations exercise the rule evaluation. The bank's identity lookup, transaction reservation and final commitment must be implemented and tested in its own systems.

7.14 Traces are part of the delivered meaning

A decision without an explanation is difficult to investigate. The runtime trace records node identifiers, operators, children, type, value or status, evidence references and source spans. Shared rule evaluations are memoized, and skipped branches have ex-

plicit markers. The trace is deterministic under the request and engine version, making it suitable for replay and comparison.

Trace fidelity is a separate verification obligation. Two programs may return true while one omits the evidence for the exception or incorrectly claims to have evaluated a skipped branch. A regulator-facing explanation assembled from that trace could be misleading even though the Boolean result matches. The comparison therefore includes trace structure and evidence, not just the top-level status.

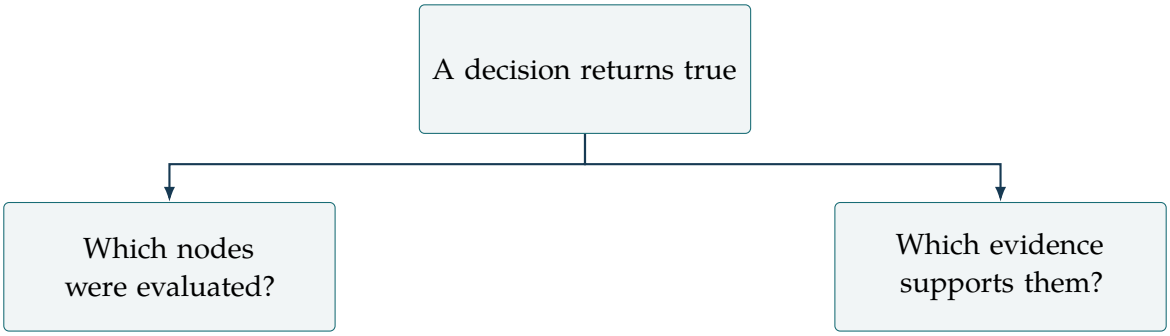


Figure 7.17: Equal Boolean results can still have unequal explanations.

Natural-language explanations are generated from the verified trace and approved source mappings. A language model may help render a readable summary, but it cannot alter the computed result or invent a justification. If a summary mentions a source that the trace and bundle do not contain, validation should reject or flag it. The authoritative explanation remains reconstructible without the summary model.

The current source-span policy associates trace nodes with the owning rule's explicit citations and retains exception citations in the bundle. That is a coarser mapping than a separate legal proof for every arithmetic operation. The report should describe the granularity honestly. A future refinement may attach more precise spans, but a dense set of citations should not be mistaken for a formal demonstration that each mapping is legally correct.

Execution hashes identify the canonical request and result under a particular backend version. They help detect altered bytes and reproduce behavior. They do not authenticate a reviewer or prove that a document came from the regulator. Authentication, source acquisition and approval are separate controls. Confusing these roles is a common source of overclaiming in systems described as auditable.

7.15 Event histories add obligations that a snapshot cannot express

Some requirements concern sequences: obtain consent before an action, honor a withdrawal, provide a document within a period, or preserve a breach after late performance. A current Boolean snapshot cannot express all of these relationships. The ex-

isting event component replays finite attributed histories using a separately versioned Java entry point, `EventReplay.replay(Map<String,Object>)`.

A stream header binds subject, category, actor, action, profile and inception. Events belong to that header; an API path cannot redirect their meaning to a different subject. Duplicate identical events are idempotent, while conflicting payloads or sequence reuse are errors. Ordering authority is explicit. If event order is unresolved, the system must not invent a sequence based solely on arrival.

Consent illustrates why completeness matters. Absence of a grant in an incomplete history does not establish that no grant occurred. A completeness watermark or a reviewed initial state supplies the relevant observation boundary. A watermark cannot cover a time later than the recording instant of its assertion. A withdrawal ends the applicable consent generation, and a later regrant uses a new evidence identity rather than resurrecting a previously withdrawn record.

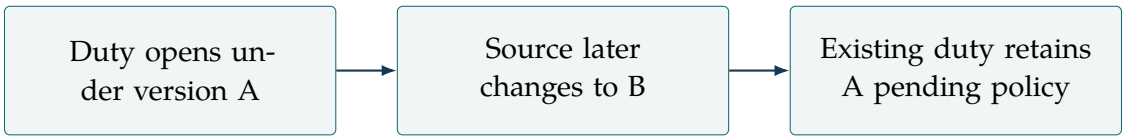


Figure 7.18: Amending the source does not automatically migrate an open obligation.

For an achievement obligation, activation opens a duty with a deadline. The selected contract uses the interval from activation up to, but excluding, the deadline for timely performance. Performance exactly at the excluded endpoint is therefore late under that contract. This is an engineering convention that must match the reviewed legal timing rule; a different legal rule requires a different specified boundary, not a casual adjustment in the host.

Late performance does not erase the historical breach. The system can record both that a duty was breached and that it was later fulfilled. Similarly, an observed action is retained even if an advisory control would have prohibited it. Deleting inconvenient observations would make replay describe an idealized compliant world rather than the bank's actual history.

Each opened duty binds the source bundle that created it. A later amendment does not automatically migrate existing duties. Migration may be appropriate, but it requires an explicit legal and operational policy about which obligations change, which dates apply and how historical findings are retained. The current MVP has no automatic migration implementation. The ensemble can identify the question and prepare alternatives; it cannot assume the answer from the new publication date alone.

7.15.1 Obligations and event histories

The decision fragment cannot represent every continuing control. For consent, let s_i describe the recorded arrangement immediately before event e_i: its client, selected categories, agreed threshold, monitoring mode, relevant reviews and withdrawal status.

Each event includes its timestamps and ordering evidence. It changes the recorded state through a versioned transition function,

$$s_{i+1} = T_v(s_i, e_i). \tag{7.2}$$

Here i orders processed events; it does not assume equal physical time steps. Version v identifies the adopted rules and ordering convention. A withdrawal event can change the availability of the streamlined arrangement for subsequent decisions. A new-funds event can trigger the designated-account checks. An annual-review event creates work that must later be evidenced as completed. The exact consequences require review of the cited guidance and the bank's operating procedure (SFC and HKMA, 2023b,c).

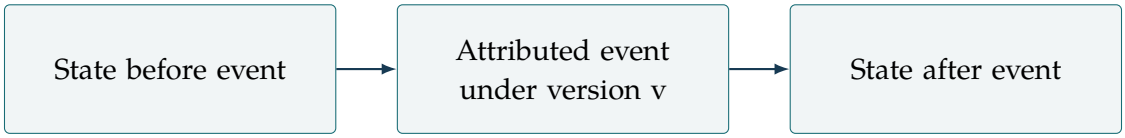


Figure 7.19: An event transition describes ordered changes to the recorded arrangement.

An obligation record should distinguish its creation, due condition, performance, breach and any later remediation. If a transaction occurs after a withdrawal, the audit view must retain the transaction and identify the issue; it cannot erase the event because the authorization view would have blocked it. This follows the useful separation of normative permission from actual occurrence in eFLINT (van Binsbergen et al., 2026).

Time requires two histories. Legal validity time records when a source or adopted rule applies. System-recorded time records when the bank received and deployed it. These support different questions: what should have applied to an event, and what version did the system actually use? The same distinction applies to facts received late or corrected later. A reconstructed historical assessment may use new evidence, but it must not overwrite the original decision and its inputs.

Event ordering is a material assumption. The prototype should store source timestamps, receipt timestamps, sequence identifiers, time zone and any uncertainty in ordering. A synthetic trace with a consent withdrawal and execution at the same timestamp is unresolved unless the available ordering or an adopted convention settles it. Calendar computations should identify whether they use anniversaries, elapsed days or another reviewed operation, drawing on the date semantics literature (Monat et al., 2024). Neither the operating system clock nor the order in which messages happen to arrive establishes legal priority.

7.15.2 The finite event profile

Apply the state model to a single stream: a grant creates generation g_1, a withdrawal closes g_1, and a later grant creates g_2. A transaction carrying g_1 after the withdrawal is

still an observed event, but it cannot be treated as an active-consent action. If the withdrawal and transaction have the same occurrence time and the connector supplies no ordering authority, the profile returns `E_ORDER_UNRESOLVED` rather than guessing from arrival order.

The trace header binds subject, category, actor, action, profile and inception. Each event has an immutable identifier, occurrence and recorded timestamps, and an authoritative sequence number when one exists. Identical duplicate bytes are idempotent; reusing an identifier with different content is an error. A grant or withdrawal can establish the state only when the stream is complete from its declared inception or a reviewed initial snapshot. An incomplete export remains an unknown history, and observed transactions remain in the audit record even when the current state would have blocked streamlined treatment.

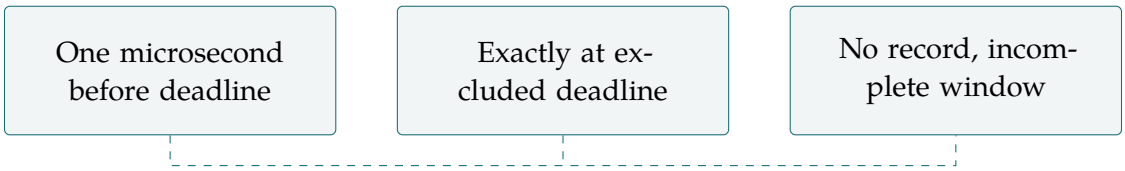


Figure 7.20: Timing and observation completeness determine different duty outcomes.

Under this profile, the first performance is timely and the second is late. In the third case, a missing record cannot yet establish breach: the observation window may simply be incomplete. The three cases require distinct results even at the same assessment time.

An obligation records actor, action, subject, source version, activation and a deadline or explicit absence of a deadline. Let a be activation and d the exclusive deadline. Under this proposed profile, a matching performance at time p is timely when

$$a \leq p < d. \tag{7.3}$$

This is the chosen boundary convention for the profile, not an assertion that every SFC deadline has the same wording. A local due date is converted to the next local midnight. An anniversary requires an explicit leap-day policy; a reviewed choice of February 28, March 1 or rejection replaces an implicit library default. Business-day calendars are deferred.

Absence of a timely record establishes breach only when a watermark shows that the observation window through d is complete. The wall clock cannot create that evidence. Without completeness the obligation stays open. Timely performance produces `SATISFIED_ON_TIME`; a complete window without it produces `BREACHED`. A subsequent matching performance produces `PERFORMED_LATE` and retains the breach. A deadline-free request can be performed but does not become overdue through an invented number of days.

For example, consider a synthetic review activated at midnight UTC on September 1 and due at midnight on September 2. A record one microsecond before the latter instant is timely; a record exactly at it is late. The fixtures contain both. These dates illustrate the runtime convention rather than an SFC-prescribed review deadline. If a late-arriving

record later proves timely performance, replay produces a corrected assessment with a new knowledge cutoff; the original assessment remains visible. Maintenance duties, pre-emptive satisfaction, waivers and legal compensation require future profiles and are rejected now.

7.16 Approve the version that was actually tested

Suppose the reviewer approves a rule on Tuesday. On Wednesday an engineer fixes a defect and rebuilds the program. The new program may be better, but Tuesday's approval cannot silently become approval of Wednesday's change. The review must identify the particular program that was tested and the interpretation it implements.

The records therefore follow a definite order. First the engineers build a candidate from the reviewed rule. The verification report identifies that candidate and records what was checked. The release decision then links the candidate, its evidence and the business scope in which it may be used. Any later change returns to the relevant checks. A familiar version name or an unchanged folder name is not enough to establish that the program is the same.



Figure 7.21: Approval applies to the particular program that the reviewer examined.

This order allows useful work before final approval. Reviewers need a concrete program and its results to inspect; building that program does not authorize its use for clients. Deployment follows the required decisions by the bank's legal, technical and business owners.

A second team should also be able to rebuild the same program from the retained rule, source code and declared tools. Differences would require investigation. Obtaining the same program helps establish which implementation the reviewer saw, although it cannot establish that the adopted legal meaning was correct. Section F.10 of the technical companion explains how the build and approval records identify one another without circular references.

The demonstration uses synthetic identities to exercise this process. A bank still needs authenticated staff identities, separation of responsibilities and protected approval records. Establishing that two copies of a program are identical does not establish who authorized either copy.

7.17 A correct decision can still race with the transaction

Suppose a control depends on consent and exposure. The host reads both, evaluates an order and then commits it. Between the read and commit, consent may be withdrawn or another order may consume the available limit. The decision was correct for its snapshot but is no longer valid for the transaction being committed. This is a concurrency problem outside the pure rule evaluator.

The host should read a consistent snapshot with version identifiers, evaluate exposure including the proposed order, and atomically validate the versions while reserving the exposure and recording the decision. If the versions changed, it rereads and reevaluates under a bounded transaction retry policy. That policy is separate from interpretation-resolution retries and must not share an ambiguous counter called simply "attempts."

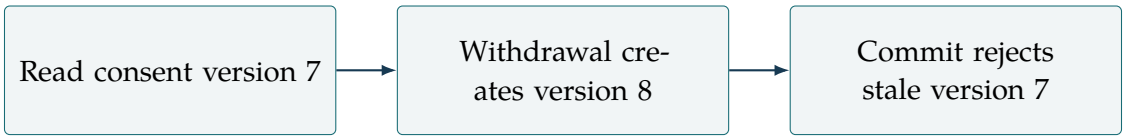


Figure 7.22: A correct snapshot decision can be stale before the transaction commits.

Idempotency protects against duplicate submissions. Repeating the same order identifier and payload should recover the same committed result or reservation. Using the same identifier with a different payload is a conflict. The identity must include the legally relevant order content, not merely a user-interface request number. Otherwise a retry can accidentally authorize a changed transaction.

Release selection has a similar race. The host records the release identity used for the decision and verifies that the applicable release policy permits it at commit. Whether an in-flight transaction can finish under a prior release is a bank policy requiring explicit treatment. The rule engine cannot resolve that question by consulting its process clock or loading whatever JAR is newest.

A rollback also has legal meaning. Restoring a previous software build can repair an implementation defect if that build still expresses the applicable rule. Restoring a superseded legal regime is different. The incident procedure must separate software rollback, operational suspension and legal-rule selection. Historical packages remain available for replay even when they cannot be used for new decisions.

7.17.1 The bank adapter and the exposure race

The adapter has four responsibilities. It selects the approved rule release for the legal entity, business profile and relevant time; resolves identities, categories and evidence into the declared inputs; obtains a consistent consent and exposure snapshot; and records the result alongside the transaction's other controls. The generated library should have

no credentials for editing these source systems. A host record should include transaction identifier, release and snapshot hashes, category and consent generation, data-version identifiers, decision timestamps and the final disposition.

Concurrency is a material part of correctness. Suppose existing gross exposure is HK$8 million and two orders each add HK$1.5 million against a HK$10 million limit. If each reads HK$8 million, both independently pass at HK$9.5 million, but together they produce HK$11 million. A perfectly correct generated formula does not prevent that race. The host must include reserved exposure and commit the new reservation under a transaction or optimistic version check. If the exposure or consent version changed after evaluation, it reloads and evaluates again. A withdrawal must invalidate a stale consent version before a new order can consume the decision. Market-exposure policy, release timing and retry limits must be agreed with the bank's order and risk owners.



Figure 7.23: Two individually acceptable reads can jointly exceed a HK$10m limit.

Each order sees HK$9.5 million when it ignores the other reservation. Together they produce HK$11 million. An atomic reservation or equivalent transaction protocol must prevent both from committing against the same unreserved balance.

An implementable host sequence is:

1. Resolve an idempotency key from the order request. A retry with the same key and unchanged payload returns its recorded disposition; a changed payload under that key conflicts.
2. Read the approved release and the subject/category consent generation, holdings, leverage and outstanding reservations with their version identifiers. Build the dated snapshot and calculate exposure including this order.
3. Evaluate the generated Java library and the host's remaining controls. If this profile is unresolved or refuted, do not consume its streamlined route.
4. Atomically validate the read versions, reserve the permitted exposure and append the decision record. On version conflict, repeat from the read step with a bounded retry policy; otherwise route to review.
5. Retain the committed record for replay. A later correction creates another assessment linked to the old one; it does not overwrite what was known earlier.

The standalone JAR demonstrates the decision step. It does not simulate this bank transaction protocol or certify concurrency in an unspecified host system. Those integration tests are required before production use.

7.18 A verification dossier that supports a release decision

The final technical dossier links the accepted interpretation to its implementation through several kinds of evidence. Named scenarios demonstrate reviewed boundary behavior. Exhaustive finite checks cover declared small domains. Property and mutation tests examine broader classes of implementation mistakes. Symbolic comparison supplies domain-qualified results and replayed witnesses. Differential execution compares Python and Java semantics. Build records identify exact bytes. Host tests exercise concurrency, idempotency and release selection.

These methods overlap but are not interchangeable. A compiler proof could cover an unbounded class of accepted expressions under stated semantics, while still assuming that the factual adapter and source interpretation are correct. A test suite can catch a concrete adapter error that lies outside that proof. An independent legal review can reject a perfectly verified expression because it misreads an exception. The dossier should state the role of each result so that one strong component does not hide a missing one.

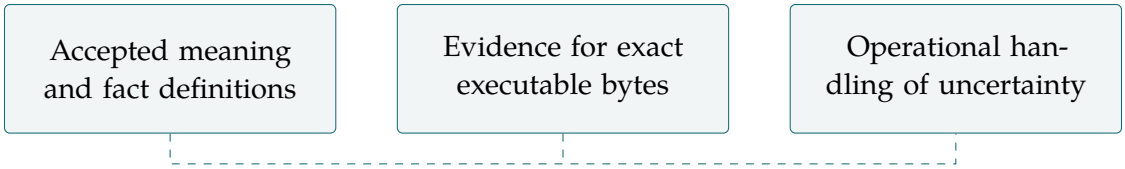


Figure 7.24: The verification dossier supports a concrete release decision.

For the current project, the retained evidence includes the initial prototype, the separate gift-control exercise, live development investigations, source checks and controlled integration. Composition and Java-host update exercises now supplement the individual-rule comparisons. Each retains the open interpretation questions that execution alone cannot answer. The next empirical step requires defensible reference judgments and a complete comparative study. The retained development runs do not measure performance on unfamiliar sources or supply the institution approval needed for a bank control.

A release reviewer should be able to answer four concrete questions from the package: what rule is being enforced; what evidence makes its inputs meaningful; what demonstrates that these exact bytes preserve the rule; and what happens when an assumption or input is uncertain. If any answer requires guessing from a model confidence score, the dossier is incomplete. Chapter 8 follows those requirements into the work of compliance, operations and engineering. The detailed service records and implementation tasks are retained in Appendix D of the technical companion.

Chapter 8

What the bank would own and operate

The implementation has to answer a practical question: who can trust a decision, and what can they inspect when the decision is challenged? A compliance reviewer owns the meaning of a circular. A business owner sets the perimeter in which that meaning will be used. Operations supplies and records the facts. Engineering makes the approved meaning executable. A later incident may require all four people to reconstruct one client or product decision.

This chapter follows that responsibility through one change. It starts with the reviewer's view of the decision, then follows the evidence into a shared typed model, tested RuleIR/Java and Catala targets, and a release. The detailed service paths, storage rules and task history remain in Appendix D of the technical companion; they are reference material for people who must build or audit the route.

8.1 Begin with the decision a person needs to make

A reviewer opening the SPI example should see the source passage, the client's financial and category evidence, the alternatives that were considered, and the case that would change the answer. The reviewer should not have to reconstruct that explanation from a list of fields. If the portfolio is missing but the alternative net-asset route is established, the screen should say why the missing value does not block this particular conclusion. If two records disagree, it should show both records and identify the person who must resolve them.

The same view serves different people in different ways. Compliance needs the source and adopted reading. Operations needs the action, owner and deadline. Engineering needs the precise values and histories used to execute the rule. An auditor needs to replay the result later. One maintained decision record can support all four views if the reader starts with the conclusion and can open the reason behind it.

8.2 The shared frontend and the Catala routes

Suppose the bank wants to calculate an amount from a client's holdings. The earlier scalar rule can compare a supplied total with a threshold, but someone must calculate that total first. A rule that selects eligible holdings and sums their amounts needs the individual items as inputs. Moving the calculation outside the rule would move its classification and completeness assumptions outside the comparison too.

The shared frontend records the proposed calculation, its source passages, scope, factual meanings and treatment of missing evidence in a *LegalRuleModel*. Each target translator receives that fixed model without making another language-model call. It first checks which operations it supports. The resulting *capability report* either permits translation or identifies the unsupported expression. A missing operation therefore remains visible before a smaller program can be mistaken for the reviewed rule.

The two Catala routes serve different needs. The compatibility route translates the supported RuleIR rules into Catala while keeping the existing decision, explanation, evidence, transaction and release interfaces. The native route translates the shared model directly. For the holdings calculation, a *record* groups each item's inclusion flag and amount; a *list* holds the records; filtering selects the included items; and summation adds their amounts. The RuleIR target reports this collection model as unsupported, while the native Catala target can execute it.

Two further types prevent information from being forced into an uninformative flag. An *option* represents either a present value or a known absence. A *payload variant* combines a category with the data appropriate to that category, such as a tagged kind of holding with its associated amount. Neither known absence nor a known category means that missing evidence has been found. Version 1 already supports these richer values under a complete-input policy. Version 2 adds scope conditions, explicit result states, defaults and exceptions, exact scaling, separately chosen rounding, and typed calendar and collection operations. It also supports pure reusable calculations, called helpers, and several named results from one interpretation. Helper calls and type definitions must have no circular dependencies.

Exactness and rounding answer different questions. Half of 101 monetary minor units is 50.5 units. An exact scaling operation reports that the result is not an integer number of units. A separately declared nearest rounding operation, with ties away from zero, returns 51. Keeping those operations distinct prevents the compiler from choosing a monetary policy implicitly. Calendar operations likewise have named conventions: adding a month with month-end clamping uses the final day of the destination month when the original day does not exist.

The input policy is part of the meaning. The `complete.v1` profile requires every declared input to be usable. Version 2 also supports `partial.v1`, which follows the observations actually used in the calculation. Consider the retained synthetic example of a record whose amount is known to be one-third but whose inclusion flag is unknown. Reading the amount returns one-third. Returning the whole record gives a partial result with the two field states preserved. Filtering on the inclusion flag leaves membership unresolved, so the sum is unknown. A complete empty list, by contrast, sums to zero; an incomplete list cannot establish that no further items exist.

RuleIR checks for conflicts across the facts in a rule's static dependency set before evaluating its expression. The partial Catala policy can inspect an unaffected field without inspecting another field's conflict. These are different policies, so the RuleIR target rejects `partial.v1`. The compatibility route retains RuleIR's original policy. In both native profiles, Python validates and encodes observations and decodes results; Catala performs the arithmetic, selection and propagation of known, unknown, conflict and

other declared result states.

Table 8.1: The executable layers used by the completed comparison.

Layer	Responsibility	Evidence boundary
Shared frontend	Retain sources and scope; record one typed interpretation, evidence policy and outputs	Common model and interpretation hashes
RuleIR target	Execute the bounded scalar and finite-duty profile through Python and Java	Existing RuleIR semantics, traces and release checks
Catala target	Lower the common profile or emit richer native Catala with declared policies	Catala build, interpreter, Java and exact-reference checks
Host boundary	Validate facts and evidence, decode outputs and preserve provenance	Input, result, package and transaction invariants

Freezing the model makes a disagreement introduced by translation easier to isolate. A missing source qualification can still be present in that common model. The source checks in Chapter 6 address this earlier opportunity for error.

8.2.1 What the completed checks establish

The retained evidence has several scopes. The modular replay sent the same frozen model and evidence policy through both targets for 44 paired cases. Each pair preserved the model and interpretation hashes, and each compiled result matched the original RuleIR evaluator's complete semantic trace. Seven richer outputs executed as native Catala, uninstrumented Java and exact checks inside the pinned Catala interpreter; the RuleIR target reported those models as unsupported. These checks establish finite translation and execution behavior for the declared controls. They do not establish that the source readings are legally correct.

Version 2 received a separate set of 91 exact checks for the extended semantics. Section 9.1 distinguishes these checks from the earlier backend tests and source-generation study. Appendix F.1 in the technical companion identifies the commands, source versions, logs and detailed regression accounting.

The common fragment retained its checked behavior, and the native target executes the documented richer models. Whether that additional expressiveness improves interpretation or review is a separate empirical question. The completed engineering tests do not supply a performance or legal-quality ranking, human review evidence or production approval.

8.2.2 The bounds that remain part of the interface

The native profile is a closed typed vocabulary with acyclic helper and type graphs. The generated interface enforces declared limits on native types, expression depth, inputs and outputs, candidate source size, observations and expanded types. Imported modules outside the pinned library set, recursive types, arbitrary Catala operators and unbounded collections remain unsupported. All-output execution now evaluates the sealed program once and returns each output with the same value, status and evidence as an individual call. A resource report exposes the type and depth expansion introduced by state encoding before compilation. The earlier directly authored native programs could raise an execution failure for conflicting consequences. That historical behavior must be distinguished from the completed shared translator, which encodes strict exceptions and structured result states. Two active exceptions conflict there even when their consequences agree.

The current native trace records branches emitted into Java and their source positions. Observation hooks are added after the ordinary compiler passes, so both the original and instrumented compilers run the invariant checks, including the nested-scope check that the first tracing implementation could not pass. Standard-library internals and source choices removed by optimization are not observed. Comparing plain Java, instrumented Java and the original Catala interpreter helps expose changes introduced by tracing, but those finite checks do not prove that the instrumentation preserves every program.

These limits are useful precisely because they are visible. An unsupported construct produces a capability report or a reviewable build failure. It is not silently rewritten as a smaller rule. A bank release would still require the source, interpretation, evidence mappings, identity controls and approvals described later in this chapter.

8.3 Follow one change from source to release

The route is easier to understand as a sequence of questions than as a software architecture. What did the circular say? Which activity and legal entity are in scope? Which reading did the bank adopt? What facts does that reading require? Which cases would expose an error? Does the program return the same result as the reviewed rule? Who approved this exact version, and when may the bank use it?

The prototype now exercises this route for public documents and synthetic facts. Consider the gift review. The reviewer begins with the selected paragraph and source edition, sees alternative readings and missing coverage, and examines a concrete distinguishing case where the fact meanings can be aligned. Separate control components keep their answers and open questions when Java is generated. If the source changes, affected work is reconsidered while the earlier decision remains reproducible.

That journey includes different kinds of progress. Retrieving the FAQ supplies context. Running a case exposes a program difference. An authorized decision adopts a meaning for the bank. The first two are available in local development; the third requires the

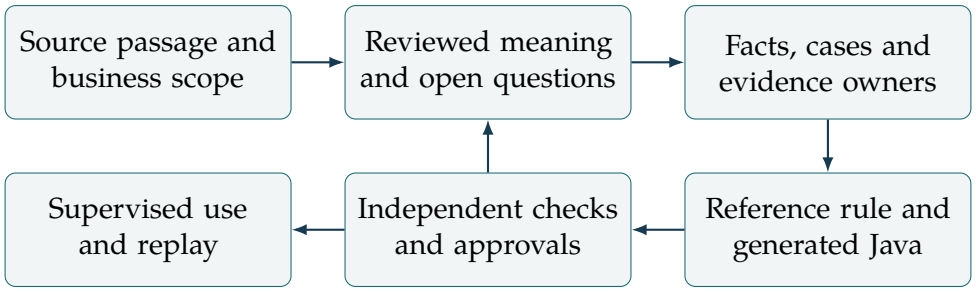


Figure 8.1: A bank release carries a reviewed meaning and its evidence through execution. Each step answers a different question.

institution's own authority and evidence. The demonstrated route therefore supplies draft controls for review, with the open questions carried through to delivery.

8.4 Keep the release tied to its meaning

A tested program is only useful when the bank can identify the rule it implements. The release therefore carries the source edition, the adopted interpretation, the factual definitions, the supported dates and activities, the test evidence and the exact Java package together. Changing one of those parts creates a new version that needs its own review. A content identifier helps detect accidental mixing of versions; it does not make an administrator, source or interpretation authoritative.

The same principle governs a correction. If a client withdraws consent on Tuesday, a Monday decision must remain reproducible with the information known on Monday. A Wednesday amendment must show which cases and clients change. The bank can replace a defective software package while preserving the legal version that applied to an earlier transaction. Historical replay is therefore part of the control's meaning, not a convenience added after deployment.

8.5 What the prototype has and has not demonstrated

The engineering record contains checks for selected rules, edge cases, deliberate mutations, release identity, withdrawal, amendment and archive restoration. The interpretation route now also generates alternatives and checks their source support. Earlier rounds included live development calls; later integration exercises used those retained responses and controlled inputs, with unresolved interpretations retained through delivery.

The distinction matters for the next investment decision. The mechanism can now be exercised, but no comparative study has measured how often it misses a requirement on unfamiliar material, how quickly a reviewer can use its reports, or how it behaves within the bank's identity and data controls. Public sources and synthetic identities remain the basis of the local demonstrations.

Those unanswered questions belong to the evaluation and operating decisions in the last two chapters. Keeping them separate prevents a successful build from being mistaken for a legal approval or a production control.

8.6 The boundary before a bank pilot

Before a supervised pilot, the bank would have to name the legal entities, products, client categories, source families and decision points included in the study. It would need an authorised legal owner, data owners for each fact, independent engineering and compliance review, and a response for unknown or conflicting evidence. The pilot should run beside the existing process so that staff can compare decisions and record material disagreements. A case that cannot be supported should stop or go to a named reviewer; it should not be converted into a plausible default.

The implementation record makes the route concrete. The management decision is whether the bank is willing to supply the authority, evidence and supervision that the current public demonstration cannot supply by itself.

Chapter 9

Measuring error, uncertainty and the value of redundancy

A prototype can pass every test it was given and still fail the question management cares about: does it find material omissions and help qualified reviewers make better decisions? That question cannot be answered by counting successful calls, model agreement or a short list of synthetic cases.

9.1 What the completed RuleIR–Catala comparison can answer

The Catala work answers a narrower question before it answers any product question: can one reviewed, typed interpretation reach two executable targets without silently changing the declared cases or evidence policy? The shared frontend makes that comparison meaningful. It gives RuleIR and Catala the same source packet, interpretation hash and typed model. Each target reports which parts it can accept. The comparison still inherits the limits of that one interpretation: agreement can preserve a missed qualification.

For the common scalar profile, the modular replay passed 44 paired cases through both targets. Each pair retained the same model and interpretation hashes, and the compiled result matched the original RuleIR evaluator's complete semantic trace. The compatibility remediation adds 344 complete decision comparisons, eight shared event cases and 202 regression tests. These are finite engineering checks of translation, execution, provenance and host behavior. They are not independent legal judgments and they are not a sample from a population of circulars.

The richer comparison asks a different question. Seven version-1 outputs exercised record/list filtering and aggregation, optional presence and absence, payload variant construction and matching, exact decimal multiplication and money aggregation. They executed as native Catala and passed the pinned interpreter and Java checks. The RuleIR target reported those models as unsupported. This is evidence that the Catala route has a wider declared implementation profile. It does not show that a wider profile produces better source interpretations, fewer omissions or safer bank decisions. The later version-2 `partial.v1` policy also differs from RuleIR's static whole-fact conflict veto. The version-2 checks therefore test that policy against its own declared reference, rather than treating a different RuleIR answer as a failed implementation of the same semantics.

The direct native converter had already demonstrated seven generated programs on 43 development cases. Host-boundary hardening kept those programs and added five cases, giving 48 passing cases. Six deliberately changed programs produced detectable wrong answers, and four forged execution records failed replay. The tasks included holdings, tiered charges, calendar deadlines, categories, repeated household calculations, nested exceptions and the two net-asset definitions in the selected SPI paragraph. These computations show why a richer target is useful, while the supplied factual classifications and small convenience sample limit what they say about source interpretation.

The later gap-closure source study compared generation attempts before the shared frontend was introduced. It used 19 of 24 authorized dispatch slots: the minimal native arm passed two of four tasks, the source-reviewed native arm passed none of four, and the RuleIR generation arm passed none of four under the frozen output and source requirements. Most failures were missing context, source quotation or executable-format failures. A small repair probe succeeded on one exposed task, bringing the combined study and repair to 21 of 24 slots. The source-reviewed arm spent part of its allowance on criticism; its format failures usually prevented the critic from being reached. Missing source context and unequal opportunities for repair are plausible explanations for the observed counts. Three previously exposed source families, provisional references and wide paired uncertainty intervals support no converter-quality ranking. There were zero human observations in the separate reviewer packet; synthetic scoring responses were used only to test that machinery.

The final version-2 work added 91 exact checks in four sealed builds. A trusted regression run passed 792 tests, followed by 69 translation tests after compatibility repairs and 35 affected tests after fixture corrections. Together these checkpoints cover 794 distinct test identifiers with no skips; they are not one final full-suite invocation or additional source-conversion replications. The subsequent merge into main passed 30 focused integration tests. None of these deterministic runs measures comparative runtime or the quality of fresh source interpretation.

The execution follow-up preserved 186 complete result records across 13 existing builds: 88 executions of the 44 paired cases, seven richer version-1 outputs and the 91 version-2 checks. It also compared batches of 1, 8, 20 and 40 outputs with individual calls, plain Java and the original interpreter. Every batch used one Java process and returned identical individual results. For the 40-output integer example, observed execution time fell from 44.09 seconds for separate calls to 1.26 seconds for the batch. These single local measurements describe that workload; they do not establish a general speed ratio or rank Catala against RuleIR. The change removes repeated execution while preserving the checked results. A subsequent complete regression passed 807 tests with no skips in 1,038.44 seconds against one unchanged implementation checkpoint. This supplies the final-suite evidence that the earlier sequence of repair runs lacked.

Table 9.1: Questions answered by the retained RuleIR–Catala evidence.

Question	Evidence	Status
Does the common model preserve declared scalar behavior?	44 paired cases and the compatibility regression records	Supported for the retained controls
Can the native Catala route execute richer shared models?	Seven rich outputs with interpreter, Java and exact-reference checks	Supported for the declared feature profile
Does Catala convert unfamiliar legal text more accurately?	No independently adjudicated, held-out paired study	Not checked
Does Catala reduce review time or improve comprehension?	No human observations or counterbalanced reviewer experiment	Not checked
Should Catala replace the RuleIR default?	No promotion criterion or uncertainty analysis for that decision	Unsupported

The remaining question is whether the extra executable vocabulary helps people interpret and review unfamiliar circulars. A future quality study needs independently adjudicated source families, equal and declared resource budgets, a held-out set, source-family-aware uncertainty and observations from human reviewers. Until that evidence exists, the defensible conclusion is that Catala is an optional target with a wider implemented feature profile, while RuleIR remains the checked baseline for its own profile.

9.2 The experiment must ask the right question

The main empirical question is whether the method discovers and reports material interpretation errors and omissions more reliably than simpler alternatives at comparable cost. A secondary question is whether its reports help reviewers make better, faster and more traceable decisions. A third concerns resource use: whether targeted resolution spends effort on questions that can actually be answered. These questions require different measurements. The protocol here is prospective as an effectiveness study: improved interpretation of SFC circulars has not yet been demonstrated. The four-method comparison can now be executed by the evaluation software. Its development checks establish dispatch, scoring and resource accounting, not the comparative result that this chapter seeks.

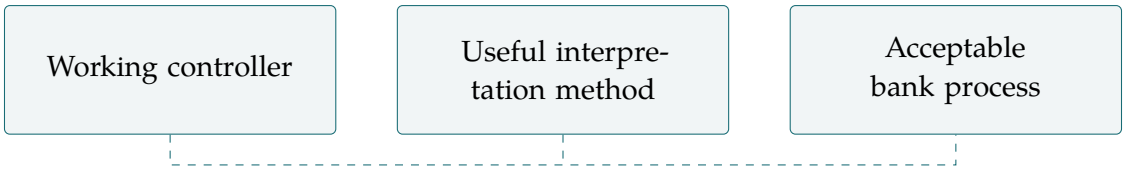


Figure 9.1: Engineering, empirical and deployment questions require different evidence.

The feared failure is a false clean result: a report appears complete and unambiguous while a material source requirement, alternative reading or factual assumption has been missed. A system that abstains on every case avoids some false clean results but may be operationally useless. Evaluation must therefore measure risk and useful coverage together. Neither raw agreement nor a low average model uncertainty score is an adequate promotion criterion.

Before a live study, write an evidence contract naming the corpus, baselines, reference process, primary outcome, uncertainty analysis, hard vetoes, resource limits and permitted conclusions. Freeze it before examining comparative results. The manuscript proposes such a protocol; it does not report a completed trial. Thresholds that depend on the bank's risk tolerance remain decisions for the responsible owners, not numbers to be invented by the implementation agent.

9.2.1 The product decision and initial comparison scope

The prototype should answer whether a shared executable specification reduces material mistakes and review effort for a defined class of regulatory changes. It should also identify the cases that still require judgment and the cost of maintaining the formalization. A demonstration that translates a clean paragraph and returns a correct number would establish too little; a project that attempts all SFC circulars at once would make the source and evaluation problems too large to diagnose.

The proposed scope is a twelve-week exercise with three connected examples. The SPI circular and annexes supply the main decision and workflow case. The two tokenisation circulars supply a version-replacement and actor-specific case. The fund-marketing circular supplies exceptions and judgment-dependent clauses. The thematic-review announcement supplies a negative classification example: the collector must recognize useful regulatory information without inventing an eligibility rule (SFC and HKMA, 2023a; SFC, 2023c, 2026c, 2023b; SFC and HKMA, 2025).

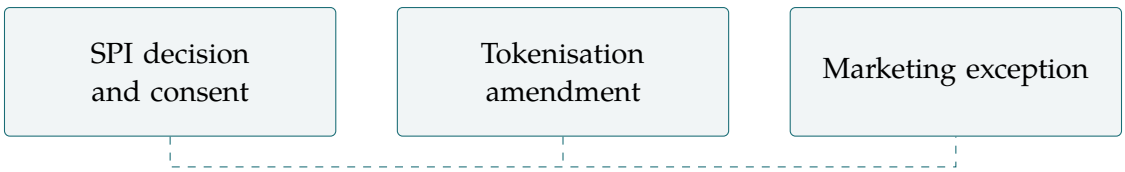


Figure 9.2: The initial source families expose different failure mechanisms.

9.3 Define the unit of evaluation

A circular contains many questions, and a single question can affect many transactions. Counting every transaction as an independent interpretation example would exaggerate sample size when they all depend on the same disputed classification. The evaluation should distinguish document, provision, interpretation question, candidate, scenario and transaction levels.

The primary unit for omission detection is a reviewed provision or interpretation question within a source packet. The source packet includes the circular and required dependencies as of a declared cut-off. Scenario-level outcomes measure how candidate rules behave under facts. Document-level summaries measure whether an apparently complete investigation misses any material issue. These views complement one another but have different denominators.

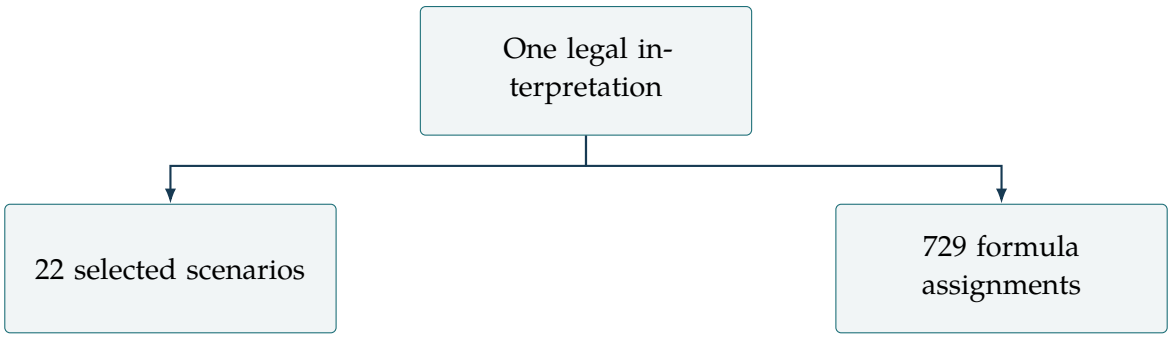


Figure 9.3: Many executions of one formula do not create many independent interpretations.

For example, the 729 ternary assignments are not 729 independent regulatory interpretations. They all arise from one selected formula and six abstract inputs. They are excellent finite-domain engineering coverage for that formula. Treating them as a sample of 729 circulars would produce meaningless uncertainty intervals for legal translation accuracy. The 22 named cases likewise share source, author and modeling assumptions.

The corpus should record these relationships explicitly. Each scenario names its source packet, provision, interpretation question and reference decision. Each candidate names its generation method and source version. Analysis can then aggregate at the appropriate level and use cluster-aware uncertainty estimates rather than treating related rows as independent observations.

An unresolved reference question is still an evaluation object. The desired output may be recognition of ambiguity, preservation of supported alternatives and escalation, rather than selection of a single label. If the dataset forces such questions into a binary answer, it rewards unjustified certainty and penalizes the behavior the bank actually needs.

9.4 Build the reference before seeing the model's answer

The reference process begins with independent source inventory. Reviewers enumerate provisions, footnotes, definitions, exceptions, dates and dependencies. A separate reader checks the inventory against the retained document. The process records which materials were unavailable and which judgments depend on them. The corpus should not silently exclude difficult documents merely because their references are inconvenient to acquire.

Some questions have an explicit outcome in an official worked example; others can use an authorized, recorded decision with the relevant facts. Such sources can supply a reference for that exact question. They still require an assessment of scope, edition and exceptions. A model-written answer or a program agreeing with its own table does not create independent support for the meaning.

For contested interpretation questions, the proposed study uses at least two appropriately qualified reviewers who form initial judgments without seeing the candidate system's answer. They state the reading, assumptions, relevant passages, plausible alternatives and distinguishing examples. The study plan must justify their number and qualifications for its target. This can involve internal compliance and legal staff; it does not make an external consultancy a prerequisite for every reference question.

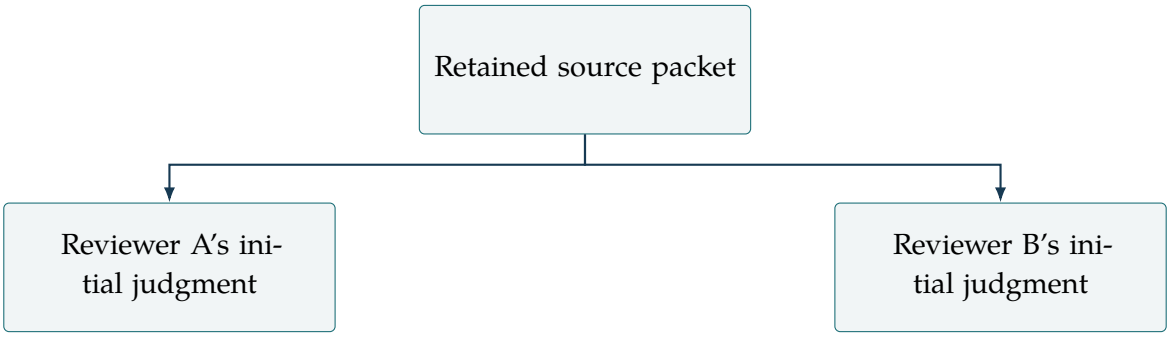


Figure 9.4: Reference construction precedes exposure to the candidate answer.

When reviewers disagree, adjudication examines the reasons. A third reviewer or panel can resolve the issue, retain multiple acceptable readings, or declare the available authority insufficient. The adjudication record preserves the original positions and the evidence that changed them. Majority voting among reviewers should not automatically settle a question of authority or source meaning.

Reviewer independence is documented, not assumed. Shared training, a common case summary, prior involvement in the implementation and access to model outputs can all influence judgments. Some collaboration is necessary for adjudication, but initial blinded judgments should remain distinguishable from the later consensus. The study reports both stages.

Reference revisions are inevitable when new evidence appears. They require a ver-

sioned correction with a reason. Comparative results are recomputed against the corrected reference, and the report explains the effect. A correction that favors one system is not automatically improper, but it must be justified without using that system's desired score as the reason.

The reference itself is not a universal truth oracle. It is an independently reviewed account of the declared source perimeter at a time. The study can measure agreement with that account, sensitivity to its material issues and handling of its unresolved questions. It cannot establish that no future authority or previously unknown source could change the interpretation.

9.5 Construct a corpus that exposes the expected failures

A useful corpus contains several kinds of difficulty: explicit Boolean conditions, exceptions, definitions supplied elsewhere, actor and activity scope, uncertain classification, amendments, transition periods, event histories, monetary units and interactions between requirements. The initial public examples can anchor the engineering harness, but a method comparison needs additional circular families that were not used to design the prompts or repair logic.

Selection should reflect the intended product perimeter. If the product focuses on private-bank distribution controls, an evaluation consisting entirely of simple product-authorization filing instructions would be a weak test of deployment readiness. Conversely, including arbitrary legal texts outside the supported language can be useful for measuring abstention, but should not be mixed with in-scope accuracy without reporting the distinction.

Temporal separation matters. A holdout containing a later amendment can reveal whether the system improperly reuses an earlier definition or priority. The source packet must state whether the task is historical interpretation as of a cut-off or current applicability review. A model with knowledge of a later rule can otherwise appear to correct a historical answer by importing information that was unavailable at the assessment date.

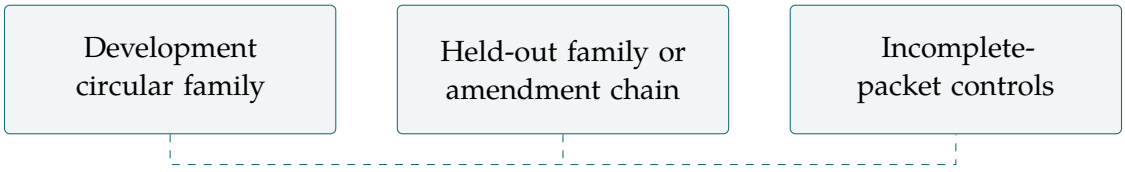


Figure 9.5: A useful corpus tests transfer and recognition of missing evidence.

Synthetic cases are valuable for controlled defects. They can isolate a missing exception, an ambiguous attachment or a contradictory fact. Their provenance must be visible, and their performance must be reported separately from naturally occurring circular

questions. A system can learn the style of seeded mutations without becoming better at discovering unanticipated omissions.

Include negative controls in which no material ambiguity is supported, as well as positive controls with known alternative readings. A method that always invents a dispute should not receive full credit for finding ambiguity. The reference should distinguish a plausible source-supported alternative from an arbitrary logical variation. Review burden from unsupported alternatives is an operational cost worth measuring.

The corpus should also include deliberately incomplete packets. The correct behavior may be to identify the missing dependency and block a conclusion. These examples test whether the system recognizes the limits of its evidence. They must not be scored as ordinary translation errors simply because no final rule is produced.

9.5.1 Fifteen cross-circular acceptance scenarios

The proposal can be made concrete before connecting to any bank system. The following cases specify the initial demonstration's intended behavior. Monetary inputs refer to the adopted financial definitions; the table does not imply that those facts alone establish SPI status. These are source-based design cases for independent compliance review. Execution evidence for the selected Java subset is reported separately below; this broader table is not claimed as completed.

The first evaluation set is easier to follow when grouped by the decision it teaches. The cases below are prospective demonstrations, not completed results. The full fifteen-case record remains in Appendix E of the technical companion; each case still has an identifier so that a later report can point to the exact fact pattern.

Table 9.2: The questions covered by the initial acceptance cases.

Question family	The contrast that the evaluation must explain
Financial alternatives	With portfolio evidence missing, HK$70 million in net assets leaves a question open; HK$80 million establishes the alternative financial route.
Ownership and category	A jointly held asset differs from a wholly owned asset. Experience in one product category does not establish experience in another.
Exposure and consent	An order before withdrawal differs from one after it. The decision must follow the adopted order of events.
Amendment and exceptions	A fee discount differs from a product-linked gift. The assessment must preserve the stated exception.
Scope and classification	A thematic-review announcement is relevant source material, but does not thereby create a numerical eligibility rule.

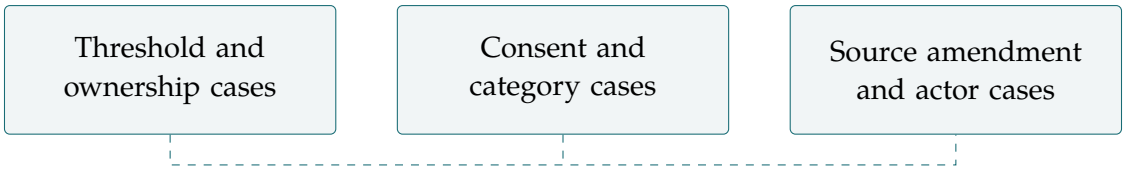


Figure 9.6: The fifteen design scenarios cover different substantive questions.

The full financial and ownership cases derive from Annex 1, paragraphs 3.1–3.4; the category and objective cases from paragraphs 4–7. The designated-account case needs paragraph 8.3 and Annex 2, FAQ 6. Withdrawal depends on the consent provisions and the adopted ordering of events. The remaining cases concern the tokenisation editions, marketing exceptions, redemption arrangements and the thematic-review announcement (SFC and HKMA, 2023b,c; SFC, 2023c, 2026c, 2023b; SFC and HKMA, 2025). This source mapping is part of the acceptance package and should be visible beside each case in the workbench.

9.6 Use a baseline ladder with comparable resources

The implemented comparison asks what is gained at each of four steps. A single reader produces one interpretation. Isolated readers produce alternatives without seeing one another's answers. Search investigates and refines those alternatives. The assurance method adds separate source inventories and checks for claims or qualifications that the proposals may have missed. These are four actual routes through the software, each given the same declared source access and resource ceilings.

Eight scripted jobs, covering two development cases across these four methods, exercised dispatch and scoring against concealed finite Java outcomes. Checks also exercised missing answers and a method that abstains on every case. This matters because a method can avoid wrong answers simply by answering nothing. The exercise tests the measurement procedure; it supplies no live-method ranking.

A more detailed research design would separate further contributions. The following five contrasts retain that proposed design as additional experiments, with names distinct from the four implemented methods. The simplest baseline is a single structured reading with the same source packet and a fixed prompt. It establishes what the ensemble adds beyond ordinary translation. A stronger baseline uses that single reading plus deterministic schema, coverage and execution checks. This prevents the proposed method from claiming credit for checks that do not require an ensemble.

A third baseline uses repeated sampling and answer aggregation, inspired by self-consistency. It receives the same total resource allowance and reports its aggregation method. A fourth uses diverse initial proposals with a fixed, deterministic resolution scheduler. The enhanced method adds the proposed hypothesis-family coverage, targeted questions and optional argumentation or tree search. This ladder isolates the contribution of each mechanism.

Table 9.3: Proposed further comparisons separate redundancy from search and verification. These five contrasts are not the implemented four-method study.

Arm	Mechanism	Question it answers
A0	One structured proposal	What does ordinary structured translation achieve?
A1	A0 plus deterministic checks	How much comes from source coverage and validation alone?
A2	Repeated proposals with aggregation	Does repeated sampling add useful error detection at equal cost?
A3	Diverse proposals and fixed resolution	Does explicit disagreement handling improve useful coverage?
A4	A3 plus targeted search or argumentation	Does the enhanced scheduler find additional material issues?

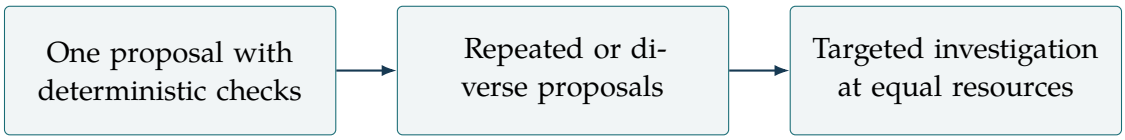


Figure 9.7: The baseline ladder separates the contribution of each added mechanism.

Equal resources do not mean merely equal numbers of model calls. Calls differ in input length, output length, retrieval, latency and cost. The study should record both token or monetary budget and elapsed time, and present comparisons at several predeclared budget levels if that is part of the question. Human reviewer access must also be comparable. Giving only the proposed method a legal reviewer would confound the method with extra expertise.

Prompt and configuration tuning use a development set. The strongest baseline should receive reasonable task-specific tuning rather than a deliberately weak prompt. The test set remains held out until the protocol is fixed. If a test failure triggers a repair, the repaired method is evaluated on a fresh holdout or reported as development evidence, not as an untouched final result.

Search methods require their own assumptions audit. A beam width, family quota, UCT exploration constant or no-progress threshold transferred from puzzles or another repository is a hypothesis. It can be a starting point, but must not become the default without target-specific evidence. The deterministic scheduler is the initial engineering baseline because its behavior is easier to inspect, not because it has been proven optimal.

9.6.1 The separate comparison of whole review workflows

The baseline ladder above compares automatic interpretation methods. A second experiment is needed for the product decision: does the complete workbench help people carry a change through review? The four workflow conditions below answer that broader question. They must not be pooled with repeated model calls as if both experiments had the same observational unit.

The principal engineering question is whether the workbench produces a more reviewable and dependable implementation of the selected requirements than the current process. The baseline must therefore be a faithful version of that process, with the same documents, permitted reference material, business scope and definition of a completed change.

Use four conditions: the existing manual process; a source-grounded LLM summary/checklist; a hand-authored specification with the workbench; and the same workbench with model-assisted drafting. The summary baseline tests whether a simpler product achieves the benefit. The two formalized conditions isolate the incremental value of the model. Optional proof/search enhancements form a later arm under the same information and budget constraints. Preparation of formal rules, examples and reusable definitions must be counted, rather than treated as free information available only to the assisted condition. This addresses the information-budget distinction exposed by the tax-reasoning literature (Jurayj et al., 2026).

Independent compliance reviewers should derive expected outcomes and required actions from the selected sources before seeing the candidate implementation. Where interpretation is contested, the expected outcome can be a review request or an explicitly adjudicated reading. Ambiguity must not be removed merely to obtain a convenient deterministic label. Cases generated from the formal model serve a different role: they exercise its paths and mutations, and should be reported separately from the independent source-derived cases.

Material errors include applying a rule to the wrong actor or category, allowing a scoped action after a relevant withdrawal, losing a required obligation, inventing a regulatory restriction, and converting decision-relevant missing information into an established fact. The exact taxonomy and severity must be agreed in weeks 1–2. A correct financial subcondition must not be scored as an incorrect trade permission merely because the test harness confuses those outputs.



Figure 9.8: The workflow study counts human preparation and review in every condition.

The study needs a small number of decisions that everyone can understand. A serious defect blocks the affected scope. Review effort matters only after the quality screen.

Unknown and review outcomes explain how the system behaves when facts or authority are insufficient. Reproducibility and source coverage protect the evidence on which those decisions rest. Appendix E of the technical companion keeps the individual measures and their proposed roles.

Table 9.4: The evidence that can change an evaluation decision.

Evidence	How it affects the decision
Material interpretation or decision defects	A missing requirement, wrong actor or wrong obligation can block the affected use. An unresolved critical defect fails the quality screen.
Review effort and useful abstention	After quality is established, measure staff time and whether unresolved cases reach the right owner. Silently treating an unresolved case as settled remains a defect.
Source coverage and implementation agreement	Missing support or disagreement between the adopted rule and its implementation can veto release within the examined scope.
Reproducibility and operating cost	Replaying the same rule, facts and history shows whether decisions can be reconstructed. Time and cost inform operation; they cannot establish legal quality.

The proposed evaluation uses at least 30 independently adjudicated change tasks, provisionally 10 for development and 20 held out. This is a feasibility budget assumption, not a statistical power calculation. Split by circular family and amendment chain; if the five initial documents cannot supply a credible separation, expand the corpus before making a generalization claim. Two compliance readers derive the expected scope, duties and case outcomes; contested interpretations remain a separate stratum rather than disappearing from the denominator.

Report paired error and reviewer-time differences, with intervals that account for reviewer and document effects. Repeated calls on one paragraph are not independent regulatory changes. Assign comparable tasks, rotate conditions where feasible and record prior exposure to avoid giving the assisted condition an unearned learning advantage.

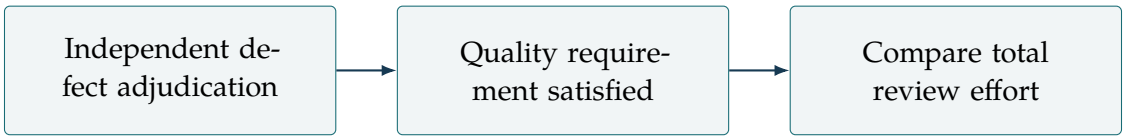


Figure 9.9: A time saving becomes relevant only after the required quality checks pass.

Zero unresolved critical false approvals or missed duties is a proposed release screen on the adjudicated cases. Passing it does not estimate a zero population error rate. Crit-

icality is defined with the bank before unblinding. If too few independent families remain for a defensible comparison, report descriptive results and the additional sample needed rather than rank viable approaches.

A small pilot may establish useful failure cases without supporting a precise speed ranking. Zero observed material errors in a finite synthetic set does not establish a zero error rate in the bank's workload. Similarly, a high mutation score says the selected perturbations were detected; it does not show that every missing legal requirement would be found. Report these limits directly and use the evidence to select the next scope of validation.

The evaluation should pause if its source inventory is wrong, its expected outcomes are not independently supportable, records are corrupted, or conditions receive materially different information without accounting for it. Those are problems with the evaluation. A failed drafting strategy or event adapter is a candidate failure: it should trigger a specified repair and repeat of the affected checks, rather than abandonment of the whole product direction.

9.7 Measure omissions, alternatives and useful abstention

Let Q be the set of independently reviewed material questions in the evaluation corpus. For each $q \in Q$, the reference records required conditions, accepted or plausible readings and whether available evidence resolves the question. A material omission occurs when the system's report fails to represent a required condition or unresolved question within its claimed scope. This definition includes missing uncertainty, not only incorrect final formulas.

A false clean result occurs when the system reports a question as resolved and eligible for review without disclosing a material unresolved defect identified by the reference. The denominator is the number of questions or documents declared clean, depending on the reported metric. A system that declares fewer clean cases may reduce this rate while increasing manual workload. The report must show both.

Define useful coverage as the fraction of eligible questions for which the system produces a reference-supported, sufficiently complete result without unresolved material defects. Define selective risk as the error rate among the questions it chooses to resolve. A risk-coverage curve varies the decision policy and shows the tradeoff. The policy must not be tuned on the final test labels after the fact.

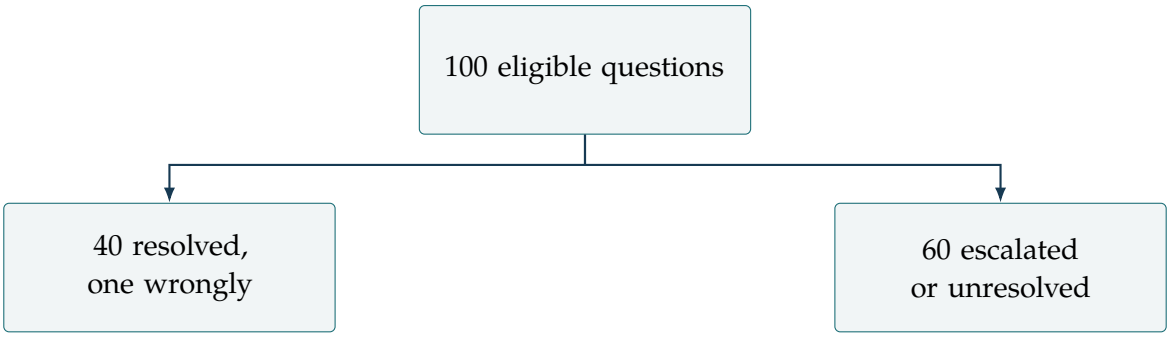


Figure 9.10: Selective risk and useful coverage need separate denominators.

In this synthetic illustration, the error rate among resolved questions is 1/40, or 2.5 percent. The fraction resolved is 40 percent; correctly resolved coverage is 39 percent if every other resolved result meets the reference standard. Reporting only one error in 100 would hide the abstention policy's effect.

Alternative-reading recall measures whether the report contains the reference's material supported alternatives. It should be assessed by meaning and consequence, not string matching. Alternative precision measures how many proposed alternatives are judged relevant and supported. These measures expose a method that increases recall by overwhelming reviewers with arbitrary possibilities.

Resolution quality has a different denominator: detected discrepancies. Measure whether the selected action obtains the needed evidence, produces a valid repair, correctly escalates, or falsely closes the issue. A successful HTTP response is not a successful resolution. The action's declared question and expected evidence type make this classification possible.

Report completeness is largely an engineering metric. It checks that every issue, role, candidate disposition and resource outcome appears. It can be tested exhaustively over bounded controller fixtures. It does not establish that the issues themselves cover every material legal question. Keeping these metrics separate prevents a perfect report-rendering score from being presented as perfect interpretation accuracy.

9.8 Severity and direction of error need explicit treatment

Not every error has the same consequence. Missing a prohibition may permit an impermissible offer; inventing a prohibition may block a legitimate activity or mislead the bank about its obligations. Both are errors relative to the accepted interpretation, even if one appears operationally conservative. The evaluation should report their directions rather than collapsing them into a single accuracy number.

Severity categories must be defined before scoring. They can consider whether the error changes an action, affects many clients, is hard to detect downstream or undermines the source and approval chain. Assigning weights is a policy choice requiring the bank's

input. The study can report unweighted counts and separate severity strata until those weights are approved.

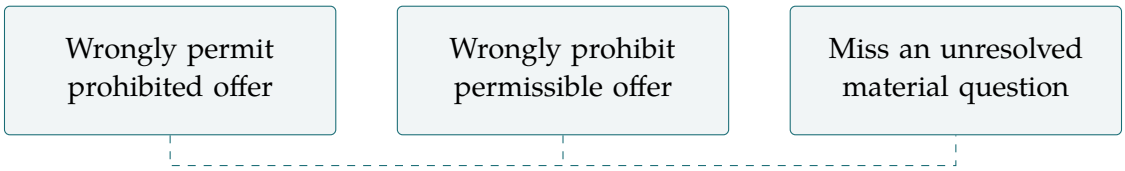


Figure 9.11: Error direction and severity must remain visible beside average accuracy.

A weighted average can hide a rare severe error among many trivial successes. Therefore severe false clean outcomes should also be reported as individual cases with their mechanisms. A predeclared severe-error veto can block promotion even when average performance is favorable. That veto should not automatically stop all research: if the failure reveals a repairable omission detector, the next planned repair phase may remain justified.

The report should distinguish an invalid experiment from a failed candidate. Leakage of test labels into prompts invalidates the comparison and may require rebuilding the evaluation. A candidate missing an exception on a valid heldout case is evidence against that candidate's readiness. An unavailable provider is an operational failure that may prevent estimating interpretation performance. These outcomes should not all be summarized as "the experiment failed."

Reviewer workload also has severity. A report containing hundreds of unsupported alternatives may conceal the important issue through overload. Measure review time, number of evidence inspections, unresolved-question quality and corrections made by reviewers. A method that catches more errors but makes the review process unmanageable needs a different presentation or search policy, not an unqualified claim of superiority.

9.9 Zero observed errors does not mean zero risk

Suppose a study observes no errors in n independent, identically distributed trials, where $n \geq 1$, with unknown error probability p. Choose $0 < \alpha < 1$. Under this simplified binomial model, the probability of seeing zero errors is $(1 - p)^n$. A one-sided upper confidence bound at level $1 - \alpha$ solves $(1 - p)^n = \alpha$, giving

$$p_{\text{upper}} = 1 - \alpha^{1/n}. \tag{9.1}$$

For a 95 percent bound and $n = 100$, this is approximately 2.95 percent. With $n = 1000$, it is approximately 0.30 percent. These calculations show why a small perfect test set cannot establish a very small error rate.

The independence assumption is substantial. One hundred scenarios derived from one circular do not generally behave like one hundred independent circulars. Shared word-

ing, definitions and prompts create clusters. The calculation is therefore illustrative unless the trial definition and sampling design justify the model. It should not be applied to the 729 truth-table assignments as a legal error bound.

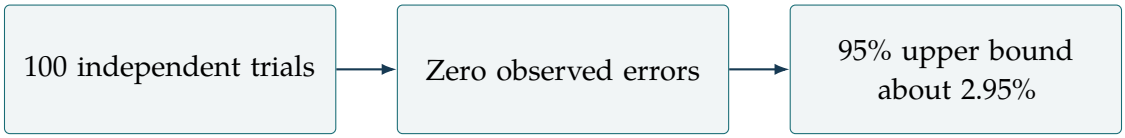


Figure 9.12: A finite perfect sample does not imply zero population risk.

For method comparisons, paired evaluation on the same source questions reduces irrelevant variation. A question-level difference records whether one method finds a material issue that the other misses. Confidence intervals or permutation procedures should preserve the document or question clustering appropriate to the sampling design. Resampling individual rows while ignoring their shared circular can produce overly narrow intervals.

9.9.1 Why the implemented comparison counts source families

The two branches of the gift FAQ share wording, authority and an interpretation problem. Treating them as independent observations would give repeated use of one source the appearance of replication. The implemented comparison first averages outcomes within a source family, then compares methods on those family averages. Unsafe acceptance and useful accepted answers are measured separately. For the former, a lower rate is desirable; for the latter, a higher rate is desirable. A system that abstains on everything has no useful accepted answers.

For one of these rates, let X_i be the proposed method's rate minus the single reader's rate in family i. Each rate lies between zero and one, so $-1 \leq X_i \leq 1$. There are $n \geq 1$ families, with mean difference $\bar{X} = n^{-1} \sum_{i=1}^{n} X_i$. Assume independent family sampling; dependence within a family is allowed. The quantity estimated is the expected average family difference under that sampling design, with equal weight per family. It is not the average per client or per transaction.

The interval calculation asks how far this average might lie from its expected value. Its two rates are defined against the study's reference outcomes: a faulty reference would compromise the comparison before any statistical calculation begins.

To bound a large positive deviation, temporarily give larger deviations more weight. For family i, subtract its expected difference, EX_i, and weight that deviation by $\exp[t(X_i - EX_i)]$, where $t > 0$ controls the emphasis. Here E denotes an average over repeated family observations under the assumed sampling design. The exponential makes large positive deviations count more; its logarithm will let us turn products from independent families into sums. Define $g_i(t) = \log E \exp[t(X_i - EX_i)]$.

At $t = 0$, the exponential is one, so $g_i(0) = 0$. Differentiating once gives the average centered deviation under these weights, so $g_i'(0) = 0$ too. Differentiating again gives the

variance under the same normalized weights. Reweighting changes probabilities, not the range of possible values: that range still has length two. Its squared distance from the interval midpoint is at most one, and variance cannot exceed that average squared distance. Hence $g_i''(t) \le 1$. Integrating from zero gives $g_i'(t) \le t$ and then $g_i(t) \le t^2/2$.

If the average deviation exceeds $r > 0$, the exponential of the summed deviations exceeds $\exp(tnr)$. Markov's inequality bounds the probability that a positive quantity exceeds a threshold by its expectation divided by that threshold. Independence lets the expectations of the family exponentials multiply; taking logs adds the bounds $g_i(t)$. Consequently,

$$P(\bar{X} - E\bar{X} \ge r) \le \exp(-ntr + nt^2/2),$$
$$P(|\bar{X} - E\bar{X}| \ge r) \le 2\exp(-nr^2/2). \tag{9.2}$$

The first exponent is minimized at $t = r$. Applying the same calculation to the negative tail gives the second line. There are six comparisons to protect: three methods against the single reader, for each of the two rates. The union bound adds their error probabilities. Choosing an overall error allowance $0 < \alpha < 1$ and solving $12\exp(-nr^2/2) = \alpha$ yields the common radius

$$r = \sqrt{\frac{2\log(12/\alpha)}{n}}, \qquad [\bar{X} - r, \bar{X} + r] \cap [-1, 1]. \tag{9.3}$$

The intersection respects the possible range of a rate difference. With one family and $\alpha = 0.05$, the radius exceeds three; every resulting interval covers the full range from minus one to one. The development fixture therefore cannot distinguish the methods through this calculation. More variations of that same family do not increase n.

This is a conditional probability argument, not a guarantee supplied by an API. A convenience sample does not acquire a population interpretation from the formula. A funded study still needs a justified sampling scheme, defensible reference outcomes and enough independent families for its intended precision. Conformal prediction, the method for constructing sets of possible answers developed in Section 9.11, addresses a different question. It has not been deployed as a guarantee of this comparison.

Multiple stochastic generations require multiple seeds or repeated provider runs, with the seed and configuration recorded where supported. Differences from one run are descriptive. If a model API does not guarantee reproducibility from a seed, the study records independent request repetitions and their limitations. The retained responses permit audit even when exact regeneration is impossible.

Tail measures such as maximum review time or the 99th percentile of action cost need particularly cautious interpretation. A few difficult circulars can dominate them, and small samples give unstable estimates. They are useful diagnostics and capacity-planning signals, but a superiority claim requires a predeclared analysis with sufficient observations and uncertainty evidence.

The evaluation report should state which candidates failed hard safety or integrity screens, which remain viable, whether any ranking is statistically supported, and which

differences are descriptive only. Passing a hard screen is not evidence that one viable method is better than another. This discipline is especially important when the apparent gain comes from a small number of rare severe errors.

9.10 Calibration asks a narrower question than truth

A calibrated score has a defined relationship to an observed event in a target population. For example, among cases assigned a particular probability range, the fraction agreeing with the independently reviewed reference should be consistent with that range, within sampling uncertainty. Calibration is not a property conferred by using the word confidence in a JSON field.

The target event must be precise. "The interpretation is correct" is too broad unless the reference process, source perimeter and acceptable alternatives are specified. A more measurable event might be "the proposed paragraph-level rule has no material discrepancy against the adjudicated reference for this corpus." Even that event depends on the quality and scope of the reference.

Calibration data must be separate from the data used to tune the score and from the final evaluation set. A score trained on one circular family may be miscalibrated on another, especially when definitions, authority structure or source quality change. Model and prompt updates can also invalidate an earlier calibration. The calibration record therefore binds the method and population versions.

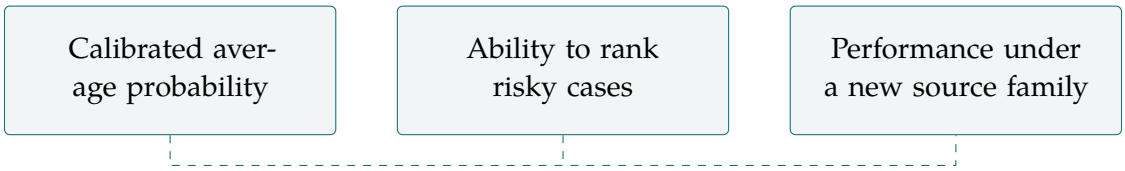


Figure 9.13: Calibration, discrimination and transfer are different properties.

Reliability diagrams and proper scoring rules can diagnose probability estimates, but a visually plausible diagram with few cases is weak evidence. Calibration also does not ensure discrimination: a model that assigns the base rate to every case can be calibrated yet unhelpful for selecting review priorities. The study should assess both risk ranking and calibration, with uncertainty intervals.

Semantic entropy can nominate uncertain answers, as discussed in Chapter 5. Its published evaluations use particular datasets and correctness proxies (Kuhn et al., 2023). A high area under a ranking curve in those tasks does not establish calibrated legal error probabilities. In the workbench, entropy may help prioritize review after a target-specific study, while mandatory source and release checks remain categorical requirements.

Low entropy is particularly dangerous under common-mode omission. Every sampled answer can concentrate on the same incomplete reading. The independent inventory and source review remain necessary even if an uncertainty score appears excellent on

average. A useful evaluation explicitly includes these unanimous-error cases rather than assuming that disagreement accompanies every mistake.

9.11 Conformal prediction can support sets under stated assumptions

The statistical assumption needed here is *exchangeability*: rearranging the calibration examples and the new example does not change their joint distribution. It rules out, for example, treating a new family of documents as comparable merely because it arrived after the calibration set.

Conformal prediction offers a principled way to construct prediction sets from calibration examples (Angelopoulos and Bates, 2022, section 1.1 and appendix D). It is relevant because the proposed product should sometimes return several candidates rather than a forced answer. Its guarantee, however, concerns a specified statistical prediction problem under exchangeability. It is not a theorem about all possible interpretations of English.

To make the mechanism concrete, suppose each task has a finite reviewed label space $\mathcal{Y}$, such as a small set of predetermined interpretation classes. A fitted scoring function $s(x, y)$ assigns larger values when label y looks less compatible with task x. The scoring function is fixed before calibration. For n calibration tasks with reviewed labels (X_i, Y_i), compute $S_i = s(X_i, Y_i)$.

To select a threshold, arrange the calibration scores from smallest to largest. Write $S_{(k)}$ for the score in position k. Choose the tolerated rate of missed true labels, α, between zero and one. The ceiling brackets below mean rounding up to the next integer. With $n \geq 1$, the rule is

$$k = \lceil (n+1)(1-\alpha) \rceil, \qquad \widehat{q} = \begin{cases} S_{(k)}, & k \leq n, \\ +\infty, & k = n+1, \end{cases} \tag{9.4}$$

The threshold $\widehat{q}$ therefore admits labels whose incompatibility scores are no larger than the selected calibration score. When the required position is beyond the available sample, it uses an infinite threshold. Write $\Gamma(x)$ for the set of admitted labels for a new task x:

$$\Gamma(x) = \{y \in \mathcal{Y} : s(x, y) \leq \widehat{q}\}. \tag{9.5}$$

The finite-sample correction uses $n+1$, not simply the empirical $1-\alpha$ quantile with an unspecified interpolation rule. The inclusive inequality also matters when scores tie.

Conditional on the separately fitted scoring rule, if the n calibration examples and the new example are exchangeable, the position of the new true-label score among the $n+1$ scores is symmetric. In the no-tie case, its rank is uniform. Excluding it requires a rank beyond k, whose probability is at most α by the definition of k. With ties and inclusive sets, the basic coverage statement is conservative. This gives marginal coverage over a new exchangeable example and the calibration sample.

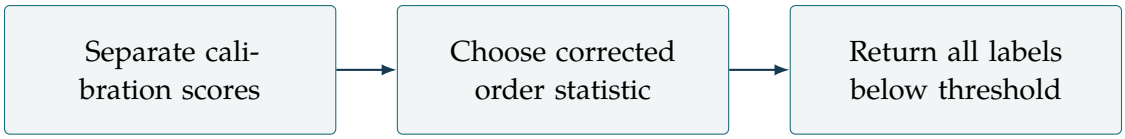


Figure 9.14: Conformal sets concern a fixed prediction problem under exchangeability.

With four calibration cases and a 90 percent target, the required rank is five. The construction therefore uses an infinite threshold and can return the entire label space. This is the honest consequence of the small sample, rather than evidence that a narrow legal answer has 90 percent reliability.

For $n = 19$ and $\alpha = 0.1$, $k = 18$, so the threshold is the eighteenth ordered calibration score. Suppose that score is 0.8 in a synthetic example. For a new task, labels scored 0.2 and 0.7 enter the set; a label scored 0.9 does not. The threshold chooses a set rather than forcing a winner between the first two. For $n = 4$ and $\alpha = 0.1$, $k = 5$, requiring the infinite threshold and potentially the entire label space. The latter example makes a practical point: a very small calibration set cannot support a narrow set with a strong finite-sample coverage target through this construction.

The guarantee does not say that each particular circular has a 90 percent chance of containing the right reading in its returned set. Nor does it guarantee coverage conditional on every rare ambiguity type. Marginal coverage can coexist with poor performance on a small important subgroup. The distinction between marginal and conditional coverage is discussed explicitly in the tutorial (Angelopoulos and Bates, 2022, section 3) and is central to a high-stakes application.

The label space is another limitation. If the generator has never proposed a material reading and the label space contains only generated candidates, a set selection method cannot include that missing reading. One can add an "unrepresented interpretation" or escalation outcome to a reviewed task formulation, but that changes the prediction problem and requires reference data. Conformal calibration does not establish semantic completeness of open-ended candidate generation.

The appropriate early experiment is therefore narrow: use a fixed, reviewed classification task or a defined error-detection score, calibrate on separate source families where exchangeability is defensible, and measure set size, coverage and subgroup behavior. Source amendments, provider changes and shifts in document type should trigger renewed assessment. The inspected official tutorial code is useful for checking quantile and set construction, but its image examples are not a ready-made legal calibration protocol.

9.12 Search coverage and semantic completeness are different

A controller can report that it explored every node in a finite generated graph. That is a meaningful computational fact. It does not establish that the graph contains every

supported legal interpretation. The generator, inventory and construction operators determine what enters the graph. Evaluation must examine both the search procedure and the generation boundary.

For each reference alternative, record whether it was generated, retained, investigated, correctly dispositioned and represented in the final report. This stage-by-stage accounting locates the failure. If an alternative was never generated, changing the scheduler may not help. If it was generated but pruned by a global beam before investigation, family preservation may be useful. If it was investigated but falsely defeated by a weak argument, the evaluation criterion needs repair.

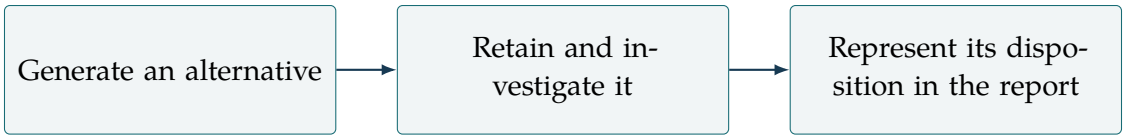


Figure 9.15: Stage-by-stage measurement locates where an interpretation was lost.

The same accounting applies to omissions. A source unit can be missed at acquisition, extraction, inventory, normative classification or rule mapping. Treating all such failures as model reasoning errors hides the actionable cause. The ensemble's modular design makes it possible to test which redundant activity actually contributed detection.

Ablations remove one component at a time under comparable budgets: the independent inventory, the alternative reader, the argument critic, the witness generator or the no-progress scheduler. If removing a component changes both resources and information access, the interpretation of the ablation is limited. The study should state whether it measures the component's total contribution or its value at fixed cost.

Search completeness claims must name the finite object. "All twenty-four retained candidates were compared on the declared Boolean domain" is testable. "All legal readings were considered" is unsupported. The report should also expose the unexplored frontier, discarded unsupported proposals and resource-limited comparisons so that reviewers can judge whether more investigation is justified.

9.13 Evaluate the reviewer, not only the model

The final product is a workbench used by people. A study should therefore compare review with and without the ensemble dossier, using a design that controls for case difficulty and learning effects. Reviewers should not see the same case in both conditions in an order that makes the second condition trivially easier. A counterbalanced or appropriately randomized assignment can reduce this problem.

Outcomes include detection of material defects, quality of the final rationale, time to a defensible decision, number of unnecessary escalations and ability to identify remaining uncertainty. Faster review is valuable only if the substantive quality is preserved.

A dashboard that encourages reviewers to accept a misleading clean badge can reduce time while increasing risk.

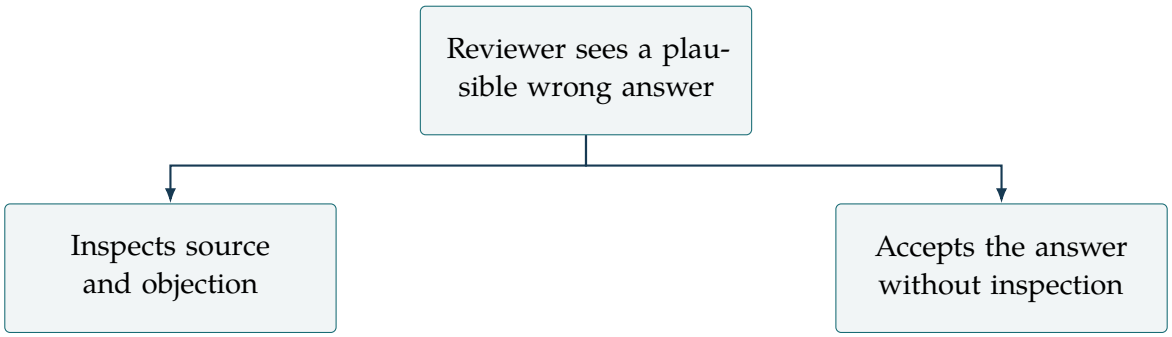


Figure 9.16: Usability evaluation should examine automation bias in consequential cases.

The study should inspect automation bias directly. Include cases where the leading model answer is wrong, where a minority candidate is right relative to the reference, and where no single answer is justified. Observe whether reviewers inspect the opposing argument and source. The purpose is not to trick staff, but to learn whether the interface supports the intended skeptical review.

Reviewer feedback also tests self-containment. Can a compliance reader understand the formal rule through its controlled-language rendering and examples? Can an engineer identify the required Java behavior without consulting an external paper? Can both explain why a solver result does not resolve an uncertain definition? These comprehension tasks are more informative than asking whether the report looks professional.

Any study involving actual bank staff, confidential materials or production workflows requires the bank's normal authorization and data controls. The public prototype can first use synthetic tasks and qualified independent readers. Its findings should be reported at that level. A successful public-source usability exercise is not evidence that every private-bank process has been validated.

9.14 Economic value and operating cost

The business case should use measured labor and maintenance effort. Let $K > 0$ be the initial cost of constructing the selected corpus, workbench and integration. Let M be the average manual cost of a comparable completed change, and let A be its assisted cost including human review, model use, operation and allocated maintenance. If those costs remain comparable across changes and $M > A$, the simple break-even count is

$$n_{\text{break-even}} = \frac{K}{M - A}. \tag{9.6}$$

This follows by comparing manual cost nM with assisted cost $K + nA$ and solving $nM = K + nA$. For a whole number of changes, the first cost-saving or cost-neutral count is

the ceiling of this ratio. It is a planning relation, not an estimated return. If $M \leq A$, this labor-saving model alone supplies no break-even case. Material-error prevention, audit response and amendment speed may have additional value, but they should be measured separately rather than assigned unsupported monetary benefits.

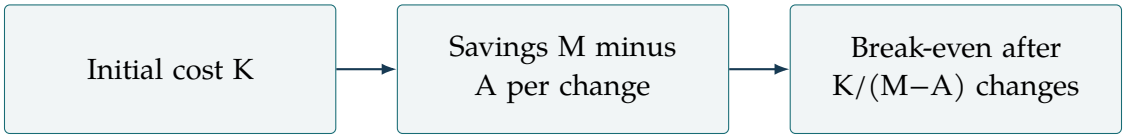


Figure 9.17: The planning relation requires positive savings on comparable changes.

For synthetic costs K = 100,000, M = 2,000 and A = 1,500 in the same currency, the break-even count is 200 comparable changes. If assisted cost rises to 2,100, this labor-saving calculation has no positive break-even point. Avoided errors would require separate evidence and valuation.

Reusable definitions can lower later costs, while new product classes and ambiguous amendments can raise them. The prototype should record both effects. Paper benchmark token costs and tax penalties are not substitutes for the bank's staffing costs, risk definitions or change volume. A credible investment decision may be to continue only for a narrow class of rule changes where the measured benefit is clear.

9.15 Promotion decisions and the next evidence needed

The study should predeclare a small set of promotion criteria and hard vetoes. A severe false clean result, broken source identity, uncounted external action, unauthorized release or incomplete mandatory report can veto promotion. Average runtime, model score and cost are explanatory until the correctness and integrity conditions are met. A method cannot trade away a required release invariant for better throughput.

Among methods that pass the hard screens, ranking requires the predeclared uncertainty analysis. If the data do not distinguish them, report them as viable under current evidence. A descriptively favorable method may justify a longer study or an optional feature, but not a new default merely because its mean score is highest in a small sample.

Consider two methods that both pass the quality screen. Suppose one takes less review time in the observed cases, but the uncertainty interval still includes no difference. Both remain candidates for further evaluation; the faster observed mean alone does not justify replacing the current method. If a critical omission is then found in one candidate, that finding changes its eligibility, regardless of its speed. A useful result report makes the distinction explicit: what passed, what failed, what remains uncertain and which next step follows. The recording fields are retained in Section E.7 of the technical companion.

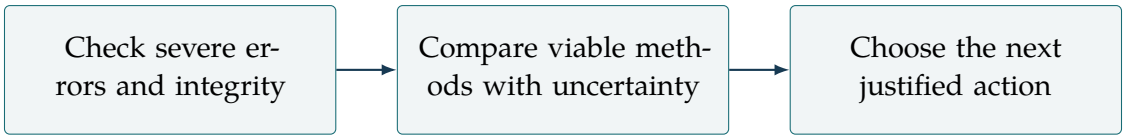


Figure 9.18: A favorable average cannot override a predeclared severe-error veto.

A failed candidate should trigger the repair that its failure supports. Missing source units suggest improving inventory and extraction diversity. Losing a minority interpretation suggests changing family retention. Repeated ineffective actions suggest a better question-selection policy. A contaminated reference requires repairing the evaluation before comparing methods again. These are specific next steps, not reasons to declare the entire research direction either successful or impossible.

Every serious run should retain a manifest with source and code identities, commands, environment, model/provider versions, configurations, seeds or request repetitions, wall time, resource use, plan and result paths. The result note should include the strongest alternative explanation and the evidence that would overturn its conclusion. This makes the evaluation reproducible enough to guide product decisions rather than merely produce an attractive benchmark table.

The implemented controller has live development observations and controlled integration checks, and the comparative evaluation machinery has been exercised. Its empirical legal-interpretation performance remains to be established on unfamiliar source families with defensible reference outcomes. The next chapter describes how the bank can preserve that distinction through shadow operation, source changes and ongoing review.

Chapter 10

Operating the product without losing its evidence

A control can preserve its source and pass its tests, yet still fail when the wrong legal entity sends it data or an amendment silently changes a past decision. The bank needs people who can recognize those failures, stop the affected use and reconstruct what happened. These responsibilities determine how a supervised prototype can become an operating control.

10.1 The bank remains responsible for the control

The workbench can organize evidence, expose alternatives, verify consequences and prevent certain unauthorized transitions. It cannot assume the bank's legal or managerial responsibility. The operational design must identify who owns source applicability, interpretation, factual mappings, software correctness and release. A system that assigns every difficult question to a generic "human review" queue has not completed that design.

Meaning review belongs to people authorized to assess the relevant regulatory requirement. Data owners establish what evidence supports facts, how long it remains valid and how corrections are represented. Engineering owners establish that the software preserves the approved specification. Release owners verify the package, approvals and operational readiness. Business owners determine the workflow response to unknown or blocked results within the bank's approved policy.

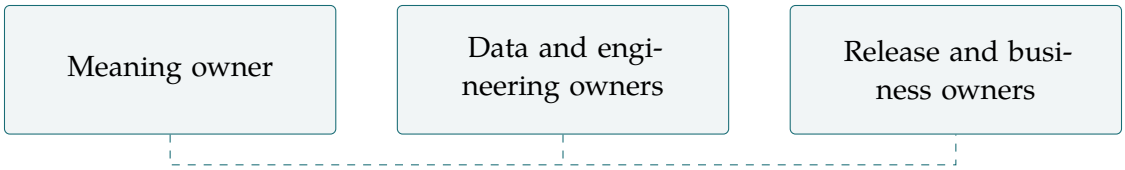


Figure 10.1: Responsibility must identify the decision each person is authorised to make.

These responsibilities can overlap institutionally, but conflicts and separation requirements must be explicit. An author should not be able to approve its own interpretation through a second display name. A model-generated reviewer identity has no authority. The local prototype's test roles exercise the mechanics; the bank deployment must bind them to enterprise identity and actual delegated responsibilities.

The product's value is not that it eliminates judgment. It makes the judgment specific and attributable. Instead of asking a reviewer to bless a long generated summary, it asks which definition applies, why an exception attaches to a given condition, or whether a source is current for the activity. The final control then carries those decisions into its executable form and future amendment review.

10.2 What prevents reliance on the present product for bank decisions

The present evidence does not establish that LegalMath is suitable for autonomous legal or compliance decisions in a bank. The code implements useful local controls, but independent legal adjudication, performance on unseen regulatory questions and the production identity, data and operational arrangements are unfinished. The issue is not a residual percentage that can be inferred from 205 passing tests. Several necessary components of the intended use have not yet been demonstrated.

This conclusion has consequences for product design. A proposed release needs a named legal entity, regulated activity, source perimeter and decision that the bank permits the software to influence. Review must distinguish the authority governing that use from the technical evidence about the software. Financial penalties, client harm and remediation duties cannot be estimated from a software test count. Their assessment depends on the actual breach and applicable legal and supervisory requirements.

10.2.1 AI governance belongs in the regulatory perimeter

The SFC's 12 November 2024 circular on generative AI language models addresses licensed corporations' relevant use of such models, including third-party components. It covers senior oversight, validation and ongoing management, information security and provider risk. Its validation discussion includes systems that combine a model with retrieval, prompts or other components. The notification provisions concern the specified high-risk uses and the existing notification requirements; they do not create a blanket approval statement for every AI application (SFC, 2024a, paras. 6, 9–16, 20–31). An applicable-use assessment is therefore required before transferring these expectations to a particular bank entity or workflow.

The accompanying appendix identifies failure modes relevant to this architecture, including retrieval that misses necessary information and prompt injection. It also discusses data and third-party risks. A citation-producing retrieval system still needs review of its source support (SFC, 2024b, appendix, paras. 9–12, 16–19).

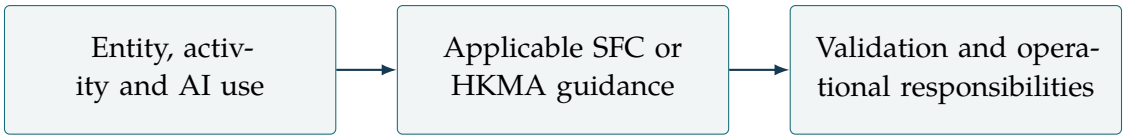


Figure 10.2: Governance obligations follow the actual use and institution.

The HKMA's 19 August 2024 circular specifically addresses consumer protection in customer-facing generative-AI applications at authorized institutions. It sets out governance, fairness, transparency and data-protection expectations, including human control and review arrangements in the described circumstances (HKMA, 2024, pp. 1–4). LegalMath's internal authoring workbench and a customer-facing explanation service are different uses. Enabling the latter would require an additional applicability and control assessment. This cited circular does not by itself determine the full risk-management requirements for an internal bank system; its referenced guidance and other applicable requirements need their own inventory.

For example, a synthetic control may correctly identify a failed wealth condition but give a client an unsupported explanation about legal ineligibility. That is a different harm from an incorrect internal Boolean calculation. The product needs approved customer explanations, an accountable review route and a record linking the explanation to the actual decision if such a use is introduced. A technically accurate trace is a starting point for that design.

10.2.2 Identity, access and evidence integrity need production controls

The inspected local implementation registers synthetic identities and their roles, and binds reviews to exact content. This is sufficient to exercise separation of author, meaning reviewer and engineering reviewer in tests. It does not establish an employee's current delegated authority, employment status, multi-factor authentication or revocation. A bank identity integration must revoke a departing reviewer's future authority while preserving the historical record of decisions genuinely authorised at the time.

Access requires a separate design. Authentication of a registered local user does not establish that the user may inspect every client's evidence or every legal entity's case. Production permissions must apply to source packets, candidate reports, exports and factual evidence, including indirect retrieval through linked records. A test should attempt to retrieve a second entity's case through every relevant interface and verify the intended denial. The current local API is not evidence that this separation has been implemented.

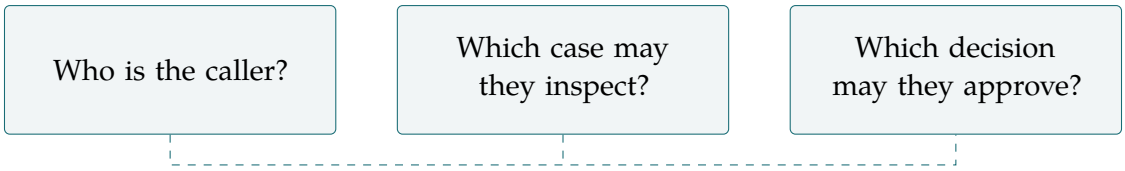


Figure 10.3: Authentication, access and decision authority require separate answers.

The audit chain is stored in the same locally administered environment as the database it records. Its hashes can expose changed records when the expected chain remains trusted. An administrator who can replace both records and the chain can defeat that comparison. The bank design therefore needs an independent integrity reference, such as appropriately controlled signed checkpoints or external immutable retention, with reviewed key custody and recovery. This is a design requirement arising from the trust model, not a claim that one named technology is mandated by every regulator.

Consider a synthetic incident in which a privileged operator replaces an approval and recomputes the later hashes. A verifier that trusts only the rewritten database can find an internally consistent history. An independently retained earlier checkpoint would instead expose the divergence. Testing accidental blob corruption does not exercise this privileged-rewrite scenario.

10.2.3 Abstention needs an operational destination

An unresolved interpretation is useful only if the affected process handles it. Every material issue needs an owner, an evidence request, a permitted interim action and a review deadline appropriate to the bank's workflow. The system must detect an unowned issue or an overdue queue rather than allowing a blocked screen to become routine ignored noise. An override needs defined authority, scope and expiry; it must not silently amend the underlying regulatory meaning.

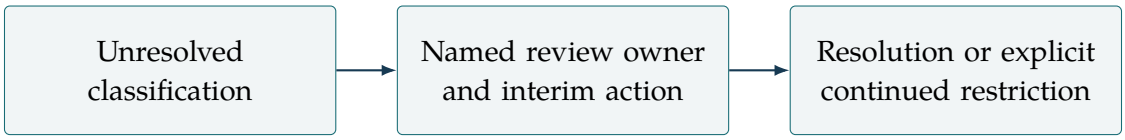


Figure 10.4: Abstention transfers a specific decision; it does not finish the operating process.

The pure Java evaluator also needs a production release-withdrawal protocol. An offline library can continue executing after its central approval is retired unless the host checks the applicable release policy. The bank must define how revocation reaches hosts, the permitted age of cached authority, behavior during disconnection and treatment of in-flight transactions. A release revoked for wrong meaning raises a different fallback question from a model-provider outage while an approved rule remains applicable.

For a synthetic race, an order is assessed just before a release is suspended, then reaches commit after suspension. The host must apply its reviewed policy to that transition and record the version used. Merely retaining the JAR hash does not decide whether the order may complete. The existing in-memory host example motivates this test but does not validate a bank's distributed order and deployment infrastructure.

10.2.4 Confidentiality and incidents cross the model boundary

Public regulatory documents and synthetic examples permit development without client data. Production work can introduce personal information, confidential advice and privileged legal analysis. The bank must identify which material can leave its environment, for what purpose, under which provider terms, and with what retention, deletion and access arrangements. Privilege and cross-border handling require qualified assessment for the actual arrangement. Encrypting transport alone does not answer those questions.

A source parser and a model provider also present different attack surfaces. The parser needs operating-system isolation and resource bounds for hostile files. The provider needs a narrow task interface and cannot inherit approval, network or database privileges from text it reads. The complete application must be tested with malicious source instructions, altered attachments and cross-case references. The focused security, release-binding and host checks rerun for this review passed 18 tests; that result covers those cases only.

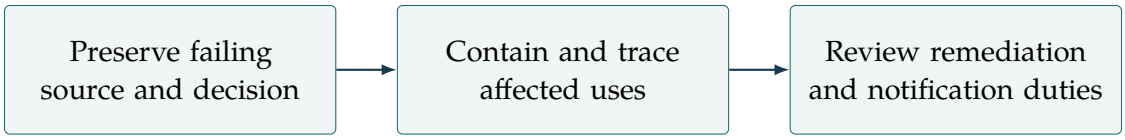


Figure 10.5: Incident response requires evidence, operational action and legal assessment.

Finally, an incident procedure must assign authority for suspension, client remediation, regulatory-notification assessment and preservation of relevant records. The product should expose affected decisions and their dependencies without deciding the bank's external communications. Rehearsing a missing exception, a compromised approval and a stale release tests three different response paths. These exercises, independent legal review and a measured shadow study are necessary next evidence for a bounded deployment decision. The current document and local tests cannot substitute for them.

10.3 Start with a narrow operational perimeter

A first deployment should name the activity, legal entity, products, client or counterparty categories, source families and decision points it supports. This perimeter is not

a marketing description. It determines which circulars enter the workbench, which factual schemas are required and which host workflows can consume the resulting package.

For example, a paragraph-10 incentive-control pilot could focus on reviewing a single component of a proposed offer before it is launched. It would need a reliable definition of the component, evidence of its product relationship and a reviewed classification process for gift and fee discount. It would not by itself cover every marketing, disclosure or distribution requirement. The host must continue to route the offer through other applicable controls.

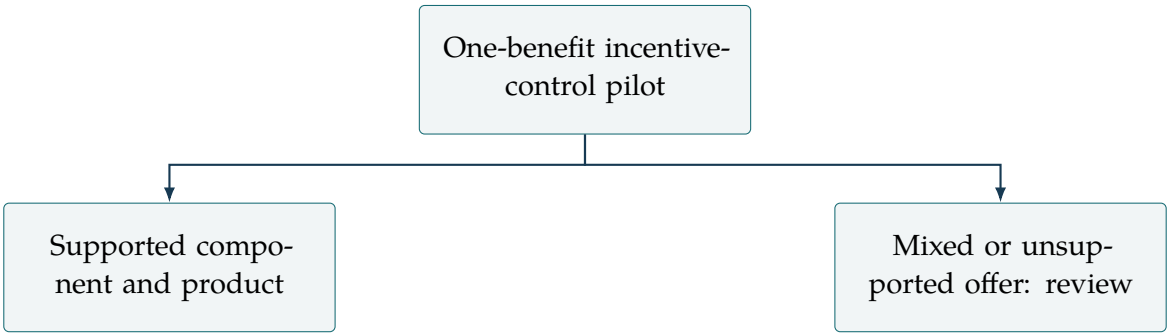


Figure 10.6: A narrow perimeter must be enforced in the input and workflow.

Narrowing the perimeter is useful when it removes an unsupported ambiguity by an explicit operational restriction. If the bank excludes mixed offers from the automated pilot and routes them to manual review, the system can assess simpler cases under a clearer subject definition. That restriction must be enforced by input validation and workflow routing, not merely stated in a project memo. Otherwise mixed offers will eventually arrive and receive an unintended answer.

The perimeter also defines negative tests. Inputs from another actor, product type or jurisdiction should produce the specified out-of-scope or unsupported-profile result. They should not be coerced into the closest known category. A controlled failure at the boundary is preferable to a plausible answer about the wrong activity.

Expansion requires evidence. Adding a new circular family may introduce temporal obligations, different authority relationships or vocabulary that the original schema does not support. The team should create a new source and method review, update the factual mappings, and evaluate the expanded task. Success on a narrow pilot is a reason to consider expansion, not proof that transfer is automatic.

10.3.1 Scope, team and deliverables

The planning assumption is one full-time implementation engineer, a part-time technical lead, a compliance specialist available for roughly two days per week, and part-time product/operations and independent review support. A formal-methods specialist should review the semantics and solver adapter at defined points. These are resource

assumptions to validate at initiation, not estimates derived from measured implementation velocity. Reduced compliance availability should reduce scope or extend the schedule rather than silently transfer legal decisions to engineers.

The corpus should begin with the cited documents and follow the dependencies needed for the selected rules. The first discovery phase will choose a tractable slice, provisionally around 30–50 atomic rules and 150–250 synthetic or appropriately approved historical scenarios. These ranges are planning hypotheses, not known counts of requirements in the circulars. The corpus must include thresholds, exceptions, missing evidence, actor distinctions, dates and at least one continuing obligation. Entire related clause families should be reserved for held-out evaluation to reduce leakage from almost identical examples.

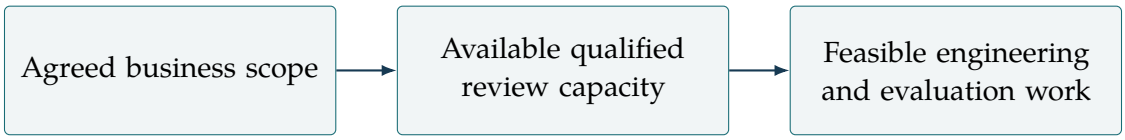


Figure 10.7: Staffing assumptions constrain the scope that can responsibly be completed.

The demonstrable deliverable is a local workbench with source comparison, typed rules, examples, explanations and an amendment view. Its export is a reviewable package with generated Java, a compiled JAR, a deterministic decision API and evidence records. The initial runtime need not call any bank service: public text and synthetic client facts can demonstrate the full path. Integration design should identify the future data feeds and control owners, while actual access follows the bank's environment and information-handling requirements.

10.4 Keep regulatory meaning distinct from bank policy

A bank can choose a more restrictive operational policy than the minimum condition identified in a circular. It may decline an uncertain promotion, require an additional approval or retain a manual check. These decisions should be represented as bank policy with their own owner, rationale and effective interval. They should not be folded into the regulatory rule and later cited as if the SFC required them.

This separation improves both accountability and maintenance. If a regulatory interpretation changes, the team can see whether the bank policy still makes sense. If a business owner changes an internal approval threshold, it does not appear that the regulator amended the circular. Reports can explain a blocked offer as blocked by bank policy pending classification, rather than falsely declaring it prohibited by the source.

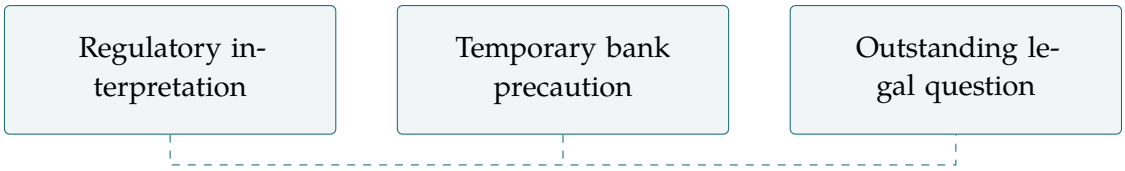


Figure 10.8: A conservative bank policy must retain its own authority and rationale.

The runtime can evaluate both kinds of controls, provided the bundles identify their authority and purpose. The host's orchestration specifies how their results combine. A regulatory condition returning false may still be blocked by a bank policy; a bank policy returning permissive cannot override a regulatory prohibition. The combination rules need their own reviewed semantics and tests.

An unresolved interpretation should not be hidden in a conservative default. Suppose the team treats every cashback as a gift until clarification. That can be a cautious temporary policy, but the uncertainty report should still state that the legal classification remains unresolved. Otherwise future reviewers may mistake a risk-management choice for settled authority and propagate it into other products.

Temporary policies need expiry or review triggers. An indefinite "pending clarification" rule can become permanent through neglect. The workbench should record the question, owner, next review condition and affected activities. A reminder is an operational aid; it does not automatically extend the policy or resolve the underlying source issue.

10.5 Shadow operation is a study, not a hidden release

Shadow operation runs the proposed control alongside the existing process without letting its result directly determine the transaction. It allows the team to observe real input quality, classification workload, latency and disagreement. The scope and access to real data require the bank's normal authorization. Shadow results must be clearly distinguished from approved production decisions.

A useful shadow study records the existing process's decision, the workbench's result, the facts available to each and the timing of those facts. When they disagree, an independent review examines the reasons. The existing process is not automatically the truth oracle; manual procedures can be wrong. Equally, the workbench is not correct merely because it produces a more detailed explanation. The study needs an adjudication process consistent with Chapter 9.

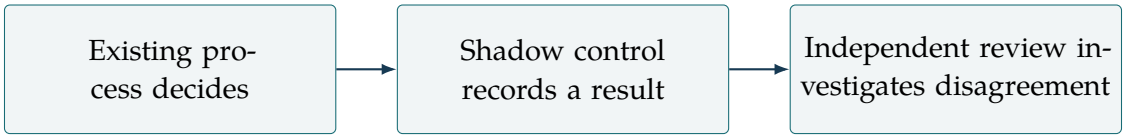


Figure 10.9: Shadow operation gathers evidence under an authorised study.

The team should predeclare what would justify promotion, what would veto it and what would trigger a repair. A severe false clean result might block promotion while motivating a targeted fix. A data-mapping defect might invalidate a subset of observations until corrected. A provider outage may reveal operational fragility without answering the interpretation question. These distinctions keep the study from becoming a succession of improvised demonstrations.

Shadow operation also reveals whether the selected unit of assessment matches business practice. If staff enter a whole campaign while the rule expects one benefit component, the system may receive apparently valid but semantically wrong inputs. The remedy may be a workflow redesign, not a more capable model. The interface should help staff identify and evidence components consistently.

Reports from shadow operation remain subject to access, retention and correction rules. They can contain sensitive facts and preliminary judgments. A later adjudication should append a correction rather than rewrite the original observation. This preserves the study's evidence and prevents an apparently perfect retrospective record from being created by editing away disagreements.

10.6 Source monitoring begins a new interpretation cycle

The workbench should monitor the source families in its approved perimeter for new publications, revisions and incorporated-material changes. Discovery creates a candidate source record; it does not automatically establish current applicability or overwrite the active rule. The acquisition process retains bytes, metadata, retrieval time and the relationship to prior versions.

A textual difference is only the first signal. A changed definition may affect many controls even when their operative paragraphs are unchanged. A formatting change may have no substantive consequence but still change the file hash. The impact service therefore combines byte identity, extracted text differences, source-unit mapping and the explicit dependency graph.

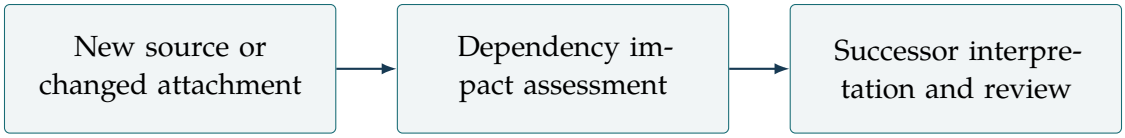


Figure 10.10: Source monitoring starts a new review; it does not silently replace an active rule.

The first impact report identifies which candidates, facts, rules, duties, reports and releases depend on the changed material. It should distinguish direct citations from indirect definition dependencies. A control can be affected through an incorporated code provision that is not quoted in its main explanation. This is why dependency closure was required before initial interpretation.

An unchanged structural dependency report is not a proof of semantic equivalence. Conversely, a changed paragraph number does not prove a changed obligation. Semantic comparison and legal review are separate actions. The ensemble can propose interpretations of the change and construct scenarios on which old and new rules differ, but a reviewer must establish the intended effective regime.

Every changed rule version requires the relevant review, build and verification steps again. Reusing a previously approved JAR because its human-readable title looks the same would break the evidence chain. Reusing a verified runtime library can be legitimate if its identity and supported semantics remain unchanged, while the new policy bundle and generated binding receive fresh evidence.

The local monitoring implementation has now exercised this sequence in a controlled example. The first gift rule omitted a fee-discount exception; its successor included that exception. For the supplied discount facts, the selected restriction changed from true to false in a separate Java host process. An unrelated control kept its answer, and the earlier packages remained available. This shows a source-and-rule update reaching the intended executable control without replacing the historical record.

The amendment was supplied as part of the exercise. It was not an independently interpreted new regulatory announcement. The demonstration therefore establishes local update and replay behaviour, while the bank must still provide its actual control register, fact owners, service identity, scheduler and release roles. The implemented scheduler also retains unfinished checks and completed but unresolved interpretations when the next visit finds no source change. An unchanged document cannot cure either problem by itself.

10.7 Amendment, actor and conditional language: tokenised products

The SFC's 20 April 2026 tokenisation circular replaces the November 2023 circular; the earlier public entry is marked accordingly. A source collector that keeps only a URL and the most recently extracted text cannot reliably reconstruct which requirements an older decision used. A collector that retains both but fails to record replacement can apply inconsistent generations together (SFC, 2023c, 2026c).

The 2026 circular also provides useful clause-level challenges. Paragraph 10 keeps responsibility for the tokenisation arrangement and ownership records with product providers despite outsourcing. Paragraph 18 addresses distributors, including providers distributing their own products. These are actor-specific requirements: the word "bank" is not a sufficient applicability condition. Paragraph 13 conditions the use of public-permissionless networks on additional proper controls; translating it into an unconditional technology ban changes the statement (SFC, 2026c, paras. 10, 13, 18).

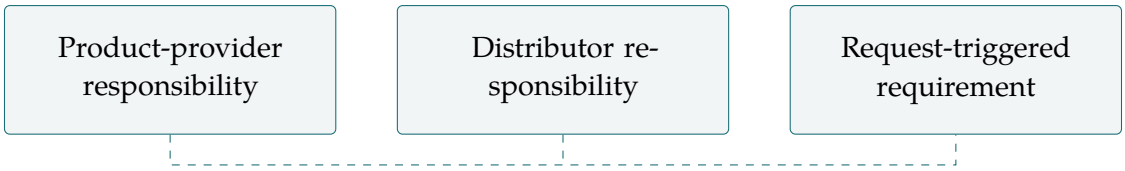


Figure 10.11: Actor and trigger distinctions survive a tokenisation rule change.

Paragraphs 14–16 illustrate several different triggers. Confirmation and demonstration upon request are distinguishable actions. Third-party verification and a legal opinion are conditioned on an SFC request in the cited paragraphs. Removing that trigger creates a different obligation; removing the obligation creates the opposite error. A request must be represented as an event, with its subject and responsible actor, and any deadline must come from a relevant source or separately identified operational policy. The product must not invent a generic deadline (SFC, 2026c, paras. 14–16).

Prior consultation and prior approval are also different. Paragraphs 20–21 and footnote 8 distinguish new products, tokenisation of existing products, material changes, and circumstances concerning tokenised share classes. Turning "consultation required" into "approval obtained" skips an action and invents a result. The proposed workbench will expose such verbs and their dependencies as separate obligations and evidence fields (SFC, 2026c, paras. 20–21 and n. 8).

10.7.1 Amendments and the release package

Every source revision should produce both a textual change view and an impact view. The former identifies changed passages, annexes and links; the latter follows dependencies to interpretations, definitions, rules, tests, data mappings and deployed controls. The 2023 and 2026 tokenisation circulars form a useful initial replacement case. A replacement marker is evidence about the document relationship, but application dates and treatment of existing arrangements still require the relevant source and review (SFC, 2023c, 2026c).

Where old and new executable rules cover the same domain, the workbench should find representative cases on which their outcomes differ. Where a new rule introduces an additional fact or actor, it should identify the new data dependency and the cases that cannot yet be evaluated. A textual difference alone cannot measure operational impact, and an unchanged threshold cannot establish that a definition or exception is unchanged.

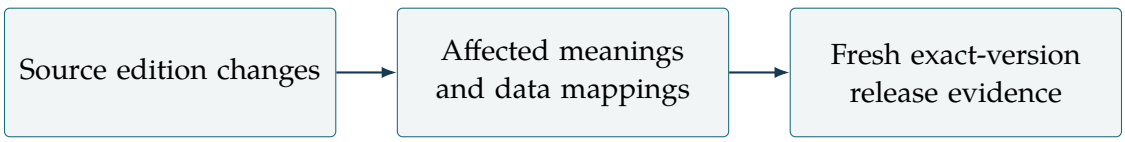


Figure 10.12: An amendment's impact follows dependencies, not only changed numbers.

The release package should contain the source snapshot, interpretation record, rule version, unresolved exclusions, approved examples, test and solver results, adapter version, data mappings and owner approvals. Decisions made with that package should record its identifier and input snapshot. Dependency analysis can identify tests and interpretation evidence that remain relevant after a change. Every changed bundle nevertheless needs fresh approval of its exact hash and a new evidence report; a result for an older hash cannot authorize its release. The historical record remains intact. This distinction permits reuse of work without obscuring what needs fresh review.

10.8 A worked amendment scenario

Consider a synthetic amendment used only to explain the process. The original selected rule contains a fee-discount exception. A later source adds a condition requiring a particular relationship between the discount and the transaction. This is not a claim that the SFC has made such an amendment; it is a controlled example of how a change would propagate.

The source custodian retains the new document and identifies the added condition. The inventory maps it to the earlier exception and opens an interpretation run. Candidate A treats the condition as applying to all offers assessed after the new commencement date. Candidate B distinguishes offers already committed before that date under a possible transition provision. The source packet must include that transition provision if one exists; the system cannot infer grandfathering from business preference.

The scenario designer creates offers before and after commencement, an offer agreed before but performed after, and an offer with incomplete timing evidence. These examples reveal whether the competing readings differ. The formal checker can compare their executable conditions once the timing concepts are defined. It cannot determine which transition interpretation the source supports without the relevant authority.

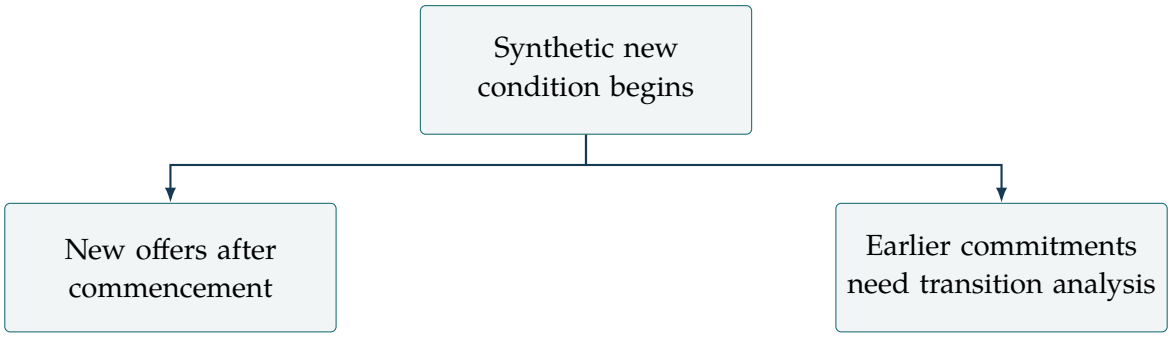


Figure 10.13: An amendment may raise a separate question about existing arrangements.

When reviewers accept the new interpretation, the data owner confirms that the required relationship and timing facts can be supplied. Engineering builds and verifies the new Java package. The release manifest binds its effective interval and applicabil-

ity profile. The host's release selector is tested at the exact boundary, including the excluded endpoint of the earlier interval.

Existing event obligations require separate treatment. If a duty was opened under the old bundle, the current MVP preserves that binding. A legal decision may require migration, continuation or another treatment. The workbench records the question and any approved migration specification. No background update silently changes the source version attached to a historical duty.

The old source, interpretation, JAR and decisions remain available for replay. A later investigation can ask what the system would have decided under the old rule with the evidence known then, or what a corrected assessment under the new rule would conclude. Those are different questions and their results carry different assessment contexts.

10.9 Bitemporal replay explains rather than rewrites history

An incident needs two replay questions. The first asks what the bank could have decided on Monday: select the rule and facts whose valid time covered the transaction and whose knowledge time was no later than Monday's assessment cut-off. The second asks what the bank should decide after a Wednesday correction: retain the same Monday valid time but allow evidence recorded on Wednesday. The two-time model keeps those questions separate; valid time describes when a fact or rule applies, while knowledge time describes when the system could have used it.

Suppose the Monday classification marked a benefit as a fee discount and a Wednesday review finds evidence that the classification should have been a gift from Monday onward. Monday replay reproduces the original input and execution. The corrected assessment may change the result, but it must be linked as a new assessment with a new knowledge cut-off. The difference identifies whether the incident concerns the rule, the data, the process or delayed information; it does not make either replay retroactively answer the other question.

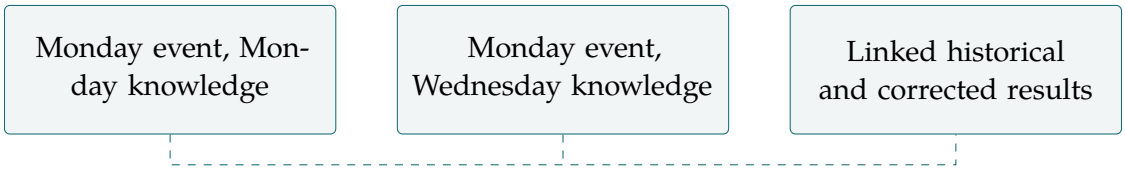


Figure 10.14: Bitemporal replay keeps the question behind each assessment explicit.

The system should label the corrected assessment and link it to the original. It should not replace the original execution hash or pretend that Wednesday's evidence was available on Monday. This distinction supports fair investigation of whether the failure concerned the rule, data, process or delayed information. It also identifies which other decisions relied on the same evidence.

Replay requires the exact historical runtime or a verified equivalent path, the bundle, fact snapshot, event history where relevant, calendars and assessment context. Retaining only a final status and a current source URL is insufficient. The retention policy must preserve these dependencies for the bank's required period; this book does not invent a statutory retention duration.

Reproducing an old decision does not endorse it. A replay can faithfully show that an earlier rule omitted an exception. That is useful evidence about the incident. The corrected rule and reassessment should be linked, with the reason for change and any operational remediation. Traceability supports accountability precisely because it can preserve an error honestly.

10.10 Operational responses to unknown and conflict

Unknown and conflict are ordinary outcomes in a system built around evidence. They should have designed workflows rather than being treated as exceptional software crashes. The host policy identifies the responsible queue, information request, permitted temporary action and escalation deadline for each reason. These choices depend on the bank's process and must be approved there.

A missing classification might route to a product specialist. Conflicting fee records might route to the data owner. An unavailable incorporated source might require legal review before any automated decision. An unsupported RuleIR operator requires engineering and meaning review together. The reason code helps select the remedy, but the workflow should expose the substantive question in ordinary language.

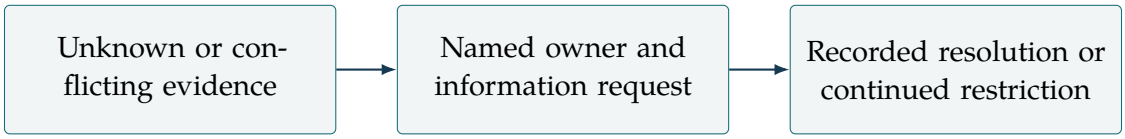


Figure 10.15: A review queue needs an operational owner and a next action.

A conflict should preserve both records and their provenance. Automatically taking the newest record can hide a disagreement between systems or an unauthorized correction. If the bank adopts a precedence rule, that rule must be explicit and reviewed for the evidence type. It should not be embedded in a generic database query that happens to order by timestamp.

Manual overrides need their own authority and scope. An override may authorize a specific operational action under bank policy while leaving the regulatory interpretation unresolved. The record should say exactly what was overridden, by whom, for which case and period, and with what rationale. It must not change the underlying source or make future cases automatically inherit the exception.

Queues themselves can fail. Unreviewed items can accumulate, deadlines can pass, and staff can misunderstand a blocked status. Monitoring should therefore track age,

owner, materiality and next action for unresolved issues. A low model error rate is little comfort if the operational process routinely ignores the issues that the model correctly escalates.

10.11 Risks that shape the implementation

The most serious risk is a wrong but internally consistent interpretation. Its controls are independent source review, exact evidence links, explicit alternatives and cases designed from the circular. Formal verification strengthens the implementation step but cannot repair that source error automatically.

The second risk is incomplete scope. A well-modeled clause can depend on an uncollected definition or a different entity's obligations. The selected corpus therefore needs an applicability and dependency inventory, with unresolved imports blocking release of affected rules. The discovery service should distinguish "not found" from "does not exist."

The third risk is operational mismatch. A correct financial expression supplied with stale valuations or an event history with missing withdrawals can still produce a harmful result. The prototype should make data freshness, actor identity, event ordering and reconciliation visible. A later bank pilot needs named owners for those feeds and a documented response to uncertainty.

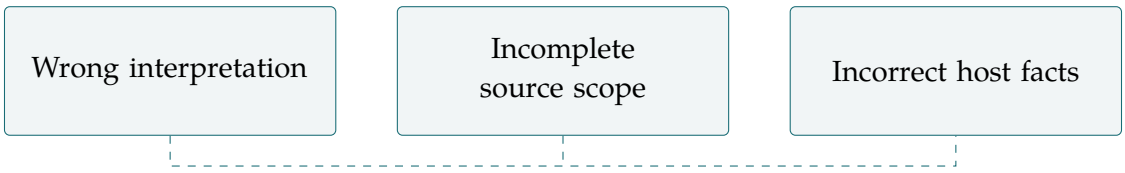


Figure 10.16: Different failure mechanisms require different controls.

The fourth risk is creating a maintenance burden greater than the one removed. Keeping one typed specification with generated views reduces duplication; adopting many engines at once increases it. Pin tool versions, preserve proof and test dependencies, and add backends only when a specific integration or assurance question justifies them. Sources remain accessible if a research tool becomes unavailable.

Public sources and synthetic cases suffice for this prototype. If later work uses client information, deployment location, access control, retention and model access must follow the bank's applicable requirements. This is an architectural constraint on that later integration, not a reason to delay the current public source study. The initial decision is whether the evidence justifies such a pilot, with a defined scope and accountable owners.

10.12 Incident analysis should locate the failed boundary

When an incorrect control or decision is discovered, the first question is where the evidence chain failed. Was the wrong source acquired? Was a qualification missed? Was the accepted interpretation encoded incorrectly? Did the host supply misclassified facts? Did a transaction race invalidate the snapshot? Did an unauthorized or stale package reach production? These mechanisms require different repairs and different assessments of affected decisions.

The incident record should preserve the exact failing example, source versions, candidate, approvals, JAR, input snapshot, result and operational action. It should also preserve the strongest competing interpretation that was available at the time. If the objection was present but suppressed, the remedy concerns reporting and review as well as generation quality.

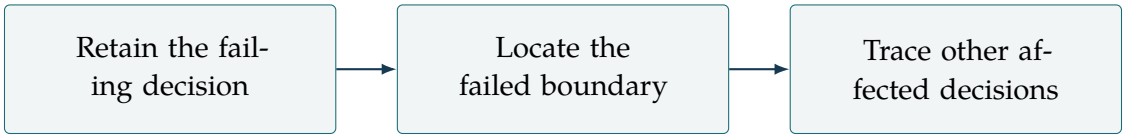


Figure 10.17: Incident analysis follows the causal dependency of the error.

Containment is a bank decision. It may involve suspending a particular offer category, returning to an approved manual process, disabling a faulty adapter or replacing an implementation while retaining the same accepted rule. The incident team must distinguish these options from reverting to a legally superseded rule. The workbench supplies impact information; it does not decide the bank's external communications or regulatory response.

Root-cause analysis should avoid a generic conclusion that "the AI hallucinated." That phrase can conceal a missing source, a contaminated oracle, an undefined classification or a budget policy that forced premature closure. A specific mechanism supports a specific preventive control. The discovered failure becomes a retained regression scenario, with its source and interpretation status clear.

The team should also ask what similar decisions may be affected. Dependency and lineage queries can identify every package using the disputed definition and every assessment using the relevant fact mapping. A database search by final status alone will miss decisions that happened to be correct for other reasons. The scope of remediation follows the causal dependency, not merely the visible outcome.

10.13 Availability failures must not change the meaning of a result

The production Java evaluator should not depend on a live model or source website. That separation lets an approved control continue to execute during an interpretation-provider outage, subject to the bank's release and source-monitoring policy. An outage in the workbench still matters for new controls and amendment review, but it need not make every existing decision nondeterministic.

The host must distinguish an unavailable interpretation service from an unknown fact and from an expired release. These conditions can require different actions. An approved, still-applicable local package may remain usable when a research provider is unavailable. A release whose effective status cannot be established presents a different problem. The fallback policy must state the conditions under which each path is permitted.

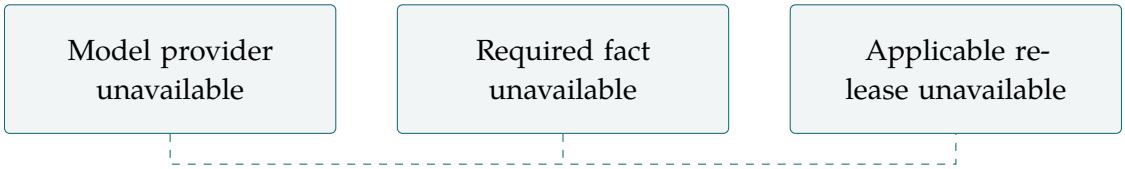


Figure 10.18: Availability failures must remain distinct from substantive false results.

No fallback may convert a technical error into a substantive false result. If a Java evaluation fails because the snapshot is malformed, returning false would imply that the selected condition was evaluated and not satisfied. The correct result is an error with an operational disposition. Likewise, a missing interpretation report cannot be replaced by a default approval flag.

The ensemble controller has its own recovery obligations. A restart recovers reserved and dispatched actions, preserves their charges and determines which results are known, late or unknown. It does not restart the investigation from zero unless an authorized successor run is created. The supervisor monitors absolute deadlines independently of the worker's progress messages.

Capacity planning should consider the difficult tail: large dependency graphs, many genuine alternatives and long-running external checks. Graph and action limits protect the service, but they create explicit incomplete outcomes. The operational owner needs to know how often those outcomes occur and whether the manual review process can absorb them. Increasing limits without measuring cost and review quality is not a complete capacity strategy.

10.14 Model updates are method changes

A new model version can change which alternatives are generated, how sources are summarized and which questions are prioritized. Even if the JSON schema remains unchanged, the interpretation method has changed. The workbench should bind provider, model, prompt and configuration versions to each run and evaluate material updates before making them the default.

The update review asks whether known omission cases remain detected, minority readings remain visible, unsupported alternatives increase, and uncertainty scores retain their measured relationship to the reference. It also checks provider behavior relevant to budgets, such as retries, tool use and token accounting. A faster model that silently changes these behaviors can invalidate the controller's operational assumptions.

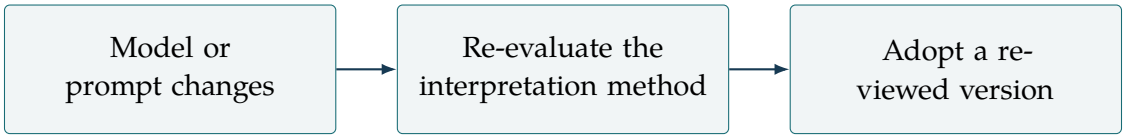


Figure 10.19: A provider update can change the method without changing the JSON schema.

Existing approved Java packages do not need to be regenerated merely because the proposal model changes. Their authority derives from the retained interpretation, review and verification evidence. A new model may identify a concern that triggers review, but its existence alone does not invalidate every prior decision. This separation limits unnecessary churn while allowing genuine discoveries to enter an accountable amendment or incident process.

Prompt changes deserve similar treatment. Adding a leading example can improve performance on one family while anchoring all members to the same interpretation. Removing a request for counterarguments can reduce cost while weakening diversity. The change record should explain the intended mechanism, evaluation evidence and remaining uncertainty. A prompt is part of the method, not incidental prose.

Search-policy changes also require evidence. Switching from a deterministic scheduler to MCTS, changing family quotas or increasing a no-progress threshold alters which questions receive attention. The study should compare these choices under equal resources and independent judgments. Passing controller invariants is necessary for either scheduler, but does not establish which investigates law more effectively.

10.15 A release readiness review with concrete evidence

A readiness review should inspect a concrete package, not approve a future promise to generate one. The package contains the accepted interpretation dossier, source cov-

erage, factual mapping, verification report, build and release manifests, operational dispositions and unresolved-risk record. All identities must agree. An approval of one candidate cannot be reused for a different generated bundle.

The meaning reviewer checks that the selected source perimeter and assumptions support the rule, that plausible alternatives have a reasoned disposition, and that any remaining uncertainty is compatible with the proposed scope. The data owner checks input definitions, evidence quality, freshness, corrections and calendar or valuation policies. Engineering checks exact behavior, traces, resource limits and integration. The release owner checks authority, deployment mechanics, monitoring and recovery.

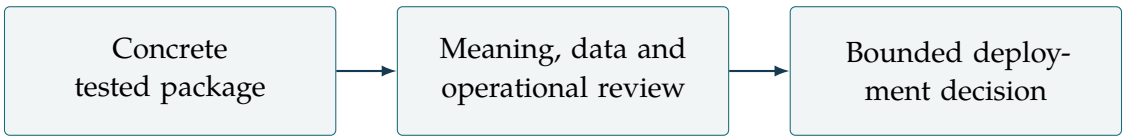


Figure 10.20: A release review inspects evidence for the exact proposed use.

A useful readiness review asks five plain questions. Has the bank accepted the meaning? Can it establish the needed facts? Does the implementation preserve that meaning? Is someone authorized to own the release? Can staff act when the answer is missing or blocked? The detailed evidence list is retained in Appendix E of the technical companion.

Table 10.1: Five questions before a bounded deployment decision.

Question for the responsible owner	Evidence to examine
Is this reading justified for the activity and date?	Source passages, alternative readings, objections and the legal reviewer's reasons.
Do the records establish the facts that the rule needs?	Definitions, ownership, dates, freshness and unresolved conflicts.
Does the program preserve the accepted rule?	Comparisons with the reference calculation, boundary cases and the exact tested version.
Are the responsible people authorized to approve its use?	Named owners, role assignments, approval decisions and audit records.
Can staff act when the control cannot decide?	Tested responses to missing or conflicting evidence, simultaneous events and incidents.

A missing item should produce a specific block and next action. It should not trigger a vague demand for more confidence. If a classification is unresolved, obtain the authority or narrow the scope. If Java traces differ, repair the implementation. If enterprise identity is not integrated, complete that integration before relying on local synthetic roles. The package makes these distinctions visible enough to assign work.

A review may approve a limited deployment with explicit exclusions and monitoring conditions. Those conditions become enforceable configuration and expiry or review triggers. They must not remain only in meeting minutes. The system should be able to show which operational path enforces each exclusion and which test would fail if it were bypassed.

10.16 Three operational cases for the implementation team

The first case is a missing definition. The source packet is intact, the candidate formula compiles, all named scenarios pass, and the ensemble agrees on the expression. An incorporated definition is unavailable. The correct report is blocked for the affected classification, even though the engineering checks pass. The next action is to obtain or adjudicate the definition, not to add more random truth-table tests.

The second case is a wrong host mapping. Reviewers accepted a component-level rule, but the host sends one Boolean classification for an entire mixed offer. The Java evaluator faithfully applies its inputs and returns a reproducible result. The error lies at the subject and fact boundary. The remedy includes a component-aware adapter, affected-case analysis and a regression scenario that would expose a whole-offer mapping. Re-proving the Boolean evaluator alone would not address the failure.

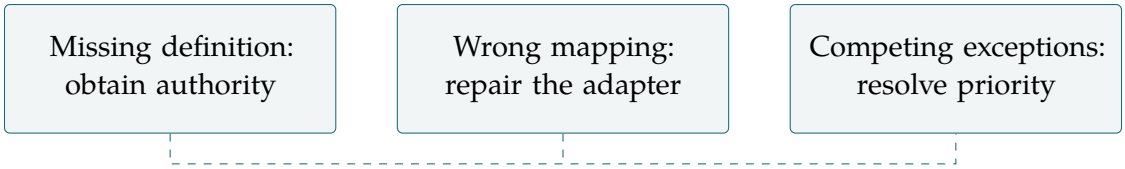


Figure 10.21: Three review cases require three different next actions.

The third case is an unresolved exception priority. Two exception guards are true. The current RuleIR default returns conflict, while a developer proposes to choose the first exception in the array. That change would make the program run, but it would introduce a priority that the specification did not authorize. The workbench must preserve conflict and ask whether the source establishes priority, whether explicit nesting is justified, or whether the rule lies outside the supported fragment.

These cases demonstrate why completion criteria must follow the claimed target. In the first, source evidence is incomplete. In the second, the application mapping is wrong relative to the accepted interpretation. In the third, a proposed implementation change would alter semantics. The same dashboard label “needs review” is not enough; the system must name the responsible boundary and the evidence needed to repair it.

A team rehearsal should walk through each case using the actual interface and reports. Staff should be able to locate the source, identify the affected candidate, understand the block and record a decision without editing hidden configuration. Rehearsal findings

become product changes and acceptance tests. A successful technical build does not substitute for this operational exercise.

10.17 Maintaining a useful literature and method record

The research record should let a new team answer why the control was designed this way. If the team changes the treatment of an exception, it needs to find the source argument, the version inspected and the local assumptions used in adapting it. A citation without that account can identify a paper while leaving the design choice unexplained.

The bibliography and reading notes distinguish published results, local derivations and proposed adaptations. Catala's treatment of defaults, Stipula's state and resource semantics, argumentation frameworks, tree search and uncertainty methods answer different questions. Combining them in one architecture does not transfer every theorem or empirical result to the combination. The method record states which mechanism is borrowed and what must be re-established locally.

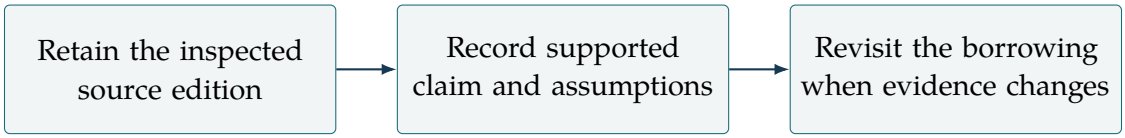


Figure 10.22: A useful literature record connects a mechanism to its local justification.

Official code is valuable evidence about practical assumptions. The semantic- clustering example in Chapter 5 shows why code and paper can require separate inspection. A pinned repository revision may implement a broader condition than the prose suggests. The correct response is to document and test the difference, not silently choose the version that makes the proposal look stronger.

The literature record should also preserve inaccessible or uninspected leads. They can guide future work, but cannot support categorical implementation claims. The present review is a substantial bounded survey of the retained sources, not a declaration that no relevant paper exists outside the library. New work can change the best engineering choice, especially in a rapidly developing area.

ResearchAssistant is useful for acquisition, parsing and source structure. MathDevMCP is useful for named formal obligations. DynareMCP contributes patterns for explicit hypotheses, falsifiers and bounded investigations. Their reuse should remain concrete and versioned. The bank should be able to identify which function, data structure or proof obligation is being borrowed, rather than relying on a broad claim that another project already solved the difficult part.

10.17.1 How to turn the citation network into a development program

The richest use of this literature is to connect each mechanism to a failure the bank can recognize. Catala and Logical English address inspectable definitions, while date semantics and CUTECat address difficult boundaries. LegalRuleML, s(LAW) and eFLINT address context, unknowns and obligations; Stipula and process-compliance work address histories. ContractCheck and SMT techniques address contradictory requirements, and the neurosymbolic papers address the fallible translation boundary. Each contribution can therefore be tested against a specific part of the bank's problem.

For each borrowed mechanism, the prototype should retain one source example, one SFC analogue, the assumptions that changed, and a test that would reveal a bad transfer. A Stipula event-ordering example becomes a consent-withdrawal trace; a Catala exception example becomes the fee-discount exception; an ARc ambiguity case becomes two proposed readings with a concrete client case on which they disagree. If the mechanism cannot represent the bank case without hidden conventions, that is evidence to revise the adapter or keep the case under human assessment. It is not a reason to force the circular into the language.

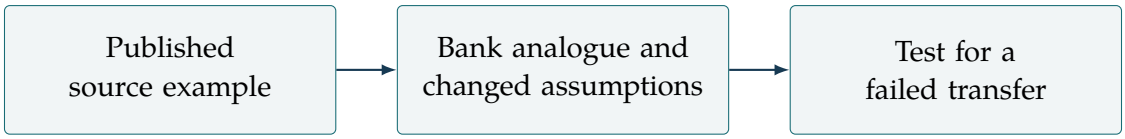


Figure 10.23: A development program borrows mechanisms through explicit comparisons.

Citation expansion should continue during development around those unresolved questions. New papers enter the library with versioned full text and technical notes, and software enters through a small reproducible example. This preserves the benefits of a broad research base while keeping the product centered on reviewable regulatory change.

10.17.2 Keeping the research usable after the original team leaves

A future reviewer should be able to recover the edition of a paper or circular that supported a design choice and find the relevant passage. The retained collection therefore keeps the source documents together with technical reading notes. Those notes distinguish a paper's theorem from an implementation, a reported experiment from a reproduced result, and a public repository from an approved bank dependency. Where extraction loses a table or mathematical symbol, the original rendered source remains the reference.

The collection also preserves earlier source versions and the reasons for changes in interpretation. This is what makes an amendment review cumulative: the next team can examine the earlier reasoning instead of trying to reconstruct it from an unexplained

citation. File naming and reproduction instructions are given in Appendix F of the technical companion.

10.18 Decisions needed at the start of a bank prototype

The first decision is the precise business perimeter: legal entity, activity, client class and product family. The second is the owner of each interpretation and the independent review process for disputed cases. The third is the mapping from regulatory definitions to actual bank records, including freshness and missing-evidence treatment. These decisions should be made on the selected source slice, with worked examples, rather than as abstract architecture questions.

The initial technical commitment should be modest and testable: one source repository, one typed decision semantics, one bounded event model, one review interface and one deployment adapter. The literature gives substantial guidance for each part. It also explains why a universal translation-and-proof promise would be unsound. A successful prototype would show which interpretations are adopted, which computations follow them, which facts support the result, and where human judgment is still required.

10.19 A practical sequence from manuscript to product

The local prototype now covers source-driven generation, investigation of alternatives, source checks, explicitly grouped controls and draft Java delivery. Live development supplied some of the retained proposals; controlled and replayed exercises tested the subsequent integration. The marketing example remains a running case, while settlement readiness exposes the difficulty of assembling separate duties. Both retained investigations have open questions. Their programs and supporting records are useful for review without becoming approved controls.

The next empirical step is defensible reference construction and a complete comparative evaluation on unfamiliar source families. The team freezes source inventories and judgments before seeing model outputs, runs the baseline ladder and records uncertainty. Failures are classified by boundary and used for planned repairs. Agreement with the author's original 22 expected answers alone would not justify using a model for unfamiliar circulars.

In parallel, institution integration must supply identity, approved data access, isolated source parsing, provider controls, signed or equivalently authenticated releases, monitoring ownership and host transaction behavior. Local packaging and a separate Java host have already been exercised; the bank's own environment and authority to release remain to be supplied. This work can proceed alongside method evaluation where dependencies permit. It should not be postponed until a final demo, because operational

constraints may change which designs are feasible.

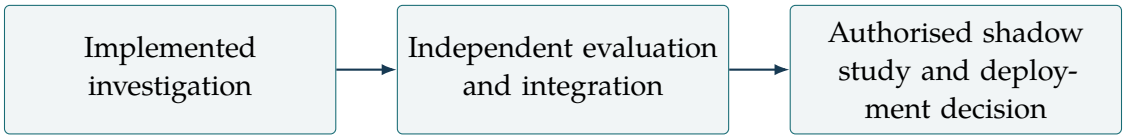


Figure 10.24: The development sequence preserves the distinction between software evidence and bank authority.

The fourth step is authorized shadow operation in a narrow activity perimeter. The study compares the workbench with the existing process and independently adjudicates disagreements. It measures both substantive quality and reviewer workload. The team rehearses source amendments, unknown facts, provider outages, worker crashes and rollback decisions.

The fifth step is a bounded deployment decision based on the concrete evidence package. The decision may approve only a subset of controls or retain manual review for disputed categories. Monitoring then continues the evidence process: new sources, unexpected disagreements and incidents create successor investigations. Deployment is not the end of interpretation assurance.

This sequence does not assign an arbitrary calendar duration or staffing number. The bank's source perimeter, access constraints and review capacity determine those estimates. The task dependencies and acceptance evidence are concrete enough to build a schedule once those inputs are known. Inventing a confident delivery date without them would add apparent precision without resolving the underlying work.

10.20 What the prototype establishes and what must be measured

The prototype now makes an interpretation investigable as well as executable. It generates alternatives, retains source-based challenges, compares programs where their facts can be aligned and combines separately scoped duties. Its local Java demonstration carries a controlled amendment into a changed answer while preserving the earlier package. Its completion records distinguish an unfinished check from a completed investigation whose meaning remains unresolved. These are implemented mechanisms with bounded development evidence.

A successful independent interpretation study can establish empirical performance on its declared task population, with uncertainty and limitations. It may support a bounded use or a choice among methods. It cannot prove that every future circular will be interpreted correctly, especially under new vocabulary, authority or distribution shift. A formal proof of the controller's release invariant similarly depends on the recorded evidence and does not prove that no source obligation was ever missed.

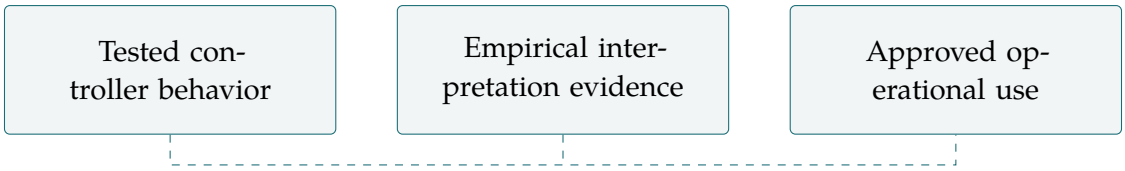


Figure 10.25: Completion at one stage does not establish the next stage's claim.

The workbench makes interpretation, transcription, coding and review inspectable, introduces checks with different opportunities to detect mistakes, and preserves questions that remain open. Whether that reduces material errors or review effort must be measured against the bank's actual process, using a fair reference and comparable resources. The current work does not supply the bank's incident rate or establish that business benefit.

The high stakes justify both ambition and precision. The system should actively search for ways its preferred reading could be wrong, retain those challenges, and stop automatic work honestly when the evidence or budget is insufficient. It should deliver code whose relationship to an approved specification can be checked, while keeping the approval of meaning visible as a separate act. That combination is a concrete engineering objective and a defensible basis for building the next version of LegalMath.

Bibliography

Daneshvar Amrollahi, Jerry Lopez, and Clark Barrett. Faithful Autoformalization via Roundtrip Verification and Repair, 2026. URL https://arxiv.org/pdf/2604.25031v2. Reviewed preprint v2, 9 May 2026.

Chenyang An et al. A Neurosymbolic Approach to Natural Language Formalization and Verification. arXiv:2511.09008, 2025. URL https://arxiv.org/pdf/2511.09008v2. First posted 2025; reviewed v2, 13 July 2026.

Anastasios N. Angelopoulos and Stephen Bates. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511, 2022. URL https://arxiv.org/pdf/2107.07511v6. Reviewed v6, 7 December 2022; first posted 2021. Split-conformal method, marginal/conditional coverage, shift discussion and appendix D proof inspected.

Grigoris Antoniou, David Billington, Guido Governatori, and Michael J. Maher. Representation Results for Defeasible Logic. arXiv:cs/0003082, 2000. URL https://arxiv.org/pdf/cs/0003082. Reviewed 2000 preprint; journal version published in 2001.

Joaquín Arias, Mar Moreno-Rebato, José A. Rodríguez-García, and Sascha Ossowski. Automated legal reasoning with discretion to act using s(LAW). Artificial Intelligence and Law, 2024. URL https://arxiv.org/pdf/2401.14511. 2024 arXiv copy; journal first published online in 2023.

Tara Athan, Guido Governatori, Monica Palmirani, Adrian Paschke, and Adam Wyner. Legal Interpretations in LegalRuleML. CEUR Workshop Proceedings, 1296, 2014. URL https://ceur-ws.org/Vol-1296/paper2.pdf. Workshop paper.

Bank of America Corporation. 4.750 percent Non-Cumulative Preferred Stock, Series SS: Prospectus Supplement. SEC filing, 2022a. 27 January; retained issuer PDF; selected terms on pages S-16–S-28.

Bank of America Corporation. Certificate of Designations of 4.750 percent Non-Cumulative Preferred Stock, Series SS. SEC Exhibit 3.1, 2022b. Effective 31 January; retained web extraction; original HTML retrieval unavailable.

Barclays PLC. 7.625% Fixed Rate Resetting Perpetual Subordinated Contingent Convertible Securities: Prospectus Supplement. https://home.barclays/content/dam/home-barclays/documents/investor-relations/fixedincome/2025/20250225-USD-AT1.pdf, 2025. Dated 18 February 2025; ISIN US06738EDC66; preserved with the accompanying prospectus in docs/prospectus.

T.J.M. Bench-Capon. HYPO's Legacy: Introduction to the Virtual Special Issue. Artificial Intelligence and Law, 25, 205–250, 2017. URL https://www.csc.liv.ac.uk/~tbc/publications/hypoLegacy.pdf. Author manuscript; virtual-special-issue introduction.

Trevor Bench-Capon and Giovanni Sartor. A Model of Legal Reasoning with Cases Incorporating Theories and Values. Artificial Intelligence, 150, 97–143, 2003. URL https://www.csc.liv.ac.uk/~tbc/publications/aijsartor.pdf. Author-hosted full text; publication year and DOI follow the journal record.

Catala contributors. Catala: compiler and core formalization repository, 2026. URL https://github.com/CatalaLang/catala. README, formalization notes and selected implementation/proof source inspected September 2026; proof not replayed.

Alison Chorley and Trevor Bench-Capon. AGATHA: Using heuristic search to automate the construction of case law theories. Artificial Intelligence and Law, 13, 9–51, 2005. URL https://www.csc.liv.ac.uk/~tbc/publications/agatha.pdf. Author PDF title page is dated 2006; Crossref records print publication 2005 and online publication 2006; cited as volume 13 (2005).

Silvia Crafa. From Legal Contracts to Legal Calculi: the code-driven normativity. Electronic Proceedings in Theoretical Computer Science, 368, 2022. URL https://arxiv.org/pdf/2209.02353. Author preprint.

Silvia Crafa, Cosimo Laneve, and Giovanni Sartor. Pacta sunt servanda: legal contracts in Stipula. arXiv:2110.11069, 2021. URL https://arxiv.org/pdf/2110.11069. Reviewed three-author 2021 preprint; later journal version has a different author list.

cvc5 contributors. cvc5: Python api and solver examples, 2026. URL https://cvc5.github.io/docs/latest/api/python/python.html. Official API; solver and encoder identities recorded with each local result.

Matthew Dahl, Varun Magesh, Mirac Suzgun, and Daniel E. Ho. Large legal fictions: Profiling legal hallucinations in large language models. *Journal of Legal Analysis*, 16 (1):64–93, 2024. doi: 10.1093/jla/laae003. URL https://arxiv.org/abs/2401.01301v2. Inspected author manuscript with online appendix, arXiv v2; official replication code retained separately.

Giorgio Delzanno, Cosimo Laneve, Arnaud Sangnier, and Gianluigi Zavattaro. Clause-reachability is undecidable in legal contracts. International Journal on Software Tools for Technology Transfer, 28, 255–275, 2026. URL https://link.springer.com/content/pdf/10.1007/s10009-026-00841-5.pdf. Published 19 March 2026; June issue.

Docling contributors. Docling: document conversion and structured document representation, 2026. URL https://github.com/docling-project/docling. Official documentation and selected source inspected September 2026; execution versions recorded separately.

Christopher A. Esterhuyse, Tim Müller, and L. Thomas van Binsbergen. eFLINT Clingo implementation and evaluation artifact, version 2, 2025a. URL https://zenodo.org/records/15470286. README, compiler and test conventions inspected; documented exclusions retained; not executed.

Christopher A. Esterhuyse, Tim Müller, and L. Thomas van Binsbergen. A Stable Model Semantics for eFLINT Norm Specifications and Model Checking Scenarios. GPCE 2025, 80–93, 2025b. URL https://pure.uva.nl/ws/files/270385564/3742876.3742882.pdf. University repository copy with cover; distinct semantics from the earlier interpreter.

Financial Crimes Enforcement Network. 31 CFR 1010.620: Due diligence programs for private banking accounts, 2025 edition. Official source, 2025a. Inspected 29 September 2026; retained edition and scope recorded locally.

Financial Crimes Enforcement Network. 31 CFR 1010.605: Definitions for correspondent and private banking accounts, 2025 edition. Official source, 2025b. Inspected 29 September 2026; retained edition and scope recorded locally.

FINMA. FINMA provides information about the basis for writing down AT1 capital instruments. https://www.finma.ch/en/news/2023/03/20230323-mm-at1-kapitalinstrumente/, 2023. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

FINMA. FINMA to appeal partial decision concerning AT1. https://www.finma.ch/en/news/2025/10/20251015-meldung-bvger-at1/, 2025a. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

FINMA. FINMA Insolvency Ordinance: consolidation of 1 October 2025. https://www.fedlex.admin.ch/filestore/fedlex.data.admin.ch/eli/cc/2025/557/20251001/de/pdf-a/fedlex-data-admin-ch-eli-cc-2025-557-20251001-de-pdf-a.pdf, 2025b. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Aditya Ghose and George Koliadis. Auditing Business Process Compliance. Business Process Management (BPM 2007), 2007. URL https://link.springer.com/content/pdf/10.1007/978-3-540-74974-5_14.pdf. Publisher chapter.

Thomas F. Gordon, Henry Prakken, and Douglas Walton. The Carneades model of argument and burden of proof. Artificial Intelligence, 171, 875–896, 2007. URL https://webspace.science.uu.nl/~prakk101/pubs/GordonPrakkenWalton2007a.pdf. Author-hosted full text; inspected formal definitions and burden-of-proof sections.

Pierre Goutagny, Aymeric Fromherz, and Raphaël Monat. CUTECat: Concolic Execution for Computational Law. Programming Languages and Systems (ESOP

2025), 2025. URL https://link.springer.com/content/pdf/10.1007/978-3-031-91121-7_2.pdf. Publisher open-access chapter.

Guido Governatori and Antonino Rotolo. Logic of Violations: A Gentzen System for Reasoning with Contrary-To-Duty Obligations. Australasian Journal of Logic, 4, 193–215, 2006. URL https://ojs.victoria.ac.nz/ajl/article/download/1780/6684. Title-page publication 2006; migrated index year 2018 is not used.

Guido Governatori, Francesco Olivieri, Antonino Rotolo, and Simone Scannapieco. Computing Strong and Weak Permissions in Defeasible Logic. arXiv:1212.0079, 2012. URL https://arxiv.org/pdf/1212.0079. Reviewed December 2012 preprint; later journal DOI.

Guido Governatori, Mustafa Hashmi, Ho-Pun Lam, Serena Villata, and Monica Palmirani. Semantic Business Process Regulatory Compliance Checking Using LegalRuleML. EKAW 2016, 746–761, 2016. URL https://inria.hal.science/hal-01572441/document. HAL author manuscript.

Neel Guha et al. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. arXiv:2308.11462, 2023. URL https://arxiv.org/pdf/2308.11462. Reviewed v1 including detailed task and evaluation appendices; no benchmark rerun.

Mustafa Hashmi, Guido Governatori, and Moe Thandar Wynn. Normative requirements for regulatory compliance: An abstract formal framework. Information Systems Frontiers, 18, 429–455, 2016. URL https://eprints.qut.edu.au/83732/23/Main.pdf. Author manuscript; online publication 2015, issue 2016.

HKMA. Consumer protection in respect of use of generative artificial intelligence, 2024. URL https://brdr.hkma.gov.hk/eng/doc-ldg/docId/20241107-1-EN. 19 August 2024; official four-page PDF retained 23 September 2026; scope is customer-facing applications.

Nils Holzenberger and Benjamin Van Durme. Connecting Symbolic Statutory Reasoning with Legal Information Extraction. Proceedings of the Natural Legal Language Processing Workshop, 2023. URL https://aclanthology.org/2023.nllp-1.12.pdf. ACL Anthology proceedings copy.

Nils Holzenberger, Andrew Blair-Stanek, and Benjamin Van Durme. A Dataset for Statutory Reasoning in Tax Law Entailment and Question Answering. arXiv:2005.05257, 2020. URL https://arxiv.org/pdf/2005.05257. Dataset uses simplified statutes, not current operative US tax law.

Hong Kong Monetary Authority. Sale and Distribution of Debt Instruments with Loss-absorption Features and Related Products.

https://brdr.hkma.gov.hk/eng/doc-ldg/docId/getPdf/20221021-3-EN/20221021-3-EN.pdf, 2022a. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Hong Kong Monetary Authority. Loss-absorption products: Annex 1, enhanced investor protection measures. https://brdr.hkma.gov.hk/eng/doc-ldg/docId/getPdf/20221021-4-EN/20221021-4-EN.pdf, 2022b. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Hong Kong Monetary Authority. Loss-absorption products: Annex 2, frequently asked questions. https://brdr.hkma.gov.hk/eng/doc-ldg/docId/getPdf/20221021-5-EN/20221021-5-EN.pdf, 2022c. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Abe Bohan Hou, Orion Weller, Guanghui Qin, Eugene Yang, Dawn Lawrie, Nils Holzenberger, Andrew Blair-Stanek, and Benjamin Van Durme. CLERC: A Dataset for Legal Case Retrieval and Retrieval-Augmented Analysis Generation. arXiv:2406.17186v2, 2024. URL https://arxiv.org/abs/2406.17186v2. Inspected arXiv v2, 27 June 2024; later proceedings edition not substituted.

Liane Huttner and Denis Merigoux. Catala: Moving Towards the Future of Legal Expert Systems. Artificial Intelligence and Law, 2022. URL https://inria.hal.science/hal-02936606/document. Accepted manuscript, 10 August 2022.

Reiner Hähnle, Cosimo Laneve, and Adele Veschetti. Formal Verification of Legal Contracts: A Translation-based Approach (Extended Version). arXiv:2509.20421, 2025. URL https://arxiv.org/pdf/2509.20421v2. Extended version, v2, 26 September 2025.

Joint Financial Intelligence Unit. Announcement of the launch of streams 2 on 2026-02-02, 2026. URL https://www.jfiu.gov.hk/en/.

JPMorgan Chase. J.P. Morgan Private Bank Asia: Legal Disclaimer. Official source, 2026. Inspected 29 September 2026; retained edition and scope recorded locally.

William Jurayj, Nils Holzenberger, and Benjamin Van Durme. Language Models and Logic Programs for Trustworthy Tax Reasoning. arXiv:2508.21051, 2025. URL https://arxiv.org/pdf/2508.21051v3. Alternate extended version of jurayj2026, v3, 5 February 2026; first posted 2025.

William Jurayj, Nils Holzenberger, and Benjamin Van Durme. Language Models and Logic Programs for Trustworthy Tax Reasoning. Proceedings of AAAI, 40(45), 38688–38698, 2026. URL https://ojs.aaai.org/index.php/AAAI/article/download/41212/45173. Published proceedings copy.

Alan Khoja, Martin Kölbl, Stefan Leue, and Rüdiger Wilhelmi. Automated consistency analysis for legal contracts. Artificial Intelligence and Law, 2025. URL

https://link.springer.com/content/pdf/10.1007/s10506-025-09456-8.pdf. Publisher version.

Elliot Kim, Avi Garg, Kenny Peng, and Nikhil Garg. Correlated errors in large language models, 2025. URL https://proceedings.mlr.press/v267/kim25e.html. ICML proceedings; section 3 and appendices A and C inspected; official analysis code retained.

John C. Knight and Nancy G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, SE-12(1):96–109, 1986. URL https://people.cs.rutgers.edu/~uli/cs673/papers/EvaluationMultiVersionProgramming86.pdf. Retained manuscript; sections 4–8 inspected; experiment not reproduced locally.

Yuta Koreeda and Christopher D. Manning. ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts. Findings of EMNLP 2021, 2021. URL https://aclanthology.org/2021.findings-emnlp.164.pdf. ACL Anthology proceedings copy.

Robert Kowalski, Jacinto Dávila, Galileo Sator, and Miguel Calejo. Logical English for Law and Education. Prolog: The Next 50 Years, 2023. URL https://www.doc.ic.ac.uk/~rak/papers/Logical%20English%20for%20Law%20and%20Education%20.pdf. Author-hosted version; third author is Galileo Sator as on title page.

Lorenz Kuhn and collaborators. Official semantic-uncertainty clustering implementation, 2023. URL https://github.com/lorenzkuhn/semantic_uncertainty/blob/20e0ee1388e776e48c1ee285e00462aabc6cf35a/code/get_semantic_similarities.py. Pinned source inspected, lines 99–114; source inspection only, no model execution.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. ICLR 2023, 2023. URL https://arxiv.org/pdf/2302.09664v3. Reviewed v3, 15 April 2023; sections 3–6 and appendix A.1–A.2. Official clustering source inspected separately; no experiment run.

Cosimo Laneve. Liquidity analysis in resource-aware programming. Journal of Logical and Algebraic Methods in Programming, 135, 100889, 2023. URL https://www.cs.unibo.it/~laneve/papers/Stipula_LiquidityFULL.pdf. Reviewed author preprint dated 5 October 2022; journal edition 2023.

Cosimo Laneve. The Stipula Platform: A Workbench for Programming and Analyzing Legal Contracts. Formal Methods for Industrial Critical Systems, 2026. URL https://www.cs.unibo.it/~laneve/papers/StipulaPlatform.pdf. Author-hosted manuscript available September 2025; publisher chapter indexed 2026.

Cosimo Laneve, Alessandro Parenti, and Giovanni Sartor. Legal Contracts Amending with Stipula. Author-hosted manuscript, 2023. URL https://www.cs.unibo.it/~laneve/papers/HO_Stipula_llncs.pdf. 2023

author-hosted edition; do not identify this PDF with the different 2024 chapter Programming Contract Amending.

Legalese contributors. L4 compiler, IDE and DEONTIC reference, 2026. URL https://github.com/legalese/l4-ide. README and doc/reference/regulative/DEONTIC.md inspected; no execution or equivalence claim.

LegalMath research notes. Paper library, technical reading notes and retrieval manifest. Local files: docs/papers/README.md, reading-notes.md and manifest.json, 2026. 38 PDF editions representing 37 works; expanded notes and coverage ledger; research snapshot 21 September 2026.

Bev Littlewood, Peter Popov, and Lorenzo Strigini. Modeling software design diversity: A review. *ACM Computing Surveys*, 33(2):177–208, 2001. URL https://openaccess.city.ac.uk/id/eprint/1951/. Retained manuscript; section 4.1, equations 4.1–4.4 inspected.

Varun Magesh, Faiz Surani, Matthew Dahl, Mirac Suzgun, Christopher D. Manning, and Daniel E. Ho. Hallucination-free? assessing the reliability of leading ai legal research tools, 2024. URL https://arxiv.org/abs/2405.20362v1. Inspected arXiv v1, 30 May 2024; historical evaluation, not a claim about current product versions.

David Makinson and Leendert van der Torre. Input/Output Logics. Journal of Philosophical Logic, 29, 383–408, 2000. URL https://icr.uni.lu/leonvandertorre/papers/jpl00.pdf. Author-hosted manuscript.

Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A Programming Language for the Law. Proceedings of the ACM on Programming Languages, 5 (ICFP), 2021. URL https://arxiv.org/pdf/2103.03198v2. arXiv v2, 2 July 2021.

Microsoft. DeBERTa large MNLI model configuration, 2026. URL https://huggingface.co/microsoft/deberta-large-mnli/raw/main/config.json. Configuration snapshot retrieved 22 September 2026; access year, not model publication year; label mapping retained locally.

Raphaël Monat, Aymeric Fromherz, and Denis Merigoux. Formalizing Date Arithmetic and Statically Detecting Ambiguities for the Law. Programming Languages and Systems (ESOP 2024), 2024. URL https://link.springer.com/content/pdf/10.1007/978-3-031-57267-8_16.pdf. Publisher open-access chapter.

Robert D. Nowak. The Geometry of Generalized Binary Search. IEEE Transactions on Information Theory, 57(12), 7893–7906, 2011. URL https://arxiv.org/abs/0910.4397v5. Reviewed arXiv v5 (25 June 2013); journal publication 2011.

OASIS LegalRuleML Technical Committee. LegalRuleML Core Specification Version 1.0, 2021. URL https://docs.oasis-open.org/legalruleml/legalruleml-core-spec/v1.0/os/legalruleml-core-spec-v1.0-os.html. OASIS Standard, 30 August 2021.

Object Management Group. Decision Model and Notation, Version 1.5, 2024. URL https://www.omg.org/spec/DMN/1.5/PDF. Formal specification; hit-policy discussion in section 8.2.11.

Office of Foreign Assets Control. FAQ 401: Indirect ownership under OFAC's 50 Percent Rule. Official source, 2014. Inspected 29 September 2026; retained edition and scope recorded locally.

Office of Foreign Assets Control. FAQ 899: Designated entities and effective dates under EO 13959, as amended. Official source, 2021a. Inspected 29 September 2026; retained edition and scope recorded locally.

Office of Foreign Assets Control. FAQ 902: Investment services for non-US persons under EO 13959, as amended. Official source, 2021b. Inspected 29 September 2026; retained edition and scope recorded locally.

Office of Foreign Assets Control. FAQ 857: Subsidiaries under the Chinese military-industrial securities restrictions. Official source, 2021c. Inspected 29 September 2026; retained edition and scope recorded locally.

Office of Foreign Assets Control. FAQ 1046: Holding and divestment of CMIC securities. Official source, 2022a. Inspected 29 September 2026; retained edition and scope recorded locally.

Office of Foreign Assets Control. FAQ 1048: Rejection and intermediation of CMIC securities transactions. Official source, 2022b. Inspected 29 September 2026; retained edition and scope recorded locally.

Office of Foreign Assets Control. FAQ 11: Who must comply with OFAC sanctions? Official source, 2024. Inspected 29 September 2026; retained edition and scope recorded locally.

Office of the Comptroller of the Currency. 12 CFR 21.21: Procedures for monitoring Bank Secrecy Act compliance, 2025 edition. Official source, 2025. Inspected 29 September 2026; retained edition and scope recorded locally.

Theo X. Olausson, Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. Proceedings of EMNLP 2023, 2023. URL https://aclanthology.org/2023.emnlp-main.313.pdf. ACL Anthology proceedings copy.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. Findings of EMNLP 2023, 2023. URL https://aclanthology.org/2023.findings-emnlp.248.pdf. ACL Anthology proceedings copy.

PIT contributors. Pit: mutation operators and command-line execution, 2026. URL https://pitest.org/quickstart/commandline/. Official documentation; mutator results do not establish legal correctness.

Potassco. clingo: Python api and answer-set solving, 2026. URL https://potassco.org/clingo/python-api/current/. Official API; finite-model semantics and enumeration limits specified in the adapter.

Henry Prakken. An abstract framework for argumentation with structured arguments. Argument and Computation, 1, 93–124, 2010. URL https://webspace.science.uu.nl/~prakk101/pubs/aspicAF.pdf. Author-hosted full text; correction page retained in the staging directory.

President of the United States. Executive Order 14032: Addressing the Threat from Securities Investments that Finance Certain Companies of the People's Republic of China. Official source, 2021. Inspected 29 September 2026; retained edition and scope recorded locally.

Securities and Exchange Commission. 17 CFR 230.902: Regulation S Definitions. https://www.govinfo.gov/content/pkg/CFR-2025-title17-vol3/pdf/CFR-2025-title17-vol3-sec230-902.pdf, 2025a. Annual edition revised 1 April 2025; section (k) inspected; later currentness is not certified.

Securities and Exchange Commission. 17 CFR 230.144A: Private Resales of Securities to Institutions. https://www.govinfo.gov/content/pkg/CFR-2025-title17-vol3/pdf/CFR-2025-title17-vol3-sec230-144A.pdf, 2025b. Annual edition revised 1 April 2025; definitions and account conditions inspected; later currentness is not certified.

Securities and Futures Commission. SFC distribution of complex and high-risk products, 18EC89. https://apps.sfc.hk/edistributionWeb/api/circular/list-content/circular/suitability/doc?lang=EN&refNo=18EC89, 2018. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Securities and Futures Commission. Circular on intermediaries engaging in tokenised securities-related activities, 2023. URL https://apps.sfc.hk/edistributionWeb/api/circular/content?refNo=23EC52&lang=EN.

Securities and Futures Commission. Circular on full adoption of e-ip application/submission system on wings, 2024a. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=24EC50. 24 October 2024, reference 24EC50; parallel period extension, mandatory channel, annual fees and footnote 1 category list inspected.

Securities and Futures Commission. Circular on launch of e-ip application/submission system on wings, 2024b. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=24EC33. 8 July 2024, reference 24EC33; launch, parallel period and footnote 2 category list inspected.

Securities and Futures Commission. Exemption for non-centrally cleared equity options from margin requirements, 2025. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=25EC71. 22 December 2025, reference 25EC71; commencement and promised Code amendment compared with January 2026 Code, Schedule 10 Part III paragraph 7(e) and footnote 9.

Securities and Futures Commission. SFC SPI FAQ. https://www.sfc.hk/en/faqs/intermediaries/supervision/Sophisticated-Professional-Investors/Sophisticated-Professional-Investors, 2026a. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Securities and Futures Commission. Transition to the new suspicious transaction reporting platform, 2026b. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=26EC2. 26 January 2026, reference 26EC2; English and traditional-Chinese official responses retained; submission methods, transitional arrangements and footnotes 2 and 3 inspected.

M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, and H. T. Cory. The British Nationality Act as a Logic Program. Communications of the ACM, 29(5), 370–386, 1986. URL https://www.doc.ic.ac.uk/~rak/papers/British%20Nationality%20Act.pdf. Author-hosted scan.

SFC. Frequently asked questions on the Code of Conduct: 30 September 2010 page, 2010. URL https://www.sfc.hk/en/faqs/intermediaries/supervision/Code-of-Conduct/30-Sep-2010---Code-of-Conduct. Retained 24 September 2026; page records 15 October 2024 updates to answers 2 and 4; historical applicability unresolved.

SFC. Code of Conduct for Persons Licensed by or Registered with the Securities and Futures Commission, 2023a. URL https://www.sfc.hk/-/media/EN/assets/components/codes/files-current/web/codes/code-of-conduct-for-persons-licensed-by-or-registered-with-the-securities-and-futures-commission/Code_of_conduct-Sep-2023_Eng-Final-with-Bookmark.pdf?rev=209e9f3b717e4d70b45bfe45a0bb6288. September 2023 edition; retained 24 September 2026.

SFC. Circular on distributors providing additional returns and other services or arrangements when marketing SFC-authorised funds. 24 October, reference

23EC46, 2023b. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=23EC46.

SFC. Circular on tokenisation of SFC-authorised investment products. 2 November, reference 23EC53, 2023c. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=23EC53. Public entry marked as replaced by the circular of 20 April 2026.

SFC. Circular to licensed corporations - use of generative ai language models, 2024a. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/intermediaries/supervision/doc?refNo=24EC55. 12 November 2024, reference 24EC55; official response retained 23 September 2026.

SFC. Non-exhaustive list of risk factors in relation to the use of generative ai language models, 2024b. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/intermediaries/supervision/doc?refNo=24EC55. Appendix to circular 24EC55; official four-page PDF retained 23 September 2026.

SFC. Circular to licensed corporations and management companies of SFC-authorised funds - Shortening of the US securities transaction settlement cycle to T+1, 2024c. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=24EC16. Circular 24EC16; retained official response and extracted text.

SFC. Code of Conduct for Persons Licensed by or Registered with the Securities and Futures Commission, 2026a. URL https://www.sfc.hk/-/media/EN/assets/components/codes/files-current/web/codes/code-of-conduct-for-persons-licensed-by-or-registered-with-the-securities-and-futures-commission/Code_of_conduct-Dec-2025_Eng-Final-with-Bookmark_Jan-2026.pdf?rev=8768a10c17c44385ab1ad8c0a29d2844. January 2026 edition; retained 24 September 2026.

SFC. Circulars: Product Authorization, 2026b. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/products/product-authorization/. Public index inspected 21 September 2026; not a complete private-bank regulatory inventory.

SFC. Circular on tokenisation of SFC-authorised investment products. 20 April, reference 26EC22, 2026c. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=26EC22.

SFC and HKMA. Joint circular to intermediaries: Streamlined approach for compliance with suitability obligations when dealing with sophisticated professional investors. 28 July, reference 23EC35, 2023a. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=23EC35.

SFC and HKMA. Streamlined approach for compliance with suitability obligations when dealing with sophisticated professional investors: Annex 1. Annex to joint circular 23EC35, 28 July, 2023b. URL https://apps.sfc.hk/edistributionWeb/api/circular/openAppendix?lang=EN&refNo=23EC35&appendix=0.

SFC and HKMA. Frequently asked questions: Streamlined approach for sophisticated professional investors, Annex 2. Annex to joint circular 23EC35, 28 July, 2023c. URL https://apps.sfc.hk/edistributionWeb/api/circular/openAppendix?lang=EN&refNo=23EC35&appendix=1.

SFC and HKMA. Joint circular to intermediaries: Concurrent SFC-HKMA thematic review of the distribution of non-exchange traded investment products. 29 September, reference 25EC48, 2025. URL https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=25EC48.

Sepehr Sharifi, Alireza Parvizimosaed, Daniel Amyot, Luigi Logrippo, and John Mylopoulos. Symboleo: Towards a Specification Language for Legal Contracts. IEEE International Requirements Engineering Conference 2020, 2020. URL https://drive.google.com/uc?export=download&id=1WXwXeLrZdaJjhSJcCrt_wBXxDvhFkq2k. Author preprint linked by official Symboleo repository; some ancillary semantics references remain anonymized.

SIX. Schedule of clearing days 2027. Published schedule, 2027. Retrieved 29 September 2026; clearing-system scope only.

Smart Contract Modelling, University of Ottawa. Symboleo editor and model checker, 2026. URL https://github.com/Smart-Contract-Modelling-uOttawa/Symboleo-Model-Checker. Official editor README, sale example and model-checker README inspected; no nuXmv replay.

Standard Chartered PLC. 7.625% Fixed Rate Resetting Perpetual Subordinated Contingent Convertible Securities: Offering Circular. https://www.sc.com/en/uploads/sites/66/content/docs/SC-PLC-1bn-AT1-Offering-Circular-final.pdf, 2025. Dated 8 January 2025; ISINs US853254DF47 and USG84228GP72; preserved in docs/prospectus.

Stipula contributors. Stipula implementation and workbench, 2026a. URL https://github.com/stipula-language/stipula-workbench. Official project repositories; workbench and prototype documentation inspected September 2026.

Stipula contributors. Stipula-to-Java/JML verification implementation, 2026b. URL https://github.com/stipula-language/stipula-KeY. Official translation repository; Translator.java and documentation inspected, no KeY replay.

Swiss Confederation. Banking Act amendment of 17 December 2021, promulgated AS 2022 732. https://www.fedlex.admin.ch/filestore/fedlex.data.admin.ch/eli/oc/2022/732/de/pdf-a/fedlex-data-admin-ch-eli-oc-2022-732-de-pdf-a.pdf, 2022. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Swiss Confederation. Federal Act on Banks and Savings Banks: German consolidation of 1 January 2024. https://www.fedlex.admin.ch/filestore/fedlex.data.admin.ch/eli/cc/51/117_121_129/20240101/de/pdf-a/fedlex-data-admin-ch-eli-cc-51-117_121_129-20240101-de-pdf-a.pdf, 2024. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Swiss Federal Administrative Court. B-2334/2023: partial judgment of 1 October 2025. https://www.bvger.ch/media-releases/3eeb1c4b-6120-4804-9512-271477ebb52a/en/b-2334-2023_te.pdf, 2025a. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Swiss Federal Administrative Court. Suspension of AT1 cases. https://www.bvger.ch/en/newsroom/media-releases/suspension-of-at1-cases-2443, 2025b. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Swiss Federal Council. Emergency liquidity ordinance: amendment of 19 March 2023 (AS 2023 136). https://www.fedlex.admin.ch/filestore/fedlex.data.admin.ch/eli/oc/2023/136/de/pdf-a/fedlex-data-admin-ch-eli-oc-2023-136-de-pdf-a.pdf, 2023a. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Swiss Federal Council. Emergency liquidity ordinance: amendment of 6 September 2023 (AS 2023 495). https://www.fedlex.admin.ch/filestore/fedlex.data.admin.ch/eli/oc/2023/495/de/pdf-a/fedlex-data-admin-ch-eli-oc-2023-495-de-pdf-a.pdf, 2023b. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Swiss Federal Council. Capital Adequacy Ordinance: German consolidation of 1 January 2025. https://www.fedlex.admin.ch/filestore/fedlex.data.admin.ch/eli/cc/2012/629/20250101/de/pdf-a/fedlex-data-admin-ch-eli-cc-2012-629-20250101-de-pdf-a.pdf, 2025. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Swiss Federal Supreme Court. 2C_565/2025: judgment of 5 March 2026. https://search.bger.ch/ext/eurospider/live/de/php/aza/http/index.php?highlight_docid=aza%3A%2F%2F05-03-2026-2C_565-2025&lang=de&type=show_document&zoom=NO, 2026. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

Tesseract contributors. Tesseract ocr: command-line and structured output documentation, 2026. URL https://github.com/tesseract-ocr/tesseract. Official documentation inspected September 2026.

Silvano Colombo Tosatto, Guido Governatori, and Pierre Kelsen. Business Process Regulatory Compliance is Hard. IEEE Transactions on Services Computing, 8(6), 958–970, 2015. URL https://eprints.qut.edu.au/100964/1/ComProof%202.pdf. Author manuscript; DOI online year 2014, issue year 2015.

UBS Group AG. SGD Tier 1 capital notes (ISIN CH1357852636): published terms and conditions. Issuer conditions for ISIN CH1357852636, 2024. Version and inspected scope retained in the legal source dossier; accessed 29 September 2026.

UK AI Security Institute. Inspect ai: task, solver and scorer documentation, 2026. URL https://inspect.aisi.org.uk/scorers.html. Official documentation; deterministic local evaluation is distinct from live-model accuracy.

L. Thomas van Binsbergen, Lu-Chi Liu, Robert van Doesburg, and Tom van Engers. eFLINT: a Domain-Specific Language for Executable Norm Specifications. GPCE 2020, 2020. URL https://ir.cwi.nl/pub/29922/29922.pdf. CWI author copy.

L. Thomas van Binsbergen, Christopher A. Esterhuyse, and Tim Müller. Reflections on the design, applications and implementations of the normative specification language eFLINT. Journal of Computer Languages, 2026. URL https://arxiv.org/pdf/2511.12276v3. Reviewed arXiv v3, 4 August 2026; journal title differs slightly; first preprint 2025.

Olivia Peiyu Wang, Sanna Wong-Toropainen, Daneshvar Amrollahi, Ryan Bai, Tashvi Bansal, Arush Garg, and Leilani H. Gilpin. Know Your Limits: On the Faithfulness of LLMs as Solvers and Autoformalizers in Legal Reasoning. arXiv:2606.16118, 2026. URL https://arxiv.org/pdf/2606.16118v2. v2, 19 June 2026; under review; methodological caveats retained.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023, 2023. URL https://arxiv.org/pdf/2203.11171v4. Reviewed v4, 7 March 2023; first posted 2022. Method, sampling, evaluation and aggregation inspected; no benchmark replay.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601, 2023. URL https://arxiv.org/abs/2305.10601v2. Reviewed arXiv v2 (3 December 2023); the method explores partial thoughts with breadth-first and depth-first search.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. Proceedings of ICML 2024, PMLR 235, 2024. URL https://arxiv.org/abs/2310.04406v3. Reviewed arXiv v3 (6 June 2024); first posted 2023.